

# Claude Code 完全指南 V2

全网最详细的 Claude Code 51万行 TypeScript 源码解读

20 篇

187 节

44,600+ 行

100+ 架构图

8 个实战 Lab

在线版（含交互 Mermaid 图表）

[https://bcefgjh.github.io/claude-code-complete-guide\\_v2/](https://bcefgjh.github.io/claude-code-complete-guide_v2/)

GitHub: [github.com/bcefgjh](https://github.com/bcefgjh)

小红书ID: **bcefgjh**

2026 年 4 月

# 目录

## 前言

---

前言：关于《Claude Code 完全指南 V2》

学习路线图：三条路径，同一种好奇心

术语表：把「行话」变成你能用的口令

环境搭建：工欲善其事，必先利其器

## 第1篇：背景故事

---

1.1 泄露事件始末：当「调试地图」搭上了公开发货的便车

1.2 源码规模全貌：五十一万行不是「一个文件」，而是一座城

1.3 社区反应与重建：当一千双手伸向同一张地图

1.4 为什么值得学：AI Agent 的力气，多半花在「AI 之外」

1.5 法律与伦理：我们读的是「公开可得文本」，但不是「任意使用许可证」

## 第2篇：小白快速上手

---

第2篇：小白快速上手 · 总览

2.2 安装全流程

2.3 第一次对话

2.4 编辑文件实操

2.5 运行命令实操

2.6 Git 工作流

2.7 命令速查表

2.8 费用与省钱技巧

## 第3篇：架构全景

---

第 3 篇 · 3.1 不是 CLI，是「以 LLM 为内核的操作系统」

3.2 四个独立入口：同一颗「大脑」，四条「大门」

3.3 目录结构全景：src/ 地图与统计心智模型

3.4 数据流全景：从一句话到工具执行再回环

3.5 技术栈深度解析：Bun、TypeScript、自研 Ink、Commander

3.6 架构对比：Claude Code、Cursor、Copilot Agent、Windsurf

3.7 启动流程详解：从进程诞生到 Agent 主循环

- 3.8 main.tsx 入口分析：4684 行如何拆解？
- 3.9 模块依赖关系：从「依赖图」理解演进边界
- 3.10 设计哲学总结：Less scaffolding, more model

## 第4篇：核心循环

---

- 4.1 QueryEngine 是什么：整本「51 万行史诗」的心脏
- 4.2 八步循环：QueryEngine 的「主菜流水线」
- 4.3 query.ts 逐段走读：从 query() 外壳到 while(true) 心室
- 4.4 消息准备与历史：对话的「行李箱」怎么打包
- 4.5 API 调用与流式响应：逐词「泵血」到终端
- 4.6 工具请求收集：从模型「意图」到可执行清单
- 4.7 静默错误修复：用户看不见的「免疫系统」
- 4.8 预算三重关卡：Token、金钱与轮次
- 4.9 循环终止条件：何时「关火收工」
- 4.10 Thinking 模式：Extended Thinking 与 thinking 块
- 4.11 并行工具执行器：isConcurrencySafe 与 Promise.all
- 4.12 协作全景图：QueryEngine 在系统中的位置

## 第5篇：提示词工程

---

- 第 5 篇：提示词工程（Prompt Engineering）
- 5.2 静态部分：系统宪法（System Constitution）
- 5.3 动态部分：当期政策（Dynamic Policy）
- 5.4 缓存边界设计：SYSTEM\_PROMPT\_DYNAMIC\_BOUNDARY
- 5.5 Token 缓存经济学（Prompt Cache Economics）
- 5.6 缓存失效 7 大陷阱
- 5.7 铁血行为约束（Behavior Constraints）
- 5.8 工具使用手册：每个工具的 prompt.ts
- 5.9 策略对比：Claude Code vs OpenAI vs Google
- 5.10 实践：设计你自己的系统提示词

## 第6篇：工具系统

---

- 第六部分 · 工具系统（6.1） — 42 个内置工具全景
- 6.2 Tool 接口设计 — Zod 校验与输入输出 Schema
- 6.3 十四步治理流水线 — 从解析到结构化输出
- 6.4 BashTool — AST 分析、Git 安全与命令黑名单
- 6.5 FileRead / FileEdit / NotebookEdit — 先读后写强制

- 6.6 搜索工具族 — Glob / Grep / LSP / ToolSearch
- 6.7 AgentTool — 子代理生成与隔离上下文
- 6.8 外部工具 — WebFetch 与 WebSearch
- 6.9 MCP 工具集成 — 动态发现、指令注入与 ReadMcpResource
- 6.10 Fail-closed 设计 — 默认「不安全」与工厂函数
- 6.11 延迟加载 — ToolSearchTool、prompt.ts 与 Token 节省
- 6.12 实践 — 构建自定义工具（步骤指南）

## 第7篇：权限与安全

---

- 7.1 为什么需要权限：从 `rm -rf` 噩梦说起
- 7.2 六种权限模式：一张总表读懂全局
- 7.3 Default、acceptEdits、Plan 详解
- 7.4 Auto 模式与后台分类器（Sonnet 4.6）
- 7.5 dontAsk 与 bypassPermissions
- 7.6 七步权限评估管道（完整流程图）
- 7.7 BashTool 与 AST 命令分析
- 7.8 沙箱执行：macOS Seatbelt 与 Linux bubblewrap
- 7.9 Fail-closed 安全哲学
- 7.10 实践：企业级权限配置与 Prompt Fatigue 缓解

## 第8篇：上下文管理

---

- 第 8 篇：上下文管理（Context Management）
- 8.2 三层压缩总览：Tier1 → Tier2 → Tier3
- 8.3 Tier1 微压缩：无模型参与的「前台清桌」
- 8.4 Tier2 自动压缩：87% 阈值与服务端策略
- 8.5 Tier3 完全压缩：九节摘要与 CoT 剥离
- 8.6 缓存感知压缩：cache\_edits 与「前缀不动」纪律
- 8.7 API 层 compaction：compact-2026-01-12 与模型支持
- 8.8 手动 /compact：节奏、焦点与可控摘要
- 8.9 成本分析：窗口长度、单价与 5 倍差距直觉
- 8.10 最佳实践：上下文管理的「日常纪律」

## 第9篇：记忆系统

---

- 第 9 篇：记忆系统（Memory System）
- 9.2 CLAUDE.md：层级、拼接与 50~200 行纪律
- 9.3 自动记忆提取：观察、写入与审计



- 9.4 双模型检索：快速 Sonnet 扫描与「最多 5 条」注入
- 9.5 精确度优先：宁缺毋滥的注入哲学
- 9.6 KAIROS 做梦模式：流水账、低活跃与夜间触发
- 9.7 做梦蒸馏：从流水账到「用户偏好」与「项目背景」
- 9.8 记忆与上下文交互：注入如何吃掉窗口预算
- 9.9 持久化：claude -c、会话边界与三层记忆的寿命
- 9.10 综合练习：从空白仓库到「可记忆」协作

## 第10篇：多Agent系统

---

- 第 10 篇：多 Agent 系统 · 10.1 蜂群作战
- 10.2 六个内建 Agent 角色全图
- 10.3 Explore 探索专家（只读）
- 10.4 Plan 规划专家（只读 · 调研不动手）
- 10.5 Coordinator 协调器（项目经理调度）
- 10.6 反偷懒机制与工人意识注入
- 10.7 Verification 验证专家（Try to break it）
- 10.8 子 Agent 缓存优化（Fork 前缀匹配）
- 10.9 防无限递归（子 Agent 不得生成子 Agent）
- 10.10 消息路由（父子 Agent 通信模式）
- 10.11 Swarm vs Coordinator（两种多 Agent 模式对比）
- 10.12 实践：设计多 Agent 工作流

## 第11篇：终端UI

---

- 第 11 篇：终端 UI · 11.1 架构总览与自研渲染栈
- 11.2 Yoga 布局：纯 TypeScript 中的 Flexbox
- 11.3 React Fiber：自定义协调器与「终端 DOM」
- 11.4 流式渲染：Async Generator 与逐词输出
- 11.5 输入处理：Kitty 协议与终端能力检测
- 11.6 虚拟滚动：长输出的性能护栏
- 11.7 Vim 模式：键位绑定与状态机
- 11.8 Diff 展示：代码修改对比子系统
- 11.9 鼠标、文本选择与 OSC 8 超链接
- 11.10 设计系统与主题：暗色、亮色与 389 组件一致性

## 第12篇：Bridge桥接

---

- 第 12 篇：Bridge 桥接 · 12.1 总览：CLI 与 IDE 的双向通道

- 12.2 跨进程通信基础: stdin/stdout 与 Unix Domain Socket
- 12.3 bridgeMain 主循环: 监听、分发、执行、返回
- 12.4 消息协议: JSON-RPC 风格的请求、响应与通知
- 12.5 JWT 认证: 验证 IDE 连接合法性
- 12.6 sessionRunner: 多会话并发与隔离
- 12.7 传输层抽象: stdio、WebSocket 与 TCP 可切换
- 12.8 IDE 集成实践: VS Code 扩展、Cursor 内置与 JetBrains 插件
- 12.9 BoundedUUIDSet: 有界 UUID 集合与防内存泄漏
- 12.10 总结: Bridge 桥接全链路检查表

## 第13篇：状态管理

---

- 第13篇：状态管理 · 第1节 createStore — 极简状态内核
- 第13篇：状态管理 · 第2节 AppState — 全局应用状态组织
- 第13篇：状态管理 · 第3节 副作用同步 — 状态变更触发外部世界
- 第13篇：状态管理 · 第4节 Memdir — 基于文件系统的记忆目录
- 第13篇：状态管理 · 第5节 History — 会话历史与续接 (claude -c)
- 第13篇：状态管理 · 第6节 Migrations — 配置文件版本迁移
- 第13篇：状态管理 · 第7节 持久化策略 — 什么该留下、什么该随风
- 第13篇：状态管理 · 第8节 架构全景 — 状态流转总图

## 第14篇：服务与集成

---

- 第14篇：服务与集成 · 第1节 API 客户端 — Anthropic Messages 与流式 SSE
- 第14篇：服务与集成 · 第2节 错误分类 — API / 网络 / 认证 / 限流
- 第14篇：服务与集成 · 第3节 MCP — Model Context Protocol 概览
- 第14篇：服务与集成 · 第4节 MCP 传输 — stdio / SSE / WebSocket
- 第14篇：服务与集成 · 第5节 LSP — Language Server Protocol 集成
- 第14篇：服务与集成 · 第6节 OAuth 2.0 + PKCE — 认证与 Token 管理
- 第14篇：服务与集成 · 第7节 特性标志 — 90+ Flags、渐进发布与实验
- 第14篇：服务与集成 · 第8节 遥测与全篇总结 — 观测、隐私与集成全景

## 第15篇：隐藏功能

---

- 第十五部分 · 隐藏功能 (15.1) — 源码中的 90+ Feature Flags 全景
- 第十五部分 · 15.2 Undercover Mode — 公开仓库中的「去标识化」隐身系统
- 第十五部分 · 15.3 Buddy Pet System — BUDDY 旗标与 18 种宠物宇宙
- 第十五部分 · 15.4 Buddy 外观反作弊 — 会话级重算与配置一致性
- 第十五部分 · 15.5 Deep Planning Mode — 长达 30 分钟的单任务深度规划

第十五部分 · 15.6 内部彩蛋 — USER\_TYPE === 'ant' 与实验输出策略

## 第16篇：Hooks/Skills/Plugins

---

第十六部分 · Hooks / Skills / Plugins 生态 (16.1) — 扩展点总览

第十六部分 · 16.2 PreToolUse — 最强大的 Hook: block / allow / modify

第十六部分 · 16.3 Skills — Markdown 封装的轻量工作流

第十六部分 · 16.4 Plugins — 重型扩展: 命令、目录与硬性标记

第十六部分 · 16.5 MCP 动态指令注入 — instructions 与系统提示词拼接

第十六部分 · 16.6 自定义命令 — Slash 命令开发

第十六部分 · 16.7 defer\_loading — 工具延迟加载与缓存保护

第十六部分 · 16.8 综合演练 — 从 Hooks 到 defer 的端到端蓝图

## 第17篇：性能与成本

---

第17篇 · 17.1 Token 经济学与定价心智模型

17.2 Prompt 缓存: 从「每轮全价」到「前缀命中」

17.3 并行预取: 启动路径上的「不等公交」

17.4 懒加载: 四十二般兵器不必一次背上身

17.5 子 Agent 缓存: 统一前缀与字节级命中

17.6 React 渲染性能: 流畅 UI 如何「省下重试」

17.7 流式管道: async generator 与逐词到达

17.8 省钱速查表: 场景 × 模型 × 动作

## 第18篇：遥测与生命周期

---

第18篇 · 18.1 进程状态调度: 前后台 Agent 与生命周期

18.2 Perfetto 性能追踪: registerPerfettoAgent()

18.3 全链路遥测: Transcript、工具计时与 API 延迟

18.4 资源清理: killShellTasksForAgent() 与连接卸载

18.5 错误恢复: 熔断、重试、退避与优雅降级

18.6 生产最佳实践: 监控、告警、日志与调试

## 第19篇：实战Lab

---

Lab 1: 从零搭建最简 Agent Loop

Lab 2: 实现工具注册与执行

Lab 3: 添加权限控制

Lab 4: 实现流式响应

Lab 5: 构建 MCP 服务器 (stdio) 并在 Agent 中使用

- Lab 6: 多 Agent 协调 (主 Agent + 只读子 Agent)
- Lab 7: 实现上下文压缩 (摘要替换旧历史)
- Lab 8: 完整项目整合与架构对照

第20篇：总结与展望

---

- 第 20 篇 · 总结与展望 (一) AI Agent 的壁垒在哪里
- 20.2 深度对比: Claude Code、Cursor、GitHub Copilot Agent、Windsurf、Aider、Continue
- 20.3 AI 编程工具未来趋势: 上下文、成本、多模态、自主性与信任
- 20.4 对开发者的启示: 架构、Token 经济学、安全与多 Agent
- 20.5 推荐学习路径: 按角色划分的进阶路线与资源
- 20.6 结语: 回顾全书、致谢读者与开源共建

附录

---

- 附录 A · 源码文件索引 (教学用模块地图)
- 附录 B · 命令速查表
- 附录 C · 自测题 (50 题)
- 附录 D · 推荐阅读与外部资源
- 附录 E · 中英术语对照表

# 前言

## 前言：关于《Claude Code 完全指南 V2》

### 本节学习目标

- 理解本书 V2 与 V1 的定位差异，以及为何以「51 万行真实源码」为主线。
- 判断自己是否属于本书的目标读者，并知道如何搭配路线图阅读。
- 掌握全书 20 篇的结构地图，能按「小白 / 开发者 / 架构师」三种节奏自学。

### 一本书的诞生：从终端里的「小助手」到可读的工程史诗

如果把 Claude Code 想象成一位住在你终端里的「私人管家」，那么 Anthropic 官方发布的 npm 包，就是这位管家每天出门时穿的「制服」。2026 年 3 月底，有研究者发现：这件制服的内衬里，不小心缝进了一张「建筑蓝图」——一张体积约 60MB 的 `cli.js.map` (Source Map)。透过它，社区得以一窥约 51.2 万行 TypeScript、1903 个文件 构成的客户端编排层。

本书 V2 要做的，不是八卦泄露本身，而是把这次「意外透明」变成 零基础也能跟上的教学叙事：像读一部长篇小说那样，从背景、地图、术语、环境，一路走进 Agent 循环、工具系统、权限与安全设计。

[Mermaid 图表 — 请在在线版本中查看]

## V2 新特色：我们改了什么？

| 维度   | V1（假设的通用 Agent 教程） | V2（本书）                                                 |
|------|--------------------|--------------------------------------------------------|
| 素材来源 | 多为概念图、伪代码、公开文档     | 以 npm 包中意外附带的 Source Map 所还原的 TS 结构为锚点                 |
| 叙事粒度 | 常停在「Agent 会调工具」    | 深入到 QueryEngine、Coordinator、Compaction、Hooks、MCP 等工程细节 |
| 读者假设 | 常假设已会 Node/CLI     | 从零解释 终端、包管理、Source Map、构建链                             |
| 可视化  | 以文字为主              | 每节至少 1~2 个 Mermaid，把「谁在调用谁」画成图                         |
| 伦理立场 | 或回避或偏激             | 专设 法律与伦理 篇，区分「误发布」与「黑客攻击」                              |

生活类比：V1 像「旅游手册上的景点照片」；V2 像「意外拿到景区施工图纸后，导游带着你边走边对照」——照片能激发兴趣，图纸能解释为什么路这么修。

## 谁适合读这本书？

[Mermaid 图表 — 请在在线版本中查看]

| 读者画像      | 你会得到什么                        | 可能觉得吃力的部分                    |
|-----------|-------------------------------|------------------------------|
| 零基础小白     | 术语表、环境搭建、大量类比与图               | 偶尔出现的 TypeScript 语法（我们会放慢讲解） |
| 日常开发者     | 模块边界、调用链、工程习惯                 | 超大仓库的心智负担（用路线图拆碎）            |
| 架构 / 安全方向 | 权限模式、验证代理、Feature Flags 等设计话题 | 需自行对照社区重建仓库的版本差异             |

## 全书 20 篇结构概览（路线图级目录）

下面这张表是「全书楼层导览」。数字代表建议阅读顺序的大致区块；具体三条路径（小白 / 开发者 / 架构师）见 [roadmap.md](#)。

| 篇次      | 主题方向（规划）                           | 一句话期待                          |
|---------|------------------------------------|--------------------------------|
| Part 00 | 前言、路线图、术语、环境                       | 把地图和装备配齐                       |
| Part 01 | 泄露背景、源码规模、社区、学习价值、法律伦理             | 知道「我们在读什么」与边界                  |
| Part 02 | 仓库解剖：入口、 <code>main</code> 、启动链路   | 从 <code>node</code> 按下回车到进程活起来 |
| Part 03 | 配置与 <code>CLAUDE.md</code> 、用户目录约定 | 像「住户公约」一样的项目记忆                 |
| Part 04 | Query 与对话状态机                       | 把聊天想成「前台接线员」                   |
| Part 05 | Agent Loop：计划—执行—观察                | 餐厅后厨的出菜流水线                     |
| Part 06 | Tool 抽象与注册表                        | 工具箱里每把扳手的编号                    |
| Part 07 | Permission Mode 与安全闸门              | 小区门禁 vs 万能钥匙                   |
| Part 08 | Compaction 与上下文压缩                  | 行李箱塞不下时的「真空收纳」                 |
| Part 09 | MCP：模型上下文协议与 Bridge                | 给 AI 接「外设驱动」                   |
| Part 10 | Hooks 与扩展点                         | 音乐会上的「间奏插段」                    |
| Part 11 | Feature Flags 与实验功能                | 游乐园里「未开放区域」的开关                 |
| Part 12 | Coordinator 与多角色协同                 | 剧组里导演、场记、灯光的分工                 |
| Part 13 | Verification Agent（若源码可见）          | 专职「复查作业」的第二意见                  |
| Part 14 | Assistant / Buddy 等子系统边界           | 同一品牌下的不同产品线                    |
| Part 15 | Plugins 与生态位                       | App Store 与系统内核的分界             |
| Part 16 | 性能：冷启动、缓存、I/O                      | 为什么有时「卡一下」                     |
| Part 17 | 测试与可观测性                            | 给黑盒装玻璃窗                        |
| Part 18 | 与 IDE / 编辑器集成思路                    | 终端代理如何「伸长手臂」                   |
| Part 19 | 对比其他 Agent 框架                      | 站在巨人肩膀上看差异                     |
| Part 20 | 附录：重建仓库、哈希与校验、延伸阅读                 | 自学者的「工具房」                      |

[Mermaid 图表 — 请在在线版本中查看]

## 如何使用本书：四种读法

- 1. 顺读法：从 Part 00 一路往下，适合时间充裕、想建立完整心智模型的人。
- 2. 问题导向：先翻 `glossary.md`，再跳到对应篇——像查字典后读例句。

3. 对照源码：先完成 `setup.md`，本地打开社区重建的目录，书中引用片段时跟着跳转。
4. 小组共读：每周固定一篇，用 Mermaid 图在白板上复现——教是最好的学。

关键源码片段（示意）：真实仓库中，入口附近往往会出现类似「启动 CLI、挂载子命令、初始化服务」的串联。下面是一段 `** pedagogical 伪代码**`，仅用于说明阅读方式（非逐字摘录）：

```
// 阅读本书时，请把下面当作「导游旗」——指向你要打开的 real 文件
async function bootstrapCli() {
  const config = await loadUserAndProjectConfig();
  const engine = new QueryEngine(config);
  await engine.attachToolsFromRegistry();
  return engine.runInteractiveLoop();
}
```

## 与 V1 对比总表（本书自我定位）

| 项目     | V1          | V2                      |
|--------|-------------|-------------------------|
| 核心隐喻   | 「学会用 Agent」 | 「读懂 Agent 背后的工程」        |
| 必备前置   | 常含隐式假设      | 显式 提供环境搭建与术语表           |
| 图表密度   | 中等          | 每节 1~2+ Mermaid         |
| 伦理与法律  | 可选          | 独立成篇                    |
| 社区重建仓库 | 或略提         | 作为实践抓手（clone、stub、依赖逆向） |

## V2 新增与强化（速查）

| V2 块        | 你多得到什么                                                                                                    |
|-------------|-----------------------------------------------------------------------------------------------------------|
| Part 00 四件套 | <code>index</code> / <code>roadmap</code> / <code>glossary</code> / <code>setup</code> 把「地图、路标、词典、帐篷」一次配齐 |
| Part 01 五连章 | 事件—规模—社区—价值—伦理，形成完整背景闭环                                                                                   |
| 图表纪律        | 每节至少 1~2 个 Mermaid，强制把「关系」画出来                                                                             |
| 对标真实体量      | 反复锚定 512K 行、1903 文件、地标文件名，避免空谈                                                                            |
| 社区语境        | 明确 重建仓库 与 官方产品 的边界，减少误读                                                                                   |

[Mermaid 图表 — 请在在线版本中查看]



## 谁可能暂时不适合本书？

| 情况        | 说明                                  |
|-----------|-------------------------------------|
| 只想「一键用神功」 | 本书偏 理解与工程，不保证读完变高手                  |
| 完全拒绝打开终端  | 至少完成 <code>setup.md</code> 的最低要求再考虑 |
| 需要法律意见    | 本书 非律师函替代品，见 1.5                    |

## 前言篇 FAQ

- 问：V2 还需要买 V1 吗？

答：若 V1 是另一套叙事，可把 V1 当「使用手册」，V2 当 源码导游；是否购买取决于你对重复内容的容忍度。
- 问：我必须读完 20 Part 吗？

答：不必。按 `roadmap.md` 选路径；最低限度也建议读完 Part 00 + Part 01。
- 问：没有官方源码授权还能学吗？

答：学习行为与材料来源需 自担合规责任；本书鼓励 教育性阅读 与 流程教训总结，见 1.5。
- 问：Mermaid 图在 GitHub 上渲染失败怎么办？

答：用 VS Code / Cursor 插件预览，或导出 SVG；本书节点标签已尽量用引号包裹含特殊字符的文本。

[Mermaid 图表 — 请在在线版本中查看]

## 读书小组建议流程（每周 90 分钟）

| 分钟    | 环节           | 产出    |
|-------|--------------|-------|
| 0~10  | 轮流复述上周一张图    | 巩固记忆  |
| 10~40 | 共读指定节        | 标注术语  |
| 40~70 | 本地搜代码关键词     | 共享屏幕  |
| 70~90 | 画一张新 Mermaid | 上传笔记库 |

生活类比：读书小组像 羽毛球固定局——重点不是单次赢分，而是 每周肌肉记忆。

## Part 00~01 文件一览（仓库内路径）

| 路径                                                        | 角色              |
|-----------------------------------------------------------|-----------------|
| <code>docs/part00-preface/index.md</code>                 | 本书导览与 V2 说明（本文） |
| <code>docs/part00-preface/roadmap.md</code>               | 三条学习路径与学时       |
| <code>docs/part00-preface/glossary.md</code>              | 30+ 术语          |
| <code>docs/part00-preface/setup.md</code>                 | 环境搭建            |
| <code>docs/part01-background/index.md</code>              | 1.1 事件始末        |
| <code>docs/part01-background/02-source-overview.md</code> | 1.2 规模与目录       |
| <code>docs/part01-background/03-community.md</code>       | 1.3 社区重建        |
| <code>docs/part01-background/04-why-learn.md</code>       | 1.4 学习价值        |
| <code>docs/part01-background/05-legal-ethics.md</code>    | 1.5 法律伦理        |

## 致谢与声明

本书 **教育目的** 讲解客户端编排层的设计思想；**不鼓励** 任何未授权分发、破解服务或绕过条款的行为。Anthropic 的产品条款与许可证以官方为准；社区重建仓库的版权与合规状态因仓库而异，请读者自行判断。

下一页建议阅读：[学习路线图](#) `roadmap.md` → [术语表](#) `glossary.md` → [环境搭建](#) `setup.md`。

## 附录：阅读心态检查表

| 自检项                          | 是 / 否                    |
|------------------------------|--------------------------|
| 我愿意先画图再记名词                   | <input type="checkbox"/> |
| 我能接受「一次只读懂一层调用栈」             | <input type="checkbox"/> |
| 我会把泄露事件当作 <b>工程案例</b> 而非娱乐新闻 | <input type="checkbox"/> |
| 我会在本地动手打开文件，而不是只滑手机          | <input type="checkbox"/> |

当你勾选越来越多「是」时，V2 的长途旅行会轻松许多。祝阅读愉快。

# 学习路线图：三条路径，同一种好奇心

## 本节学习目标

- 在 小白路径、开发者路径、架构师路径 之间做出选择，并了解各自投入时间。
- 能用 Mermaid 图向他人解释三条路径的分叉与汇合点。
- 把「篇次」与「周计划」对应起来，避免在 51 万行代码前迷失。

## 为什么需要路线图？

想象你要逛一座 51 万间房 的博物馆：如果没有地图，你会在第一个展厅就累趴下。本书把内容拆成 20 个 Part (见 [index.md](#))，再用三条 学习路径 告诉你：若只有周末、若每晚一小时、若想一口气吃透架构，分别该怎么走。

[Mermaid 图表 — 请在在线版本中查看]

## 时间估算的说明

下表中的「预计学习时间」指：以中文阅读 + 偶尔本地打开文件 + 手绘/复现 Mermaid 为准，不包括 你从零重写一个 Claude Code 的时间。

| 路径    | 覆盖篇次                   | 预计总时长（宽松）   | 预计总时长（紧凑）  |
|-------|------------------------|-------------|------------|
| 小白路径  | Part 01~05（侧重 01 与 02） | 约 18~28 小时  | 约 10~14 小时 |
| 开发者路径 | Part 03~10             | 约 35~50 小时  | 约 22~30 小时 |
| 架构师路径 | Part 01~20 全书          | 约 80~120 小时 | 约 50~70 小时 |

生活类比：小白路径像「学会看地铁图并坐通三条线」；开发者路径像「考下驾照并开城市快速路」；架构师路径像「读完整个城市的交通规划白皮书」。

## 路径一：小白路径（建议 Part 01~05）

适合：第一次听说 Source Map、npm 包体积、Agent Loop；希望先建立信心再碰大仓库。

[Mermaid 图表 — 请在在线版本中查看]

小白路径章节清单

| 顺序 | 文件 / 篇                                  | 你要搞定什么                                           |
|----|-----------------------------------------|--------------------------------------------------|
| 1  | part00-preface/index.md                 | 知道全书结构与 V2 特色                                    |
| 2  | glossary.md                             | 先扫一遍，不必背，留个印象                                    |
| 3  | setup.md                                | 本机能 <code>git clone</code> 、能打开 <code>.ts</code> |
| 4  | part01-background/index.md              | 理解事件时间线与 Source Map 角色                           |
| 5  | part01-background/02-source-overview.md | 记住「1903 文件 / 512K 行」量级                           |
| 6  | part01-background/03-community.md       | 知道社区如何「拼图」                                       |
| 7  | part01-background/04-why-learn.md       | 建立「AI 之外 90%」的心智                                 |
| 8  | part01-background/05-legal-ethics.md    | 建立合法合规阅读习惯                                       |
| 9  | (后续 Part 02~05)                         | 入口、 <code>main</code> 、Query、Loop 初识             |

小白路径 Mermaid：知识依赖

[Mermaid 图表 — 请在在线版本中查看]

里程碑：能向朋友用 3 分钟 讲清楚「cli.js.map 是什么」「为什么这不算传统意义上的黑客入侵」。

路径二：开发者路径（建议 Part 03~10）

适合：熟悉 TypeScript / Node，想快速定位「工具怎么注册」「权限怎么拦」「上下文怎么压」。

[Mermaid 图表 — 请在在线版本中查看]

### 开发者路径周节奏（示例）

| 周次 | 主题                  | 建议产出              |
|----|---------------------|-------------------|
| 1  | Part 01 速览 + 本地仓库跑通 | 一份你自己的「目录导览笔记」    |
| 2  | 入口与配置链路             | 手绘启动序列图           |
| 3  | Agent Loop + Tool   | 表格列出 10 个你最关心的工具名 |
| 4  | 权限 + Compaction     | 写一页「如果我来设计 2.0」草案 |
| 5  | MCP + Hooks         | 给团队内部分享 20 分钟     |

**关键源码阅读提示（示意）：**在超大文件中，优先搜索 `class`、`export async function`、以及 `registerTool` 一类语义锚点——像在海里找浮标。

```
// 开发者路径常用「阅读锚点」伪代码
interface ToolRegistry {
  register(def: ToolDefinition): void;
  dispatch(name: string, input: unknown): Promise<unknown>;
}
```

### 路径三：架构师路径（全书 Part 01~20）

**适合：**要做内部 Agent 平台、关心审计与多租户、需要对比业界方案。

[Mermaid 图表 — 请在在线版本中查看]

### 架构师路径时间拆分（参考）

| 模块    | 内容块        | 建议学时    |
|-------|------------|---------|
| 基础    | Part 00~01 | 8~12 h  |
| 核心链路  | Part 02~05 | 15~22 h |
| 工具与安全 | Part 06~09 | 18~25 h |
| 扩展与编排 | Part 10~15 | 20~30 h |
| 性能与对比 | Part 16~20 | 15~22 h |

**生活类比：**架构师路径不是「旅游」，而是「带着测量仪做城市体检」——你会反复问：边界在哪？失效模式是什么？谁能绕过？

## 三条路径的汇合点

无论你从哪条路上来，最终都会在以下概念上 **汇合**（后续篇目会展开）：

| 汇合概念            | 一句话             |
|-----------------|-----------------|
| Agent Loop      | 计划、行动、观察、再计划的循环 |
| Tool            | 模型以外的「手脚」       |
| Permission Mode | 哪些手脚要先敲门        |
| Compaction      | 记忆太长时的裁剪与摘要     |
| MCP             | 标准化的外接能力协议      |

[Mermaid 图表 — 请在在线版本中查看]

## 自测：你该选哪条路？

| 问题                                 | 更偏「是」则倾向          |
|------------------------------------|-------------------|
| 我看到 <code>tsconfig.json</code> 会心慌 | 小白路径              |
| 我能独立写一个 Express 中间件                | 开发者路径             |
| 我要写 ADR 和安全评审                      | 架构师路径             |
| 我只想周末翻翻                            | 小白路径 + 只读 Part 01 |
| 我要给团队定规范                           | 架构师路径 + 强制读完法律伦理篇 |

## 与本书其他文档的衔接

- 术语迷路 → [glossary.md](#)
- 机器没配好 → [setup.md](#)
- 全书导览 → [index.md](#)

## 附录：每日 45 分钟模板（可复制）

| 分钟    | 动作                               |
|-------|----------------------------------|
| 0~5   | 复习昨天的一张 Mermaid 图                |
| 5~25  | 读一节正文 + 对照术语表                    |
| 25~40 | 本地打开 1~2 个相关 <code>.ts</code> 文件 |
| 40~45 | 用三句话写「今天我搞懂了...」                 |

坚持四周，小白路径的核心篇就能扎实走完第一轮。下一程，我们在 [glossary.md](#) 里把名词换成你能使唤的「口令」。

## 术语表：把「行话」变成你能用的口令

### 本节学习目标

- 掌握 30+ 个本书高频术语，能在对话中正确使用英文原名与中文释义。
- 通过 **生活类比** 把抽象概念钉在记忆里。
- 借助 **Mermaid 关系图** 理解术语之间的调用与包含关系。

### 使用说明

- 下表按 **概念域** 分组；每组附 **类比** 与 **在 Claude Code 语境下的提示**。
- 若某实现细节随社区重建仓库版本变化，本书以 **教学一致性** 优先，必要时注明「以你本地 checkout 为准」。

[Mermaid 图表 — 请在在线版本中查看]

## A. Agent 与对话核心

| 术语    | 英文 / 缩写            | 解释                                                 | 生活类比                         |
|-------|--------------------|----------------------------------------------------|------------------------------|
| 智能体循环 | Agent Loop         | 模型在「思考 → 选行动 → 看结果 → 再思考」之间循环，直到任务结束或被打断。          | 餐厅后厨：接单、切配、下锅、尝味、补盐，直到出餐。    |
| 查询引擎  | QueryEngine        | 把用户输入、历史消息、系统提示等 <b>编排</b> 成可提交给模型与工具层的一次「查询」处理管线。 | 机场值机柜台：收证件、打登机牌、行李贴条、指引登机口。  |
| 协调器   | Coordinator        | 在多模块或多子代理之间分配步骤、合并结果、处理冲突的高层角色（具体类名以源码为准）。         | 电影场记 + 副导演：谁上场、谁先说话、台词错了谁重拍。 |
| 对话压缩  | Compaction         | 当上下文过长时，对历史进行 <b>摘要、裁剪或结构化折叠</b> ，腾出 token 窗口。     | 行李箱装不下时，把厚羽绒服换成真空袋。          |
| 验证代理  | Verification Agent | 用于对主代理输出或工具结果做 <b>二次校验</b> 的逻辑（如静态检查、对拍、规则引擎）。     | 试卷上的「第二阅卷老师」。                |

## B. 工具、权限与安全

| 术语   | 英文 / 缩写         | 解释                                        | 生活类比                     |
|------|-----------------|-------------------------------------------|--------------------------|
| 工具   | Tool            | 模型可调用的外部能力单元（读文件、跑命令、搜索等），通常有 schema 与实现。 | 瑞士军刀上的每一把小工具。            |
| 权限模式 | Permission Mode | 决定工具调用前是否需要用户确认、默认允许列表、沙箱边界等策略集合。         | 小区门禁：业主卡 / 访客登记 / 装修临时证。 |
| 功能开关 | Feature Flags   | 按用户、环境或随机百分比打开/关闭某功能，用于灰度与实验。             | 游乐园「试运营」项目：只有持蓝票的游客能进。   |
| 钩子   | Hooks           | 在生命周期的固定点插入自定义逻辑（前后置、审计、改写）。              | Word 文档的「保存前自动跑宏」。       |

[Mermaid 图表 — 请在在线版本中查看]



## C. 协议、桥接与项目记忆

| 术语                        | 英文 / 缩写                   | 解释                                              | 生活类比                    |
|---------------------------|---------------------------|-------------------------------------------------|-------------------------|
| 模型上下文协议                   | MCP                       | 一种让 AI 应用以标准方式连接外部数据源与工具生态的协议（Anthropic 推动）。    | USB-C：不同厂家设备用同一接口充电传数据。 |
| 桥                         | Bridge                    | 在 CLI 与 MCP 服务、子进程或宿主环境之间转发消息与生命周期的组件。          | 口岸大桥：车辆换向、检查、计费。        |
| <a href="#">CLAUDE.md</a> | <a href="#">CLAUDE.md</a> | 项目根或约定路径下的 <b>持久说明文件</b> ，给模型提供风格、命令、目录约定等长期记忆。 | 新同事入职文件夹里的「我们团队怎么协作」。   |

## D. 工程与构建相关

| 术语                      | 英文 / 缩写                 | 解释                                                                  | 生活类比                    |
|-------------------------|-------------------------|---------------------------------------------------------------------|-------------------------|
| Source Map              | <code>.map</code>       | 将压缩/打包后的 JS 映射回原始 TS 源的文件，便于调试。                                     | 翻译书里的「边注」：英文版页码对照中文版段落。 |
| npm 包                   | npm package             | 发布到 npm registry 的 tarball，内含 <code>package.json</code> 声明的入口与文件列表。 | 超市货架上的「预制菜包装盒」。         |
| <code>.npmignore</code> | <code>.npmignore</code> | 告诉 npm <b>不要</b> 打包哪些路径；若配置失误，可能把不该上传的文件打进包。                        | 搬家纸箱上写「此箱勿封」却还是被胶带封死了。  |
| Stub 模块                 | Stub                    | 仅占位、返回空实现或最小实现的模块，用于让 TypeScript 编译通过。                              | 电影拍摄时的「替身站位」，后期再换真演员。   |
| Coordinator（文件域）        | coordinator/            | 源码树中常与「多步骤任务编排」相关的目录（具体以仓库为准）。                                      | 交响乐团指挥台附近的乐谱架区。         |

## E. 产品与社区语境

| 术语          | 英文 / 缩写                  | 解释                                       | 生活类比                         |
|-------------|--------------------------|------------------------------------------|------------------------------|
| Claude Code | Claude Code              | Anthropic 的终端 AI 编程代理产品，负责在本地/远程环境中辅助开发。 | 住在终端里的「结对程序员」。               |
| 客户端编排       | client orchestration     | 相对「模型推理服务」而言，运行在用户侧的调度、工具、UI、配置逻辑。       | 外卖 App：店家做饭是云端，骑手调度是客户端周边系统。 |
| 泄露 vs 入侵    | leak vs breach           | 泄露常指 误发布 敏感产物；入侵强调 未授权访问。二者法律评价不同。       | 把日记本落在公交上 vs 有人撬锁进你家偷日记。     |
| 社区重建        | community reconstruction | 爱好者根据泄露或公开片段 恢复可浏览源码树 的努力，常缺构建脚本。        | 用碎瓷片拼出花瓶形状，但可能缺几块。           |

## F. 扩展术语（补足 30+）

| 术语                 | 简要解释                                                       |
|--------------------|------------------------------------------------------------|
| Assistant          | 常与对话侧、建议侧 UI 或子系统相关（具体命名以源码为准）。                            |
| Buddy              | 源码树中可能出现的辅助模块/实验名，宜对照目录而非臆测功能。                             |
| Plugins            | 插件机制：在核心外加载扩展命令或工具。                                        |
| Commands           | 用户可触发的子命令实现目录（如 <code>src/commands</code> 一类）。             |
| Services           | 长生命周期服务：配置、遥测、会话等（如 <code>src/services</code> ）。           |
| Utils              | 通用工具函数集合，避免业务目录臃肿。                                         |
| main.tsx / main.ts | CLI 或 UI 入口之一，常体积巨大，是阅读锚点。                                 |
| query.ts           | 与查询/会话管线强相关的核心文件之一。                                        |
| Tool.ts            | 工具抽象定义或基类可能出现的位置。                                          |
| R2                 | Cloudflare R2 对象存储；若构建链上传产物，可能与发布物存储相关（教学语境下常作为「CDN/桶」代称）。 |
| Bun                | 高性能 JS 运行时/工具链；某些构建默认行为（如生成 Source Map）会影响发布物。             |
| TypeScript         | 带类型的 JavaScript 超集；本书源码主线语言。                               |
| Registry           | 工具/命令的注册表模式：名字到实现的映射。                                      |
| Telemetry          | 匿名或脱敏的使用数据上报（若实现存在，应以隐私政策为准）。                              |
| Sandbox            | 限制文件/网络/子进程权限的执行环境策略（概念层）。                                 |

## G. 关系速查：谁包含谁？

[Mermaid 图表 — 请在在线版本中查看]

## H. 易混淆对比表

| 对比组      | A          | B               | 记忆口诀                         |
|----------|------------|-----------------|------------------------------|
| 协议 vs 实现 | MCP        | Bridge          | MCP 是「插座标准」，Bridge 是「插线板」。   |
| 循环 vs 引擎 | Agent Loop | QueryEngine     | Loop 像「心跳」，Engine 像「血液循环系统」。 |
| 压缩 vs 权限 | Compaction | Permission Mode | 一个管「记不记得住」，一个管「让不让做」。        |
| 钩子 vs 插件 | Hooks      | Plugins         | Hooks 像「事件」，Plugins 像「配件包」。  |

## I. 关键源码「指路牌」（示意片段）

下列代码 仅为教学示意，帮助你把术语和「可能在源码里长什么样」联系起来：

```
// PermissionMode: 策略枚举或配置对象（示意）
type PermissionMode = "default" | "acceptEdits" | "plan" | "bypass";

// Tool: 名字 + 入参 schema + 执行体（示意）
interface ToolDefinition {
  name: string;
  description: string;
  execute: (input: unknown) => Promise<unknown>;
}

// Compaction: 可能在对话历史过长时触发（示意）
async function maybeCompactThread(thread: Message[]): Promise<Message[]> {
  if (estimateTokens(thread) > SOFT_LIMIT) return summarize(thread);
  return thread;
}
```

## J. 推荐阅读顺序（针对术语表）

1. 先扫 A、B 两组（Agent + 工具权限）。
2. 再读 C（MCP / [CLAUDE.md](#)）。
3. 遇到构建新闻时回到 D（Source Map / npm）。
4. 写读书报告前复习 H（易混表）。

## K. 空白笔记区（可自行打印填写）

我自己造的句子（用上 5 个术语）

术语表会随全书更新而增补。若你在社区重建仓库里发现 **同名不同义** 的文件（例如两个 `Tool.ts` ），以 **路径前缀** 区分，并在笔记本里画一张「同名文件地图」——这是阅读超大仓库的必备习惯。下一节：`setup.md` 把环境一次配稳。

## 环境搭建：工欲善其事，必先利其器

### 本节学习目标

- 在 macOS / Linux / Windows（WSL2 推荐）上完成 **Node.js**、**Bun（可选）**、**Git** 的安装与版本自检。
- 能使用 **HTTPS 或 SSH** 克隆社区重建仓库，并理解为何可能没有完整 `package.json` 。
- 配置 **VS Code / Cursor** 等编辑器，获得 TypeScript 跳转、搜索、大纲视图的良好体验。

## 总览：你的「考古工作台」长什么样？

[Mermaid 图表 — 请在在线版本中查看]

**生活类比**：读 51 万行源码像 **考古发掘**——Node/Bun 是铲子，Git 是记录坑位的笔记本，编辑器是放大器和标签盒。

## 1. 操作系统建议

| 系统      | 建议               | 原因                 |
|---------|------------------|--------------------|
| macOS   | 直接跟本书步骤          | 与多数开发者文档一致         |
| Linux   | 任选主流发行版          | 路径与权限更透明           |
| Windows | 优先 WSL2 (Ubuntu) | 避免路径大小写、脚本换行、符号链接坑 |

[Mermaid 图表 — 请在在线版本中查看]

## 2. 安装 Node.js (推荐 LTS)

### 2.1 为什么需要 Node?

即使 Claude Code 的某些脚本用 Bun 构建，**社区重建仓库** 往往仍依赖 Node 生态工具（`npm`、`npx`、语言服务）。把 Node 装好，等于备好「通用螺丝刀」。

### 2.2 安装方式对比

| 方式            | 优点    | 缺点      |
|---------------|-------|---------|
| 官网安装包         | 最简单   | 多版本切换不便 |
| nvm / fnm     | 多版本共存 | 多一步学习成本 |
| 包管理器 (brew 等) | 与系统集成 | 版本可能滞后  |

### 2.3 自检命令

在终端执行：

```
node -v
npm -v
```

期望：`node` 为 **当前 LTS 主版本线**（例如 v22.x 或官方当时推荐的 LTS，以 [nodejs.org](https://nodejs.org) 为准）；`npm` 能正常输出版本号。

## 2.4 常见故障

| 现象                                   | 可能原因         | 处理方向                                       |
|--------------------------------------|--------------|--------------------------------------------|
| <code>command not found: node</code> | 未安装或未加入 PATH | 重装或检查 shell 配置                             |
| 权限错误                                 | 全局目录不可写      | 用 nvm/fnm 或修复目录权限                          |
| 公司代理                                 | 证书拦截         | 配置 <code>npm config set proxy</code> 或信任证书 |

## 3. 安装 Bun（可选但推荐）

### 3.1 Bun 与本次泄露新闻的关系

公开讨论中提到：Bun 默认行为可能生成 Source Map，若发布流程未剔除，容易导致 `.map` 进入 npm tarball。学习 Bun 有助于你理解 构建链默认值 = 安全与合规的盲区。

### 3.2 安装

官方一键脚本会随时间更新，请以 <https://bun.sh> 为准。装好后自检：

```
bun -v
```

### 3.3 Node vs Bun：并存策略

[Mermaid 图表 — 请在在线版本中查看]

生活类比：Node 像 国标电源；Bun 像 快充头——很多设备两个都能充，但别混用不认识的线。

## 4. 安装 Git

### 4.1 为什么必须会 Git?

社区仓库频繁更新；你要 `diff`、`blame`、`checkout` 到某提交 对照本书例子。不会 Git，就像考古不用分层记录。

## 4.2 安装与配置

```
git --version
git config --global user.name "Your Name"
git config --global user.email "you@example.com"
```

## 4.3 SSH vs HTTPS

| 方式    | 适合   | 备注                                            |
|-------|------|-----------------------------------------------|
| HTTPS | 新手最快 | 可能需要 token                                    |
| SSH   | 长期开发 | 需生成 <code>~/.ssh/id_ed25519</code> 并加到 GitHub |

[Mermaid 图表 — 请在在线版本中查看]

# 5. 克隆「社区重建」源码的方法

## 5.1 仓库示例（教学引用）

本书背景中常提及：

- `leaked-claude-code`（体量与 star 数会变化）
- `sanbuphy` 等分析向仓库（star 较高，偏结构化导读）

**重要：**本书 **不提供** 具体侵权或未授权完整分发物的链接；请你只克隆 **你确认合法** 的公开仓库，并遵守其许可证与 GitHub 政策。

## 5.2 克隆命令模板

```
mkdir -p ~/src && cd ~/src
git clone https://github.com/OWNER/REPO.git claude-code-study
cd claude-code-study
git log -1 --oneline
```



### 5.3 你可能遇到的「不完整仓库」

| 情况                          | 说明        | 应对                                     |
|-----------------------------|-----------|----------------------------------------|
| 缺 <code>package.json</code> | 重建者只提供源码树 | 用编辑器只读浏览，不强求 <code>npm i</code>        |
| 大量 <code>stub</code>        | 为通过类型检查占位 | 阅读接口签名，理解意图                            |
| 依赖需逆向                       | 私有包名未公开   | 把 <code>import</code> 当「购物清单」，逐项查公开替代品 |

[Mermaid 图表 — 请在在线版本中查看]

## 6. 推荐编辑器配置 (VS Code / Cursor)

### 6.1 必装扩展 (建议)

| 扩展                           | 作用      |
|------------------------------|---------|
| TypeScript / JavaScript (内置) | 类型提示与跳转 |
| ESLint (如项目有配置)              | 风格与常见错误 |
| Markdown All in One          | 读本书配套笔记 |
| Mermaid 预览类扩展                | 预览本文图表  |

### 6.2 `settings.json` 片段 (可选)

下列设置提升大仓库体验 (路径与键名以编辑器版本为准):

```
{
  "editor.tabSize": 2,
  "editor.wordWrap": "on",
  "typescript.tsserver.maxTsServerMemory": 8192,
  "search.exclude": {
    "**/node_modules": true,
    "**/dist": true
  }
}
```

### 6.3 大文件阅读技巧

| 技巧     | 快捷键 / 操作（常见绑定）                                      |
|--------|-----------------------------------------------------|
| 按符号跳转  | Command Palette → Go to Symbol                      |
| 全局搜索   | <code>rg</code> 或编辑器搜索（尊重 <code>.gitignore</code> ） |
| 折叠无关区域 | Fold All / 手动折叠                                     |
| 书签     | 扩展或 TODO 注释                                         |

[Mermaid 图表 — 请在在线版本中查看]

生活类比：读 4000 行文件像读 **字典**：没人从第一页背到最后一页，都是先查索引。

### 7. 可选：终端增强

| 工具                                     | 用途                                         |
|----------------------------------------|--------------------------------------------|
| <code>tmux</code>                      | 分屏：一边 <code>git</code> ，一边 <code>rg</code> |
| <code>fd</code> / <code>ripgrep</code> | 比默认搜索更快（可选）                                |

示例 `ripgrep`：

```
rg "class QueryEngine" -n .
```

### 8. 网络与磁盘

| 资源 | 粗估               | 建议                                      |
|----|------------------|-----------------------------------------|
| 磁盘 | 视仓库大小，数 GB 级 不罕见 | 预留空间给 <code>.git</code>                 |
| 网络 | clone 可能较慢       | 镜像或浅克隆 <code>git clone --depth 1</code> |

[Mermaid 图表 — 请在在线版本中查看]

## 9. 验证清单（打勾即用）

| 检查项                     | 命令 / 结果                       |
|-------------------------|-------------------------------|
| Node 可用                 | <code>node -v</code>          |
| Git 可用                  | <code>git --version</code>    |
| 能克隆公开仓库                 | <code>git clone ...</code> 成功 |
| 编辑器能打开 <code>.ts</code> | 有语法高亮                         |
| 能全局搜索字符串                | <code>rg</code> 或 IDE 搜索成功    |

## 10. 下一步

环境就绪后，请回到 **Part 01**：

- `../part01-background/index.md` 了解事件时间线
- `glossary.md` 随时查词

## 附录 A：WSL2 快速备忘（Windows）

1. 启用 WSL2 与虚拟机平台（微软文档为准）。
2. 安装 Ubuntu。
3. 在 Ubuntu 内安装 Node、Git。
4. 项目文件放在 Linux 文件系统（`/home/...`），不要放在 `/mnt/c/...` 以避免 I/O 与权限问题。

## 附录 B：公司电脑合规提示

若你在企业设备上操作：

- 先问清 是否允许 克隆第三方重建仓库。
- 勿将 内部代码 与外部仓库混在同一工作区。
- 遥测、上传、云同步策略以公司安全规范为准。

## 附录 C：故障树（简版）

---

[Mermaid 图表 — 请在在线版本中查看]

当你的编辑器第一次成功跳转到某个 `export function` 的定义时，你会感到：51 万行不再是洪水，而是一条 **可以溯流而上** 的河。装备齐了吗？我们进背景篇。

---

## 第1篇：背景故事

### 1.1 泄露事件始末：当「调试地图」搭上了公开发货的便车

#### 本节学习目标

- 按时间线复述 2026-03-31 前后社区观察到的 npm 包异常与 Source Map 争议。
- 解释 cli.js.map、体积从约 17MB 到约 31MB 等事实在工程上意味着什么。
- 理解 Bun 默认 Source Map、.npmignore 漏洞、R2 存储桶 等关键词在叙事中的角色（区分「传闻」与「可验证机制」）。

#### 导语：一盒变重的「快递」

想象你在网上买了一盒 标称 17MB 的饼干。快递到手一掂，沉得离谱——上秤发现 31MB。你撕开包装，发现除了饼干，还塞了一本 60MB 厚的「工厂内部工艺流程图册」（而且图册本该只留在厂里）。

对软件来说，那本「图册」就是 Source Map：它让工程师能把压缩后的 cli.js 对照回 TypeScript 源码行号。对终端用户而言，它通常 不该出现在公开发布的 npm tarball 里——就像顾客不该收到工厂图纸。

本书这一部分讲 故事脉络；下一节讲 源码规模；法律与伦理见 05-legal-ethics.md。

#### 人物与发现（公开叙述层面）

公开讨论中常提到安全研究者 Chaofan Shou 对 Anthropic 发布到 npm 的 Claude Code v2.1.88 包的观察：包体异常膨胀，并指向 Source Map 文件的存在。具体技术细节以研究者公开材料与 npm 上可核验的发布物为准。

[Mermaid 图表 — 请在在线版本中查看]

生活类比：研究者像 质检员——不是「黑进工厂」，而是「买到市售品后发现多了一张不该附带的图纸」。

## 体量数字：为什么社区会震惊？

下面数字来自本书 教学引用 的背景材料（具体以你本地核验的包版本为准）：

| 项目             | 约数 / 描述                       |
|----------------|-------------------------------|
| 包体积变化          | 约 17MB → 31MB                 |
| 多出的 Source Map | 约 60MB 级别讨论（cli.js.map）       |
| 还原规模叙事         | 约 51.2 万行 TypeScript、1903 个文件 |
| 内容性质           | 客户端编排层；非 模型权重                 |

[Mermaid 图表 — 请在在线版本中查看]

类比：模型权重像 可口可乐秘方；客户端源码像 自动售货机的机械结构——泄露后者仍然严重，但不要混谈成「配方被偷」。

## 时间线：把散落的报道拼成一张图

下列时间线为 教学用整理，日期与事件描述综合公开讨论与历史类似案例；正式引用请回溯一手来源。

[Mermaid 图表 — 请在在线版本中查看]

### 表格版时间线

| 阶段   | 时间（约）      | 发生了什么               | 技术侧关注点         |
|------|------------|---------------------|----------------|
| 先例记忆 | 2025-02    | 社区回忆/对比「曾经也发生过」     | 发布流程是否系统性风险    |
| 触发观察 | 2026-03-31 | npm 包体积与 map 文件引发讨论 | tarball 内容清单审计 |
| 扩散分析 | 2026-04    | 重建仓库、导图、文章井喷        | 合规、版权与教育边界     |

[Mermaid 图表 — 请在在线版本中查看]

## Source Map 到底是什么？

### 一句话

Source Map 是 压缩/打包代码 与 原始源码位置 之间的索引。

### 稍微展开

当你把 TypeScript 编译成单文件 `cli.js` 时：

- 浏览器或 Node 实际执行的是 `cli.js`。
- 若同时生成 `cli.js.map`，调试器可以把错误栈 映射回 `.ts` 行号。

若 `.map` 与 `.js` 一起发布到 npm，下载者就有机会 高度还原 源码结构（配合反编译/格式化工具）。

[Mermaid 图表 — 请在在线版本中查看]

### 类比表

| 概念                      | 类比              |
|-------------------------|-----------------|
| <code>cli.js</code>     | 餐厅给顾客的「成品菜」     |
| <code>cli.js.map</code> | 后厨墙上贴的「切配与火候笔记」 |
| npm 包                   | 外卖盒             |
| 误塞 map                  | 外卖盒里夹了整张后厨 SOP  |

## Bun 与「默认生成 Source Map」的讨论

公开分析中常提到：Bun 在构建链路里 可能默认生成 Source Map。这意味着：

- 若 CI/CD 没有显式关闭 map 输出，产物目录里会出现 `.map`。
- 若 `.npmignore` 或 `files` 白名单 没有排除 map，tarball 就会 带着地图出货。

**重要：**这是 **流程与默认值** 的讨论，不是「Bun 有错」或「某一方故意」的单选题——工程上更像 **瑞士奶酪模型**：每层一个小洞，叠在一起就会穿。

[Mermaid 图表 — 请在在线版本中查看]

## .npmignore 与 files 字段：「装箱单」事故

npm 打包规则可以简单理解为：

- `package.json` 的 `files` 字段：「只装这些」。
- `.npmignore`：「别装这些」（类似 `.gitignore`）。

若规则 **漏写**、**被覆盖**、或与构建输出目录 **不同步**，就会出现「以为没装，其实装了」。

| 机制                      | 作用   | 典型失误                                                 |
|-------------------------|------|------------------------------------------------------|
| <code>files</code>      | 白名单  | 写了 <code>dist</code> 但没排除 <code>dist/**/*.map</code> |
| <code>.npmignore</code> | 黑名单  | 规则不覆盖新目录                                             |
| 预处理脚本                   | 复制产物 | 把 <code>build/</code> 整个拷进待发布树                       |

**生活类比：**搬家时列了「书房要装箱」，结果把书房 **抽屉里的机密笔记本** 也扫进去了——不是小偷进屋，是 **装箱单不细**。

## R2 存储桶：它在叙事里扮演什么？

部分讨论将发布物与 **Cloudflare R2** 一类对象存储联系起来，用以解释 **大文件分发** 或 **构建产物托管**。本书建议读者：

- 把 R2 理解为「**放安装包的仓库货架**」；
- 具体某产品是否使用、如何使用，以官方架构为准，避免过度臆测。

[Mermaid 图表 — 请在在线版本中查看]



## 2025 年 2 月：「又一次」为何刺痛神经？

若同类事件 重复发生，社区情绪会从「偶然」转向「系统性疑问」：

| 视角 | 问题                       |
|----|--------------------------|
| 工程 | 发布 checklist 是否缺自动化校验？   |
| 合规 | 是否应监控 tarball 文件类型与大小基线？ |
| 生态 | 竞争对手、恶意镜像站是否会长期存档？       |

[Mermaid 图表 — 请在在线版本中查看]

## 与「黑客攻击」的边界（预告）

本节只建立事实感；法理与伦理展开见 1.5。此处给出口诀：

- 误发布：像 把机密复印件落在复印店取件台。
- 入侵：像 有人撬锁进文印室偷复印件。

二者都可能造成损失，但 归责与响应 不同。

## 关键「源码片段」长什么样？（教学示意）

你下载到的 map 并不会等于「漂亮的 .ts 文件树」，但会极大降低还原成本。下面是一段 虚构的 map 片段结构示意：

```
{
  "version": 3,
  "file": "cli.js",
  "sources": ["../src/entry.ts", "../src/tooling/registry.ts"],
  "sourcesContent": ["/ * 可能被内联的源码片段 */"],
  "mappings": "AAAA;AAAA;AAAA;"
}
```

真实文件中 mappings 是压缩编码；sourcesContent 是否存在取决于生成选项。**\*\* point\*\***：只要信息足够，工具链就能 重建 近似原始仓库。

## 本节小结表

| 关键词        | 你应带走的理解                  |
|------------|--------------------------|
| v2.1.88    | 体量异常引发 scrutiny 的版本锚点之一  |
| cli.js.map | Source Map；调试友好 ≠ 公开发布友好 |
| 17→31MB    | 用户侧可感知的「变重」              |
| Bun        | 默认值与构建链一环                |
| .npmignore | 装箱黑名单；失误可导致 map 漏网       |
| R2         | 对象存储叙事中的「货架」隐喻           |
| 编排层        | 泄露讨论聚焦客户端逻辑，不是权重         |

## 延伸思考题

1. 若你是发布负责人，会如何设计「禁止 .map 入 tarball」的 CI 断言？
2. Source Map 是否应区分 内网调试包 与 公网包 两种发布通道？
3. 社区重建仓库对 安全研究 是助力还是误导（许可证碎片化）？

## 下一节导航

- 1.2 源码规模全貌：[02-source-overview.md](#)
- 术语复习：[../part00-preface/glossary.md](#)

## 附录：对话剧本（帮助小白复述）

小明：听说 Claude Code 被黑了？

小红：更准确说，是 公开发布的安装包 里 多带了调试地图，别人能 还原大量 TS 源码。这不等于「模型权重泄露」，但很严肃。

小明：为啥会带？

小红：常见是 构建默认生成 map + 发布清单没剔除 + 监控没报警 叠在一起——像搬家装箱不检查抽屉。

## 附录：检查清单（企业发布用）

| 步骤 | 检查项                                           |
|----|-----------------------------------------------|
| 1  | <code>npm pack --dry-run</code> 列出 tarball 内容 |
| 2  | 搜索 <code>.map</code> 后缀                       |
| 3  | 对比上一版本体积差异阈值                                  |
| 4  | 对 <code>sourcesContent</code> 内联策略做显式配置       |
| 5  | Artifacts 分「公网」「内网调试」通道                       |

当时间线在你脑海里从散点连成折线，你会发现：这不是单一「坏人坏事」的惊悚片，而是一部 **流程喜剧里的悲剧桥段**——而我们要做的，是从桥段里学到 **如何把门关好**。继续读 1.2，看看这 51 万行「城市」的地图长什么样。

## 1.2 源码规模全貌：五十一万行不是「一个文件」，而是一座城

### 本节学习目标

- 用 **表格 + 图** 建立对 **1903 文件 / 约 51.2 万行** 的数量级直觉。
- 认识主要目录分工：`tools`、`commands`、`services`、`utils`、`coordinator`、`assistant`、`buddy`、`plugins` 等。
- 知道 `main.tsx`（约 4684 行）、`query.ts`（约 1730 行）、`Tool.ts`（约 793 行）这类「地标文件」在阅读策略中的意义。

### 开场：别被数字吓到

51.2 万行 听起来像一辈子读不完。但城市也有上千万人口——你不需要认识每一个人，只需要 **地铁图、行政区、几个地标广场**。

本书把 Claude Code 客户端编排层想象成一座 **垂直城市**：

- **地下**：工具与系统调用（ `tools` 、部分 `services` ）
- **地面**：命令行入口与用户交互（ `commands` 、 `main` ）
- **空中**：协调、插件、扩展（ `coordinator` 、 `plugins` ）
- **市政厅**：配置、会话、核心查询管线（ `query` 、 `services` ）

[Mermaid 图表 — 请在在线版本中查看]

## 规模总表（教学用锚点数字）

下列数字来自本书背景材料；你本地 checkout 可能因重建版本不同而略有偏差。

| 指标    | 数量级        | 备注                    |
|-------|------------|-----------------------|
| 文件数   | 1903       | 含 TS/TSX 等（以社区统计口径为准） |
| 代码行数  | 约 512,000  | TypeScript 为主         |
| 超大单文件 | 多个 1000+ 行 | 阅读时宜「符号跳转」而非线性刷       |

[Mermaid 图表 — 请在在线版本中查看]

生活类比：1903 个文件像 1903 个街区门牌；你不会挨家敲门，你会查 邮编分区。

## 地标文件：三个「城市广场」

| 文件（示例路径以重建仓库为准）       | 约行数  | 角色比喻                  | 阅读建议              |
|-----------------------|------|-----------------------|-------------------|
| <code>main.tsx</code> | 4684 | 机场航站楼：人流进出、指示牌最多      | 先画「启动→子命令→主循环」    |
| <code>query.ts</code> | 1730 | 市政调度中心：一次 query 如何被处理 | 对照 Agent Loop 术语表 |
| <code>Tool.ts</code>  | 793  | 工具博物馆导览册：抽象层级高        | 先读接口再读实现          |

[Mermaid 图表 — 请在在线版本中查看]

类比：

- `main` 像 酒店前台：你一进大堂就被分流。
- `query` 像 客服工单系统：问题编号、转接、回访。
- `Tool` 像 外包服务商名录：每项服务有合同模板（schema）。

目录结构鸟瞰（概念图）

下列目录名为 **典型** 命名；请以你本地树为准。

[Mermaid 图表 — 请在在线版本中查看]

表格：目录 → 职责（教学版）

| 目录          | 一句话职责                | 生活类比              |
|-------------|----------------------|-------------------|
| tools       | 对外能力的具体实现与注册         | 工具房里的电钻、扳手        |
| commands    | CLI 子命令、用户显式操作入口     | 遥控器上的按钮           |
| services    | 长生命周期能力：配置、会话、集成     | 物业中心：水电抄表         |
| utils       | 共享小函数，避免复制粘贴         | 公用螺丝刀抽屉           |
| coordinator | 多步骤任务协调、状态推进         | 婚礼司仪的流程卡          |
| assistant   | 与助手体验相关的模块集合         | 导游小旗与讲解词          |
| buddy       | 辅助/伙伴式交互或实验功能（以源码为准） | 旅伴 App：提醒喝水       |
| plugins     | 插件加载与扩展边界            | App Store 里的第三方模块 |

[Mermaid 图表 — 请在在线版本中查看]

模块数量叙事：4756 与「文件」的差异

背景材料中有时出现「4756 个模块文件」一类统计，与 1903 文件 似乎不一致。常见原因包括：

| 口径    | 可能含义                     |
|-------|--------------------------|
| 物理文件数 | 磁盘上的 <code>.ts</code> 数量 |
| 模块单元数 | 含 barrel 导出、子路径、生成文件等统计  |
| 工具链统计 | bundler 图节点数             |

建议：把数字当作 量级灯塔，不要在本书学习阶段纠结个位数差异。

[Mermaid 图表 — 请在在线版本中查看]

## 依赖与构建：为什么「clone 了也跑不起来」？

社区重建版常见痛点：

| 痛点                          | 解释               |
|-----------------------------|------------------|
| 缺 <code>package.json</code> | 无法一键安装依赖         |
| 60+ npm 依赖 需逆向              | 私有 scope 或内部包名占位 |
| 90+ stub                    | 只为类型检查存在的空壳      |

[Mermaid 图表 — 请在在线版本中查看]

类比：你拿到的是 建筑剖面图 而不是 精装房——能学结构，别强求当晚入住。

## 关键源码片段（示意）：Tool 抽象

下面是一段 教学伪代码，展示阅读 `Tool.ts` 一带时你会寻找的结构：

```
// 示意：真实源码更复杂
export interface ToolContext {
  cwd: string;
  signal: AbortSignal;
}

export abstract class Tool {
  abstract readonly name: string;
  abstract run(input: unknown, ctx: ToolContext): Promise<unknown>;
}
```

读到这里你可以自问：

- 1. 权限在哪里介入？（前后是否有 `PermissionMode`）
- 2. 遥测是否包裹 `run`？
- 3. 错误如何回到 `QueryEngine`？

## 关键源码片段（示意）：Query 管线

```
// 示意：query 文件常包含「状态机」味道
type QueryPhase = "collect" | "plan" | "act" | "summarize";

async function advanceQuery(state: QueryState): Promise<QueryState> {
  switch (state.phase) {
    case "collect":
      return { ...state, phase: "plan" };
    case "plan":
      return { ...state, phase: "act" };
    default:
      return state;
  }
}
```

类比：Query 像 快递路由扫描——包裹每到一个分拣中心就换状态。

## 阅读策略对比表

| 策略                             | 适合                                                                      | 风险           |
|--------------------------------|-------------------------------------------------------------------------|--------------|
| 从 <code>main</code> 顺序往下滚      | 想有「电影感」                                                                 | 极易在 4000 行迷路 |
| 从 <code>Tool</code> 抽象倒推       | 想搞清能力边界                                                                 | 可能缺上下文       |
| 从 <code>commands</code> 用户路径切入 | 有 CLI 使用经验                                                              | 可能漏后台服务      |
| 折中（推荐）                         | <code>main</code> 找启动 → <code>query</code> 找状态 → <code>tools</code> 找实现 | 需自己做笔记       |

[Mermaid 图表 — 请在在线版本中查看]

## 体量对比：和常见开源项目比一比

| 项目类型      | 粗略体量感  | 说明                   |
|-----------|--------|----------------------|
| 小型 CLI 工具 | 数千~一万行 | 单仓库可读性强              |
| 中型应用      | 数万~十万行 | 需要模块化纪律              |
| 本客户端编排层   | 五十万行级  | 接近「语言服务」或大型 IDE 插件体量 |
| 浏览器内核     | 千万行以上  | 不在同一比较平面             |

结论：这是 严肃工业软件，不是课后习题。

## Mermaid：文件依赖的「虚构示例」

真实依赖需用工具生成；下图仅帮助建立直觉：

[Mermaid 图表 — 请在在线版本中查看]

## 与 Part 00 术语的挂钩

| 术语            | 在目录中的「家」                                        |
|---------------|-------------------------------------------------|
| Tool          | <code>tools/</code> + <code>Tool.ts</code> 一带   |
| QueryEngine   | <code>query.ts</code> 及关联 <code>services</code> |
| MCP / Bridge  | 常在 <code>services</code> 或集成子目录                 |
| Hooks         | 可能在 <code>core</code> 生命周期附近                    |
| Feature Flags | <code>utils</code> 或 <code>services</code> 配置层  |

## 实战小练习

- 本地运行 `find . -name '*.ts' | wc -l` 与仓库说明对比。
- 列出体积最大的 10 个文件：`find . -name '*.ts' -print0 | xargs -0 wc -l | sort -n | tail`。
- 为 `coordinator` 画一张 仅含公开导出符号 的草图。



## 下一节导航

- 1.3 社区反应： [03-community.md](#)
- 路线图： [../part00-preface/roadmap.md](#)

## 附录：「行数」如何误导人？

| 陷阱     | 说明              |
|--------|-----------------|
| 生成代码   | 可能膨胀行数但不代表人工复杂度 |
| 长模板字符串 | UI 或提示词占行       |
| 重复模式   | 多个类似 command 文件 |

[Mermaid 图表 — 请在在线版本中查看]

## 附录：笔记本模板

今日地标文件： \_\_\_\_\_  
三个最有趣符号名： \_\_\_\_\_  
一句类比： \_\_\_\_\_

当你能在白板上画出 `main → query → tool` 的箭头，你已经拥有这座城的 **旅游巴士路线**。下一站：社区如何在废墟旁搭起帐篷、架起望远镜—— [03-community.md](#)。

# 1.3 社区反应与重建：当一千双手伸向同一张地图

### 本节学习目标

- 了解 **GitHub** 上出现的代表性仓库类型：**镜像/重建**（如 `leaked-claude-code`）、**分析导读**（如 `sanbuphy`）、以及其他二次整理（如 `xorespesp` 等命名在传播中的出现）。

- 理解 PR #41391、#41518 一类数字在社区叙事中的含义（常以「向官方或相关仓库提议修复/讨论」的形式出现——具体以链接与描述为准）。
- 掌握社区重建的三类硬骨头：缺 package.json/tsconfig/build、60+ 依赖逆向、90+ stub 模块。

## 故事开头：图书馆闭馆了，但目录卡飞出来了

如果把官方闭源产品比作 闭馆图书馆，Source Map 事件就像 目录卡被风吹到广场上。于是：

- 有人 把卡片一张张捡起装订（镜像仓库）；
- 有人 写导读书告诉你哪张卡片对应哪个书架（分析仓库）；
- 有人 尝试复原借书系统（补 package.json、补脚本）。

这不是浪漫化——这是在描述 信息一旦公开扩散后的典型生态反应。

[Mermaid 图表 — 请在在线版本中查看]

## 仓库光谱：不是只有一种「社区版」

### 1. leaked-clause-code 一类：镜像/聚合

传播中常提到 leaked-clause-code 仓库获得 数百 star 量级（具体数字随时间变化）。这类仓库往往：

- 提供 可浏览 的目录树；
- 可能附带 说明 与 对比；
- 许可证与合规状态各异，读者必须自行判断。

类比：像 复印店装订册——方便传阅，但不等于出版社正式出版物。

### 2. sanbuphy 分析向仓库：导读与地图

背景信息称 sanbuphy 相关仓库可达 万级 star（以 GitHub 实时为准）。此类工作价值常在：

- 模块说明、调用关系图、阅读顺序建议；
- 降低 零基础 读者的入门成本。

类比：像 故宫导览小程序——你不一定懂建筑学，但能知道「先中轴线再东西六宫」。

3. xorespesp 等名称

不同帖子会点名不同仓库；本书不绑定单一链接。建议你以「高星 + 最近更新 + 明确许可证」三要素筛选。

[Mermaid 图表 — 请在在线版本中查看]

Star 数说明：热度 ≠ 正确性

| 信号     | 能说明什么    | 不能说明什么     |
|--------|----------|------------|
| 高 star | 话题性强、传播广 | 法律上「可自由使用」 |
| 低 star | 可能更细分或较新 | 技术差        |
| fork 多 | 二次实验活跃   | 上游维护稳定     |

生活类比：网红餐厅排队长，不代表 食品安全评级 最高。

Pull Request 叙事：#41391 与 #41518

社区讨论里常出现 PR 编号，用来指代：

- 对 相关开源周边 的修复建议；
- 对 示例项目 的文档修订；
- 或对 误发布链路 的 工具链改进提案。

本书 无法在不引用实时链接的情况下 保证每个编号对应的标题永久准确。请你在阅读时：

1. 打开 PR 链接看 标题与描述；
2. 分辨它是 官方仓库 还是 社区 fork；
3. 看 合并状态 与 讨论焦点。

[Mermaid 图表 — 请在在线版本中查看]

### 教学用对照表（占位符式）

| PR 编号（示例） | 可能主题方向（需自行点开核对）       |
|-----------|-----------------------|
| #41391    | 与发布物、忽略规则、或文档相关的修补讨论  |
| #41518    | 与工具链、map 处理、或工作流相关的跟进 |

注意：上表为 叙事脚手架；写论文或新闻报道时请替换为 可核验引用。

### 重建挑战一：没有「户口本」的源码树

缺 package.json

| 后果        | 体验               |
|-----------|------------------|
| 无依赖声明     | npm install 无从下手 |
| 无 scripts | 无法 npm run build |
| 无入口字段     | 难对齐「谁最先执行」       |

类比：你捡到一整车 乐高散件，但没有盒子封面图。

缺 tsconfig.json / 构建脚本

| 后果       | 体验        |
|----------|-----------|
| 编辑器推断弱   | 跳转与类型提示打折 |
| 无法整体 tsc | 只能「单片阅读」  |

[Mermaid 图表 — 请在在线版本中查看]

### 重建挑战二：60+ npm 依赖逆向

这意味着什么？

即使 .ts 文件齐了，import 语句仍可能指向：

- 公开包；
- 作用域私有包（@internal/... 一类示意）；
- 已改名或已废弃 包。

社区策略

| 策略          | 优点     | 缺点       |
|-------------|--------|----------|
| 用公开替代品 stub | 能编译    | 运行行为不同   |
| 最小实现 mock   | 可跑部分路径 | 工作量大     |
| 只读不做运行时     | 合规风险低  | 不能「一键启动」 |

[Mermaid 图表 — 请在在线版本中查看]

类比：菜谱写「秘制酱料」，你得用 近似调料 复刻味道。

重建挑战三：90+ stub 模块

stub 是什么？

Stub 是「仅占位」的模块：导出空对象、抛 not implemented、或返回默认值，让类型检查先过。

| stub 的善意 | stub 的恶意（对学习者的） |
|----------|-----------------|
| 让 IDE 不红 | 误以为功能真存在        |
| 标注依赖边界   | 运行时才爆炸          |

[Mermaid 图表 — 请在在线版本中查看]

生活类比：舞台剧 B 角 站台上撑场，观众以为戏完整，其实主角台词还没背。

## 社区协作模式：像开源，但不等于开源

| 维度  | 典型开源项目 | 社区重建树      |
|-----|--------|------------|
| 许可证 | 明确     | 可能混乱或缺失    |
| 上游  | 可提 PR  | 不一定有官方回应渠道 |
| 目标  | 跑起来    | 读起来 已是胜利   |

### 仓库价值象限（表格版）

[Mermaid 图表 — 请在在线版本中查看]

| 象限         | 特征   | 建议          |
|------------|------|-------------|
| 高可读 + 许可清晰 | 接近理想 | 优先 star 与跟进 |
| 高可读 + 许可模糊 | 常见   | 只学习不分发      |
| 低可读        | 价值有限 | 换分析向仓库      |
| 低可读 + 高风险  | 避免   | 不碰          |

## 与本书 V2 的衔接：我们如何使用社区材料？

1. 概念与目录 以多源交叉为主。
2. 代码片段 标注为「示意」除非逐字来自你本地文件。
3. 伦理与法律 强制读 1.5。

## 关键源码片段（示意）：stub 长什么样？

```
// stub/logger.ts (示意)
export const logger = {
  info: (..._args: unknown[]) => {},
  error: (..._args: unknown[]) => {},
};
```

看到这种文件，请在笔记本写：「此处声音被关静音」——读主流程时别指望日志帮你调试。

## 时间线：社区热度（示意）

[Mermaid 图表 — 请在在线版本中查看]

## 对比表：你在社区能获得的三种「战利品」

| 战利品     | 你得到什么 | 你得不到什么     |
|---------|-------|------------|
| 镜像树     | 文件文本  | 官方支持       |
| 分析文章    | 心智模型  | 100% 准确调用链 |
| 可跑 fork | 部分演示  | 与商业版一致的行为  |

## 下一节导航

- 1.4 为什么值得学：[04-why-learn.md](#)
- 术语：[../part00-preface/glossary.md](#)

## 附录：安全与研究伦理清单

| 问题              | 建议            |
|-----------------|---------------|
| 我能转发 tarball 吗？ | 先看清版权与条款      |
| 我能用来训练模型吗？      | 另涉数据权利，不在本书范围 |
| 我能公开完整镜像吗？      | 高风险，咨询法律顾问    |

## 附录：给教师的课堂讨论题

- 「Star 数」是不是同行评审？
- stub 会不会让 学生误以为系统已完成？
- 如果把此案例写进 软件工程课，应放在哪一章（发布工程？安全？合规？）

社区的力量在于 快速把沉默的二进制变成可讨论文本；社区的边界在于 合法与尊重原作者劳动。下一站，我们回答最朴素的问题：「学它到底图啥？」—— [04-why-learn.md](#)。

## 1.4 为什么值得学：AI Agent 的力气，多半花在「AI 之外」

### 本节学习目标

- 理解核心论点：「AI Agent 90% 的工作量在 AI 之外」——编排、工具、权限、上下文、产品化。
- 把 4756 个模块文件（或等价统计口径）还原为 可感知的工程现实：不是炫技，是分工。
- 建立四大学习价值框架：架构、工程化、安全设计、性能优化。

### 一个反直觉的真相：魔法在台上，戏班在台下

如果把 大语言模型 想象成 台上一分钟的天才魔术师，那么 Claude Code 这类产品就是 整个马戏团：

- 灯光音响（终端 UI / 编辑器集成）
- 道具机关（Tool 与 MCP）
- 保安与检票（Permission Mode）
- 剧本与提词（系统提示、[CLAUDE.md](#)、Compaction）
- 巡演后勤（配置、遥测、发布）

观众鼓掌给魔术师，但 巡演能不能跑下来，取决于台下这帮人。

[Mermaid 图表 — 请在在线版本中查看]

本节论点：读这 51 万行，是在读 马戏团运营手册，不是在读「魔术揭秘」。



## 「90% 在 AI 之外」到底指什么？

### 拆解表

| 工作块 | 「在 AI 之外」的部分            | 生活类比       |
|-----|-------------------------|------------|
| 输入  | 读取本地文件、git 状态、linter 输出 | 秘书先帮你整理桌面  |
| 行动  | 调 CLI、改文件、跑测试           | 手替你去拧螺丝    |
| 约束  | 哪些行动要确认、哪些默认拒绝          | 家长给未成年人的门禁 |
| 记忆  | 会话窗口、摘要、项目级长期说明         | 旅行箱与旅行日记分工 |
| 失败  | 重试、超时、降级、错误展示           | 外卖小哥改路线    |

[Mermaid 图表 — 请在在线版本中查看]

饼图比例为 **教学修辞**；真实项目应靠度量（日志、耗时剖析）验证。

## 4756 个「模块文件」意味着什么？

### 不是 4756 个「聪明大脑」

初学者容易把「模块」想成 **4756 个 AI**。实际上，它们多数是：

| 类型    | 可能的职责         | 类比      |
|-------|---------------|---------|
| 命令    | 解析 argv、子命令分发 | 遥控器按键固件 |
| 工具实现  | 单一能力封装        | 瑞士军刀每一格 |
| 服务    | 会话、配置、网络      | 物业办公室   |
| 类型与协议 | DTO、事件、接口     | 合同模板    |
| UI 片段 | 渲染、表格、进度      | 仪表盘刻度   |

[Mermaid 图表 — 请在在线版本中查看]

## 与 1903 物理文件的关系

见 1.2：统计口径不同会导致数字不同。你要记住的是 **数量级**——这是一个 **需要工具化阅读** 的仓库，而不是「周末随手翻完」的博客。

## 学习价值一：理解 AI Agent 架构（心智模型的底盘）

### 你将能回答的问题

| 问题        | 读源码前   | 读源码后（目标）                      |
|-----------|--------|-------------------------------|
| 工具调用谁来做？  | 「模型吧？」 | <b>编排层</b> 解析、校验、执行、回收结果      |
| 权限怎么做细？   | 「弹窗？」  | <b>策略模式</b> 、默认、白名单、会话级覆盖     |
| 上下文太长怎么办？ | 「截断？」  | <b>Compaction</b> 、摘要策略、结构化折叠 |
| 外接数据库怎么接？ | 「写脚本？」 | <b>MCP</b> 、Bridge、生命周期       |

[Mermaid 图表 — 请在在线版本中查看]

类比：学开车不只是学「引擎爆炸原理」，还要学 **离合器、刹车、路权**——Agent 产品同理。

## 学习价值二：工程化实践（能抄作业的那种）

### 典型工程题在源码里都有「标本」

| 工程题       | 可能在源码里的痕迹                        |
|-----------|----------------------------------|
| 大仓库如何拆模块？ | 目录边界、 <code>index.ts</code> 导出策略 |
| 如何避免循环依赖？ | 实际也可能存在技术债——对比「理想 vs 现实」         |
| 如何做配置分层？  | 用户目录 / 项目目录 / 环境变量               |
| 如何做错误边界？  | tool 执行 try/catch、用户可见错误码        |

```
// 教学示意：错误边界常见写法
async function runToolSafely(run: () => Promise<unknown>) {
  try {
    return { ok: true as const, value: await run() };
  } catch (e) {
    return { ok: false as const, error: normalizeError(e) };
  }
}
```

类比：像看 米其林后厨的分工表——你未必开餐厅，但你能学会「什么叫专业」。

## 学习价值三：安全设计（默认不信任）

Agent = 高权限自动化 = 高风险面

| 风险    | 设计回应（概念层）         |
|-------|-------------------|
| 误删文件  | 权限确认、沙箱、回收策略      |
| 泄露密钥  | 忽略规则、敏感扫描、脱敏日志    |
| 供应链投毒 | 依赖锁定、完整性校验（视实现而定） |
| 诱导攻击  | 系统提示、工具白名单、人机确认   |

[Mermaid 图表 — 请在在线版本中查看]

类比：自动驾驶不仅要会开，还要 知道什么时候必须让人类握方向盘。

## 学习价值四：性能优化（体感来自毫秒堆叠）

| 体感问题   | 可能优化面                |
|--------|----------------------|
| 启动慢    | 懒加载、分包、减少同步 IO       |
| 打字卡    | UI 与重任务解耦、Worker（若有） |
| 大仓库搜索慢 | 索引、忽略规则、限流           |

[Mermaid 图表 — 请在在线版本中查看]

类比：像 地铁高峰调度——单列车快不够，要全线协同。

## 和「只学 Prompt」的路线对比

| 路线           | 优点        | 盲区      |
|--------------|-----------|---------|
| 只学 Prompt    | 上手快       | 很难做可靠产品 |
| 学 Agent 编排源码 | 理解边界与失败模式 | 门槛高、耗时  |
| 两者结合         | 理论与实践互补   | 需要自律路线图 |

[Mermaid 图表 — 请在在线版本中查看]

## 谁可能「不值得」按本书深度学？

诚实地列一下：

| 情况           | 建议                |
|--------------|-------------------|
| 只想用产品完成任务    | 官方文档优先            |
| 完全不想碰代码      | 读本书 Part 00~01 即可 |
| 期待读完就能复制商业产品 | 法律与工程都不现实         |

## 关键源码片段（示意）：编排层「包住」模型

```
// 示意：客户端编排常见结构
async function handleUserTurn(userText: string) {
  const state = await loadConversationState();
  const policy = await resolvePermissionMode(state);
  const prompt = buildSystemPrompt({ state, policy });
  const modelOut = await callModel({ prompt, userText });
  const actions = parseToolCalls(modelOut);
  return executeWithPolicy(actions, policy);
}
```

看到 `resolvePermissionMode` 与 `executeWithPolicy` 了吗？那就是 90% 戏班的入口。

## 与职业路径的映射

| 角色          | 读本书的收益           |
|-------------|------------------|
| 前端/Node 工程师 | 提升对大型 TS 仓库的阅读力  |
| 平台工程师       | 借案例理解工具化自动化边界    |
| 安全工程师       | 借案例建立 Agent 威胁模型 |
| 学生          | 把「AI 课」落到软件工程地面  |

## 下一节导航

- 1.5 法律与伦理：[05-legal-ethics.md](#)
- 学习路线图：[../part00-preface/roadmap.md](#)

## 附录：21 天「价值强化」日记模板

| 天 | 今日问题 | 在源码中找的锚点 | 一句收获 |
|---|------|----------|------|
| 1 |      |          |      |
| 2 |      |          |      |

## 附录：反驳常见偏见

| 偏见            | 回应         |
|---------------|------------|
| 「不就是个壳？」      | 壳决定 可靠性与边界 |
| 「模型换代就白学？」    | 编排模式 跨模型复用 |
| 「我读不完 51 万行？」 | 本书教你 用地图读城 |

当你能平静地说出「模型只是组件之一」，你就从 AI 观众席 坐到了 工程排练厅。最后一站：法律与伦理——[05-legal-ethics.md](#)。

## 1.5 法律与伦理：我们读的是「公开可得文本」，但不是「任意使用许可证」

### 本节学习目标

- 区分 **Source Map 误发布** 与 **黑客入侵** 在叙事与归责上的差异（不等同于法律结论）。
- 明确本书立场：**教育目的** 的架构学习；**不鼓励** 侵权分发与恶意利用。
- 了解 **Anthropic** 可能的回应方向（以官方声明为准）与 **开源社区常见讨论轴线**。

### 第一句话：先把情绪分类

当新闻标题写「源码泄露」时，读者脑子里可能同时出现两种电影：

1. 间谍片：有人潜入服务器偷走核心机密。
2. 生活片：印刷厂误把内部流程单订进杂志里卖给读者。

**Source Map 进入 npm 包** 更接近哪一种？在多数公开叙述里，它更像 **流程失误导致的敏感产物外流**，而不是典型的 **入侵窃取**。

但这句判断是 **大众传播层面的分类**，不是 **法院判决**。

[Mermaid 图表 — 请在在线版本中查看]

**生活类比**：同样是「你的东西被别人拿到了」，**被偷** 与 **你自己掉在路上** 的处理路径不同。

## 概念对照表：泄露 vs 黑客攻击（教学用）

| 维度         | Source Map 误发布（常见描述） | 黑客攻击（典型描述）  |
|------------|----------------------|-------------|
| 起点         | 构建/发布链路              | 未授权访问       |
| 主观意图（讨论焦点） | 过失/流程缺陷              | 恶意/牟利/破坏    |
| 技术入口       | 公开 npm 下载            | 漏洞利用、凭证窃取等  |
| 归责讨论       | 供应链、CI/CD、清单审计       | 取证、追责、应急响应  |
| 用户体感       | 「更新包变大」              | 「服务宕机/数据被改」 |

[Mermaid 图表 — 请在在线版本中查看]

**重要：**现实案件可能 **混合多种因素**；本书表格是 **学习脚手架**。

## 教育目的使用：本书允许的「读法」

### 我们鼓励的

| 行为                     | 说明             |
|------------------------|----------------|
| 个人本地阅读 社区重建或你自己合法获得的文本 | 建立架构认知         |
| 画流程图、写笔记、课堂讨论          | 引用时注明来源与不确定性   |
| 把教训写进内部培训：发布工程、合规清单    | 脱敏、不要夹带未授权完整源码 |

### 我们不鼓励的

| 行为                   | 原因           |
|----------------------|--------------|
| 未经授权 大规模再分发完整源码树     | 版权与许可风险      |
| 用于 绕过产品条款 或 克隆商业产品   | 合同与不正当竞争风险   |
| 将材料用于 训练模型 并声称「合法默认」 | 数据权利复杂，需单独评估 |

[Mermaid 图表 — 请在在线版本中查看]

**类比：**大学解剖课允许你 **学习人体结构**，不允许你 **把标本挂网上卖**。

## Anthropic 的回应：应如何阅读新闻？

企业回应可能包含（以官方为准）：

- **承认/调查** 发布物异常；
- **撤回/替换** 某些版本包；
- **加强** 发布流程与监控；
- **声明** 影响范围（例如强调非模型权重）。

本书无法提供 **实时声明全文**；请你训练一个习惯：

1. 找 **官方博客 / 安全公告 / npm 版本变更** 一手材料；
2. 区分 **事实陈述** 与 **公关措辞**；
3. 对「完全无害」类表述保持 **技术性追问**（地图是否可还原？是否含注释与路径信息？）。

[Mermaid 图表 — 请在在线版本中查看]

## 开源社区的讨论轴线：大家在吵什么？

轴线 A：「这算不算开源？」

| 观点  | 要点                  |
|-----|---------------------|
| 不算  | 无明确 OSI 许可证意图；属泄露文本 |
| 像开源 | 可阅读、可讨论、社区重建活跃      |

本书立场：法律状态 ≠ 社区行为。你可以 **像读开源一样读**，但别 **像用开源一样默认授权**。

轴线 B：「研究价值 vs 道德风险」

[Mermaid 图表 — 请在在线版本中查看]

|       | 低伤害       | 高伤害       |
|-------|-----------|-----------|
| 高研究价值 | 架构教学、合规改进 | 需强约束与伦理审查 |
| 低研究价值 | 八卦传播      | 恶意利用      |



轴线 C：「对竞品与生态的影响」

- 正向：推动行业 发布工程 成熟；
- 负向：刺激 恶意 fork 与 钓鱼工具链。

研究者的位置：Chaofan Shou 叙事中的角色

公开描述中，研究者承担的是 观察者与通报者 功能：

| 常见伦理要求     | 说明           |
|------------|--------------|
| 负责任披露      | 给厂商修复窗口（若适用） |
| 避免利用漏洞造成破坏 | 区分验证与攻击      |
| 证据链清晰      | 可复核、可重复      |

类比：记者报道「超市货架上有异物」，不是鼓励顾客哄抢，而是推动 召回与整改。

开发者个人合规清单（建议打印）

| 步骤 | 自检                 |
|----|--------------------|
| 1  | 我是否只在自己机器阅读？       |
| 2  | 我是否避免把完整树上传公有云？    |
| 3  | 我是否在公司政策允许范围内？     |
| 4  | 我是否区分「学习笔记」与「再发布」？ |
| 5  | 我是否在文章里避免煽动性用语？    |

[Mermaid 图表 — 请在在线版本中查看]

## 与学生、教师的特别说明

| 角色 | 建议                         |
|----|----------------------------|
| 学生 | 把案例当 软件工程与安全课 教材，不要当「破解教程」 |
| 教师 | 明确 作业边界：分析架构可以，传播未授权完整包不可以 |

## 关键「法律词汇」小白解释

| 词    | 超短解释                |
|------|---------------------|
| 版权   | 对表达形式的保护            |
| 许可   | 版权人允许你怎么用           |
| 合同   | 使用产品时同意的条款          |
| 商业秘密 | 非公知、能带来价值、采取保密措施的信息 |

本书 **不是法律意见**。涉诉或商业决策请咨询执业律师。

## 与 1.1~1.4 的闭环

[Mermaid 图表 — 请在在线版本中查看]

- 1.1 告诉你发生了什么；
- 1.2 告诉你规模；
- 1.3 告诉你社区怎么做；
- 1.4 告诉你为什么值得学；
- 1.5 告诉你 学的时候别踩线。

## 源码片段（示意）：发布清单审计思路

```
# 教学示意：在发布前检查 tarball 是否含 map
npm pack --dry-run 2>/dev/null | rg "\\map$" || true
```

这不是「攻击技巧」，这是 每个发布工程师都该会的自检。

## 下一程

完成 Part 01 后，请回到路线图选择路径：

- `../part00-preface/roadmap.md`
- 进入 Part 02（全书规划）时，请携带 两张图：事件时间线 + `main-query→tool` 调用草图。

## 附录：常见问答（FAQ）

- 问：我读社区仓库会违法吗？

答：取决于 你所在法域、仓库许可、你做什么。个人本地学习通常讨论较多，但别替代律师意见。
- 问：我能写「完整还原指南」吗？

答：写作本身可能涉 教唆分发 风险；更稳妥是写 架构级分析 与 发布工程教训。
- 问：官方会不会追究读者？

答：一般公共讨论聚焦 责任方与流程；但 大规模传播 另当别论。

## 附录：情绪日记（自我调节用）

| 我读新闻时的情绪 | 更建设性的下一步                        |
|----------|---------------------------------|
| 兴奋       | 打开 <code>setup.md</code> 配环境    |
| 愤怒       | 写「发布 checklist」给团队              |
| 恐惧       | 只读公开合法材料                        |
| 冷漠       | 先读 <code>04-why-learn.md</code> |

## 结语：学习的尊严来自边界

真正专业的学习者，会把 好奇心 变成 可验证的理解，把 热点 变成 长期能力。当我们尊重法律与伦理边界时，知识才会 站得住脚。

恭喜你读完 《Claude Code 完全指南 V2》第 1 篇。下一篇，我们将从 入口文件 真正走进这座城——记得带上地图。



## 第2篇：小白快速上手

---

## 第2篇：小白快速上手 · 总览

---

**本篇定位：**零基础读者从「听说过 AI 写代码」到「能在终端里用 Claude Code 干活」的入门路径。

**前置知识：**会用电脑、能打开终端（命令行）、愿意复制粘贴命令。

### 本篇学习目标

---

完成第2篇后，你应该能够：

1. **说清楚：**AI 编程助手分哪三档，Claude Code 属于哪一档、和 Cursor 等工具有何不同。
2. **装得上：**在 macOS / Linux / Windows 上安装 Claude Code，并配置 API 密钥。
3. **聊得动：**启动 `claude`，完成第一次对话，看懂终端里的输入输出。
4. **改得了：**让代理读取、创建、修改文件，并理解 `y/n` 权限提示在保护什么。
5. **跑得通：**批准或拒绝 Bash 命令，知道测试、`npm`、`git` 等常见用法。
6. **管得好 Git：**在安全前提下让代理协助提交、分支与 PR，避开危险操作。
7. **查得快：**会用斜杠命令与速查表，并了解费用与省钱习惯。

### 2.1 什么是 AI 编程助手？

---

#### 一句话定义

AI 编程助手是用大语言模型帮你理解代码、改代码、跑命令的一类工具。差别在于：它只能「给建议」，还是能「动手改文件、跑终端」。

#### 生活类比：装修房子的三种方式

想象你要装修一套房子：

| 方式              | 像什么                                                   | 你能得到什么                      |
|-----------------|-------------------------------------------------------|-----------------------------|
| 代码补全            | 师傅站在旁边，你砌墙时他递砖、递水泥                                    | 你仍是一砖一瓦自己干，只是少打字、少查文档       |
| 对话助手            | 你打电话问设计师：「客厅怎么配色？」他口述方案                               | 方案很详细，但锤子还在你手里，你得自己买材料、自己施工 |
| 自主代理<br>(Agent) | 你雇一个工长：你说「把厨房改成开放式」，他拆墙、走水电、对接橱柜—— <b>每步关键动作会请你签字</b> | 你管目标和验收， <b>执行链条由代理串联</b>   |

Claude Code 就是第三种里的「终端工长」：在命令行里运行，能读项目、改文件、跑 `npm` / `git` / 测试，并通过权限提示让你把关。

[Mermaid 图表 — 请在在线版本中查看]

### 三等级技术对比（从「补全」到「代理」）

| 维度         | 代码补全（如 Copilot 行内） | 对话助手（如 ChatGPT 网页） | 自主代理（如 Claude Code）    |
|------------|--------------------|--------------------|------------------------|
| 主要界面       | 编辑器内联              | 聊天窗口               | 终端 + 文本界面（如 React Ink） |
| 是否改你磁盘上的文件 | 通常只插入光标处           | 不直接改，除非你复制粘贴       | 可直接编辑工作区文件             |
| 是否跑终端命令    | 一般不跑               | 一般不跑（或需插件）         | 可提议并执行（需你批准）           |
| 上下文        | 当前文件/邻近文件          | 你粘贴的内容 + 对话        | 整仓搜索、多文件、工具链           |
| 适合场景       | 快写函数、补样板           | 问概念、要小段代码          | 多步骤需求：重构、修 bug、跑测试     |

[Mermaid 图表 — 请在在线版本中查看]

### Claude Code 的核心画像（与本篇后文呼应）

- 形态：终端里的 AI 编程代理，不是浏览器插件为主。
- 四类能力：编辑文件、运行命令、搜索代码、协调工作流（多步任务拆成读→改→测→提交）。

- **技术栈（实现侧）**：以 **Bun** 为运行时、**TypeScript** 编写、**React Ink** 做终端 UI（你看到的是字符界面，背后是成熟组件模型）。
- **工具与权限**：内置约 **42 个工具**、多种**权限模式**（后文安装与实操会反复遇到「是否允许读/写/执行」）。
- **计费**：走 **Anthropic API**，按 **Token** 计费；具体区间见 `08-cost-tips.md`。

### Claude Code 与 Cursor / Copilot Agent / Windsurf 对比

说明：各产品迭代快，下表侧重**入门读者该理解的差异维度**，而非某一版本的精确功能清单。

| 对比项        | Claude Code       | Cursor            | GitHub Copilot（含 Agent 向能力） | Windsurf        |
|------------|-------------------|-------------------|-----------------------------|-----------------|
| 主战场        | 终端                | 编辑器深度集成           | 编辑器 + GitHub 生态             | 编辑器 + 流程化 Agent |
| 和你日常写代码的位置 | 另开一个终端会话，与 IDE 并行 | 几乎「长在 VS Code 系里」 | 在支持的 IDE 里                  | 在支持的 IDE 里      |
| 改文件方式      | 工具调用写入工作区         | 多光标/Composer 等    | 依模式与版本而定                    | 流式编辑与任务流        |
| 跑命令        | 强项：Bash 工具链 + 审批  | 有终端集成，体验因版本而异     | 依环境与扩展而定                    | 常与工作流绑定         |
| 适合谁        | 习惯终端、希望代理完整操作仓库的人 | 希望少切窗口、编辑器即中枢的人   | 已在 GitHub/Microsoft 栈里的人    | 喜欢「任务流」编辑器体验的人  |

#### 选型口诀：

- 你已经 **IDE 不离手**：Cursor / Windsurf 可能更「顺手」。
- 你希望 **SSH 到服务器、纯终端环境、CI 旁路调试**：Claude Code 很合拍。
- 你公司 **强制 GitHub Copilot**：可把它当「补全 + 部分代理」，与 Claude Code 是否并存取决于团队政策与预算。

### 本节小结

- AI 编程助手不是单一物种：从**补全**、**对话**到**代理**，自主程度递增。

- Claude Code = 终端里的自主代理，强项是文件 + 命令 + 搜索 + 多步工作流，用权限 gate 保证你不失控。
- 与 Cursor 等相比：战场在终端，适合愿意把「工长」放在 shell 里的开发者。

## 本篇章节导航

| 章节  | 文件                                 | 内容提要               |
|-----|------------------------------------|--------------------|
| 2.1 | 本文                                 | AI 助手三等级、产品对比、装修类比 |
| 2.2 | <a href="#">02-installation.md</a> | 安装与 API 密钥         |
| 2.3 | <a href="#">03-first-chat.md</a>   | 第一次对话              |
| 2.4 | <a href="#">04-edit-files.md</a>   | 编辑文件与权限            |
| 2.5 | <a href="#">05-run-commands.md</a> | 运行命令与审批            |
| 2.6 | <a href="#">06-git-workflow.md</a> | Git 与安全边界          |
| 2.7 | <a href="#">07-cheatsheet.md</a>   | 斜杠命令与快捷键           |
| 2.8 | <a href="#">08-cost-tips.md</a>    | Token 计费与省钱        |

## 延伸阅读（概念备忘）

# 你在后文会反复看到的词

Token        – API 计费的文本单位（可粗理解为「词块」）

工具 Tool    – 模型调用的能力：读文件、写文件、执行 bash 等

权限 Permission – 是否允许本次读/写/执行；常表现为 y/n

工作区 Workspace – 你 cd 进去的那个项目根目录

## 常见误解（入门必读）

| 误解                      | 实际情况                          |
|-------------------------|-------------------------------|
| 「装了 Claude Code 就不用学编程」 | 代理加速执行，但架构、调试、Code Review仍依赖你 |
| 「代理永远懂我的项目」             | 它依赖读到的文件与工具结果；交代不清就会猜         |
| 「按 y 只是走流程」             | y 是法律意义上的授权；恶意或错误命令同样可能执行     |
| 「终端里的一定比 IDE 里安全」       | 安全取决于权限与审查，与界面形态无必然关系         |



## 学习路径建议（7 天极简版）

| 天 | 建议动作                                                   |
|---|--------------------------------------------------------|
| 1 | 读完 2.1–2.2，完成安装与 <code>claude</code> 启动                |
| 2 | 完成 2.3，纯问答 + 读文件各 3 次                                  |
| 3 | 完成 2.4，创建与修改各 2 个文件                                    |
| 4 | 完成 2.5， <code>npm test</code> 或等价命令闭环                  |
| 5 | 完成 2.6，一次完整 <code>commit</code> （不 push）               |
| 6 | 背 <code>/help</code> 与速查表，整理自己的 <code>CLAUDE.md</code> |
| 7 | 读 2.8，给自己定「模型 + 压缩」规则                                  |

[Mermaid 图表 — 请在在线版本中查看]

## 术语迷你表

| 英文         | 中文习惯说法         |
|------------|----------------|
| Agent      | 代理 / 智能体       |
| Tool       | 工具（读文件、执行命令等）  |
| Permission | 权限 / 批准        |
| Workspace  | 工作区 / 项目根目录    |
| Context    | 上下文（当前对话+附带内容） |
| Token      | 计费与窗口单位（词块）    |

## 把「Claude Code」放进你的日常工位

你可以把日常开发想象成**双屏**：左屏 IDE 写代码，右屏终端跑 `claude` 做「工长」。  
它不是要替代 IDE，而是把 **多文件、多命令、多步骤** 的脏活累活串成流水线——**你仍然在掌舵**。

| 工位元素                     | 角色                  |
|--------------------------|---------------------|
| IDE                      | 细粒度编辑、调试器、可视化 Git   |
| 终端 + Claude Code         | 批量读写、跑脚本、搜全仓、生成提交说明 |
| 官方文档与 <code>/help</code> | 纠偏与版本新特性            |

下一章：[2.2 安装全流程](#) →

## 2.2 安装全流程

**本节目标：**在 macOS、Linux 或 Windows 上装好 Claude Code，配置 Anthropic API 密钥，并用 `claude` 命令完成首次验证。

**预计时间：**首次约 15–30 分钟（含下载 Node 与网络等待）。

### 学习目标

- 能解释：为什么需要先装 **Node.js**（用于全局 `npm` 包）。
- 能独立完成：`npm install -g @anthropic-ai/claude-code`。
- 能在 [Anthropic Console](#) 创建 **API Key** 并妥善保存。
- 能在项目目录执行 `claude`，看到正常启动而非报错。

### 安装前的直觉：Claude Code 住在哪里？

把 Claude Code 想成「装在系统里的一个全局命令」：

[Mermaid 图表 — 请在在线版本中查看]

- **Node.js**：提供运行 JavaScript 工具链的环境（Claude Code 以 `npm` 包分发）。
- `npm install -g`：把 CLI 装到「全局」，任意目录都能敲 `claude`。
- **API Key**：证明「这是你付费/配额内的调用」，别泄露、别提交到 Git。

# 第一步：安装 Node.js

## 为什么要装 Node?

Claude Code 通过 npm 发布。没有 Node，就没有稳定的 npm 来装全局 CLI。  
(实现上 Claude Code 自身技术栈含 Bun/TS 等，但你作为用户当前主流安装路径仍是 Node + npm。)

## 版本建议

- 使用 当前的 LTS（长期支持）版 Node 通常最省心。
- 安装完成后在终端验证：

```
node -v
npm -v
```

应看到类似：

```
v22.x.x
10.x.x
```

## macOS 安装方式（任选其一）

| 方式       | 适合人群    | 命令或操作                                                             |
|----------|---------|-------------------------------------------------------------------|
| 官网安装包    | 不想折腾    | <a href="https://nodejs.org/">https://nodejs.org/</a> 下载 LTS .pkg |
| Homebrew | 已用 brew | brew install node                                                 |
| nvm      | 需要多版本切换 | 按 nvm 文档安装后再 nvm install --lts                                    |

## Linux 安装方式（示例）

Debian / Ubuntu（示意，版本号以官方为准）：

```
# 示例：使用 NodeSource 或发行版仓库，任选团队认可的方式
curl -fsSL https://deb.nodesource.com/setup_lts.x | sudo -E bash -
sudo apt-get install -y nodejs
```

Fedora / RHEL 系：

```
sudo dnf install nodejs npm
```

验证同样使用 `node -v` 与 `npm -v` 。

### Windows 安装方式

| 方式     | 说明                                                                   |
|--------|----------------------------------------------------------------------|
| 官网 MSI | 从 <a href="https://nodejs.org">nodejs.org</a> 安装 LTS，勾选「Add to PATH」 |
| winget | <code>winget install OpenJS.NodeJS.LTS</code> （以本机 winget 源为准）       |
| WSL2   | <b>推荐：</b> 在 WSL 的 Ubuntu 里按 Linux 步骤安装，终端体验更接近文档示例                  |

[Mermaid 图表 — 请在在线版本中查看]

**小白提示：**若在 Windows 原生 PowerShell 里路径、换行与 macOS 不同，优先查官方文档或团队规范；很多开发者最终会选择 **WSL2 + Ubuntu** 统一环境。

## 第二步：全局安装 Claude Code

在任意终端执行：

```
npm install -g @anthropic-ai/claude-code
```

### 常见现象

| 现象                                     | 可能原因           | 处理方向                                                                       |
|----------------------------------------|----------------|----------------------------------------------------------------------------|
| <code>EACCES</code> 权限错误               | 全局目录无写权限       | macOS/Linux：配置 npm 前缀到用户目录，或用 <code>nvm</code> ；避免长期 <code>sudo npm</code> |
| 下载很慢                                   | 网络或 registry   | 换网络、配置镜像（遵守公司合规）                                                           |
| <code>command not found: claude</code> | PATH 未包含全局 bin | 重新打开终端；检查 <code>npm bin -g</code> 输出是否在 PATH                               |

安装完成后验证：

```
claude --version
# 或
claude -h
```

若能输出版本或帮助信息，说明 CLI 已就位。

## 第三步：获取 Anthropic API 密钥

1. 浏览器打开：<https://console.anthropic.com/>
2. 登录或注册账号。
3. 进入 API Keys（名称可能随控制台改版略有不同），**Create Key**。
4. **复制密钥**，粘贴到安全位置（密码管理器）。
5. **不要把密钥写进公开仓库、截图发群、或贴在 issue 里。**

### 配置到环境变量（推荐）

macOS / Linux（当前终端会话临时生效）：

```
export ANTHROPIC_API_KEY="sk-ant-api03-....."
```

写入 shell 配置文件（长期生效，示例为 zsh）：

```
echo 'export ANTHROPIC_API_KEY="sk-ant-api03-....."' >> ~/.zshrc
source ~/.zshrc
```

Windows（PowerShell 当前会话）：

```
$env:ANTHROPIC_API_KEY = "sk-ant-api03-....."
```

具体密钥前缀以控制台显示为准；若团队使用 **配置文件或 SSO**，以内部文档为准。

[Mermaid 图表 — 请在在线版本中查看]

## 第四步：首次运行 **claude** 验证

1. 进入一个测试用目录（避免误改重要项目）：

```
mkdir -p ~/sandbox-cc-test
cd ~/sandbox-cc-test
git init # 可选：许多 workflows 假设在仓库内
```

2. 启动：

```
claude
```

3. 期望：出现交互界面或提示，能够输入自然语言；若报错，记录完整报错英文/中文。

故障排查速查

| 报错线索                     | 检查项                                                         |
|--------------------------|-------------------------------------------------------------|
| API key / authentication | <code>echo \$ANTHROPIC_API_KEY</code> 是否为空；Windows 检查对应 env |
| network / timeout        | 代理、防火墙、公司 MITM 证书                                           |
| rate limit               | 配额、账单、组织策略                                                  |

与「42 工具 / 权限模式」的关系（预告）

安装本身不展示全部工具。进入交互后，当代理需要 读文件、改文件、执行 `bash` 时，你会看到权限提示——这正是多工具 + 权限模式的外在表现。详情见 `04-edit-files.md` 与 `05-run-commands.md`。

本节命令清单（可复制）

```
# 1) 检查 Node
node -v && npm -v

# 2) 全局安装
npm install -g @anthropic-ai/claude-code

# 3) 验证 CLI
claude --version

# 4) 设置密钥（示例）
export ANTHROPIC_API_KEY="你的密钥"

# 5) 在空目录试跑
mkdir -p ~/sandbox-cc-test && cd ~/sandbox-cc-test
claude
```

## 小结

- **Node.js + npm** 是官方分发路径上的标准入口；装好后用 **全局 npm 包** 得到 `claude` 命令。
- **API Key** 来自 Anthropic 控制台，通过环境变量注入最通用。
- **首次验证** = 在测试目录运行 `claude`，确认能连上 API 并进入交互。

上一章: [2.1 总览](#) ← · 下一章: [2.3 第一次对话](#) →

## 2.3 第一次对话

**本节目标：**启动 `claude`，完成一轮最简单的问答，看懂终端里「你说了什么 / 模型回了什么 / 中间有没有工具」，并能用一张图描述「思考—行动—回答」的节奏。

### 学习目标

- 能在空项目或练习目录中**稳定启动** Claude Code。
- 能区分：**用户输入**、**助手可见回复**、**工具调用块**（若界面展示）。
- 能用生活类比理解：代理不是「瞬间魔法」，而是**多步推理 + 可选动作**。

### 启动：像打开一个「专注聊天的终端窗口」

在已配置 `ANTHROPIC_API_KEY` 的前提下，进入目录并执行：

```
cd ~/sandbox-cc-test    # 或你的真实项目根
claude
```

**生活类比：**你不是打开微信聊天框，而是打开一个只谈代码与命令的窗口——它能看到当前文件夹里的文件（在得到读权限后），但**不会自动乱改**，除非你让它做并批准相关权限。

[Mermaid 图表 — 请在在线版本中查看]

## 第一次说什么？三个由浅入深的示例

---

### 示例 A：纯问答（通常不涉及改文件）

在提示符处输入（中文即可，以下为示意）：

```
用三句话解释什么是 Git 里的 commit，完全不懂的人也能听懂。
```

你可能看到：

- 一段结构化解释（分段、列表）。
- 没有文件变更、没有执行 `git` 命令——因为任务只需语言回答。

### 示例 B：与当前目录相关（可能触发读目录/读文件）

先放一个文件：

```
echo "hello" > note.txt
```

再问：

```
当前目录下有哪些文件？note.txt 里写了什么？
```

此时代理往往会走 **列出目录 / 读取文件** 的工具链（具体界面以你版本为准）。  
你会遇到 **权限确认**（例如是否允许读取）——**先按提示选 y/n**，这是安全设计。

### 示例 C：明确要求「只解释、不要改」

```
接下来只回答我的问题，不要修改任何文件、不要运行命令。
```

这有助于你**观察纯对话形态**，建立信心；后续再放开让代理动手。

---

## 理解输入与输出格式

---

### 你的输入（User Message）

- 自然语言，中英均可。
- 可以很长：贴日志、贴报错、描述需求。
- 建议习惯：一次一个目标，例如「先解释报错，再给出修复步骤」。



## 模型的输出 (Assistant)

常见包含几类信息 (不同 UI 皮肤下排版不同):

| 块类型    | 含义          | 小白怎么读            |
|--------|-------------|------------------|
| 说明文字   | 推理与计划       | 先看这段, 确认它理解对你的任务 |
| 工具调用提议 | 将执行读/写/bash | 注意是否要你按 y 批准     |
| 工具结果摘要 | 命令输出、文件片段   | 核对是否符合你预期        |
| 最终总结   | 可执行结论       | 保存为笔记或照做         |

## 伪代码: 一次对话在系统里大致长什么样

下面不是真实 API 载荷, 只是帮助建立心智模型:

```
[System] 你是终端编程代理, 工作目录 /Users/you/project ...
[User]   用三句话解释 commit ...
[Assistant] commit 就像给项目拍一张「可回退的快照」...
```

当需要工具时, 可想象为额外插入:

```
[Assistant → Tool] Read(path="note.txt")
[Tool → Assistant] hello
[Assistant] 文件里只有一行 hello ...
```

[Mermaid 图表 — 请在在线版本中查看]

## 「思考过程」怎么可视化?

Claude Code 的具体 UI 会随版本调整, 但你可以用**固定套路**观察「思考感」:

1. **计划段**: 先列 1—3 步 (例如「先读 package.json」「再跑测试」)。
2. **行动段**: 出现工具名或命令预览。
3. **结果段**: 文件片段、终端输出截断展示。
4. **收尾段**: 总结 + 下一步建议。

**生活类比**: 像医生问诊——先问病史 (读文件), 再开检查 (跑命令), 最后给处方 (改代码); 你不会希望医生不做检查就乱开刀。

## 第一次对话的推荐检查清单

- [ ] `ANTHROPIC_API_KEY` 已设置，`claude` 能启动。
- [ ] 完成至少 1 次纯问答。
- [ ] 完成至少 1 次涉及读文件的问答，并看清权限提示。
- [ ] 知道如何 退出（常见为 `exit`、Ctrl+C 或界面提示；以当前版本帮助为准）。
- [ ] 知道 `/help` 可查看内置命令（详见 `07-cheatsheet.md`）。

## 常见新手问题

| 问题           | 简短回答                         |
|--------------|------------------------------|
| 我说中文可以吗？     | 可以；模型对中文支持良好。                |
| 它会自动把我电脑删光吗？ | 危险操作通常需你批准；仍建议用测试目录练习。       |
| 为什么有时很慢？     | 模型推理 + 网络 + 工具执行耗时；复杂任务多步更久。 |
| 报错看不懂怎么办？    | 把完整报错复制进对话，并说明「我刚执行了什么命令」。   |

## 与后文衔接

- 要改文件了 → 阅读 [2.4 编辑文件实操](#)，重点 `FileRead` → `FileEdit` 与 `y/n`。
- 要跑 `npm test` 了 → 阅读 [2.5 运行命令实操](#)。

## 小结

- 第一次对话 = 启动 `claude` + 从纯问答过渡到「读当前目录」。
- 终端里的回复往往不是「一句话」，而是 计划 → 工具 → 结果 → 总结 的链条。
- 权限提示是安全阀：先读提示再按 `y`，养成肌肉记忆。

## 对话截图替代：自建「对话记录」习惯

终端内容不便截图时，可在练习笔记本里固定记录四列：

| 时间 | 我输入的要点 | 代理做了什么（读/写/命令） | 我的收获或失误 |
|----|--------|----------------|---------|
|    |        |                |         |

坚持 5 次以后，你会明显更清楚钱和时间花在哪。

## 与「React Ink 终端 UI」相关的体验提示

Claude Code 的界面用 **React Ink** 等技术绘制，可能出现：

- **重绘**：输出更新时整屏滚动方式与纯 shell 不同。
- **热键冲突**：某些终端默认快捷键与 TUI 抢焦点。

若遇到显示错乱，优先尝试：**放大终端窗口、换一款终端应用、更新 Claude Code 版本。**

## 示例对话脚本（可复制微调）

### 脚本 1：自我介绍式

我是编程新手。请用不超过 200 字说明：你会如何帮我学这个仓库？  
先不要读文件，只基于一般经验回答。

### 脚本 2：约束式

接下来请用「步骤 1/2/3」回答。若需要工具，请先说明将使用哪类工具以及为什么。

### 脚本 3：复盘式

上一轮回答里，哪些结论是确定的，哪些只是推测？请用两列表格说明。

## 本节与核心概念对照

| 核心概念       | 在本节的体现                           |
|------------|----------------------------------|
| 终端 AI 编程代理 | 你在 shell 里启动 <code>claude</code> |
| 四种能力       | 本节以对话为主；读文件在示例 B 露头              |
| 42 工具      | 不必数清；先建立「有时会调用工具」的直觉             |
| 权限模式       | 首次在工具调用处遇到 <code>y/n</code>      |

## 延伸阅读（心智模型）

[Mermaid 图表 — 请在在线版本中查看]

上一章: [2.2 安装](#) ← · 下一章: [2.4 编辑文件](#) →

## 2.4 编辑文件实操

**本节目标：**让 Claude Code **创建并修改**你磁盘上的文件；理解 **先读后写** 的常见模式；对 **y/n 权限提示** 有明确判断标准。

**安全提醒：**在练习目录操作；涉及删除、批量重写时格外谨慎。

### 学习目标

- 能口述：**为什么代理往往要先读文件再编辑。**
- 能完成：创建新文件、修改现有文件、让代理解释 **diff 思路**。
- 看到权限提示时，能判断：**这次允许会不会越界。**

### 生活类比：改作文 vs 瞎编

- **不读文件就改** = 没看你作文就重写——容易跑题、覆盖错段落。
- **FileRead → 再 FileEdit** = 先通读，再在原稿上批改——上下文对齐，冲突更少。

Claude Code 内置多类工具（总量约 **42** 个工具的概念），其中与文件强相关的大致包括：**读取、搜索、写入/编辑**等。不同版本命名可能略有差异，但\*\*「读→改」\*\*节奏是通用的。

[Mermaid 图表 — 请在在线版本中查看]

## 准备工作：一个最小练习项目

```
mkdir -p ~/cc-edit-demo
cd ~/cc-edit-demo
claude
```

在对话里可以说：

请在本目录创建一个 README.md，用中文写项目名称「练习仓」和一句简介，不要写其他文件。

## 流程拆解：FileRead → FileEdit（概念层）

下表用逻辑名称描述流程；你终端里看到的工具名可能叫 `Read`、`Edit`、`Write` 等，不必死记拼写，重在理解顺序。

| 步骤            | 代理在做什么      | 你为什么要在意                          |
|---------------|-------------|----------------------------------|
| 1. 发现目标       | 决定改哪个路径     | 路径错了会改错文件                        |
| 2. Read       | 拉取当前内容      | 避免覆盖无关部分、对齐编码风格                  |
| 3. 计划补丁       | 决定插入/替换哪些片段 | 大改前先看它计划是否合理                     |
| 4. Edit/Write | 应用变更        | 此时磁盘会真的变化                        |
| 5. 汇报         | 总结改了什么      | 你用 <code>git diff</code> 或打开文件核对 |

[Mermaid 图表 — 请在在线版本中查看]

## 权限提示（y/n）到底在问什么？

当界面出现类似「是否允许读取/写入某路径」时，可以按下面问题快速决策：

| 自检问题                 | 若答案为「是」则倾向          |
|----------------------|---------------------|
| 路径是否在我本意的项目内？        | 更安全                 |
| 是否是敏感文件（.env、密钥、私钥）？ | 先 n，改用脱敏示例          |
| 是否会大面积删除我看不见的内容？     | 先 n，要求「先展示 diff 计划」 |
| 我是否备份或已提交 Git？       | 更有底气按 y             |

生活类比：装修工要拆一面墙，先让你签字——y 不是「随便你」，而是「我确认这面墙可以拆」。

---

## 实操示例 1：创建新文件

---

你说：

```
新建 src/greet.ts，导出一个函数 greet(name: string): string,
返回 `Hello, ${name}!`, 并加一行注释说明用途。
```

期望结果：

- 代理可能先确认目录结构，再写入文件。
- 若需创建 `src/`，可能伴随写权限提示。

你本地验证：

```
cat src/greet.ts
```

---

## 实操示例 2：修改现有文件

---

先手动建一个「有瑕疵」的文件：

```
mkdir -p src
printf "export function add(a,b){return a+b}\n" > src/math.ts
```

你说：

```
给 src/math.ts 里的 add 补上 TypeScript 类型标注，并格式化得更易读，不要改函数行为。
```

期望观察点：

- 是否出现 Read 再 Edit。
- 改完后运行（若你环境有 `tsc`）：

```
npx tsc --noEmit 2>/dev/null || echo "若无 tsconfig 可忽略"
```

## 源码片段：改前 vs 改后（示意）

---

改前 `src/math.ts`：

```
export function add(a,b){return a+b}
```

改后（示意，非唯一正确答案）：

```
/** 返回两数之和 */  
export function add(a: number, b: number): number {  
  return a + b;  
}
```

在对话里你可以追加一句：

请用要点列表说明：你改了哪些 token，为什么不会改坏行为。

这能训练你读代理的解释，而不是盲信。

---

## 实操示例 3：拒绝与纠正

---

若代理提议写入错误路径，直接说：

不要写在 `src/math.ts`，请改为 `src/utils/math.ts`，其他要求不变。

若你已按了 `y` 但发现不对：

- 立即用 Git 回滚（若已 `git init`）：

```
git checkout -- src/math.ts  
# 或  
git restore src/math.ts
```

这衔接下一篇 Git 专章的安全习惯。

---

## 与「42 工具 / 权限模式」的关系

---

- 工具多 ≠ 一次全用上；与文件相关的几次调用就会拼出完整故事。

- **权限模式决定**：是否每次询问、是否信任某目录、是否允许大范围替换。入门阶段建议**偏保守：多询问、少自动**。

[Mermaid 图表 — 请在在线版本中查看]

## 故障排查

| 现象       | 可能原因       | 建议                |
|----------|------------|-------------------|
| 提示无写权限   | 目录只读或沙箱    | 检查目录权限；换到有写权限的路径  |
| 改完编码乱码   | 原文件非 UTF-8 | 说明编码；必要时先统一 UTF-8 |
| 与本地编辑器冲突 | 同时手改同一文件   | 一方改时另一方先停         |

## 本节练习清单

- [ ] 创建一个新文件并通过 `cat` 验证。
- [ ] 修改一个已有文件并理解代理的**读→改**顺序。
- [ ] 故意给错路径，观察代理如何纠正或询问。
- [ ] 至少一次对 `y/n` 选择 `n`，看代理是否给出替代方案。

## 小结

- **文件编辑**是代理最有力的能力之一，也是**最需要你盯紧**的能力。
- `FileRead` → `FileEdit` 是为减少「瞎改」的标准节奏。
- `y/n` 是你的**签字权**；不确定就 `n`，并要求**先展示计划或 diff**。

上一章：[2.3 第一次对话](#) ← · 下一章：[2.5 运行命令](#) →

# 2.5 运行命令实操

**本节目标**：让 Claude Code 提议并（在你批准后）**执行终端命令**；理解 **Bash 类工具**在干什么；建立**命令审批**的安全习惯。



**安全提醒：**永远先看完整命令再按 y；涉及 `rm`、`curl` | `sh`、改系统配置要特别警惕。

## 学习目标

- 能说明：为什么执行命令必须经过你确认（默认情况下）。
- 能完成：让代理运行 `npm`、`git`、测试命令，并阅读终端输出摘要。
- 知道：代理看到的输出可能截断，必要时自己本地重跑对照。

## 生活类比：你是机长，代理是副驾驶

- 副驾驶可以提议：「建议现在放下起落架（跑测试）」。
- 你推杆才算数：按 y 才是「我授权执行」。
- 恶劣天气要复飞：命令里有可疑管道、远程脚本、递归删除——果断 n。

[Mermaid 图表 — 请在在线版本中查看]

## BashTool / Shell 执行：工作原理（小白版）

实现细节上，Claude Code 使用 `Bun` 运行时 + `TypeScript` 组织逻辑，并通过工具层把「跑一条 shell 命令」封装成模型可调用的能力。你在 UI 里常看到：

1. 命令预览：完整或接近完整的字符串。
2. 工作目录：在哪个文件夹执行（`cwd`）。
3. 超时/输出限制：过长输出会被截断展示。

不必背内部类名，记住用户侧事实即可：这是一条会在你机器上真实执行的 shell 命令。

| 概念      | 人话                      |
|---------|-------------------------|
| 工具调用    | 模型说「我要跑终端」              |
| 审批 gate | 终端前加一道你的 y/n            |
| 退出码     | 0 通常成功，非 0 常表示失败（依命令而定） |

[Mermaid 图表 — 请在在线版本中查看]

## 命令审批机制：你应该看什么？

在按 `y` 之前，至少检查：

| 检查项         | 反例（要警惕）                                                 |
|-------------|---------------------------------------------------------|
| 是否是你认识的子命令？ | <code>curl ...   bash</code>                            |
| 路径是否在你的项目内？ | <code>cd / &amp;&amp; rm -rf</code>                     |
| 会不会改全局环境？   | <code>sudo ...</code> 、 <code>brew install</code> （视场景） |
| 会不会上传代码或密钥？ | 不明 <code>scp</code> 、 <code>rsync</code> 到陌生主机          |

好习惯：让代理先解释每条 `flag`，你再批准。

## 实操示例 1： `npm` 脚本

在已有 `package.json` 的项目里（可让代理先生成一个最小 demo）：

你说：

请读取 `package.json`，列出 `scripts` 里有哪些命令，并建议一个适合检查代码风格的命令；  
若存在则运行它，运行前把完整命令给我看，我批准后再执行。

期望：出现 `npm run lint` 或类似；你核对后按 `y`。

## 实操示例 2：单元测试

你说：

运行测试套件。若失败，根据报错定位到具体测试文件并给出修复建议；修复前先说明打算改哪些行。

期望节奏：

1. 跑 `npm test` / `pnpm test` / `pytest` 等（以项目为准）。
2. 解析失败日志。

3. 可能回到 读文件 → 改文件（见上一节）。

---

## 实操示例 3: `git` 只读命令（为下一节铺垫）

---

你说：

```
只运行 git 只读命令: git status 和 git log -1 --oneline, 不要改任何东西。
```

这练习两件事：命令审批 + 区分只读与写入。

---

## 输出截断与「自己再跑一遍」

---

代理贴给你的终端输出可能是摘要。当出现：

- 报错行号对不上；
- 似乎成功但本地仍失败；

请在自己的终端原样重跑：

```
npm test 2>&1 | tee /tmp/test.log
```

把 `/tmp/test.log` 关键段贴回对话。

---

## 与四种能力的关系

---

Claude Code 的四类能力里，本节对应「运行命令」，并常与「搜索代码」「编辑文件」组合：

[Mermaid 图表 — 请在在线版本中查看]

## 常见坑

| 坑          | 说明                                         |
|------------|--------------------------------------------|
| 环境不一致      | 代理用的 shell 环境变量可能与你手动终端不同；以文档与实测为准         |
| 交互式命令      | <code>vim</code> 、密码输入类命令不适合自动化；改要非交互 flag |
| Windows 路径 | WSL 与 CMD 路径差异；统一用一种终端                     |

## 本节练习清单

- [ ] 批准一次 `npm` 类命令，确认输出被正确解释。
- [ ] 拒绝一次命令（按 `n`），观察代理是否换用更安全步骤。
- [ ] 对失败测试做一次「本地重跑 + 贴日志」闭环。

## 小结

- **Bash/Shell 工具** = 把 shell 命令接到模型上；**真实执行**发生在你机器上。
- **审批机制**是最后一道防线：**看清再 y**。
- **验证**时信日志、信本地复现，而不只信摘要。

## Bash 单行 vs 多行：让代理「展示再执行」

有时代理会拼接长命令，你可要求：

把将要执行的命令用代码块完整展示，换行符保持可读；  
我回复「批准」后你再请求执行。

这能减少「一行里藏十个管道」的误按 `y`。

## 环境变量与命令行为

| 变量族                        | 可能影响                                           |
|----------------------------|------------------------------------------------|
| <code>PATH</code>          | 找到的是哪个 <code>node</code> / <code>python</code> |
| <code>NODE_ENV</code>      | 依赖是否走 dev/prod 分支                              |
| <code>HTTP(S)_PROXY</code> | 企业网络下 API 与包下载                                 |

若「我手动终端可以、代理不行」，第一件事：对比 `env` 输出（注意脱敏）。

## 非交互标志速查（减少卡住）

| 工具  | 常见非交互 flag（示例）                                  |
|-----|-------------------------------------------------|
| npm | <code>npm ci</code> 、 <code>--yes</code> （视子命令） |
| apt | <code>-y</code>                                 |
| npx | <code>--yes</code>                              |

原则：自动化时尽量避免 `vim`、`less` 内分页停留。

## 实操示例 4：失败命令的「分层诊断」

你说：

上次 `bash` 命令退出码非 0。请按顺序：  
1) 复述命令与工作目录  
2) 解释 `stderr` 关键行  
3) 给出两条修复路径：保守与激进  
不要自动重试，等我选择。

这训练你把代理当参谋，而不是自动驾驶。

## 与 Bun 运行时（实现侧）的一句话关系

Claude Code 自身常用 `Bun` 等现代运行时开发与分发；你在本机执行的 `npm` 包装 CLI 后，内部如何启动属于实现细节。省钱与稳定仍取决于：少跑无意义命令、先看再 `y`。

## 本节命令审批「红名单」(建议直接 n)

| 模式      | 示例特征                                                          |
|---------|---------------------------------------------------------------|
| 远程管道安装  | <code>curl ...   sh</code> 、 <code>wget -O- ...   bash</code> |
| 递归删除    | <code>rm -rf /</code> 、 <code>rm -rf /*</code> 、删上级目录         |
| 乱配 sudo | 不明目的的 <code>sudo tee /etc/...</code>                          |
| 乱改远程    | <code>git push --force</code> (见 Git 专章)                      |

## 练习记录表

| 日期 | 批准的命令 | 退出码 | 事后评价 (值/不值) |
|----|-------|-----|-------------|
|    |       |     |             |

上一章: [2.4 编辑文件](#) ← · 下一章: [2.6 Git 工作流](#) →

## 2.6 Git 工作流

**本节目标:** 让 Claude Code 协助 `status` / `diff` / `commit` / 分支 / PR 草稿; 理解 Git 安全协议: 哪些操作默认应拒绝或必须人工复核。

**安全提醒:** 永远不要让代理在未经你理解的情况下执行 **改写历史的危险命令**。

## 学习目标

- 能区分: 只读 `git` 命令 vs 写入历史/远程 的命令。
- 能完成: 一次由代理辅助的 `add + commit` (信息清晰)。
- 能背熟两条红线: 不随意 `push --force`、不随意 `reset --hard`。

## 生活类比: Git 是「时光机 + 交接班日志」

- `git status`: 看看桌上文件谁动过。
- `git diff`: 具体改了哪几行。

- `commit`：写一条交接说明，以后自己能看懂。
- PR (Pull Request)：请同事/CI 评审的「正式申请」。

代理适合帮你起草说明、整理改动、跑检查；是否合并、是否强推，必须是你或团队的决策。

[Mermaid 图表 — 请在在线版本中查看]

## Git 安全协议（本指南的默认约定）

下列命令在新手与多数团队场景下应极其谨慎或由人工单独执行：

| 命令族                                | 风险              | 本指南态度                                 |
|------------------------------------|-----------------|---------------------------------------|
| <code>git push --force / -f</code> | 覆盖远程历史，可能删掉同事提交 | 默认 <b>不要</b> 让代理做；确需时由你亲自执行并确认        |
| <code>git reset --hard</code>      | 丢弃本地未提交/未保护改动   | 默认 <b>不要</b> ；先备份或 <code>stash</code> |
| <code>git clean -fd</code>         | 删除未跟踪文件         | 高风险；先看 <code>git clean -n</code>      |
| 改写历史的 rebase + push                | 协作冲突、误丢记录       | 仅在你理解影响时                              |

[Mermaid 图表 — 请在在线版本中查看]

一句话：代理可以是「秘书」，但签字的必须是你。

## 实操示例 1：从改动到提交

前置：在练习仓库里让代理改过文件（见 2.4）。

你说：

执行 `git status` 和 `git diff --stat`，总结改了哪些文件；  
然后写一个符合 Conventional Commits 的 commit message 草稿（中文说明即可），  
我确认后你再给出 `git add` 与 `git commit` 的具体命令——我批准后再运行。

你检查：

- `git diff` 是否包含意外文件（如 `.env`）？
- commit message 是否准确？

## 实操示例 2：分支与功能开发（轻量流程）

你说：

基于当前 main 创建分支 feature/greet-readme，切换过去，把 README 里加一行「由 Claude Code 练习生成」，然后提交。不要 push。

期望：出现 `git checkout -b` 或 `git switch -c`（依 Git 版本）。

## 实操示例 3：PR 描述草稿

当你已 push 分支（由你决定何时 push）后：

你说：

根据本次分支相对 main 的 diff，写一份 PR 描述：包含背景、改动点、测试说明、风险；用 Markdown 输出，我复制到 GitHub/GitLab。

代理通常不需要写权限即可完成（纯文本）；若它提议跑 `gh pr create`，同样要走 命令审批。

## 与「协调工作流」能力的关系

四类能力中的 协调工作流，在 Git 场景体现为：

1. 读差异 → 2. 跑测试 → 3. 修失败 → 4. 再 diff → 5. 提交说明

这是一串有顺序的多步任务，代理擅长串联，但每一步仍要你把关关键闸门。

[Mermaid 图表 — 请在在线版本中查看]

## `.gitignore` 与密钥（必读习惯）

在让代理「顺手提交」前，确认：

- `.env`、`*_key.pem`、令牌文件已在 `.gitignore`。
- 若代理提议 `git add .`，先看列表再 y。



## 团队协作时的额外规则

| 场景       | 建议                           |
|----------|------------------------------|
| 公司预提交钩子  | 代理跑命令可能触发 husky；失败时把钩子日志贴回对话 |
| 主分支保护    | 不能直接 push main 时，一律走 PR      |
| fork 工作流 | 先确认 remote 名称与上游             |

## 故障排查

| 问题      | 方向                                                                                                 |
|---------|----------------------------------------------------------------------------------------------------|
| 提交包含大文件 | 用 <code>git reset</code> (soft) 撤回提交索引；改 <code>.gitignore</code> ；必要时 <code>git rm --cached</code> |
| 合并冲突    | 先 <code>git status</code> ，人工打开冲突文件；代理可协助读冲突块                                                      |
| 身份未配置   | 配置 <code>user.name</code> / <code>user.email</code> （一次性）                                          |

## 本节练习清单

- [ ] 完成一次 `add + commit`，message 由代理起草、你改过再提交。
- [ ] 明确拒绝一次 `force push` 类提议（若代理误提）。
- [ ] 生成一份 `PR 描述 Markdown` 并保存为团队模板雏形。

## 小结

- Git 协作里，代理擅长 草稿与自动化命令串联；你负责 范围、历史安全、远程策略。
- 安全协议：默认远离 `push --force` 与 `reset --hard`。
- 提交前永远 `status + diff`，密钥不进仓。

### `git add -p` 与代理协作（进阶但实用）

当你只想提交一部分改动时，可以要求：

不要 `git add .`，请教我使用 `git add -p` 交互式挑选；  
若环境不允许交互，请给出等价方案（例如拆分文件或手动备份后还原）。

这能显著降低「把调试垃圾文件一并提交」的概率。

## 提交信息模板（Conventional Commits 中文混合版）

| 字段      | 示例                                                                            |
|---------|-------------------------------------------------------------------------------|
| type    | <code>feat</code> / <code>fix</code> / <code>docs</code> / <code>chore</code> |
| scope   | <code>web</code> 、 <code>api</code>                                           |
| subject | <code>修复登录态过期未刷新</code>                                                       |

整行示例：`fix(web): 处理 token 过期后的静默登出`

让代理按此模板生成时，加上：「subject 不超过 50 字」。

## 远程与分支：先画图再动手

[Mermaid 图表 — 请在在线版本中查看]

口语指令：

请用 ASCII 或 Mermaid 画出当前分支相对 main 的关系，  
并说明如果要开 PR 还需要哪些 git 命令（不要执行）。

## 与「6 种权限模式」的交叉点

Git 写入（commit、checkout）往往触发写工作区或改 `.git` 相关操作；在较严权限下会多次询问。  
建议：练习阶段保持严格；熟练后可在可信仓库路径适度放宽，但永远不要对全局 `~` 目录放权。

## 团队协作话术（复制给同事）

我们约定：Claude Code 可以协助 `git status/diff/commit message`，  
但任何 `--force`、`--hard`、历史改写类命令必须人工在本机执行。  
PR 描述可由 AI 起草，但责任人必须通读 `diff`。

## 故障案例簿（填空用）

| 现象          | 当时执行的命令 | 根因 | 正确姿势 |
|-------------|---------|----|------|
| 提交巨大二进制     |         |    |      |
| 把 .env 提交上去 |         |    |      |

上一章：[2.5 运行命令](#) ← · 下一章：[2.7 命令速查表](#) →

## 2.7 命令速查表

**本节目标：**一页收藏 斜杠命令、常用自然语言指令、快捷键与配置入口。

**说明：**Claude Code 持续迭代，个别命令名以你本机 `/help` 为准；下表覆盖入门最常用的语义。

### 学习目标

- 能快速打开 `/help` 并定位所需子命令。
- 知道 `/compact`、`/memory`、`/model`、`/cost` 分别在什么场景用。
- 能把自然语言指令写成「目标 + 约束 + 验收标准」三段式。

### 斜杠命令总览（大表）

下列命令为**教学整理版**；若与当前版本不一致，以终端内 `/help` 输出为准。

| 命令                                          | 典型作用                | 小白什么时候用                            |
|---------------------------------------------|---------------------|------------------------------------|
| <code>/help</code>                          | 显示内置帮助              | 忘了有哪些斜杠命令时 <b>第一站</b>              |
| <code>/compact</code>                       | 压缩/整理上下文            | 对话很长、开始变慢或答非所问时                    |
| <code>/clear</code>                         | 清空屏幕或会话上下文（依版本）     | 想「换话题从零开始」                         |
| <code>/memory</code>                        | 管理长期记忆相关项（依版本）      | 想固定团队规范、个人偏好                       |
| <code>/model</code>                         | 切换或查看模型             | 要在 <b>Opus / Sonnet</b> 等与成本、速度间权衡 |
| <code>/cost</code>                          | 查看费用/用量线索（依版本）      | 心里没底时看一眼「烧钱节奏」                     |
| <code>/permissions</code> 或类似               | 权限模式相关（名称以 help 为准） | 调整 <b>工具审批</b> 的严格程度               |
| <code>/init</code> / <code>/doctor</code> 类 | 初始化、诊断环境（若有）        | 安装异常、连不上 API 时                     |
| <code>/resume</code> 类                      | 恢复会话（若有）            | 断线后继续刚才的上下文                        |
| <code>/export</code> 类                      | 导出对话（若有）            | 需要留档复盘                             |

[Mermaid 图表 — 请在在线版本中查看]

## 常用自然语言指令模板

把指令想成「目标 + 约束 + 验收」，代理更少跑偏。

| 场景    | 模板示例                                                                                   |
|-------|----------------------------------------------------------------------------------------|
| 只读分析  | 「只阅读 <code>src/a.ts</code> ，解释 <code>foo</code> 函数做什么， <b>不要修改任何文件。</b> 」              |
| 小步改代码 | 「在 <code>utils/date.ts</code> 增加 <code>formatIso(d: Date)</code> ，补单测，改完 <b>跑测试。</b> 」 |
| 修 bug | 「复现步骤：... 当前报错：... 请先 <b>定位文件与行号</b> ，再给出 <b>最小修复。</b> 」                               |
| 重构    | 「把 <code>legacy.js</code> 里重复逻辑抽到 <code>helpers</code> ， <b>行为不变</b> ，用测试证明。」          |
| Git   | 「 <code>git status</code> 里这些改动，帮我写 <b>commit message</b> ， <b>不要 push。</b> 」          |
| 省钱    | 「回答尽量短；若需长文先列提纲； <b>不要随便读整个 node_modules。</b> 」                                        |

生活类比：你对快递员说「放门口鞋柜上、别按门铃」——**约束越清楚，事故越少。**

## 键盘快捷键（通用习惯 + 终端注意）

终端应用快捷键因系统与终端模拟器而异，下表为常见约定；若无效请查本机终端设置。

| 操作      | macOS 常见            | Windows/Linux 常见        |
|---------|---------------------|-------------------------|
| 中断当前任务  | Ctrl + C            | Ctrl + C                |
| 清屏      | Cmd + K 或 Ctrl + L  | Ctrl + L                |
| 粘贴      | Cmd + V             | Ctrl + Shift + V 或右键    |
| 搜索滚动缓冲区 | Cmd + F             | Ctrl + Shift + F（视终端而定） |
| 光标到行首/尾 | Ctrl + A / Ctrl + E | 同左（readline）            |

[Mermaid 图表 — 请在在线版本中查看]

**提示：** Claude Code 基于 **React Ink** 等终端 UI 技术，部分快捷键可能与 shell **抢焦点**；冲突时以官方文档为准。

## 配置选项与环境变量（速查）

以下为入门最常遇到的配置入口；完整列表以官方 README / 文档为准。

| 类型   | 名称/示例                   | 作用                 |
|------|-------------------------|--------------------|
| 环境变量 | ANTHROPIC_API_KEY       | API 鉴权             |
| 环境变量 | ANTHROPIC_BASE_URL（若适用） | 自定义 API 端点（企业网关等）  |
| 项目文件 | CLAUDE.md               | 给代理的固定说明：技术栈、命令、禁忌 |
| 忽略文件 | .gitignore              | 避免把密钥与构建产物提交       |
| 权限相关 | 依版本的 settings / config  | 控制工具自动批准范围         |

**CLAUDE.md** 最小示例：

```
# 项目说明
- 包管理: pnpm
- 测试: pnpm test
- 禁止: 不要执行 rm -rf / 与任何 force push
- 风格: TypeScript strict, 优先函数式小组件
```

## 与「6 种权限模式」相关的使用口诀

权限模式名称以你版本为准，可从 `/help` 或设置中查看。入门可记：

| 态度   | 行为                                                   |
|------|------------------------------------------------------|
| 练习期  | 能多问就多问；所有写盘与 <code>bash</code> 都手动 <code>y</code>    |
| 日常开发 | 信任子目录（如 <code>src/</code> ），敏感路径仍询问                  |
| 高危目录 | <code>.env</code> 、 <code>~/.ssh</code> 、系统目录 — 永远警惕 |

[Mermaid 图表 — 请在在线版本中查看]

## 自然语言 → 工具链 对照（概念）

| 你说的话        | 代理可能做的事                       |
|-------------|-------------------------------|
| 「看看这个报错」    | 读日志文件 / 读相关源码                 |
| 「跑一下测试」     | Bash: <code>npm test</code> 等 |
| 「搜一下谁调用了 X」 | 代码搜索工具                        |
| 「提交一下」      | git 命令序列（需你审）                 |

## 调试对话的三句「咒语」

1. 「先复述你理解的任务，再动手。」
2. 「**每一步**只做一件事，做完停顿等我确认。」
3. 「若需要权限，**先解释**为什么需要。」

## 本节自测

- [ ] 在会话里输入 `/help` ，截图或复制保存到笔记。
- [ ] 故意发一个很长的对话后试 `/compact` （若有），观察前后差异。
- [ ] 写好自己的 `CLAUDE.md` 骨架并放入真实项目。

## 小结

- 斜杠命令是终端里的「快捷菜单」： `/help` 为根。
- 自然语言写清楚 目标 / 约束 / 验收，代理更稳。
- 配置优先 `CLAUDE.md` + 环境变量，与 权限模式 搭配使用。

## 按场景查表：我该说什么？

| 我想... | 一条指令示例                                          |
|-------|-------------------------------------------------|
| 只看不动手 | 「只读分析，不要写文件不要跑命令」                               |
| 小步迭代  | 「每次只改一个文件，改完等我确认」                               |
| 加速写测试 | 「为 <code>src/foo.ts</code> 写 vitest 用例，覆盖边界」    |
| 国际化   | 「把界面文案抽到 <code>locales/zh.json</code> ，不要改业务逻辑」 |
| 性能    | 「先 profile 再优化，禁止过早微优化」                         |

## 与内置「约 42 工具」的映射（概念级）

不必背诵工具 ID，可按动词记忆：

| 动词 | 你常下的口令                                |
|----|---------------------------------------|
| 读  | 「读取 <code>...</code> 前 80 行」          |
| 搜  | 「全仓搜 <code>deprecatedX</code> 的引用」    |
| 写  | 「新建 <code>...</code> 」「把第 N 段替换为 ...」 |
| 跑  | 「在非交互方式跑 <code>pnpm test</code> 」     |
| 协调 | 「列出任务清单再逐项执行」                         |

[Mermaid 图表 — 请在在线版本中查看]

## 退出、挂起与会话生命周期

| 动作     | 常见操作                                                   |
|--------|--------------------------------------------------------|
| 结束当前程序 | <code>exit</code> 或 <code>Ctrl+C</code> （注意是否中断正在跑的命令） |
| 暂时离开   | 关闭终端标签前确认无长任务                                          |
| 续聊     | 若版本支持 resume，按 <code>/help</code> 查找对应命令               |

## 「反模式」口令合集（少踩坑）

| 反模式        | 为什么不好              |
|------------|--------------------|
| 「你随便改」     | 范围无限 → Token 与风险双高 |
| 「读整个项目」    | 上下文爆炸 → 又贵又慢       |
| 「帮我执行网上脚本」 | 供应链风险              |
| 「不用测试」     | 回归不可见              |

## 备份：纯文本速记卡

|                        |              |
|------------------------|--------------|
| <code>/help</code>     | → 帮助根入口      |
| <code>/compact</code>  | → 压缩上下文      |
| <code>/memory</code>   | → 记忆与偏好（依版本） |
| <code>/model</code>    | → 模型切换       |
| <code>/cost</code>     | → 费用线索（依版本）  |
| <code>Ctrl+C</code>    | → 中断         |
| <code>CLAUDE.md</code> | → 项目固定说明     |
| 先 y 前三问                | → 路径？敏感？可回滚？ |

## 版本漂移时的自检清单

- [ ] 运行 `/help`，与本页表格逐条对照是否改名或新增。
- [ ] 记录你当前 `claude --version` 在笔记顶部。
- [ ] 重大升级后重读官方 Release Notes 一段。

上一章：[2.6 Git 工作流](#) ← · 下一章：[2.8 费用与省钱](#) →



## 2.8 费用与省钱技巧

**本节目标：**理解 Anthropic API 按 Token 计费；知道 Opus / Sonnet 大致价差与适用场景；掌握 缓存、/compact、[CLAUDE.md](#)、模型选择 等降本习惯。

**声明：**单价与套餐以 [Anthropic 官方定价页](#) 为准；下文 **美元区间** 为社区常见经验量级，用于建立直觉，**非财务承诺**。

### 学习目标

- 能解释：输入 Token 与 输出 Token 为何都计费。
- 能说明：提示缓存（prompt caching）为何可能显著省钱。
- 能执行：`/compact`、手动压缩长对话、用 `CLAUDE.md` 固定重复信息。
- 能根据任务选 合适模型：笨重任务用强模型，机械活用最便宜够用的。

### Token 计费原理（小白版）

把 Token 粗想成「模型读写的词块」：英文常 1 词≈若干 token，中文通常 **字数越多 token 越多**。

| 项目       | 人话                                        |
|----------|-------------------------------------------|
| 输入 Token | 你发的指令 + 代理读的文件 + 工具返回的内容， <b>进入模型窗口都算</b> |
| 输出 Token | 模型生成给你的文字、计划、代码块                          |
| 计费       | 一般 <b>输入价 ≠ 输出价</b> ；总价 = 各段用量 × 单价       |

[Mermaid 图表 — 请在在线版本中查看]

**生活类比：**Token 像**通话分钟**——不只听你说话，对方复述整本说明书也会狂烧分钟。

### Opus 与 Sonnet：怎么选？（对比表）

具体名称以控制台为准；以下为**能力 vs 成本**的通用对比框架。

| 维度   | Opus 系（更强）        | Sonnet 系（均衡/更快） |
|------|-------------------|-----------------|
| 复杂推理 | 通常更强              | 足够应对多数日常开发      |
| 速度   | 往往更慢              | 往往更快            |
| 单价   | 更高                | 相对较低            |
| 适合任务 | 架构设计、疑难 bug、跨模块重构 | 改接口、写测试、文档、脚手架  |

口诀：越「想破头」的活越值得 Opus；越「有套路」的活越适合 Sonnet。

[Mermaid 图表 — 请在在线版本中查看]

## 缓存：为什么能省很多钱？

提示缓存（Prompt Caching）的大意是：把重复出现的长上下文（例如固定系统说明、反复附带的 `CLAUDE.md`、稳定的大段代码）标记为可缓存后，后续请求重读这些部分时按 **缓存读取价** 计费，而非全价输入。

| 场景              | 无缓存                             | 有缓存（直觉）              |
|-----------------|---------------------------------|----------------------|
| 长会话反复带同一份「项目宪法」 | 每次都按普通输入全价算                     | 大块只付 <b>缓存命中</b> 的低价 |
| 典型体感（社区经验）      | 重度 Opus 会话 约 \$50—100（无缓存、长上下文） | 约 \$10—19（有缓存、相似前缀多） |

官方常给出 **缓存读取** 远低于普通输入的单价；教学中可记一个数量级：**缓存读取约 \$0.50 / 百万 Token** 这一档（以官网为准）。

**注意：**能否命中缓存、如何组织前缀，依赖 **API 与客户端实现**；Claude Code 用户侧重点是：**减少无意义重复、把稳定内容放固定文件**。

[Mermaid 图表 — 请在在线版本中查看]

### `/compact`：对话太长就「收成摘要」

当上下文膨胀时，模型「看得见」的窗口里塞满旧对话，**既贵又容易糊涂**。

| 做法                            | 作用                |
|-------------------------------|-------------------|
| <code>/compact</code> （若版本支持） | 把冗长历史 压缩成较短摘要 再继续 |
| 新开会话 + 粘贴「当前状态」               | 人工做极简 handoff     |

与 60% 规则：社区经验是 上下文用到约 60% 时，主动压缩或拆会话，性价比与准确率更稳（非严格科学阈值，当作习惯）。

[Mermaid 图表 — 请在在线版本中查看]

**CLAUDE.md：把「每次都要交代的话」写一次**

重复说明技术栈、命令、目录结构 = 每轮都在烧同样的输入 Token。

推荐固定写清：

- 包管理器与安装命令（`pnpm i` / `npm ci`）
- 测试命令
- 目录约定（`apps/web`、`packages/core`）
- 安全红线（禁止的操作）

示例片段：

```
# Claude 项目说明
- 安装: pnpm install
- 测试: pnpm test
- 主应用在 apps/web
- 不要读取或上传 .env
```

## 省钱技巧清单（可打印）

| 技巧                                 | 预期收益          |
|------------------------------------|---------------|
| 稳定前缀放 <code>CLAUDE.md</code>       | 少重复、易触发缓存友好结构 |
| 长对话用 <code>/compact</code>         | 降输入长度、减糊涂回复   |
| ~60% 上下文时主动压缩/拆会话                  | 控制单价与质量       |
| 简单任务选 Sonnet                       | 直接降单位成本       |
| 让模型「先给计划再动手」                       | 减少无效工具往返      |
| 避免「全文搜索整个大仓」式指令                    | 缩小读文件范围       |
| 本地先 <code>git diff</code> / 小范围贴代码 | 减少整文件反复读取     |

## 费用直觉区间（再次强调：非承诺）

以下为 **教学用区间**，帮助你判断「会不会很贵」：

| 场景               | 经验区间（美元，粗略）                     |
|------------------|---------------------------------|
| 无缓存、长会话、强模型 Opus | 约 \$50—100                      |
| 有缓存、重复前缀多、合理压缩   | 约 \$10—19                       |
| 缓存读取单价量级         | 约 \$0.50 / 百万 Token（读缓存段；以官网为准） |

**务必：**以控制台 **用量与账单** 为准，定期 `/cost` 或到网页后台核对。

## 与 Claude Code 技术栈的间接关系

Claude Code 使用 Bun + TypeScript + React Ink 等实现终端体验；**计费不发生在 Bun 层**，而是调用 Anthropic API 时按 Token 结算。性能优化（更快 UI）改善体验，但省钱核心仍在 Token 策略。

[Mermaid 图表 — 请在在线版本中查看]

## 本节自测

- [ ] 用自己的话向同事解释「输入 Token 从哪来」。
- [ ] 写一份 `CLAUDE.md`，列出三条以上稳定信息。
- [ ] 在长对话中试一次 `/compact`（若可用），记录前后体感。
- [ ] 打开官方定价页，记下 **当前** Sonnet/Opus 的输入输出单价。

## 小结

- Token = 计费的「字数/词块」；工具返回的大段文本也会推高输入。
- 缓存 + 压缩 + `CLAUDE.md` 是三板斧：少重复、短上下文、固定前缀。
- 模型选择是直接的杠杆：**难活强模型，套路活省着用**。

## 账单异常排查（从便宜到费事）

| 步骤 | 做什么                          |
|----|------------------------------|
| 1  | 在控制台看 按模型/按天 聚合是否突增          |
| 2  | 回忆是否 反复粘贴超大日志                |
| 3  | 检查是否 无压缩长会话 连跑数小时            |
| 4  | 搜索仓库是否有人 提交了 API Key（立即轮换密钥） |

## 与「四种能力」相关的计费热点

| 能力    | 容易烧 Token 的行为   | 省钱替代                              |
|-------|-----------------|-----------------------------------|
| 编辑文件  | 反复整文件读写         | 指定行区间；先 <code>git diff</code> 贴片段 |
| 运行命令  | 超大 stdout 全进上下文 | 管道到文件，只贴摘要                        |
| 搜索代码  | 太宽的关键词          | 缩小目录；先用 <code>rg</code> 本地粗搜      |
| 协调工作流 | 一步失败仍疯狂重试       | 要求「失败即停，给两条假设」                    |

[Mermaid 图表 — 请在在线版本中查看]

## 会话预算制（给团队或个人）

| 角色      | 建议                                  |
|---------|-------------------------------------|
| 个人学习    | 每日设心理上限；用 <code>/cost</code> 或控制台对照 |
| 团队 Lead | 强模型仅开给攻坚频道；日常 Sonnet                |
| 公司财务    | 走组织账单 + 项目标签（若产品支持）                 |

## 延伸阅读：官方与社区

- 定价以 Anthropic 官网 为准；本文区间仅供数量级感受。
- 社区中的「省钱技巧」需二次验证：**安全**（不泄露代码）与**合规**（公司政策）。

## 空白模板：我的省钱规则（请填写）

- 默认模型：\_\_\_\_\_
- 何时升级到强模型：\_\_\_\_\_
- 上下文超过 \_\_\_\_% 必 compact/新开会话
- 禁止代理读取：\_\_\_\_\_ 目录
- 每月预算心理线：\_\_\_\_\_ 元/美元

上一章：[2.7 命令速查表](#) ← · 返回：[2.1 总览](#) ←

## 第3篇：架构全景

### 第 3 篇 · 3.1 不是 CLI，是「以 LLM 为内核的操作系统」

#### 学习目标

完成本节后，你将能够：

1. 用「操作系统」隐喻理解 Claude Code 的整体定位，而不仅把它当成「又一个命令行工具」
2. 从多入口、进程/会话、权限、文件系统、插件生态五个维度，论证它与普通 CLI 的本质差异
3. 读懂一张「分层操作系统」式的架构示意图，并能向他人复述各层职责
4. 建立后续章节（四入口、目录结构、数据流、技术栈）的心理地图

#### 3.1.1 核心论点：内核是 LLM，外壳是运行时与策略

如果把传统 CLI 工具比作一把瑞士军刀——功能再多，本质仍是「你拨一下、它动一下」的被动工具——那么 Claude Code 更接近一台装了智能调度器的计算机：

| 维度   | 普通 CLI 工具              | Claude Code（操作系统隐喻）      |
|------|------------------------|--------------------------|
| 调度中心 | 无，或仅脚本顺序执行             | LLM 持续规划、决策、纠错           |
| I/O  | stdin/stdout 为主        | 终端 UI、Bridge、MCP、SDK 多通道 |
| 资源   | 单次命令作用域                | 工具、服务、会话状态 长期协作          |
| 安全   | 通常「全有或全无」（当前 shell 权限） | 多层门控（权限、确认、沙箱策略等）        |
| 扩展   | 子命令/插件零散               | 插件、Skills、MCP 形成生态位      |

**生活类比：**普通 CLI 像自动售货机——你按按钮、掉货；Claude Code 像带门禁与工牌的办公楼——有前台（入口路由）、物业（服务层）、安保（权限）、装修队（工具执行），而「大楼的大脑」是 LLM 对任务的持续推理。

### 3.1.2 为什么说它是「操作系统」而非「大脚本」？

操作系统（OS）的典型特征包括：多程序入口、进程/任务生命周期、权限与安全模型、统一文件与设备抽象、可扩展的软件生态。Claude Code 在工程形态上与之对应：

1. **多入口**：同一套核心能力，可通过 CLI、`main.tsx` 启动路径、MCP 服务模式、SDK 编程模式进入（详见 `02-four-entries.md`）。
2. **进程/会话管理**：Agent 会话、Bridge 与 IDE 的跨进程通信、远程模式等，都涉及**长生命周期**与状态恢复，而非「一条命令结束即消失」。
3. **权限系统**：工具调用前的允许/拒绝、用户确认、策略配置——类似 OS 的 **capability** 思想。
4. **文件系统访问**：大量工具围绕读写、搜索、Git、diff 展开，相当于在「用户工作区」这一**挂载卷**上作业。
5. **插件生态**：Plugins、Skills、MCP 服务器——类似 OS 上的**驱动与扩展包**。

[Mermaid 图表 — 请在在线版本中查看]

### 3.1.3 分层视角：从「终端画面」到「模型决策」

再缩小一层粒度，可以把运行时看成 **呈现层 → 应用编排 → 模型内核 → 工具与 IO**：

[Mermaid 图表 — 请在在线版本中查看]

类比：L1 是**屏幕和键盘**；L2 是**窗口管理与任务栏**；L3 是**CPU 执行指令**（这里换成 LLM 生成「下一步」）；L4 是**磁盘、网卡、USB**（工具与外部世界）。注意 L4 → L2 的回路：工具结果会回到编排层，进入下一轮推理——这是 Agent 循环，不是单次管道。



### 3.1.4 与普通「开发类 CLI」的对比表

| 对比项 | 典型 CLI（如构建器、脚手架） | Claude Code                      |
|-----|------------------|----------------------------------|
| 主循环 | 解析参数 → 执行 → 退出   | 对话/Agent 循环，可长期驻留                |
| 状态  | 多为无状态或配置文件       | 会话、历史、压缩上下文、Bridge 会话等           |
| 扩展点 | 固定子命令            | MCP、Plugin、Skill、内部 Feature Flag |
| UI  | 日志行              | 自研 Ink + Yoga 的富终端 UI            |
| 风险面 | 本地命令即用户权限        | 显式门控 + 可配置策略（多道安检）               |

[Mermaid 图表 — 请在在线版本中查看]

### 3.1.5 关键源码「锚点」（阅读路线）

以下路径为**教学用锚点**：具体仓库版本可能微调，但职责分区稳定，便于你在源码树中「着陆」：

```
// 概念锚点：命令行入口通常集中注册子命令（Commander.js）
// 实际文件：src/main.tsx（体量很大，宜配合 08-main-entry.md 分段阅读）
program
  .name("claude")
  .description("...")
  // .command(...) 注册各命令 → 对应 src/commands/
```

```
// 概念锚点：工具注册与调用 — OS 的「系统调用」入口
// 实际目录：src/tools/（数十个工具模块）
export const toolDefinitions = [
  // read, write, bash, grep, ...
];
```

**学习建议**：先记住「一个内核（LLM）+ 多入口 + 工具 syscall + 权限门控」四句话，再进入 `02-four-entries.md` 看四种「开机方式」。

### 本节小结

- Claude Code 的产品形态在终端，但系统形态是围绕 LLM 的长期运行、可扩展、带安全策略的代理平台。
- 用操作系统分层图理解 入口 / 协调 / 模型 / 工具 / 持久化 最不容易迷路。

- 与普通 CLI 的本质区别不在「能不能跑命令」，而在是否有 Agent 闭环与工程化安全与扩展。

下一节：[02-four-entries.md](#) —— 四种独立入口如何分流到同一套核心能力。

### 3.1.6 术语迷你表（本篇通用）

| 术语              | 零基础解释                         |
|-----------------|-------------------------------|
| Agent           | 能多步执行任务、会根据结果调整策略的程序化「搭档」     |
| tool / tool_use | 模型发起、运行时真正执行的「能力调用」（读文件、跑命令等） |
| Bridge          | 与 IDE 的跨进程通信子系统，让终端代理与编辑器协同   |
| MCP             | 标准协议层，便于不同客户端接入同一后端能力         |
| Reconciler      | React 将「状态变更」映射到「终端节点更新」的协调器  |

### 3.1.7 安全与能力的「再平衡」示意图

[Mermaid 图表 — 请在在线版本中查看]

读图方式：任何把 能力 拉满的产品，必须在 安全 与 体验 上付利息；Claude Code 选择 显式门控 来付息，而不是假装没有风险。

### 3.1.8 常见追问（FAQ）

Q：既然有 Bridge，为什么还要强终端？

A：终端在 SSH、CI、低图形环境 下仍是 最大公约数；Bridge 是 增强面，不是 替代面。

Q：把 LLM 叫「内核」会不会过度炒作？

A：这是 教学隐喻：强调 调度权在模型；真实计算机内核有严格数学定义，二者不可混为一谈。

Q：我是前端，为什么要读 `tools/` ？

A：因为 Agent 产品的「前端」不仅是 UI，还包括 工具契约 与 错误反馈如何回到模型——这决定交互是否「聪明」。

### 3.1.9 动手练习（不写代码也能做）

1. 打开官方或开源仓库，搜索 `command()` 与 `Tool`，各记下 5 个命中文件名。
2. 用本节的 OS 分层图，给每个文件名 标上所属层。
3. 向一位同事用 2 分钟 讲解「为什么不是普通 CLI」——不讲技术名词，只讲 装修队类比。

### 3.1.10 与后续章节的导航关系

[Mermaid 图表 — 请在在线版本中查看]

完成 3.1~3.4 后，你具备「动态视角」；完成 3.5~3.9 后，你具备「静态地图」；3.10 负责把地图收束为价值观，便于你做 二次开发与内部选型。

## 3.2 四个独立入口：同一颗「大脑」，四条「大门」

### 学习目标

完成本节后，你将能够：

1. 说出 Claude Code 的 CLI、初始化、MCP、SDK 四种入口各自解决什么问题
2. 在脑图中定位「用户从哪进来」与「核心 Agent 循环在哪汇合」
3. 理解为什么「多入口」是操作系统式软件的特征，而不是简单的「多子命令」
4. 为后续阅读 `main.tsx`、Bridge、MCP 传输层文档预留接口认知

### 3.2.1 为什么需要四个入口？

生活类比：同一栋写字楼（核心能力），可以：

- 正门刷卡（CLI：开发者日常终端交互）
- 装修入驻登记（初始化：首次配置、引导、写入设置）
- 外卖与物流通道（MCP：外部客户端通过标准协议「投递任务」）
- 物业 API（SDK：把能力嵌入你自己的 Node/Bun 应用）

若只有一个 `claude chat` 入口，就像办公楼只开一扇门——流量形态和集成方式会被迫扭曲。四入口的本质是 `deployment profile`（部署剖面）分离。

### 3.2.2 四入口一览表

| 入口     | 典型触发方式                      | 主要用户            | 核心价值          |
|--------|-----------------------------|-----------------|---------------|
| CLI    | 终端执行主程序                     | 日常开发者           | 全功能 TUI、交互、调试 |
| 初始化流程  | <code>init</code> / 首次向导    | 新用户 / 新机器       | 降低上手摩擦、写入配置   |
| MCP 模式 | 作为 MCP Server 启动            | IDE / 编排器 / 第三方 | 标准协议互操作       |
| SDK 模式 | 在代码中 <code>import</code> 调用 | 工具作者、自动化        | 可编程、可测试、可嵌入   |

[Mermaid 图表 — 请在在线版本中查看]

### 3.2.3 CLI 入口（`main.tsx` + Commander）

CLI 是最直观的入口：用户在 shell 中启动进程，参数解析 → 子命令分发 → 启动 Ink 应用或一次性任务。

[Mermaid 图表 — 请在在线版本中查看]

关键直觉：`main.tsx` 体量极大（教学材料中常引用约 4684 行 量级），它不仅是 `main()`，更是命令注册表 + 全局副作用初始化的聚集地。精读时宜按「子命令文件」切片，而非通读。

伪代码锚点：

```
// 教学用伪代码：展示「入口 = 注册 + 解析 + 派发」
import { Command } from "commander";

const program = new Command();
program.command("mcp").action(runMcpServerMode);
program.command("init").action(runInitFlow);
// default: 进入交互式 CLI
program.parse(process.argv);
```

### 3.2.4 初始化流程入口

初始化解决的是「这台机器、这个仓库、这位用户」三元组的首次契约：

- 配置目录、认证、模型端点（视版本与产品策略）
- 可能与 **Feature Flags**、遥测开关、向导 UI 联动
- 与 CLI 共享 **services/**（日志、配置、环境探测）

[Mermaid 图表 — 请在在线版本中查看]

类比：不是「买咖啡」（单次交易），而是「办会员卡 + 录入口味偏好」——之后每次 CLI 启动都更顺滑。

### 3.2.5 MCP 服务模式入口

MCP（Model Context Protocol）模式下，Claude Code 作为 **Server**，对外暴露能力；客户端（可能是 Cursor、自建编排器等）通过协议与之对话。

[Mermaid 图表 — 请在在线版本中查看]

价值：把「终端里的代理」变成 **可插拔的后端服务**。同一套 **tools/** 与权限逻辑可被多种宿主复用。

阅读延伸：**src/services/** 下与 MCP 传输、会话相关的模块（具体文件名随版本变化，抓住 **transport / session** 关键词即可）。

### 3.2.6 SDK 编程模式入口

SDK 模式面向「我想在自己的脚本里驱动 Claude Code」：

- 更强的 **可测试性**（相对交互式 TUI）
- 适合 CI、批处理、内部平台集成
- 与 CLI 共享类型与核心服务，但 **UI 层可能极薄或旁路**

[Mermaid 图表 — 请在在线版本中查看]

类比：CLI 是「去餐厅吃饭」；SDK 是「订外卖到你家厨房」——菜还是同一套后厨标准，只是取餐接口不同。

### 3.2.7 四入口如何「汇合」到同一核心？

无论哪条门进入，最终都希望在 同一套：

- 工具定义与执行管线
- 权限与确认策略
- 上下文与系统提示词构建
- 流式响应解析与 `tool_use`

上汇合，否则会出现「MCP 能做的事 CLI 不能做」的 语义分叉，维护成本爆炸。

[Mermaid 图表 — 请在在线版本中查看]

### 3.2.8 常见误解

| 误解                     | 澄清                      |
|------------------------|-------------------------|
| 「MCP 是另一个 Claude Code」 | 它是 同一核心的不同传输与宿主         |
| 「SDK 会绕过权限」            | 策略由 配置与实现 决定；SDK 不等于无门禁 |
| 「init 只是一次性脚本」         | 它与 全局服务 深度耦合，影响后续所有入口体验 |

### 本节小结

- 四入口 = 四种 接入形态，不是四个无关程序。
- Commander 在 CLI 路径上扮演 前台路由；MCP/SDK 是 侧门与货梯。
- 读懂「汇合点」后，阅读 `main.tsx` 时就知道 哪些代码是入口特化，哪些是共享心脏。

上一节：[index.md](#) · 下一节：[03-directory-structure.md](#)

### 3.3 目录结构全景：src/ 地图与统计心智模型

#### 学习目标

完成本节后，你将能够：

- 1. 说出 `src/` 下 `tools`、`commands`、`services`、`bridge`、`components` 等一级目录的职责
- 2. 用一张「树 + 表」快速估算模块规模（文件数会随版本波动，掌握数量级即可）
- 3. 理解「为什么组件目录极大、工具目录中等、bridge 自成子系统」
- 4. 制定自己的源码阅读路线（从入口 → 命令 → 协调器 → 工具）

#### 3.3.1 生活类比：城市功能区

把 `src/` 想象成一座城市：

| 目录                                                             | 类比       | 一句话职责                   |
|----------------------------------------------------------------|----------|-------------------------|
| <code>commands/</code>                                         | 各级政府办事大厅 | 用户显式发起的子命令入口            |
| <code>tools/</code>                                            | 市政工程队    | 真正「动手」的读写、执行、搜索         |
| <code>services/</code>                                         | 水电气网络    | 认证、配置、MCP、遥测等横切能力       |
| <code>coordinator/</code>                                      | 交通指挥中心   | 多代理、任务编排、路由（视版本命名）      |
| <code>assistant/</code> (KAIROS)                               | 专家顾问团    | 高阶助手逻辑、与模型交互的专项         |
| <code>buddy/</code>                                            | 随行助理     | 辅助交互、伙伴式体验相关            |
| <code>components/</code>                                       | 商业地产幕墙   | Ink/React 终端 UI 组件，数量巨大 |
| <code>bridge/</code>                                           | 城际轻轨     | 与 IDE 的跨进程协议与会话         |
| <code>plugins/</code> 、 <code>skills/</code>                   | 加盟店与培训手册 | 扩展与可复用技能                |
| <code>voice/</code> 、 <code>vim/</code> 、 <code>remote/</code> | 专线设施     | 语音、Vim 模式、远程会话          |

#### 3.3.2 规模统计表（教学用近似值）

说明：下列数字来自本教程编写时的典型开源快照与社区梳理，**仅作量级参考**。你本地克隆后应以 `find` / `cloc` 为准。

| 路径（相对 <code>src/</code> ）                    | 近似文件数 | 备注            |
|----------------------------------------------|-------|---------------|
| <code>tools/</code>                          | 约 42  | 每个工具一个或一组文件   |
| <code>commands/</code>                       | 50+   | 与 CLI 子命令大致对应 |
| <code>bridge/</code>                         | 约 31  | IDE 通信子系统集中   |
| <code>components/</code>                     | 约 389 | UI 密度最高       |
| <code>services/</code>                       | 数十    | 横切模块          |
| <code>utils/</code>                          | 数十    | 通用工具函数        |
| <code>coordinator/</code>                    | 若干    | 编排与多代理        |
| <code>assistant/</code>                      | 若干    | KAIROS 相关     |
| <code>plugins/</code> 、 <code>skills/</code> | 若干    | 生态扩展          |

[Mermaid 图表 — 请在在线版本中查看]

### 3.3.3 `src/` 树形全景（精简教学版）

下列树为 **结构教学版**：省略大量文件，仅保留一级与关键二级节点。

```
src/
├─ main.tsx           # 巨型 CLI 入口 (Commander 路由、全局初始化)
├─ commands/         # 50+ 子命令实现
├─ tools/             # ~42 工具 (Agent 的「手」)
├─ services/         # 认证、配置、MCP、遥测、错误处理...
├─ utils/            # 通用函数、字符串、路径、重试等
├─ coordinator/      # 编排、多代理协作相关
├─ assistant/        # KAIROS 等助手子系统
├─ buddy/            # 伙伴式交互
├─ plugins/           # 插件加载与扩展点
├─ skills/           # Skills 机制
├─ voice/            # 语音子系统
├─ vim/              # Vim 模式绑定与逻辑
├─ remote/           # 远程会话相关
├─ bridge/           # ~31 文件：IDE 协议、会话、传输
├─ components/       # ~389 个 React/Ink 组件
└─ ...               # 其他随版本增减的目录
```

[Mermaid 图表 — 请在在线版本中查看]



### 3.3.4 三个「高密度」目录为什么要单独记住？

#### 1) `tools/`：Agent 的「系统调用」

每个工具 ≈ 向模型暴露的一个 `capability`。工具多 ≠ 功能杂，而是 **边界清晰**：读文件、写文件、执行 `bash`、搜索、提交信息...

[Mermaid 图表 — 请在在线版本中查看]

#### 2) `bridge/`：IDE 不是「另一个 UI」，而是 协程式共生

Bridge 子系统处理 消息协议、会话、传输、与 IDE 的能力对齐。没有它，Claude Code 仍是强终端产品；有了它，变成 编辑器内工作流 的一等公民。

#### 3) `components/`：终端 UI 也是 前端工程

389 个组件说明：终端并非「`printf` 日志」，而是 完整组件树 + 布局 + 交互状态机。

### 3.3.5 与其他篇的映射

| 你想搞懂...       | 优先目录                                                  |
|---------------|-------------------------------------------------------|
| 子命令从哪进        | <code>commands/</code> + <code>main.tsx</code>        |
| 模型如何改仓库       | <code>tools/</code>                                   |
| 为什么 IDE 能联动   | <code>bridge/</code>                                  |
| 配置、登录、MCP     | <code>services/</code>                                |
| 多代理怎么跑        | <code>coordinator/</code> + <code>assistant/</code>   |
| Feature Flags | <code>services/</code> （遥测与 <code>flags</code> 相关子模块） |

### 3.3.6 自己动手：刷新统计数据（可选）

在克隆的仓库根目录（示例命令，按你环境调整）：

```
# 仅统计一级目录下直接子文件数（示例）
find src/tools -maxdepth 1 -type f | wc -l
find src/commands -maxdepth 1 -type f | wc -l
find src/bridge -type f | wc -l
find src/components -type f | wc -l
```

类比：别背「精确人口普查」，要背「哪个区是商业区、哪个区是工业区」。

## 本节小结

- `src/` 的骨架 = 入口 (main/commands) + 手 (tools) + 基础设施 (services/utils) + 编排 (coordinator/assistant) + 界面 (components) + IDE (bridge) + 扩展 (plugins/skills)。
- 文件数量级的意义在于分配 阅读耐心：UI 与 bridge 需要「按场景」读，不宜盲扫。
- 下一节我们将沿 数据流 把目录串成动态故事：[04-data-flow.md](#)

上一节：[02-four-entries.md](#)

### 3.3.7 统计口径说明：如何自己数文件？

教学文档中的 42 / 50+ / 31 / 389 等数字，建议你在本地用统一口径复核：

| 统计对象        | 建议命令思路                                             | 说明             |
|-------------|----------------------------------------------------|----------------|
| 某目录下 TS 文件数 | <code>find path -name '*.ts'   wc -l</code>        | 含子目录           |
| 仅顶层命令文件     | <code>maxdepth 1</code>                            | 更接近「子命令入口文件」语义 |
| 组件数         | <code>find components -name '*.tsx'   wc -l</code> | TSX 占比高        |

[Mermaid 图表 — 请在在线版本中查看]

注意：行数与文件数不是一回事——`main.tsx` 可能单文件抵上百个小 util。

### 3.3.8 「横向」阅读法：按场景而非按字母序

| 场景             | 阅读 bundle                                                       |
|----------------|-----------------------------------------------------------------|
| 想改终端配色与布局      | <code>components/</code> + <code>ink/</code> + Design System 文档 |
| 想加一个新工具        | <code>tools/</code> + 权限相关 + 测试                                 |
| 想理解 IDE 如何下发指令 | <code>bridge/</code> + <code>services/</code>                   |
| 想理解多代理         | <code>coordinator/</code> + <code>assistant/</code> + swarm 专章  |

[Mermaid 图表 — 请在在线版本中查看]

### 3.3.9 反模式：目录漫游三小时一无所获

- 从 `utils/` 随机点开：容易陷入 碎片化函数，失去全局观。
- 从 `components/` 首文件顺序通读：组件之间 耦合方向 不明显，建议 从页面/容器组件向下。
- 忽略 `bridge/`：如果你关心 IDE 集成，跳过它会 系统性盲区。

### 3.3.10 与数据流篇的对照记忆

[Mermaid 图表 — 请在在线版本中查看]

- 目录回答：代码住在哪
- 数据流回答：运行时谁先于谁

两篇合起来，才是 可读源码的双眼。

## 3.4 数据流全景：从一句话到工具执行再回环

### 学习目标

完成本节后，你将能够：

1. 画出 用户输入 → 系统提示词 → API → 流式响应 → tool 执行 → 结果回注 → 下一回合 的闭环
2. 解释「流式」为何同时服务 体验（逐字显示）与 协议（提前解析 tool 片段）
3. 区分 模型侧消息 与 本地侧状态（会话、权限、压缩上下文）
4. 将数据流与 `tools/`、`services/`、`coordinator/` 的职责对齐

### 3.4.1 生活类比：急诊分诊台

你把症状告诉分诊护士（用户输入），护士整理成病历摘要（系统提示词 + 上下文），医生思考并开检查单（模型 + `tool_use`），检验科出结果（工具执行），医生再根据结果决定下一步（下一回合）。这不是线性流水线，而是 带反馈的循环。

### 3.4.2 主循环数据流（总图）

[Mermaid 图表 — 请在在线版本中查看]

### 3.4.3 序列图：完整回合（教学版）

下列序列图刻意 略去 具体类名，突出 数据如何流动：

[Mermaid 图表 — 请在在线版本中查看]

### 3.4.4 系统提示词与上下文：「模型看到的宇宙」

| 组成部分  | 典型内容                     | 本地来源（概念）               |
|-------|--------------------------|------------------------|
| 系统提示词 | 行为约束、工具说明格式、安全策略摘要       | 模板 + 动态拼接              |
| 对话历史  | user/assistant/tool 消息序列 | 会话存储                   |
| 仓库元数据 | 路径、Git 状态摘要、项目结构提示       | 启动预取 + 工具              |
| 压缩/摘要 | 过长历史的摘要块                 | context compression 服务 |

[Mermaid 图表 — 请在在线版本中查看]

**关键直觉：**模型 **不直接看到你的硬盘**；它看到的是 **你允许进入提示词的文本与结构化 tool 结果**。这就是安全与隐私的 **第一道闸门**（后续还有权限与确认）。

### 3.4.5 流式响应：为什么不是「等整包 JSON」？

- 1. **体验：**终端 UI 可以 **逐 token 渲染**，降低体感延迟。
- 2. **协议：**部分实现中，工具调用参数可能 **随流到达**，解析器需 **增量状态机**。
- 3. **中断：**用户可取消或限流，流式更易协作 **取消令牌**。

[Mermaid 图表 — 请在在线版本中查看]

### 3.4.6 工具执行管线：从 `tool_use` 到真实副作用

[Mermaid 图表 — 请在在线版本中查看]

**类比：**`tool_use` 是 **法院传票**；权限系统是 **法警**；执行器是 **执行官**——没有法警与执行官，传票不会自动变成现实动作。

### 3.4.7 与 Bridge / MCP / SDK 的差异点（数据流视角）

| 通道      | 用户输入从哪来      | 工具结果展示在哪        |
|---------|--------------|-----------------|
| CLI TUI | 终端组件事件       | Ink 组件树         |
| Bridge  | IDE 消息       | IDE UI + 可能回显终端 |
| MCP     | 客户端 JSON-RPC | 由客户端决定          |
| SDK     | 函数参数         | 由宿主程序记录         |

**共同点：**尽量共享 `Orchestrator` → `API` → `ToolLayer` 主轴，避免重复实现。

[Mermaid 图表 — 请在在线版本中查看]

### 3.4.8 错误与重试：数据流里的「暗河」

生产级 Agent 循环不仅有「成功路径」，还包括：

- API 429 / 5xx：退避重试、切换路由（视实现）
- 工具失败：将 `stderr` / 异常 封装为 `tool_result`，让模型自我纠错
- 用户拒绝权限：将拒绝原因反馈给模型，请求替代方案

伪代码：

```
// 教学用：工具结果无论成功失败，都应结构化回注
async function runTool(call: ToolCall): Promise<ToolResult> {
  try {
    const out = await executor.dispatch(call);
    return { ok: true, content: out };
  } catch (err) {
    return { ok: false, error: serializeError(err) };
  }
}
```

### 3.4.9 与压缩 / 记忆的关系（点到为止）

当历史过长时，上下文压缩或 记忆提取会在 CTX 阶段改写进入模型的内容：

[Mermaid 图表 — 请在在线版本中查看]

详见其他篇「context compression / memory」专章；本篇只需记住：数据流在进 API 前可能被「减肥」。

## 本节小结

- Agent 的本质是 带工具反馈的循环，不是 HTTP 一次请求。
- 流式同时服务 UI 与解析；工具层是副作用边界。
- 多入口只替换 输入输出适配层，不应替换 脊柱数据流。

上一节：[03-directory-structure.md](#) · 下一节：[05-tech-stack.md](#)

## 3.5 技术栈深度解析：Bun、TypeScript、自研 Ink、Commander

### 学习目标

完成本节后，你将能够：

- 1. 解释为何 Claude Code 选择 **Bun** 作为运行时，以及它与 Node.js 的取舍
- 2. 说明 **TypeScript** 在大型 Agent 系统中的工程价值（不仅是「类型好看」）
- 3. 描述 **自研 Ink + 纯 TS Yoga** 的技术动机：布局一致性、可控性、性能
- 4. 定位 **Commander.js** 在 `main.tsx` 中的角色：声明式子命令路由

#### 3.5.1 总览：技术栈一张表

| 层级     | 选型                      | 一句话                               |
|--------|-------------------------|-----------------------------------|
| 运行时    | Bun                     | 极速启动、原生 TS、工具链一体化                 |
| 语言     | TypeScript              | 大规模协作、工具契约与 schema 友好             |
| 终端 UI  | 自研 React Ink + Yoga(TS) | 非标准 npm <code>ink</code> 包；布局引擎可控 |
| CLI 路由 | Commander.js            | 子命令注册与参数解析                        |
| React  | 与自定义 reconciler 配合      | 终端 Fiber 树 + 增量更新                 |

[Mermaid 图表 — 请在在线版本中查看]

### 3.5.2 Bun vs Node.js：不是「谁更好」，而是「谁更贴脸」

| 维度    | Node.js              | Bun               |
|-------|----------------------|-------------------|
| 启动延迟  | 成熟但偏重                | 通常更低，适合 CLI 频繁启动  |
| TS 开箱 | 需 transpile 或 loader | 原生 TS 路径更顺        |
| 包管理   | npm/pnpm/yarn 生态     | 内置安装器（团队策略决定是否混用） |
| 生态兼容  | 最广                   | 高兼容但偶有边缘包差异       |
| 调试心智  | 资料最多                 | 增长快，需跟版本          |

生活类比：Node 像 **国际机场**——航班最全；Bun 像 **城市快线**——去市中心（交互式 CLI）更快。Agent 产品对 **冷启动 + 反馈延迟** 敏感，Bun 的画像更匹配。

[Mermaid 图表 — 请在在线版本中查看]

批判性思维：若你的插件生态强依赖 仅 Node 测试过的 **native addon**，需评估 Bun 兼容性——这是所有新运行时的共同问题。

### 3.5.3 TypeScript：为什么 Agent 系统「几乎必然」TS 化？

- 工具契约：工具入参/出参与 **JSON schema**、RPC 消息高度同构；TS 类型 + zod 等库减少线上才发现的字段错误。
- 大规模协作：`components/` 数百文件，无类型导航成本指数上升。
- 重构安全：权限模型、消息协议一改，**类型报错**是最好的回归测试之一。

[Mermaid 图表 — 请在在线版本中查看]

类比：TS 像 **建筑结构工程师签字**——不是保证永不塌，而是 **明显违规**会在图纸上标红。

### 3.5.4 自研 Ink：为什么不用标准 npm `ink`？

公开文档与社区源码阅读中常见结论：Claude Code 使用深度定制 Ink，原因通常包括：



| 动机   | 说明                              |
|------|---------------------------------|
| 布局控制 | 终端复杂面板需要更细的 测量与重排 策略            |
| 性能   | 大组件树下减少 reconcile 与输出抖动         |
| 交互   | 与 Vim 模式、滚动、diff 视图等深度耦合        |
| 一致性  | 与内部 Design System 对齐（见终端 UI 专章） |

### Yoga: Flexbox 在终端里「算几何」

浏览器有布局引擎；终端只有 行列网格。Yoga（Meta 开源）负责 Flexbox 约束求解。Claude Code 侧为 纯 TypeScript 移植/集成，避免 native 绑定带来的 跨平台与构建痛点。

[Mermaid 图表 — 请在在线版本中查看]

生活类比：React 是 剧本；Fiber 是 导演分镜；Yoga 是 舞台布景测量；ANSI 输出是 演员站位。

### 关键源码片段（教学还原）

```
// 概念来自 ink/components/Box.tsx: flex 属性进入 Yoga
function Box({ flexDirection = "row", children, ...style }: Props) {
  return (
    <ink-box
      style={{
        flexDirection,
        ...style,
      }}
    >
      {children}
    </ink-box>
  );
}
```

`<ink-box>` 说明：宿主元素不是 DOM，而是 Ink 自定义 内部类型，由 reconciler 创建对应 终端节点。

### 3.5.5 Commander.js: `main.tsx` 的「交通总枢纽」

`main.tsx`（约 4684 行 量级）承载：

- 全局 `import` 副作用顺序（需谨慎）
- `program` 实例上挂载大量 `.command()`

- option 与 hook（版本检查、telemetry、debug）

```
// 教学用最小示例：展示 Commander 的心智模型
import { Command } from "commander";

const program = new Command();
program
  .name("claude-code")
  .option("-d, --debug", "verbose logging");

program
  .command("doctor")
  .description("自检环境")
  .action(async () => {
    // -> src/commands/... 类似结构
  });

await program.parseAsync(process.argv);
```

[Mermaid 图表 — 请在在线版本中查看]

阅读策略：不要线性阅读 4000 行；用「搜 `.command(`」建立索引表，再跳读对应 `src/commands/*.ts`。

### 3.5.6 技术栈组合带来的「系统性格」

[Mermaid 图表 — 请在在线版本中查看]

- 快：Bun + 流式 API + 增量渲染 → 体感敏捷
- 重 UI：Ink 组件爆炸 → 前端工程化
- 强 CLI：Commander → 可发现性（`--help` 文化）

### 3.5.7 常见面试式追问（自测）

| 问题              | 简答方向                     |
|-----------------|--------------------------|
| 为什么不用 Electron？ | 终端优先、远程 SSH、低图形依赖        |
| Yoga 解决什么？      | 布局计算；不是语法高亮本身            |
| 自研 Ink 的最大成本？   | 与上游同步、维护 reconciler 边缘行为 |

### 本节小结

- Bun + TS 服务 启动速度与工程规模；Ink + Yoga 服务 终端复杂 UI；Commander 服务 CLI 可维护路由。
- 把这三者连起来，就理解为何它不是「几十行的 python 脚本代理」。

上一节：[04-data-flow.md](#) · 下一节：[06-comparison.md](#)

## 3.6 架构对比：Claude Code、Cursor、Copilot Agent、Windsurf

### 学习目标

完成本节后，你将能够：

1. 用一张表对比 执行环境、安全模型、扩展方式、成本结构 四个维度
2. 用三个生活类比记住：坐旁边（Cursor）、远程虚拟机（Copilot Agent）、本机代理 + 多层安检（Claude Code）
3. 理解「没有绝对最好，只有场景最匹配」的选型逻辑
4. 将 Windsurf 放入「编辑器原生 Agent」谱系中定位

#### 3.6.1 三维隐喻：你在哪干活？

| 工具                   | 一句话隐喻               | 你的电脑角色           |
|----------------------|---------------------|------------------|
| Cursor               | 专家坐你旁边看你操作          | 主战场；AI 深嵌编辑器     |
| GitHub Copilot Agent | 专家在**另一间机房（VM）**折腾  | 你的机器更多是「遥控台」     |
| Claude Code          | 专家直接用你的电脑，但要过 9 层安检 | 主战场；终端/Bridge 驱动 |
| Windsurf             | 编辑器内的 Cascade 流     | 主战场；产品化工作流       |

[Mermaid 图表 — 请在在线版本中查看]

### 3.6.2 执行方式对比

| 维度         | Claude Code              | Cursor          | Copilot Agent      | Windsurf       |
|------------|--------------------------|-----------------|--------------------|----------------|
| 主交互面       | 终端 TUI + (可选) IDE Bridge | IDE 内嵌面板        | 以 Agent 会话为中心 (托管) | IDE 内嵌 Cascade |
| 代码访问路径     | 本机工作区直接读写                | 本机工作区 (编辑器 API) | 远程克隆/沙箱 (产品演进中)    | 本机工作区          |
| 适合 SSH 远程机 | 强 (终端原生)                 | 中 (依赖远程开发链路)    | 强 (不依赖你本机编译环境)     | 中              |
| 批处理 / 脚本化  | 强 (SDK、CLI)              | 中               | 中                  | 弱到中            |

[Mermaid 图表 — 请在在线版本中查看]

### 3.6.3 安全模型对比 (重点)

Claude Code 常被描述为「直接用你电脑，但多层安检」：

- **权限门控**：敏感工具需显式允许或用户确认（策略可配置）
- **可见性**：动作发生在你的用户权限下——与「方便」同源的风险需靠 UX 与策略缓解
- **审计**：终端日志、Git 状态、部分遥测（视版本与设置）

Cursor：

- 深度集成编辑器与 LSP；**凭据与文件访问**通常跟随编辑器工作区信任模型
- 用户感知为「AI 在 IDE 里改」——心理距离近，但本质仍是本机副作用

Copilot Agent：

- **隔离红利**：远程环境可限制对本地机的直接触碰
- **数据出境与代码上传**：企业合规维度需单独评估

Windsurf：

- 与本机工作区交互；安全故事与 **IDE 信任边界 + 产品策略** 绑定

| 安全维度    | Claude Code | Cursor    | Copilot Agent | Windsurf |
|---------|-------------|-----------|---------------|----------|
| 默认信任面   | 本机用户权限      | 本机 + 插件   | 托管环境边界        | 本机       |
| 隔离强度    | 中（进程内 + 策略） | 中         | 高（相对本地）       | 中        |
| 用户可感知确认 | 强（工具批准 UX）  | 中到强       | 中             | 中到强      |
| 企业治理    | 依赖本地策略与网络   | 依赖 IDE 策略 | 易推「不上本地」叙事    | 依赖供应商    |

[Mermaid 图表 — 请在在线版本中查看]

说明：上图中「9 层」为 **教学概括**，真实实现是 **矩阵式策略**，并非严格串行 9 个函数；但利于零基础建立「**不是裸 exec**」的直觉。

### 3.6.4 扩展性对比

| 扩展机制       | Claude Code                          | Cursor       | Copilot Agent  | Windsurf  |
|------------|--------------------------------------|--------------|----------------|-----------|
| MCP        | 原生服务端/客户端生态位                         | 支持 MCP（宿主侧）  | 视产品与版本         | 视产品与版本    |
| 插件 / Rules | Plugins、Skills、 <code>.md</code> 技能包 | Rules、MCP、扩展 | 生态在 GitHub 侧演进 | 规则/工作流产品化 |
| 可编程嵌入      | SDK 模式强                              | 弱（偏配置）       | 中              | 弱到中       |
| 开源可控       | 可读源码（许可证以仓库为准）                       | 闭源为主         | 闭源为主           | 闭源为主      |

[Mermaid 图表 — 请在在线版本中查看]

上图仅为 **定性示意**；精确产品能力请以官方文档为准。

### 3.6.5 成本与计费心智

计费模型随时间与套餐变化极大，本节只给 **架构视角**：

| 角度   | Claude Code          | Cursor | Copilot Agent   | Windsurf |
|------|----------------------|--------|-----------------|----------|
| 算力在哪 | 调用模型 API（你或组织的密钥/套餐） | 供应商打包  | 常与 Copilot 订阅绑定 | 供应商打包    |
| 本机成本 | CLI 进程 + 工具执行        | IDE 常驻 | 本地较轻            | IDE 常驻   |

类比：Claude Code 像 **自己加油的车**（API/用量可见）；部分 IDE Agent 像 **共享汽车会员**（套餐打包，单价不透明）。

### 3.6.6 选型决策树（零基础友好）

[Mermaid 图表 — 请在在线版本中查看]

### 3.6.7 综合对比总表

| 项目   | Claude Code          | Cursor         | GitHub Copilot Agent | Windsurf     |
|------|----------------------|----------------|----------------------|--------------|
| 执行拓扑 | 本机终端/Bridge          | 本机 IDE         | 托管为主                 | 本机 IDE       |
| 交互范式 | 对话式 Agent + 工具       | 编辑中辅助 + Agent  | Agent 任务流            | Cascade 工作流  |
| 安全直觉 | 本机权限 + 显式门控          | 本机 IDE 信任边界    | 远程沙箱隔离               | 本机 IDE 信任边界  |
| 扩展   | MCP、Plugin、Skill、SDK | MCP、Rules      | GitHub 生态            | 产品内工作流       |
| 开源   | 可读源码为主               | 闭源             | 闭源                   | 闭源           |
| 典型人群 | 多仓库、脚本化、终端派          | 前端/全栈日常 IDE 用户 | GitHub 中心化团队         | 追求 IDE 一体化体验 |

### 本节小结

- 执行环境决定风险轮廓：**本机直连** vs **远程托管** 没有绝对优劣。
- Claude Code 的差异点 = **终端原生** + **可编程 SDK** + **MCP** + **显式工具权限链**。
- 对比表应随产品发布更新；架构维度比「功能 checkbox」更耐久。

上一节: [05-tech-stack.md](#) · 下一节: [07-startup.md](#)

### 3.6.8 Windsurf 补充: 「编辑器原生 Agent」谱系

Windsurf 通常被放在 深度 IDE 集成 象限: 强调 工作流一体化 (编辑、索引、Agent 面板、文档跳转)。与 Claude Code 的 终端优先 形成互补:

| 维度     | Windsurf     | Claude Code      |
|--------|--------------|------------------|
| 主战场    | 编辑器内         | 终端 + (可选) Bridge |
| 心智入口   | Cascade / 面板 | CLI / SDK        |
| 脚本化自动化 | 相对弱          | SDK 强            |

[Mermaid 图表 — 请在在线版本中查看]

### 3.6.9 企业场景下的四条「治理问题」

| 治理问题     | Claude Code | Cursor    | Copilot Agent | Windsurf |
|----------|-------------|-----------|---------------|----------|
| 代码是否离开本机 | 取决于 API 与配置 | 取决于供应商与模式 | 常见上传至托管侧      | 取决于供应商   |
| 审计日志     | CLI + 服务可组合 | IDE 生态    | 平台侧为主         | 平台侧为主    |
| 最小权限     | 工具级策略       | 工作区信任     | 沙箱边界          | 工作区信任    |
| 供应链 (开源) | 可读源码为主      | 闭源        | 闭源            | 闭源       |

表格为 架构向提问清单, 不是合规结论; 真实采购需法务与信息安全评估。

### 3.6.10 迁移成本: 团队已经在用 X, 怎么办?

[Mermaid 图表 — 请在在线版本中查看]

建议: 先做 2 周试点, 用同一批任务 (重构、修 bug、写测试) 记录 完成时间、返工次数、事故数, 再谈「信仰」。

### 3.6.11 第二幅对比心智图：「谁握着鼠标？」

[Mermaid 图表 — 请在在线版本中查看]

读图：不是「谁更 AI」，而是 交互循环绑定在何种系统调用接口上。

### 3.6.12 本篇自检清单

- [ ] 我能用 1 句话区分 Cursor 与 Claude Code 的 执行面
- [ ] 我能解释 托管 Agent 的安全 红利与代价
- [ ] 我知道 Windsurf 在表格中处于 IDE 原生 位置，而非终端工具

完成以上三点，对比篇即可 毕业。

## 3.7 启动流程详解：从进程诞生到 Agent 主循环

### 学习目标

完成本节后，你将能够：

1. 按时间顺序复述 Commander 解析 → 环境预取 → 核心初始化 → 主循环 四阶段
2. 解释为何 CPU/内存/OS/Git 等信息适合 并行预取（降低首屏与首 token 前阻塞）
3. 在时序图中标出 可与 UI 渲染重叠 的阶段
4. 对接 08-main-entry.md 中 main.tsx 的分段阅读策略

### 3.7.1 生活类比：餐厅开店流程

早上开店时，优秀店长会 同时 做几件事：检查水电表（系统信息）、确认今日进货单（Git/项目状态）、打开收银机（配置加载）、布置门面（首屏 UI）。串行每件事会让第一位顾客等太久；并行 + 关键路径合并才是体验。



### 3.7.2 启动阶段总览

| 阶段           | 主要工作                                  | 用户可感知                  |
|--------------|---------------------------------------|------------------------|
| P0 进程启动      | Bun 解析入口、装载 <code>main.tsx</code> 依赖图 | 极短延迟                   |
| P1 Commander | <code>argv</code> 解析、命中子命令、全局 option  | <code>--help</code> 秒回 |
| P2 并行预取      | CPU/内存/OS、Git 状态、环境探测                 | 缩短「可输入」时间              |
| P3 核心装配      | services 挂载、日志 sink、特性开关、认证状态         | 错误或 banner             |
| P4 Agent/UI  | Ink 根组件挂载、会话恢复、进入主循环                  | 交互就绪                   |

[Mermaid 图表 — 请在在线版本中查看]

### 3.7.3 时序图：启动路径（默认交互模式）

[Mermaid 图表 — 请在在线版本中查看]

### 3.7.4 为何「并行预取」值得单独强调？

- Git 状态（分支、脏文件、最近提交）常被注入 系统提示词或上下文卡片——早拿到早装配。
- OS/CPU/内存 可能用于 遥测分层、特性开关、错误诊断（`doctor` 类命令也复用）。
- 预取失败不应阻塞启动：应有 降级（空值 + 稍后重试），否则 SSH 弱网环境体验崩。

[Mermaid 图表 — 请在在线版本中查看]

伪代码：

```
// 教学用：表达「并行 + 容错」意图
async function prefetchBootstrapContext(cwd: string) {
  const [sys, git, env] = await Promise.all([
    safe(() => readSystemSummary()),
    safe(() => readGitSummary(cwd)),
    safe(() => probeEnvironment()),
  ]);
  return { sys, git, env, partialFailures: ... };
}
```

### 3.7.5 Commander 子命令的「早退」路径

并非所有启动都进入 Agent 主循环：

| 子命令类型                                        | 典型行为                     | 是否进入 TUI |
|----------------------------------------------|--------------------------|----------|
| <code>--version</code> / <code>--help</code> | 打印后 <code>exit(0)</code> | 否        |
| <code>doctor</code>                          | 自检后退出                    | 否或极简 UI  |
| <code>mcp</code>                             | 进入服务模式                   | 否（另一主循环） |
| 默认                                           | 会话式 CLI                  | 是        |

[Mermaid 图表 — 请在在线版本中查看]

### 3.7.6 初始化顺序：小心「循环依赖」

大型 `main.tsx` 常见风险：`import` 副作用顺序导致难以调试的循环依赖。工程上常见对策：

- `analytics` / 日志 模块 延迟 `attach sink`（见 `services` 专章）
- 重型单例 延迟到 已知子命令路径 再加载
- 动态 `import` 切分 MCP/SDK 路径

[Mermaid 图表 — 请在在线版本中查看]

### 3.7.7 与「数据流篇」的衔接点

启动结束时，`bootstrap context` 会流入 `ContextBuilder`：

[Mermaid 图表 — 请在在线版本中查看]

### 3.7.8 故障排查清单（零基础）

| 现象       | 可能阶段         | 排查方向           |
|----------|--------------|----------------|
| 卡在首屏很久   | P2 预取        | Git 超大仓库、网络卷延迟 |
| 一启动就报错退出 | P3 服务        | 认证、配置路径、权限     |
| UI 花屏/闪烁 | P4 Ink       | 终端兼容、TERM、字体宽度 |
| 子命令找不到   | P1 Commander | argv、别名、安装版本   |

### 3.7.9 第二幅时序图：强调「预取与 UI 重叠」

在真实实现中，**首帧渲染** 有时可与 **部分预取** 重叠（取决于版本优化）：

[Mermaid 图表 — 请在在线版本中查看]

是否重叠属于 **实现细节**；本节目的是让你知道：**启动优化**常发生在这个交叉区。

### 本节小结

- 启动 = 路由（Commander） + 情报（并行预取） + 装配（services） + 呈现与循环（Ink + Agent）。
- **并行预取**是操作系统式软件常见的 **hide latency** 技巧。
- 阅读 `main.tsx` 时，用 **子命令** 与 **import 深度** 两轴切片，避免淹没在单行文件中。

上一节：[06-comparison.md](#) · 下一节：[08-main-entry.md](#)

## 3.8 `main.tsx` 入口分析：4684 行如何拆解？

### 学习目标

完成本节后，你将能够：

1. 解释为什么 `main.tsx` 会膨胀到 数千行（注册表 + 全局初始化 + 模式分支）
2. 给出 推荐的模块阅读顺序（从 Commander 索引到各 `commands/` 文件）
3. 识别 五类代码块：import 区、program 定义、全局 hook、子命令注册、默认入口
4. 建立 调用链心智图：从 `parse` 到具体 `action` 函数

### 3.8.1 生活类比：总机接线室

`main.tsx` 像 公司总机：不处理每一笔业务细节，但 所有电话都得先经过它，它决定 转接到哪个分机（子命令）。若把分机逻辑也全写在总机室里，房间就会 4684 行 这么拥挤——因此源码里常有「注册在此，实现在 `commands/`」的模式。

### 3.8.2 结构分解（五段论）

行号为 教学示意；你本地仓库请以搜索与标签为准。

| 区块           | 典型内容                                             | 阅读优先级  |
|--------------|--------------------------------------------------|--------|
| A：顶部 import  | 副作用敏感；全局面单例                                      | ★ 最后精读 |
| B：program 基座 | <code>name/description/version</code> 、全局 option | ★★★★   |
| C：中间件式 hook  | 预检、telemetry、debug、环境变量                          | ★★★★   |
| D：子命令注册      | <code>.command(...).action(...)</code> 高密度区      | ★★★★★★ |
| E：parse 与兜底  | <code>parseAsync</code> 、默认行为、未命中处理              | ★★★★★  |

[Mermaid 图表 — 请在在线版本中查看]

### 3.8.3 调用链：从进程到业务 handler

[Mermaid 图表 — 请在在线版本中查看]

### 3.8.4 关键源码片段（高度简化的教学模型）

下列代码 不是 逐字拷贝开源仓库，而是展示 结构骨架，便于你对照真源码：

```
// --- 区块 B: program 基座 (示意) ---
import { Command } from "commander";

const program = new Command();
program
  .name("claude")
  .description("Agentic coding assistant")
  .version(VERSION_FROM_BUILD);

program.option("--debug", "enable verbose diagnostics");
```

```
// --- 区块 D: 子命令注册 (示意) ---
program
  .command("doctor")
  .description("Check local environment")
  .action(async () => {
    const { runDoctor } = await import("./commands/doctor");
    await runDoctor();
  });

program
  .command("mcp")
  .description("Run MCP server mode")
  .action(async () => {
    const { runMcpServer } = await import("./commands/mcp");
    await runMcpServer();
  });
```

```
// --- 区块 E: 解析 (示意) ---
await program.parseAsync(process.argv);
```

阅读技巧：在仓库内搜索 `.command(` 的命中列表，生成你自己的「子命令目录」CSV，比死记行号更有效。

### 3.8.5 模块初始化顺序：为什么 import 区「看起来很乱」？

[Mermaid 图表 — 请在在线版本中查看]

原则：越靠前越像操作系统内核初始化——动一行可能影响全局。新手应 避免「顺手改 import 顺序」。

### 3.8.6 `main.tsx` vs `commands/`：职责边界

| 文件                         | 应该薄还是厚？ | 原因                            |
|----------------------------|---------|-------------------------------|
| <code>main.tsx</code>      | 中等偏厚    | 注册表天然膨胀；但仍应 <b>避免</b> 把复杂业务写满 |
| <code>commands/*.ts</code> | 业务主场    | 可测、可拆、可懒加载                    |

[Mermaid 图表 — 请在在线版本中查看]

### 3.8.7 4684 行的「反模式信号」与工程现实

不是缺点，而是权衡：

- **优点**：所有入口 **一眼可搜**；发布流水线简单（单入口）
- **风险**：合并冲突频繁、新人心理门槛高
- **缓解**：动态 `import`、把大块逻辑迁到 `commands/`、`bootstrap/` 类目录（若版本已拆分）

### 3.8.8 实战阅读路线（90 分钟版）

1. 15 min：从 `package.json` 找到实际 `bin` 指向的入口文件（通常为 `main.tsx` 或编译产物）。
2. 20 min：全文搜索 `.command(`，把子命令名抄到笔记。
3. 25 min：选一个 **非默认** 子命令（如 `doctor`）顺藤摸到 `commands/`，走通 `import` 链。
4. 20 min：回到默认路径，找 `Ink render(` 或等价调用，理解 **UI 挂载点**。
5. 10 min：对照 `07-startup.md` 把 **预取** 相关函数在 `main` 中 **打书签**。

[Mermaid 图表 — 请在在线版本中查看]

### 3.8.9 与四入口的映射（交叉引用）

| 模式   | 在 <code>main.tsx</code> 中的体现                                   |
|------|----------------------------------------------------------------|
| CLI  | 默认 <code>action</code> + <code>Ink</code>                      |
| Init | <code>init</code> 子命令注册                                        |
| MCP  | <code>mcp</code> 子命令或 <code>flag</code> 分支                     |
| SDK  | 可能 独立入口文件；或 <code>main</code> 中极薄 <code>re-export</code> （视版本） |

### 3.8.10 调试建议（零基础）

```
# 示意：打印 argv 与版本（勿在生产脚本滥用）
claude --help
claude doctor
```

在源码中临时加日志时，优先使用 已有 `debug flag` 路径，而不是散落 `console.log` ——否则多进程/Bridge 场景下 日志交错 难读。

### 本节小结

- `main.tsx` = CLI 操作系统 `bootloader` + 子命令注册表。
- 调用链终点在 `commands/`，能力链终点在 `tools/` + `services/`。
- 学会 搜索驱动阅读，比 线性通读 4684 行 更符合人类认知。

上一节：[07-startup.md](#) · 下一节：[09-dependencies.md](#)

## 3.9 模块依赖关系：从「依赖图」理解演进边界

### 学习目标

完成本节后，你将能够：

- 读出 核心模块依赖图 的主干方向（UI → 编排 → 服务 → 工具）
- 解释为何要 隔离 `analytics`、配置、错误处理 以避免循环依赖

3. 将 60+ npm 依赖 按职能 分桶 (CLI、React、网络、协议、工具库)
4. 识别 「内部包」 (如定制 `ink/`) 与外部 npm 的边界

### 3.9.1 生活类比：城市供水管网

应用模块像建筑，依赖像水管。循环依赖等于 A 楼用水要经过 B 楼，B 又要经过 A——一旦爆管，全城停水。工程上通过 水厂 (services)、分区阀门 (接口层) 把网拓扑变成 有向无环图 (DAG) 倾向。

### 3.9.2 核心内部模块依赖图 (概念 DAG)

[Mermaid 图表 — 请在在线版本中查看]

读图规则：自上而下通常更安全；底向上引用呈现层 = 红色警报 (多数情况下应避免)。

### 3.9.3 第二幅图：Agent 回合内的「动态依赖」

静态 import 图之外，运行时还有 每轮对话 的数据依赖：

[Mermaid 图表 — 请在在线版本中查看]

### 3.9.4 npm 依赖分桶 (60+ 包的读法)

具体包名与版本以仓库 `package.json` / `bun.lock` 为准。下表为 教学分桶，便于建立心智模型。



| 分桶         | 代表依赖类型                                                                | 在 Claude Code 中的角色 |
|------------|-----------------------------------------------------------------------|--------------------|
| CLI 与进程    | <code>commander</code> 、 <code>which</code> 、 <code>execa</code> 类    | 参数路由、子进程           |
| React 生态   | <code>react</code> 、 <code>react-reconciler</code> （若显式）              | 终端组件               |
| 终端与样式      | <code>chalk</code> 、 <code>ansi-*</code> 、 <code>wrap-ansi</code> 等   | 输出与测量              |
| 网络与 API    | <code>fetch</code> 封装、 <code>ws</code> 等                              | 模型与远程服务            |
| 协议与数据      | <code>zod</code> 、 <code>protobuf</code> / <code>msgpack</code> 类（若有） | 校验与序列化             |
| Git 与 diff | <code>simple-git</code> 等                                             | 仓库操作与展示            |
| 测试与构建      | <code>vitest</code> / <code>eslint</code> 等 devDeps                   | 质量保障               |

[Mermaid 图表 — 请在在线版本中查看]

学习策略：不要背 60 个包名；遇到报错再 定位到桶，查文档效率更高。

### 3.9.5 关键内部包： `ink/` 不是 `node_modules`

| 类别        | 位置                               | 备注                            |
|-----------|----------------------------------|-------------------------------|
| 定制终端渲染    | 仓库内 <code>ink/</code> （示例路径，随版本） | 与 npm <code>ink</code> 不等价    |
| 布局        | Yoga TS 相关绑定                     | 与 <code>components/</code> 协同 |
| Bridge 协议 | <code>bridge/</code> TS 模块       | IDE 消息类型密集                    |

[Mermaid 图表 — 请在在线版本中查看]

### 3.9.6 循环依赖治理：案例式理解

#### analytics 零依赖模式（概念）

```
// 设计意图（摘自架构文档常见写法）
// analytics 模块不 import 其他业务模块
// 通过 attachSink 在启动晚期接线

let sink: Sink | null = null;

export function attachAnalyticsSink(s: Sink) {
  sink = s;
  flushQueue();
}

export function logEvent(name: string, meta: object) {
  if (!sink) enqueue(name, meta);
  else sink.send(name, meta);
}
```

[Mermaid 图表 — 请在在线版本中查看]

类比：单向门——谁都能投递事件，但 analytics 不回调业务。

### 3.9.7 tools/ 对 services/ 的依赖原则

| 允许      | 谨慎       | 避免              |
|---------|----------|-----------------|
| 读配置、日志  | 重量级单例    | 工具 import UI 组件 |
| 调用网络客户端 | 隐式全局可变状态 | 工具间横向 import 成网 |

### 3.9.8 bridge/ 依赖特点

- 强依赖 services/ （会话、配置、日志）
- 弱耦合 components/ （消息驱动 UI 更新，而非直接 new 组件）
- 与 远程模式、IDE 能力协商 交叉

[Mermaid 图表 — 请在在线版本中查看]

### 3.9.9 依赖健康度自检清单

| 问题     | 工具建议                                                                                    |
|--------|-----------------------------------------------------------------------------------------|
| 谁引用了谁? | <code>madge</code> 、 <code>skott</code> 、或 <code>ripgrep</code> <code>from \"../</code> |
| 循环依赖?  | <code>madge --circular</code>                                                           |
| 包体积异常? | <code>bun pm</code> / bundle 分析 (若适用)                                                   |
| 重复依赖?  | <code>npm ls</code> / lockfile 审查                                                       |

### 3.9.10 与「技术栈篇」的交叉对照

| 栈层  | 主要外部依赖         | 主要内部模块                                         |
|-----|----------------|------------------------------------------------|
| CLI | Commander      | <code>main.tsx</code> 、 <code>commands/</code> |
| UI  | React + 内部 ink | <code>components/</code> 、 <code>ink/</code>   |
| 模型  | HTTP 客户端库      | <code>services/</code> 内客户端封装                  |
| 工具  | 各类 util        | <code>tools/</code>                            |

### 3.9.11 小结图：依赖 = 约束未来的力

[Mermaid 图表 — 请在在线版本中查看]

### 本节小结

- 内部依赖图以 `services/tools/coordinator` 为枢纽；外部 npm 以 分桶理解即可。
- 防循环是大型 TS 项目的隐形架构；analytics 模式是经典教科书案例。
- 真仓库请始终以 `lockfile + madge` 为准，文档只提供 导航语义。

上一节: [08-main-entry.md](#) · 下一节: [10-philosophy.md](#)

## 3.10 设计哲学总结：Less scaffolding, more model

### 学习目标

完成本节后，你将能够：

1. 用一句话解释「Less scaffolding, more model」在 Agent 架构中的含义与边界
2. 说明 事件驱动 与 流式处理 如何共同塑造终端体验
3. 将 权限门控 视为产品设计而非单纯安全补丁
4. 理解 Feature Flags 在「渐进发布」中的系统位置

### 3.10.1 核心理念：少脚手架，多模型

**Scaffolding** 在此泛指：硬编码业务流程、庞大 if-else 决策树、过度手写的任务状态机。**Model** 指：让 LLM 在 工具边界清晰 的前提下承担 规划、纠错、解释。

| 取向   | 优点       | 风险              |
|------|----------|-----------------|
| 多脚手架 | 行为可预测、易测 | 维护成本随场景指数爆炸     |
| 多模型  | 泛化强、迭代快  | 需 工具/权限/验证 三角约束 |

[Mermaid 图表 — 请在在线版本中查看]

生活类比：模型像 经验丰富的船长；工具像 舵轮与引擎；权限像 海事法规。你要给船长 决策权，但不能给他 随意凿船底的能力。

### 3.10.2 事件驱动：UI、Bridge、工具结果的「多源时钟」

Claude Code 不是「请求-响应一次结束」，而是 事件流：

- 用户按键、粘贴、命令面板
- IDE Bridge 消息
- 模型 token 流
- 子进程 stdout 流
- 工具完成/失败回调

[Mermaid 图表 — 请在在线版本中查看]

第二幅图：从「轮询」到「推送」

[Mermaid 图表 — 请在在线版本中查看]

要点：事件驱动让 多源异步 共存；缺点是 调试复杂——需要良好日志与可重复会话导出。

3.10.3 流式处理：体验与协议的双宿主

- 1. 体验：终端 UI 像聊天软件一样 逐字出现。
- 2. 协议：工具调用可能以 增量 JSON 形式到达，需要 流式解析器 与 强校验（ zod 等）。

| 非流式    | 流式    |
|--------|-------|
| 等待整包响应 | 边到边展示 |
| 解析简单   | 需要状态机 |
| 延迟体感高  | 延迟体感低 |

[Mermaid 图表 — 请在在线版本中查看]

3.10.4 权限门控：产品哲学，不是「吓唬用户」

权限系统在架构上实现三类目标：

- 1. 安全：限制高危操作（删除、远程访问、凭据）
- 2. 可解释：用户知道「AI 将要做什么」
- 3. 可配置：团队策略与个人偏好共存

[Mermaid 图表 — 请在在线版本中查看]

与「9 层安检」叙事的关系：安检不是羞辱用户，而是 把不可逆动作变成可撤销决策。

### 3.10.5 Feature Flags：渐进发布与 A/B

Feature Flags 让团队 在同一二进制 上：

- 灰度新工具、新 UI
- 快速关闭事故功能
- 分层遥测（与 GrowthBook 等集成，见 services 专章）

[Mermaid 图表 — 请在在线版本中查看]

生活类比：Flags 像 剧院彩排时的活动挡板——观众席（全量用户）还没看到新布景，但演员（部分用户）可以先走台。

### 3.10.6 「Less scaffolding」的边界：什么时候该多写代码？

| 场景                      | 倾向                |
|-------------------------|-------------------|
| 确定性协议（序列化、CRC、Git 对象格式） | 多代码、少模型           |
| 模糊需求探索（产品原型、重构建议）       | 多模型、强工具           |
| 合规审计                    | 多日志、多策略、模型辅助但不可越权 |

[Mermaid 图表 — 请在在线版本中查看]

### 3.10.7 遥测与隐私：哲学的一体两面

- 事件驱动 让系统可观测；可观测 又需要 最小化采集原则。
- 常见工程约束：禁止字符串型敏感元数据、采样、可关闭。

（实现细节见 `services/analytics` 相关文档。）

### 3.10.8 与操作系统隐喻的收束

回到 `index.md` 的隐喻：若 LLM 是 内核，则：

- **事件 + 流式** = 中断与 DMA（直观类比，非严格学术）
- **权限** = capability
- **Feature Flags** = 内核参数动态调优

[Mermaid 图表 — 请在在线版本中查看]

### 3.10.9 本篇（第 3 章）学习路线回顾

[Mermaid 图表 — 请在在线版本中查看]

#### 3.10.10 自测题（简答）

1. 为什么「多模型」必须配「清晰工具边界」？
2. 事件驱动架构下，**状态单一真源**应放在哪一层（概念即可）？
3. Feature Flags 与「分支发布」相比，最大工程优势是什么？

参考答案方向：

1. 否则模型会 **幻觉调用** 或 **参数胡编**，副作用不可控。
2. 通常在 **协调器 + store**；UI 只投影。
3. **同一产物渐进开闸**，降低发布矩阵复杂度。

### 本篇结语

第 3 篇从 **操作系统隐喻** 出发，经 **入口、目录、数据流、技术栈、竞品、启动、主入口、依赖**，最后落到 **设计哲学**。你此刻应能 **在源码树里不迷路**，并 **用架构语言** 与团队讨论演进。

若你接下来要深入：**工具系统** → 第 3 篇工具专章；**Bridge** → 第 9 篇；**状态管理** → 第 7 篇。

上一节：[09-dependencies.md](#) · 返回目录：[index.md](#)

## 第4篇：核心循环

### 4.1 QueryEngine 是什么：整本「51 万行史诗」的心脏

#### 本节学习目标

- 用一句话说清：QueryEngine 在 Claude Code 里扮演什么角色，以及为何称它为「心脏」。
- 理解 **异步生成器 (async generator)** 如何支撑「可暂停、可流式、可恢复」的对话循环。
- 建立心智模型：约 **51.2 万行**（俗称「半百万级」）TypeScript 中，大量模块最终都为 **query 循环** 供血。

#### 一句话定位

在泄露版源码树中，`query.ts`（约 1730 行）集中实现了 **QueryEngine**：把「用户输入 → 模型推理 → 工具执行 → 再推理」这条链路，编排成一个 **可中断、可观测、可预算** 的无限接近 `while (true)` 的工程循环。

**生活类比**：如果把 Claude Code 比作一座 **24 小时营业的综合体**，那么：

| 楼层（子系统）         | 比喻            | 与 QueryEngine 的关系                    |
|-----------------|---------------|--------------------------------------|
| UI / TUI        | 前台与显示屏        | 消费 <code>StreamEvent</code> ，把字「蹦」出来 |
| 工具系统            | 后勤与外包团队       | 被循环 <b>调度</b> 执行                     |
| 权限系统            | 安保与审批         | 在工具真正落地前 <b>拦截或放行</b>                |
| 压缩 / Compaction | 仓库理货员         | 在循环的「准备消息」阶段 <b>腾空间</b>              |
| QueryEngine     | <b>中央调度电梯</b> | 决定这一趟先上谁、到哪层、是否继续跑                   |

[Mermaid 图表 — 请在在线版本中查看]



## 为什么是「心脏」而不是「大脑」？

- 大脑更像 模型本身（Anthropic 云端推理）与 系统提示词：负责「想」。
- 心脏负责 节律与泵血：在固定节拍内把 上下文、工具结果、错误恢复、预算检查 泵进泵出，让整个 CLI 活着。

[Mermaid 图表 — 请在在线版本中查看]

没有 QueryEngine，你仍可以「单次调用 API」；但 Agent 行为（多轮、工具、自动重试、压缩）会散落成脚本，无法形成 可维护的产品级编排。

## 异步生成器：比 Promise 多一维的「时间切片」

普通 `async function` 像 一次性外卖：调用 → 等到全好 → 返回整单。

`async function*`（异步生成器）像 自动贩卖机按步骤出货：

1. 你投币（发起 `query()`）。
2. 机器先掉一瓶水（`yield` 一个 `StreamEvent`：UI 立刻显示）。
3. 你不必等「所有货道清空」才能看第一瓶上的标签。
4. 若你中途按了退款（用户中断），机器可以在「下一步出货前」停住。

[Mermaid 图表 — 请在在线版本中查看]

### 与 QueryEngine 的对应关系

| 生成器能力                                    | 在 QueryEngine 中的用途                     |
|------------------------------------------|----------------------------------------|
| <code>yield</code> 部分结果                  | 流式输出：逐词 / 逐块刷新 TUI                     |
| 函数体在 <code>yield</code> 之间暂停             | 等工具：把 <code>tool_result</code> 拼回消息再续跑 |
| <code>for await</code> 消费                | UI、日志、测试桩可以 边收边渲染                      |
| <code>throw</code> / <code>return</code> | 致命错误、用户取消、预算耗尽时 结束迭代                   |

**类比强化：**自动贩卖机每一格出货，都对应一次 `yield`；若某一格卡住了（API 错误），心脏层可以 **静默重试** 或换货道（退避），用户只看到「稍慢了一点」，未必看到异常堆栈——详见 [4.7 静默错误修复](#)。

## 「512K 行」到底在忙什么？

本书前言将泄露还原规模概括为约 51.2 万行 TypeScript。数量级上，你可以把它记成「半百万行」量级的客户端编排层。

这些代码并非每一行都在 `query.ts` 里，但 产品行为 上，多数模块是 径向依赖 QueryEngine 的：

[Mermaid 图表 — 请在在线版本中查看]

**教学结论：**读源码时，不要从随机文件「布朗运动」式乱翻。以 **QueryEngine** 为圆心，向外辐射：你每读懂一个相邻模块，就等于给心脏 接上一根更清晰的血管。

### `query()` 与 `queryLoop()`：外壳与心室

泄露版结构（概念级）可记为：

| 符号                       | 角色    | 记忆口诀                                    |
|--------------------------|-------|-----------------------------------------|
| <code>query()</code>     | 外壳    | 参数校验、遥测、把「第一次消息」交给循环                    |
| <code>queryLoop()</code> | 心室    | <code>while (true)</code> 或等价结构，真正跑 8 步 |
| <code>State</code>       | 心电图记录 | 轮次、累积 token、是否已压缩等                      |

下面是一段 教学用伪代码（与真实 `query.ts` 行号不必逐字对齐，但结构一致）：

```
// 概念骨架：异步生成器 = 可暂停的循环
export async function* query(
  input: QueryInput,
  deps: QueryDependencies,
): AsyncGenerator<StreamEvent, QueryResult, undefined> {
  // 外壳：一次性准备（权限模式、会话 id、feature flags 等）
  const initialState = await buildInitialState(input, deps);
  yield* queryLoop(initialState, deps);
}

async function* queryLoop(
  state: State,
  deps: QueryDependencies,
): AsyncGenerator<StreamEvent, QueryResult, undefined> {
  while (true) {
    // 8 步循环见下一节文档；此处仅示意「yield 贯穿全程」
    const events = prepareMessagesMaybeCompact(state, deps);
    // ... API streaming ...
    for await (const ev of streamFromApi(...)) {
      yield ev; // 前台立刻看到「字在流」
    }
    // ... 收集 tool_use、执行工具、预算检查、决定 break 或 continue ...
  }
}
```

## 与本 Part 其他小节的路线图

| 小节        | 文件                                       | 你将深入什么                     |
|-----------|------------------------------------------|----------------------------|
| 4.2       | <a href="#">02-eight-steps.md</a>        | 8 步循环总图 + 厨师做菜类比           |
| 4.3       | <a href="#">03-source-walkthrough.md</a> | <code>query.ts</code> 逐段走读 |
| 4.4~4.6   | 消息 / 流式 / 工具                             | 循环的「输入—感知—手」               |
| 4.7~4.9   | 错误 / 预算 / 终止                             | 循环的「免疫系统与刹车」               |
| 4.10~4.12 | Thinking / 并行 / 生态                       | 高级特性与全景图                   |

[Mermaid 图表 — 请在在线版本中查看]

## 小结

- **QueryEngine** 是 Claude Code 的 **节律器官**：它不替代模型「思考」，但决定 **何时间、问什么、出错怎么办、钱和 token 还够不够**。

- 异步生成器 让这一过程 对用户可见（流式）、对工程 可组合（yield 事件）、对工具调用 可插拔（暂停—恢复）。
- 把 51 万行 读成「围绕 query 循环的星系」，你会少很多迷路。

## 术语快闪表（本篇专用）

| 术语                       | 一句话                                                     |
|--------------------------|---------------------------------------------------------|
| AsyncGenerator           | 可多次 <code>yield</code> 、可 <code>return</code> 最终值的异步生成器 |
| StreamEvent              | 泵给 UI/日志的「增量事实」事件                                       |
| State                    | 一轮会话在循环里的可变账本                                           |
| <code>tool_use</code>    | 模型签发的「外包申请单」                                            |
| <code>tool_result</code> | 客户端回填的「外包回执」                                            |
| Compaction               | 为省窗口而做的历史摘要/裁剪                                          |
| Circuit breaker          | 连续失败后的 强制停机 护栏                                          |

[Mermaid 图表 — 请在在线版本中查看]

## 自测（不用交作业，心里答即可）

1. 为什么 `yield` 比「等整段 JSON」更适合 CLI?
2. `query()` 与 `queryLoop()` 各适合放哪类逻辑?
3. 无 `tool_use` 时循环为何可以停?
4. 87% 与 100% 窗口有何本质区别?
5. `isConcurrencySafe` 是为了解决哪类 bug?

► 参考答案提示（点击展开）

## 推荐阅读顺序（零基础）

[Mermaid 图表 — 请在在线版本中查看]

若你 时间极紧：只读 4.1 + 4.2 + 4.12，也能建立 可用的心智模型；细节再按需回溯。

下一篇：[4.2 八步循环：从备菜到上桌。](#)

## 4.2 八步循环：QueryEngine 的「主菜流水线」

### 本节学习目标

- 背下 8 步循环 的顺序与职责，并能用自己的话复述。
- 通过 Mermaid 流程图 建立「数据—控制流」双视角。
- 用 厨师做菜 类比，把抽象步骤映射到可感知场景。

### 八步口诀（建议朗读三遍）

1. 准备消息（压缩如需）
2. 调用 API（streaming）
3. 收集响应 + 工具请求
4. 处理错误（静默修复）
5. 执行工具
6. 检查预算（token / 金钱 / 轮次）
7. 有工具则发送结果继续
8. 无工具则退出

下面这张表是「职责—产出」速查。

| 步骤 | 核心动作                                     | 主要产出                                       | 若失败常见后果       |
|----|------------------------------------------|--------------------------------------------|---------------|
| 1  | 组装 <code>messages[]</code> ，触发压缩         | 可发送的上下文窗口                                  | 压缩熔断、上下文爆炸    |
| 2  | <code>stream: true</code> 调 Messages API | SSE / chunk 流                              | 网络错误 → 进入步骤 4 |
| 3  | 解析 <code>content</code> 块                | 文本 + <code>tool_use</code> 列表              | 解析异常 → 重试或降级  |
| 4  | 分类错误、退避、重试                               | 恢复后的流或终止决策                                 | 用户无感或温和提示     |
| 5  | 并行/串行执行工具                                | <code>tool_result</code> 块                 | 权限拒绝、工具超时     |
| 6  | 估算 token、费用、轮次                           | <code>continue</code> / <code>break</code> | 硬停止           |
| 7  | 把工具结果追加进历史                               | 下一轮 <code>messages</code>                  | 历史不一致         |
| 8  | 无 <code>tool_use</code>                  | <code>return</code> 最终结果                   | 正常结束          |

## 总览流程图（控制流）

[Mermaid 图表 — 请在在线版本中查看]

## 数据流视角：消息袋如何变重又变轻

[Mermaid 图表 — 请在在线版本中查看]

### 厨师类比：

- 步骤 1：开餐前清点食材（历史），冰箱塞不下就做 **真空压缩**（Compaction）。
- 步骤 2~3：开火炒菜，师傅边做边尝（streaming），同时记下「要去冷库取哪几样配料」（`tool_use`）。
- 步骤 5~7：小工取回配料（工具执行），把新配料倒进锅（`tool_result`），**同一道菜继续炒**（下一轮循环）。

## 逐步详解（与厨师流水线对照）

---

### 步骤 1：准备消息（压缩如需）

- 做什么：把 `State` 里的会话历史整理成 API 接受的 `messages` 数组；若上下文占用超过阈值（教学中常记 约 87% 窗口），触发 自动压缩。
- 厨师类比：备菜台只那么大，先把 切好的半成品 装进统一规格的保鲜盒（摘要），腾出台面。

详见：[4.4 消息准备与历史](#)。

### 步骤 2：调用 API（streaming）

- 做什么：向 Anthropic Messages 端点发起 流式 请求；底层多为 SSE 或等价 chunk 协议。
- 厨师类比：师傅炒菜时 不断翻动（chunk），而不是炒完一整锅才端出来。

详见：[4.5 API 调用与流式响应](#)。

### 步骤 3：收集响应 + 工具请求

- 做什么：在流式解析器里拼接完整 `assistant` 消息；扫描 `content` 中的 `tool_use` 块，收集 `id`、`name`、`input`。
- 厨师类比：师傅一边炒一边 写小纸条：「需要去冷库取 X、Y」。

详见：[4.6 工具请求收集](#)。

### 步骤 4：处理错误（静默修复）

- 做什么：对 429 速率限制、5xx、网络抖动 等做 退避 + 重试；尽量不让用户看到堆栈。
- 厨师类比：油烟机突然响了（API 咳嗽），师傅 关火等两秒再开（backoff），客人只感觉「火候稳」。

详见：[4.7 静默错误修复](#)。

### 步骤 5：执行工具

- 做什么：根据工具元数据决定 并行（`isConcurrencySafe`）或 串行；与权限系统交互后真正执行。
- 厨师类比：洗生菜和烧开水 可同时进行；但「切生肉」与「拌沙拉」可能要 分开台面（串行）。

详见：[4.11 并行工具执行器](#)。

### 步骤 6：检查预算

- 做什么：三重关卡——Token 窗口（常记 200K 量级）、金钱预算、maxTurns 轮次。
- 厨师类比：餐厅打烊前要看 煤气表、食材成本单、今日接单次数。

详见：[4.8 预算三重关卡](#)。

### 步骤 7：有工具则发送结果继续

- 做什么：把每个 `tool_use` 的 `tool_result` 按 API 规范塞回 `messages`，`continue` 外层循环。
- 厨师类比：小工把取回的配料 交回灶台，师傅 继续炒同一道菜。

### 步骤 8：无工具则退出

- 做什么：若模型本轮 未请求工具（或仅输出文本），组装最终 UI 状态，`return` / `break` 离开 `queryLoop`。
- 厨师类比：菜已装盘，关火收工。

详见：[4.9 循环终止条件](#)。

## 序列图：一轮迭代中的「时间顺序」

[Mermaid 图表 — 请在在线版本中查看]

### 「while(true)」并不可怕：退出条件在 6、7、8 汇聚

初学者看到 `while (true)` 会紧张。实际上 出口 由三类条件共同把守：

| 出口类型 | 典型触发                    | 在哪一步最密集                      |
|------|-------------------------|------------------------------|
| 正常完成 | 无 <code>tool_use</code> | 步骤 8                         |
| 预算耗尽 | token / 钱 / 轮次          | 步骤 6                         |
| 用户中断 | 信号 / UI 取消              | 任意 <code>yield</code> 间可协作取消 |
| 致命错误 | 不可恢复、压缩熔断               | 步骤 1 或 4                     |



[Mermaid 图表 — 请在在线版本中查看]

## 与异步生成器的结合点

八步循环 不是 一次性跑完再返回；而是在 步骤 2~3 持续 `yield`：

```
// 教学伪代码：步骤 2-3 与 yield 交织
for await (const chunk of apiStream) {
  const partial = parser.push(chunk);
  if (partial?.type === "content_block_delta") {
    yield { kind: "stream", delta: partial }; // UI 实时显示
  }
}
const finalAssistant = parser.finalize();
const toolUses = extractToolUses(finalAssistant);
```

这解释了为何 **心脏** 必须用生成器：步骤与可观测性 同构。

## 小结

- 8 步 是 QueryEngine 的 **主节拍**：准备 → 问模型 → 收工具意图 → 容错 → 动手 → 算账 → 有工具再转一圈 → 没工具就停。
- 厨师做菜 类比帮助记忆：备料—翻炒—写配料单—处理灶火不稳—取料—看成本—回锅—装盘。
- 下一篇将把这一切 钉回 `query.ts` 的真实骨架：[4.3 query.ts 逐段走读](#)。

## 4.3 `query.ts` 逐段走读：从 `query()` 外壳到 `while(true)` 心室

### 本节学习目标

- 能在白板上画出 `query` → `queryLoop` → `State` → 循环体 四层结构。
- 理解 **为何** 把「初始化」与「无限迭代」拆开，而不是写成一个巨型函数。
- 对照下面 **分段伪代码**，在真实仓库中快速定位同名符号（函数名可能因版本略有差异）。

## 读源码的心态：1730 行不是 1730 个概念

`query.ts` 行数多，主要来自：

| 来源                        | 你应关注的抽象                                              |
|---------------------------|------------------------------------------------------|
| 类型定义与联合判别                 | <code>StreamEvent</code> 、 <code>ToolUseBlock</code> |
| 遥测 / 日志 / Feature Flag 分支 | 可略读，知道「有开关」即可                                        |
| 错误处理与重试                   | 与 <a href="#">4.7</a> 对照                             |
| 工具与权限交接                   | 与 <a href="#">4.6</a> 、Part 权限篇对照                    |

读法：先抓 骨架，再按需「钻血管」。

[Mermaid 图表 — 请在在线版本中查看]

## 地图：`query.ts` 推荐的「五段式」划分

下面分区名是 教学标签，真实文件可能把部分逻辑拆到 `*.ts` 友元模块，但 语义分区 稳定。

| 区段                             | 典型内容                                                         | 关联小节                |
|--------------------------------|--------------------------------------------------------------|---------------------|
| A. 导入与类型                       | SDK、内部 <code>Message</code> 、 <code>Tool</code> 类型           | 全书各处                |
| B. <code>query()</code> 外壳     | 参数归一、遥测、构造初始 <code>State</code>                              | 本节                  |
| C. <code>queryLoop()</code> 签名 | <code>AsyncGenerator</code> 的 <code>yield</code> 类型          | 本节                  |
| D. <code>while (true)</code> 体 | 8 步循环主逻辑                                                     | <a href="#">4.2</a> |
| E. 辅助纯函数                       | <code>extractToolUses</code> 、 <code>estimateTokens</code> 等 | 4.4~4.8             |

[Mermaid 图表 — 请在在线版本中查看]

## 区段 A：导入层——心脏的「供血名单」

教学上你把 `import` 想成 术前核对器械：

```
// 教学示意: import 区块常出现这些「家族」
import Anthropic from "@anthropic-ai/sdk"; // 或等价封装
import type { MessageParam, ContentBlock } from "../types/messages";
import { runToolExecutor } from "../tool-executor"; // 名称示意
import { maybeCompactConversation } from "../compaction"; // 名称示意
import { checkBudgets } from "../budget"; // 名称示意
```

| 依赖家族       | 用途                                              |
|------------|-------------------------------------------------|
| SDK / HTTP | 真正发起 Messages 请求                                |
| 消息类型       | 保证 <code>role</code> 与 <code>content</code> 块合法 |
| 工具执行器      | 步骤 5                                            |
| 压缩模块       | 步骤 1                                            |
| 预算模块       | 步骤 6                                            |

## 区段 B: `query()` 外壳——一次性装配

外壳原则：把「只该做一次」的事从循环里踢出去，避免每轮重复付成本。

[Mermaid 图表 — 请在在线版本中查看]

## 伪代码：外壳长什么样？

```
export async function* query(  
  input: QueryInput,  
  ctx: QueryContext,  
) : AsyncGenerator<StreamEvent, QueryResult, undefined> {  
  // 1) 参数与模式归一（例如从 CLI flag 合并）  
  const mode = resolvePermissionMode(input, ctx.config);  
  
  // 2) 构造初始 State: 轮次=0、历史来自会话存储  
  let state: State = {  
    turn: 0,  
    messages: await ctx.session.loadMessages(input.threadId),  
    model: input.model,  
    compactionFailures: 0,  
    // ... 预算指针、元数据 ...  
  };  
  
  // 3) 可选：最外层 try/finally 做 span.end()  
  try {  
    yield* queryLoop(state, ctx, mode);  
  } finally {  
    await ctx.telemetry.flush();  
  }  
}
```

生活类比：`query()` 像 婚礼前的彩排安排：确认场地、名单、流程表；真正 每一遍走位 在 `queryLoop()`。

## 区段 C：`queryLoop()` ——异步生成器的「心室入口」

注意返回类型：`AsyncGenerator<Yield, Return, Next>`。

```
async function* queryLoop(  
  state: State,  
  ctx: QueryContext,  
  mode: PermissionMode,  
) : AsyncGenerator<StreamEvent, QueryResult, undefined> {  
  while (true) {  
    // ... 见下一区段 ...  
  }  
}
```

| 泛型参数   | 在 QueryEngine 中的实例化直觉            |
|--------|----------------------------------|
| Yield  | StreamEvent （流式 token、工具状态、系统提示） |
| Return | QueryResult （最终文本、用量统计、退出原因）     |
| Next   | 多为 undefined （CLI 不向生成器「投喂」中间值）  |

[Mermaid 图表 — 请在在线版本中查看]

### 区段 D：while (true) 循环体——八步落地

下面是 结构级伪代码，把 4.2 的八步 嵌进真实控制形状：

```

while (true) {
  state.turn += 1;

  // === 1. 准备消息 (压缩如需) ===
  state.messages = await prepareMessagesWithCompaction(state, ctx);
  if (state.compactionCircuitOpen) {
    return finalizeResult(state, "compaction_circuit_breaker");
  }

  // === 2. 调用 API (streaming) ===
  const stream = await createMessageStream(state.messages, ctx, { stream: true });

  // === 3. 收集响应 + 工具请求 (yield 交织) ===
  let assistantMsg: AssistantMessage;
  try {
    assistantMsg = await collectAssistantMessage(stream, (ev) => {
      // 边收边 yield — 这是「心脏泵血」的关键
      // 在真实代码里可能是 generator 内部函数或 async iterator 包装
    });
  } catch (e) {
    // === 4. 处理错误 (静默修复) ===
    const recovered = await handleStreamError(e, ctx);
    if (!recovered) {
      return finalizeResult(state, "fatal_api_error");
    }
    continue; // 重试本轮
  }

  state.messages.push(assistantMsg);

  const toolUses = extractToolUses(assistantMsg);

  // === 6. 预算检查 (工具执行前后都可能出现, 实现可能分两处) ===
  const budget = checkBudgets(state, ctx, assistantMsg);
  if (!budget.ok) {
    return finalizeResult(state, budget.reason);
  }

  if (toolUses.length === 0) {
    // === 8. 无工具则退出 ===
    return finalizeResult(state, "completed");
  }

  // === 5. 执行工具 ===
  const toolResults = await executeTools(toolUses, state, ctx, mode);

  // === 7. 有工具则发送结果继续 ===
  state.messages.push(...toolResults);
  continue;
}

```

## collectAssistantMessage 与 yield 如何缝合?

真实实现常见两种风格:

| 风格                                                                      | 优点   | 缺点    |
|-------------------------------------------------------------------------|------|-------|
| 在 <code>queryLoop</code> 内联 <code>for await</code> + <code>yield</code> | 易读   | 函数会膨胀 |
| 传入 <code>emit(ev)</code> 回调，由外层 <code>yield</code>                      | 分层清晰 | 跳转多   |

[Mermaid 图表 — 请在在线版本中查看]

## 区段 E: `State` 初始化——循环在记什么账？

`State` 不是 Redux 那种全局 store，而是「本轮会话的循环账本」。

| 字段（教学名）                                   | 含义                                   |
|-------------------------------------------|--------------------------------------|
| <code>turn</code> / <code>maxTurns</code> | 当前轮次与用户配置上限                          |
| <code>messages</code>                     | 完整对话历史（含 tool 块）                     |
| <code>compactionFailures</code>           | 连续压缩失败次数，触 熔断（教学中常记 连续 3 次）          |
| <code>usage</code>                        | 累积 input/output token，用于 87% 阈值 与 费用 |
| <code>flags</code>                        | 是否用户取消、是否进入只读模式等                     |

[Mermaid 图表 — 请在在线版本中查看]

### 初始化时与循环中的变化

| 阶段                         | <code>messages</code>                                | <code>turn</code>                     | <code>compactionFailures</code> |
|----------------------------|------------------------------------------------------|---------------------------------------|---------------------------------|
| <code>query()</code> 结束初始化 | 来自磁盘 / 内存会话                                          | <code>0</code> 或 <code>1</code> （依实现） | <code>0</code>                  |
| 每轮工具后                      | 追加 <code>assistant</code> + <code>tool_result</code> | <code>+1</code>                       | 成功压缩则清零失败计数                     |
| 压缩连续失败                     | 可能不变或部分回滚                                            | –                                     | <code>+1</code> ，达阈值则熔断         |

## 如何自己在仓库里「锚定」行号？

- 搜索 `async function* query` 或 `function* query`。

2. 搜索 `queryLoop`、`**while (true) **`（注意有些版本用 `for (;;)` ）。
3. 搜索 `tool_use`、`**tool_result **` 字符串——快速跳到 步骤 3、5、7。
4. 打开 Git blame（若你有重建仓库）：看 最近修改 是否集中在 `streaming` 解析。

[Mermaid 图表 — 请在在线版本中查看]

## 小结

- `query()` 负责 一次性的装配与收尾；`queryLoop()` 负责 可重复的八步节拍。
- `State` 是循环的 账本：历史、轮次、压缩失败、用量全在这里汇聚。
- `while (true)` 的合法性由 预算、无工具、熔断、用户取消 共同保证出口。

下一篇：[4.4 消息准备与历史](#)。

## 4.4 消息准备与历史：对话的「行李箱」怎么打包

### 本节学习目标

- 能画出 `messages[]` 数组中 `system` / `user` / `assistant` 的相对位置与典型内容。
- 理解 为何要压缩：窗口上限、成本、延迟三者的三角约束。
- 记住教学中常用的 87% 阈值 与 连续 3 次压缩失败熔断 的工程意图。

## 消息数组：不是聊天记录那么简单

Anthropic Messages API 把一次「多轮对话」建模为 有序消息列表。在 QueryEngine 里，这个列表既是：

- 输入：每轮 API 请求的 payload；
- 输出载体：模型返回的 `assistant` 消息会被 追加 回去；
- 工具闭环：`tool_use` 与 `tool_result` 必须 成对、同序 出现在历史中。

[Mermaid 图表 — 请在在线版本中查看]



### 三种角色的「语气」与职责

| 角色                     | 典型内容                                     | 可否包含 <code>tool_use</code> ? |
|------------------------|------------------------------------------|------------------------------|
| <code>system</code>    | 行为规范、工具说明摘要、项目 <code>CLAUDE.md</code> 注入 | 否（概念上）                       |
| <code>user</code>      | 人类输入、或 工具结果包装块                           | 否                            |
| <code>assistant</code> | 自然语言 + <code>tool_use</code> 块           | 是                            |

生活类比：

- `system` 像 员工手册：所有人进场前读一遍。
- `user` 像 顾客点单 + 仓库回货单（工具结果是「后台告诉前台」的事实）。
- `assistant` 像 店长口头回复 + 手写采购条（`tool_use`）。

### `content` 块：一条消息内部的「乐高」

单条消息常见结构（教学简化）：

```
type Message = {
  role: "user" | "assistant" | "system";
  content: ContentBlock[] | string; // 复杂场景多为块数组
};

type ContentBlock =
  | { type: "text"; text: string }
  | { type: "tool_use"; id: string; name: string; input: unknown }
  | { type: "tool_result"; tool_use_id: string; content: unknown }
  // thinking 等扩展见 4.10
;
```

[Mermaid 图表 — 请在在线版本中查看]

### 为何用块数组而不是纯字符串？

| 方案   | 优点              | 缺点           |
|------|-----------------|--------------|
| 纯字符串 | 人类可读            | 工具调用难结构化、难校验 |
| 块数组  | API 原生、可混合文本与工具 | 客户端需维护拼接顺序   |

## 历史追加规则：工具轮的特殊形状

当模型在一轮里发出工具调用，后续 **必须** 出现携带 `tool_result` 的 `user` 消息（或 SDK 约定的等价结构），且 `tool_use_id` 对齐。

[Mermaid 图表 — 请在在线版本中查看]

**错误示例（教学）：**漏掉 `tool_result` 或顺序颠倒 → API 400 或模型 **胡编**（以为工具已执行）。

## 压缩（Compaction）：行李箱满了怎么办？

### 触发条件（教学口径）

| 信号                     | 含义                       |
|------------------------|--------------------------|
| 估算上下文 token ≥ 窗口的 ~87% | 主动压缩，留出安全余量给下一轮输出        |
| 用户显式触发                 | 部分产品形态支持「整理上下文」          |
| 硬失败前                   | 作为 <b>避免请求被拒</b> 的最后一道整理 |

[Mermaid 图表 — 请在在线版本中查看]

### 连续 3 次失败熔断：为什么需要？

- **避免死循环：**压缩本身也调用模型 → 可能再失败 → 再压缩.....
- **保护用户钱包：**无意义的重复摘要请求会 **烧钱**。
- **可观测性：**熔断后应 **明确退出原因**（见 [4.9](#)）。

**生活类比：**行李箱拉链卡住了，你不会无限 **用力硬拉三次以上**——第三次还失败就该 **换箱子策略**（清空、托运、分装），而不是和拉链搏斗一整天。

## 压缩策略家族（概念表）

| 策略     | 做法                                   | 代价        |
|--------|--------------------------------------|-----------|
| 滑动窗口   | 丢掉最早若干轮                              | 丢事实，可能忘需求 |
| 摘要替换   | 用短摘要替代大块 <code>user/assistant</code> | 摘要可能丢细节   |
| 结构化保留  | 保留「当前任务描述 + 最近 N 轮」                  | 实现复杂      |
| 工具轨迹裁剪 | 删冗长 <code>tool_result</code> ，保留结论   | 调试信息变少    |

真实 `query.ts` 往往组合多种策略，并受 **Feature Flag** 控制。

## prepareMessages 伪代码

```
async function prepareMessagesWithCompaction(
  state: State,
  ctx: QueryContext,
): Promise<Message[]> {
  let messages = clone(state.messages);

  const usageRatio = await estimateContextUsageRatio(messages, state.model);
  if (usageRatio < 0.87) {
    return injectSystemIfNeeded(messages, ctx);
  }

  try {
    messages = await runCompaction(messages, ctx);
    state.compactionFailures = 0;
  } catch (e) {
    state.compactionFailures += 1;
    if (state.compactionFailures >= 3) {
      state.compactionCircuitOpen = true;
      throw new CompactionCircuitOpenError();
    }
    // 可能：降级为滑动窗口
    messages = fallbackTrim(messages);
  }

  return injectSystemIfNeeded(messages, ctx);
}

function injectSystemIfNeeded(messages: Message[], ctx: QueryContext): Message[] {
  // system 可能在队首合并，也可能与项目记忆拼接
  const system = buildSystemPrompt(ctx);
  if (messages[0]?.role === "system") {
    return [mergeSystem(messages[0], system), ...messages.slice(1)];
  }
  return [{ role: "system", content: system }, ...messages];
}
```

## 与 QueryEngine 八步的衔接

[Mermaid 图表 — 请在在线版本中查看]

| 步骤 1 子任务 | 说明               |
|----------|------------------|
| 归一化角色    | 修正畸形历史（崩溃恢复）     |
| 注入系统提示   | CLAUDE.md、工具列表摘要 |
| 压缩判断     | 87% 阈值           |
| 失败熔断     | 连续 3 次           |

## 小结

- `messages[]` 是 QueryEngine 的 主数据结构：角色有序、`content` 分块、工具成对。
- 压缩 是 步骤 1 的核心压力阀：87% 触发、三次失败 熔断是典型工程护栏。
- 下一篇进入 步骤 2~3 的动脉：[4.5 API 调用与流式响应](#)。

# 4.5 API 调用与流式响应：逐词「泵血」到终端

### 本节学习目标

- 理解 Messages API 在 QueryEngine 中的调用形状（`stream: true`）。
- 能把 SSE / chunk 事件映射到 `yield StreamEvent`。
- 区分 网络字节流、SDK 事件、UI 增量 三层。

## 非流式 vs 流式：餐厅上菜两种模式

| 模式  | 用户体验      | QueryEngine 亲和度                     |
|-----|-----------|-------------------------------------|
| 非流式 | 等整段生成完才显示 | 差：长推理像卡死                            |
| 流式  | 边生成边显示    | 优：与 <code>async generator</code> 同构 |

生活类比：非流式像 等整桌菜齐才上；流式像 火锅边涮边吃——你立刻有反馈，心理等待时间更短。

[Mermaid 图表 — 请在在线版本中查看]

## Anthropic Messages API：教学级请求骨架

真实字段名以 SDK 为准，下面是 结构等价 的 JSON 示意：

```
{
  "model": "claude-sonnet-4-20250514",
  "max_tokens": 8192,
  "stream": true,
  "system": "你是 Claude Code 代理.....",
  "messages": [
    { "role": "user", "content": "请读取 package.json" }
  ],
  "tools": [ /* 工具 schema 列表 */ ]
}
```

| 字段       | QueryEngine 侧职责        |
|----------|------------------------|
| model    | 来自配置 / CLI flag        |
| messages | <a href="#">4.4</a> 准备 |
| tools    | 工具注册表序列化               |
| stream   | 必须为 true 才能边下边解析       |

## 流式响应：事件类型「家族树」

不同版本 SDK 命名略有差异，教学中可归纳为：

[Mermaid 图表 — 请在在线版本中查看]

**content\_block\_delta**：UI 最关心的节点

```
// 教学伪类型
type TextDelta = {
  type: "content_block_delta";
  index: number;
  delta: { type: "text_delta"; text: string };
};

type ToolInputDelta = {
  type: "content_block_delta";
  index: number;
  delta: { type: "input_json_delta"; partial_json: string };
};
```

| Delta 类型         | 说明                            |
|------------------|-------------------------------|
| text_delta       | 用户可见的逐段文本                     |
| input_json_delta | tool_use.input 的 JSON 增量（需拼接） |
| thinking 相关      | 见 <a href="#">4.10</a>        |

从字节到 **yield**：三层解码器

[Mermaid 图表 — 请在在线版本中查看]

教学伪代码： `for await` + `yield`

```
async function* streamAssistant(
  apiStream: AsyncIterable<StreamRawEvent>,
): AsyncGenerator<StreamEvent, AssistantMessage, undefined> {
  const acc = createMessageAccumulator();

  for await (const ev of apiStream) {
    acc.ingest(ev);

    // 立刻把「可展示增量」泵出去
    if (ev.type === "content_block_delta" && ev.delta.type === "text_delta") {
      yield {
        kind: "assistant_text_delta",
        text: ev.delta.text,
      };
    }

    if (ev.type === "content_block_delta" && ev.delta.type === "input_json_delta") {
      yield {
        kind: "tool_input_delta",
        index: ev.index,
        partialJson: ev.delta.partial_json,
      };
    }
  }

  return acc.finalizeAssistantMessage();
}
```

要点： `return` 的 `AssistantMessage` 在 TypeScript 异步生成器里需通过 消费者 特殊方式获取；真实代码可能用 辅助结构 或 非 generator 子函数。此处强调 概念： `yield` 负责「过程」， `finalize` 负责「结果」。

背压与取消：流式不是「无限快」

| 问题        | 工程对策                                      |
|-----------|-------------------------------------------|
| UI 渲染跟不上  | UI 侧节流 (requestAnimationFrame / batching) |
| 用户 Ctrl+C | <code>AbortController</code> 中断 fetch     |
| 网络抖动      | <a href="#">4.7</a> 重试                    |

[Mermaid 图表 — 请在在线版本中查看]



usage

与流式结束：

message\_delta

流式结束前，服务器常通过 `message_delta` 报告 本消息 的 token 用量。QueryEngine 把它 累加进 State，供 [4.8 预算](#) 使用。

```
// 教学示意
if (event.type === "message_delta" && event.usage) {
  state.usage.input_tokens += event.usage.input_tokens ?? 0;
  state.usage.output_tokens += event.usage.output_tokens ?? 0;
}
```

| 字段                         | 用途                              |
|----------------------------|---------------------------------|
| <code>input_tokens</code>  | 窗口占用、计费                         |
| <code>output_tokens</code> | 计费、是否逼近 <code>max_tokens</code> |

### 错误在流中途出现怎么办？

| 阶段          | 表现                    | 归属步骤       |
|-------------|-----------------------|------------|
| 连接建立前       | <code>fetch</code> 失败 | 4.7        |
| 头几个 chunk 后 | HTTP 非 200            | 4.7        |
| 解析到一半       | JSON 行损坏              | 4.7，可能重试整轮 |

[Mermaid 图表 — 请在在线版本中查看]

### 与八步循环的对照

| 八步       | 本节覆盖                                      |
|----------|-------------------------------------------|
| 2 调用 API | 请求构造、 <code>stream: true</code>           |
| 3 收集响应   | accumulator 完结为 <code>assistant</code> 消息 |
| 4 错误     | 流中断的恢复策略（详见 4.7）                          |

## 小结

- 流式 让 QueryEngine 与 异步生成器 天然契合： `content_block_delta` → `yield`。
- 解码链路是 字节 → 行 → JSON 事件 → 累积消息 → UI。
- 用量统计 在流末尾回填 State，服务预算系统。

下一篇： [4.6 工具请求收集](#)。

## 4.6 工具请求收集：从模型「意图」到可执行清单

### 本节学习目标

- 能从 `assistant` 消息的 `content` 中 识别并提取 `tool_use` 块。
- 理解 `ToolUseBlock` 在内部如何传递，直到 工具执行器 接手。
- 知道 多工具一轮 时，顺序与 ID 的重要性。

## 工具调用在 API 里的长相

模型不「直接跑函数」；它输出结构化 `tool_use` 块，由客户端 代为执行 再把结果喂回。

```
type ToolUseBlock = {
  type: "tool_use";
  id: string;           // 全局唯一，用于 tool_result 对齐
  name: string;         // 与注册表中的工具名一致
  input: unknown;       // 通常是对象，来自 JSON 解析
};
```

生活类比：模型写 外包申请单（`tool_use`），行政（QueryEngine）盖章编号（`id`），后勤部（工具执行器）真正采购（读文件、跑命令），再把 发票复印件（`tool_result`）交回模型。

[Mermaid 图表 — 请在在线版本中查看]

## 流式阶段如何「长」出完整 `tool_use`

工具调用的 `input` 常通过 `input_json_delta` 一块块到达，需要 拼接 + `JSON.parse`。

[Mermaid 图表 — 请在在线版本中查看]

### 教学伪代码：块索引与 buffer

```
class StreamingAccumulator {
  private buffers = new Map<number, string>();

  pushToolInputDelta(index: number, partial: string) {
    const prev = this.buffers.get(index) ?? "";
    this.buffers.set(index, prev + partial);
  }

  finalizeToolUse(index: number, name: string, id: string): ToolUseBlock {
    const raw = this.buffers.get(index) ?? "{}";
    let input: unknown;
    try {
      input = JSON.parse(raw);
    } catch {
      input = { _parse_error: true, raw };
    }
    return { type: "tool_use", id, name, input };
  }
}
```

| 风险                           | 缓解                                         |
|------------------------------|--------------------------------------------|
| JSON 未闭合就 <code>parse</code> | 只在 <code>content_block_stop</code> 后 parse |
| 模型生成非法 JSON                  | 标记错误，走 <a href="#">4.7</a> 或工具内纠错          |

### `extractToolUses`：从消息到数组

当流结束，`assistant` 消息应包含 0...N 个工具块（可与 `text` 块混排）。

```
function extractToolUses(msg: AssistantMessage): ToolUseBlock[] {
  if (typeof msg.content === "string") {
    return [];
  }
  return msg.content.filter((b): b is ToolUseBlock => b.type === "tool_use");
}
```

[Mermaid 图表 — 请在在线版本中查看]

## 多工具并行 vs 顺序：收集阶段要保守

收集本身不改变顺序——它只做 列表化。是否并行由 [4.11](#) 决策。

| 场景                                               | 典型策略                                     |
|--------------------------------------------------|------------------------------------------|
| 三个 <code>read_file</code> 不同路径                   | 可能并行（若均 <code>isConcurrencySafe</code> ） |
| <code>edit</code> 后立刻 <code>read_file</code> 同路径 | 常需串行                                     |

[Mermaid 图表 — 请在在线版本中查看]

## 分发：从 `ToolUseBlock` 到「具体实现」

[Mermaid 图表 — 请在在线版本中查看]

## 教学伪代码

```
async function dispatchToolUses(
  uses: ToolUseBlock[],
  ctx: QueryContext,
  mode: PermissionMode,
): Promise<ToolResultBlock[]> {
  const plan = buildExecutionPlan(uses, ctx.registry);
  const results: ToolResultBlock[] = [];

  for (const group of plan.groups) {
    if (group.mode === "parallel") {
      const chunk = await Promise.all(
        group.items.map((u) => executeOne(u, ctx, mode)),
      );
      results.push(...chunk);
    } else {
      for (const u of group.items) {
        results.push(await executeOne(u, ctx, mode));
      }
    }
  }

  return results;
}
```

### tool\_result 块：收集的「镜像」

每个 `tool_use` 必须 恰好对应 一个 `tool_result`（教学中一对一；复杂实现可能有批处理，但 API 约束仍以 id 对齐为准）。

```
type ToolResultBlock = {
  type: "tool_result";
  tool_use_id: string;
  content: unknown;
};
```

| 字段                       | 常见错误        |
|--------------------------|-------------|
| <code>tool_use_id</code> | 拼写错、重复用 id  |
| <code>content</code>     | 把大对象直接塞爆上下文 |

## 与八步循环的编号对齐

| 八步 | 本节对应                                                |
|----|-----------------------------------------------------|
| 3  | 从流与最终消息 提取 <code>tool_use</code>                    |
| 5  | 执行（收集之后）                                            |
| 7  | 把 <code>tool_result</code> 写回 <code>messages</code> |

[Mermaid 图表 — 请在在线版本中查看]

## 小结

- `tool_use` 是模型意图的 结构化锚点；`id` 是 闭环凭证。
- 流式 JSON 需要 缓冲—解析—校验 三件套。
- 收集与 执行 解耦：前者产出 `ToolUseBlock[]`，后者对接 权限 + 注册表。

下一篇：[4.7 静默错误修复](#)。

# 4.7 静默错误修复：用户看不见的「免疫系统」

### 本节学习目标

- 能对错误做 粗分类：API 错误 / 429 速率限制 / 网络超时 / 解析失败。
- 理解 退避（backoff）+ 抖动（jitter）+ 上限（cap） 三件套。
- 区分 用户应知 与 应静默 的边界，避免「沉默式失败」的伦理风险。

## 为何需要「静默」？

QueryEngine 面向 交互式 CLI：用户心理模型是「在和助手聊天」，而不是「在盯 nginx 日志」。

| 若每次都抛堆栈 | 体验后果     |
|---------|----------|
| 打断心流    | 用户以为产品坏了 |
| 暴露内部路径  | 信息泄露风险   |
| 重复可恢复错误 | 显得「不智能」  |

生活类比：优秀服务员不会在客人面前 大喊厨房着火了；他会说「请稍等，我们为您重新上一份」——背后可能是 重试一次出餐。

[Mermaid 图表 — 请在在线版本中查看]

## 错误分类矩阵

| 类型         | 典型信号                               | 默认可恢复? | 常见策略          |
|------------|------------------------------------|--------|---------------|
| 429 速率限制   | HTTP 429、 <code>retry-after</code> | 是      | 尊重 header 等待  |
| 5xx 服务端    | 502/503/504                        | 是      | 指数退避重试        |
| 网络超时       | <code>ETIMEDOUT</code>             | 是      | 重试 + 降并发      |
| DNS / 断网   | <code>ENOTFOUND</code>             | 视情况    | 有限次重试后提示      |
| 401/403 鉴权 | 密钥错、组织策略                           | 否      | 必须明确提示用户      |
| 400 请求体    | 消息结构不合法                            | 多半否    | 记录上下文，可能需开发介入 |
| 压缩熔断       | 连续失败计数                             | 否      | 终止并解释原因       |

[Mermaid 图表 — 请在在线版本中查看]

## 重试策略：指数退避 + 抖动

### 公式（教学）

[  $\text{delay} = \min(\text{cap}, \text{base} \times 2^{\text{attempt}} + \text{jitter})$  ]

| 参数     | 典型取值（示意）   |
|--------|------------|
| base   | 200ms~1s   |
| cap    | 30s~120s   |
| jitter | 随机 0~250ms |

为何要有 jitter？ 大量客户端同时 429 时，若都 精确同一秒重试，会造成 重试风暴。

[Mermaid 图表 — 请在在线版本中查看]

### 教学伪代码

```
async function withSilentRetries<T>(  
  fn: () => Promise<T>,  
  opts: { maxAttempts: number; baseMs: number; capMs: number },  
) : Promise<T> {  
  let attempt = 0;  
  // eslint-disable-next-line no-constant-condition  
  while (true) {  
    try {  
      return await fn();  
    } catch (e) {  
      attempt += 1;  
      if (attempt >= opts.maxAttempts || !isRetriableError(e)) {  
        throw e;  
      }  
      const delay = computeBackoffWithJitter(attempt, opts.baseMs, opts.capMs);  
      await sleep(delay);  
      yieldRetryTelemetry(e, attempt); // 用户不可见  
    }  
  }  
}
```

### 「静默」不是「隐瞒」：可观测性必须存在

| 渠道      | 内容                  |
|---------|---------------------|
| 本地日志文件  | 完整 stack、request id |
| 遥测（若启用） | 错误码聚合               |
| UI      | 一句「正在重试…」或进度条       |



[Mermaid 图表 — 请在在线版本中查看]

**伦理边界：**若错误导致 **任务未完成**，最终仍应 **明确告知**「未完成的原因」，而不是无限重试后假装成功。

## 与流式解析错误的交织

| 阶段                            | 错误            | 处理                                     |
|-------------------------------|---------------|----------------------------------------|
| 半条 SSE 行                      | JSON.parse 失败 | 丢弃坏行或整轮重试                              |
| <code>message_stop</code> 未到达 | 连接断开          | 视为可恢复，重试本轮                             |
| <code>tool_use</code> JSON 坏  | 输入非法          | 生成 <code>tool_result</code> 描述错误，让模型自纠 |

[Mermaid 图表 — 请在在线版本中查看]

## 与八步循环的挂钩

| 八步  | 静默修复发生点       |
|-----|---------------|
| 2~3 | API 流、解析      |
| 5   | 工具执行超时（可重试一次） |
| 1   | 压缩子请求失败（计入熔断） |

## 小结

- 分类决定 **重试还是停机**；429/5xx/超时 常走静默恢复。
- **退避 + jitter + cap** 是工业标准，防止 **重试风暴**。
- **静默面向 UI**，**日志/遥测面向真相**——二者缺一不可。

## 重试次数与退避：推荐配置表（教学）

下表 **不是** 泄露源码的硬编码值，而是 **工业 CLI** 常见量级，便于你做实验或读代码时对照。

| 错误族     | 建议 <code>maxAttempts</code> | <code>baseMs</code> | <code>capMs</code> | 备注                           |
|---------|-----------------------------|---------------------|--------------------|------------------------------|
| 429     | 5~8                         | 500                 | 60000              | 优先读 <code>retry-after</code> |
| 503/502 | 3~5                         | 300                 | 30000              | 可略激进                         |
| 网络抖动    | 3                           | 200                 | 15000              | 配合 DNS 缓存                    |
| 解析半包    | 1~2                         | 0                   | 0                  | 多 重拉整流 而非死 parse             |

[Mermaid 图表 — 请在在线版本中查看]

isRetriableError

判别伪代码

```
function isRetriableError(e: unknown): boolean {
  if (isHttpError(e)) {
    if (e.status === 429) return true;
    if (e.status >= 500 && e.status <= 599) return true;
    if (e.status === 408) return true; // Request Timeout
    return false;
  }
  if (isNodeNetError(e)) {
    return ["ECONNRESET", "ETIMEDOUT", "EAI_AGAIN"].includes(e.code);
  }
  return false;
}
```

| 返回 <code>false</code> 的典型情况              | 处理                                    |
|------------------------------------------|---------------------------------------|
| 401                                      | 引导用户检查 <code>ANTHROPIC_API_KEY</code> |
| 400 + <code>invalid_request_error</code> | 记录 <code>messages</code> 摘要到日志，避免上传密钥 |

与 QueryEngine 八步的「挂钩点」速查

[Mermaid 图表 — 请在在线版本中查看]

| 步骤  | 可静默? | 典型用户感知      |
|-----|------|-------------|
| 2~3 | 常可   | 「稍等」或无明显提示  |
| 5   | 部分可  | 工具卡片显示重试中   |
| 1   | 有限   | 压缩失败或触发熔断文案 |

## 可观测性字段：建议 always-on

| 字段                       | 用途                                                                          |
|--------------------------|-----------------------------------------------------------------------------|
| <code>request_id</code>  | 对齐 Anthropic 支持工单                                                           |
| <code>attempt</code>     | 区分首次与重试                                                                     |
| <code>error_class</code> | <code>429</code> / <code>5xx</code> / <code>net</code> / <code>parse</code> |
| <code>backoff_ms</code>  | 复盘退避是否合理                                                                    |

## 常见误解

| 误解            | 澄清                   |
|---------------|----------------------|
| 「静默 = 吞掉错误」   | 错：应 记录 + 聚合，只是 默认不刷屏 |
| 「所有 400 都可重试」 | 错：多数 400 需 修请求体      |
| 「重试越多越好」      | 错：会放大 雪崩 与 账单        |

下一篇：[4.8 预算三重关卡](#)。

# 4.8 预算三重关卡：Token、金钱与轮次

### 本节学习目标

- 说出 **三重预算** 各自守护什么，以及 **任一超限** 如何终止循环。
- 理解 **~200K 上下文窗口** 在教学中的数量级意义（具体以模型卡为准）。

- 能把 **软预警（87% 压缩）** 与 **硬停止（预算）** 区分开。

## 一句话：三重关卡是「加油站 + 钱包 + 末班车」

| 关卡                         | 类比       | 技术对象                           |
|----------------------------|----------|--------------------------------|
| Token 预算                   | 油箱表显剩余里程 | 上下文窗口 + 累计用量                   |
| 金钱预算                       | 钱包里的今日额度 | 组织/用户的费用上限配置                   |
| 轮次预算 <code>maxTurns</code> | 末班车还剩几站  | 单轮 <code>queryLoop</code> 迭代计数 |

任一失败 → QueryEngine 不得继续 调 API（否则既贵又可能直接被服务端拒绝）。

[Mermaid 图表 — 请在在线版本中查看]

## 关卡一：Token 预算（200K 窗口的教学口径）

### 为何是「窗口」而不是「累计聊天字数」？

大模型每次推理看到的是 **当前请求里塞进去的全部 token**（系统提示 + 历史 + 工具结果）。因此：

- **长历史** 会挤占窗口；
- **大工具输出** 可能一次性吃掉数万 token；
- **输出长度** 也受 `max_tokens` 约束。

教学中常把旗舰上下文记为 **约 200K tokens** 量级——真实数字请始终以 **模型发布说明** 为准。

| 概念       | 说明                                   |
|----------|--------------------------------------|
| 上下文占用    | 估算 <code>messages</code> 序列化后的 token |
| 软阈值 ~87% | 触发压缩，留出响应余量（见 <a href="#">4.4</a> ）  |
| 硬上限 100% | 再请求可能被拒或严重截断                         |

[Mermaid 图表 — 请在在线版本中查看]

## 估算 vs 精确

| 方法                           | 精度   | 成本    |
|------------------------------|------|-------|
| 本地 tokenizer 估算              | 中    | 低     |
| 依赖 API 返回 <code>usage</code> | 高    | 已付请求费 |
| 混合：估算 + 周期校准                 | 工程常用 | 中     |

```
// 教学伪代码：预算检查点之一
function tokenBudgetOk(state: State, model: string): BudgetVerdict {
  const estimated = estimateTokens(state.messages, model);
  const limit = getContextWindow(model); // 如 200_000
  if (estimated >= limit) {
    return { ok: false, reason: "token_window_exceeded" };
  }
  return { ok: true };
}
```

## 关卡二：金钱预算（\$ 限制）

部分部署形态允许配置 单次会话 / 日 / 月 的费用上限，或对接 组织预算 API。

| 信号来源   | 说明                              |
|--------|---------------------------------|
| 配置项    | <code>maxCostUsd</code> 等（名称示意） |
| 用量 API | 云端账单聚合（延迟更高）                    |
| 本地累计   | 按 <code>usage * 价格表</code> 近似   |

生活类比：手机 流量封顶——未到月底也可能被停，因为「本地计数器」说你已经超了。

[Mermaid 图表 — 请在在线版本中查看]

## 教学伪代码

```
type PriceTable = {
  inputPerMtokUsd: number;
  outputPerMtokUsd: number;
};

function addCost(state: State, usage: UsageDelta, prices: PriceTable) {
  const dollars =
    (usage.input_tokens / 1_000_000) * prices.inputPerMtokUsd +
    (usage.output_tokens / 1_000_000) * prices.outputPerMtokUsd;
  state.usage.estimated_cost_usd += dollars;
}

function moneyBudgetOk(state: State, cap?: number): BudgetVerdict {
  if (cap == null) return { ok: true };
  if (state.usage.estimated_cost_usd > cap) {
    return { ok: false, reason: "money_budget_exceeded" };
  }
  return { ok: true };
}
```

## 关卡三：轮次限制 maxTurns

一轮通常指 queryLoop 的一次完整迭代：准备消息 → 调模型 → （可能有）工具 → 写回历史。

| 为何需要？   | 说明         |
|---------|------------|
| 防无限循环   | 模型反复调用同一工具 |
| 防成本爆炸   | 每轮都花钱      |
| 防 UX 失控 | 用户以为「卡死」   |

```
function turnBudgetOk(state: State, maxTurns: number): BudgetVerdict {
  if (state.turn > maxTurns) {
    return { ok: false, reason: "max_turns_exceeded" };
  }
  return { ok: true };
}
```

生活类比：出租车 **跳表次数** 有上限——不是路不能继续开，而是 **产品规则** 说「本次行程最多绕路 N 圈」。

[Mermaid 图表 — 请在在线版本中查看]

## 检查时机：放在循环的哪个节拍？

实现可能在 多个点 调用 `checkBudgets`，教学中记住「出大钱之前」即可。

| 时机                          | 理由                     |
|-----------------------------|------------------------|
| 每轮开始                        | 防止上下文已炸还请求             |
| 收到 <code>assistant</code> 后 | 已产生新用量                 |
| 工具执行前                       | 防止并行工具放大 I/O 与后续 token |

[Mermaid 图表 — 请在在线版本中查看]

### 聚合伪代码

```
type BudgetVerdict =
  | { ok: true }
  | { ok: false; reason: string };

function checkBudgets(
  state: State,
  cfg: { maxTurns: number; moneyCapUsd?: number; model: string },
): BudgetVerdict {
  const a = turnBudgetOk(state, cfg.maxTurns);
  if (!a.ok) return a;

  const b = tokenBudgetOk(state, cfg.model);
  if (!b.ok) return b;

  const c = moneyBudgetOk(state, cfg.moneyCapUsd);
  if (!c.ok) return c;

  return { ok: true };
}
```

## 与压缩的协同：软 vs 硬

| 机制         | 类型 | 目的          |
|------------|----|-------------|
| 87% 压缩     | 软  | 腾空间，尽量避免硬失败 |
| Token 预算超限 | 硬  | 再跑会失败或浪费钱   |
| 熔断         | 硬  | 压缩自己失败太多次   |

[Mermaid 图表 — 请在在线版本中查看]

## 用户可见文案 vs 内部原因码

| reason（内部）            | 用户文案方向            |
|-----------------------|-------------------|
| token_window_exceeded | 上下文过长，请开启新会话或精简文件 |
| money_budget_exceeded | 已达费用上限，请联系管理员调整   |
| max_turns_exceeded    | 本轮交互步数过多，请拆分任务    |

## 与八步循环的对照

| 八步 | 本节           |
|----|--------------|
| 6  | 检查预算 —— 三重关卡 |

## 小结

- Token / 金钱 / 轮次 是三道独立闸门，逻辑 OR 式失败：一个不行就停。
- 87% 压缩 是 避免 触发 token 硬限制的「软护栏」。
- 预算检查应 贴近花钱点，并输出 可诊断的 reason。

下一篇：[4.9 循环终止条件](#)。

# 4.9 循环终止条件：何时「关火收工」

### 本节学习目标

- 列举 至少五种 正常与异常终止路径。
- 能把 「无工具」正常退出 与 预算耗尽 从 UI 表现上区分开。
- 理解 用户中断 如何协作式地打断 `async generator`。



## 终止 ≠ 失败：先分「出锅」与「糊锅」

| 类型        | 是否成功    | 用户心理    |
|-----------|---------|---------|
| 正常终止（无工具） | 成功      | 「说完了」   |
| 用户主动取消    | 中性      | 「我不想等了」 |
| 预算耗尽      | 受限成功/失败 | 「被规则挡住」 |
| 致命错误      | 失败      | 「出问题了」  |

[Mermaid 图表 — 请在在线版本中查看]

### 条件一：正常终止——本轮无 `tool_use`

当模型输出的 `assistant` 消息 不再包含 任何 `tool_use` 块，QueryEngine 认为「推理闭环已闭合」。

[Mermaid 图表 — 请在在线版本中查看]

#### 伪代码锚点

```
const toolUses = extractToolUses(assistantMsg);
if (toolUses.length === 0) {
  return {
    status: "completed",
    finalText: extractVisibleText(assistantMsg),
    usage: state.usage,
  };
}
```

生活类比：厨师尝完汤说「可以上桌了」，没有再开采购单——这一桌结束。

### 条件二：预算耗尽（三重其一）

详见 4.8。终止时 `status` 应携带 `reason`，便于 UI 与日志一致。

| reason                             | 典型 UI      |
|------------------------------------|------------|
| <code>token_window_exceeded</code> | 建议压缩 / 新会话 |
| <code>money_budget_exceeded</code> | 费用上限       |
| <code>max_turns_exceeded</code>    | 步数上限       |

[Mermaid 图表 — 请在在线版本中查看]

### 条件三：用户中断（协作式取消）

CLI 中常见 `AbortController` 或 信号（SIGINT）。QueryEngine 应在：

- `fetch` 层绑定 `signal`；
- 工具执行层支持超时/取消；
- `yield` 之间检查 `cancelled` 标志。

[Mermaid 图表 — 请在在线版本中查看]

### 教学伪代码

```
async function* queryLoop(state: State, ctx: QueryContext) {
  while (true) {
    if (ctx.abortSignal.aborted) {
      return { status: "cancelled", reason: "user_abort" };
    }
    // ... 八步 ...
  }
}
```

注意：协作式意味着：若某工具 忽略 `signal`，终止会「慢半拍」——这是工程现实，不是理论缺陷。

## 条件四：致命 API / 鉴权错误

| 场景       | 是否重试 | 终止方式                         |
|----------|------|------------------------------|
| 401/403  | 否    | 立即返回，明确提示检查密钥                |
| 400 结构错误 | 通常否  | 带 <code>request_id</code> 记录 |
| 模型不存在    | 否    | 配置错误                         |

[Mermaid 图表 — 请在在线版本中查看]

## 条件五：压缩熔断（连续失败）

当自动压缩 连续失败 达到阈值（教学中 3 次），继续对话风险是 无限烧钱 + 无限错误。

```
if (state.compactionCircuitOpen) {
  return {
    status: "aborted",
    reason: "compaction_circuit_breaker",
  };
}
```

生活类比：吸尘器 三次堵死 还硬吸，会烧电机——必须 停机清灰。

## 条件六：空安全与实现守护（少见但真实）

| 守护             | 说明                                      |
|----------------|-----------------------------------------|
| 历史损坏           | 无法修复的 <code>messages</code> → 终止并建议重置会话 |
| 工具注册表为空但模型强要工具 | 降级或终止（视产品策略）                            |

QueryResult

形状（教学）

```
type QueryResult = {
  status:
    | "completed"
    | "cancelled"
    | "budget_exceeded"
    | "fatal_error"
    | "compaction_circuit_breaker";
  reason?: string;
  finalText?: string;
  usage: UsageCounters;
  diagnostics?: { requestId?: string };
};
```

| 字段          | 用途         |
|-------------|------------|
| status      | UI 主分支     |
| reason      | 细粒度分析与翻译文案 |
| diagnostics | 对接支持工单     |

与 async generator 的 return

消费者侧（简化）：

```
const it = query(input, ctx);
for await (const ev of it) {
  render(ev);
}
// TS 里获取 generator return 值需额外 API；真实项目常把 result 放在最后一条 StreamEvent
```

教学中记住：终止 要么体现为 最后的 `StreamEvent: done`，要么为 `query()` `Promise` 解析值——具体以源码为准。

[Mermaid 图表 — 请在在线版本中查看]

## 终止后的世界：资源清理

| 资源        | 动作                        |
|-----------|---------------------------|
| HTTP 连接   | <code>finally</code> 里释放  |
| 子进程       | <code>kill</code> 或等待优雅退出 |
| 临时文件      | 清理 sandbox                |
| Telemetry | <code>flush()</code>      |

## 与八步循环的对照

| 八步 | 终止关联        |
|----|-------------|
| 6  | 预算失败 → 终止   |
| 8  | 无工具 → 正常终止  |
| 4  | 不可恢复错误 → 终止 |
| 1  | 压缩熔断 → 终止   |

## 小结

- 正常终止 的核心判据：无 `tool_use` 。
- 预算与用户 是两大「外部刹车」。
- 熔断与鉴权 是「不能再自欺欺人往下跑」的 硬出口。

下一篇：[4.10 Thinking 模式](#)。

# 4.10 Thinking 模式：Extended Thinking 与 `thinking` 块

### 本节学习目标

- 解释 **Extended Thinking** 在 API 层面的基本语义（模型先「想」再答）。
- 理解 `thinking` 内容块 与 `text` / `tool_use` 的共存关系。

- 评估 Thinking 对 工具调用时机、token 计费、UI 展示 的影响。

## 为何需要「先想后说」？

常规模型输出是 直接文本；复杂推理任务中，用户希望看到 中间推理链（或至少让模型在内部有更长的推理预算）。Extended Thinking 让模型可以产出 `thinking` 块——在教学上可视为「草稿纸」。

生活类比：数学题 草稿纸 与 答题卡 分离——草稿可以不交，但答题卡要工整；Thinking 类似 草稿，`text` 类似 正式答案。

[Mermaid 图表 — 请在在线版本中查看]

## 消息内容中的块类型扩展

在 4.4 的基础上，增加：

```
type ThinkingBlock = {
  type: "thinking";
  thinking: string; // 或结构化字段，视 API 版本而定
};

type AssistantContentBlock =
  | { type: "text"; text: string }
  | ThinkingBlock
  | ToolUseBlock;
```

| 块类型                   | UI 默认       | 是否进入用户剪贴板 | 典型策略      |
|-----------------------|-------------|-----------|-----------|
| <code>thinking</code> | 可折叠 / 仅调试可见 | 依产品设置     | 尊重用户隐私与简洁 |
| <code>text</code>     | 完整展示        | 是         | 主通道       |
| <code>tool_use</code> | 显示为工具卡片     | 是（元数据）    | 与执行器绑定    |

## 流式事件：Thinking 也有 delta

与 `text_delta` 类似，可能存在 `thinking_delta`（名称以 SDK 为准）。

[Mermaid 图表 — 请在在线版本中查看]

### 教学伪代码

```
for await (const ev of stream) {
  if (isThinkingDelta(ev)) {
    yield { kind: "thinking_delta", text: ev.delta.thinking };
  }
  if (isTextDelta(ev)) {
    yield { kind: "assistant_text_delta", text: ev.delta.text };
  }
}
```

### 对工具调用的影响：「想完再动手」？

| 行为                | 说明                                               |
|-------------------|--------------------------------------------------|
| 同一轮内顺序            | 常见顺序： <b>thinking</b> → <b>text/tool</b> （非严格保证） |
| 多 <b>tool_use</b> | Thinking 可能 <b>预先规划</b> 多个调用                     |
| 错误恢复              | 若 thinking 显示「我搞错了」，模型可能在 <b>下一轮</b> 自纠          |

[Mermaid 图表 — 请在在线版本中查看]

**教学结论：**QueryEngine 不必等 thinking 结束才开始收集 **tool\_use**——收集逻辑仍 以块类型为准；但 UI 可能 延迟高亮工具卡片 直到 thinking 折叠完成（产品决策）。

### Token 与计费：Thinking 通常不免费

| 维度   | 影响                             |
|------|--------------------------------|
| 输入窗口 | thinking 文本占用 <b>输出侧</b> token |
| 费用   | 常计入 <b>output tokens</b>       |
| 压缩   | 长 thinking 加速触达 <b>87% 阈值</b>  |

**生活类比：**草稿纸也用本子页数——**页数（token）** 会计入总消耗。

## 与历史压缩的摩擦

压缩策略需要决定：**是否保留 thinking**。

| 策略 | 优点      | 缺点    |
|----|---------|-------|
| 保留 | 调试价值高   | 占空间   |
| 丢弃 | 省 token | 丢推理轨迹 |
| 摘要 | 平衡      | 实现复杂  |

[Mermaid 图表 — 请在在线版本中查看]

## 安全与隐私：Thinking 可能更「直白」

| 风险     | 缓解                               |
|--------|----------------------------------|
| 泄露内部策略 | 默认不记录到共享日志                       |
| 误导用户   | UI 标注「模型思考过程，非最终结论」              |
| 合规     | 企业版可 <b>关闭</b> Extended Thinking |

## QueryEngine 集成检查清单

| 检查项                          | 说明                                                |
|------------------------------|---------------------------------------------------|
| 解析器                          | 识别 <code>thinking</code> 起止事件                     |
| Accumulator                  | 与 <code>text</code> 、 <code>tool_use</code> 同索引管理 |
| <code>extractToolUses</code> | 忽略 thinking，只扫 <code>tool_use</code>              |
| <code>yield</code>           | 单独事件类型，供 UI 订阅                                    |
| 预算                           | 将 thinking token 计入 <code>usage</code>            |



## 与八步循环的映射

| 八步  | Thinking 相关     |
|-----|-----------------|
| 2~3 | 流式解析新增 delta 类型 |
| 1   | 压缩是否保留 thinking |
| 6   | token 预算更快触线    |

## 小结

- Extended Thinking 通过 `thinking` 块 暴露或强化推理过程。
- QueryEngine 把它当作 另一类 `content` 块 做流式累积与 `yield`。
- 工具调用 仍以 `tool_use` 为准；Thinking 主要影响 体验、计费与压缩策略。

## API / SDK 侧：如何「打开」Extended Thinking（概念）

具体字段名以 当前 SDK 与模型能力 为准，教学中可记三层：

| 层级   | 示意                                                                   |
|------|----------------------------------------------------------------------|
| HTTP | 请求体中带 扩展思考开关（或模型族隐式支持）                                               |
| SDK  | <code>client.messages.stream({ ..., /* thinking options */ })</code> |
| 响应   | 流里出现 <code>thinking</code> 相关事件                                      |

[Mermaid 图表 — 请在在线版本中查看]

本书不绑定某一泄露版本的 **精确字段名**，避免与你本地重建仓库不一致；你在 `query.ts` 附近搜索 `thinking`、`extended` 即可锚定。

## UI 策略矩阵

| 用户画像  | 推荐展示                            |
|-------|---------------------------------|
| 默认开发者 | 折叠面板 + 「显示思考过程」快捷键              |
| 演示模式  | 仅显示 <code>text</code> ，避免泄露中间推理 |
| 调试模式  | 完整 thinking + 原始事件时间线           |

[Mermaid 图表 — 请在在线版本中查看]

## 与工具治理的交叉：Thinking 里「计划」是否算承诺？

| 观点     | 工程处理                                       |
|--------|--------------------------------------------|
| 计划可变更  | 以实际 <code>tool_use</code> 为准，thinking 仅为解释 |
| 计划误导用户 | UI 提示「可能修订」                                |

这能避免 把草稿当合同 的法律/体验风险。

## 调试清单：Thinking 相关 bug 从哪查？

- 事件类型未识别 → 流解析器缺分支，`thinking` 被丢弃。
- 块顺序错乱 → `index` 与 `content_block_start` 映射错误。
- `token` 暴涨 → 长 thinking 未参与压缩策略。
- 工具未触发 → 误以为 thinking 结束才算「模型说完」— 应检查 `tool_use` 是否已闭合。

[Mermaid 图表 — 请在在线版本中查看]

## 与异步生成器：`yield` 顺序保证

Thinking delta 与 text delta 交错 `yield` 时，UI 应 按时间序 渲染；不要在客户端 重排 为「全文 thinking 在前」除非产品明确要求。

| 做法   | 结果           |
|------|--------------|
| 保序   | 与模型真实节奏一致    |
| 强行重排 | 可能 误导 用户理解因果 |

下一篇：[4.11 并行工具执行器](#)。

## 4.11 并行工具执行器：`isConcurrencySafe` 与 `Promise.all`

### 本节学习目标

- 解释 **为何** 默认谨慎串行，**何时** 可以并行。
- 理解 `isConcurrencySafe`（或等价标记）在注册表中的角色。
- 能写出 **分区算法** 的教学伪代码：`Promise.all` + 串行尾部。

### 工具执行不是「越多越好并发」

并行能 **降低墙钟时间**，但会引入：

| 风险    | 例                                                |
|-------|--------------------------------------------------|
| 竞态    | 两个 <code>edit</code> 写同一文件                       |
| 依赖    | <code>read_file</code> 必须在 <code>write</code> 之后 |
| 副作用顺序 | 两个 <code>bash</code> 改同一全局状态                     |
| 权限风暴  | 同时弹 10 个确认框                                      |

生活类比：洗碗机 **可以** 和洗衣机 **并行**；但「切生肉」与「拌即食沙拉」**不应** 并行——交叉污染。

[Mermaid 图表 — 请在在线版本中查看]

## isConcurrencySafe：注册表上的「绿灯」

每个工具在注册时携带元数据（名称示意）：

```
type ToolDefinition = {
  name: string;
  description: string;
  inputSchema: unknown;
  isConcurrencySafe: boolean; // 核心标记
  handler: (input: unknown, ctx: ToolContext) => Promise<unknown>;
};
```

| isConcurrencySafe | 含义（教学口径）            |
|-------------------|---------------------|
| true              | 无交叉副作用，或副作用隔离；只读类优先 |
| false             | 可能写共享资源、需顺序语义       |

[Mermaid 图表 — 请在在线版本中查看]

## 并行 vs 串行：决策流程图

[Mermaid 图表 — 请在在线版本中查看]

## 保守策略（默认）

即使标记为 safe，也可加 **全局上限**：

| 护栏               | 说明               |
|------------------|------------------|
| maxParallelTools | 例如最多 4 个         |
| 资源键冲突检测          | 对 path 参数做归一化比较  |
| 用户权限模式           | acceptEdits 下更保守 |

## 教学伪代码：分区 + 执行

```
type PlannedGroup =
  | { mode: "parallel"; items: ToolUseBlock[] }
  | { mode: "serial"; items: ToolUseBlock[] };

function planToolExecution(
  uses: ToolUseBlock[],
  registry: ToolRegistry,
): PlannedGroup[] {
  const groups: PlannedGroup[] = [];
  let buffer: ToolUseBlock[] = [];

  const flushParallel = () => {
    if (buffer.length) {
      groups.push({ mode: "parallel", items: buffer });
      buffer = [];
    }
  };

  for (const u of uses) {
    const def = registry.get(u.name);
    const safe = def?.isConcurrencySafe ?? false;

    if (safe) {
      buffer.push(u);
    } else {
      flushParallel();
      groups.push({ mode: "serial", items: [u] });
    }
  }
  flushParallel();
  return groups;
}

async function executePlan(
  plan: PlannedGroup[],
  ctx: QueryContext,
  mode: PermissionMode,
): Promise<ToolResultBlock[]> {
  const out: ToolResultBlock[] = [];

  for (const g of plan) {
    if (g.mode === "parallel") {
      const chunk = await Promise.all(
        g.items.map((u) => runSingleTool(u, ctx, mode)),
      );
      out.push(...chunk);
    } else {
      for (const u of g.items) {
        out.push(await runSingleTool(u, ctx, mode));
      }
    }
  }
}
```

```
    return out;
  }
```

Promise.all

失败语义

| 行为     | 说明                                     |
|--------|----------------------------------------|
| 一个工具抛错 | 整组 reject（默认）                          |
| 工程变体   | <code>Promise.allSettled</code> 收集部分失败 |

QueryEngine 常选择：将错误封装进 `tool_result`，让模型 下一轮自纠，而不是整个进程崩溃。

[Mermaid 图表 — 请在在线版本中查看]

```
async function runSingleTool(
  u: ToolUseBlock,
  ctx: QueryContext,
  mode: PermissionMode,
): Promise<ToolResultBlock> {
  try {
    const data = await executeWithPermission(u, ctx, mode);
    return { type: "tool_result", tool_use_id: u.id, content: data };
  } catch (e) {
    return {
      type: "tool_result",
      tool_use_id: u.id,
      content: { error: String(e) },
    };
  }
}
```

与权限系统的时序

并行工具可能 同时 请求权限决策：

[Mermaid 图表 — 请在在线版本中查看]

UI 需 合并或排队 弹窗，避免 竞态导致的错点。

## 与 4.6 工具收集 的边界

| 阶段     | 职责                                |
|--------|-----------------------------------|
| 收集     | 产出 有序 <code>ToolUseBlock[]</code> |
| 本节的执行器 | 在 不改变语义顺序 的前提下 重排执行调度             |

注意: `tool_result` 在消息里仍应按 API 要求 与 `tool_use` 对齐; 并行只是 墙钟优化。

## 与八步循环的对照

| 八步 | 本节               |
|----|------------------|
| 5  | 执行工具 —— 并行化发生在这里 |

## 小结

- 并行 是优化; 正确性 优先靠 `isConcurrencySafe` + 资源冲突检测。
- `Promise.all` 适合 同质、独立只读 工具群。
- 错误应 落袋为 `tool_result`, 维持 QueryEngine 循环韧性。

下一篇: [4.12 协作全景图](#)。

# 4.12 协作全景图: QueryEngine 在系统中的位置

### 本节学习目标

- 画出 QueryEngine 与 工具、权限、UI、压缩、记忆 模块的 依赖方向。
- 理解 为何 QueryEngine 不应「什么都自己实现」, 而应 编排。
- 能向他人讲解: 一次用户回车 触发的跨模块调用链。

## 一张总图：心脏与五脏六腑

[Mermaid 图表 — 请在在线版本中查看]

生活类比：QueryEngine 是 指挥；乐手是各子系统。指挥 不亲自拉小提琴，但决定 何时齐奏、何时独奏。

## 与工具系统：注册表 + 执行器

| 交互点                      | 方向            | 说明                           |
|--------------------------|---------------|------------------------------|
| 序列化 <code>tools[]</code> | QE → API      | 来自 <code>ToolRegistry</code> |
| <code>tool_use</code> 解析 | API → QE      | 在流式累积器完成                     |
| 真正执行                     | QE → Executor | <a href="#">4.11</a>         |
| 结果回注                     | Executor → QE | <code>tool_result</code> 块   |

[Mermaid 图表 — 请在在线版本中查看]

## 与权限系统：闸门在「执行前」

| 若无权限模块   | 后果   |
|----------|------|
| 模型可随意删文件 | 安全事故 |
| 无法审计     | 合规失败 |

[Mermaid 图表 — 请在在线版本中查看]

QueryEngine 不 内嵌复杂策略，而是调用 `askPolicy(tool, input, mode)` 式接口——具体模式见本书权限篇。



## 与 UI: StreamEvent 是契约

| 事件类型（示意）                          | UI 行为    |
|-----------------------------------|----------|
| <code>assistant_text_delta</code> | 增量打字     |
| <code>tool_card</code>            | 展示工具名与参数 |
| <code>permission_prompt</code>    | 弹确认      |
| <code>budget_warning</code>       | 黄条提示     |
| <code>done</code>                 | 显示用量与耗时  |

[Mermaid 图表 — 请在在线版本中查看]

解耦价值：同一 QueryEngine 可接 CLI / VSCode 插件 / 自动化测试桩。

## 与压缩系统：步骤 1 的外包

| 职责划分                     | QueryEngine | Compaction 模块 |
|--------------------------|-------------|---------------|
| 决定 何时 压                  | ✓（阈值判断）     |               |
| 决定 怎么 压                  |             | ✓（摘要算法、调用模型）  |
| 写回 <code>messages</code> | ✓（接收产物）     |               |

[Mermaid 图表 — 请在在线版本中查看]

详见：[4.4](#)。

## 与记忆系统：`CLAUDE.md`、会话持久化

| 数据源                       | 注入点                                          |
|---------------------------|----------------------------------------------|
| 项目 <code>CLAUDE.md</code> | <code>system</code> 或首条 <code>user</code> 前缀 |
| 用户偏好                      | <code>QueryContext.config</code>             |
| 跨会话记忆                     | 上层存储（若启用）                                    |

[Mermaid 图表 — 请在在线版本中查看]

QueryEngine 读取 上下文，但 不负责 长期学习的伦理与策略——那是产品与合规层的决策。

## 与 Hooks / MCP（预告）

| 扩展    | 典型挂钩点                     |
|-------|---------------------------|
| Hooks | 工具前后、会话开始结束               |
| MCP   | 工具作为 远程注册 注入 ToolRegistry |

[Mermaid 图表 — 请在在线版本中查看]

本书后续篇章会展开；此处只需 定位：QueryEngine 是 编排中心，MCP 是 工具供应侧。

## 反模式：QueryEngine 变成「上帝对象」

| 反模式                     | 症状     | 正解                   |
|-------------------------|--------|----------------------|
| 把 tokenizer 粘进 query.ts | 文件无限膨胀 | 独立 token-estimate 模块 |
| UI 逻辑混入循环               | 难测试    | yield 纯事件            |
| 权限 if-else 散落           | 难审计    | PermissionPipeline   |

## 端到端：用户按一次回车的故事

[Mermaid 图表 — 请在在线版本中查看]

## 小结

- QueryEngine 位于中心，但 能力向外委托：工具、权限、压缩、记忆、UI 各司其职。
- StreamEvent 是跨层契约；messages[] 是跨轮契约。

- 读懂协作图，你就读懂了 51 万行仓库的主干叙事。

本篇回顾： [返回 4.1 索引](#)。

---

## 第5篇：提示词工程

# 第 5 篇：提示词工程（Prompt Engineering）

Claude Code 完全指南 V2 · 全新篇章 · 从「写一段提示」到「理解系统如何拼装提示」

## 本篇学习目标

完成本篇后，你应能够：

1. **解释** `getSystemPrompt()` 如何把多段文本「编译」成最终系统提示词，而不是把它当成单一静态字符串。
2. **区分** 静态「系统宪法」与动态「当期政策」的职责边界，并说明各自对缓存与行为的影响。
3. **读懂** `SYSTEM_PROMPT_DYNAMIC_BOUNDARY` 在工程上的意图：把可缓存前缀与每次必变后缀切开。
4. **预判** 哪些用户侧或环境侧变化会让提示词缓存失效，从而推高 Token 成本。
5. **对照** 行为约束与工具手册在提示词中的角色：前者是铁律，后者是可检索的操作说明。

## 5.1 动态拼装机制：系统提示不是「一篇文章」

### 生活类比：餐厅后厨的「固定菜谱 + 当日特供」

想象一家餐厅：

- **固定菜谱**（对应静态部分）：刀工规范、卫生标准、上菜顺序、口味基调——**天天一样**，可以印在墙上长期不改。
- **当日特供**（对应动态部分）：今天进了什么海鲜、哪桌客人过敏、现在几点是否还能做复杂菜——**每餐不同**，必须写在小黑板上。

Claude Code 的系统提示词同理：底层「怎么做 AI 员工」是宪法；「此刻为谁、在什么环境、读到了什么记忆」是政策。二者拼在一起，才是模型真正看到的系统消息。

## 核心论断：`getSystemPrompt()` 像编译器，不是「打开一个 .txt」

在工程实现上，系统提示词通常由函数 **动态拼装**：

- 多个模块各自产出片段（身份、规范、工具说明、环境、记忆……）。
- 按固定顺序 **concat**（或模板填充）成一个大字符串。
- 可能还有 **条件分支**（例如内部用户类型、是否启用 MCP）。

因此，你在文档里看到的「系统提示词示例」往往是 **某次运行的快照**；源码里真正重要的是 **拼装管线** 与 **边界标记**。

## 拼装流程概览（概念伪代码）

下面用 TypeScript 风格伪代码表达「编译器式」输出（教学用，非某版本逐行源码）：

```
// 概念示意：系统提示 = 静态缓存前缀 + 动态后缀
function getSystemPrompt(ctx: SessionContext): string {
  const staticCore = buildStaticConstitution(); // 身份、规范、哲学、风险、工具总则、语气

  const dynamicTail = [
    buildSessionPreamble(ctx),
    buildMemoryInjections(ctx.memories, { max: 5 }),
    buildEnvironmentBlock(ctx.env),
    loadClaudeMd(ctx.workspace),
    buildMcpPluginSection(ctx.mcp),
    buildTokenBudgetHint(ctx.budget),
  ].join("\n\n");

  return (
    staticCore +
    "\n\n" +
    SYSTEM_PROMPT_DYNAMIC_BOUNDARY +
    "\n\n" +
    dynamicTail
  );
}
```

要点：

- **静态块** 尽量稳定，便于 **prompt caching**（前缀命中）。
- **动态块** 每轮/每会话可能变化，放在 **边界标记之后**，避免「污染」整个前缀缓存。

## Mermaid：从「模块」到「最终 system 字符串」

[Mermaid 图表 — 请在在线版本中查看]

Mermaid: `getSystemPrompt()` 数据流（编译器视角）

[Mermaid 图表 — 请在在线版本中查看]

为什么要这样设计？（表格小结）

| 维度  | 若全是静态          | 若全是动态       | 静动分离         |
|-----|----------------|-------------|--------------|
| 缓存  | 易命中，但无法感知环境    | 难命中，成本高     | 前缀命中 + 后缀更新  |
| 一致性 | 行为稳，但「不知此刻上下文」 | 上下文全，但规则易漂移 | 规则稳、上下文新     |
| 可测试 | 易测静态块          | 难测组合爆炸      | 分测静态生成器与动态注入 |

与本篇其他章节的关系

| 章节   | 文件                                      | 你将深入的内容                            |
|------|-----------------------------------------|------------------------------------|
| 5.2  | <code>02-static-constitution.md</code>  | 六大静态模块各自写什么、为何不变                   |
| 5.3  | <code>03-dynamic-policy.md</code>       | 六大动态注入如何塑造「当期智能」                   |
| 5.4  | <code>04-cache-boundary.md</code>       | 边界标记如何把缓存风险「关进后院」                  |
| 5.5  | <code>05-token-economics.md</code>      | 缓存单价与长对话成本估算                       |
| 5.6  | <code>06-cache-pitfalls.md</code>       | 七种失效陷阱与规避                          |
| 5.7  | <code>07-behavior-constraints.md</code> | 行为铁律、工具死规矩、Fail-closed             |
| 5.8  | <code>08-tool-manuals.md</code>         | 每工具 <code>prompt.ts</code> 如何写给模型读 |
| 5.9  | <code>09-comparison.md</code>           | 与 OpenAI / Google 工具策略对照           |
| 5.10 | <code>10-practice.md</code>             | 动手设计迷你 Agent 系统提示                  |

自测清单（学完 5.1 应能回答）

- 为什么说「系统提示词 = 编译器输出」比「提示词 = 配置文件」更贴切？
- 静态块与动态块若顺序颠倒，对缓存与一致性分别有什么风险？
- 若把「环境信息」误放进静态块，会带来什么工程后果？

## 本篇路线图（章节依赖）

[Mermaid 图表 — 请在在线版本中查看]

建议阅读顺序：5.1 → 5.2 → 5.3 → 5.4，再按需跳读 5.5—5.8；5.9 适合与架构师讨论选型；5.10 适合工作坊收尾。

## 术语速查（本篇口径）

| 术语                             | 含义                                             |
|--------------------------------|------------------------------------------------|
| 系统宪法                           | 静态、低频变更的系统提示前缀，承载身份与底线规范                       |
| 当期政策                           | 动态、高频变更的后缀，承载环境与会话事实                           |
| <code>getSystemPrompt()</code> | 将多段文本拼装为最终 system 字符串的入口（教学名）                  |
| Fail-closed                    | 元数据未显式声明安全能力时，按「不安全/不并行」保守处理                   |
| ToolSearchTool                 | 按需检索工具手册（如各工具 <code>prompt.ts</code> ）的机制（教学名） |

## 常见误解澄清

| 误解                    | 实际情况                            |
|-----------------------|---------------------------------|
| 「系统提示词就是运维写死的几页 Word」 | 多为 运行时拼装，且分静动两层                 |
| 「提示词越长模型越聪明」          | 过长前缀 稀释注意力、抬高成本；应用 分层 + 延迟手册    |
| 「缓存是云厂商魔法，与提示词无关」     | 前缀稳定性 直接决定命中率与账单                |
| 「只要禁止 cat/sed 就安全了」   | 还需 Git 协议、盲改拦截、Fail-closed 等工程层 |

## 与「工具调用」章节的边界（预告）

本篇聚焦 system 字符串如何构成；具体 JSON Schema、流式 tool\_calls、错误重试 属于 API 与 宿主实现细节。阅读时请记住：提示词定义倾向，代码定义底线。

## 编者注

---

- 函数名、常量名（如 `getSystemPrompt`、`SYSTEM_PROMPT_DYNAMIC_BOUNDARY`）与真实代码库可能逐字一致也可能不同，请以你所用版本为准。
  - 价格与缓存规则以 **供应商当期文档** 为准；本篇算例仅建立 **数量级直觉**。
- 

## 导航

---

- **上一篇**：（依全书目录链接）
  - **下一篇**：[5.2 静态部分：系统宪法](#)
- 

本篇为 V2 教学文稿，技术细节以官方实现与版本更新为准；文中伪代码与标记名用于对齐常见架构表述。

---

## 5.2 静态部分：系统宪法（System Constitution）

---

### 学习目标

---

- 说出 Claude Code 风格系统提示中 **静态六大块** 各自解决什么问题。
  - 解释 **为何** 静态块要尽量稳定（与 prompt cache、行为一致性相关）。
  - 能判断某段新文案应归入静态还是动态，避免破坏缓存结构。
  - 用一张表对比「宪法」与「政策」在失败时的表现差异。
- 

### 生活类比：国家的宪法 vs 每日新闻头条

---

- **宪法**：公民基本权利、政府组织原则——**不常改**，改了要全民严肃讨论。
- **头条**：今天股市、天气、突发事件——**天天变**，但不该推翻宪法里的底线。

静态系统提示就是 **AI 雇员的宪法**：无论今天在哪台机器、哪个仓库、哪次会话，**底线能力、工具纪律、风险意识、语气** 都应一致。这样产品体验可预期，工程上还能让 **长前缀缓存** 持续命中。

---



## 静态六大组成部分（教学模型）

以下为与 Claude Code 架构 **对齐的常见拆分**；具体段落标题以实际实现为准。

### 1. 身份定位（Who am I）

**作用：**锚定模型在产品中的角色——例如「终端里的编程助手」「可调用工具的 Agent」，而非泛泛的聊天机器人。

**为何静态：**

- 角色漂移会导致用户困惑（一会儿像导师，一会儿像客服）。
- 身份描述通常 **不随仓库变化**，适合进入缓存前缀。

**典型内容方向：**

- 与产品名、场景（CLI / IDE / Web）一致的自我描述。
- 「你不是通用搜索引擎替代品」这类 **边界声明**（若产品需要）。

### 2. 系统规范（Operational norms）

**作用：**定义 **工作方式**：如何规划、如何汇报、如何对待用户指令优先级。

**为何静态：**

- 属于 **组织级纪律**，不应因「今天多装了一个 MCP」而改变。
- 与身份一起构成 **稳定行为 OS**。

**典型内容方向：**

- 先理解再动手 vs 直接执行的默认策略（产品选择）。
- 对用户不透明操作的禁止或限制（视安全模型而定）。

### 3. 任务哲学（Task philosophy）

**作用：**回答「什么叫把事做完」——例如：**聚焦用户请求、反对范围蔓延、反对为炫技而抽象**。

**为何静态：**

- 这是 **价值观层** 约束，和具体任务无关。
- 常与 `getSimpleDoingTasksSection` 类 **短平快铁律** 一起出现（详见 [5.7](#)）。

---

## 4. 风险规范 (Risk & safety rails)

作用：在 不替代完整安全审计 的前提下，降低删库、泄密、误操作概率。

为何静态：

- 风险底线 不应 因会话而放松。
- 与合规、内部红线一致，属于长期文本。

典型内容方向：

- 对密钥、令牌、隐私数据的处理原则。
  - 对破坏性命令（如 `rm -rf`、强制 Git 操作）的态度（具体禁令常落在工具手册与 Bash 协议中）。
- 

## 5. 工具使用规范 (Tools — global rules)

作用：跨工具 的总则：何时并行、何时必须先读后写、Fail-closed 默认值等。

为何静态：

- 工具 名单与细则 可能动态加载，但 总则（纪律）通常稳定。
- 避免模型在不同会话里一会儿「爱用 cat」一会儿「爱用 read」。

与 5.8 的关系：

- 本章是 交通规则；`prompt.ts` 是 每辆车的说明书。
- 

## 6. 语气风格 (Voice & tone)

作用：统一 输出体验：简洁/教学式/正式、是否默认中文、列表偏好等。

为何静态：

- 品牌与 UX 要求 长期一致。
  - 不宜让「今天记忆片段」偶然改写全局语气（除非产品刻意允许）。
-

## Mermaid：静态块在拼装管线中的位置

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid：静态 vs 动态在「失败模式」上的差异

[Mermaid 图表 — 请在在线版本中查看]

## 为什么「不变」如此重要？（工程视角）

### 1. Prompt caching 的前缀稳定性

若静态块频繁因 小改动（例如加时间戳、插入随机 ID）而整体变化：

- 缓存 前缀无法命中。
- 长对话下输入 Token 费用 线性放大。

结论：静态块应像 \*\* semver 主版本\*\* 一样谨慎修改；日常波动应塞进动态块。

### 2. 行为一致性与可调试性

当用户反馈「昨天和今天表现不一致」时：

- 静态稳定 → 更容易归因到 动态注入（记忆、环境、[CLAUDE.md](#)、MCP）。
- 静态乱动 → 难以区分是模型漂移还是提示词漂移。

### 3. 安全与合规审计

审计人员通常希望：底线规则 有明确版本与变更记录。静态块天然适合放进 变更管理；动态块更适合 运行时日志。

## 表格：六大静态块 — 职责 / 稳定性 / 典型反模式

| 模块   | 核心职责    | 应保持稳定原因      | 常见反模式                                   |
|------|---------|--------------|-----------------------------------------|
| 身份定位 | 角色与边界   | UX 一致、减少角色漂移 | 把「当前仓库名」写进身份                            |
| 系统规范 | 工作流与优先级 | 可预期、可测试      | 每轮插入不同「口号」                              |
| 任务哲学 | 完成定义与克制 | 防止范围蔓延       | 把具体 bug 描述写进哲学段                         |
| 风险规范 | 安全与隐私底线 | 合规、不可会话级削弱   | 用动态块覆盖删库禁令                              |
| 工具总则 | 跨工具纪律   | 并行/先读后写等一致   | 在总则里写某个工具的 JSON 细节                      |
| 语气风格 | 品牌与可读性  | 体验一致         | 让 <a href="#">CLAUDE.md</a> 覆盖全局语气（若冲突） |

## 源码片段（概念）：静态块生成器

```
/**
 * 教学用伪代码：静态部分只依赖「产品配置」，不依赖「本次会话」。
 */
function buildStaticConstitution(product: ProductConfig): string {
  return [
    renderIdentity(product.name, product.surface), // 身份定位
    renderSystemNorms(product.normsVersion),      // 系统规范
    renderTaskPhilosophy(),                        // 任务哲学
    renderRiskRails(product.redlines),             // 风险规范
    renderGlobalToolRules(),                       // 工具总则
    renderVoiceAndTone(product.locale, product.voice),
  ].join("\n\n");
}
```

要点：`SessionContext` 不进 此函数（或仅进与「版本号」无关的字段），才能从工程上保证 缓存友好。

## 静态块的「版本治理」建议（给自建 Agent 的读者）

| 实践    | 说明                                      |
|-------|-----------------------------------------|
| 单一来源  | 静态文本集中在一处生成，避免复制粘贴分叉                    |
| 变更记录  | 每次改静态块记版本号与原因，便于回滚                      |
| 长度预算  | 过长的宪法挤占用户消息与工具结果空间                      |
| 与动态对齐 | 动态块不得与宪法 语义打架（例如宪法要求先读后写，动态却鼓励「直接 sed」） |

---

## 自测题

---

1. 「当前工作目录路径」应放在静态块还是动态块？为什么？
  2. 若你把「记忆片段」误写入静态块，缓存与隐私分别会出现什么问题？
  3. 工具总则与单个工具的 `prompt.ts` 应如何分工，才能既死板又可扩展？
- 

## 导航

---

- [← 5.1 动态拼装](#)
  - [5.3 动态部分：当期政策 →](#)
- 

## 5.3 动态部分：当期政策 (Dynamic Policy)

---

### 学习目标

---

- 列举 六大动态注入 及其「每次不同」的来源。
  - 解释动态块如何赋予 Agent 环境感知 与 会话连贯。
  - 能说明 记忆片段上限（如 5 条）的工程原因（上下文、噪声、成本）。
  - 绘制一条从「仓库文件」到「模型读到的系统提示」的数据路径。
- 

### 生活类比：导航里的「实时路况」

---

离线地图告诉你：道路规则、限速、单行道（像静态宪法）。

导航 App 叠加：此刻拥堵、施工、你家的目的地（像动态政策）。

没有路况，你仍能开车；没有规则，路况再好也会乱套。Claude Code 的动态块就是「实时路况层」——让模型知道 此刻 为谁、在哪、读过什么、能用什么插件、还剩多少预算。

---

# 动态六大注入内容（教学模型）

## 1. 会话引导（Session preamble）

**是什么：**本轮或本会话的 开场白式上下文——例如任务摘要、用户显式目标、多轮对话中的「当前子任务」指针。

**为何每次不同：**

- 会话 ID、用户最新一句、子任务切换都会变。
- 若写进静态块，等于 所有用户共用同一段开场，产品不可用。

**环境感知价值：**

- 把「用户刚说了什么」压缩成 对模型可见的短期意图，减少遗忘与跑题。

## 2. 记忆片段（Memory injections, 常上限 5 条）

**是什么：**从长期记忆存储里检索出的 短摘要，注入系统提示（条数受限）。

**为何每次不同：**

- 检索依赖 当前查询 / 会话主题；不同问题命中不同记忆。
- 用户可能随时 新增 / 修正 记忆。

**为何常限制 ≤5 条：**

| 原因    | 说明                            |
|-------|-------------------------------|
| 上下文窗口 | 记忆过长挤压用户消息与工具输出               |
| 噪声    | 过多记忆引入 错误先验（模型过度「觉得你过去一定...」） |
| 成本    | 动态前缀越长，缓存命中越难，费用越高            |
| 延迟    | 检索与拼装本身耗时                     |

## 3. 环境信息（Environment）

**是什么：**OS、Shell、路径、时区、语言、Node 版本、Git 状态摘要等 运行时事实。

**为何每次不同：**

- 用户换机器、换目录、升级运行时，事实即变。

- 同一静态宪法无法承载 **无限多环境**。

环境感知价值：

- 避免模型 **假设** 用户在 Linux 而实为 macOS。
  - 对路径分隔符、权限、包管理器的建议更接地气。
- 

#### 4. **CLAUDE.md** (仓库级指令)

**是什么：**工作区内的项目说明文件（名称以产品为准），通常包含：构建命令、测试命令、目录约定、团队规范。

**为何每次不同：**

- **换仓库即换内容**；同一用户在项目 A 与 B 的规则不同。
- 文件可能被用户 **随时编辑**。

环境感知价值：

- 把 **领域知识** 从模型先验转移到 **可验证的本地文本**。
  - 减少「通用建议」与「此仓库实际做法」的冲突。
- 

#### 5. MCP 插件说明 (MCP tool / server blurbs)

**是什么：**当前会话启用的 MCP 服务器、工具名、参数约束、调用注意事项的 **拼接说明**。

**为何每次不同：**

- 用户 **增减 MCP**、改配置、换密钥，工具可见集即变。
- 与 [5.6 缓存失效](#) 中「加减 MCP」强相关。

环境感知价值：

- 让模型知道 **此刻真能调用什么**，而不是背诵训练数据里的过时 API。
- 

#### 6. Token 预算提示 (Budget hints)

**是什么：**对剩余上下文、压缩策略、何时需要总结的对话 **管理提示**（实现各异）。

**为何每次不同：**

- 每轮对话消耗不同，剩余预算 **随轮次变化**。

- 长对话中可能触发 不同的截断或摘要策略，提示需同步更新。

环境感知价值：

- 引导模型 省着用上下文：少废话、多引用、必要时主动要用户精简输入。

Mermaid：六大动态块的数据来源

[Mermaid 图表 — 请在在线版本中查看]

Mermaid：从仓库到 system 提示的路径

[Mermaid 图表 — 请在在线版本中查看]

表格：动态块 — 变化触发因素 / 风险

| 动态块                       | 典型触发变化         | 主要风险         |
|---------------------------|----------------|--------------|
| 会话引导                      | 新消息、子任务切换      | 摘要错误导致误解用户意图 |
| 记忆                        | 检索词变化、记忆库更新    | 陈旧或矛盾记忆      |
| 环境                        | 换目录、升级运行时      | 过时环境块误导命令建议  |
| <a href="#">CLAUDE.md</a> | 用户编辑、切分支       | 与代码实际不一致     |
| MCP                       | 增删插件、改 tool 列表 | 缓存失效、工具幻觉    |
| Token 预算                  | 每轮消耗           | 过度激进截断损失信息   |



## 源码片段（概念）：动态尾部的拼装

```
type Memory = { id: string; text: string; score: number };

function buildDynamicTail(ctx: SessionContext): string {
  const memories = topK(ctx.memoryCandidates, 5); // 最多 5 条

  return [
    renderSessionPreamble(ctx.turnSummary),
    renderMemories(memories),
    renderEnvironment(ctx.envSnapshot),
    renderClaudeMd(ctx.workspace.claudeMdText),
    renderMcpSection(ctx.mcp.tools),
    renderTokenBudget(ctx.meter.remainingTokens),
  ]
  .filter(Boolean)
  .join("\n\n");
}
```

## 「环境感知」的三层模型（进阶）

| 层级     | 内容                                | 是否适合动态注入            |
|--------|-----------------------------------|---------------------|
| L1 运行时 | cwd、OS、版本号                        | 是                   |
| L2 仓库  | <a href="#">CLAUDE.md</a> 、目录结构摘要 | 是（常需工具先读后注入）        |
| L3 组织  | 内部策略、合规                           | 常放静态或受控配置；慎用用户可编辑文件 |

## 与静态宪法的协作原则

- 动态不推翻静态：例如宪法要求先读后写，动态块不应写「可直接 sed」。
- 冲突时显式排序：在提示中写明 优先级（产品决策），避免模型仲裁。
- 可验证优先：环境块里尽量 事实（可 `uname`），少 猜测。

## 自测题

- 为什么「记忆」不适合无上限注入？
- MCP 工具列表变化如何同时影响 行为 与 成本？
- [CLAUDE.md](#) 与「任务哲学」若冲突，你会把裁决规则写在哪一层？

## 导航

- [← 5.2 静态宪法](#)
- [5.4 缓存边界设计 →](#)

## 5.4 缓存边界设计：SYSTEM\_PROMPT\_DYNAMIC\_BOUNDARY

### 学习目标

- 解释 **边界标记** 在「静态前缀 + 动态后缀」架构中的角色。
- 对比 **边界之上** 与 **边界之下** 的内容在 **稳定性**、**缓存**、**风险** 上的差异。
- 说明为何边界设计是 **工程上的精妙一刀**，而非文档排版技巧。
- 能向同事用一张表说清：什么必须永远在边界 **上方**。

### 生活类比：精装书的「固定正文」与「可撕便签」

一本教材：

- **胶印正文**：出版社一次排版，成千上万册相同 → 像 **可缓存的静态宪法**。
- **便签 / 勘误条**：贴在末页，每个学生拿到的可能不同 → 像 **动态政策**。

**SYSTEM\_PROMPT\_DYNAMIC\_BOUNDARY** 就是 出版社规定的「便签只能贴在勘误页」：

- 你改便签，**不必重印整本书**（类比：不必使整个 system 前缀失效）。
- 若你把便签糊在第一章中间，**整本书的「相同页」定义就崩了**（类比：缓存前缀无法稳定命中）。

### 核心概念：边界是「缓存一致性」的语义锚点

在支持 **prompt caching**（或前缀缓存）的推理栈中，供应商通常对 **完全相同的前缀** 给予较低单价。

因此产品侧目标非常明确：

1. **最大化** 从 system 开头到某一点为止的 **字节级稳定**。

2. 把必然变化的内容 挤到该点 之后，使变化 局部化。

边界标记 `SYSTEM_PROMPT_DYNAMIC_BOUNDARY`（名称教学化；实现可能是常量字符串）正是 切分点 声明：

```
[ 静态：系统宪法 · 超长稳定前缀 ]
<<< SYSTEM_PROMPT_DYNAMIC_BOUNDARY >>>
[ 动态：当期政策 · 每会话/每轮可变 ]
```

模型会把整段都读进去；缓存子系统 则可能对 前半 与 后半 采用不同计费与失效策略。

边界「之上」与「之下」：对比表

| 维度   | 边界之上（静态宪法）          | 边界之下（动态政策）                                    |
|------|---------------------|-----------------------------------------------|
| 变化频率 | 低（版本发布级）            | 高（会话 / 轮次级）                                   |
| 典型内容 | 身份、规范、哲学、风险、工具总则、语气 | 会话引导、记忆、环境、 <a href="#">CLAUDE.md</a> 、MCP、预算 |
| 缓存诉求 | 强烈希望长期命中            | 接受频繁失效                                        |
| 隐私敏感 | 一般较低（无用户数据）         | 高（记忆、路径、仓库信息）                                 |
| 调试归因 | 变更应可版本化             | 变更常与上下文采集相关                                   |

精妙设计体现在哪里？

1. 失效局部化（Localization of invalidation）

没有边界时，常见反模式是：

- 在静态块末尾 顺手 拼接 `new Date().toISOString()` 「方便调试」。
- 结果：整个 system 每秒变一次，缓存 永久报废。

有边界时，工程约定清晰：

- 时间戳、会话 ID、记忆 → 只能出现在边界 下。

- Code review 一眼可抓违规。

## 2. 心智模型对齐 (Human + machine)

新同事读代码：

- 看到常量 `SYSTEM_PROMPT_DYNAMIC_BOUNDARY` 即理解 架构分層。
- 比散落在各处的 `if (dynamic)` 更易维护。

## 3. 与安全策略同构 (Optional coupling)

部分实现会把「可审计的静态红线」完整放在边界上侧，使得：

- 动态块再花里胡哨，也 较难 在结构上「盖掉」宪法（仍需防 语义冲突）。

---

## Mermaid：边界在拼装中的「刀位」

[Mermaid 图表 — 请在在线版本中查看]

---

## Mermaid：错误放置导致的缓存雪崩（反例）

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段（概念）：边界常量与拼接

```
/** 教学用：显式可检索的切分标记 */
export const SYSTEM_PROMPT_DYNAMIC_BOUNDARY =
  "<<<<CC_DYNAMIC_POLICY_BOUNDARY>>>>";

export function assembleSystemPrompt(parts: {
  staticConstitution: string;
  dynamicPolicy: string;
}): string {
  return [
    parts.staticConstitution.trimEnd(),
    "",
    SYSTEM_PROMPT_DYNAMIC_BOUNDARY,
    "",
    parts.dynamicPolicy.trim(),
  ].join("\n");
}
```

注意：真实产品中的标记字符串可能 **刻意避免** 与用户/仓库内容撞车；有时会用极少见 Unicode 或长随机常量（教学简化为可读字符串）。

## 边界上下内容示例（虚构、缩略）

### 边界之上（摘录）

你是 CLI 中的编程助手。你必须遵守工具纪律：读文件用专用读取工具，不得用 `shell` 文本命令替代.....  
(中略：风险、语气)

### 边界之下（摘录）

```
## 当前会话
工作目录： /Users/alice/proj-foo
Shell： zsh, OS: darwin

## 记忆（最多 5 条）
- 用户偏好：提交信息用中文。

## CLAUDE.md
运行测试：pnpm test

## MCP 工具
- mcp__linear__create_issue ...
```

可见：上侧 像员工手册；下侧 像 今日值班白板。

# 与「ToolSearchTool / 延迟加载」的边界关系（预告）

工具细则可能 **不在** 全量塞进静态块，而是通过 ToolSearchTool 按需检索各工具目录的 `prompt.ts`（见 [5.8](#)）。

此时仍建议：

- 工具总则（并行、先读后写、Fail-closed）留在 **边界之上** 或 **极稳定** 区域。
- 单个工具的长说明 可走检索链，避免 **静态前缀无限膨胀**。

## Checklist：Code Review 问句

1. 本次 MR 是否改变了边界 **之上** 的文本？若是，是否评估缓存与全用户影响？
2. 是否有 **用户数据** 误入边界之上？（隐私与缓存复用风险）
3. 动态块是否 **重复** 了宪法内容导致维护分叉？

## 自测题

1. 为什么说边界标记主要是 **工程与计费结构** 问题，而不只是「排版好看」？
2. 若把 `CLAUDE.md` 全文挪到边界之上，对 **多仓库场景** 有何后果？
3. 边界之下的内容是否「可以随便写」？还有哪些约束（长度、冲突、安全）？

## 导航

- [← 5.3 动态政策](#)
- [5.5 Token 缓存经济学 →](#)

# 5.5 Token 缓存经济学（Prompt Cache Economics）

## 学习目标

- 理解 **缓存读取单价** 相对 **常规模型输入单价** 的数量级差异（教学用比例：约 90% 减免）。

- 会用简单算术估算 **长对话** 在「高命中」vs「零命中」下的费用区间。
- 解释 **命中率** 如何非线性地影响总成本。
- 读懂一张 **命中率-成本因子** 关系图（Mermaid）。

## 生活类比：图书馆借书 vs 复印整本

- **常规模型输入**：每次把厚书 **全文复印** 交给教授读——贵。
- **缓存读取**：教授 **记得** 上周读过同一本的前 200 页，只收 **重读已备案部分** 的低价——便宜。

Prompt caching 的本质：**稳定前缀不必每次全价「重新教」**。Claude Code 把 **静态宪法** 尽量放在前缀，就是在 **优化复印费**。

## 关键价格假设（教学示例，非实时报价）

以下数字 **仅用于课堂推算**；请以 Anthropic 控制台与文档的 **当期定价** 为准。

| 计费项              | 假设单价（示例）           | 备注                       |
|------------------|--------------------|--------------------------|
| 模型输入（未命中缓存）      | \$5 / 百万 tokens    | 与官方 Opus 量级同阶的整数假设       |
| 缓存读取（cache read） | \$0.50 / 百万 tokens | 约为常规模型输入的 1/10 → 节省约 90% |
| 缓存写入（若有）         | 另计，本篇从略            | 首次建立前缀时可能发生              |

用户给出的经验表述：缓存读取约 \$0.50/百万，正常输入约 \$5/百万 → 节省 80—90%（与 1/10 一致）。

## 100 轮 Opus 级对话：从 \$50—100 降到 \$10—19（推演）

### 设定（简化模型）

- 每轮 **用户 + 助手 + 工具** 往返后，进入下一请求时，**system 前缀 + 历史** 中有 **S tokens** 属于「可缓存前缀」结构（教学上常包含长 system 与累积历史，具体以平台规则为准）。
- 为突出 **system 静态块** 的收益，假设每轮 **新增** 用户与助手内容合计 **U tokens**。
- **仅演示数量级**：设每轮请求里，有 **P = 200,000 tokens** 的前缀在本次请求中走 **缓存读取价**，每轮另有 **Q = 10,000 tokens** 走 **常规模型输入价**（新内容、未命中部分）。

真实产品中 P/Q 随历史长度、截断策略、是否多模态等剧烈变化；此处取「长上下文 + 高前缀重复」的保守夸张例，用于展示 降费潜力。

零缓存命中（极端差）

每轮全部按 \$5/百万：

- 每轮 tokens 假设  $P + Q = 210,000$
- 100 轮总 tokens  $\approx 21,000,000$
- 费用  $\approx 21 \times 5 = \text{\$}105$

（与用户说的 \$50—100 同量级，可通过调低 P/Q 对齐。）

高缓存命中（前缀按 \$0.50/百万）

同一轮：

- P 走缓存读： $200,000 \times \$0.50/1e6 = \$0.10$
- Q 走常规模型输入： $10,000 \times \$5/1e6 = \$0.05$
- 每轮合计  $\approx \$0.15$

100 轮  $\approx \$15$

若 Q 略大或 P 略小，落在 \$10—19 区间非常自然。

算术小结表

| 场景    | 每轮费用（示例公式）                          | 100 轮近似 |
|-------|-------------------------------------|---------|
| 全价前缀  | $(P+Q) \times \$5/M$                | ~\$105  |
| 前缀缓存读 | $P \times \$0.5/M + Q \times \$5/M$ | ~\$15   |

结论（教学）：长且稳定的前缀 是降本核心；动态污染前缀 是回本杀手（见 5.6）。

Mermaid：成本结构饼图意念（流程版）

[Mermaid 图表 — 请在在线版本中查看]



## Mermaid：命中率与「等效单价倍数」

设一次请求中，前缀占比为  $r$  (0—1)，仅两档价：缓存读  $c$ ，全价  $f$ ，且忽略写入费。

等效每 token 单价（极度简化）：

$$[\text{eff}] = r \cdot c + (1-r) \cdot f$$

当  $c = f/10$ ：

| 命中率 $r$ (前缀走缓存) | 等效相对全价的比例 (近似) |
|-----------------|----------------|
| 0%              | 100%           |
| 50%             | 55%            |
| 80%             | 28%            |
| 90%             | 19%            |
| 95%             | 14.5%          |

[Mermaid 图表 — 请在在线版本中查看]

上图用 **流程链** 表达「 $r$  越高、相对全价成本越低」的单调关系；精确数值以下表为准（兼容不支持 xychart 的渲染器）。

| $r$  | 相对全价成本 % |
|------|----------|
| 0    | 100      |
| 0.5  | 55       |
| 0.8  | 28       |
| 0.9  | 19       |
| 0.95 | 14.5     |

## 缓存命中率对成本的「杠杆效应」

- 前缀越长、重复越高 → 绝对节省的美元越大（线性于 tokens）。
- 小幅命中率下滑 → 在高价区多烧的钱 **不成比例地痛**（因为贵的是 \$5/M 那段）。
- 动态块膨胀 不仅多 token，还可能 **打碎前缀** → 双重打击。

## 给团队的可执行建议（与提示词工程联动）

| 动作                 | 预期收益                 |
|--------------------|----------------------|
| 静态宪法与动态政策 硬边界      | 稳定前缀，提升命中            |
| 禁止在静态区写时间戳 / 随机 ID | 避免伪变化                |
| MCP 工具列表稳定化（少抖动）   | 减少动态区对前缀的「牵连」（视实现而定） |
| 监控「缓存读写 tokens」指标  | 把降本变成可观测 KPI         |

## 源码片段（概念）：按层统计 tokens（观测用）

```
type Pricing = { modelPerM: number; cacheReadPerM: number };

function estimateInputCost(tokens: {
  cacheRead: number;
  fullPrice: number;
}, p: Pricing): number {
  return (
    (tokens.cacheRead * p.cacheReadPerM) / 1_000_000 +
    (tokens.fullPrice * p.modelPerM) / 1_000_000
  );
}

// 例: cacheRead=200_000, fullPrice=10_000 → 约 $0.15（沿用本节假设价）
```

## 灵敏度：Q 与 P 同时变化时（100 轮）

固定 单价假设（\$5/M 全价，\$0.5/M 缓存读），只调 每轮 token 结构：

| 每轮 P（缓存读） | 每轮 Q（全价） | 每轮费用    | 100 轮合计 |
|-----------|----------|---------|---------|
| 200,000   | 10,000   | \$0.15  | ~\$15   |
| 150,000   | 20,000   | \$0.175 | ~\$17.5 |
| 100,000   | 30,000   | \$0.20  | ~\$20   |
| 50,000    | 40,000   | \$0.225 | ~\$22.5 |

读表直觉：即使 前缀变短（P 降），只要 全价区 Q 不大，总账单仍可能落在 **\$10—25** 量级。若 完全无缓存，每轮费用改为  $(P+Q) \times \$5/M$ （例如 P=200k、Q=10k 时每轮约 **\$1.05**），100 轮约

\$105，与「高命中」场景相差约 7 倍（该倍数随 P/Q 与价差假设而变）。

## Mermaid：长对话下的「前缀滚雪球」

[Mermaid 图表 — 请在在线版本中查看]

若前缀因陷阱被污染，滚雪球的是 **费用** 而不是 **红利**。

## 与产品指标对接（观测面板建议）

| 指标                   | 含义           | 异常时排查           |
|----------------------|--------------|-----------------|
| cache_read_tokens    | 走低价读的 tokens | 突然归零 → 提示词或模型切换 |
| input_tokens 全价部分    | 新增/未命中       | 对话变长或截断策略变      |
| cost_per_turn        | 每轮美元         | 与命中率表对照         |
| static_block_version | 静态宪法版本       | 发版后对比命中曲线       |

## 自测题

1. 若缓存读价不是  $f/10$  而是  $f/5$ ，100 轮示例总费用会上浮多少（定性即可）？
2. 为什么「长对话」比「短对话」更吃缓存红利？
3. 你会如何把「降本」写成 CI 里的告警规则（指标 + 阈值思路）？

## 导航

- [← 5.4 缓存边界](#)
- [5.6 缓存失效陷阱 →](#)

## 5.6 缓存失效 7 大陷阱

### 学习目标

- 列举 七种 常见导致 prompt 缓存前缀失效或命中率暴跌的做法。
- 每个陷阱能给出 反面示例 与 规避手段。
- 建立「改提示词 = 改计费」的成本意识。

### 生活类比：指纹打卡机

公司门禁认 指纹。你 整过容（全脸变化）或 手指贴了新创可贴（局部变化），识别率都会掉。

缓存前缀就像 指纹模板：任何你以为「无所谓」的小改动，都可能让系统认为 换了一根手指。

### 总览表

| # | 陷阱                                | 典型症状         | 核心规避                        |
|---|-----------------------------------|--------------|-----------------------------|
| 1 | 改大小写 / 空白规范化                      | 前缀哈希突变       | 统一 formatter、禁止随意 trim 规则变更 |
| 2 | 加减 MCP 工具                         | 工具列表变化       | 稳定工具集、批量变更、版本化说明块           |
| 3 | 换模型                               | 供应商侧缓存键变     | 升级评估缓存命中回归                  |
| 4 | 注入时间戳                             | 每轮前缀不同       | 只放动态区 或 日志侧                 |
| 5 | 改 <code>tool_choice</code> / 工具模式 | API 行为变、拼接变  | 谨慎改默认调用策略                   |
| 6 | 静态区混入用户路径                         | 多用户互相污染或无法命中 | 路径仅动态区                      |
| 7 | 提示词 A/B 无版本隔离                     | 实验污染生产缓存统计   | 分环境、分 key、分前缀               |

## 陷阱 1: 改大小写与空白

### 现象

`FileRead` 与 `fileread`、`\n\n` 与 `\n`、行尾空格，在 **字节级** 都不相等。某些团队在「美化提示词」时无意改了大小写，导致 **缓存整段失效**。

### 反面示例

```
- You MUST use FileRead for reading files.  
+ You must use fileread for reading files.
```

### 避免方法

- 提示词 **规范化管道**（单一 builder），禁止手改散落副本。
- Code review 关注 **非语义 diff**（大小写、换行）。

## 陷阱 2: 加减 MCP 工具

### 现象

MCP 工具清单常拼进 **system 或 tools 模式描述**。增删一个工具 → 描述字符串变 → **前缀变化**。

### 反面示例

```
# 今日注入  
可用工具: Read, Edit, Bash, mcp__slack__post_message # 新增一行
```

### 避免方法

- 批量** 更新工具说明，减少日内抖动。
- 把易变清单 **固定放在动态区** 且接受失效，但 **勿** 向上渗透污染静态宪法（架构约定）。
- 对实验性 MCP 使用 **单独 profile**。

## 陷阱 3：换模型

---

### 现象

不同模型族可能对应 不同计费与缓存实现；即便文本相同，平台也可能 不复用 另一模型的缓存条目。

### 反面示例

「本周从 A 模型切到 B 模型，费用报表突然难看」——部分来自 命中归集变化。

### 避免方法

- 模型升级做 A/B 费用对比（同任务、同前缀策略）。
- 大版本切换时 预期 短期命中率波动。

---

## 陷阱 4：加时间戳 / 随机 trace id

---

### 现象

调试时在 system 顶部加入：

```
const banner = `Build time: ${Date.now()}`;
```

每请求变一次 → 缓存永久 0 命中。

### 避免方法

- 时间信息放 日志，不要进 可缓存前缀。
- 若必须给模型「当前时间」，放在 动态区 并 接受 该区不命中。

---

## 陷阱 5：改 `tool_choice` 或工具调用模式

---

### 现象

部分 API 层在切换 `tool_choice`（如 auto / required / none）时，会 重组消息或 system 附加说明，间接改变前缀或系统拼装路径。

## 反面示例

全局把 `tool_choice` 从 `auto` 改为 `required`，触发不同的 工具调用引导语 注入。

## 避免方法

- 变更默认工具策略前读 供应商文档 与 内部拼装 diff。
- 分阶段灰度，观察 `cache read tokens` 曲线。

## 陷阱 6：静态区混入用户特定信息

### 现象

为「省事」把用户名、home 路径、仓库绝对路径写进 静态宪法生成器：

- 多用户场景下 无法共享缓存 或 错误共享（严重）。
- 单用户场景也会因 路径变化 频繁失效。

### 反面示例

```
你是专为 /Users/alice/my-app 服务的助手.....
```

### 避免方法

- 用户与路径 永远在 `SYSTEM_PROMPT_DYNAMIC_BOUNDARY` 之下。

## 陷阱 7：A/B 实验污染前缀

### 现象

同时在产线注入多种提示变体，但 无稳定前缀；或把实验 flag 随机插进前缀中间。

### 反面示例

```
[[experiment_variant=37]] # 随机  
你是.....
```

## 避免方法

- 实验变体 整块替换 且 版本号稳定（按 cohort 固定）。
- 指标按 变体 ID 分桶看缓存命中。

## Mermaid：陷阱 4 的因果链

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid：七陷阱分类（树状）

[Mermaid 图表 — 请在在线版本中查看]

## 快速自查脚本思路（伪代码）

```
function auditPromptForCaching(s: string): string[] {
  const issues: string[] = [];
  if (/^d{4}-d{2}-d{2}T/.test(s)) issues.push("疑似 ISO 时间戳");
  if (/^trace[_-]?id/i.test(s)) issues.push("疑似 trace id");
  if (/Users\/|C:\\Users\\\/i.test(s)) issues.push("疑似用户路径");
  return issues;
}
```

## 自测题

1. 为什么「美化换行」也可能比「改一句中文翻译」更伤缓存？
2. MCP 变更同时影响 **功能** 与 **成本** 的路径各是什么？
3. 若必须把「当前时间」给模型，你会放在哪一层、如何接受成本？

## 导航

- [← 5.5 Token 经济学](#)



- [5.7 铁血行为约束 →](#)

## 5.7 铁血行为约束（Behavior Constraints）

### 学习目标

- 复述 `getSimpleDoingTasksSection` 类 任务哲学铁律（不加无关功能、不过度抽象等）。
- 背熟 工具使用死规矩：读用 `FileRead`、改用 `FileEdit`、无依赖并行。
- 说明 BashTool Git 安全协议 三条：不 `push --force`、不 `reset --hard`、总是新 commit 不 amend。
- 解释 Fail-closed： `isConcurrencySafe` 与 `isReadOnly` 默认 `false` 的工程含义。
- 说明 FileEditTool 强制先 FileReadTool：盲改拦截逻辑的产品价值。
- 了解 `USER_TYPE === 'ant'` 分支对内部员工可能 更激进 的策略差异（概念层）。

### 生活类比：手术室的核对清单

再厉害的医生也要：先读病历再开刀、不擅自加戏（额外器官别乱切）、危险操作双人确认。

Agent 的「铁血约束」就是 手术室清单：不是为了刁难模型，是为了 可恢复、可审计、少翻车。

### 一、任务哲学铁律（`getSimpleDoingTasksSection` 教学归纳）

以下为对常见「Doing tasks」短章节的语义归纳；函数名对齐你的技术说明。

| 铁律        | 含义                     | 反面行为        |
|-----------|------------------------|-------------|
| 不加无关功能    | 只做用户要的，拒绝「我顺便帮你重构整个项目」 | scope creep |
| 不过度抽象     | 够用即可，避免为模式而模式          | 引入不必要框架     |
| 不乱加注释     | 注释服务维护，不服务「显得我很努力」     | 满屏噪声注释      |
| 不给时间估计    | 避免幻觉式 deadline         | 「大约 5 分钟搞定」 |
| 失败先诊断再换策略 | 先定位根因，再换工具/思路          | 随机重试三连      |
| 结果如实汇报    | 失败说失败，不确定说不确定          | 粉饰输出        |

## 源码片段（概念）

```
function getSimpleDoingTasksSection(): string {
  return [
    "- 不添加用户未要求的无关功能。",
    "- 避免过度抽象：优先最小可行修改。",
    "- 不堆砌无意义注释。",
    "- 不对耗时给出武断估计。",
    "- 出错时先诊断再更换策略，禁止盲目重试。",
    "- 如实汇报结果与限制，不夸大成功。",
  ].join("\n");
}
```

## 二、工具使用「极死板」规范

### 核心规矩

- 1. 读文件：使用 `FileRead`（或产品等价物），禁止用 `cat` / `head` 等 shell 代替（除非产品明确允许例外）。
- 2. 改文件：使用 `FileEdit`（或结构化补丁工具），禁止用 `sed -i` 等盲改。
- 3. 无依赖的工具调用：必须并行 发起，降低延迟。

### 为什么死板？

| 理由      | 说明                            |
|---------|-------------------------------|
| 可审计     | structured read/edit 可记录路径与版本 |
| 跨平台     | shell 文本命令在 Windows/引号/转义上脆弱  |
| 与安全钩子结合 | 工具层可做策略拦截（盲改、敏感路径）            |

### 反面示例

```
# 反模式：用 cat 「读」——绕过 FileRead 的统一策略
cat src/app.ts
```

```
# 反模式：sed 原地改——难以 diff、难以回滚对齐
sed -i '' 's/foo/bar/' src/app.ts
```

推荐心智

「Shell 是胶水，文件真相在专用工具里。」

三、BashTool：Git 安全协议（教学版）

| 禁止 / 要求                          | 说明                                                  |
|----------------------------------|-----------------------------------------------------|
| 禁止 <code>git push --force</code> | 避免覆盖他人历史，破坏协作                                       |
| 禁止 <code>git reset --hard</code> | 防止未备份工作丢失（除非产品另有撤销机制）                               |
| 总是新 commit                       | 用新提交记录变更，不要 <code>git commit --amend</code> 改写已发布历史 |

生活类比

Git 历史像 银行流水：你可以 新记一笔冲正，但不该 偷偷改昨天那条记录（amend / force push 的协作风险）。

四、Fail-closed 设计

定义

对工具元数据中的开关：

- `isConcurrencySafe` 默认 `false`
- `isReadOnly` 默认 `false`

除非工具作者 显式证明 并发安全或只读，否则调度器 保守处理。

工程含义

| 默认值                            | 若误标为 true 的后果 | Fail-closed 避免 |
|--------------------------------|---------------|----------------|
| <code>isConcurrencySafe</code> | 并行调用 竞态、文件损坏  | 默认不并行          |
| <code>isReadOnly</code>        | 误以为只读而 并行+缓存  | 默认按「有副作用」处理    |

## 伪代码

```
interface ToolMeta {
  isConcurrencySafe?: boolean;
  isReadOnly?: boolean;
}

function isSafeToParallelize(meta: ToolMeta): boolean {
  return meta.isConcurrencySafe === true; // 必须显式 true
}

function treatAsReadOnly(meta: ToolMeta): boolean {
  return meta.isReadOnly === true;
}
```

## 五、FileEditTool：强制先 FileReadTool

### 规则

在允许编辑前，系统校验：**目标文件已被 FileRead（或等价）读取 且版本仍有效**；否则 **拦截盲改**。

### 价值

- 防止模型在 **未看到最新内容** 的情况下覆盖他人修改。
- 降低 **上下文幻觉** 导致的灾难性补丁。

### 流程示意（Mermaid）

[Mermaid 图表 — 请在在线版本中查看]

## 六、`USER_TYPE === 'ant'`：内部员工分支（概念）

部分部署为 **内部用户** 提供 **更激进** 策略（更高自动性、更宽工具范围等），通过：

```
const policy =
  ctx.userType === "ant"
    ? loadInternalAggressivePolicy()
    : loadDefaultPolicy();
```

教学提醒：

- 这是 **风险与信任模型** 选择，不是「能力更强」的同义词。
- 对外用户默认应 **更保守**，避免合规与数据泄露面扩大。

Mermaid：行为约束在提示词中的层次

[Mermaid 图表 — 请在在线版本中查看]

表格：约束落点（提示 vs 代码）

| 约束类型          | 主要靠提示词 | 主要靠代码拦截 |
|---------------|--------|---------|
| 不加无关功能        | 是      | 难       |
| 不用 cat 读文件    | 是      | 可部分     |
| 禁止 force push | 是      | 应用层强烈建议 |
| FileEdit 先读   | 提示辅助   | 应硬拦截    |
| Fail-closed   | 解释性    | 调度器实现   |

自测题

1. 为什么「禁止时间估计」属于 **产品体验** 而不只是风格？
2. Fail-closed 与「默认允许」在 **并行调度** 上各适合什么组织文化？
3. 若移除 FileEdit 先读拦截，最可能出现的 **真实事故** 是什么？

导航

- [← 5.6 缓存陷阱](#)
- [5.8 工具使用手册 →](#)

# 5.8 工具使用手册：每个工具的 prompt.ts

## 学习目标

- 理解 42 个工具按需延迟加载 与 ToolSearchTool 注入 的协作方式。
- 能描述 工具目录 prompt.ts 的职责：写给 模型 看的说明，而非给人类运维读的长文。
- 分析一份 BashTool 手册示例：参数约束、Git 子协议、反模式。
- 区分 全局工具总则（宪法层）与 单工具手册（条例层）。

## 生活类比：宜家家具说明书

全屋装修规范（宪法）说：「用电钻注意安全」。

具体到 书柜 B：步骤图、螺丝编号、「不要用锤子敲卡扣」——这是 B 的说明书。

prompt.ts 就是 每件家具的折叠说明书：模型 按需翻页，而不是一次塞满 42 本。

## 架构：ToolSearchTool 与延迟加载

### 要点（对齐你的技术信息）

- 全量 42 个工具的 长描述 若全部塞进 system → 前缀爆炸、成本高、噪声大。
- ToolSearchTool（名称教学化）允许模型 检索 当前任务相关的工具手册片段。
- 每个工具目录维护 prompt.ts（或等价模块），导出 短而硬 的使用说明字符串。

### Mermaid：从「我要干活」到「读到正确手册」

[Mermaid 图表 — 请在在线版本中查看]

### Mermaid：延迟加载 vs 全量注入

[Mermaid 图表 — 请在在线版本中查看]

prompt.ts

写什么？（结构模板）

| 区块         | 内容               |
|------------|------------------|
| 一句话职责      | 这个工具解决什么问题       |
| 何时用 / 何时不用 | 与相邻工具的分工         |
| 参数契约       | 必填、可选、格式、路径约定    |
| 硬禁令        | 与 Bash/Git/读写的边界 |
| 好例子 / 坏例子  | 各 1 组即可          |
| 并行与依赖      | 能否与谁并行；是否必须先读    |

源码片段（概念）：工具 prompt 模块

```
// tools/bash/prompt.ts - 教学示例
export const bashToolPrompt = `
## BashTool

用途：在受控环境中执行 shell 命令以完成构建、测试、包管理。

何时使用：
- 需要运行 npm/pnpm/pytest 等项目命令。
- 需要 git status/diff 等只读查询（仍须遵守 Git 安全协议）。

何时不使用：
- 读取文件内容 → 使用 FileReadTool，不要用 cat。
- 修改源文件 → 使用 FileEditTool，不要用 sed -i。

Git 安全协议（摘要）：
- 禁止 git push --force。
- 禁止 git reset --hard。
- 提交使用新 commit；禁止 git commit --amend 改写已共享历史。

并行：与其他无数据依赖的工具调用并行发起。
`.trim();
```

## BashTool 手册扩展示例（可直接当模板骨架）

```
## BashTool - 扩展手册（示例）

### 安全边界
- 假定所有命令可被用户审计；避免在命令中拼接密钥。
- 长输出可能截断；需要完整内容时优先专用只读工具或分页策略（若产品提供）。

### 推荐模式
- 先 `pwd` / `git status` 建立情境，再执行目标命令（在需要时）。
- 构建失败时，保留完整错误输出片段，便于诊断。

### 反模式
- 用 `rm -rf` 清理构建产物前未确认路径。
- 用 `curl | sh` 安装未知脚本。

### 与 File 工具协作
1. FileRead 获取配置或源码上下文。
2. Bash 运行与上下文一致的命令。
3. FileEdit 根据结果改代码；勿用 awk/sed 代替 FileEdit。
```

### 表格：总则 vs `prompt.ts`

| 维度   | 全局工具总则（静态宪法侧） | 单工具 <code>prompt.ts</code> |
|------|---------------|----------------------------|
| 受众   | 所有工具调用        | 单一工具                       |
| 稳定性  | 高             | 中（随工具演进）                   |
| 长度   | 短             | 可中等，但检索片段宜短                |
| 典型内容 | 并行纪律、先读后写     | 参数、反模式、Git 细则              |

### 「42 个工具」治理建议

- 每个工具单一文件，避免 README 与 prompt 分叉。
- 索引一致性：ToolSearch 的元数据（名称、标签）与 `prompt.ts` 同步更新。
- 抽检：新工具上线前跑一轮 **模型开卷考试**（给定任务，看是否先检索正确手册）。

### 与 Fail-closed / 盲改拦截的衔接

- `prompt.ts` 写明 **必须先 FileRead** 是软约束。



- FileEdit 硬拦截 是 硬约束。

二者叠加，降低模型钻空子概率。

prompt.ts

长度与信息密度指南

| 篇幅            | 建议              | 原因           |
|---------------|-----------------|--------------|
| 极短 (< 30 行)   | 适合高频、语义单一工具     | 检索命中后噪声低     |
| 中等 (30—120 行) | Bash、Git、测试运行器等 | 需示例与反模式      |
| 过长 (> 200 行)  | 拆成「总则 + 场景附录」   | 避免单次注入吞没用户消息 |

写作原则：模型 不会 像人类一样「通读全书」，而是 边做边查。段落标题要 可扫描，禁令要 可执行（动词开头）。

与 42 个工具的版本协同

```
// 概念：工具版本 bump 时同步索引，避免「搜得到但说明过期」
export const TOOL_MANUAL_INDEX = [
  { name: "FileRead", version: 3, path: "tools/file-read/prompt.ts" },
  { name: "Bash", version: 12, path: "tools/bash/prompt.ts" },
  // ...
] as const;
```

| 变更类型    | 需要更新                    |
|---------|-------------------------|
| 新增必填参数  | prompt.ts + Schema + 索引 |
| 仅修复 bug | 宿主代码为主；prompt 可选微调      |
| 重命名工具   | 破坏性：索引、历史会话、缓存          |

Mermaid：手册片段如何进入当轮上下文

[Mermaid 图表 — 请在在线版本中查看]

## BashTool 手册「反面教材」片段（刻意糟糕）

```
## Bash
你可以运行任何命令。随便 push。
```

问题：无 Git 协议、无 File 工具分工、无并行提示 → 模型行为 **不可预期**。对比前文「扩展手册示例」可见差异。

## 自测题

1. 若把 42 个 `prompt.ts` 全文塞进静态宪法，会破坏哪两个工程目标？
2. ToolSearchTool 检索失败时，产品应 **降级** 为什么体验（全量？拒绝？）？
3. Bash 手册里为什么要 **重复** Git 三条（若宪法已写过）？

## 导航

- [← 5.7 行为约束](#)
- [5.9 策略对比 →](#)

# 5.9 策略对比：Claude Code vs OpenAI vs Google

## 学习目标

- 从 **系统提示结构**、**工具描述组织**、**并行与失败策略** 三方面对比三家常见路线。
- 说明 **Claude Code** 的「静动边界 + 延迟工具手册」在 **成本与纪律** 上的取舍。
- 能向团队解释：为何 **不是**「谁的提示词更长谁更强」。

## 生活类比：三家快递公司的员工手册

- **A 公司**：手册很厚，但分 **总部红线** 与 **站点每日告示**——像静动分离。
- **B 公司**：总部发 **标准 API 卡片**（函数名、JSON Schema），站点自己贴便利贴——像 function calling 生态。

- C 公司：强调 多模态与工具声明格式，各城法规用插件描述——像 Gemini tools。

没有绝对赢家，只有 场景匹配。

### 对比总表

| 维度             | Claude Code（本篇模型）                                  | OpenAI（function calling 生态）     | Google（Gemini tools）          |
|----------------|----------------------------------------------------|---------------------------------|-------------------------------|
| 系统提示           | 静动拆分，边界下注入环境/记忆/MCP                                | 常由应用拼装 system + developer       | system instruction + tools 声明 |
| 工具描述           | 总则 + 每工具 <code>prompt.ts</code> ，可 ToolSearch 延迟加载 | JSON Schema + 简短 description 为主 | 工具定义 + 声明式约束（随 API 演进）        |
| 纪律 enforcement | 强提示 + 硬拦截（如先读后改）                                   | 多依赖应用层中间件                       | 依赖应用层与模型遵循                    |
| 并行             | 明确「无依赖并行」文化                                        | 由客户端决定是否并行 tool 调用              | 依客户端与模型能力                     |
| 缓存意识           | 产品化考虑 prompt cache 前缀                              | 视应用；Responses/缓存能力因产品而异         | 视应用与模型栈                       |
| 典型强项           | 长上下文编程流、CLI 一体                                     | 生态与集成广度                         | 多模态、部分谷歌云一体场景                 |

### Claude Code：提示词策略摘要

1. `getSystemPrompt()` 编译器模型：静态宪法 + `SYSTEM_PROMPT_DYNAMIC_BOUNDARY` + 动态政策。
2. 行为铁律 与 工具死规矩 双管齐下；关键路径 代码拦截。
3. 工具手册 延迟加载，控制前缀长度，服务 成本与专注。

### Mermaid：Claude Code 提示词栈

[Mermaid 图表 — 请在在线版本中查看]

## OpenAI：function calling 典型策略

### 特征

- 工具以 **函数列表** 形式提供给 API；模型在 **消息流** 中产出 `tool_calls`。
- **描述字段** 受 token 预算限制，团队常在「短描述」与「行为完整」间权衡。
- **系统提示** 常由 **应用** 拼接：安全策略、RAG、记忆、业务规则。

### 与 Claude Code 的差异（教学观点）

| 点      | OpenAI 生态常见现象 | Claude Code 倾向 |
|--------|---------------|----------------|
| 工具说明位置 | 紧附在 schema    | 总则 + 可检索手册     |
| 硬约束    | 依赖中间件/沙箱      | 提示 + 专用工具链拦截   |
| 前缀治理   | 因应用而异，需自建规范   | 产品内建静动边界意识     |

### Mermaid：OpenAI 风格消息流（简化）

[Mermaid 图表 — 请在在线版本中查看]

## Google：Gemini tools 典型策略

### 特征

- **声明式工具** 与 **系统 instruction** 组合；多模态输入常见。
- 工具与 **谷歌生态**（搜索、地图、企业数据）集成时，提示词侧重 **能力边界与合规**。
- 具体字段名、模式随 **API 版本** 演进较快，提示工程要 **版本锁定**。

### 与 Claude Code 的差异（教学观点）

| 点     | Gemini 路线             | Claude Code 路线    |
|-------|-----------------------|-------------------|
| 上下文形态 | 强多模态                  | 编程 CLI 常以文本+工具为主  |
| 工具发现  | 应用定义                  | ToolSearch + 延迟手册 |
| 团队心智  | 「instruction + tools」 | 「宪法/政策/手册」三层      |

## 何时借鉴哪一种？

| 你的场景        | 可借鉴 Claude Code | 可借鉴 OpenAI | 可借鉴 Google |
|-------------|-----------------|------------|------------|
| 长对话编程 Agent | 静动边界、缓存意识       | 丰富中间件示例    | 多模态需求      |
| 大量第三方 API   | 延迟工具手册思路        | schema 驱动  | 声明式工具      |
| 企业合规审计      | 硬拦截 + 日志        | 策略网关       | 云一体身份与权限   |

## 源码片段（概念）：三家「工具注册」形态对比

```
// OpenAI 风格: schema 即契约
const openaiTools = [
  {
    type: "function",
    function: {
      name: "read_file",
      description: "Read a UTF-8 text file by path.",
      parameters: { type: "object", properties: { path: { type: "string" } }, required: ["path"] },
    },
  },
];

// Claude Code 风格: 运行时检索 prompt.ts 片段 + 硬规则在宿主
const claudeCodeTooling = {
  globalRules: "FileRead for read; FileEdit for edit; parallelize independent tools.",
  lazyManuals: { bash: () => import("./tools/bash/prompt").then((m) => m.bashToolPrompt) },
};

// Gemini 风格 (极度简化): 工具声明 + instruction
const geminiStyle = {
  systemInstruction: "You are a coding agent. Follow org policies.",
  tools: [{ name: "run_shell", description: "..." }],
};
```

## 常见误区

1. 误区：「OpenAI 不用写长 system，全靠 schema。」  
正解：复杂 Agent 仍然需要 分层 system；schema 无法承载全部协作策略。
2. 误区：「Claude Code 只靠提示词约束。」  
正解：关键路径有 代码级 Fail-closed 与盲改拦截。

3. 误区：「换一家 API 只要复制提示词。」
- 正解：工具格式、并行语义、缓存机制不同，需要 重写拼装层。

## 迁移检查表：从 OpenAI 风格迁到「静动 + 手册」风格

| 步骤 | 动作                                                                 |
|----|--------------------------------------------------------------------|
| 1  | 从现有 monolithic system 中 剪切 所有含用户路径/时间的句子                           |
| 2  | 将剩余 红线规则 固化为 <code>buildStaticConstitution</code>                  |
| 3  | 将 环境、记忆、插件 迁入 <code>buildDynamicPolicy</code>                      |
| 4  | 把过长工具说明迁到 <code>per-tool prompt</code> ，保留 <code>schema</code> 短描述 |
| 5  | 加 边界常量 与 集成测试（快照哈希稳定）                                              |

## Google 路径：多模态块放在哪？

| 内容                  | 建议层级                   |
|---------------------|------------------------|
| 「如何描述图片中的 UI bug」总则 | 可放 静态（若不含用户图像）         |
| 当前截图 bytes / 文件名    | 动态 或 user 消息，勿进可共享缓存前缀 |
| 工具返回的富媒体            | 消息链，与 system 手册分离      |

## Mermaid：三家的「单一职责」卡通对比

[Mermaid 图表 — 请在在线版本中查看]

## 延伸阅读（官方与生态）

- 以 各平台定价页 校准缓存单价与写入费。
- 以 各 API 版本 changelog 校准工具声明字段名（尤其 Gemini）。
- 开源 Agent 框架（LangGraph、Semantic Kernel 等）常混用多家 API：对照本篇分层表 做适配层。

## 自测题

---

1. 为什么说 ToolSearch 延迟加载 同时 服务「注意力」与「成本」？
  2. function calling 的 schema 描述过短会导致什么 典型模型错误？
  3. 若你的 Agent 需要强多模态，你会如何把「静动边界」思想 迁移 过去？
- 

## 导航

---

- [← 5.8 工具手册](#)
  - [5.10 实践练习 →](#)
- 

## 5.10 实践：设计你自己的系统提示词

---

### 学习目标

---

- 按 Claude Code 模式 为「迷你 Agent」划分 静态宪法 与 动态政策。
  - 写出 `SYSTEM_PROMPT_DYNAMIC_BOUNDARY` 等价标记 两侧的目录级大纲。
  - 起草 任务哲学短章（对齐 `getSimpleDoingTasksSection` 精神）。
  - 为 2 个虚构工具 各写一份微型 `prompt.ts` 片段。
  - 列出自己的 缓存失效风险清单（至少 5 条）。
- 

### 生活类比：开一家只有两名员工的小店

---

你是老板，要写两本册子：

1. 员工守则（天天贴在墙上）——静态。
2. 今日进货与特价（每天早上更新）——动态。

本练习就是 写这两本册子，并规定 只能把特价写在黑板下半截（边界之下）。

---

### 练习 A：选定你的迷你 Agent

---

从以下选一（或自拟）：

| 编号 | Agent 类型 | 用户场景                |
|----|----------|---------------------|
| A1 | 个人记账助手   | 解析自然语言支出、分类、周报      |
| A2 | 仓库文档巡检   | 检查 README 链接、过期日期   |
| A3 | 课程作业批改辅助 | 对作业给出规范反馈，不直接给抄答案步骤 |

写下一句「产品使命」（≤30 字）。

## 练习 B：静态宪法六段大纲

复制下表并 填空（每格 1—3 条要点即可，不必长文）。

| 静态模块 | 你的要点 |
|------|------|
| 身份定位 |      |
| 系统规范 |      |
| 任务哲学 |      |
| 风险规范 |      |
| 工具总则 |      |
| 语气风格 |      |

### 任务哲学短章（模板）

```
## 任务哲学 (Doing tasks)

- 不添加用户未要求的无关功能。
- 避免过度抽象；优先最小可行修改。
- 不堆砌无意义注释。
- 不对耗时给出武断估计。
- 出错时先诊断再更换策略。
- 如实汇报结果与限制。
```

要求：至少 改写其中 2 条 以贴合你的 Agent 场景（例如记账助手强调「不臆造金额来源」）。



## 练习 C：动态政策六段大纲

| 动态模块                               | 你将注入什么字段？（列变量名） |
|------------------------------------|-----------------|
| 会话引导                               |                 |
| 记忆（≤5）                             |                 |
| 环境信息                               |                 |
| 项目文件（类 <a href="#">CLAUDE.md</a> ） |                 |
| 插件说明（类 MCP）                        |                 |
| Token 预算                           |                 |

## 练习 D：拼装函数（手写伪代码）

在下列骨架中补全你的函数名与字段：

```
const SYSTEM_PROMPT_DYNAMIC_BOUNDARY = "<<<<MY_AGENT_DYNAMIC>>>>";

function getSystemPrompt(ctx: MySessionContext): string {
  const staticConstitution = buildStaticConstitution(/* ? */);
  const dynamicPolicy = buildDynamicPolicy(ctx);
  return `${staticConstitution}\n\n${SYSTEM_PROMPT_DYNAMIC_BOUNDARY}\n\n${dynamicPolicy}`;
}
```

交付物：实现 `buildStaticConstitution` 与 `buildDynamicPolicy` 的 返回字符串大纲（可用 Markdown 拼接）。

## 练习 E：两个工具的 `prompt.ts`

### 工具 1（只读型）示例骨架

```
export const queryLedgerPrompt = `
## QueryLedgerTool
用途：按日期范围查询账本条目。
何时用：用户问「本周餐饮支出」。
何时不用：需要写入新支出 → 用 AppendExpenseTool。
参数：from (ISO date), to (ISO date), category?: string
.trim();`
```

## 工具 2（写入型）示例骨架

```
export const appendExpensePrompt = `
## AppendExpenseTool
用途：追加一条支出记录。
硬规则：写入前必须先 QueryLedgerTool 读取重叠区间，避免重复记账（若你的业务需要）。
`.trim();
```

要求：把工具名/参数改成 你的 Agent 的虚构 API，但必须包含：

- 何时用 / 何时不用
- 参数契约
- 至少 1 条反模式

## 练习 F：缓存失效清单

至少列出 5 条 你会主动避免 的做法，映射到本篇 5.6 的陷阱编号（可重复类型）。

| 你的反模式                           | 对应陷阱类型 | 你的规避  |
|---------------------------------|--------|-------|
| 例：在静态区写 <code>Date.now()</code> | #4 时间戳 | 改放动态区 |
|                                 |        |       |
|                                 |        |       |

## Mermaid：画出你的 Agent 拼装流

将方框文字换成你的命名：

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid：你的工具与边界关系

[Mermaid 图表 — 请在在线版本中查看]

## 评分 rubric（自评用）

| 项目   | 及格         | 良好          |
|------|------------|-------------|
| 静动分离 | 有边界标记      | 静态无用户路径/时间戳 |
| 铁律   | 覆盖 4+ 条精神  | 与场景定制化结合    |
| 工具手册 | 2 个工具有意图分区 | 含反模式与并行提示   |
| 缓存意识 | 5 条风险      | 能联系到成本指标    |

## 参考答案思路（仅提示，非唯一解）

- 记账助手：风险规范侧重 不连接真实银行密钥；动态块放 货币与Locale。
- 文档巡检：工具总则强调 先读后改链接；动态块放 仓库根路径与分支名。
- 作业辅助：任务哲学强调 启发不代写；风险规范写 学术诚信边界。

## 交付检查清单

- [ ] `getSystemPrompt()` 草图可运行（伪代码级）
- [ ] 边界之上无用户数据
- [ ] `getSimpleDoingTasksSection` 风格短章已场景化
- [ ] 两个 `prompt.ts` 草稿完成
- [ ] 缓存风险表  $\geq 5$  行

## 导航

- [← 5.9 策略对比](#)
- 本篇完 · 返回 [5.1 索引](#)

## 第6篇：工具系统

### 第六部分 · 工具系统 (6.1) — 42 个内置工具全景

**导航：**本部分共 12 节。本节为总览；后续各节深入接口、治理流水线、各类工具与工程实践。

#### 学习目标

完成本节学习后，你应该能够：

1. **枚举** Claude Code 生态中约 42 个内置工具，并按「文件 / 搜索 / 执行 / 外部」四大类归类。
2. **解释** 每类工具在对话中的典型职责，以及为何需要如此拆分（权限、安全、Token 效率）。
3. **对照** 官方或实现中的命名习惯（如 `FileRead`、`Bash`），在文档与配置中快速定位能力边界。
4. **预判** 哪些操作会触发权限提示、子进程或网络 I/O，从而在架构设计时预留 Hook 与审计点。

#### 生活类比：工具箱与专柜

把 Claude Code 的工具系统想象成一家**大型五金超市**：

- **文件类**像「档案柜 + 订书机」：只能动你授权范围内的纸张（工作区），改错可版本管理回滚。
- **搜索类**像「索引员 + 放大镜」：不直接改货，只告诉你东西在哪、长什么样。
- **执行类**像「车间里的机床」：能切能焊，但必须戴护目镜（权限与沙箱），误操作代价高。
- **外部类**像「电话与快递」：连到外界（网页、搜索 API、MCP 服务），需要额外信任与配额。

42 个工具不是 42 把一模一样的锤子，而是**分工明确的专柜**——柜员（运行时）按流程取货、登记、再交给顾客（模型）结构化结果。

## 42 工具功能分类总表

下表按**功能域**归纳内置能力（具体名称以你所用版本为准；数量为教学用「约 42」口径）。空单元格表示该域下无对应子类或合并到其他节讲解。

| 功能域 | 代表工具 / 能力                              | 主要职责              | 典型风险         |
|-----|----------------------------------------|-------------------|--------------|
| 文件  | FileRead、FileEdit、NotebookEdit         | 读/写文本与笔记本单元格      | 覆盖写、路径穿越     |
| 搜索  | Glob、Grep、LSP、ToolSearch               | 模式匹配、符号与引用、按需发现工具 | 大仓库扫描耗Token  |
| 执行  | Bash、PowerShell、Agent、REPL             | Shell/子代理/交互解释器   | 任意命令、数据外泄    |
| 外部  | WebFetch、WebSearch、MCP、ReadMcpResource | HTTP、检索、MCP 工具与资源 | SSRF、提示注入、隐私 |

### 子类速查（教学用清单）

| 类别       | 工具族（示例）                          | 一句话         |
|----------|----------------------------------|-------------|
| File     | Read / Edit / Notebook           | 先读后写（见 6.5） |
| Search   | Glob / Grep / LSP / ToolSearch   | 本地索引与「工具目录」 |
| Exec     | Bash / PowerShell / Agent / REPL | 强治理与隔离      |
| External | Web / MCP                        | 出站与插件边界     |

## Mermaid：工具全景与数据流

下列流程图展示**用户意图** → **模型选工具** → **治理层** → **工具实现** → **结构化回传**的粗粒度关系（节点 ID 无空格，标签用引号包裹中文）。

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段（概念性）：工具注册伪代码

下列伪代码展示「工具名 → 实现 → Schema」的常见组织方式，与真实仓库结构类似，便于你在阅读源码时对照（非逐行拷贝自某一版本）。

```
// 概念：工具注册表 (Registry)
type ToolName =
  | "FileRead"
  | "FileEdit"
  | "Glob"
  | "Grep"
  | "Bash"
  | "WebFetch"
  | "mcp__server__toolName";

interface ToolDefinition<I, O> {
  name: ToolName;
  description: string;
  inputSchema: ZodSchema<I>;
  outputSchema: ZodSchema<O>;
  isReadOnly: boolean;
  isConcurrencySafe: boolean;
  call(input: I): Promise<O>;
}

const registry = new Map<ToolName, ToolDefinition<unknown, unknown>>();

function registerTool<I, O>(def: ToolDefinition<I, O>) {
  registry.set(def.name, def as ToolDefinition<unknown, unknown>);
}
```

要点：

- 名称既是模型可见的 capability，也常作为遥测与权限策略的键。
- **Schema**在运行前约束参数，减少「胡编字段」导致的失败。
- 布尔标记参与 fail-closed 默认值（详见 6.10）。

## 与后续章节的映射

| 本节概念                   | 深入章节                             |
|------------------------|----------------------------------|
| Zod 与 Tool 接口          | <a href="#">6.2 工具接口设计</a>       |
| 解析→Hook→权限             | <a href="#">6.3 十四步治理流水线</a>     |
| Bash / Shell           | <a href="#">6.4 BashTool</a>     |
| 先读后写                   | <a href="#">6.5 文件三件套</a>        |
| Glob / Grep / LSP      | <a href="#">6.6 搜索工具族</a>        |
| 子代理                    | <a href="#">6.7 AgentTool</a>    |
| Web                    | <a href="#">6.8 外部工具</a>         |
| MCP                    | <a href="#">6.9 MCP 集成</a>       |
| 默认不安全                  | <a href="#">6.10 Fail-closed</a> |
| ToolSearch / prompt.ts | <a href="#">6.11 延迟加载</a>        |
| 自建工具                   | <a href="#">6.12 实践</a>          |

## 小结

- 42 个内置工具可按 文件 / 搜索 / 执行 / 外部 理解，核心是权限、安全与 Token 经济的平衡。
- 总图帮助你把「模型选工具」放到「治理流水线」之下，避免把工具当成裸 API。
- 下一节将聚焦 Tool 接口与 Zod Schema，说明输入输出如何成为可校验的契约。

## 自测题

1. 为何「执行类」与「文件类」通常要分开治理？
2. ToolSearch 更靠近「搜索」还是「元工具」？它解决什么问题？
3. MCP 工具命名空间化（如 mcp\_... ）对权限策略有什么好处？

下一节：[6.2 Tool 接口设计 \(Zod 校验、输入输出 schema\)](#)

## 四十二工具教学枚举（按域占位）

下表为课程用占位清单：真实产品版本可能增删或改名，学习时以官方文档与源码注册表为准。目的有二：一是建立心智地图，二是练习权限分级思维。

| 序号    | 教学用名称           | 域     | 备注                    |
|-------|-----------------|-------|-----------------------|
| 1     | FileRead        | 文件    | 只读                    |
| 2     | FileEdit        | 文件    | 先读后写                  |
| 3     | NotebookEdit    | 文件    | 单元格级                  |
| 4     | Glob            | 搜索    | 路径模式                  |
| 5     | Grep            | 搜索    | 正则/文本                 |
| 6     | LSP             | 搜索/语义 | 需语言服务                 |
| 7     | ToolSearch      | 搜索/元  | 延迟发现                  |
| 8     | Bash            | 执行    | AST+黑名单               |
| 9     | PowerShell      | 执行    | Windows               |
| 10    | Agent           | 执行    | 子代理                   |
| 11    | REPL            | 执行    | 交互解释器                 |
| 12    | WebFetch        | 外部    | 受控 HTTP               |
| 13    | WebSearch       | 外部    | 检索 API                |
| 14    | MCP（动态 tools）   | 外部    | 多工具                   |
| 15    | ReadMcpResource | 外部    | 资源 URI                |
| 16—42 | 其余内置与组合能力       | 混合    | 视版本：压缩、多模态、诊断、VCS 辅助等 |

**说明：**第 16—42 项在不同发行版中对应**具体命名**（如编辑器的专用工具、测试运行器、Git 状态查询等）。教学上可把它们理解为：**在四大域之上叠加快捷封装**，仍服从同一套 **Tool 接口 + 治理流水线**。



## 术语速查

| 术语          | 解释                          |
|-------------|-----------------------------|
| Tool        | 模型可调用的原子能力，含 schema 与实现     |
| Schema      | 用 Zod/JSON Schema 描述的入参出参契约 |
| Hook        | Pre/Post 注入点，可改写、拒绝或审计      |
| Fail-closed | 信息不足时选更保守默认（如非并发安全）         |
| 延迟加载        | 推迟注入完整工具描述以降低首包 Token       |
| MCP         | 第三方工具与资源的插件协议               |

## 常见问题（FAQ）

- 问：42 是硬性数字吗？

答：教学口径。实际以注册表为准；架构方法比精确个数重要。
- 问：为何执行类工具特别少但风险最高？

答：Shell/子进程/子代理的 blast radius 大，产品侧往往用更少工具 + 更强策略而非堆数量。
- 问：模型选错工具怎么办？

答：靠 描述质量、prompt.ts、ToolSearch、以及 Schema 错误可恢复 四件事一起兜。
- 问：内置工具与 MCP 工具能混排吗？

答：可以，但必须在 UI 与策略上区分信任级别（见 6.9、6.10）。

## 版本与实现差异（阅读建议）

| 你手中的产物    | 建议对照                     |
|-----------|--------------------------|
| 开源宿主      | registry 初始化文件、tools/ 目录 |
| 闭源 IDE 插件 | 官方「工具列表」文档               |
| 企业私有化     | 内部策略包：允许的工具子集            |

学习路径推荐：6.1 → 6.2 → 6.3 建立骨架，再按兴趣跳读 Bash / 文件 / MCP。

## 本节知识点回顾（ checklist ）

- [ ] 能不看资料画出四大域工具地图
- [ ] 能解释「治理层」为何夹在模型与工具之间
- [ ] 能举例说明哪类工具应默认非 `concurrencySafe`
- [ ] 能说明 Web 与 MCP 在信任模型上的差异

## 延伸阅读（与本部分其他节）

| 主题            | 章节                                         |
|---------------|--------------------------------------------|
| Zod 双端校验      | <a href="#">6.2</a>                        |
| PreToolUse 顺序 | <a href="#">6.3</a>                        |
| curl 替代       | <a href="#">6.4</a> · <a href="#">6.8</a>  |
| 先读后写          | <a href="#">6.5</a>                        |
| ToolSearch    | <a href="#">6.6</a> · <a href="#">6.11</a> |

# 6.2 Tool 接口设计 — Zod 校验与输入输出 Schema

前置阅读：[6.1 42 个工具全景](#)

## 学习目标

完成本节学习后，你应该能够：

1. **描述** Tool 抽象中常见的字段：`name`、`description`、`inputSchema`、`outputSchema`、`call` 等。
2. **说明** 为何选用 **Zod**（或同类库）做运行时校验，而不是仅依赖 TypeScript 编译期类型。
3. **区分** 「模型生成的 JSON」与「运行时强类型对象」，并理解校验失败时的 fail-closed 行为。
4. **设计** 一份最小可用的工具 Schema：必填字段、可选字段、联合类型与错误消息友好化。
5. **关联** 后续治理流水线中「解析输入 → 校验 Schema」两步的职责边界。

# 生活类比：海关申报单与开箱查验

Tool 的输入输出 Schema 就像出入境申报单：

- 旅客（模型）填写的每一项都必须落在**规定栏目**里；格式不对，柜台直接拒收（校验失败）。
- **申报单**是「对外可见的表格」（JSON / 结构化字段），**开箱**是「实际货物」（运行时对象），两者必须一致。
- **输出 Schema** 则是「回程时必须带回的收据格式」——海关（宿主应用）只接受标准格式的回执，避免乱七八糟的自然语言混入业务逻辑。

Zod 在这里扮演**既看表格又看货物的稽查员**：先问「有没有带违禁品字段」，再问「类型对不对」。

## Tool 接口核心要素（表）

| 要素                        | 作用       | 与模型的关系              | 与宿主的关系         |
|---------------------------|----------|---------------------|----------------|
| <code>name</code>         | 稳定标识、遥测键 | function calling 名称 | 注册表、权限策略       |
| <code>description</code>  | 能力说明     | 选工具依据               | 可裁剪/压缩进 system |
| <code>inputSchema</code>  | 入参契约     | 生成合法参数              | 拒绝非法调用         |
| <code>outputSchema</code> | 出参契约     | 收到可解析结果             | 管道下游消费         |
| <code>call()</code>       | 副作用边界    | 不直接可见               | 审计、超时、取消       |

## Zod：为何需要运行时校验

TypeScript 类型在编译后消失，而模型产出的参数来自**网络与解析**，属于**不可信输入**。Zod 提供：

1. **运行时解析** — `safeParse` / `parse` 将未知数据变为强类型或明确错误。
2. **可序列化的 JSON Schema 对齐** — 不少实现会把 Zod 转为 JSON Schema 供 API 描述。
3. **精细化错误** — `path` + `message` 便于回灌给模型自我修正（与治理流水线「二次修正输入」呼应）。

## 源码片段：最小 Tool 定义

```
import { z } from "zod";

// 输入：路径 + 可选编码
const FileReadInput = z.object({
  path: z.string().min(1, "path 不能为空"),
  encoding: z.enum(["utf8", "base64"]).optional(),
});

// 输出：内容与元数据
const FileReadOutput = z.object({
  content: z.string(),
  lineCount: z.number().int().nonnegative(),
  truncated: z.boolean(),
});

type FileReadInputT = z.infer<typeof FileReadInput>;
type FileReadOutputT = z.infer<typeof FileReadOutput>;

async function fileReadToolCall(input: FileReadInputT): Promise<FileReadOutputT> {
  // ... 实现省略
  return {
    content: "",
    lineCount: 0,
    truncated: false,
  };
}

function validateFileReadInput(raw: unknown) {
  const r = FileReadInput.safeParse(raw);
  if (!r.success) {
    return { ok: false as const, error: r.error.flatten() };
  }
  return { ok: true as const, data: r.data };
}
```

要点：

- `unknown` 入口强制你先校验再使用。
- `z.infer` 保证 Schema 即类型源（SSOT）。
- 错误结构可映射为工具层的标准 `ToolError`。

## 源码片段：输出也要校验

部分实现会对工具返回值再做一次 Zod 校验，防止实现 bug 或子进程污染：

```
async function invokeTool(rawInput: unknown) {
  const inParsed = FileReadInput.safeParse(rawInput);
  if (!inParsed.success) {
    return { type: "invalid_input", issues: inParsed.error.issues };
  }

  const rawOut = await fileReadToolCall(inParsed.data);
  const outParsed = FileReadOutput.safeParse(rawOut);
  if (!outParsed.success) {
    return { type: "invalid_output", issues: outParsed.error.issues };
  }

  return { type: "ok", data: outParsed.data };
}
```

这体现了 **双端契约**：模型与工具实现都必须尊重 Schema。

---

## Mermaid：校验在流水线中的位置

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

---

## 进阶：联合类型与判别字段

工具参数常出现「模式切换」，用 `discriminatedUnion` 可减少非法组合：

```
const SearchMode = z.discriminatedUnion("mode", [
  z.object({ mode: z.literal("text"), pattern: z.string() }),
  z.object({ mode: z.literal("regex"), pattern: z.string(), flags: z.string().optional() }),
]);
```

对模型而言，清晰的枚举与字面量比「一堆可选 boolean」更容易填对。

## 与 JSON Schema 的互操作（概念表）

| 场景     | Zod 优势                     | JSON Schema 优势 |
|--------|----------------------------|----------------|
| TS 一体化 | <code>z.infer</code> 天然同步  | 需代码生成或手写类型     |
| 动态注册   | 可在运行时组合                    | 生态广、部分平台原生     |
| 错误信息   | <code>issue.path</code> 友好 | 依赖实现           |

许多产品会 **Zod → JSON Schema** 导出，用于 OpenAPI 或客户端。

## 常见反模式

| 反模式                                | 后果        | 建议                            |
|------------------------------------|-----------|-------------------------------|
| 仅用 TS 类型、不校验                       | 运行时注入/缺字段 | 入口 <code>unknown</code> + Zod |
| Schema 过宽（全 <code>z.any()</code> ） | 失去治理意义    | 收紧字段，版本化演进                    |
| 输出不校验                              | 静默数据损坏    | 对关键工具校验 output                |
| 错误信息泄露路径                           | 信息暴露      | 对用户/模型分级脱敏                    |

## 小结

- **Tool 接口** = 名称 + 描述 + **输入/输出 Schema** + **call**；Schema 是模型与宿主之间的**法律合同**。
- **Zod** 把「模型 JSON」变成「可信对象」，并向上层提供**可恢复的错误**。
- **双端校验**（入参 + 出参）在关键工具上值得投入。

## 自测题

1. 若省略输出 Schema 校验，攻击面或故障面分别可能是什么？
2. `safeParse` 与 `parse` 在治理流水线里哪个更适合默认使用？为什么？
3. 如何将 Zod 错误映射为「可供模型二次修正」的提示语？

上一节：[6.1 全景](#) · 下一节：[6.3 十四步治理流水线](#)

## 6.3 十四步治理流水线 — 从解析到结构化输出

前置阅读: [6.2 Tool 接口设计](#)

### 学习目标

完成本节学习后，你应该能够：

1. **按顺序复述** 14 步治理流水线各步职责：从解析输入到 PostToolUse、结构化输出。
2. **解释** `PreToolUse` 为何可以**改写或拦截**调用，以及它与「正式权限决策」的次序关系。
3. **说明**「投机分类器」在工程上的目的：预判风险、提前降速或加策略。
4. **关联** 遥测日志、二次修正输入与 `tool.call()` 之间的数据流。
5. **对照** 自家产品：哪些步骤可省略、哪些**不可省略**（安全与合规）。

### 生活类比：银行大额转账

把一次工具调用想象成柜台办理大额转账：

1. **解析**客户填单（JSON）是否缺栏。
2. **Schema** 校验格式是否合法。
3. `validateInput` 业务规则（收款人黑名单等）。
4. **投机分类器**像风控预审：疑似诈骗先标红。
5. `PreToolUse` 像主管改单：可要求补材料或直接拒办。
6. **正式权限**像授权岗：看角色、额度、时间。
7. **二次修正**像客户签字确认最终金额。
8. `tool.call()` 真正划账。
9. **遥测**记账本、供审计。
10. `PostToolUse` 事后短信通知、触发反洗钱规则。
11. **结构化输出**给客户一张**标准回执**（非手写便条）。

流水线越长，**延迟**可能越高，但**事故率**通常越低——工程上要在二者间折中。

## 十四步清单（总表）

| 步骤 | 名称（教学用）          | 一句话职责                  |
|----|------------------|------------------------|
| 1  | 解析输入             | 将原始 tool_call 解析为结构化对象 |
| 2  | 校验 Schema        | Zod / 等价校验入参形状与类型      |
| 3  | validateInput    | 业务层额外规则（路径白名单等）        |
| 4  | 投机分类器            | 启发式 / 小模型预判风险等级        |
| 5  | PreToolUse Hook  | 可改写参数、短路拒绝、附加元数据       |
| 6  | 正式权限决策           | 用户批准、策略引擎、角色 ACL       |
| 7  | 二次修正输入           | 权限结果回填、参数归一化           |
| 8  | tool.call()      | 执行真实副作用或只读操作           |
| 9  | 遥测日志             | 事件、耗时、结果摘要、关联 traceId  |
| 10 | PostToolUse Hook | 审计、缓存失效、副作用链           |
| 11 | 结构化输出            | 映射为统一 ToolResult       |
| 12 | （扩展）重试策略         | 可选：可恢复错误的有限重试          |
| 13 | （扩展）速率限制         | 可选：令牌桶防刷               |
| 14 | （扩展）结果裁剪         | 可选：大 payload 截断与哈希存档   |

**说明：**不同实现可能合并或拆分步骤；上表在「核心 11 步」基础上补充 3 个常见扩展，凑满**教学用 14 步**口径，便于与「解析→校验→Hook→权限→执行→日志→输出」完整闭环对齐。若你内部分档为严格 11 步，可将 12–14 视为子阶段。

为与需求中的**精确 14 步**一致，下列**编号版**与开篇需求逐条对应：



| 步号 | 阶段                      |
|----|-------------------------|
| 1  | 解析输入                    |
| 2  | 校验 Schema               |
| 3  | validateInput           |
| 4  | 投机分类器预判风险               |
| 5  | PreToolUse Hook（可改写/拦截） |
| 6  | 正式权限决策                  |
| 7  | 二次修正输入                  |
| 8  | tool.call()             |
| 9  | 遥测日志                    |
| 10 | PostToolUse Hook        |
| 11 | 结构化输出                   |
| 12 | （实现可选）输出 Schema 再校验     |
| 13 | （实现可选）敏感字段脱敏            |
| 14 | （实现可选）返回模型前的压缩/引用       |

## Mermaid：完整流程图（主路径 + 拦截）

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：流水线骨架（伪代码）

```
type ToolCallResult =
  | { status: "ok"; output: unknown }
  | { status: "error"; code: string; message: string; retryable?: boolean };

async function runGovernedToolCall(
  raw: unknown,
  tool: ToolDefinition<unknown, unknown>,
  ctx: ExecutionContext,
): Promise<ToolCallResult> {
  // 1 解析输入
  const parsed = parseToolCallPayload(raw);
  if (!parsed.ok) return { status: "error", code: "parse", message: parsed.error };

  // 2 校验 Schema
  const schemaResult = tool.inputSchema.safeParse(parsed.data);
  if (!schemaResult.success) {
    return { status: "error", code: "schema", message: formatZod(schemaResult.error) };
  }

  // 3 validateInput
  const vi = await tool.validateInput?.(schemaResult.data, ctx);
  if (vi && !vi.ok) return { status: "error", code: "validateInput", message: vi.reason };

  // 4 投机分类器
  const risk = await speculativeClassifier(tool.name, schemaResult.data, ctx);

  // 5 PreToolUse
  const pre = await ctx.hooks.preToolUse({ tool: tool.name, input: schemaResult.data, risk });
  if (pre.action === "deny") return { status: "error", code: "pre_hook", message: pre.reason };
  let input = pre.input ?? schemaResult.data;

  // 6 正式权限
  const auth = await ctx.policy.authorize({ tool: tool.name, input, risk });
  if (!auth.allowed) return { status: "error", code: "auth", message: auth.reason };

  // 7 二次修正
  input = auth.normalizedInput ?? input;

  // 8 call
  const started = Date.now();
  let output: unknown;
  try {
    output = await tool.call(input, ctx);
  } catch (e) {
    emitTelemetry({ tool: tool.name, phase: "call", ms: Date.now() - started, error: String(e) });
    return { status: "error", code: "exception", message: String(e) };
  }

  // 9 遥测
  emitTelemetry({ tool: tool.name, phase: "call", ms: Date.now() - started, ok: true });

  // 10 PostToolUse
  const post = await ctx.hooks.postToolUse({ tool: tool.name, input, output });
```

```
output = post.output ?? output;

// 11-14 结构化、可选校验、脱敏、压缩
const structured = structureToolResult(tool.name, output);
return { status: "ok", output: structured };
}
```

## 关键交互：PreToolUse 与权限的边界

| 维度    | PreToolUse | 正式权限            |
|-------|------------|-----------------|
| 典型实现  | Hook 函数、插件 | UI 确认、RBAC、策略引擎 |
| 能否改参数 | 常允许        | 一般只追加约束/归一化     |
| 失败体验  | 快速拒绝、可带建议  | 用户可见的授权窗        |
| 测试策略  | 单元 + 集成    | E2E + 合规用例      |

反模式：把「是否允许访问网络」只放在 PostToolUse —— 太迟，流量已发出。

## 遥测字段建议（表）

| 字段               | 用途                  |
|------------------|---------------------|
| traceId / spanId | 分布式追踪               |
| toolName         | 聚合 QPS、错误率          |
| riskScore        | 分类器效果评估             |
| durationMs       | 性能回归                |
| outcome          | ok / denied / error |
| payloadBytes     | 大结果监控               |

## 小结

- 治理流水线把「模型想做的事」变成「可被审计、可被拒绝、可修正的事」。
- PreToolUse 与 权限 是两道闸门；投机分类器为闸门提供先验。
- 遥测 + PostToolUse 连接运维与合规，结构化输出连接下一轮对话。

## 自测题

1. 若 PreToolUse 允许无限改写参数，可能引入何种攻击面？
2. 「二次修正输入」与「Schema 校验」有何本质区别？
3. 哪些步骤在只读工具上可以简化仍不破坏安全？

上一节： [6.2](#) · 下一节： [6.4 BashTool](#)

## 6.4 BashTool — AST 分析、Git 安全与命令黑名单

前置阅读： [6.3 十四步治理流水线](#)

### 学习目标

完成本节学习后，你应该能够：

1. 说明 BashTool 为何需要 **AST 级**（或 shell 解析器级）命令分析，而不是简单字符串 `includes`。
2. 列举 Git 安全协议常见规则：禁止强制推送、保护分支、凭证泄露场景。
3. 描述 环境变量管理策略：继承、白名单、敏感键过滤。
4. 解释 `curl` / `wget` 默认禁用的动机与绕过成本。
5. 将 Bash 调用映射到治理流水线：权限、PreToolUse、遥测字段。

### 生活类比：化工厂反应釜

Bash 像反应釜上的总阀门：一开就可能同时影响温度、压力、进料。AST 分析像读工艺流程图——你得知道并行管道里每一支走向哪里，而不是只看阀门上的标签字。

`curl` / `wget` 默认禁用，类似禁止私自外接软管：不是永远不能接，而是要经过额外审批（专用策略或替代工具如受控的 `WebFetch`）。

## BashTool 能力边界（表）

| 能力      | 说明                     | 典型治理      |
|---------|------------------------|-----------|
| 命令解析    | shell 词法/语法            | 防注入、拆分管道  |
| 工作目录    | <code>cwd</code> 约束    | 锁定仓库根     |
| 超时 / 取消 | SIGTERM → SIGKILL      | 防僵尸       |
| 输出上限    | stdout/stderr 截断       | Token 与内存 |
| 环境      | <code>env</code> 注入与过滤 | 密钥防泄漏     |

## AST 解析：为何不够「字符串匹配」

下列命令都包含 `rm` 子串，但风险完全不同：

```
echo "not rm really"
rm -rf /
git commit -m "fix rm bug"
```

仅靠 `includes("rm")` 会产生大量误报；仅靠黑名单单词会漏掉：

```
$(which destructive_cmd)
bash -c '...嵌套...'
```

工程方向：

- 使用 **shell parser**（如 bash 的 AST、或 `shlex` 类分词 + 有限状态）得到**命令名与参数节点**。
- 对**简单场景**可退化为「首个 token 为命令」的启发式，但需标注置信度。

## 源码片段：命令名提取（概念）

```
import { parse } from "some-shell-parser"; // 示意

interface ParsedCommand {
  argv0: string;
  args: string[];
  pipelines: boolean;
  hasSubshell: boolean;
}

function extractTopLevelCommands(script: string): ParsedCommand[] {
  const ast = parse(script);
  // 遍历 AST: Command、Pipeline、CompoundCommand 等节点
  return flattenToCommands(ast);
}

function isBlockedCommand(argv0: string, blocklist: Set<string>): boolean {
  const base = argv0.replace(/^.*/, ""); // 去路径
  return blocklist.has(base);
}
```

要点： `argv0` 规范化（basename）可减少 `./curl` 绕过。

## Git 安全协议（教学清单）

| 规则                                                                          | 目的             |
|-----------------------------------------------------------------------------|----------------|
| 禁止 <code>git push --force</code> 到共享主分支                                     | 防历史重写事故        |
| 限制 <code>git filter-branch</code> / 大范围重写                                   | 防供应链与审计断裂      |
| 拦截含 <code>token</code> 、 <code>password</code> 的 <code>git clone</code> URL | 防凭证写入 shell 历史 |
| 对 <code>.git</code> 外操作只读模式可选                                               | 防误改对象库         |
| <code>git config</code> 写全局配置需额外授权                                          | 防持久化后门         |

这些规则可放在 `validateInput` 或 `PreToolUse`：在进程 `spawn` 之前拦截。

## 环境变量管理

| 策略      | 描述                                            | 适用  |
|---------|-----------------------------------------------|-----|
| 继承宿主    | 简单，易泄露                                        | 开发机 |
| 白名单     | 仅 <code>PATH</code> 、 <code>HOME</code> 等     | CI  |
| 注入只读元数据 | <code>CI=true</code> 、 <code>REPO_ROOT</code> | 流水线 |
| 剥离敏感键   | 去掉 <code>AWS_*</code> 、 <code>OPENAI_*</code> | 多租户 |

```
const SAFE_ENV_KEYS = new Set(["PATH", "HOME", "LANG", "CI"]);

function filterEnv(env: NodeJS.ProcessEnv): NodeJS.ProcessEnv {
  const out: NodeJS.ProcessEnv = {};
  for (const k of SAFE_ENV_KEYS) if (k in env) out[k] = env[k]!;
  return out;
}
```

## 命令黑名单：`curl` / `wget` 默认禁

| 命令                                    | 风险                         | 常见替代                              |
|---------------------------------------|----------------------------|-----------------------------------|
| <code>curl</code>                     | 任意出站、管道到 <code>bash</code> | <code>WebFetch</code> （受控 URL 策略） |
| <code>wget</code>                     | 同左                         | 同左                                |
| <code>nc</code> / <code>netcat</code> | 反向 shell                   | 禁止或审批                             |
| <code>ssh</code>                      | 隧道与数据渗出                    | 审批 + 目标白名单                        |

默认禁不等于永远不能：可通过 策略覆盖、专用配置文件 或 拆分工具（例如只允许 `curl` 访问内网镜像且经 `PreHook` 改写 URL）。

## Mermaid：Bash 调用决策

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## PowerShell 侧记

在 Windows 路径下，`PowerShell` 工具面临类似问题：**别名、编码、执行策略**。治理上可复用同一流水线，替换解析器与黑名单表。

### 测试用例建议（表）

| 用例                                        | 期望      |
|-------------------------------------------|---------|
| <code>echo hello</code>                   | 允许      |
| <code>curl https://evil</code>            | 默认拒绝    |
| <code>git push --force origin main</code> | 拒绝或二次确认 |
| <code>rm -rf /</code>                     | 拒绝      |
| 嵌套子 shell 调用黑名单                           | 解析器应识别  |

### 遥测与审计字段

| 字段                         | 说明             |
|----------------------------|----------------|
| <code>argv0_flat</code>    | 顶层命令列表 JSON    |
| <code>blockedReason</code> | 黑名单 / Git / 策略 |
| <code>exitCode</code>      | 进程退出码          |
| <code>truncated</code>     | 输出是否裁剪         |

### 常见反模式

| 反模式           | 后果          |
|---------------|-------------|
| 正则匹配整条命令      | 极易绕过        |
| 无超时           | 挂死占用 worker |
| 全量环境继承        | 密钥进子进程      |
| 超大 stdout 不截断 | OOM 与账单爆炸   |



## 小结

---

- BashTool 的核心是解析 → 策略 → 受控执行 → 裁剪回传。
  - Git 安全与出站黑名单是高频事故源，应前置在 spawn 之前。
  - 环境变量是第二攻击面，白名单 + 剥离是稳妥默认。
- 

## 自测题

---

1. 为何「首个 token」启发式在 `env VAR=value cmd` 场景下可能误判？
2. 若允许 `curl`，PreToolUse 可插入哪些约束降低 SSRF？
3. AST 解析失败时，fail-closed 应「拒绝」还是「降级为人工审核」？

上一节：[6.3](#) · 下一节：[6.5 文件工具](#)

---

## 6.5 FileRead / FileEdit / NotebookEdit — 先读后写 强制

---

前置阅读：[6.1 全景](#) · [6.3 治理流水线](#)

---

## 学习目标

---

完成本节学习后，你应该能够：

1. 解释 `FileEditTool` 强制先 `FileReadTool` 的产品与安全理由。
  2. 描述「盲改拦截」在实现上的几种形态：会话缓存、ETag、版本向量。
  3. 对比 文本 `FileEdit` 与 `NotebookEdit` 在 Schema 与冲突处理上的差异。
  4. 列举 路径规范化、`..` 穿越、符号链接跟随等常见校验点。
  5. 将 文件三件套接入 PreToolUse / 权限：按扩展名、目录、行数阈值分级。
-

## 生活类比：手术核对清单

先读后改像手术前核对患者手环：你不能凭记忆动刀。`FileEdit` 在没有当前文件内容上下文（或等价的版本令牌）时拒绝执行，就像**未核对 ID 拒绝麻醉**——麻烦一秒，避免灾难级误切。

**Notebook** 则是「多部位手术」：每个单元格有独立 ID，改错一格不影响整本病历的页码体系（但若结构错乱仍会事故）。

## 三工具职责表

| 工具                        | 主要职责                     | 典型输入                                                               | 典型输出                                                                   |
|---------------------------|--------------------------|--------------------------------------------------------------------|------------------------------------------------------------------------|
| <code>FileRead</code>     | 读文本/二进制元数据               | <code>path</code> , <code>offset?</code>                           | <code>content</code> , <code>truncated</code> , <code>lineCount</code> |
| <code>FileEdit</code>     | 补丁/替换/整段写                | <code>path</code> , <code>old</code> / <code>new</code> 或结构化 diff  | <code>applied</code> , <code>conflicts</code>                          |
| <code>NotebookEdit</code> | 改 <code>.ipynb</code> 单元 | <code>cellId</code> , <code>source</code> , <code>cellType?</code> | <code>notebookVersion</code>                                           |

## 强制先读：策略表

| 机制                        | 做法                                      | 优点      | 缺点      |
|---------------------------|-----------------------------------------|---------|---------|
| 会话指纹                      | 近期 <code>FileRead</code> 记录 path + hash | 实现快     | 会话外不共享  |
| 显式 <code>readToken</code> | Read 返回 token, Edit 必须带上                | 强一致     | API 冗长  |
| 版本号                       | 文件 mtime / content hash                 | 可检测并发修改 | 时钟/缓存问题 |

## 源码片段：盲改拦截（概念）

```
interface SessionReadCache {
  path: string;
  contentHash: string;
  readAt: number;
}

const recentReads = new Map<string, SessionReadCache>();

async function fileReadTool(input: { path: string }): Promise<{ content: string; readToken:
  const content = await fs.readFile(input.path, "utf8");
  const contentHash = sha256(content);
  const readToken = sign({ path: input.path, contentHash, ts: Date.now() });
  recentReads.set(input.path, { path: input.path, contentHash, readAt: Date.now() });
  return { content, readToken };
}

async function fileEditTool(input: { path: string; readToken: string; patch: string }) {
  const verified = verifyReadToken(input.readToken);
  if (!verified.ok) {
    return { error: "blind_edit_blocked", hint: "请先使用 FileRead 获取 readToken" };
  }
  const snap = recentReads.get(input.path);
  if (!snap || snap.contentHash !== verified.payload.contentHash) {
    return { error: "stale_read", hint: "文件已变化请重新读取" };
  }
  // ... 应用 patch
}
```

真实产品可能用 更短的 nonce 或 把 read 结果缓存在宿主侧；思想一致：无读凭证则拒绝写。

## NotebookEdit：结构化编辑

Notebook 是 JSON，编辑应尽量避免「整文件字符串替换」：

```
const NotebookCellEdit = z.object({
  path: z.string(),
  cellId: z.string(),
  newSource: z.string(),
  mergeOutputs: z.boolean().optional(),
});
```

生活类比：改幻灯片某一页的备注，不应重排整份 PPTX 的 XML——应定位到 `slideId`。

## Mermaid：先读后写状态机

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## 路径与安全校验清单

| 检查项                               | 说明                                           |
|-----------------------------------|----------------------------------------------|
| <code>path.resolve</code> + 仓库根前缀 | 防 <code>..</code> 逃逸                         |
| 符号链接                              | 是否跟随、是否允许跳出根                                 |
| 二进制大文件                            | 读时截断、禁止编辑                                    |
| <code>.env</code> / 密钥文件          | 更高权限或脱敏展示                                    |
| 行尾与编码                             | 统一 <code>LF</code> / <code>utf8</code> 减少伪冲突 |

## PreToolUse 钩子示例（伪代码）

```
hooks.preToolUse = async ({ tool, input }) => {
  if (tool === "FileEdit") {
    const p = (input as any).path as string;
    if (p.endsWith(".pem") || p.includes(".ssh/")) {
      return { action: "deny", reason: "敏感路径需人工模式" };
    }
  }
  return { action: "allow" };
};
```

## 与 Glob / Grep 的协作

| 步骤      | 工具                                        |
|---------|-------------------------------------------|
| 发现候选文件  | <code>Glob</code>                         |
| 确认内容上下文 | <code>Grep</code> / <code>FileRead</code> |
| 小范围修改   | <code>FileEdit</code>                     |
| 笔记本实验   | <code>NotebookEdit</code>                 |

## 冲突与合并（概念表）

| 场景               | 策略                             |
|------------------|--------------------------------|
| 同时编辑             | hash 不匹配 → 要求重读                |
| patch 不匹配        | 返回 hunks 失败列表                  |
| Notebook cell 删除 | <code>cellId</code> 不存在 → 明确错误 |

## 遥测建议

| 事件                              | 字段                                                |
|---------------------------------|---------------------------------------------------|
| <code>file_read</code>          | <code>bytes</code> , <code>truncated</code>       |
| <code>file_edit</code>          | <code>hunkCount</code> , <code>applyResult</code> |
| <code>blind_edit_blocked</code> | <code>path</code> 哈希化                             |

## 常见反模式

| 反模式            | 后果                |
|----------------|-------------------|
| 允许无读直接写        | 覆盖他人变更、模型幻觉 patch |
| 整文件覆盖大仓库       | 费用与审查困难           |
| Notebook 当纯文本改 | JSON 结构损坏         |

## 小结

- `FileRead` 是 `FileEdit` 的前置条件；盲改拦截把「幻觉式修改」变成可恢复错误。
- `NotebookEdit` 应以 `cell` 身份为第一公民。
- 路径与敏感文件策略应落在 `validateInput` / `PreToolUse`。

## 自测题

1. `readToken` 与 Git 本身提供的版本控制有何互补关系？
2. 若多进程并发写同一文件，仅靠会话缓存是否足够？
3. Notebook 输出区 (outputs) 是否应默认清空以防泄露旧图？

上一节：[6.4 BashTool](#) · 下一节：[6.6 搜索工具](#)

## 6.6 搜索工具族 — Glob / Grep / LSP / ToolSearch

前置阅读：[6.1 全景](#) · [6.11 延迟加载](#) (ToolSearch 可对照阅读)

### 学习目标

完成本节学习后，你应该能够：

1. 区分 `Glob`、`Grep`、`LSP`、`ToolSearch` 的适用场景与返回形态。
2. 解释为何搜索类工具通常标为只读且更易并发安全（与 fail-closed 默认值对照）。
3. 说明 `ToolSearch` 与「42 工具全景」的关系：元层发现能力，延迟注入节省 Token。
4. 列举大仓库上使用 Grep/Glob 的配额与截断策略。
5. 关联 LSP 工具与 IDE 能力：定义跳转、引用查找、符号重命名前置。

### 生活类比：图书馆四件套

- `Glob` = 书架分区目录：「三楼科技类所有书名」。
- `Grep` = 全文索引卡片：「哪本书第几页出现关键词」。

- LSP = 专业编目员：告诉你术语的标准定义与他人引用关系。
- ToolSearch = 问管理员「你们馆有哪些查询服务」——先问目录，再决定用哪台终端。

## 四工具对比总表

| 工具         | 输入要点                                                           | 输出要点   | Token 风险 | 典型失败  |
|------------|----------------------------------------------------------------|--------|----------|-------|
| Glob       | <code>pattern</code> , <code>cwd?</code>                       | 路径列表   | 中（结果多）   | 模式过宽  |
| Grep       | <code>pattern</code> , <code>path?</code> , <code>glob?</code> | 匹配行+文件 | 高        | 正则灾难  |
| LSP        | <code>uri</code> , <code>position</code> , <code>method</code> | 结构化符号  | 中        | 服务未就绪 |
| ToolSearch | <code>query</code>                                             | 工具名+摘要 | 低        | 词不达意  |

## Glob：模式匹配与性能

```
const GlobInput = z.object({
  pattern: z.string(),
  root: z.string().optional(),
  limit: z.number().int().positive().max(5000).optional(),
});
```

实践：

- 默认 `limit` 防止 `**/*` 爆炸。
- 排除 `node_modules` 、 `.git` 为内置约定（可配置）。

## Grep：内容与正则

```
const GrepInput = z.object({
  pattern: z.string(),
  path: z.string().optional(),
  glob: z.string().optional(),
  caseInsensitive: z.boolean().optional(),
  contextLines: z.number().int().min(0).max(5).optional(),
});
```

生活类比：在整库复印前先用便利贴标上下文（`contextLines`），否则每 hit 一页 A4，账单惊人。

## LSP：语言服务桥接

LSP 工具将 Language Server Protocol 能力暴露给模型：

| 能力  | 教学用 LSP 方法                           | 模型用途 |
|-----|--------------------------------------|------|
| 跳定义 | <code>textDocument/definition</code> | 找实现  |
| 引用  | <code>textDocument/references</code> | 影响分析 |
| 符号  | <code>workspace/symbol</code>        | 粗定位  |

**注意：**需要 IDE 或 headless language server **已启动**且项目可索引。

## ToolSearch：按需发现工具

**核心思想：**不在 system prompt 里一次性塞满 42 个工具说明，而是让模型先 **搜索工具目录**，再加载相关工具的 `prompt.ts` 片段（详见 6.11）。

| 维度   | 说明              |
|------|-----------------|
| 延迟加载 | 仅注入候选工具 schema  |
| 查询   | 自然语言 → 匹配工具名/标签 |
| 失败降级 | 返回「最接近的 3 个」    |

## Mermaid：搜索工具协作

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]



## 源码片段：Grep 结果截断

```
interface GrepHit {
  file: string;
  line: number;
  text: string;
}

function truncateGrepHits(hits: GrepHit[], maxHits: number, maxChars: number) {
  const out: GrepHit[] = [];
  let chars = 0;
  for (const h of hits) {
    if (out.length >= maxHits) break;
    if (chars + h.text.length > maxChars) break;
    out.push(h);
    chars += h.text.length;
  }
  return { hits: out, truncated: hits.length > out.length };
}
```

## 权限与治理

| 工具            | 常见默认              |
|---------------|-------------------|
| Glob/Grep/LSP | 只读，较宽松            |
| ToolSearch    | 只读；结果可能泄露工具名（低敏感） |

PreToolUse 可限制 Grep 的 `path` 必须在子目录 `packages/app` 内等。

## 与 Bash 的边界

| 需求                     | 优先                       |
|------------------------|--------------------------|
| 搜字符串                   | <code>Grep</code>        |
| <code>find + wc</code> | 仍可能 Bash，但 Grep/Glob 更可控 |
| <code>git grep</code>  | 若需版本感知，可在 Grep 内封装       |

## 遥测维度

| 指标             | 用途          |
|----------------|-------------|
| hitCount       | 调整默认 limit  |
| regexCompileMs | 恶意 ReDoS 监测 |
| lspMethod      | 语言服务器健康度    |

## 常见反模式

| 反模式            | 后果        |
|----------------|-----------|
| 无 limit 的 Glob | 数万路径打爆上下文 |
| 巨型正则           | CPU 拒绝服务  |
| LSP 未就绪仍重试     | 卡死会话      |

## 小结

- Glob/Grep 解决「在哪、有什么」；LSP 解决「语义上是什么关系」。
- ToolSearch 解决「有哪些工具可用」，与延迟加载强相关。
- 搜索层尽量 只读 + 截断，为后续 FileEdit 铺路。

## 自测题

1. 何时应用 LSP 而非 Grep？举一对互斥场景。
2. ToolSearch 的结果若包含 MCP 工具名，会引入何种提示注入风险？
3. 如何为 Grep 设计「渐进式」结果（先文件列表再展开）？

上一节：[6.5 文件工具](#) · 下一节：[6.7 AgentTool](#)

## 6.7 AgentTool — 子代理生成与隔离上下文

前置阅读: [6.3 治理流水线](#)

### 学习目标

完成本节学习后，你应该能够：

- 1. 解释 AgentTool 如何将复杂任务委派给子代理（subagent）执行。
- 2. 描述 子上下文隔离的动机：Token 预算、权限降级、提示注入面。
- 3. 对比 主循环与子循环在工具白名单、模型参数、历史压缩上的差异。
- 4. 列举 子代理完成的典型任务：大范围重构调研、并行探索、沙箱验证。
- 5. 说明 子代理返回结果如何结构化合并回主会话，避免「对话套娃」失控。

### 生活类比：项目经理与专项小组

主代理像项目经理：把握目标与边界，不亲自拧每一颗螺丝。AgentTool 像成立专项小组——小组有自己的会议室（隔离上下文）、自己的工具柜（子工具白名单）、结束后只交验收报告（结构化摘要），而不是把上千页会议记录塞回总经理邮箱。

### AgentTool 能力要素（表）

| 要素    | 说明                 |
|-------|--------------------|
| 任务描述  | 自然语言 + 可选结构化约束     |
| 子系统提示 | 角色、输出格式、禁止项        |
| 工具子集  | 如仅 Glob+Grep+Read  |
| 预算    | 最大步数、最大 Token、墙钟时间 |
| 返回契约  | JSON Schema / 固定章节 |

## 隔离上下文：三层含义

| 层       | 隔离内容            | 目的              |
|---------|-----------------|-----------------|
| 消息历史    | 子代理不见完整用户生平     | 减噪、保密           |
| 文件视图    | 可选 chroot / 子目录 | 缩小 blast radius |
| 密钥与 MCP | 子集或空            | 防数据渗出           |

## 源码片段：子代理调度（概念）

```
interface AgentToolInput {
  objective: string;
  constraints?: string[];
  allowedTools?: string[];
  maxSteps?: number;
}

interface AgentToolOutput {
  summary: string;
  artifacts: Array<{ type: "file"; path: string } | { type: "json"; data: unknown }>;
  traceId: string;
}

async function agentToolCall(input: AgentToolInput, parentCtx: ExecutionContext): Promise<AgentToolOutput> {
  const childCtx = parentCtx.fork({
    toolAllowlist: input.allowedTools ?? DEFAULT_SUBAGENT_TOOLS,
    history: seedMessages(input.objective),
    budget: { maxSteps: input.maxSteps ?? 40 },
  });

  const runner = new AgentLoop(childCtx);
  const final = await runner.runUntilStop();

  return {
    summary: compressForParent(final.messages),
    artifacts: collectArtifacts(final.messages),
    traceId: childCtx.traceId,
  };
}
```

要点： `fork` 显式传递策略，避免隐式继承全部父权限。

## Mermaid：主代理与子代理

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## 工具白名单建议（示例表）

| 子任务类型 | 推荐子集                      |
|-------|---------------------------|
| 代码调研  | Glob, Grep, FileRead, LSP |
| 文档撰写  | FileRead, WebFetch(可选)    |
| 运行测试  | Bash(沙箱), FileRead        |
| 大范围改  | 谨慎加入 FileEdit + 先读后写      |

## 与 Bash 的边界

| 用 Bash | 用 AgentTool |
|--------|-------------|
| 单条确定命令 | 多步探索 + 决策   |
| 脚本化流水线 | 需模型判断分支     |

反模式：为跑 `npm test` 专门 spawn 子代理 —— 过重；直接 Bash 更便宜。

## 结构化合并策略

| 策略   | 描述                                                          |
|------|-------------------------------------------------------------|
| 模板摘要 | 固定 <code>## 结论</code> <code>## 证据</code> <code>## 风险</code> |
| 引用路径 | 仅回传路径列表 + 关键片段                                              |
| 差异包  | 子代理生成 patch，父代理审核                                           |

## 安全：提示注入与子代理

子代理若可 `WebFetch`，可能把恶意页面指令当真理。应：

- 对子代理 降级 Web 权限；
- `PostToolUse` 扫描外源内容中的「忽略上文」模式；
- 记录 父→子 的不可篡改 `objective` 哈希。

## 遥测

| 字段                                                     | 用途      |
|--------------------------------------------------------|---------|
| <code>parentTraceId</code> / <code>childTraceId</code> | 关联追踪    |
| <code>subSteps</code>                                  | 成本归因    |
| <code>toolHistogram</code>                             | 子代理行为画像 |

## 常见反模式

| 反模式         | 后果    |
|-------------|-------|
| 子代理继承全部 MCP | 攻击面翻倍 |
| 无步数上限       | 费用失控  |
| 子结果全文灌父上下文  | 上下文爆炸 |

## 小结

- `AgentTool` = 受预算约束的二次循环 + 隔离上下文 + 结构化回传。
- 白名单与 `fork` 是安全关键，不是可选优化。
- 适用于高分支、多文件、需试探的任务，而非单步 `shell`。

## 自测题

1. 子代理的 `traceId` 为何应与主会话关联？

2. 若子代理需要写库，如何设计「父审核再落盘」？
3. 何时应优先并行多个子代理 vs 单个子代理多步？

上一节：[6.6 搜索工具](#) · 下一节：[6.8 外部工具](#)

## 预算调参指南（表）

| 参数                    | 保守默认         | 放宽条件      | 风险        |
|-----------------------|--------------|-----------|-----------|
| <code>maxSteps</code> | 20—40        | 仓库极大且只读   | 费用 ↑      |
| 墙钟超时                  | 2—5 min      | CI 内可延长   | 占用 worker |
| 允许工具数                 | 3—6          | 任务经人工批准   | 攻击面 ↑     |
| 返回摘要最大长度              | 2k—8k tokens | 需要附 patch | 父上下文膨胀    |

生活类比：给专项小组会议时长上限和会议室人数上限，不是不信任，而是控制加班费。

## 子代理输出契约模板（示例）

```
## 结论
(1-3 句)

## 证据
- 文件路径 + 关键片段引用
- 或工具调用 ID / 时间戳

## 未解决问题
- 需要父代理或用户决策的点

## 建议的下一步
- 具体可执行动作
```

宿主可将该模板写进 子系统提示，降低合并难度。

## 故障排查

| 现象     | 可能原因        | 排查                           |
|--------|-------------|------------------------------|
| 子代理空转  | 工具白名单过窄     | 临时开只读 Bash 日志                |
| 结果过长   | 无摘要模板       | 强制结构化节                       |
| 与父结论冲突 | 目标歧义        | 收紧 <code>objective</code> 文本 |
| 费用暴增   | maxSteps 过大 | 分段委派                         |

## 与 REPL 工具的差异（简表）

| 维度  | AgentTool | REPL       |
|-----|-----------|------------|
| 控制流 | 模型多步循环    | 通常单行求值或短会话 |
| 状态  | 子上下文隔离    | 会话内变量持续    |
| 适用  | 探索+决策     | 快速验算/画图    |

## 小结补充

- 子代理不是免费午餐：预算与模板是必选项。
- 白名单比「子代理里再开全集工具」更安全。
- 与 6.3 流水线 结合：子循环内部仍应跑完整治理（仅策略可裁剪）。

# 6.8 外部工具 — WebFetch 与 WebSearch

前置阅读：[6.4 BashTool](#)（出站对比） · [6.3 治理流水线](#)

## 学习目标

完成本节学习后，你应该能够：



1. 区分 `WebFetch`（拉取 URL 内容）与 `WebSearch`（检索索引）的输入输出与风险模型。
2. 解释 为何用专用 HTTP 工具替代 Bash 里的 `curl`：策略集中、SSRF 防护、响应大小上限。
3. 列举 Web 工具在治理流水线中的检查点：URL 白名单、重定向限制、内容类型过滤。
4. 说明 搜索结果 `snippet vs 全文` 对 Token 与版权的影响。
5. 关联 `ReadMcpResource`（在 6.9 详述）与 Web 工具的边界：HTTP 通用 vs MCP 资源模型。

## 生活类比：持证记者 vs 私人侦探

- `WebFetch` 像持证记者采访指定地址：你只能去报备过的门牌（URL 策略），进门最多录音 N 分钟（响应截断），危险品室（二进制下载）默认不进。
- `WebSearch` 像报社资料室：管理员给你剪报摘要（`snippet`），未必附原文全书；要快、要广，但细节需再 `Fetch`。

## WebFetch vs WebSearch（总表）

| 维度     | WebFetch                                                        | WebSearch                                                           |
|--------|-----------------------------------------------------------------|---------------------------------------------------------------------|
| 输入     | <code>url</code> , <code>headers?</code> , <code>method?</code> | <code>query</code> , <code>locale?</code> , <code>timeRange?</code> |
| 输出     | HTML/文本片段、状态码元数据                                                | 结果列表（标题、URL、摘要）                                                     |
| 主要风险   | SSRF、恶意内容、超大响应                                                  | 查询注入、结果投毒（SEO）                                                      |
| 与 Bash | 替代 <code>curl</code> 主路径                                        | 替代「手搓搜索引擎 API」                                                      |

## WebFetch：实现要点

| 控制点                         | 说明                                                 |
|-----------------------------|----------------------------------------------------|
| DNS 重绑定                     | 解析后再次校验 IP 段                                       |
| 内网 IP                       | 默认拒绝 <code>10/8</code> 、 <code>192.168/16</code> 等 |
| 重定向                         | 限制次数，每跳重验                                          |
| <code>Content-Length</code> | 预检上限                                               |
| 超时                          | 连接 + 首字节 + 总时长                                     |

---

## 源码片段：URL 策略（概念）

---

```
const FetchInput = z.object({
  url: z.string().url(),
  maxBytes: z.number().int().positive().max(2_000_000).optional(),
});

function isUrlAllowed(url: URL, policy: FetchPolicy): boolean {
  if (policy.blockPrivateIp && isPrivateIp(url.hostname)) return false;
  if (policy.allowlistHosts && !policy.allowlistHosts.has(url.hostname)) return false;
  if (url.protocol !== "https:" && policy.httpsOnly) return false;
  return true;
}
```

---

## WebSearch：结果形态

---

```
const SearchHit = z.object({
  title: z.string(),
  url: z.string().url(),
  snippet: z.string(),
  publishedAt: z.string().optional(),
});

const WebSearchOutput = z.object({
  hits: z.array(SearchHit).max(15),
  provider: z.string(),
});
```

生活类比：菜单（hits）与后厨实物（fetch 全文）分开，避免一上来端整头烤全羊。

---

---

## Mermaid：外部工具在流水线中的位置

---

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

---

## 内容与提示注入

| 风险             | 缓解          |
|----------------|-------------|
| 页面含「忽略上文」      | 内容消毒、引用分隔符  |
| 搜索引擎摘要被 SEO 污染 | 多源交叉、低权重采纳  |
| PDF/HTML 隐写    | 类型白名单、纯文本管线 |

## 与 MCP 的分工

| 场景                | 工具              |
|-------------------|-----------------|
| 任意公开文档            | WebFetch        |
| 需登录或 OAuth 的 SaaS | MCP 工具          |
| 稳定资源 URI          | ReadMcpResource |

## 遥测

| 事件                        | 字段                                                           |
|---------------------------|--------------------------------------------------------------|
| <code>web_fetch</code>    | <code>host</code> , <code>status</code> , <code>bytes</code> |
| <code>web_search</code>   | <code>provider</code> , <code>hitCount</code>                |
| <code>ssrf_blocked</code> | <code>reason</code>                                          |

## 常见反模式

| 反模式              | 后果       |
|------------------|----------|
| 允许 file:// Fetch | 本地文件读出   |
| 无截断              | 上下文与内存爆炸 |
| 搜索结果直接当真理        | 错误事实传播   |

## 小结

- WebFetch 解决受控拉取；WebSearch 解决发现与导航。
- 替代 curl 的价值在策略与遥测集中。
- SSRF + 大小 + 注入 是三连击，应在前置阶段处理。

## 自测题

1. 重定向到 `file://` 时应如何处理？
2. 为何搜索摘要仍可能触发提示注入？
3. WebFetch 返回 Markdown 与 HTML 各有什么利弊？

上一节：[6.7 AgentTool](#) · 下一节：[6.9 MCP 工具](#)

## WebSearch 提供商维度（概念表）

| 维度         | 说明                |
|------------|-------------------|
| 地域合规       | 结果是否受区域法规影响       |
| 索引新鲜度      | 新闻/文档滞后           |
| snippet 长度 | 影响后续是否必须 Fetch    |
| 配额与计费      | QPS、按调用计费         |
| 可观测性       | 是否返回可追溯 requestId |

生活类比：选搜索 API 像选通讯社——不只看价格，还要看发稿速度与版权边界。

## 重试与退避

| 场景        | 建议            |
|-----------|---------------|
| 429 / 5xx | 指数退避 + 最大重试次数 |
| DNS 失败    | 快速失败，提示检查网络策略 |
| TLS 错误    | 记录证书指纹，勿静默忽略  |

```
async function fetchWithRetry(url: string, opts: { maxRetries: number }) {
  let delay = 300;
  for (let i = 0; i < opts.maxRetries; i++) {
    try {
      return await controlledFetch(url);
    } catch (e) {
      if (i === opts.maxRetries - 1) throw e;
      await sleep(delay);
      delay *= 2;
    }
  }
  throw new Error("unreachable");
}
```

## 缓存策略

| 类型        | 适用    | 注意        |
|-----------|-------|-----------|
| URL 正文短缓存 | 静态文档  | 与鉴权响应勿混缓存 |
| 搜索结果不缓存   | 强时效查询 | 避免陈旧事实    |
| ETag      | 大页面增量 | 需服务端支持    |

## 与 REPL / Bash 的边界

| 需求            | 推荐                |
|---------------|-------------------|
| 拉取公开 API JSON | WebFetch（策略化）     |
| 在本地算矩阵        | REPL / 语言运行时      |
| 克隆 Git 仓库     | 专用 VCS 工具或受控 Bash |

避免「WebFetch 拉脚本 | bash」这种二段式绕过 —— 应在策略层识别组合风险。

## 合规与版权提示（教学）

| 实践         | 说明                   |
|------------|----------------------|
| snippet 展示 | 控制长度，注明来源 URL        |
| 全文抓取       | 遵守 robots 与服务条款（视场景） |
| 内网文档       | 走 VPN + 内网 URL 白名单   |

## FAQ

- 问：为何不直接把 HTML 全量塞进上下文？

答：Token、版权、XSS 消毒三重压力；应用 `readability` 类提取与上限。
- 问：WebSearch 结果能当作引用来源吗？

答：只能当线索；关键事实应 Fetch 原文 或 交叉验证。
- 问：与 ReadMcpResource 如何二选一？

答：公网匿名可读 → WebFetch；需登录/组织内资源 URI → MCP（见 6.9）。

## 小结补充

- WebFetch + WebSearch 常成对出现：先搜后取。
- 重试、缓存、合规 属于生产必选项，而非「以后再说」。
- 对外部工具保持 fail-closed：默认假设内容不可信，需消毒与截断。

# 6.9 MCP 工具集成 — 动态发现、指令注入与 ReadMcpResource

前置阅读：[6.8 外部工具](#) · [6.2 Tool 接口](#)

## 学习目标

完成本节学习后，你应该能够：

- 1. **解释** MCP (Model Context Protocol) 工具如何**动态发现**并映射到宿主工具注册表。
- 2. **描述** 命名空间化工具名 (如 `mcp__server__toolName`) 在权限与遥测上的好处。
- 3. **识别** **指令注入** 风险：恶意 MCP 服务器返回的描述与 schema 如何影响模型行为。
- 4. **区分** MCP **工具调用**与 `ReadMcpResource` 读取资源的语义差异。
- 5. **列举** 将 MCP 接入治理流水线的要点：PreToolUse、速率限制、凭证隔离。

## 生活类比：外包服务商进场

把 MCP 当成持证外包厂商：主机厂（宿主应用）提供**统一门禁与工单系统**（协议），外包带自己的**扳手套装**（工具列表）与**零件目录**（资源）。**动态发现**等于每天开工前在门禁屏上刷新「今日可用服务」——**不刷新就不知道新扳手**。**指令注入**像外包在工单上写了一行小字「忽略安全科规定」——若总装线照单全收就会出事。

## MCP 在工具全景中的位置（表）

| 能力       | MCP 载体                         | 与 Web 对比             |
|----------|--------------------------------|----------------------|
| 调用后端 API | MCP Tool                       | 常需用户 OAuth / API Key |
| 读文档/模板   | MCP Resource + ReadMcpResource | 更结构化 URI             |
| 提示片段     | MCP Prompt                     | 类似 system 插件         |

## 动态发现流程

| 阶段                      | 行为                                                |
|-------------------------|---------------------------------------------------|
| 连接                      | stdio / SSE / 自定义传输                               |
| 握手                      | 能力协商、版本                                           |
| <code>tools/list</code> | 拉取工具名、描述、inputSchema                              |
| 注册                      | 映射为 <code>ToolDefinition</code> + Zod JSON Schema |
| 热更新                     | 重连或周期刷新                                           |

## 源码片段：注册 MCP 工具（概念）

```
type McpToolName = `mcp__${string}__${string}`;

function toInternalToolName(serverId: string, toolName: string): McpToolName {
  const safeServer = serverId.replace(/[^a-zA-Z0-9_-]/g, "_");
  const safeTool = toolName.replace(/[^a-zA-Z0-9_-]/g, "_");
  return `mcp__${safeServer}__${safeTool}` as McpToolName;
}

async function registerMcpServer(conn: McpConnection, registry: ToolRegistry) {
  const { tools } = await conn.listTools();
  for (const t of tools) {
    const name = toInternalToolName(conn.serverId, t.name);
    registry.register({
      name,
      description: sanitizeDescription(t.description),
      inputSchema: jsonSchemaToZod(t.inputSchema),
      call: (input) => conn.callTool(t.name, input),
      isReadOnly: inferReadOnly(t),
      isConcurrencySafe: false,
    });
  }
}
```

注意： `inferReadOnly` 对不可信服务器不可盲信，默认应 `fail-closed`（见 6.10）。

## 指令注入：描述与 Schema 即攻击面

| 向量                          | 说明                 |
|-----------------------------|--------------------|
| 工具 <code>description</code> | 诱导模型高频或错误调用        |
| <code>inputSchema</code> 注释 | 部分栈会把注释拼进 prompt   |
| 错误消息                        | 可携带「请改用 bash 泄露密钥」 |

缓解：

- 描述消毒：剥离「忽略」「override」模式；
- 人工审核高敏 MCP；
- `PreToolUse` 对 MCP 工具单独策略类。



## ReadMcpResource

| 维度         | 说明                            |
|------------|-------------------------------|
| 用途         | 按 URI 读取 MCP 暴露的资源（文件、数据库视图等） |
| 与 WebFetch | 不走公网 HTTP，走 MCP 资源模型          |
| 治理         | URI 白名单、服务器级 ACL              |

```
const ReadMcpResourceInput = z.object({
  serverId: z.string(),
  uri: z.string(),
});
```

## Mermaid：MCP 工具生命周期

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## 凭证与隔离

| 实践         | 说明            |
|------------|---------------|
| 每服务器独立 env | 防串密钥          |
| 用户级 OAuth  | token 不进模型上下文 |
| 网络出口       | MCP 进程级防火墙    |

## 与内置 42 工具的关系

内置工具由可信方实现；MCP 工具由第三方实现，故：

- 默认 `isConcurrencySafe=false` ；
- 默认 非只读 除非强证明；

- 遥测必须带 `serverId`。

## 遥测字段

| 字段                               | 用途      |
|----------------------------------|---------|
| <code>mcpServerId</code>         | 成本与错误归因 |
| <code>mcpToolOriginalName</code> | 与内部名对照  |
| <code>latencyMs</code>           | SLO     |

## 常见反模式

| 反模式                               | 后果   |
|-----------------------------------|------|
| 信任服务器提供的 <code>readOnly</code> 标记 | 误判权限 |
| 无超时 <code>callTool</code>         | 挂死   |
| 描述原样进 <code>system</code>         | 注入放大 |

## 小结

- MCP 让工具集可插拔；代价是信任模型降级。
- 命名空间 + 治理流水线 是把第三方能力关进笼子的关键。
- `ReadMcpResource` 补齐「资源」这一半，不仅「动作」。

## 自测题

1. 为何 MCP 工具名要加 `serverId` 前缀而非仅用 `toolName`？
2. 若 MCP 返回 Markdown 中含隐藏指令，宿主应如何处理？
3. `tools/list` 热更新时如何避免竞态注册？

上一节：[6.8](#) · 下一节：[6.10 Fail-closed](#)

## MCP 能力三角（表）

| MCP 原语   | 映射到教学工具                        | 模型感知方式           |
|----------|--------------------------------|------------------|
| Tool     | <code>mcp__server__name</code> | function calling |
| Resource | <code>ReadMcpResource</code>   | URI + mime       |
| Prompt   | 模板注入（若宿主支持）                    | 系统/用户消息片段        |

## 传输层与安全

| 传输         | 优点     | 注意       |
|------------|--------|----------|
| stdio      | 简单、易本地 | 进程生命周期绑定 |
| SSE / HTTP | 远程可部署  | 需 TLS、鉴权 |
| 自定义        | 企业内网   | 审计与零信任   |

生活类比：选传输像选通勤方式——本地步行（stdio）省事，远程出差（HTTP）要带证件与加密行李箱。

`callTool`

### 负载形态

| 字段                       | 用途                 |
|--------------------------|--------------------|
| <code>content</code> 块数组 | 文本、图像、资源引用         |
| <code>isError</code>     | 与 HTTP 200 解耦的业务失败 |

宿主应把 `isError` 映射为统一 `ToolResult`，避免模型把错误当正文。

## 策略包示例（伪 YAML）

```
mcp_servers:
  - id: crm_prod
    trust: low
    tool_overrides:
      "*":
        isReadOnly: false
        isConcurrencySafe: false
        require_approval: true
  - id: docs_internal
    trust: medium
    resource_allowlist_uris:
      - "internal://docs/**"
```

## 与 6.3 流水线的检查点对照

| 步             | MCP 特化                       |
|---------------|------------------------------|
| validateInput | URI 与 serverId 绑定校验          |
| PreToolUse    | 描述消毒、参数脱敏                    |
| PostToolUse   | 将 content 块规范化               |
| 遥测            | 必须含 <code>mcpServerId</code> |

## FAQ

- 问：能否禁止某个 MCP 服务器的全部写操作？

答：可以，在注册层统一 `require_approval` 或覆盖 `isReadOnly` 为 false 且拦截写类工具名模式。
- 问：MCP 工具与内置工具同名怎么办？

答：内部名 必须命名空间化（`mcp_...`），永不与内置短名冲突。
- 问：动态 schema 更新如何通知模型？

答：会话级 system 补丁 或 要求重新 ToolSearch；避免静默改契约。

## 小结补充

- MCP 把生态做大，也把攻击面做大；策略包是企业落地关键。
- `isError` 与 HTTP 状态 可能不一致，宿主需统一语义。
- 传输、鉴权、审计 与工具本身同等重要。

## 6.10 Fail-closed 设计 — 默认「不安全」与工厂函数

前置阅读： [6.2 Tool 接口](#) · [6.9 MCP](#)

### 学习目标

完成本节学习后，你应该能够：

1. 解释 `isConcurrencySafe=false` 与 `isReadOnly=false` 作为默认值的 fail-closed 含义。
2. 说明 为何「未显式声明安全」的工具应视为需串行或需锁。
3. 设计 工厂函数 `defineTool()`，强制调用方传入安全能力位，避免遗漏。
4. 对照 只读搜索工具与 Bash 工具在安全位上的典型覆盖。
5. 关联 权限 UI：默认拦截、显式放行与审计日志。

### 生活类比：化工管道默认阀门

Fail-closed 像常闭电磁阀：断电（信息缺失）= 停止流动。若你不知道一条管道是单行道还是双向，默认按会爆炸处理——关阀比赌运气便宜。

`isConcurrencySafe` 默认 `false`：不知道能否并行，就不要并行。`isReadOnly` 默认 `false`：不知道是否改状态，就按会改状态要求授权。

## 两个布尔位的语义（表）

| 字段                | true       | false（默认 fail-closed） |
|-------------------|------------|-----------------------|
| isReadOnly        | 不写盘、不改外部系统 | 可能写盘或副作用 → 需严格策略      |
| isConcurrencySafe | 可并行调用无竞态   | 可能有竞态 → 串行或加锁         |

## 为何默认「不安全」

| 场景     | 若错误标 readOnly: true | 若错误标 concurrencySafe: true |
|--------|---------------------|----------------------------|
| 文件写    | 绕过写权限               | 交错写入损坏                     |
| MCP 工具 | 静默改远端数据             | 重复扣款                       |
| Bash   | 极危                  | 管道竞态                       |

原则：只有可证明的性质才能标 true；缺失证明 → 保守默认。

## 源码片段：危险默认值（反例）

```
// 反模式：默认值过于乐观
interface ToolDefBad {
  isReadOnly?: boolean; // 缺省 undefined → 被当成 falsy 仍可能逻辑错误
  isConcurrencySafe?: boolean;
}

function runBad(def: ToolDefBad) {
  const readOnly = def.isReadOnly ?? true; // 若有人误用「缺省即只读」会灾难
  // ...
}
```

## 源码片段：工厂函数强制显式（推荐）

```
type SafetyFlags =
  | { isReadOnly: true; isConcurrencySafe: true }
  | { isReadOnly: true; isConcurrencySafe: false }
  | { isReadOnly: false; isConcurrencySafe: false };

interface ToolConfig<I, O> extends SafetyFlags {
  name: string;
  inputSchema: ZodSchema<I>;
  outputSchema: ZodSchema<O>;
  call(input: I): Promise<O>;
}

function defineTool<I, O>(config: ToolConfig<I, O>) {
  // 编译器：必须传入 SafetyFlags 联合的某一枝，不能省略
  return {
    ...config,
    // 运行期再次断言，防止 as any 逃逸
    meta: {
      isReadOnly: config.isReadOnly,
      isConcurrencySafe: config.isConcurrencySafe,
    },
  };
}

// 示例：只读且可并发
const globTool = defineTool({
  name: "Glob",
  isReadOnly: true,
  isConcurrencySafe: true,
  inputSchema: GlobInput,
  outputSchema: GlobOutput,
  call: async () => ({ paths: [] }),
});

// 示例：写操作必须 isReadOnly: false, 通常 isConcurrencySafe: false
const fileEditTool = defineTool({
  name: "FileEdit",
  isReadOnly: false,
  isConcurrencySafe: false,
  inputSchema: FileEditInput,
  outputSchema: FileEditOutput,
  call: async () => ({ applied: true }),
});
```

## Mermaid：调度器如何消费安全位

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## 与治理流水线的衔接

| 步骤   | fail-closed 影响           |
|------|--------------------------|
| 权限决策 | 非只读 → 默认弹窗               |
| 并行优化 | 非 concurrencySafe → 禁止合并 |
| 缓存   | 非只读 → 禁止缓存响应             |

## MCP 与第三方扩展

对 MCP 工具：

- 禁止信任对端声明的 `readOnly` 除非有本地证明（静态列表或人工勾选）；
- 默认 `isConcurrencySafe: false`；
- 可在 `registerMcpServer` 内覆盖为更保守。

## 测试矩阵（概念表）

| 工具       | <code>isReadOnly</code> | <code>isConcurrencySafe</code> |
|----------|-------------------------|--------------------------------|
| Glob     | true                    | true                           |
| Grep     | true                    | true（视实现，可能 false）             |
| FileRead | true                    | true                           |
| FileEdit | false                   | false                          |
| Bash     | false                   | false                          |
| MCP 未知   | false                   | false                          |



## 常见反模式

| 反模式             | 后果     |
|-----------------|--------|
| 「大部分是只读」默认 true | 写穿权限   |
| 为性能盲目并行         | 数据竞争   |
| 用注释代替布尔位        | 调度器读不到 |

## 审计与 UI

| 事件                             | 记录        |
|--------------------------------|-----------|
| <code>tool_safety_flags</code> | 注册时快照     |
| <code>override</code>          | 管理员改标记需留痕 |

## 小结

- Fail-closed = 缺省最保守 + 证明才可放宽。
- 工厂函数 + 联合类型 能在编译期减少遗漏。
- 调度与权限 应消费同一套元数据，避免分叉逻辑。

## 自测题

1. 若 `isReadOnly=true` 但实现里写了日志文件，属于什么失败模式？
2. 何时 `isReadOnly=true` 仍不应 `isConcurrencySafe=true` ？
3. 如何把 fail-closed 策略做成可配置的企业策略包？

上一节：[6.9 MCP](#) · 下一节：[6.11 延迟加载](#)

# 6.11 延迟加载 — ToolSearchTool、prompt.ts 与 Token 节省

前置阅读：[6.1 全景](#) · [6.6 搜索工具](#)

## 学习目标

完成本节学习后，你应该能够：

- 1. **解释** 为何一次性向模型暴露 42 个工具的**完整说明**会浪费 Token。
- 2. **描述** `ToolSearchTool` **按需注入**的工作流：查询 → 命中工具 → 挂载 schema 与描述片段。
- 3. **说明** 每个工具目录下的 `prompt.ts`（或等价文件）写给谁看、包含哪些内容。
- 4. **对比** 「全量工具列表」与「分层工具列表」在**首包延迟**与**误选率**上的权衡。
- 5. **列举** 实现延迟加载时的坑：schema 版本、缓存失效、命中排序。

## 生活类比：餐厅菜单与厨房小抄

全量工具像一本 200 页菜单：顾客（模型）从头读到尾才点菜，点餐时间（首 Token）很长。延迟加载像先给分类目录，顾客说「我想吃鱼」，服务员只递上海鲜页再加今日特供小抄（`prompt.ts`）。ToolSearch 就是\*\*「请问有哪几种做法」\*\*的问询台。

## Token 经济学（表）

| 策略                | system+tools 体积 | 误选风险 | 实现复杂度 |
|-------------------|-----------------|------|-------|
| 全量暴露              | 高               | 低    | 低     |
| 静态分组              | 中               | 中    | 中     |
| ToolSearch + 动态挂载 | 低               | 中-高  | 高     |

## ToolSearchTool 流程

| 步 | 行为                                                       |
|---|----------------------------------------------------------|
| 1 | 模型调用 <code>ToolSearch</code> , <code>query</code> 为任务关键词 |
| 2 | 宿主在工具元数据索引中检索 (BM25/向量/标签)                               |
| 3 | 返回 Top-K 工具名 + 一行摘要                                      |
| 4 | 模型选定后, 宿主注入完整 <code>inputSchema</code> 与详细描述             |
| 5 | 下一轮可 <code>call</code> 该工具                               |

## prompt.ts: 写给 AI 看的说明

| 区块      | 内容示例                    |
|---------|-------------------------|
| 何时使用    | 「多文件重命名用我, 不用 Bash sed」 |
| 输入诀窍    | 「path 必须相对仓库根」          |
| 反模式     | 「不要对大二进制用 FileRead」     |
| 与其他工具协作 | 「先 Grep 再 FileEdit」     |

生活类比: `prompt.ts` 是岗位培训卡片, 不是给人看的 README 全文。

## 源码片段：工具元数据索引（概念）

```
interface ToolPromptModule {
  whenToUse: string;
  tips?: string[];
  antiPatterns?: string[];
}

interface ToolCatalogEntry {
  name: string;
  summary: string;
  tags: string[];
  loadPrompt: () => Promise<ToolPromptModule>;
  loadSchema: () => Promise<{ input: ZodSchema<unknown> }>;
}

const catalog: ToolCatalogEntry[] = [
  {
    name: "FileEdit",
    summary: "对文本文件应用结构化编辑，需先读",
    tags: ["write", "patch", "file"],
    loadPrompt: () => import("./tools/FileEdit/prompt.js").then((m) => m.default),
    loadSchema: () => import("./tools/FileEdit/schema.js").then((m) => m.FileEditInput),
  },
];

async function toolSearch(query: string, k: number) {
  const ranked = rankByQuery(catalog, query); // 简化：关键词打分
  return ranked.slice(0, k).map((e) => ({ name: e.name, summary: e.summary }));
}

async function activateTool(name: string, session: Session) {
  const entry = catalog.find((e) => e.name === name);
  if (!entry) throw new Error("unknown tool");
  const [prompt, schema] = await Promise.all([entry.loadPrompt(), entry.loadSchema()]);
  session.attachTool({ name, prompt, inputSchema: schema.input });
}
```

## Mermaid：延迟加载时序

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## 与 Tool 接口的衔接

延迟加载不改变 Tool 接口，只改变何时把 `description` + `inputSchema` 放进可见上下文：

| 阶段  | 模型可见                               |
|-----|------------------------------------|
| 初启  | 核心工具（Read/Bash/Search）+ ToolSearch |
| 激活后 | 额外工具的完整契约                          |

## 缓存与失效

| 策略    | 说明             |
|-------|----------------|
| 按版本哈希 | schema 变更 bump |
| 会话级缓存 | 避免重复 import    |
| 冷启动预热 | 对高频工具同步加载      |

## 风险：检索失误

| 问题     | 缓解            |
|--------|---------------|
| 搜不到    | 返回「相近」+ 建议关键词 |
| 搜错工具   | 多轮澄清；保留安全默认值  |
| 注入恶意条目 | 目录签名、可信源      |

## 遥测

| 指标                               | 用途        |
|----------------------------------|-----------|
| <code>toolsearch_query</code>    | 优化索引与别名   |
| <code>activate_latency_ms</code> | import 性能 |
| <code>mismatch_rate</code>       | 选错工具比例    |

## 常见反模式

| 反模式             | 后果           |
|-----------------|--------------|
| 动态加载无版本         | 新旧 schema 混用 |
| prompt.ts 过长    | 激活后仍爆 Token  |
| 无 ToolSearch 回退 | 新用户不会用库      |

## 小结

- 延迟加载把「工具说明书」从首包挪到真正需要时。
- `ToolSearchTool` 是元工具，`**prompt.ts**` 是面向模型的操作手册。
- 权衡：Token↓ 可能带来 误选↑，需检索与默认核心工具兜底。

## 自测题

1. 哪些工具应始终常驻而不走延迟加载？
2. `prompt.ts` 与 `description` 字段如何避免重复冗长？
3. ESM 动态 `import()` 在 Serverless 冷启动下有何影响？

上一节：[6.10 Fail-closed](#) · 下一节：[6.12 实践](#)

## 常驻工具集建议（表）

| 常驻          | 理由         |
|-------------|------------|
| FileRead    | 几乎所有开发任务需读 |
| Glob 或 Grep | 导航仓库       |
| ToolSearch  | 延迟加载入口     |
| Bash（受限）或替代 | 用户期望跑测试    |

| 可延迟          | 理由      |
|--------------|---------|
| NotebookEdit | 仅笔记本任务  |
| WebFetch     | 仅联网任务   |
| 低频 MCP 工具    | 减攻击面与体积 |

## 与「工具裁剪」关系

**裁剪**：从全集删永不用工具。**延迟加载**：全集仍在，但描述与 schema 按需出现。二者可组合：先裁剪高危/无用，再对长尾延迟。

## FAQ

- 问：延迟加载会增加一轮对话吗？

答：通常会（先 ToolSearch 再调用）；换得 首包更小 与 更聚焦上下文。
- 问：如何避免循环依赖（搜不到就永远不用）？

答：核心工具常驻 + 文档化典型工作流（如「改配置先 Grep」）。
- 问：prompt.ts 能否用多语言？

答：可以，但应与用户界面语言策略一致，避免模型混读。

## 小结补充

- ToolSearch + prompt.ts + 动态 import 是三位一体的 Token 策略。
- 常驻最小集决定「是否多一轮」与「误选率」的平衡。
- 监控 `toolsearch_query` → 激活工具 漏斗，持续优化索引与别名。

# 6.12 实践 — 构建自定义工具（步骤指南）

前置阅读：[6.2 Tool 接口](#) · [6.3 治理流水线](#) · [6.10 Fail-closed](#)

## 学习目标

完成本节学习后，你应该能够：

- 1. 按步骤 从零实现一个宿主可加载的自定义工具：目录结构、Schema、`call`、注册。
- 2. 编写 配套的 `prompt.ts`，让模型稳定选对工具并填对参数。
- 3. 接入 PreToolUse / 权限策略，演示拦截与参数改写。
- 4. 选择 正确的 `isReadOnly` / `isConcurrencySafe` 默认与覆盖条件。
- 5. 验证 输入输出 Zod 校验与遥测字段是否齐全。

## 生活类比：定制专用印章

自定义工具像刻一枚法务专用章：要有印模尺寸（Schema）、用印流程（治理流水线）、谁能盖（权限）、盖完留底（遥测）。刻完还要给文员一张\*\*「何时盖此章」\*\*的便签（`prompt.ts`），否则章会被盖在无关合同上。

## 步骤总览（表）

| 步号 | 步骤                            | 产出                                              |
|----|-------------------------------|-------------------------------------------------|
| 1  | 定义能力与边界                       | 一句话 spec                                        |
| 2  | 设计入参出参 Zod                    | <code>Input</code> / <code>Output</code> schema |
| 3  | 实现 <code>call</code> 纯逻辑      | 无模型依赖                                           |
| 4  | <code>defineTool</code> 包装安全位 | fail-closed 元数据                                 |
| 5  | 编写 <code>prompt.ts</code>     | whenToUse + tips                                |
| 6  | 注册到 Registry                  | 名称唯一                                            |
| 7  | 接 Hook / 权限                   | 风险分级                                            |
| 8  | 单元测试 + 契约测试                   | <code>safeParse</code> 用例                       |
| 9  | 遥测与文档字符串                      | 运维可观测                                           |
| 10 | 灰度发布                          | 特性开关                                            |



## 步骤 1: 规格示例

---

工具名: `InvoiceTotal`

做什么: 给定本地 JSON 路径, 汇总 `items[].amount` 返回总和与币种。

不做什么: 不访问网络; 不修改文件。

→ 可得: `isReadOnly: true`, `isConcurrencySafe: true` (若文件只读快照)。

---

## 步骤 2—3: Schema 与实现

---

```
import { z } from "zod";

const InvoiceTotalInput = z.object({
  path: z.string().min(1),
  currency: z.string().length(3).optional(),
});

const InvoiceTotalOutput = z.object({
  total: z.number(),
  currency: z.string(),
  itemCount: z.number().int().nonnegative(),
});

type In = z.infer<typeof InvoiceTotalInput>;
type Out = z.infer<typeof InvoiceTotalOutput>;

async function invoiceTotalCall(input: In): Promise<Out> {
  const raw = await fs.readFile(input.path, "utf8");
  const data = JSON.parse(raw) as { items?: Array<{ amount: number; currency?: string }> };
  const items = data.items ?? [];
  const currency = input.currency ?? items[0]?.currency ?? "USD";
  const total = items.reduce((s, it) => s + it.amount, 0);
  return { total, currency, itemCount: items.length };
}
```

## 步骤 4: defineTool 包装

```
const invoiceTotalTool = defineTool({
  name: "InvoiceTotal",
  isReadOnly: true,
  isConcurrencySafe: true,
  inputSchema: InvoiceTotalInput,
  outputSchema: InvoiceTotalOutput,
  call: invoiceTotalCall,
  validateInput: async (input, ctx) => {
    const resolved = path.resolve(ctx.repoRoot, input.path);
    if (!resolved.startsWith(ctx.repoRoot)) {
      return { ok: false, reason: "路径必须在仓库内" };
    }
    return { ok: true };
  },
});
```

## 步骤 5: prompt.ts

```
// tools/InvoiceTotal/prompt.ts
export default {
  whenToUse:
    "当用户给出仓库内发票 JSON 路径并需要汇总金额时使用；不要用于任意二进制文件。",
  tips: [
    "path 使用相对仓库根路径",
    "若多币种需先在业务层规范，本工具仅做简单默认",
  ],
  antiPatterns: ["不要用本工具读取 .env 或密钥文件"],
};
```

## 步骤 6: 注册

```
registry.register(invoiceTotalTool);
```

## 步骤 7: Hook 示例

```
hooks.preToolUse = async ({ tool, input }) => {
  if (tool === "InvoiceTotal") {
    const p = (input as any).path as string;
    if (p.endsWith(".pem")) return { action: "deny", reason: "敏感扩展名" };
  }
  return { action: "allow" };
};
```

## Mermaid：自定义工具生命周期

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## 步骤 8：测试清单（表）

| 用例      | 期望               |
|---------|------------------|
| 缺字段     | Zod 失败           |
| 路径穿越    | validateInput 失败 |
| 超大 JSON | 可另加限额            |
| 并发两次    | 结果稳定             |

```
test("InvoiceTotal rejects traversal", async () => {
  const r = await runValidate(invoiceTotalTool, { path: "../../etc/passwd" });
  expect(r.ok).toBe(false);
});
```

## 步骤 9：遥测

```
emitTelemetry({
  tool: "InvoiceTotal",
  itemCount: output.itemCount,
  durationMs: dt,
});
```

## 步骤 10：灰度

| 手段   | 说明                  |
|------|---------------------|
| 特性开关 | 仅内测 workspace       |
| 配额   | 每会话 N 次             |
| 影子模式 | 执行但结果不返回模型，只记录 diff |

## 进阶：复用治理流水线

自定义工具禁止绕过统一入口；即使内部调用也应走：

```
parse → schema → validateInput → hooks → authorize → call → telemetry → post。
```

## 与 MCP 的对比

| 维度    | 宿主内自定义 | MCP 插件 |
|-------|--------|--------|
| 信任    | 高      | 低      |
| 部署    | 随版本发布  | 独立进程   |
| 默认安全位 | 仍建议保守  | 必须保守   |

## 常见反模式

| 反模式                             | 后果   |
|---------------------------------|------|
| call 内直接 <code>fetch</code> 无策略 | SSRF |
| 无 output 校验                     | 静默错误 |
| prompt 与实现不一致                   | 模型误用 |

## 小结

- 自定义工具 = 契约（Zod） + 实现（call） + 安全位 + 模型说明（prompt.ts） + 治理注册。
- 测试应覆盖 Schema、路径规则、Hook。
- 发布用特性开关与遥测闭环。

## 自测题

- 若工具只读但依赖进程全局缓存，并发安全位应如何标？
- `validateInput` 与 Zod 的边界应如何划分？
- 如何把自定义工具纳入 ToolSearch 索引？

上一节：[6.11 延迟加载](#) · 返回：[6.1 全景](#)

## 第7篇：权限与安全

### 7.1 为什么需要权限：从 `rm -rf` 噩梦说起

**本篇定位：** Claude Code 完全指南 V2 · 第 7 篇「权限与安全」导论。后续章节将展开六种模式、七步管道与沙箱细节。

#### 学习目标

完成本节学习后，你应该能够：

1. **解释** 为什么「能跑命令的 AI」必须自带权限闸门，而不是完全信任模型输出。
2. **识别** 典型高危操作（删除、覆盖、外泄、供应链）与「误操作」之间的区别。
3. **理解** 权限系统解决的是「谁、在何种模式下、对哪些资源、执行什么动作」的可审计决策问题。
4. **建立** 「默认保守、显式放行」的心理模型，为 7.2—7.10 的模式与管道章节打底。
5. **列举** 至少五种与 Shell 相关的常见事故模式，并能对应到后续章节中的缓解手段（沙箱、deny、ask、写入边界）。
6. **区分** 「用户点了同意」与「策略上应该允许」——前者是交互事实，后者是安全策略，二者不可混为一谈。

#### 生活类比：智能管家与钥匙串

想象你雇了一位**反应极快、知识面极广**的管家，但他：

- 会**字面执行**你随口说的「把没用的都删掉」；
- 分不清「清理下载文件夹」和「清空整个家」；
- 偶尔被钓鱼话术诱导，去「帮主人装个工具」，实则执行了陌生脚本。

你不会把**保险柜钥匙、房产证、银行卡密码**一次性全交出去，而是：

- **默认**：先看一眼再签字（对应 Default 等模式的「确认」）；
- **分区**：卧室能进，邻居车库不能进（对应写入仅限项目目录等规则）；

- **黑名单**：明确禁止「代签大额合同」（对应 deny / 命令黑名单）。

Claude Code 的权限体系，本质上就是这把**可配置的钥匙串**。管家越聪明，钥匙串越要**细粒度、可撤销、可审计**。

核心信息：**rm -rf** 为何是教科书级噩梦

| 风险维度    | 典型场景                                                                           | 没有闸门时会发生什么               |
|---------|--------------------------------------------------------------------------------|--------------------------|
| 不可逆删除   | <code>rm -rf /</code> 、 <code>rm -rf ~</code> 、 <code>rm -rf .</code> （路径解析错误） | 数据永久丢失，备份链断裂             |
| 范围误判    | 通配符、变量展开、 <code>find ... -delete</code>                                        | 「以为删的是 build，实际删的是 repo」 |
| 供应链     | <code>curl   bash</code> 、下载后执行                                                | 远程脚本可篡改本地环境与密钥           |
| 横向移动    | 任意网络拉取、写系统目录                                                                   | 单项目被攻破后影响整台机器            |
| 权限提升错觉  | <code>sudo</code> 、修改 shell 配置、写 crontab                                       | 一次批准可能变成持久后门             |
| 误合并/误覆盖 | <code>git reset --hard</code> 、强制推送策略被脚本化                                      | 协作仓库历史被破坏                |

扩展案例：不是「恶意」，而是「太勤快」

许多事故并非模型「想害你」，而是：

1. **过度泛化**：你说「清理临时文件」，它选了更「彻底」的命令。
2. **上下文缺失**：不知道 `.env` 里是真密钥还是占位符。
3. **路径脆弱性**：相对路径、`cd` 后状态、CI 工作目录与本地不一致。
4. **工具链组合**：格式化工具 + `git add -A` + 提交脚本，一步走错全盘皆输。

因此，权限设计的起点是：**假设聪明人会犯傻、聪明模型会过度优化**。

权限系统要回答的三个问题

[Mermaid 图表 — 请在在线版本中查看]

1. **谁在请求**：用户显式触发、Auto 模式下的后台分类器、CI 中的 `dontAsk` 等，责任归属与可追溯性不同。
2. **什么动作**：读文件、改仓库、跑 Bash、访问网络——后果与可逆性不同；同一动作在不同目录上风险也不同。
3. **在什么模式下**：从「只读分析」到「全跳过护栏」是一条光谱，没有一种模式适合所有场景；选错模式的代价比多点一次确认更大。

---

## Mermaid：一次草率命令如何滚雪球

---

[Mermaid 图表 — 请在在线版本中查看]

---

## 说明性源码片段：权限决策的「伪代码」骨架

---

以下不是 Claude Code 真实源码，而是帮助理解「闸门插在哪里」的示意：

```
// 示意：工具调用前的统一入口（概念层）
type PermissionDecision = "allow" | "ask" | "deny";

async function invokeTool(tool: Tool, input: unknown, context: SessionContext) {
  const decision: PermissionDecision = await evaluatePermissions({
    tool,
    input,
    mode: context.permissionMode,
    rules: context.mergedRules,
    sandbox: context.sandboxProfile,
  });

  if (decision === "deny") {
    return failClosed("blocked_by_policy", { tool: tool.name });
  }
  if (decision === "ask") {
    await requireUserApproval({ tool: tool.name, preview: summarize(input) });
  }
  return tool.run(input, { sandbox: context.sandboxProfile });
}
```

要点：

- 在真正 `exec` / `write` 之前就要完成决策；事后补救往往来不及。
- `failClosed` 表示「拿不准就拒绝」，详见 7.9。
- 规则顺序 `deny → ask → allow`、首次匹配生效，将在 7.6 与 7.10 反复强调。



## 与「写入仅限项目目录」的关系（预告）

你将在 7.8 与 7.10 看到明确约束：只能写项目目录及子目录，不能写父目录。这与 `rm -rf` 类事故形成互补：

| 约束类型  | 防什么                                       | 典型失败若缺失        |
|-------|-------------------------------------------|----------------|
| 写入边界  | 逃逸到 <code>../</code> 、家目录、系统路径            | 单项目任务污染全局环境    |
| 命令黑名单 | <code>curl</code> / <code>wget</code> 默认禁 | 一键拉取远程 payload |
| 沙箱    | 文件系统与网络隔离                                 | 误触敏感路径或外连      |

## 与后续章节的路线图

| 小节   | 文件                                     | 你将学到                                                             |
|------|----------------------------------------|------------------------------------------------------------------|
| 7.2  | <code>02-six-modes.md</code>           | 六种权限模式一张总表                                                       |
| 7.3  | <code>03-basic-modes.md</code>         | Default / acceptEdits / Plan 的日常用法                               |
| 7.4  | <code>04-auto-mode.md</code>           | Auto 与后台分类器（Sonnet 4.6）                                          |
| 7.5  | <code>05-advanced-modes.md</code>      | <code>dontAsk</code> （CI）与 <code>bypassPermissions</code> （隔离容器） |
| 7.6  | <code>06-evaluation-pipeline.md</code> | 七步评估管道完整流程                                                       |
| 7.7  | <code>07-bash-ast.md</code>            | Bash 子命令与 AST 级检查思路                                              |
| 7.8  | <code>08-sandbox.md</code>             | Seatbelt / bubblewrap 隔离                                         |
| 7.9  | <code>09-fail-closed.md</code>         | Fail-closed 安全哲学                                                 |
| 7.10 | <code>10-practice.md</code>            | 企业级实践与 allowlist 缓解 Prompt fatigue                               |

## FAQ（与本节强相关）

问：用户每次都点「允许」，权限系统还有意义吗？

答：有意义。系统仍可提供默认 deny、沙箱、写入边界、审计轨迹；交互疲劳用 allowlist 缓解（7.10），而不是直接关掉护栏。

问：是不是只要不用 Bash 就安全了？

答：不是。编辑工具写错路径、依赖安装脚本、插件市场供应链等，同样属于「高后果动作」，需要 Edit 路径检查 与 七步管道（7.6–7.7）。

问：最危险的组合是什么？

答：高权限模式 + 无隔离环境 + 宽规则。例如在非容器内长期 `bypassPermissions` (7.5)，等价于把钥匙串扔掉。

## 小结

- **模型不是运维**：它的目标是「完成任务」，不是你的数据保全顾问。
- **权限是产品特性**：把「危险动作」变成可解释、可重复、可审计的决策。
- 下一节从六种模式的总表开始，把「我此刻处于哪种信任级别」说清楚。

## 自测 checklist

- [ ] 能否用一句话说明：为什么 `curl|bash` 与 `rm -rf` 同属「必须过闸门」类操作？
- [ ] 能否画出「模式 → 是否询问 → 是否沙箱」的粗粒度心智图（可先手绘，再在 7.2 对照）？
- [ ] 是否理解「规则顺序 deny→ask→allow、首次匹配」将在 7.6 正式展开？
- [ ] 能否举出一个「非恶意但后果严重」的自动化例子，并说明哪一类规则会拦截它？

## 术语速查（本篇）

| 术语          | 含义                           |
|-------------|------------------------------|
| Fail-closed | 不确定则拒绝，见 7.9                 |
| 沙箱          | OS 级隔离执行环境，见 7.8             |
| 七步管道        | 从工具 deny 到安全护栏的完整评估链，见 7.6   |
| allowlist   | 显式列出可自动通过的安全命令，缓解反复确认，见 7.10 |

## 延伸阅读：事故时间线（虚构但典型）

下列时间线用于团队内部分享，帮助新成员建立肌肉记忆：

| 时刻 | 事件                               | 若有权限闸门                    |
|----|----------------------------------|---------------------------|
| T0 | 模型建议删除 <code>dist/</code>        | 可能仅 ask 或 allow（低风险目录）    |
| T1 | 路径写成 <code>../dist</code> 或变量为空  | Edit/Bash 路径检查或 deny 拦截   |
| T2 | 附带 <code>find ... -delete</code> | Bash AST 层识别高危子命令（7.7）    |
| T3 | 用户连点三次「允许」                       | allowlist 应吸收重复安全命令（7.10） |
| T4 | 发现 <code>.env</code> 被删          | 审计日志定位「谁批准的」              |

## 与「Prompt injection」的边界说明

权限系统主要防的是高后果动作被误执行，而不是替代：

- 代码审查流程
- 依赖漏洞扫描
- 密钥轮换与 HSM

把它想成车速限制 + 安全气囊：不保证不出事故，但显著降低伤亡。

下一篇：[7.2 六种模式对比表](#)

## 7.2 六种权限模式：一张总表读懂全局

**本篇定位：**在 7.1 建立「为什么需要权限」之后，本节用**对比表 + 决策图**把六种模式一次性放到同一坐标系里，避免碎片化记忆。

### 学习目标

完成本节学习后，你应该能够：

1. **默写或复述** 六种模式的名称及其一句话定位（信任级别 + 典型场景）。
2. **对照表格** 说明：在每种模式下，**只读**、**编辑**、**Shell 命令** 三类动作默认是「自动 / 询问 / 拒绝」中的哪一种倾向。
3. **解释** `bypassPermissions` 为何必须绑定**隔离容器**，而不能当作日常开发快捷键。

4. **选择** 在代码阅读、重度重构、CI 流水线中分别更适合的模式组合。
5. **理解** Auto 模式中「后台分类器（Sonnet 4.6）」与其他模式的本质差异：**策略由模型在后台评估**，而非仅靠静态规则。
6. **预判** `dontAsk` 下「未预批准即拒绝」对脚本稳定性的影响。

## 生活类比：机场安检等级

| 模式                | 类比                             |
|-------------------|--------------------------------|
| Plan              | 只许站在玻璃外看飞机，不许登机（纯只读分析）         |
| Default           | 经济舱：随身行李要过机，托运要贴条（只读省事，动刀要确认）  |
| acceptEdits       | 常旅客：安检快一点，但液体仍有限制（编辑免批，命令仍要确认） |
| Auto              | 智能闸机：先 AI 看你的包像不像危险品，再决定要不要开箱  |
| dontAsk           | 货运清单：只收「单子上的货」，多一件都不要（CI）      |
| bypassPermissions | 消防演练专用通道：仅限封闭场地，出了场地就是事故       |

## 核心总表：六种模式对比

| 模式                | 只读类操作        | 编辑类操作    | Shell / 外部命令 | 典型用户        | 主要风险若误用                      |
|-------------------|--------------|----------|--------------|-------------|------------------------------|
| Default           | 倾向自动（免批）     | 需确认      | 需确认          | 日常开发默认      | 反复弹窗（可用 allowlist 缓解，见 7.10） |
| acceptEdits       | 倾向自动         | 免批       | 需确认          | 大规模重构、文档批量改 | 错误编辑扩散快                      |
| Plan              | 自动（在只读允许范围内） | 禁止 / 不执行 | 禁止 / 不执行     | 读代码、画架构、评审  | 无法落地改代码                      |
| Auto              | 只读常自动通过      | 依分类器与规则  | 依分类器与规则      | 想减少打断又不愿全开  | 分类误判                         |
| dontAsk           | 仅预批准集合内自动    | 同左       | 同左           | CI、自动化流水线   | 配置过宽=流水线变攻击面                 |
| bypassPermissions | 理论上全放行       | 全放行      | 全放行          | 仅隔离容器内调试    | 在主机上使用=灾难                    |

**注意：**上表描述的是**产品语义层面**的倾向；具体某一工具调用仍要经过 **7.6 七步管道**（工具 deny、ask、Bash 子命令检查、安全护栏等）。不要把「模式」理解成绕过管道的万能开关—— `bypassPermissions` 是例外中的例外。

## Mermaid：六种模式的信任光谱

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid：用户请求如何先「选模式」再「过管道」

[Mermaid 图表 — 请在在线版本中查看]

## 说明性配置片段：模式在 CLI / 设置中的示意

以下为教学用伪配置，键名以概念为主，请以你所用版本的官方文档为准：

```
# 示意：权限模式（概念键名）
permission_mode: default    # default | accept_edits | plan | auto | dont_ask | bypass_permissions

# 示意：与 dontAsk 配套的预批准前缀/命令（概念）
preapproved_commands:
  - "npm test"
  - "pnpm lint"
  - "git status"
```

要点：

- 模式解决「这一会话的信任基调」。
- 预批准列表解决「在 dontAsk 下什么仍允许自动跑」。
- 规则文件里的 `deny → ask → allow` 解决「具体到子命令/路径」的细粒度（7.6、7.10）。

## 模式与「命令黑名单」的叠加关系

你已知道：`curl` / `wget` 默认在黑名单中（拉取远程脚本的高风险通道）。模式不会 magically 撤销硬安全策略——除非进入 `bypassPermissions` 且环境本身已隔离。

| 组合                      | 预期行为（语义层）                                |
|-------------------------|------------------------------------------|
| Default + 黑名单命令         | 管道早期 <b>deny</b> 或强制 <b>ask</b> ，以产品实现为准 |
| Plan + 任意写/执行           | 应在模式层即限制，不进入执行                           |
| dontAsk + 非预批准          | <b>拒绝</b> （fail-closed 友好）               |
| bypassPermissions + 黑名单 | 可能仍被企业策略拦截； <b>不应依赖</b> 此组合              |

## 与 Auto 模式的一行预告

Auto 会启动 Sonnet 4.6 后台分类器：

- 匹配 `allow` / `deny` 规则 → **即时决策**；
- 只读操作 → 常 **自动通过**；
- 判定「不安全」→ 可能进入 **两阶段 XML 分类器** 做更稳的归类（详见 7.4）。

## 小结对照卡（建议收藏）

| 我想.....                 | 优先考虑                       |
|-------------------------|----------------------------|
| 只聊天分析、不动手               | Plan                       |
| 日常写代码、偶尔跑命令             | Default                    |
| 大批量改文件、少被打断             | acceptEdits                |
| 想少弹窗、又不敢全开              | Auto（配合规则细化）               |
| CI 里稳定跑固定脚本             | dontAsk + 窄预批准             |
| 仅在 Docker/K8s 隔离里调试极端问题 | bypassPermissions（仍要最小化时长） |

## 深入阅读索引

| 主题                              | 章节                  |
|---------------------------------|---------------------|
| Default / acceptEdits / Plan 细节 | <a href="#">7.3</a> |
| Auto 分类器                        | <a href="#">7.4</a> |
| dontAsk / bypassPermissions     | <a href="#">7.5</a> |
| 七步管道                            | <a href="#">7.6</a> |

## 自测题

- 为什么说 `acceptEdits` 是「编辑加速」而不是「安全加速」？
- `Plan` 模式下，若仍能对磁盘写入，可能违背哪条安全原则（提示：7.9）？
- 若 CI 使用 `dontAsk`，但预批准里写了 `bash -c`，可能引入什么问题？

## 附录：英文标识与中文称呼对照

| 中文叙述常用 | 英文/配置中常见写法        |
|--------|-------------------|
| 默认模式   | Default           |
| 接受编辑   | acceptEdits       |
| 计划/只读  | Plan              |
| 自动     | Auto              |
| 勿询问    | dontAsk           |
| 绕过权限   | bypassPermissions |

## 常见反模式（团队规范可照抄禁止）

| 反模式                                   | 为何危险         | 替代做法                |
|---------------------------------------|--------------|---------------------|
| 长期 <code>bypassPermissions</code> 在本机 | 等同关闭产品安全层    | 短生命周期容器 + 最小时长      |
| CI <code>dontAsk</code> + 过宽预批准       | 流水线可被打成远程执行器 | 前缀锁定 + 哈希钉扎脚本       |
| 新人默认 <code>acceptEdits</code>         | 误改扩散         | 先 Plan/Default 熟悉仓库 |
| 把 Auto 当「永不询问」                        | 分类器会错        | 关键目录 deny 优先        |

## 模式切换的检查清单

在切换模式前自问：

1. 下一小时我会不会跑网络命令或装依赖？ → 谨慎从 Plan 切到 Default。
2. 是否会改 50+ 文件？ → 考虑 acceptEdits，但先提交或开分支。
3. 是否在共享 runner 上？ → 优先 dontAsk + 窄规则，而不是 Auto。
4. 是否在客户 VPC 内？ → 禁止 bypass，除非安全团队书面批准 + 隔离证明。

上一篇：[7.1 为什么需要权限](#) · 下一篇：[7.3 Default/acceptEdits/Plan](#)



## 7.3 Default、acceptEdits、Plan 详解

**本篇定位：**三种最常用、语义最清晰的模式——**默认平衡**、**编辑加速**、**只读分析**。掌握它们后，再读 Auto / dontAsk 会轻松很多。

### 学习目标

完成本节学习后，你应该能够：

1. **准确描述** Default 模式下「只读免批、编辑与命令要确认」的产品语义，并举例三种典型弹窗场景。
2. **对比** acceptEdits 与 Default：在什么情况下换用能省时间，在什么情况下换用会放大错误。
3. **说明** Plan 模式的边界：适合架构梳理、风险评审、onboarding，而不适合需要落地改代码的任务。
4. **绘制** 三者「读写执行」三维空间中的位置关系（可对照本节 Mermaid）。
5. **结合** 七步管道（7.6）理解：模式**不替代**工具级 deny/ask。
6. **制定** 个人/团队默认策略：例如新人首周 Plan、核心库分支 Default、大规模格式化分支 acceptEdits。

### 生活类比：图书馆、编辑部、施工队

| 模式          | 类比     | 你能做什么                   |
|-------------|--------|-------------------------|
| Plan        | 图书馆阅览室 | 看书、做笔记、画思维导图；不能把书撕了重写   |
| Default     | 杂志社编辑部 | 随便读；改稿、发稿、叫印刷厂要主编签字     |
| acceptEdits | 排版车间   | 批量改样式、统一术语；但叫「外包印刷」仍要签字 |

## 核心对照表

| 维度          | Default         | acceptEdits | Plan         |
|-------------|-----------------|-------------|--------------|
| 读文件 / 分析    | 倾向 免批（在策略允许范围内） | 同左          | 核心用途：只读分析    |
| 编辑工作区       | 需确认             | 免批          | 不执行 / 禁止写入路径 |
| Bash / 外部命令 | 需确认             | 需确认         | 不执行          |
| 典型心智负担      | 中等（弹窗频率适中）      | 低（改文件快）     | 最低（但无法动手）    |
| 误操作后果       | 可控（编辑前有闸）       | 编辑错误可快速扩散   | 低（几乎不能写）     |

## Mermaid：三种模式下的请求路径（简化）

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid：何时从 Plan 切到 Default（决策树）

[Mermaid 图表 — 请在在线版本中查看]

## Default 模式：日常开发的「主档位」

### 语义（牢记）

- 只读类：在策略允许下常 自动通过，减少「读个文件也问一遍」的疲劳。
- 编辑类：需要确认——防止模型一次性改坏大量文件。
- 命令类：需要确认——`rm`、`git push --force`、包管理等都应可见、可取消。

### 适用场景

- 功能开发、修 bug、小步提交。
- 与 Code Review 流程配合：本地改动有闸，远程评审有第二道眼。
- 对接 allowlist：把 `git status`、`npm test` 等高频且低风险命令放进允许列表，缓解 Prompt fatigue（7.10）。

### 说明性片段：Default 下的「编辑闸门」示意

```
// 示意：Default - 编辑工具需显式批准
function shouldPromptForEdit(mode: PermissionMode, tool: string): boolean {
  if (mode === "plan") return true; // Plan 下应在更前阶段拦截
  if (tool === "edit" && mode === "default") return true;
  if (tool === "edit" && mode === "accept_edits") return false;
  return true; // 默认保守
}
```

## acceptEdits 模式：批量编辑的加速器

### 语义（牢记）

- 编辑免批：适合「重命名贯穿全库」「格式化」「批量改 import」等。
- 命令仍要确认：防止在「省掉编辑弹窗」的错觉下，顺手批准了高危 Shell。

### 风险与缓解

| 风险             | 缓解                     |
|----------------|------------------------|
| 模型改错 100 个文件   | 先开分支；小步 commit；用测试守护   |
| 与业务逻辑无关的「智能重排」 | Code review；关键路径加 deny |
| 误以为命令也免批       | 团队文档醒目标注「仅编辑免批」        |

### 何时切换

- 切换前：`git status` 干净或已提交检查点。
- 切换后：完成大批量任务后切回 Default，避免「长期免批编辑」成为习惯。

## Plan 模式：纯只读分析

### 语义（牢记）

- 纯只读分析：阅读、解释、画架构图、列出风险点、对比方案。
- 不执行写操作与 Shell（与产品实现对齐；若你的版本有细微差异，以官方文档为准）。

## 适用场景

- 新人阅读巨型单体仓库。
- 安全评审、合规预审（「这段代码会访问哪些外部系统？」）。
- 客户环境只允许「看」不允许「动」的演示。

## 与 Auto 的区分

- **Plan**：模式层就约束为只读路径，**不依赖**后台分类器做「能不能写」的取舍。
- **Auto**：仍可能执行经分类与规则允许的写/命令；信任模型不同（7.4）。

---

## 三种模式与「写入仅限项目目录」

---

无论你用哪种模式，**写入限制**通常仍然成立：

- 只能写项目目录及子目录；
- 不能写父目录（防止 `../` 逃逸）。

因此：

- **acceptEdits** 加速的是「项目内批量改」，不是「写到 `$HOME/.ssh`」。
- **Plan** 若尝试写父目录，应在 **Edit 路径检查**（管道第 3 步）或更早失败。

---

## 团队规范模板（可粘贴到内部 Wiki）

---

```
## Claude Code 模式使用规范
```

1. 默认：全员 **\*\*Default\*\***。
2. **release** 分支：禁止 **\*\*acceptEdits\*\***；仅 hotfix owner 可申请临时切换并在当日切回。
3. 架构评审会议：共享屏幕使用 **\*\*Plan\*\***。
4. 大规模 codemod：专用分支 + **\*\*acceptEdits\*\*** + CI 必须通过。
5. 任何 **\*\*bypassPermissions\*\*** 必须 ticket + 安全审批（见 7.5）。

## 小结

| 模式          | 一句话                |
|-------------|--------------------|
| Default     | 读省事，改和跑命令要你说「行」    |
| acceptEdits | 改可以一路畅通，跑命令仍要你说「行」 |
| Plan        | 只动嘴（分析）不动手（写/执行）   |

## 自测

- Default 下，为什么「只读免批」不等于「任何读取都免批」？（提示：敏感路径、.git）
- acceptEdits 最适合与哪三种 Git 工作流习惯搭配？
- Plan 模式能否替代代码静态分析工具？为什么？

## 故障排查速查

| 现象              | 可能原因             | 建议                  |
|-----------------|------------------|---------------------|
| 只读也总弹窗          | ask 规则过宽或工具被标为敏感 | 检查 deny/ask 顺序（7.6） |
| Plan 下仍能写       | 版本差异或插件绕过        | 核对官方 changelog      |
| acceptEdits 不生效 | 实际模式未切换或管道 deny  | 查工具级规则              |

## 场景演练（对照三种模式）

| 任务描述                                                     | 推荐模式           | 理由                                    |
|----------------------------------------------------------|----------------|---------------------------------------|
| 「用自然语言解释 <code>auth</code> 模块调用链」                        | Plan           | 不需要写盘或跑命令                             |
| 「修复登录 bug 并跑单测」                                          | Default        | 小步编辑与命令均需可见                           |
| 「全仓库把 <code>legacy_api</code> 重命名为 <code>v2_api</code> 」 | acceptEdits    | 编辑量大；命令仍单独确认                          |
| 「评估是否引入新依赖」                                              | Plan → Default | 先只读查现有依赖图，再切换执行 <code>pnpm why</code> |

## 与「命令黑名单」的日常配合

在 Default 与 acceptEdits 下，用户仍可能遇到「想 `curl` 文档」被拒：

| 用户意图         | 更安全替代                                                              |
|--------------|--------------------------------------------------------------------|
| 拉安装脚本        | 用镜像内已缓存包或内部 Artifactory                                            |
| 下载开源 release | 浏览器下载到 <code>~/Downloads</code> 后 手动 <code>cp</code> 进项目（仍受写入边界约束） |
| API 调试       | 只读 Plan 分析 + 专用 Postman/Insomnia，不把密钥塞进 Shell 历史                   |

上一篇：[7.2 六种模式](#) · 下一篇：[7.4 Auto 模式](#)

## 7.4 Auto 模式与后台分类器（Sonnet 4.6）

**本篇定位：**Auto 模式把一部分「该不该弹窗」的决策交给后台运行的 Sonnet 4.6 分类器，在规则匹配与安全降级之间找平衡。本节讲清原理、边界与运维注意点。

### 学习目标

完成本节学习后，你应该能够：

- 说明** Auto 模式中后台分类器的职责：评估工具调用意图与风险标签，映射到 allow/deny/ask。
- 描述** 「规则即时匹配 → 只读自动通过 → 不安全时两阶段 XML 分类器」的决策链。
- 对比** Auto 与 Default：前者减少打断，后者更可预测；能向经理解释取舍。
- 识别** Auto 的失效模式：分类错误、规则缺口、对抗性提示与异常工具输入。
- 配置** 思路：用 **deny 优先** 兜住高危路径与命令，Auto 负责「灰色地带」。
- 理解** 「Sonnet 4.6 在后台运行」对延迟、成本与隐私的影响（定性层面）。

### 生活类比：机场预检与复检

- 即时规则匹配：**液体容量一眼超标 → 直接拒，不用讨论。

- **只读自动通过**：空手通过金属探测门 → 默认放行。
- **两阶段 XML 分类器**：包里有奇怪形状 → 先机器断层扫描（第一阶段），仍可疑再人工开箱按清单逐项勾选（第二阶段，结构化 XML 便于程序解析）。

Auto 不是「完全不问」，而是把问的时机交给模型 + 规则，而不是事事找用户。

## 核心信息结构化

| 环节              | 行为                     | 目的                 |
|-----------------|------------------------|--------------------|
| 规则匹配 allow/deny | 命中即 即时决策               | 确定性高的策略不走模型        |
| 只读操作            | 倾向 自动通过                | 降低读文件类摩擦           |
| 判定不安全 / 不确定     | 两阶段 XML 分类器            | 结构化输出，减少胡言乱语导致的误执行 |
| 与七步管道关系         | Auto 不豁免 工具 deny、路径检查等 | 模式是「用户交互策略层」之一     |

## Mermaid：Auto 模式决策总览

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid：两阶段分类器与下游系统

[Mermaid 图表 — 请在在线版本中查看]

## 说明性伪代码：分类器输出约束为 XML

```
<!-- 示意：第二阶段期望的结构化片段（教学用） -->
<permission_decision version="1">
  <verdict>ask</verdict>
  <risk_family>network_fetch</risk_family>
  <confidence>0.72</confidence>
  <rationale>Command resembles package install with postinstall script.</rationale>
</permission_decision>
```

```
// 示意：解析层只信字段，不信散文
function parseClassifierXml(raw: string): Decision {
  const doc = safeParseXml(raw);
  const verdict = doc.query("verdict")?.text; // allow | ask | deny
  if (!["allow", "ask", "deny"].includes(verdict)) {
    return { kind: "ask", reason: "malformed_classifier_output" };
  }
  return { kind: verdict as Verdict };
}
```

要点：XML（或等价结构化格式）让程序可做 **schema 校验**；若解析失败，应 **ask 或 deny**，而不是默认 allow（fail-closed，7.9）。

## 为什么「只读」常自动通过

只读操作（在沙箱与路径策略允许范围内）通常：

- **可逆性高**（不修改工作区）；
- **频次高**（探索代码库）；
- **用户价值大**（减少无意义打断）。

但注意：读取 **.env**、密钥、**.ssh** 配置仍可能被 **安全护栏** 或 **内容特定 ask** 拦截（7.6 第 6、7 步）。

## Auto 与命令黑名单的关系

**curl / wget** 默认禁用 属于硬策略。Auto 分类器不应被理解为「可以聪明到绕过黑名单」——正确做法是：

| 层级       | 谁说了算                      |
|----------|---------------------------|
| 工具级 deny | 硬拒，无覆盖（管道第 1 步）           |
| 分类器      | 在 未 deny 的集合里细分 ask/allow |
| 用户       | 最终对 ask 路径拍板              |

## 运维与协作建议

1. 关键仓库：对 **\*\*/secrets/\*\***、**.env\***、**\*.pem** 配 deny 优先。



- 2. **监控**：记录分类器 verdict 分布，发现 `allow` 异常升高要复盘规则。
- 3. **版本钉扎**：分类器模型升级（如 Sonnet 系列小版本）可能改变行为，升级后跑一轮回归场景。
- 4. **隐私**：后台分类会上传**工具摘要**类信息到模型 API（依产品实现），敏感项目应用本地规则减小上传面。

## Auto vs Default 选择表

| 你的优先级              | 更合适的模式     |
|--------------------|------------|
| 可预测、可审计、培训新人       | Default    |
| 熟练用户、规则已细化、愿承担分类误差 | Auto       |
| 强合规、只读会议           | Plan （7.3） |

## 常见误解澄清

| 误解           | 事实              |
|--------------|-----------------|
| Auto = 全自动不问 | 仍可能 ask；deny 仍硬 |
| 分类器不会错       | 会错；用 deny 与沙箱兜底 |
| Auto 可替代代码审查 | 不能；只辅助权限交互      |

## 小结

- **Auto** 用 **Sonnet 4.6** 在后台做**风险分类**，把「灰色地带」从用户肩上卸下一部分。
- **规则匹配**负责「黑白分明」；**只读**负责「高频低害」；**XML 两阶段**负责「结构化 + 可解析 + 可 fail-closed」。
- 与 **七步管道** 叠加使用，**永远不要用** Auto 作为唯一安全层。

## 自测

- 1. 若 XML 解析失败，系统应采取 allow 还是 ask/deny？为什么？
- 2. 举出一个「只读但仍敏感」的路径，说明它为何不应仅依赖「只读自动通过」。

3. deny 规则与分类器同时存在时，谁先谁后？（提示：首次匹配）

### 参考串联

- 七步管道：[7.6](#)
- Fail-closed：[7.9](#)
- 企业 allowlist：[7.10](#)

### 监控指标建议（定性）

| 指标                       | 异常信号               |
|--------------------------|--------------------|
| <code>allow</code> 比例周环比 | 突然上升 → 规则被改宽或分类器漂移 |
| <code>ask</code> 集中在某仓库  | 该仓库路径规则缺失          |
| 用户手动改为 Default           | 对 Auto 不信任的投票      |

### 与「只读自动通过」的例外讨论

以下只读仍可能需要 ask 或 deny（取决于企业基线）：

| 路径/内容                                    | 原因           |
|------------------------------------------|--------------|
| <code>**/*secret*</code>                 | 命名即敏感        |
| <code>*.pem</code> / <code>id_rsa</code> | 密钥材质         |
| <code>.claude/settings.local.json</code> | 可能含 token    |
| 巨大二进制                                    | 非安全但涉带宽/隐私政策 |

### 版本与模型说明（声明）

本节中 Sonnet 4.6 与 两阶段 XML 分类器 描述依据产品公开语义整理；具体模型名、端点与日志字段以你所用版本为准。升级大版本后应复测三类用例：**纯读**、**安全写**、**明显恶意链**。

上一篇：[7.3 基础模式](#) · 下一篇：[7.5 高级模式](#)

## 7.5 dontAsk 与 bypassPermissions

**本篇定位：** dontAsk 服务 CI 与确定性自动化；bypassPermissions 是「核弹选项」，**仅应在隔离容器中使用**。本节把边界、风险与合规话术写清楚。

### 学习目标

完成本节学习后，你应该能够：

- 1. **复述** dontAsk 的核心语义：**只执行预批准**，否则 **拒绝**；说明为何适合 CI。
- 2. **解释** bypassPermissions：**全跳过交互闸门**；为何文档与内规应写「仅隔离容器」。
- 3. **设计** 一套「预批准命令」清单的粒度：前缀锁定、哈希钉扎、禁止 `bash -c` 泛化。
- 4. **向安全团队陈述** 使用 bypassPermissions 的审批材料：威胁模型、时长、网络策略、日志留存。
- 5. **对比** dontAsk 与 Auto：前者 **确定性**，后者 **模型参与**。
- 6. **识别** 典型事故：共享 runner 上误开 bypass、预批准里写了 `curl`。

### 生活类比

| 模式                | 类比                                                 |
|-------------------|----------------------------------------------------|
| dontAsk           | 无人值守自动售货机：只认「编好号的货道」；你塞硬币想买没上架的货 → <b>直接退币（拒绝）</b> |
| bypassPermissions | 消防演习时拆掉办公楼门禁：只在封闭演练场有效；若在日常办公楼常态拆除 → <b>全员失窃风险</b> |

## 核心对照表

| 维度   | dontAsk                  | bypassPermissions    |
|------|--------------------------|----------------------|
| 用户交互 | 无（未预批准则拒）                | 无（全跳过）               |
| 适用场景 | CI、自动化、回归脚本              | 仅 Docker/K8S/VM 隔离调试 |
| 确定性  | 高（清单驱动）                  | 低（依赖环境隔离）            |
| 配错后果 | 流水线失败（fail-closed 友好）    | 主机被一锅端（fail-open 灾难） |
| 与黑名单 | 预批准仍不应包含 curl/wget 类（应拒） | 技术上可能执行，绝对不应依赖       |

## Mermaid： dontAsk 执行流

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid： bypassPermissions 与容器边界

[Mermaid 图表 — 请在在线版本中查看]

## dontAsk：预批准清单设计原则

### 推荐做法

| 原则         | 说明                                                              |
|------------|-----------------------------------------------------------------|
| 前缀锁定       | <code>npm run test --</code> 固定脚本名，避免自由拼接                       |
| 禁止泛化 shell | 不写 <code>sh -c</code> / <code>bash -c</code> 通配                 |
| 哈希钉扎       | 对辅助脚本 <code>sha256sum</code> 校验后再执行                             |
| 环境变量最小化    | CI secret 仅注入必要键                                                |
| 分环境清单      | <code>ci-frontend.yaml</code> / <code>ci-backend.yaml</code> 分离 |

## 说明性 YAML 片段

```
# 示意: dontAsk 预批准 (教学)
permission_mode: dont_ask
preapproved_commands:
  - "pnpm -s test"
  - "pnpm -s lint"
  - "git diff --stat"
# 明确不要写:
# - "curl https://..."
# - "bash -c '...任意...'"
```

## bypassPermissions: 何时「理论上可用」

仅当同时满足：

1. 进程在隔离文件系统内（容器 rootfs，不包含宿主 `$HOME`）。
2. 网络策略默认拒绝或严格 egress allowlist。
3. 生命周期短：排障结束立即销毁容器。
4. 审批与 ticket：谁、何时、为何、如何验证无数据渗出。
5. 日志外送到公司 SIEM（若合规要求）。

## 伪代码：双重门禁

```
function assertBypassAllowed(ctx: RuntimeContext) {
  if (ctx.permissionMode !== "bypass_permissions") return;

  if (!ctx.isolatedContainer) {
    throw new SecurityError("bypassPermissions requires isolated container");
  }
  if (!ctx.approvalTicketId) {
    throw new SecurityError("missing approval ticket");
  }
  // 仍可叠加：宿主机上直接拒绝启动该模式
}
```

## dontAsk vs Auto（给架构师的表）

|         | dontAsk | Auto          |
|---------|---------|---------------|
| 决策主体    | 清单 + 管道 | 清单 + 管道 + 分类器 |
| CI 可重复性 | 优       | 中（模型输出需约束）    |
| 灵活性     | 低       | 高             |
| 误配风险类型  | 清单过宽    | 规则 + 模型双误差    |

经验：CI 用 dontAsk，人机协作用 Default/Auto。

## 合规话术模板（提交给安全审批）

申请事项：在隔离构建容器内短时启用 `bypassPermissions`  
时间窗口：2026-04-02 14:00-16:00 UTC  
原因：复现仅当权限全放行时触发的工具链 bug  
隔离证明：镜像 ID xxx，无宿主挂载，egress 仅 registry 白名单  
数据：仓库不含 PII；构建日志脱敏后上传 bucket yyy  
回滚：容器销毁；模式恢复 dontAsk

## 与七步管道的关系

即使 `bypassPermissions`，在企业加固版实现中仍可能保留：

- 宿主机级别的 `Seatbelt/bubblewrap`（7.8）
- 某些 硬编码 `deny`（视产品策略）

教学上请假设：`bypass` 跳过的是「用户交互」，不保证跳过所有 OS 策略——以实际版本为准。

## 命令黑名单提醒

`curl/wget` 默认禁：dontAsk 的预批准若试图放行，应在 `code review` 阶段拒绝合并。`bypass` 在容器内也不应成为下载任意脚本的借口。

## 小结

| 模式                | 一句话                    |
|-------------------|------------------------|
| dontAsk           | 只跑清单上的；清单外的统统拒绝——CI 之友 |
| bypassPermissions | 跳过询问的核弹——只配待在隔离容器里     |

## 自测

- 为什么 `preapproved_commands: ["bash -c"]` 几乎等于放弃 dontAsk？
- bypassPermissions 与「写入仅限项目目录」同时成立时，攻击面还剩哪些？
- 若 CI 需要 `curl` 下载依赖，更安全的替代方案是什么？（提示：缓存层、镜像预装、私有 registry）

## 故障排查

| 现象           | 可能原因                 |
|--------------|----------------------|
| dontAsk 下全失败 | 命令与清单字符串不完全一致（空格/参数） |
| bypass 仍弹窗   | 企业策略或宿主机拦截           |
| CI 间歇通过      | Auto 与 dontAsk 混用    |

## 时间盒（Time-boxing）实践

对 bypassPermissions 建议强制 时间盒：

| 字段          | 示例                                                             |
|-------------|----------------------------------------------------------------|
| 开始 UTC      | 2026-04-02T06:00:00Z                                           |
| 结束 UTC      | 2026-04-02T08:00:00Z                                           |
| 审批人         | <a href="mailto:security@example.com">security@example.com</a> |
| 容器镜像 digest | <code>sha256:abcd...</code>                                    |

到期后 CI 与本地 profile 自动回退 至 Default/dontAsk，避免「临时绕过」变成永久配置。

上一篇: [7.4 Auto 模式](#) · 下一篇: [7.6 七步管道](#)

## 7.6 七步权限评估管道（完整流程图）

**本篇定位：**把分散的规则串成一条有序管道。顺序很重要：`deny` → `ask` → `allow`，首次匹配生效。本节是第 7 篇的理论中枢。

### 学习目标

完成本节学习后，你应该能够：

- 按顺序背诵** 七个步骤，并说明每一步的输入/输出。
- 解释** 为何「工具级 `deny`」是硬拒且无覆盖（在标准语义下）。
- 说明** 「工具级 `ask`」与「沙箱例外」的关系（第 2 步）。
- 指向** Bash 与 Edit 的专项检查分别属于第 3 步的哪类逻辑（7.7 详解）。
- 区分** 第 5 步「用户交互要求」与第 6 步「内容特定 `ask`」。
- 列举** 第 7 步安全护栏涉及的典型路径：`.git`、`.claude`、shell 配置等。

### 生活类比：海关七道岗

| 步骤         | 类比                     |
|------------|------------------------|
| 1 工具 deny  | 绝对禁运清单：毒品一律扣，无「领导批条」   |
| 2 工具 ask   | 特许通道也要抽检：外交邮袋可过，但仍可能开箱 |
| 3 工具特定检查   | 品类专检：肉类检疫、电子产品辐射       |
| 4 工具实现拒绝   | 仪器故障/读数异常：宁可滞留         |
| 5 用户交互     | 你本人到场签字                |
| 6 内容特定 ask | 行李里发现液体超量：单独问这一瓶       |
| 7 安全护栏     | 国门生物安全：土壤、种子一律特殊处置     |



## 七步管道总表

| 步骤 | 名称          | 要点                                                                     |
|----|-------------|------------------------------------------------------------------------|
| 1  | 工具级 deny 规则 | 硬拒，无覆盖；命中即停                                                            |
| 2  | 工具级 ask 规则  | 可定义「沙箱例外」等：仍可能触发询问或受限执行                                                |
| 3  | 工具特定检查      | <b>Bash</b> ：子命令/AST； <b>Edit</b> ：路径是否在允许写入范围                         |
| 4  | 工具实现拒绝      | 运行时实现层拒绝（参数不合法、越权等）                                                    |
| 5  | 用户交互要求      | 当前模式若要求确认 → 弹窗/CLI 提示                                                  |
| 6  | 内容特定 ask    | 命中敏感内容模式 → 即使前面偏 allow 也可能要问                                           |
| 7  | 安全护栏        | <code>.git</code> 、 <code>.claude</code> 、 <code>shell</code> 配置等全局敏感区 |

## Mermaid：七步管道（自上而下）

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid：规则顺序 deny → ask → allow（首次匹配）

[Mermaid 图表 — 请在在线版本中查看]

## 各步详解（精简版）

### 第 1 步：工具级 deny（硬拒无覆盖）

- 目的：把「永不希望发生」的动作挡在最前。
- 特性：命中后不再被 allow 或用户「我记得批准过」覆盖——这是防社会工程与误点的关键。
- 示例：`rm -rf /`、`curl` 下载远程脚本、写入 `~/.ssh`。

### 第 2 步：工具级 ask（沙箱例外）

- 目的：对「需要知道一下」的动作统一打标。

- **沙箱例外**：某些 ask 规则可配合 **受限沙箱**（7.8）——例如允许在只读网络下试跑，但仍记录。
- **与 deny 区别**：ask 是「可谈」，deny 是「免谈」。

### 第 3 步：工具特定检查

- **Bash**：解析命令 AST，识别管道、子 shell、逻辑与；匹配黑名单子命令（7.7）。
- **Edit**：校验目标路径位于**项目目录及子目录**，禁止写父目录 `../` 逃逸。

### 第 4 步：工具实现拒绝

- SDK/适配器层发现非法参数、越界索引、与模式冲突时直接拒绝。
- 这一步保证「即使规则文件写错」，实现层仍有最后一道类型安全。

### 第 5 步：用户交互要求

- **Default**：编辑与命令常在此步前已倾向 ask。
- **acceptEdits**：编辑可能跳过，命令仍可能进入。
- **Plan/dontAsk/bypass**：各自短路逻辑不同（见前文各篇）。

### 第 6 步：内容特定 ask

- 即使工具类型安全，**内容**可能敏感：例如文件中出现 `AWS_SECRET_ACCESS_KEY=`。
- 这一步解决「读看起来无害，但内容不能外传」类问题。

### 第 7 步：安全护栏

- **.git**：防止破坏对象库、hook 投毒。
  - **.claude**：防止规则与技能被篡改闭环。
  - **Shell 配置**：`.bashrc`、`.zshrc` 等持久化执行面。
-

## 说明性源码片段：管道编排器

```
type StepResult = "continue" | "deny" | "ask" | "allow";

async function permissionPipeline(req: ToolRequest): Promise<FinalDecision> {
  if (matchFirst(req, rules.deny)) {
    return finalize("deny", { step: 1, reason: "tool_deny" });
  }
  if (matchFirst(req, rules.ask)) {
    const r = await handleAskWithSandboxExceptions(req);
    if (r === "deny") return finalize("deny", { step: 2 });
    if (r === "ask") return finalize("ask", { step: 2 });
  }
  if (!toolSpecificChecks(req)) {
    return finalize("deny", { step: 3, reason: "bash_or_path" });
  }
  if (toolRuntimeReject(req)) {
    return finalize("deny", { step: 4 });
  }
  if (modeRequiresUserPrompt(req)) {
    return finalize("ask", { step: 5 });
  }
  if (contentSpecificAsk(req)) {
    return finalize("ask", { step: 6 });
  }
  if (safetyGuardrails(req)) {
    return finalize("deny", { step: 7 });
  }
  return finalize("allow", { step: 7 });
}
```

## 与六种模式的「交汇点」

模式主要影响 第 5 步 及之前的是否短路（例如 Plan 在更上层限制写/执行）。但 第 1 步 deny 通常不因模式而消失——除非进入企业不允许的 bypass 路径。

## 写入限制在管道中的位置

| 约束      | 典型步骤                      |
|---------|---------------------------|
| 仅项目目录可写 | 第 3 步 Edit 路径检查 + 第 7 步护栏 |
| 禁止父目录   | 路径规范化后比对 project root     |

## 命令黑名单在管道中的位置

| 约束            | 典型步骤                       |
|---------------|----------------------------|
| curl/wget 默认禁 | 第 1 步 deny 或第 3 步 Bash 子命令 |

## 小结 Checklist

- [ ] deny 最先，硬拒无覆盖
- [ ] ask 其次，可结合沙箱
- [ ] Bash/Edit 专项检查
- [ ] 实现层可拒
- [ ] 模式驱动用户交互
- [ ] 内容敏感再问
- [ ] 全局护栏收尾

## 自测

1. 若 deny 与 allow 同时「匹配」，最终以谁为准？为什么叫「首次匹配」？
2. 第 6 步与第 5 步的触发条件有何本质不同？
3. 安全护栏为何放在最后一步而不是最前？（提示：性能与语义分工）

## 与相邻章节映射

| 步骤        | 深入阅读                                      |
|-----------|-------------------------------------------|
| 3 Bash    | <a href="#">7.7</a>                       |
| 7 护栏 + 沙箱 | <a href="#">7.8</a> 、 <a href="#">7.9</a> |
| 规则维护      | <a href="#">7.10</a>                      |

上一篇: [7.5 高级模式](#) · 下一篇: [7.7 Bash AST](#)

## 7.7 BashTool 与 AST 命令分析

**本篇定位：**Shell 是**最高后果**接口之一。BashTool 通过 **AST（抽象语法树）**级解析，识别子命令、管道与逻辑组合，而不是 naive 的字符串 `includes("rm")`。本节讲思路、边界与绕过防御。

### 学习目标

完成本节学习后，你应该能够：

- 1. **解释** 为何正则匹配 Shell 文本不可靠，而 AST 分析更接近真实语义。
- 2. **识别** Bash 管道中的「隐蔽子命令」：`curl ... | bash`、命令替换 ``...``、`$(...)`。
- 3. **说明** 七步管道中 **第 3 步** 如何将 Bash 检查与 deny/ask 规则衔接。
- 4. **列举** 与 `curl/wget` **默认禁** 相关的典型 AST 节点类型（简单命令、前缀赋值等）。
- 5. **设计** 团队级 **allowlist** 缓解 Prompt fatigue 时，如何避免被 `&&` 链接绕过。
- 6. **理解** AST 分析的局限：heredoc、别名、`eval`、外部解释器。

### 生活类比：安检 X 光机 vs 只看行李标签

- **字符串包含检测：**只看行李箱挂牌写着「衣服」——里面可能是液体。
- **AST 分析：**X 光成像分辨结构：瓶子在角落、形状异常 → 进一步开包（ask/deny）。

### 核心对比表：字符串 vs AST

| 方法       | 优点     | 缺点                                                |
|----------|--------|---------------------------------------------------|
| 子串黑名单    | 实现快    | <code>c\uurl</code> 、 <code>'c'url</code> 、变量展开绕过 |
| token 分词 | 比子串稳   | 不懂管道与控制结构                                         |
| AST 级    | 覆盖组合结构 | 仍需处理别名/eval/多解释器                                  |

## Mermaid: Bash 请求在管道中的位置

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid: 管道与命令替换的数据流

[Mermaid 图表 — 请在在线版本中查看]

## AST 节点类型（教学用枚举）

以下为示意性 TypeScript，帮助建立心智模型：

```
// 示意: Bash AST 节点 (大幅简化)
type BashNode =
  | { kind: "simple"; argv: string[]; assignments: Record<string, string> }
  | { kind: "pipeline"; cmds: BashNode[] }
  | { kind: "list"; op: "&&" | "||" | ";"; left: BashNode; right: BashNode }
  | { kind: "group"; subshell: boolean; body: BashNode }
  | { kind: "redirect"; target: BashNode; op: ">" | ">>" | "<"; file: string };

interface BashAnalyzer {
  collectSimpleCommands(root: BashNode): SimpleCommand[];
  expandAliases(cmd: string): string; // 可能不可解析 → fail-closed
}
```

要点：

- **pipeline**：必须左右都扫；`curl ... | bash` 的危险在**组合**，不单在左或右。
- **list**：`&&` 链接的**每一段**都要策略评估；`allowlist` 不能只批最左段。
- **group / subshell**：递归进入子树。

## 说明性伪代码：访问者遍历

```
function analyzeBash(root: BashNode, policy: Policy): Decision {
  const simples = flattenSimpleCommands(root);
  for (const s of simples) {
    const name = s.argv[0];
    if (policy.denyCommands.has(name)) {
      return { verdict: "deny", reason: `denied_command:${name}` };
    }
  }
  if (containsDownloadPipeToShell(root)) {
    return { verdict: "deny", reason: "curl_pipe_shell" };
  }
  if (policy.askCommands.hasAny(simples)) {
    return { verdict: "ask" };
  }
  return { verdict: "continue" };
}
```

## curl / wget 默认禁：AST 视角

| 场景                                  | AST 特征                      | 期望策略        |
|-------------------------------------|-----------------------------|-------------|
| 单独 <code>curl URL</code>            | simple + argv0=curl         | deny 或强 ask |
| <code>curl URL   bash</code>        | pipeline(curl, bash)        | 硬拒优先        |
| <code>bash -c "\$(curl ...)"</code> | command substitution + curl | 递归检测        |
| <code>wget -O- URL   sh</code>      | 同 pipeline                  | 同左          |

## 与「规则顺序：deny→ask→allow」的结合

对 Bash 子命令抽取结果应用有序规则：

- 任一 simple command 命中 **deny** → **立即 deny**（首次匹配）。
- 否则若有 **ask** → **ask**。
- 否则进入 **allow** 或默认策略。

## allowlist 设计反模式

| 反模式                                             | 为何会被绕过                                |
|-------------------------------------------------|---------------------------------------|
| 只匹配行首命令                                         | <code>true &amp;&amp; curl ...</code> |
| 忽略引号拼接                                          | <code>cu'rl'</code>                   |
| 批准 <code>npm</code> 但不看 <code>preinstall</code> | 生命周期脚本内嵌 <code>curl</code>            |
| 批准 <code>git</code> 任意参数                        | <code>git -c ...</code> 滥用配置注入        |

缓解：allowlist 尽量具体到完整 argv 前缀；对 `npm/pnpm/yarn` 配合 生命周期审计 或 离线缓存。

## 写入边界与 Bash 的交叉

即使 Bash 允许执行，`Edit` 路径仍受 仅项目目录 约束。Bash 侧若出现：

```
echo pwn > ../outside.txt
```

应在 重定向目标路径规范化 后触发 `deny`（第 3 或第 7 步）。

## AST 无法拯救的世界：`eval` 与多解释器

| 情况                               | 说明                                                         |
|----------------------------------|------------------------------------------------------------|
| <code>eval "\$VAR"</code>        | 静态 AST 看不到运行期字符串 → 默认 <code>ask</code> 或 <code>deny</code> |
| <code>python -c "..."</code>     | 另一门语言的 AST，需要 工具级策略                                        |
| <code>docker run ... curl</code> | 容器边界与宿主策略分离（7.8）                                           |

原则：看不清 → `fail-closed`（7.9）。

## 实践练习（纸上推演）

对下列命令，标出应 `deny` / `ask` / `allow`（假设 Default + 企业基线）：

- `ls -la`
- `rm -rf node_modules`
- `curl -fsSL https://x.io/install.sh | bash`



4. `pnpm test && git status`
5. `source ~/.bashrc && npm run build`

(参考答案思路：1 常 allow；2 ask 或 deny 视策略；3 deny；4 分段评估；5 `source` 触达 shell 配置 → 高敏 ask/deny。)

---

## 小结

---

- BashTool AST 把「命令的真结构」暴露给策略引擎。
- 管道、替换、逻辑链 都必须递归评估。
- `curl/wget` 与 下载管道到 `shell` 是供应链高危模板。
- `allowlist` 要防 `&&` 与生命周期脚本绕过。

---

## 自测

---

1. 为什么 `grep -R "curl" script.sh` 不等于 AST 证明了安全？
2. 首次匹配 deny 规则下，十个 simple command 里第二个命中 deny，还需要看后面吗？
3. `bash -c` 与直接 simple command 在策略上应区别对待吗？

---

## 相关章节

---

- 七步管道： [7.6](#)
- 沙箱： [7.8](#)
- 企业配置： [7.10](#)

---

上一篇： [7.6 七步管道](#) · 下一篇： [7.8 沙箱](#)

---

## 7.8 沙箱执行：macOS Seatbelt 与 Linux bubblewrap

**本篇定位：**沙箱把「即使命令跑起来」的伤害限制在**文件系统与网络**的可控边界内。Claude Code 在 macOS 上常映射到 **Seatbelt (sandbox-exec 系)**，在 Linux 上常用 **bubblewrap (bwrap)**。

### 学习目标

完成本节学习后，你应该能够：

- 1. **说明** 沙箱解决的是哪类威胁：误删、越权读、外向 C2、横向移动。
- 2. **对比** Seatbelt 与 bubblewrap 的适用 OS 与典型限制维度。
- 3. **解释** 「文件系统隔离」与「写入仅限项目目录」如何叠加。
- 4. **描述** 网络隔离对 `curl` / `git fetch` 等命令的可见影响。
- 5. **向团队说明** 沙箱不是「防模型坏」，而是「防命令真坏」。
- 6. **识别** 沙箱的常见逃逸面：挂载泄漏、Unix socket、ptrace、内核漏洞（定性了解即可）。

### 生活类比：生物实验室负压舱

- **沙箱：**你在负压舱里做实验；碎了试管，污染**主要在舱内**，不扩散到整栋楼。
- **无沙箱：**在食堂操作台做同实验——后果自负。

### 核心对照表

| 平台    | 典型技术                       | 常见限制                           |
|-------|----------------------------|--------------------------------|
| macOS | Seatbelt / Sandbox profile | 读写路径集合、网络、子进程能力                |
| Linux | bubblewrap                 | mount 命名空间、PID 网络命名空间、drop cap |

## Mermaid：沙箱在工具执行栈中的位置

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid：文件系统视图（示意）

[Mermaid 图表 — 请在在线版本中查看]

## Seatbelt（macOS）要点

| 主题      | 说明                              |
|---------|---------------------------------|
| Profile | 声明式规则：可读路径、可写路径、网络出入            |
| 与签名关系   | 系统级沙箱常与企业 MDM 策略交织；教学聚焦概念       |
| 调试成本    | 权限拒绝日志需 Console / log stream 辅助 |

### 说明性 profile 片段（概念，非真实语法照搬）

```
;; 示意：Seatbelt 风格「仅项目可写」概念
(allow file-read-data (subpath "/workspace/project"))
(allow file-write-data (subpath "/workspace/project"))
(deny file-write-data (subpath "/Users"))
```

## bubblewrap（Linux）要点

| 主题          | 说明                                                                            |
|-------------|-------------------------------------------------------------------------------|
| 核心思想        | 新 mount namespace + 最小 rootfs + 可选 <code>--unshare-net</code>                 |
| 常用 flags 概念 | <code>--ro-bind</code> 、 <code>--bind</code> 、 <code>--die-with-parent</code> |
| 与容器关系       | 比 Docker 更轻；适合 CLI 工具封装                                                       |

## 说明性命令行（教学拼接）

```
# 示意: bwrap 仅绑定项目目录为读写，系统路径只读或不可见
bwrap \
  --ro-bind / / \
  --bind /home/user/proj /home/user/proj \
  --chdir /home/user/proj \
  --unshare-pid \
  --proc /proc \
  --dev /dev \
  bash -lc "npm test"
```

真实生产参数需安全团队评审；上例仅为结构演示。

## 网络隔离层级

| 级别                 | 效果          | 副作用                                         |
|--------------------|-------------|---------------------------------------------|
| 完全断网               | 杜绝外连 C2     | <code>npm i</code> 、 <code>go mod</code> 失败 |
| 仅 allowlist egress | 允许 registry | 配置复杂                                        |
| 允许 DNS 限制          | 降低数据渗出      | 仍可能 DNS 隧道（高级威胁）                            |

实践：开发机可较松，CI 用 `dontAsk` + 断网或白名单（7.5、7.10）。

## 与七步管道第 2 步「沙箱例外」

工具级 `ask` 可声明：在沙箱 `profile A` 下允许试运行，在 `profile B` 下必须用户批准。这样把「试一试」的风险关在舱内。

[Mermaid 图表 — 请在在线版本中查看]

## 写入限制：项目目录 vs 沙箱 bind

| 机制                    | 作用                              |
|-----------------------|---------------------------------|
| 逻辑路径检查（Edit/Bash 重定向） | 即使沙箱松，也拒绝 <code>../</code> 写父目录 |
| 沙箱 bind               | 从 OS 层使宿主路径不可见                  |

二者 互补：逻辑层防配置错误，OS 层防底层绕过尝试。

## 命令黑名单在沙箱内是否仍重要？

重要。沙箱不是万能的：

- 若沙箱仍可见 项目内 `.env`，`cat` 即可泄密，无需网络。
- 若允许 `Unix domain socket` 连到宿主服务，可能横向移动。
- `curl 禁` 减少「沙箱内下载 payload」的概率，降低与内核/容器逃逸链组合风险。

## 故障排查表

| 现象           | 可能原因                                     |
|--------------|------------------------------------------|
| 构建失败 EACCES  | 沙箱可写路径未包含 <code>node_modules</code> 缓存目录 |
| git clone 失败 | 网络 unshare                               |
| 仅 macOS 复现   | Seatbelt profile 与 Linux 不一致             |

## 与 bypassPermissions 的再次强调

在 `bypassPermissions` 下，用户交互跳过，但企业加固仍可能套沙箱。绝对不应把 `bypass` 理解为「宿主机裸奔合理」。

## 小结

- macOS Seatbelt 与 Linux bubblewrap 提供 FS + 网络 维度的隔离。
- 与 七步管道、写入边界、黑名单 叠加，形成纵深防御。

- 沙箱配置是运维 + 安全共有资产，应与仓库 README 同步变更。

### 自测

- 沙箱能防住「模型读了 `.env` 并通过聊天外泄」吗？
- 为什么「项目目录可写 + 禁止父目录」在逻辑层仍然必须？
- bubblewrap 的 `--ro-bind / /` 与最小 rootfs 方案各有什么权衡？

### 延伸阅读线索

- Fail-closed 哲学：[7.9](#)
- 企业实践：[7.10](#)

### 跨平台一致性检查表

团队同时有 macOS 与 Linux 开发者时，建议维护一张对齐表：

| 能力                     | macOS Seatbelt               | Linux bwrap                           |
|------------------------|------------------------------|---------------------------------------|
| 项目目录读写                 | profile <code>subpath</code> | <code>--bind</code> rw                |
| 禁止 <code>\$HOME</code> | deny 规则                      | 不 bind home                           |
| 网络                     | entitlement / profile        | <code>--unshare-net</code> 或 iptables |
| 临时目录                   | 明确 <code>/tmp</code> 子路径     | <code>--tmpfs /tmp</code>             |

目标：同一仓库在两种 OS 上「能跑同一套测试」且「都碰不到父目录敏感文件」。

上一篇：[7.7 Bash AST](#) · 下一篇：[7.9 Fail-closed](#)

## 7.9 Fail-closed 安全哲学

**本篇定位：**Fail-closed（故障关闭）指：当策略引擎**不确定**、组件**故障**或输入**无法解析**时，默认选择 **拒绝或询问**，而不是 **允许**。这是权限系统与「产品要好用」张力下的安全基石。

## 学习目标

完成本节学习后，你应该能够：

1. **用一句话定义** fail-closed，并对比 fail-open 在 AI 工具场景下的后果。
2. **列举** 至少五种应触发 fail-closed 的条件：规则解析失败、AST 错误、XML 分类器损坏、沙箱启动失败等。
3. **解释** 为何「工具级 deny 无覆盖」是 fail-closed 在策略层的体现。
4. **向非技术同事说明**：为什么「系统坏了宁可停工」比「系统坏了全开」更符合安全合规。
5. **设计** 日志字段，使 fail-closed 事件可事后审计。
6. **理解** dontAsk 模式下「未预批准即拒绝」与 fail-closed 的一致性。

## 生活类比：电梯急停 vs 电梯失控

- **Fail-closed**：传感器异常 → **刹车关门停运**——乘客抱怨，但没人坠梯。
- **Fail-open**：传感器异常 → **忽略限制全速运行**——短期「还能用」，长期「会出人命」。

对能执行 Shell 的 AI 助手来说，一次 fail-open 的代价可能是整个代码库与密钥。

## 核心对照：Fail-closed vs Fail-open

| 维度    | Fail-closed | Fail-open     |
|-------|-------------|---------------|
| 默认倾向  | 拒绝 / 询问     | 允许            |
| 用户体验  | 可能「卡住」      | 流畅直到出事        |
| 合规友好度 | 高           | 低             |
| 运维信号  | 明确报错需修      | 静默埋雷          |
| 适用场景  | 高后果工具调用     | 低后果只读展示（仍需谨慎） |

## Mermaid：决策不确定时的汇合点

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid：与七步管道的关系

[Mermaid 图表 — 请在在线版本中查看]

## 应触发 Fail-closed 的典型条件

| 条件         | 示例                       | 期望行为                            |
|------------|--------------------------|---------------------------------|
| 规则文件语法坏    | YAML 缩进错误                | 拒绝加载或回退最严默认                     |
| Bash 解析失败  | 非法语法、不完整的引号              | <b>deny</b> 或 <b>ask</b> ，不允许执行 |
| 分类器 XML 损坏 | 缺字段、非法 verdict           | <b>ask</b> 或 <b>deny</b> (7.4)  |
| 沙箱启动失败     | bwrap 参数不兼容              | 不裸执行全权限（理想）                     |
| 路径规范化异常    | <code>realpath</code> 失败 | 禁止写入                            |
| 模式未知/枚举越界  | 配置拼写错误                   | 回退 <b>Default</b> 或 <b>Plan</b> |

## 说明性源码：统一「安全降级」

```
type SafeOutcome = { kind: "allow" } | { kind: "ask"; reason: string } | { kind: "deny"; reason: string }

function failClosed(reason: string): SafeOutcome {
  return { kind: "deny", reason };
}

function ambiguousClassifierOutput(raw: string): SafeOutcome {
  // 解析失败 → 绝不默认 allow
  return { kind: "ask", reason: `classifier_malformed:${hash(raw)}` };
}

function bashParseError(detail: string): SafeOutcome {
  return failClosed(`bash_parse_error:${detail}`);
}
```

## 「Deny 无覆盖」= 策略层 Fail-closed

当 **deny** 与后续 **allow** 冲突时，以 **deny 首次匹配** 为准 (7.6)。这等价于：



黑名单的语义强度高于白名单 —— 在高后果域，这是 fail-closed 的自然实现。

## dontAsk 与 Fail-closed 的一致性

dontAsk：仅预批准执行，未匹配 → 拒绝。这不是「openness」，而是 清单驱动的 closed world：

| 事件     | 行为                                   |
|--------|--------------------------------------|
| 命令不在清单 | 拒绝（fail-closed）                      |
| 清单含歧义项 | 应在 code review 拒绝合并（组织层 fail-closed） |

## 与 Prompt Fatigue 的张力

用户讨厌反复点「允许」，但 fail-closed 要求 不能 因疲劳改成默认 allow。正确缓解是：

- allowlist 常用安全命令（7.10）
- 更细的 deny 减少误 ask
- 模式选择（Default vs acceptEdits）减少无关弹窗

错误缓解：「关闭权限」——fail-open。

## 审计日志建议字段

| 字段                  | 用途                                               |
|---------------------|--------------------------------------------------|
| event               | permission_denied / sandbox_failed / parse_error |
| step                | 七步管道序号                                           |
| tool                | bash / edit / ...                                |
| reason_code         | 稳定枚举，便于 SIEM                                     |
| hash_redacted_input | 隐私友好指纹                                           |
| mode                | Default / ...                                    |

## 合规叙事模板（对内）

我们采用 fail-closed：当权限子系统无法证明某动作安全时，不会默认执行。  
这会导致偶尔的自动化失败，需要修正规则或预批准清单——这是预期成本。

## 反模式清单

| 反模式                        | 后果          |
|----------------------------|-------------|
| 解析失败 → allow               | 直接打开攻击面     |
| 日志仅 debug 级别               | 事故后无迹可循     |
| 用 bypass 解决 fail-closed 噪声 | 饮鸩止渴        |
| CI 失败就改宽 dontAsk           | 流水线变 RCE 入口 |

## 小结

- Fail-closed 是 AI 代码助手权限设计的道德与工程底线。
- 解析失败、沙箱失败、分类器损坏 都应导向 deny/ask，而非 allow。
- 与 deny 优先、首次匹配、dontAsk 清单世界 同一哲学。

## 自测

1. 若企业 KPI 推动「降低弹窗率」，如何在不违背 fail-closed 前提下优化？
2. fail-closed 是否意味着「永远 deny」？与 ask 的关系？
3. 举一个你所在技术栈中「解析失败」的真实案例（如 JSON/YAML）。

## 相关章节

- 七步管道：[7.6](#)
- Auto XML：[7.4](#)
- 企业实践：[7.10](#)

## 与 GDPR / 等保叙述的衔接（高层）

审计人员常问：「系统故障时会不会误执行？」 Fail-closed 的回答模板：

在策略或解析子系统故障时，本助手默认不执行高后果操作；必要时降级为人工确认。默认不允许执行优于错误执行。

这与数据处理「最小必要原则」叙事一致：无把握则不处理。

## 灰度发布中的 Fail-closed

新规则上线建议 灰度：

阶段1：仅日志记录「would\_deny」不真拒  
阶段2：对 10% 会话真拒  
阶段3：全量

若阶段1发现误杀率过高，修规则而不是跳至 allow。灰度是观测手段，不是放宽安全理由。

上一篇：[7.8 沙箱](#) · 下一篇：[7.10 实践](#)

## 7.10 实践：企业级权限配置与 Prompt Fatigue 缓解

本篇定位：把前几节的理论落到可复制的配置策略上：规则顺序 deny→ask→allow、allowlist、CI dontAsk、沙箱基线、审计。目标是在不牺牲 fail-closed 的前提下，让团队「少点鼠标」。

### 学习目标

完成本节学习后，你应该能够：

- 1. 起草 一份团队级 deny 基线：密钥路径、shell 配置、父目录写入、 curl|bash 模板。
- 2. 设计 ask 规则与 沙箱例外 的组合，覆盖「想试但怕」的场景。

3. 编写 allowlist 缓解 Prompt fatigue：精确前缀、`&&` 全链评估意识。
4. 落地 CI：`dontAsk` + 窄预批准 + 网络白名单/缓存。
5. 组织 规则代码评审：谁有权改 `deny`、如何版本化、如何回滚。
6. 解释 为何 首次匹配 要求把最严 `deny` 放最前。

## 生活类比：公司采购审批流

- `deny`：法律部黑名单供应商——一票否决。
- `ask`：超过 5 万要 VP 批。
- `allow`：办公文具包月额度内自动过。
- `allowlist`：常用快递账号，不用每次填表——但不能把「任意收款人」加进去。

## 核心原则表

| 原则                                                        | 操作要点                                                                   |
|-----------------------------------------------------------|------------------------------------------------------------------------|
| <code>deny</code> → <code>ask</code> → <code>allow</code> | 文件内严格排序；审查时先看 <code>deny</code>                                        |
| 首次匹配生效                                                    | 更具体的 <code>deny</code> 排在泛化 <code>allow</code> 之前                      |
| 写入边界                                                      | 任何规则不得默许写父目录                                                           |
| 黑名单                                                       | <code>curl</code> / <code>wget</code> 保持禁或强 <code>ask</code> ；与供应链政策一致 |
| <code>fail-closed</code>                                  | 解析失败 → 不执行                                                             |

## Mermaid：企业规则分层

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid：Prompt Fatigue 缓解闭环

[Mermaid 图表 — 请在在线版本中查看]

## 说明性规则文件（教学 YAML）

```
# 示意：规则顺序 deny → ask → allow（首次匹配）
version: 1
rules:
  - name: block-remote-download
    action: deny
    tool: bash
    pattern: "(curl|wget)\\b"
    note: "默认禁 curl/wget；用私有 registry 与缓存"

  - name: block-parent-write
    action: deny
    tool: edit
    path_regex: '^\\.\\.\\.\\./'
    note: "禁止写父目录"

  - name: block-dot-git-direct
    action: deny
    tool: edit
    path_regex: '^(^|/\\.\\.\\.\\.git(/|$))'
    note: "安全护栏与专用工具处理"

  - name: ask-install-scripts
    action: ask
    tool: bash
    pattern: "(npm|pnpm|yarn)\\s+(i|install)\\b"

  - name: allow-status-test
    action: allow
    tool: bash
    pattern: "^git\\s+status\\b"

  - name: allow-pnpm-test
    action: allow
    tool: bash
    pattern: "^pnpm\\s+-s\\s+test\\b"
```

真实产品字段名以官方为准；此处强调 **顺序** 与 **语义**。

## CI 配置清单 (dontAsk)

| 检查项                     | 说明                                 |
|-------------------------|------------------------------------|
| 预批准字符串                  | 与脚本 完全一致 (含 <code>-s</code> )      |
| 禁止 <code>bash -c</code> | 除非拆分进多条明确批准                        |
| 镜像预装依赖                  | 减少安装类 ask/deny 噪声                  |
| egress                  | 仅 registry; 禁止公网 <code>curl</code> |
| 密钥                      | 最小 scope; 不写回仓库                    |

```
# 示意: CI 会话
permission_mode: dont_ask
preapproved_commands:
  - "pnpm -s test"
  - "pnpm -s lint"
  - "git diff --stat"
isolated: true
sandbox_profile: ci-minimal
```

## 沙箱基线建议 (与 7.8 对齐)

| 环境    | 文件系统        | 网络           |
|-------|-------------|--------------|
| 本地开发  | 项目读写 + 缓存目录 | 较松, 可配合用户防火墙 |
| 共享跳板机 | 严格 bind     | 白名单          |
| CI    | 最小 rootfs   | 白名单或离线缓存     |

## 角色与权限 (RBAC 类比)

| 角色           | 能否改组织 deny | 能否加 allow    |
|--------------|------------|--------------|
| 安全团队         | 是          | 评审后          |
| 团队 Tech Lead | 否          | 是 (团队范围)     |
| 个人开发者        | 否          | 仅本地实验, 不进主分支 |

## 审计与复盘

1. 每周导出 Top deny / ask 原因码。
2. 每月复盘：有无误杀；是否应升格为 deny。
3. 事件响应：若发现绕过 AST 的技巧，更新 Bash 策略（7.7）。

## 与六种模式的落地建议

| 场景          | 推荐模式                       |
|-------------|----------------------------|
| 默认开发        | Default                    |
| codemod 周   | acceptEdits + 特性分支         |
| 架构评审会       | Plan                       |
| 熟练工程师 + 细规则 | Auto                       |
| CI          | dontAsk                    |
| 隔离排障        | bypassPermissions（短时 + 审批） |

## 命令黑名单治理

| 动作                     | 负责人                               |
|------------------------|-----------------------------------|
| 申请开放 <code>curl</code> | 安全例外流程 + 替代方案评估                   |
| 引入新下载工具                | <code>aria2c</code> 等同样高危，默认 deny |
| 供应链扫描                  | 与预批准 install 脚本联动                 |

## 源码片段：合并规则时排序

```
function mergeRules(layers: RuleLayer[]): Rule[] {
  const merged = layers.flatMap((l) => l.rules);
  // 示意: 保证 deny 全在 ask 前、ask 全在 allow 前
  return merged.sort((a, b) => rank(a.action) - rank(b.action));
}

function rank(action: "deny" | "ask" | "allow") {
  return action === "deny" ? 0 : action === "ask" ? 1 : 2;
}
```

---

## 小结

---

- 企业级权限 = 组织基线 + 仓库规则 + 沙箱 + CI 模式 + 审计。
  - Prompt fatigue 用 allowlist 与 缓存/镜像 解决，不用 fail-open。
  - deny→ask→allow 与 首次匹配 是维护时的「防呆设计」。
- 

## 交付物模板（可直接复制为工单）

---

```
标题: 新增 allowlist 条目申请
命令: pnpm -s test
理由: 每日 200+ 次 ask, 已审查无 install 生命周期脚本
风险: 低 (只读测试, 无网络 fetch)
评审: @security @tl
回滚: 删除对应 allow 规则提交
```

---

## 自测

---

1. 为什么 allow 规则不能写在 deny 之前? 举反例。
  2. 团队合并规则冲突时, 以「仓库优先」还是「组织优先」更合理?
  3. 如何在度量上区分「烦人的 ask」与「必要的 ask」?
-



## 第 7 篇回顾

| 节    | 关键词                          |
|------|------------------------------|
| 7.1  | rm -rf、为什么需要闸门               |
| 7.2  | 六种模式总表                       |
| 7.3  | Default / acceptEdits / Plan |
| 7.4  | Auto、Sonnet 4.6、XML 两阶段      |
| 7.5  | dontAsk、bypass+容器            |
| 7.6  | 七步管道                         |
| 7.7  | Bash AST                     |
| 7.8  | Seatbelt、bubblewrap          |
| 7.9  | Fail-closed                  |
| 7.10 | 企业实践、allowlist               |

上一篇: [7.9 Fail-closed](#) · 返回: [7.1 导论](#)

## 第8篇：上下文管理

# 第 8 篇：上下文管理 (Context Management)

Claude Code 完全指南 V2 · 从「对话越长越贵」到「预算、压缩与缓存」的系统化理解

## 本篇学习目标

完成本篇后，你应能够：

1. 用**预算思维**描述约 200K token 上下文窗口：它不是「无限记事本」，而是**可耗尽、可计价、可优化的资源**。
2. **区分** 三层压缩 (Tier1 微压缩 → Tier2 自动压缩 → Tier3 完全压缩) 的触发条件、是否调用模型、以及失败熔断策略。
3. **解释** 缓存感知压缩 (如 `cache_edits`) 如何在「剪掉臃肿工具输出」的同时，尽量**不破坏可缓存前缀**。
4. 对照 API 层 **compaction** (如 `compact-2026-01-12` 头) 与交互层 `/compact` 命令：谁在服务端做摘要、谁给你「焦点提示」的控制权。
5. **估算** 长上下文请求的**相对成本** (例如 30K vs 150K 量级差异)，并在 **~60% 使用量** 时主动介入，而不是等到撞墙。

## 8.1 为什么「上下文管理」是 Claude Code 的必修课

### 生活类比：行李箱与登机行李额

把一次长会话想象成国际航班：

- 200K token 像是航空公司给你的**总行李额度** (托运+随身有上限)。
- 工具返回的大段日志 像是你在免税店不断往里塞的「纪念品」——很快箱子就满了。
- 压缩 (compaction) 不是把记忆删掉，而是把「占体积但不影响决策」的东西**折叠成摘要**或**丢掉过期副本**，让你还能继续装下一段旅程必需品。

如果你从不整理行李，最后要么**超重加钱**（更多输入 token 成本），要么**塞不进去**（触发更激进的压缩甚至丢信息）。

核心论断：上下文 = 工作记忆 + 证据链 + 系统拼装

在 Claude Code 中，上下文通常同时承载：

| 成分      | 典型内容                                             | 对窗口的影响      |
|---------|--------------------------------------------------|-------------|
| 系统与规范   | 身份、工具策略、项目规则                                     | 相对稳定，利于缓存前缀 |
| 用户与助手消息 | 需求、方案、追问                                         | 持续增长        |
| 工具 I/O  | <code>read_file</code> 、 <code>grep</code> 、测试输出 | 极易膨胀        |
| 隐式状态    | 当前任务焦点、未决错误                                      | 往往散落在消息里    |

因此，上下文管理的目标不是「越短越好」，而是：**在可接受的信息损失下，把窗口留给高价值 token。**

200K 窗口：预算表（教学用示意）

下列数字用于建立直觉；实际计费以官方定价与计量为准。

| 概念   | 说明                                                      |
|------|---------------------------------------------------------|
| 窗口上限 | 约 200K token 量级的可用上下文（产品/模型代际可能调整）                      |
| 软预警  | 建议在总占用约 60% 时开始整理（手动 <code>/compact</code> 、减少大输出、拆分任务） |
| 硬压力  | 更高占用会提高触发 Tier2 自动压缩 的概率                                |
| 成本直觉 | 同模型下，输入 token 越多，单次请求通常越贵（见第 9 节定量对照）                   |

Mermaid：上下文在会话中的「流入—膨胀—压缩」循环

[Mermaid 图表 — 请在在线版本中查看]

源码片段（概念）：把「窗口占用」当成可观测指标

下面用伪代码表达「你应该在脑子里有的仪表板」（非绑定某一发行版的真实字段名）：

```
type ContextBudget = {
  maxTokens: 200_000;
  usedTokens: number;
  toolResultsBytes: number;
  cachePrefixStable: boolean;
};

function recommendAction(b: ContextBudget): string {
  const ratio = b.usedTokens / b.maxTokens;
  if (ratio >= 0.87) return "预计触发 Tier2: 准备接受自动摘要或手动 /compact";
  if (ratio >= 0.60) return "建议主动整理: 删大输出、拆分任务、焦点压缩";
  return "正常推进";
}
```

本篇路线图（10 节）

| 节    | 文件                                  | 主题                             |
|------|-------------------------------------|--------------------------------|
| 8.1  | <code>index.md</code>               | 总览与预算思维                        |
| 8.2  | <code>02-three-tiers.md</code>      | 三层压缩总览                         |
| 8.3  | <code>03-micro-compaction.md</code> | Tier1 微压缩                      |
| 8.4  | <code>04-auto-compaction.md</code>  | Tier2 自动压缩与熔断                  |
| 8.5  | <code>05-full-compaction.md</code>  | Tier3 完全压缩与九节摘要                |
| 8.6  | <code>06-cache-aware.md</code>      | 缓存感知与 <code>cache_edits</code> |
| 8.7  | <code>07-api-compaction.md</code>   | API 头与模型支持                     |
| 8.8  | <code>08-manual-compact.md</code>   | <code>/compact</code> 与焦点提示    |
| 8.9  | <code>09-cost-analysis.md</code>    | 成本与窗口经济学                       |
| 8.10 | <code>10-best-practices.md</code>   | 最佳实践清单                         |

与相邻篇章的关系

- 第 5 篇（提示词工程）：静态/动态边界决定缓存前缀是否稳定；压缩策略必须尊重该边界。
- 第 9 篇（记忆系统）：记忆注入会增加上下文消耗；压缩后仍要靠项目记忆找回长期约束。

### 常见误解澄清

| 误解           | 更准确的表述                                          |
|--------------|-------------------------------------------------|
| 「压缩 = 失忆」    | 压缩是 <b>信息折叠</b> ；Tier 越高，折叠越激进，但仍尽量保留任务连续性的骨架。  |
| 「只要窗口够大就不用管」 | 窗口大 ≠ 成本低； <b>长输入</b> 仍可能显著增加费用。                |
| 「手动压缩一定比自动好」 | 手动给你 <b>焦点与节奏</b> ；自动在 <b>阈值</b> 下更省心力。二者应组合使用。 |

### 练习（自检）

1. 用你自己的项目举一个「工具输出膨胀」的例子，并说明它主要伤害**延迟**、**费用**还是**可读性**。
2. 画一张草图：你的对话里哪些块应该尽量稳定以命中缓存，哪些块必然每轮变化。

### 小结

上下文管理的核心不是背诵阈值数字，而是建立三件事：**预算意识**（200K 不是无限）、**分层策略**（微/自动/完全）、**缓存纪律**（剪枝时尽量不动前缀）。后续各节把这些原则落到机制与操作上。

### 延伸阅读（概念索引）

- Tier1: **无模型参与**的快速减负。
- Tier2: **87% 阈值触发** + **连续 3 次失败熔断**。
- Tier3: **九节摘要** + chain-of-thought 后剥离。
- 手动: `/compact` + **焦点提示**。
- API: `compact-2026-01-12` 与 **Opus 4.6 / Sonnet 4.6** 支持面。

（以上产品细节以你所用客户端/服务端版本说明为准；本篇以教学模型解释机制。）

### 术语表（本篇）

| 术语           | 含义                       |
|--------------|--------------------------|
| Compaction   | 压缩/折叠上下文以降低占用或成本         |
| Tier         | 压缩分级策略（激进程度与模型参与度不同）     |
| Cache prefix | 可缓存的提示前缀；破坏它可能让「便宜的前缀」失效 |
| Focus hint   | 手动压缩时提示模型应优先保留什么         |

## 附录：Mermaid 语法自检清单

本篇所有 Mermaid 图遵循：

- 使用 `flowchart` / `graph` 关键字与明确方向（如 `LR`、`TB`）。
- 节点文本避免未转义的引号嵌套混乱；必要时用 `["节点文本"]` 形式。
- 不在节点标签中使用会破坏解析的 `<>` 未闭合片段。

下一节：`02-three-tiers.md` —— 把三层压缩放进一张「决策树」，你会第一次完整看到：什么情况下系统会「悄悄整理」，什么时候会「大刀阔斧」。

## 8.1 补充：把「上下文」拆成可管理的块

### 工具输出为何是头号膨胀源

在一次典型重构会话中，模型可能多次：

- 读取大文件全文；
- 运行测试并捕获完整 `stdout/stderr`；
- 对目录做递归列举。

这些输出对当时一步推理可能必要，但对十步之后往往只剩一句「当时测试失败原因是 X」有价值。Tier1 的目标就是：在不调用模型的情况下，先把这种「过期证据」移出前排。

### Mermaid：上下文块的「保质期」

[Mermaid 图表 — 请在在线版本中查看]

表：各块在压缩中的典型命运

| 块类型          | Tier1 | Tier2    | Tier3     |
|--------------|-------|----------|-----------|
| 最近用户消息       | 保留    | 保留/压缩视策略 | 进入摘要骨架    |
| 旧工具输出（非最近 5） | 删除/截断 | 强摘要      | 极大折叠      |
| 系统提示稳定前缀     | 尽量不碰  | 尽量不碰     | 尽量不碰      |
| CoT 草稿       | 可能保留  | 可能被摘要    | Tier3 后剥离 |

伪代码：会话循环中的「预算守门人」

```
async function onTurnEnd(ctx: Session) {
  tier1MicroCompaction(ctx); // 无模型：清旧工具结果，保留最近 5

  if (ctx.fillRatio >= 0.87) {
    const ok = await tier2AutoCompaction(ctx); // 服务端策略
    if (!ok) ctx.autoFailStreak += 1;
    else ctx.autoFailStreak = 0;

    if (ctx.autoFailStreak >= 3) {
      pauseAggressiveAuto("熔断：连续 3 次自动压缩失败");
    }
  }

  if (ctx.mustRecoverFromPressure) {
    tier3FullCompaction(ctx); // 九节摘要 + 剥离 CoT
  }
}
```

与「提示词缓存」的交界

即使你从不手动压缩，缓存命中仍可能因为以下行为变差：

- 在系统前缀后频繁插入大块动态内容；
- 让工具输出穿插在「希望稳定」的提示结构之前（取决于具体拼装顺序）；
- 同一轮内反复改动同一段前缀文本。

第 8.6 节会专门讲 `cache_edits` 思路：手术刀式删除工具结果，而不是「重写整段对话」。

实操清单（现在就能用）

| 时机    | 动作                                | 目的              |
|-------|-----------------------------------|-----------------|
| 任务切换  | 新开线程或手动 <code>/compact</code> 带焦点 | 减少串台与旧日志干扰      |
| 大文件阅读 | 让模型分段读或指定行范围                      | 控制单次工具输出        |
| CI 日志 | 先本地过滤再粘贴关键段                       | 避免万行日志进上下文      |
| 长讨论   | 60% 占用时整理                         | 降低 Tier2/3 触发概率 |

FAQ

- Q：200K 是「硬上限」还是「建议值」？

A：教学上把它当硬预算理解最安全；触及上限会引发更强压缩或失败模式，具体表现随版本变化。
- Q：我为什么要在 60% 就行动？

A：留出余量给下一轮工具输出突刺；否则你会频繁触发自动策略，摘要节奏不再由你掌控。
- Q：压缩会影响代码修改正确性吗？

A：可能影响细节回忆（例如很久以前贴过的栈追踪）。应对办法是：把关键约束写入 [CLAUDE.md](#) 或仓库文档（见第 9 篇）。

本篇自测题（简答）

1. 解释 Tier1 为何强调「无模型参与」：它省下了什么成本？
2. Tier2 的「熔断」保护的是哪一类失败场景？
3. 为什么说「缓存感知」是压缩系统的工程难点之一？

8.2 三层压缩总览：Tier1 → Tier2 → Tier3

把「整理对话」当成一条有梯度的逃生通道：先扫地，再收纳，最后搬家式打包。

本节学习目标

1. 口述 三层压缩的触发意图：从「低成本减负」到「服务端自动摘要」再到「最后手段结构化折叠」。



- 2. 对比 各层是否调用模型、对**信息保真度**的影响、以及与**缓存前缀**的关系。
- 3. 画出 决策流：占用上升时系统更可能走哪条路径，**熔断**在何处介入。
- 4. 解释 Tier3 的 **九节摘要** 在工程上解决什么问题：让模型在极端压力下仍保留任务连续性骨架。
- 5. 联系 手动 `/compact` 与 API compaction：它们分别站在**交互层**与**传输层**的什么位置。

## 生活类比：办公室整理的三档力度

- **Tier1（微压缩）**：同事下班前把桌面上上周的打印稿收进抽屉，只留下最近五份待办——**不需要**请外部顾问（模型）来评估哪些该扔。
- **Tier2（自动压缩）**：行政发现工位堆到 **87%** 容积率，启动标准流程：统一扫描、归档、贴标签——由**服务端策略**执行，强调自动化与一致性。
- **Tier3（完全压缩）**：办公室要搬迁，只能带走集装箱有限体积：必须把资料**按九大类装箱单**重排，并把临时草稿（类比 chain-of-thought）**剥离**，只留对外可交付结论。

## 核心机制一览表

| 层级    | 名称（教学称呼） | 典型触发            | 模型参与       | 主要动作                     | 备注                   |
|-------|----------|-----------------|------------|--------------------------|----------------------|
| Tier1 | 微压缩      | 持续进行/轻量条件       | 否          | 清理旧工具结果， <b>保留最近 5 个</b> | 成本低、延迟小              |
| Tier2 | 自动压缩     | 约 <b>87%</b> 阈值 | 由策略决定（服务端） | 自动摘要/折叠                  | <b>连续 3 次失败 → 熔断</b> |
| Tier3 | 完全压缩     | 压力极高/最后手段       | 通常涉及更强摘要   | <b>九节摘要 + CoT 后剥离</b>    | 保骨架、丢枝节              |

## Mermaid：三层压缩状态机（概念）

[Mermaid 图表 — 请在在线版本中查看]

## Tier1：微压缩的「边界条件」

Tier1 的关键不是「更聪明」，而是更便宜、更确定：

- 清旧工具结果：旧 `run_terminal_cmd` 输出、`read_file` 大块内容等，往往占据大量 token。
- 保留最近 5 个：保证模型仍能看到刚刚发生的证据链，避免「刚跑完测试但输出被瞬间蒸发」的荒谬体验。

伪代码：Tier1 只做结构化剪枝

```
function tier1MicroCompaction(messages: Message[]) {
  const toolResults = messages.filter(isToolResult);
  if (toolResults.length <= 5) return;

  const victims = toolResults.slice(0, -5); // 旧的
  for (const m of victims) {
    m.content = "[tool output elided by Tier1 micro-compaction]";
    // 或硬删除/移出热上下文，视产品实现而定
  }
}
```

表：Tier1 保留/清理直觉

| 内容         | Tier1 倾向     |
|------------|--------------|
| 最近 5 个工具结果 | 保留           |
| 更早的工具结果    | 清理/占位        |
| 用户自然语言     | 通常不动（除非另有策略） |
| 系统提示       | 不动           |

## Tier2：自动压缩与 87% 阈值

把 87% 理解成「再不放水，下一轮工具输出可能把桶顶翻」的预警线：

- 触发：上下文占用达到阈值附近（教学用 0.87）。
- 执行者：服务端策略（客户端只感受结果：摘要变短、某些细节变糊）。
- 失败与熔断：若自动压缩连续失败 3 次，应停止盲目重试，避免抖动、循环与额外费用；转入更保守路径或提示用户介入（具体 UX 以版本为准）。

## Mermaid: Tier2 请求—响应回路

[Mermaid 图表 — 请在在线版本中查看]

## Tier3: 完全压缩与九节摘要

Tier3 是「最后手段」：当继续堆原文会伤害可用性/成本/稳定性时，必须把对话**重构**为更小但仍可执行的表示。

### 九节摘要（教学版结构）

1. **意图**：用户到底想达成什么。
2. **概念**：领域名词、约束、定义。
3. **文件**：关键路径与模块职责。
4. **错误**：未解决问题与复现信息。
5. **消息**：必须保留的用户原话要点。
6. **任务**：分解后的 TODO 与完成标准。
7. **当前工作**：下一步建议动作。
8. （若你的资料版本为 9 节，可在此扩展「环境/依赖/风险」等固定栏目）
9. **链式思考后剥离**：把中间推理移出持久上下文，只留结论。

注：「九节」是结构化信息架构的教学归纳；具体字段命名以实现为准。

## 片段：九节摘要的 Markdown 骨架

```
## Compaction Summary (Tier3)

### 1. Intent
- ...

### 2. Concepts
- ...

### 3. Files
- path/to/a.ts - ...

### 4. Errors
- ...

### 5. Message Highlights
- ...

### 6. Tasks
- [ ] ...

### 7. Current Focus
- Next: ...

### 8. Environment / Constraints
- ...

### 9. Reasoning Stripped
- (CoT removed; conclusions only below)
- ...
```

## chain-of-thought 后剥离：为什么重要

CoT 对单次推理有帮助，但对长期上下文往往是高噪声：

- Token 占用高；
- 可能包含未验证猜想；
- 不利于人类审阅与 diff。

Tier3 的策略倾向：结论入库，草稿出栈。

## 与其他章节映射

| 主题                                    | 详述位置                              |
|---------------------------------------|-----------------------------------|
| <code>cache_edits</code> 手术刀          | <code>06-cache-aware.md</code>    |
| API 头 <code>compact-2026-01-12</code> | <code>07-api-compaction.md</code> |
| <code>/compact</code> 焦点提示            | <code>08-manual-compact.md</code> |
| 费用与 60% 建议                            | <code>09-cost-analysis.md</code>  |

## 对比表：什么时候你该手动介入？

| 信号        | 建议                                          |
|-----------|---------------------------------------------|
| 占用 ~60%   | 主动整理、拆任务、减少工具大块输出                           |
| 频繁自动摘要    | 用 <code>/compact</code> 给出 <b>焦点</b> ，减少误摘要 |
| 关键证据可能被卷走 | 把结论写入文件或 <code>CLAUDE.md</code> （第 9 篇）     |
| 熔断发生      | 降低并行工具调用；缩小读取范围；新开会话                        |

## 练习

1. 用你自己的一次长调试会话，标注哪些消息应进 Tier1 的「冷区」。
2. 试写一份「九节摘要」草稿，看看能否在 **2K token** 内保留任务连续性。

## 小结

三层压缩不是三个孤立功能，而是一条**梯度**：Tier1 负责「便宜地减掉明显垃圾」，Tier2 在**阈值**附近做服务端自动化，Tier3 用**结构化摘要**在极限条件下保住主线。理解这条梯度，你才能在成本、稳定性与正确性之间做权衡。

## 附录：阈值记忆口诀

- **60%**：你开始管；
- **87%**：系统更可能强力自动压；

- **3 连失败**：自动策略熔断，别再赌运气。

---

## 8.3 Tier1 微压缩：无模型参与的「前台清桌」

---

不等「房间满了」才收拾：每轮对话后悄悄扔掉过期工具小票，只留下最近五张。

---

### 本节学习目标

1. **说明** Tier1 微压缩的设计目标：**低延迟、低成本、确定性**，因此**不调用模型**。
2. **描述** 「清旧工具结果、保留最近 5 个」在用户体验上的意义：既减体积，又不断裂即时证据链。
3. **识别** 哪些内容最适合 Tier1 处理，哪些**绝不能**只靠规则硬删（需要人工或 Tier2/3）。
4. **对照** Tier1 与缓存：Tier1 应尽量走**结构化删除**，避免重写系统前缀（详见 8.6）。
5. **演练** 如何在日常使用中**减少 Tier1 也救不了的膨胀**（例如万行日志、重复读全文）。

---

### 生活类比：咖啡店只保留最近五张小票

---

咖啡师台面空间有限：

- 做完的订单里，**最近五杯**的备注还可能被追问（「刚才那杯少冰对吗？」）。
- 更早的小票可以收进垃圾桶，只保留**总额统计**或**是否退款**这类结构化记录——而且**不需要**请店长（模型）来读每张票决定扔不扔，店员按规则做就行。

Tier1 就是这名店员：**按规则清桌**，不叫店长开会。

---

## Tier1 做什么 / 不做什么

| 维度          | Tier1 典型行为        |
|-------------|-------------------|
| 旧工具输出       | 删除、截断或替换为占位符      |
| 最近工具输出      | 保留最近 5 个（教学约定）    |
| 用户文本消息      | 默认不处理（避免误删需求）     |
| 助手最终答复      | 默认不处理（避免误删结论）     |
| 需要「理解语义」的合并 | 不做（交给更高 Tier 或人类） |

## Mermaid：Tier1 在对话回合中的插入点

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：按「工具结果序号」滑动窗口

教学用 TypeScript 伪代码，展示「最近 5 个」如何实现为滑动窗口：

```
type ToolMessage = { role: "tool"; id: string; content: string };

function retainLatestToolResults(msgs: ToolMessage[], keep = 5): ToolMessage[] {
  const idxs = msgs.map((m, i) => ({ m, i })).filter((x) => x.m.role === "tool");

  if (idxs.length <= keep) return msgs;

  const dropCount = idxs.length - keep;
  const dropSet = new Set(idxs.slice(0, dropCount).map((x) => x.i));

  return msgs.map((m, i) =>
    dropSet.has(i)
      ? { ...m, content: "[elided: Tier1 micro-compaction]" }
      : m
  );
}
```

### 变体：按「字节预算」而非条数

当单个工具输出极大时，「保留 5 条」仍可能爆窗口；工程上常见补强是：

```
function budgetAwareElide(tool: ToolMessage, maxChars: number) {
  if (tool.content.length <= maxChars) return tool;
  return {
    ...tool,
    content:
      tool.content.slice(0, maxChars) +
      `\\n... [truncated ${tool.content.length - maxChars} chars]`,
  };
}
```

表：典型工具输出在 Tier1 下的命运

| 工具类型                       | 风险  | Tier1 策略倾向           |
|----------------------------|-----|----------------------|
| <code>read_file</code> 大文件 | 极高  | 旧次优先 elide           |
| 测试 stdout                  | 高   | 保留最近失败日志更久（若实现支持优先级） |
| <code>grep</code> 结果       | 中-高 | 大量匹配时优先截断            |
| <code>ls -R</code>         | 高   | 强烈建议用户避免；Tier1 只能善后  |

为什么 Tier1 不该调用模型

| 若调用模型           | 结果           |
|-----------------|--------------|
| 每次清桌都推理         | 成本上升、延迟上升    |
| 需要额外上下文来理解「重要性」 | 可能反向增加 token |
| 规则可判定的事         | 应用确定性算法更可靠   |

Tier1 的哲学：把 80% 体积垃圾用 1% 成本清掉。

Mermaid：Tier1 与更高 Tier 的职责切分

[Mermaid 图表 — 请在在线版本中查看]



## 实操建议：让 Tier1 「少加班」

| 做法                          | 说明              |
|-----------------------------|-----------------|
| 分段 <code>read_file</code>   | 只读必要行范围         |
| 日志先 <code>tail / rg</code>  | 不要把整个 CI 输出喂给模型 |
| 大 JSON 先 <code>jq</code> 过滤 | 保留错误字段即可        |
| 同类工具少并行爆量                   | 避免同一轮塞回 10 份大结果 |

## 边界案例（教学）

### 案例 A：用户要求「对比三次 `read_file`」

若 Tier1 已 elide 其中两次，模型可能无法对比。对策：

- 把对比结论落地成文件；
- 或在对比完成前避免进入需要 Tier1 大幅清理的阶段。

### 案例 B：安全相关的完整审计日志

规则删除可能丢掉合规需要的原貌。对策：

- 日志保留在仓库外文件；
- 上下文只保留摘要 + 指针（路径、命令、时间）。

## 与第 8.6 节的关系（预告）

Tier1 若实现不谨慎，可能通过「整段重写消息数组」破坏缓存前缀；更理想的是 `cache_edits`：像手术刀一样只改工具节点文本。你现在只需记住：**结构化、局部、可审计**。

## FAQ

**Q：为什么是「5」而不是 3 或 10？**

**A：**教学文档采用你提供的产品化描述；本质是滑动窗口宽度。过窄易断链，过宽减负不足。

**Q：Tier1 会删我的代码吗？**

A：Tier1 针对的是消息里的工具输出副本，不是仓库文件系统（除非另有工具写入）。

练习

- 1. 打开你最近一段长会话，数一下工具消息条数，标出若只保留 5 条会丢失什么信息。
- 2. 写一条「用户指令模板」，要求模型读文件时始终带行号范围。

小结

Tier1 微压缩是上下文治理的第一道闸：**无模型、偏规则、对工具输出最敏感**。它用「最近 5 个」在**减负与连贯**之间取平衡。你要做的是：少用超大工具输出挑战这道闸。

自检答案提示

- 「无模型」省下的是：**推理 token、往返延迟、非确定性**。
- 「保留 5 个」保住的是：**短窗口内的因果链**（刚运行的命令、刚读的文件）。

扩展阅读：消息图模型

把会话看成有向图有助于理解 Tier1：

| 节点        | 边                         |
|-----------|---------------------------|
| user      | → assistant               |
| assistant | → tool_call               |
| tool      | → tool_result → assistant |

Tier1 主要裁剪 **tool\_result** 叶子 上的冗余，而不破坏「最近推理链」的主干。

Mermaid：工具结果滑动窗口可视化

[Mermaid 图表 — 请在在线版本中查看]

## 场景推演：五次 `read_file` 后第六次对比

- 1. 用户要求对比 6 个配置文件。
- 2. 若 Tier1 只保留 5 个工具结果，最早那份可能变占位。
- 3. 对策：对比前声明「把结论写入 `docs/diff-summary.md`」，或分批对比 (3+3)。

## 对照表：Tier1 与「人类手动删消息」

| 维度   | Tier1 | 人类删消息     |
|------|-------|-----------|
| 速度   | 毫秒级   | 慢         |
| 一致性  | 高     | 因人而异      |
| 语义理解 | 无     | 有         |
| 误伤风险 | 可预测   | 可能删错 user |

## 术语小抄

| 英文               | 中文      |
|------------------|---------|
| micro-compaction | 微压缩     |
| elide            | 省略/占位替换 |
| sliding window   | 滑动窗口    |

## 8.4 Tier2 自动压缩：87% 阈值与服务端策略

水位涨到警戒线，水库自动泄洪；若闸门卡死三次，别再硬开——先换策略。

### 本节学习目标

- 1. 解释 87% 阈值在工程上的意义：为「下一轮工具输出突刺」预留安全余量。

2. **描述** Tier2 由**服务端策略**驱动的含义：客户端体验为「上下文突然变短/更摘要化」。
3. **说明** **连续 3 次失败熔断**：防止重试风暴、成本螺旋与不可用状态抖动。
4. **对比** Tier2 与 Tier1：何时必须引入**语义级摘要**（通常更贵、更慢）。
5. **列举** 用户侧可执行的降压手段，降低 Tier2 触发频率与失败率。

## 生活类比：小区二次供水与限流阀

住户用水高峰时，水箱液位上升到 87%：

- 物业系统自动开启**排放与分流**（类比服务端摘要），避免漫灌。
- 若阀门故障连续三次，系统应**熔断报警**，而不是无限电动重试烧电机——否则整个泵房都可能挂。

## 触发与执行：一张总表

| 项目   | 教学描述                                             |
|------|--------------------------------------------------|
| 触发阈值 | 上下文占用约 87% ( <code>fillRatio &gt;= 0.87</code> ) |
| 执行主体 | <b>服务端策略</b> （非纯本地规则）                            |
| 成功效果 | 体积下降、对话可继续                                       |
| 失败计数 | 每次失败 <code>failStreak++</code>                   |
| 熔断条件 | <b>连续 3 次失败</b> → 暂停激进自动策略（概念）                   |

## Mermaid：占用率与 Tier2 触发

[Mermaid 图表 — 请在在线版本中查看]

## 服务端策略可能做什么（概念清单）

具体算法随版本迭代；此处给学习者「预期集合」。

| 策略            | 目的                    |
|---------------|-----------------------|
| 合并相邻助手说明性段落   | 减少重复修辞                |
| 对旧轮次用户消息做要点抽取 | 保留意图，删口语              |
| 对工具输出做模板化摘要   | 保留 exit code、关键行      |
| 丢弃极低价值中间态     | 例如重复 <code>pwd</code> |

## 源码片段：失败计数与熔断（伪代码）

```
type AutoCompactionState = {
  failStreak: number;
  lastError?: string;
};

async function maybeAutoCompact(
  ctx: Context,
  state: AutoCompactionState
): Promise<boolean> {
  if (ctx.fillRatio < 0.87) return true;

  if (state.failStreak >= 3) {
    console.warn("circuit open: auto-compaction paused");
    return false;
  }

  try {
    await serverSideCompaction(ctx);
    state.failStreak = 0;
    return true;
  } catch (e) {
    state.failStreak += 1;
    state.lastError = String(e);
    return false;
  }
}
```

## Mermaid：序列图——三次失败路径

[Mermaid 图表 — 请在在线版本中查看]

## 表：Tier2 成功 vs 失败的典型原因

| 成功因素       | 失败诱因            |
|------------|-----------------|
| 工具输出可被模板化  | 输出高度非结构化        |
| 用户允许丢失细节   | 用户强制「全文保留」类需求冲突 |
| 上游 API 健康  | 限流/超时           |
| 上下文本身可分段摘要 | 上下文含大量互指代词缺主语   |

## 用户侧降压清单（减少 Tier2 触发）

| 动作                                | 机制                |
|-----------------------------------|-------------------|
| 60% 起手动 <code>/compact</code> 带焦点 | 主动摘要，降低冲到 87% 的概率 |
| 拆分 Epic 任务                        | 单会话更短             |
| 减少「整仓扫描」式工具调用                     | 控制回注体积            |
| 把中间结论写入文件                         | 上下文只保留指针          |

## 与 API 层 compaction 的交界

Tier2 常与 API 头（如 `compact-2026-01-12`）协同：客户端声明能力，服务端选择摘要管线。详见 `07-api-compaction.md`。

## 深度阅读：为什么「熔断」是成本工程

若无熔断：

- 每次失败可能仍附带大上下文上传；
- 重试抖动造成 N 倍费用；
- 用户看到「卡住」会更频繁强制中断，产生更多垃圾状态。

熔断把系统从振荡推到可诊断的稳定失败。

## 表：熔断后建议操作

| 步骤 | 说明                                                       |
|----|----------------------------------------------------------|
| 1  | 保存当前任务焦点到 <code>NOTES.md</code> 或 <code>CLAUDE.md</code> |
| 2  | 新开会话，携带焦点摘要                                              |
| 3  | 缩小工具范围，先复现再扩展                                            |
| 4  | 检查网络/配额/模型可用性                                            |

## 练习

1. 假设 `max=200000`，计算 87% 对应的 token 数。
2. 写一段「会话开场白」，在新线程里恢复被 Tier2 摘要掉的大型重构上下文。

## FAQ

- Q：失败三次后是不是永远不能自动压了？

A：教学上描述为熔断暂停；真实产品可能允许冷却恢复或手动复位。以版本说明为准。
- Q：87% 是精确值还是区间？

A：理解为策略阈值；实现可能有滞回，避免在边界来回抖动。

## 小结

Tier2 自动压缩是高压下的泄洪阀：在 87% 附近启动，由服务端做语义级处理，并用连续三次失败熔断避免系统与用户双输。你的最佳配合是在 60% 前主动整理，让 Tier2 更少「不得不出手」。

## 附录：阈值与行数对照（示意）

| maxTokens | 87%     | 60%     |
|-----------|---------|---------|
| 200,000   | 174,000 | 120,000 |

用于建立数量级直觉，非承诺 SLA。

## 扩展：滞回阈值（概念）

工程上为避免在 86%~88% 之间反复触发，可能使用：

触发阈值 87%  
解除阈值 82%（示意）

你只需知道：存在滞回时，体感上「有时刚过线没立刻压」是正常的。

## Mermaid：失败类型分类

[Mermaid 图表 — 请在在线版本中查看]

## 表：日志字段建议（自建观测时）

| 字段                              | 用途                                   |
|---------------------------------|--------------------------------------|
| <code>fill_ratio_before</code>  | 触发原因分析                               |
| <code>compact_latency_ms</code> | SLA                                  |
| <code>fail_streak</code>        | 熔断诊断                                 |
| <code>profile</code>            | 与 <code>compact-2026-01-12</code> 对齐 |

## 与组织治理：何时升级工单

| 条件      | 建议          |
|---------|-------------|
| 单项目持续熔断 | 检查工具脚本是否灌日志 |
| 全局熔断率上升 | 平台配额/区域故障   |
| 仅某模型    | 模型路由配置      |



## 练习补充

- 设计一个「熔断后」用户可见提示文案（中文，不超过 80 字）。

## 8.5 Tier3 完全压缩：九节摘要与 CoT 剥离

搬家时只能带走集装箱：把散件按装箱单归类，草稿纸烧掉，只留签字版结论。

### 本节学习目标

- 定义 Tier3 「完全压缩」：最后手段，在极端压力下重构上下文为结构化骨架。
- 背诵并理解 九节摘要 的信息架构：意图、概念、文件、错误、消息、任务、当前工作 + 扩展节 + CoT 剥离。
- 解释 chain-of-thought 后剥离：降低噪声与 token，保留可执行结论。
- 对照 Tier3 与 Tier1/Tier2：何时规则不够、何时自动摘要不够。
- 实操 能根据九节模板手工重建一段「将死会话」的连续性。

### 生活类比：庭审卷宗 vs 律师草稿纸

诉讼结束时：

- 卷宗（Tier3 产物）必须包含：当事人意图、争议焦点、关键证据目录、未决事项、下一步程序——像九节目录。
- 律师的演算草稿（CoT）可以销毁或归档到不允许带入庭审的文件盒：不是「没发生过」，而是不应再占用主卷宗的页数。

## 九节摘要：字段说明表

| 节 | 名称        | 应回答的问题           |
|---|-----------|------------------|
| 1 | 意图        | 用户最终要什么交付物？      |
| 2 | 概念        | 有哪些术语/约束必须统一？    |
| 3 | 文件        | 哪些路径是「主战场」？      |
| 4 | 错误        | 现在卡住的失败是什么？如何复现？ |
| 5 | 消息        | 用户哪些原话不可改写？      |
| 6 | 任务        | TODO 列表与完成定义？    |
| 7 | 当前工作      | 下一步最小动作？         |
| 8 | 环境/依赖（扩展） | 版本、分支、运行命令？      |
| 9 | CoT 剥离    | 中间推理删除后，保留了哪些结论？ |

第 8 节可根据团队模板替换为「风险/回滚/安全」等固定栏目。

## Mermaid：Tier3 的输入—输出变换

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：把对话折叠为九节（伪实现）

```
type Tier3Summary = {
  intent: string;
  concepts: string[];
  files: { path: string; note: string }[];
  errors: { title: string; repro: string }[];
  messageHighlights: string[];
  tasks: { id: string; done: boolean; text: string }[];
  currentFocus: string;
  environment: string;
  strippedCoT: { keptConclusions: string[] };
};

function buildTier3Summary(longThread: string): Tier3Summary {
  // 真实实现：由更强摘要模型或规则流水线完成
  return {
    intent: extractIntent(longThread),
    concepts: extractConcepts(longThread),
    files: extractFiles(longThread),
    errors: extractErrors(longThread),
    messageHighlights: extractUserQuotes(longThread),
    tasks: extractTasks(longThread),
    currentFocus: extractNextStep(longThread),
    environment: extractEnv(longThread),
    strippedCoT: { keptConclusions: extractConclusionsOnly(longThread) },
  };
}
```

## chain-of-thought 剥离：操作守则

| 守则          | 原因            |
|-------------|---------------|
| 结论必须可独立成立   | 否则剥离 CoT 会丢因果 |
| 关键数字/路径写进正文 | 避免「我记得大概是」    |
| 未验证假设不得写死   | 防止错误固化成「事实」   |
| 剥离物可外置到文件   | 需要审计时仍可追溯     |

### 片段：剥离前后对照（示意）

```
# 剥离前（节选）
我先猜是路由问题..... 不对，再看 middleware.....
哦日志里 403 来自 authz..... 所以应该改 policy.....

# 剥离后（应写入第 4/7 节）
错误：403 来自 authz policy X。
下一步：修改 policy X 并补集成测试。
```

### Mermaid：九节之间的依赖关系

[Mermaid 图表 — 请在在线版本中查看]

### Tier3 使用时机（判断表）

| 场景               | 是否倾向 Tier3 |
|------------------|------------|
| Tier2 熔断 + 仍超高占用 | 是          |
| 用户明确要「重置但不断任务」   | 是          |
| 仅需删旧工具输出         | 否（Tier1）   |
| 中等占用、可接受服务端自动处理  | 优先 Tier2   |

### 手工演练：从混乱对话生成九节

假设对话主题是「把 Express 迁移到 Fastify」：

```
### 1. Intent
完成 Express → Fastify 迁移，并通过现有集成测试。

### 2. Concepts
- 中间件签名差异
- 请求生命周期钩子映射

### 3. Files
- `src/server.ts` - 入口替换
- `src/routes/*` - 路由注册方式

### 4. Errors
- 测试 `auth.test.ts` 超时：待复现日志

### 5. Message Highlights
- 用户要求：「保持对外 API 不变」

### 6. Tasks
- [ ] 迁移入口
- [ ] 对齐 middleware
- [ ] 修复 auth 测试

### 7. Current Focus
先定位 `auth.test.ts` 超时是路由还是插件初始化。

### 8. Environment
Node 22, pnpm, 分支 `feat/fastify`.

### 9. Reasoning Stripped
保留结论：超时更可能来自插件 await 链，而非路由层。
```

## 与手动 `/compact` 的关系

Tier3 是系统级「硬重置骨架」；用户级 `/compact` 则可用**焦点提示**引导摘要保留哪些节更重要。见 `08-manual-compact.md`。

## 风险与缓解

| 风险         | 缓解                           |
|------------|------------------------------|
| 九节写错导致后续全偏 | 关键事实附 文件路径 + 命令              |
| 过度剥离丢证据    | 大日志放文件，摘要里留指针                |
| 团队字段不一致    | 固定模板提交到仓库 <code>docs/</code> |

## 练习

---

1. 找一段你自己的长 ChatGPT 线程，手工填九节，看能否在 1500 汉字内说清。
  2. 标出哪些句子必须来自「用户原话」（第 5 节）。
- 

## FAQ

---

Q：九节必须严格九个吗？

A：教学上是信息完备性的 checklist；实现可合并字段，但别丢维度。

Q：Tier3 会不会删掉我还没同意的修改？

A：Tier3 处理的是对话上下文，不是 Git；但模型可能遗忘未落地变更的意图——所以要及时 `commit` 或写 `CLAUDE.md`。

---

## 小结

---

Tier3 完全压缩用九节摘要把「散乱的对话—工具证据」变成可续写的项目状态机；再通过 CoT 剥离把高噪声推理移出热上下文。掌握这套装箱单，你就不怕长会话「物理上还在、逻辑上散了」。

---

## 附录：九节模板（纯 Markdown，可复制）

---

```
## Tier3 / Nine-Section Summary

1. Intent:
2. Concepts:
3. Files:
4. Errors:
5. Message Highlights:
6. Tasks:
7. Current Focus:
8. Environment:
9. Stripped CoT – Kept Conclusions:
```

---

## 8.6 缓存感知压缩：cache\_edits 与「前缀不动」纪律

整理书架时只抽走几本厚书，而不是把整面墙重新粉刷——否则「常读区」的标签全得重贴。

### 本节学习目标

- 1. **解释** 提示词 **prompt caching** 的基本收益：稳定前缀命中 → 更低费用/更快首包（视产品与模型支持而定）。
- 2. **说明** 朴素压缩（整段重写对话）如何**破坏缓存前缀**，从而「省下的上下文」被「上涨的单价」反噬。
- 3. **描述** **cache\_edits** 思路：**手术刀式删除或替换工具结果**，尽量保持前缀字节级稳定。
- 4. **对照** 「删消息」与「改消息」对缓存的不同影响（概念层面）。
- 5. **制定** 团队规范：哪些块必须append-only，哪些块允许被 **cache\_edits** 触碰。

### 生活类比：超市会员价标签

超市对「会员价」贴在**固定展板上**（缓存前缀）：

- 若店员每天把整块展板撕掉重贴（整段重写上下文），系统要重新录入、重新核对——成本高。
- 更好的做法是：只替换**可变价签插槽里的数字**（**局部 edit**），展板底板不动。

**cache\_edits** 就是替换插槽，而不是换整块板。

### 缓存前缀为何脆弱

| 行为              | 对前缀的影响                         |
|-----------------|--------------------------------|
| 在前缀后 append 新消息 | 通常 <b>不破坏</b> 既有前缀匹配（取决于服务商规则） |
| 修改前缀中任意字符       | <b>通常破坏</b>                    |
| 重排早期消息顺序        | <b>高风险</b>                     |
| 把工具输出插到前缀之前     | <b>极高风险</b> （改变前缀边界）           |

精确语义以 Anthropic/OpenAI 等官方缓存文档为准；本节给 Claude Code 学习者的工程直觉。

## Mermaid：朴素压缩 vs cache\_edits

[Mermaid 图表 — 请在在线版本中查看]

### 源码片段： `cache_edits` 伪代码

```
type CacheEdit = {
  messageId: string;
  field: "content";
  oldHash: string;
  newContent: string;
};

function applyCacheAwareCompaction(
  messages: Message[],
  edits: CacheEdit[]
): Message[] {
  const byId = new Map(messages.map((m) => [m.id, m]));

  for (const e of edits) {
    const m = byId.get(e.messageId);
    if (!m) continue;
    if (hash(m.content) !== e.oldHash) {
      // 并发冲突：放弃此 edit 或重算
      continue;
    }
    m.content = e.newContent;
  }

  return messages; // 顺序不变、非目标消息不变
}
```



### 片段：只 elide 工具结果节点

```
{
  "type": "cache_edits",
  "edits": [
    {
      "message_id": "toolmsg_0182",
      "replace_content_with": "[tool output elided; see artifacts/run-182.log]"
    }
  ]
}
```

### 表：可编辑 vs 不可编辑（团队规范示例）

| 消息类型    | 是否建议 cache_edits | 备注       |
|---------|------------------|----------|
| 系统提示静态段 | 否                | 动了就失缓存   |
| 用户原始消息  | 谨慎               | 语义敏感     |
| 工具结果    | 是                | 体积大户     |
| 助手最终答复  | 视情况              | 可能影响连贯索引 |

### Mermaid：上下文拼装顺序与「前缀边界」

[Mermaid 图表 — 请在在线版本中查看]

### 与 Tier1 的协同

Tier1「清旧工具结果」若实现为：

- 好：对 tool 消息内容做原地替换或定点删除；
- 差：把整轮 messages 重新序列化导致无关空格/字段顺序变化。

## 实操：让缓存「有东西可命中」

| 建议         | 解释       |
|------------|----------|
| 少改系统提示模板   | 提高静态段稳定性 |
| 动态内容集中在边界后 | 保护前缀     |
| 大内容走文件指针   | 工具输出变小   |
| 合并无意义的小修小补 | 减少前缀抖动频率 |

## 反模式清单

| 反模式                     | 后果         |
|-------------------------|------------|
| 每次压缩都 pretty-print JSON | 空白变化 → 前缀变 |
| 随机重排消息                  | 哈希变        |
| 把 timestamp 写入静态前缀      | 前缀永不稳定     |
| 同一策略两种字符串拼接顺序           | 难以命中       |

## 练习

- 画你项目里「静态系统提示」与「动态后缀」的分界线。
- 写三条 `cache_edits` 规则，只针对 `grep` 与 `read_file` 工具消息。

## FAQ

- Q：我怎么知道前缀有没有命中？

A：看计费面板/日志中的 cache read/write 指标（以平台为准）。
- Q：cache\_edits 是用户可见 API 吗？

A：可能是内部/底层表示；学习者只需理解局部编辑思想。

## 小结

缓存感知压缩的核心是：能删工具结果，就别重写世界。`cache_edits` 用手术刀维护前缀稳定，让「减负」与「省钱」一致起来，而不是互相打架。

## 附录：与第 5 篇边界的对照

| 概念   | 第 5 篇重点                                     | 本篇重点          |
|------|---------------------------------------------|---------------|
| 动态边界 | <code>SYSTEM_PROMPT_DYNAMIC_BOUNDARY</code> | 压缩不跨越边界胡改     |
| 静态宪法 | 可缓存                                         | Tier1/2/3 都慎动 |

## 扩展：哈希稳定性的最小示例

```
import { createHash } from "node:crypto";

function stableHash(messages: Message[]) {
  const canonical = JSON.stringify(messages, Object.keys(messages[0] ?? {}).sort());
  return createHash("sha256").update(canonical).digest("hex");
}
```

工程上常改用更细粒度的「前缀字节范围哈希」；此处仅说明：序列化细节影响命中。

## Mermaid：edit 只触达子树

[Mermaid 图表 — 请在在线版本中查看]

## 表：服务商缓存语义备忘（学习者）

| 概念           | 记住一句话         |
|--------------|---------------|
| prefix cache | 前缀相同部分可复用计费   |
| TTL          | 缓存可能过期，别当永久磁盘 |
| breakpoints  | 断点位置影响命中      |

以官方文档为准。

## 实战：审查一次 PR 中的「压缩实现」

检查清单：

1. 是否原地修改工具消息而非重建全数组？
2. 是否避免对 system 块 `trim()` ？
3. 是否有单元测试覆盖「edit 后哈希」？

## 练习补充

3. 列举三个会改变 JSON 序列化从而破坏缓存的「无害」代码改动。

## 8.7 API 层 compaction: `compact-2026-01-12` 与模型支持

在 HTTP 头上贴一张「请用新版收纳盒」的标签，服务端才知道该走哪条摘要管线。

## 本节学习目标

1. 说明 API 层 compaction 与客户端 `/compact` 的差异：声明能力/协议版本 vs 交互命令。
2. 识别 头字段 `compact-2026-01-12`（教学命名）所代表的「2026-01-12 版压缩协议/行为」。
3. 列举 Opus 4.6 与 Sonnet 4.6 在自动摘要链路中的支持面（概念：二者均支持该类能力）。
4. 解释「自动摘要」在传输层的价值：用户少操心，但仍受阈值与策略约束。
5. 对照 API compaction 与 Tier2：一个在协议层声明，一个在会话层表现为占用压力反应。

## 生活类比：快递箱上的「易碎/冷链」标签

同样的包裹：

- 贴上 冷链 标签，分拣系统会走冷库线路；
- 没贴标签，可能当普通件处理，夏天就化了。

API 头里的 compaction 标记类似：告诉基础设施用哪套处理规则。

## 概念架构表

| 层级    | 角色   | 典型载体                               |
|-------|------|------------------------------------|
| 交互层   | 用户触发 | <code>/compact</code> + 焦点提示       |
| 会话层   | 占用治理 | Tier1/2/3                          |
| API 层 | 能力协商 | <code>compact-2026-01-12</code> 等头 |

## Mermaid：请求中的 compaction 协商

[Mermaid 图表 — 请在在线版本中查看]

## 头字段示例（教学用伪 HTTP）

```
POST /v1/messages HTTP/1.1
Content-Type: application/json
X-Client: claude-code
X-Compaction-Profile: compact-2026-01-12

{
  "model": "claude-sonnet-4-6",
  "messages": [ ... ]
}
```

真实头名称以实现为准；此处展示「把版本钉在协议上」的意图。

## 模型支持矩阵（教学）

| 模型族        | 支持 API 层自动摘要（概念） |
|------------|------------------|
| Opus 4.6   | 支持               |
| Sonnet 4.6 | 支持               |

说明：「支持」指产品路线中该模型可参与/可受益于对应 compaction 管线，而非保证每一请求都触发摘要。

## 源码片段：客户端如何附加 profile（伪代码）

```
type HttpHeaders = Record<string, string>;

function buildHeaders(session: Session): HttpHeaders {
  return {
    "Content-Type": "application/json",
    "X-Compaction-Profile": "compact-2026-01-12",
    "X-Session-Id": session.id,
  };
}

async function sendModelRequest(body: unknown, session: Session) {
  return await fetch(session.apiBase, {
    method: "POST",
    headers: buildHeaders(session),
    body: JSON.stringify(body),
  });
}
```

## Mermaid：不同 profile 的分支

[Mermaid 图表 — 请在在线版本中查看]

## 自动摘要：用户可感知的信号

| 信号       | 可能含义       |
|----------|------------|
| 对话突然「变短」 | 服务端已折叠早期细节 |
| 旧栈追踪不见   | 被摘要为一句错误描述 |
| 任务仍在     | 九节/要点仍保留   |

## 与 Tier2 阈值的关系

API profile 决定「怎么压」；Tier2 阈值决定「何时压」。二者组合：

何时压：fillRatio >= 0.87（会话层）  
怎么压：compact-2026-01-12（API 层）

## 兼容性注意

| 场景    | 建议                     |
|-------|------------------------|
| 旧客户端  | 可能缺少新 profile → 摘要行为不同 |
| 多模型切换 | 确认目标模型在矩阵内             |
| 自建代理  | 透传 compaction 相关头      |

## 练习

- 解释如果把 compaction 头去掉，Tier2 是否仍可能存在？
- 画一张表对比「只有 Tier2」与「Tier2 + 新 profile」的风险差异。

## FAQ

Q：日期 2026-01-12 是什么？

A：教学用版本钉；代表某次行为变更基线，便于日志与排障。

Q：会改变计费吗？

A：摘要可能改变输入 token 结构，从而影响费用；以平台账单为准。

## 小结

API 层 compaction 用 compact-2026-01-12 这类头把「摘要策略版本」显式化，使 Opus 4.6 / Sonnet 4.6 等模型在服务端可走一致的自动摘要路径。它与 Tier2 何时触发互补，与用户 /compact 主动控制焦点互补。

## 附录：排障 checklist

| 现象        | 检查                   |
|-----------|----------------------|
| 摘要行为与文档不符 | profile 头是否下发        |
| 只有部分会话异常  | 是否模型混用               |
| 费用异常      | 缓存命中 + 摘要前后 token 对比 |

## 扩展：版本钉与可观测性

把 `compact-2026-01-12` 写入客户端版本遥测：

```
{
  "client": "claude-code",
  "version": "x.y.z",
  "compaction_profile": "compact-2026-01-12",
  "model": "claude-opus-4-6"
}
```

排障时对比：升级前后 profile 是否漂移。

## Mermaid：多模型路由

[Mermaid 图表 — 请在在线版本中查看]

## 表：客户端升级检查表

| 检查项   | 通过标准        |
|-------|-------------|
| 头发送   | 抓包可见        |
| 模型白名单 | 仅 4.6 族走新管线 |
| 回滚路径  | 可关闭 profile |



## 与 Sonnet / Opus 的选用提示（概念）

| 倾向     | 说明                      |
|--------|-------------------------|
| Sonnet | 更快扫描/执行，适合高频 compact 辅助 |
| Opus   | 更强推理，适合极难摘要保留因果         |

具体路由以产品为准。

## 练习补充

3. 写一段「发布说明」文案，告知团队新 compaction profile 上线与回滚方式。

## 8.8 手动 `/compact`：节奏、焦点与可控摘要

吸尘器的高级模式不是「更强吸力」，而是「告诉它重点吸沙发角」。

### 本节学习目标

- 使用 `/compact` 在对话中段主动触发压缩，而不是被动等待 Tier2。
- 编写 有效的焦点提示 (focus hint)：告诉模型摘要后必须保留什么、可以牺牲什么。
- 对比 `/compact` 与 API 自动摘要：控制权、可预期性与适用场景。
- 规避 误摘要：把不可丢失的事实绑定到文件路径/测试名/命令。
- 规划 压缩后的下一句用户消息，使新上下文「一键续写」。

### 生活类比：摄影师选片

拍了一整天婚礼：

- 自动选片像 Tier2：算法按规则删重复，但可能删掉你喜欢的抓拍。
- `/compact` + 焦点像你告诉修图师说：「保留新人与父母那组，其他可以狠裁」——结果更贴合意图。

/compact

是什么（概念）

| 维度        | 描述                            |
|-----------|-------------------------------|
| 类型        | 手动命令（聊天内）                     |
| 目的        | 主动整理上下文                       |
| 关键能力      | 支持 焦点提示                       |
| 与 Tier 关系 | 可能调用与 API/Tier2 类似的摘要能力，但由你发起 |

## Mermaid：手动压缩介入点

[Mermaid 图表 — 请在在线版本中查看]

## 焦点提示模板库

### 模板 A：保留错误与复现

```
/compact
焦点：保留当前 TypeError 的完整堆栈与复现命令；
可删：早期探索性 read_file 全文与已解决的 lint 输出。
```

### 模板 B：保留架构决策

```
/compact
焦点：保留我们已确认的模块边界与公开 API 列表；
可删：讨论过程的口语往返与中间假设。
```

### 模板 C：保留未合并 PR 状态

```
/compact
焦点：保留 PR #482 的审查意见摘要与待改文件列表；
可删：与 PR 无关的本地实验输出。
```

## 源码片段：CLI 解析 `/compact`（伪实现）

```
type CompactCommand = {
  kind: "compact";
  focus?: string;
};

function parseSlashCommand(input: string): CompactCommand | null {
  if (!input.startsWith("/compact")) return null;
  const rest = input.replace("/compact", "").trim();
  return { kind: "compact", focus: rest || undefined };
}

async function handleCompact(cmd: CompactCommand, session: Session) {
  await session.compact({
    userFocus: cmd.focus,
    preserveRecentToolResults: 5, // 可能与 Tier1 协同
  });
}
```

## 表：何时用手动 vs 自动

| 场景                | 手动 <code>/compact</code> | 等 Tier2 |
|-------------------|--------------------------|---------|
| 你要赶 deadline，必须可控 | 优先                       | 次选      |
| 占用缓慢上升            | 可选                       | 可等      |
| 刚完成大阶段，准备新阶段      | 强建议                      | 不必等     |
| 你明确知道什么是噪音        | 强建议                      | 风险更高    |

## Mermaid：焦点提示的信息流

[Mermaid 图表 — 请在在线版本中查看]

## 实操步骤（建议流程）

- 保存 当前工作区状态（`git status` 干净或已提交 WIP）。
- 写焦点：3~6 条 bullet，区分「必须保留 / 可删」。

3. 执行 `/compact` 粘贴焦点。
4. 下一条消息用「续写协议」开场：

我们继续上一段任务。以下事实仍以仓库为准：

- 分支：`feature/x`
- 失败测试：`packages/foo/__tests__/bar.test.ts`

请基于压缩后的摘要，从「修复 `bar.test`」继续。

## 与 Tier3 九节的关系

你可以把焦点提示看成用户版的九节权重：强调第 4 节（错误）和第 7 节（当前工作），其余让摘要器自由折叠。

## 反模式

| 反模式                       | 为什么糟          |
|---------------------------|---------------|
| <code>/compact</code> 无焦点 | 可能删掉你心里的关键细节  |
| 焦点写太长                     | 摘要提示本身占 token |
| 焦点互相矛盾                    | 模型无所适从        |
| 压缩后不交代续写协议                | 容易「失忆式乱改」     |

## 练习

1. 给你当前真实任务写一段 50 字内 焦点提示。
2. 模拟压缩后第一条用户消息，包含 分支名 + 一个文件路径。

## FAQ

- Q： `/compact` 会动我的磁盘文件吗？

A：通常动的是会话上下文；文件除非你另有工具操作。
- Q：焦点提示必须用中文吗？

A：用你最会写约束的语言即可；团队统一更重要。

## 小结

`/compact` 把压缩从「系统不得不」变成「我主动要」：配合**焦点提示**，你在 **60%** 占用前后就能把对话整理成下一阶段可读的形态，减少对 Tier2/3 的被动依赖。

## 附录：与 `CLAUDE.md` 联动

压缩会丢细节，项目记忆不会：

| 内容      | 放会话 | 放 <code>CLAUDE.md</code> |
|---------|-----|--------------------------|
| 一次性调试   | 是   | 否                        |
| 长期命令/约定 | 可   | 是                        |
| 团队规范    | 可   | 是                        |

详见第 9 篇。

## 扩展：`/compact` 与其他斜杠命令的协同

| 命令                        | 关系        |
|---------------------------|-----------|
| <code>/compact</code>     | 整理上下文     |
| 新会话                       | 更激进「重置」   |
| <code>CLAUDE.md</code> 编辑 | 长期约束不依赖会话 |

## Mermaid：压缩前后的「首条用户消息」策略

[Mermaid 图表 — 请在在线版本中查看]

### 表：焦点提示的长度建议

| 长度       | 评价         |
|----------|------------|
| < 30 字   | 可能信息不足     |
| 50~200 字 | 甜区         |
| > 500 字  | 可能浪费 token |

### 反例改写练习

反例： `/compact` 别删东西

改写： `/compact` 焦点：保留 X；可删：Y；SoT: path Z

### 与无障碍/国际化

若团队双语：

```
/compact
Focus (EN): Keep failing stack trace for test T.
可删：早期探索性输出。
```

模型通常能处理混合指令，但焦点句建议一种语言主写。

### 练习补充

3. 记录一次你真实使用 `/compact` 前后的「模型行为差异」（主观即可）。

## 8.9 成本分析：窗口长度、单价与 5 倍差距直觉

同一模型下，「多带行李登机」往往要多付钱——上下文越长，单次请求的输入侧成本越可能陡升。

## 本节学习目标

1. **建立** 定量直觉：在可比设定下，**30K token** 请求与 **150K token** 请求的成本量级比约为 **1 : 5**（教学用：\$0.09 vs \$0.45）。
2. **解释** 为何「窗口有 200K」不等于「每次都应该用满」：费用近似随**有效输入规模**缩放。
3. **采纳 60% 使用量**介入建议：为工具输出突刺与多轮累积留安全垫。
4. **拆解** 成本构成：**输入 token**、**缓存命中**、**输出 token**、**摘要额外调用**（若有）。
5. **制定** 个人/团队的「上下文预算 SLO」：例如单任务会话目标占用上限。

## 生活类比：搬家公司按车厢体积计费

你叫同一辆车：

- 只装 **30 箱**（类比 ~30K token 输入），按起步价 + 少量立方 —— **便宜一档**。
- 装到 **150 箱**（类比 ~150K），车厢要更大、装卸更久、油费更高 —— **大约贵好几倍**。

模型请求里，长上下文就像塞满车厢：不是不能装，而是**每次搬运都更贵**。

## 教学用定价对照表（示意）

下列数字用于**比例直觉**；真实价格以官方页面为准。

| 单次请求有效输入规模（示意） | 示意费用（USD） | 相对倍数 |
|----------------|-----------|------|
| ~30K tokens    | \$0.09    | 1×   |
| ~150K tokens   | \$0.45    | 5×   |

核心结论：**5 倍**不是「模型变贵」，而是「你一次性喂给模型的上下文体积」变大导致的**输入侧计价**结果。

## Mermaid：token 体积与费用敏感度

[Mermaid 图表 — 请在在线版本中查看]

## 为什么 60% 就要介入（与第 8.1 节呼应）

设 `max = 200K`：

| 比例   | token 量级 | 解读                                |
|------|----------|-----------------------------------|
| 60%  | ~120K    | 建议介入：整理、拆分、 <code>/compact</code> |
| 87%  | ~174K    | Tier2 高概率触发                       |
| 100% | 200K     | 强压缩/失败风险上升                        |

60% 的意义：你还剩 ~80K 的「缓冲」给：

- 下一轮 `read_file` / 测试输出；
- 多轮 user/assistant 文本；
- 可能的摘要提示本身。

## 源码片段：费用估算器（伪代码）

```
type TieredPrice = { upToTokens: number; usdPerMillion: number }[];

function estimateInputCost(inputTokens: number, tiers: TieredPrice): number {
  // 教学用：简化为线性单价示意
  const usdPerMillion = 3; // 假设值
  return (inputTokens / 1_000_000) * usdPerMillion;
}

function compareTwoLoads() {
  const a = estimateInputCost(30_000, []);
  const b = estimateInputCost(150_000, []);
  return { low: a, high: b, ratio: b / a }; // 期望 ratio ≈ 5
}
```

## 片段：把「工具输出预算」纳入会话策略

```
const SESSION_INPUT_BUDGET = 120_000; // 60% of 200k

function beforeToolCall(estimatedReturn: number, current: number) {
  if (current + estimatedReturn > SESSION_INPUT_BUDGET) {
    warn("将超过建议预算：先 /compact 或缩小工具范围");
  }
}
```



## Mermaid：成本与压缩策略的博弈

[Mermaid 图表 — 请在在线版本中查看]

表：降本杠杆排序（从高影响到低）

| 杠杆                        | 机制           | 难度 |
|---------------------------|--------------|----|
| 控制工具输出体积                  | 直接降低输入 token | 低  |
| 提高缓存前缀命中                  | 降低有效计费输入     | 中  |
| 60% <code>/compact</code> | 避免长期高水位徘徊    | 低  |
| 拆分会话                      | 每会话峰值更低      | 中  |
| 把结论写入仓库                   | 上下文只保留指针     | 中  |

## 与缓存的关系（费用第二曲线）

即使「原始对话很长」，若 `cache read` 命中率高，账单可能好于 naive 计数。反之：

- 频繁破坏前缀 → 命中差 → 费用接近「全量重读」。

因此第 8.6 节的 `cache_edits` 同时也是成本工程。

## 场景推演 A：调试循环

| 轮次  | 行为        | 输入体积趋势         |
|-----|-----------|----------------|
| 1–5 | 每次贴 8K 日志 | 线性上升           |
| 6   | 达 60%     | 应压缩或外置日志       |
| 若不做 | 10+       | 冲 87%，Tier2 介入 |

教训：日志进文件，上下文只留 50 行关键。

## 场景推演 B：大重构

| 阶段 | 建议                                 |
|----|------------------------------------|
| 设计 | 新开会话 + <code>CLAUDE.md</code> 记录决策 |
| 实施 | 多会话按子模块切                           |
| 验证 | 测试输出截断                             |

## 表：「便宜但错」vs「贵但对」

| 策略            | 单次费用 | 总费用风险    | 正确性风险 |
|---------------|------|----------|-------|
| 上下文极简但不交代约束   | 低    | 高（返工）    | 高     |
| 上下文饱满但不整理     | 高    | 高（抖动+重试） | 中     |
| 60% 整理 + 文件指针 | 中    | 低        | 低     |

## 练习

1. 用你自己的账单（若可得）验证「长输入」与费用的相关性。
2. 给团队写一条 **SLO**：单会话建议不超过多少 K token（自行定义）。

## FAQ

**Q：输出 token 不重要吗？**

A：重要；但本篇强调上下文体积带来的 5x 直觉，因为许多用户忽略了输入侧。

**Q：压缩本身花钱吗？**

A：若摘要需要额外模型调用，可能产生**附加费用**；但与长期高水位相比，常更划算。

## 小结

把 30K vs 150K 的 \$0.09 vs \$0.45（约 5 倍）刻进肌肉记忆，你会自然接受 60% 介入：不是焦虑，而是**预算管理**。配合工具输出控制与缓存纪律，总成本与稳定性通常同时变好。

## 附录：速算卡片

200K 窗口  
60% ≈ 120K ← 建议开始整理  
87% ≈ 174K ← Tier2 高概率

同模型、可比单价下：  
输入 ×5 规模 ≈ 输入费用 ×5 (示意)

## 延伸阅读

- `03-micro-compaction.md`：先 cheap 地砍体积。
- `06-cache-aware.md`：别让压缩毁掉缓存红利。
- `08-manual-compact.md`：用焦点把「该花钱保留的」钉死。

## 案例表：三类开发者的一周账单差异（虚构但比例可信）

| 角色 | 习惯                 | 周费用示意 |
|----|--------------------|-------|
| A  | 每轮贴万行日志            | 高     |
| B  | 60% compact + 日志外置 | 中低    |
| C  | 极短上下文但反复返工         | 中高    |

## 术语对照

| 英文             | 中文       |
|----------------|----------|
| input tokens   | 输入 token |
| prompt caching | 提示缓存     |
| compaction     | 压缩/摘要    |

## 与产品指标对齐（建议）

若你在组织内推广 Claude Code，可跟踪：

1. 平均会话峰值 token（或 proxy：消息大小）。
2. compact 触发次数 / 千次请求。
3. 人均周费用与返工率（PR revert 率）。

---

## 反思题

---

1. 你愿意为「少返工」支付多少额外输入 token？
2. 哪些信息绝对应从上下文迁移到 Git 与 `CLAUDE.md`？

---

## 结语

---

成本分析的本质不是抠门，而是把 token 花在刀刃上：长上下文是工具，不是荣誉勋章。

---

## 8.10 最佳实践：上下文管理的「日常纪律」

---

像健身计划一样：不靠一次极限冲刺，靠每天 10 分钟的好习惯。

---

## 本节学习目标

---

1. 整合 Tier1/2/3、缓存感知、API compaction、手动 `/compact` 与成本意识，形成可执行清单。
2. 制定 个人工作流：从打开会话到收尾归档，上下文全程可控。
3. 识别 高风险习惯（万行日志、整仓 `read`、永不 compact）并给出替代方案。
4. 对齐 团队规范：代码审查时同时审查「对话是否可复现」（通过仓库而非聊天）。
5. 复盘 一次故障：用九节摘要模板做事后归档。

---

## 生活类比：厨房「边做边洗」

---

大厨炒菜不堆碗：

- Tier1 是顺手冲一下砧板；

- 60% 整理是每两道菜洗一批锅；
- Tier3 是打烊大扫除——不应成为日常唯一手段。

### 总览：一页纸清单（建议打印）

| 时机      | 动作                                  | 对应机制        |
|---------|-------------------------------------|-------------|
| 会话开始    | 明确目标一句话 + 分支名                       | 降低跑题膨胀      |
| 每 3-5 轮 | 扫一眼占用/消息长度                          | 预警          |
| ~60%    | <code>/compact</code> + 焦点 或 新会话    | 主动治理        |
| 大工具调用前  | 评估返回体积                              | 预防          |
| 里程碑     | 决策写入 <code>CLAUDE.md</code> /README | 记忆外置        |
| 故障      | 九节摘要进 <code>docs/incidents/</code>  | Tier3 思维人工化 |

### Mermaid：推荐日常工作流

[Mermaid 图表 — 请在在线版本中查看]

### 实践 1：工具调用「先问体积」

#### 反例

```
# 不要：直接让助手 cat 巨型日志进聊天
cat full-ci.log
```

#### 正例

```
# 先定位，再精读
rg -n "FAIL|Error" full-ci.log | head -n 80
sed -n '1200,1280p' full-ci.log
```

## 实践 2：分层记忆，不把聊天当数据库

| 信息类型     | 应存何处               |
|----------|--------------------|
| 命令如何启动服务 | CLAUDE.md / README |
| 本次调试临时栈  | 会话（可压缩）            |
| 合规审计原始日志 | 对象存储/日志系统          |

## 实践 3：焦点提示的标准 Operating Procedure

- 1. Must keep（3 条以内）
- 2. Nice to drop（明确许可摘要器删）
- 3. Source of truth（仓库路径/测试名）

```
/compact
Must keep: failing test name + last green commit hash + migration file list.
Nice to drop: early grep dumps.
SoT: repo branch feature/x, file packages/api/src/migrate.ts
```

## Mermaid：团队规范中的「责任边界」

[Mermaid 图表 — 请在在线版本中查看]

## 实践 4：与 CI/CD 的接口

| CI 输出            | 建议         |
|------------------|------------|
| JUnit XML        | 只摘失败用例名    |
| Docker build log | 只摘 ERROR 段 |
| E2E 录屏           | 不上传；给链接    |

## 实践 5：熔断发生时的 playbook

1. 停：不要疯狂重试同一 compact 路径。
2. 存：`git stash` 或 commit WIP。
3. 写：九节摘要到文件。
4. 开：新会话携带摘要路径。
5. 缩：缩小工具范围复现。

## 表：习惯红榜 vs 黑榜

| 红榜          | 黑榜           |
|-------------|--------------|
| 分段读文件       | 反复全文 read    |
| 失败先最小复现     | 一次并行 10 个重工具 |
| 60% compact | 等到模型卡顿才慌     |
| 指针代替粘贴      | 把二进制当文本塞聊天   |

## 源码片段：团队 hook 伪代码（可选）

```
// 在本地 CLI 包装层提示用户（概念）
onBeforeSendPrompt(ctx => {
  if (ctx.estimatedInputTokens > 120_000) {
    ui.warn("已超过 60% 建议阈值：考虑 /compact 或拆分任务");
  }
});
```

## 实践 6：代码审查中的「可复现性检查」

审查者问：

1. 若聊天丢失，能否靠 README 跑起来？
2. 关键环境变量是否记录在 `.env.example` ？
3. 迁移步骤是否写进 `docs/` ？

## 与第 9 篇的接口

| 本篇                        | 第 9 篇                        |
|---------------------------|------------------------------|
| 控制热上下文体积                  | <code>CLAUDE.md</code> 提供冷约束 |
| <code>/compact</code> 丢细节 | 自动记忆/项目记忆补长期偏好               |

注意：记忆注入也耗 token——记忆不是免费午餐。

## 复盘模板（事后）

```
## Incident Context Archive (Nine-Section)

1. Intent: ...
2. Concepts: ...
...

Artifacts:
- logs/2026-04-02-api-timeout.zip
```

## 练习

- 1. 写你的「个人上下文 SLO」三条。
- 2. 与同伴交换 `/compact` 焦点模板，互相挑刺。

## FAQ

Q：小团队也要这么正式吗？  
A：最小集：60% + 日志不外灌 + 决策进 README 就赢过大多数人。

Q：会不会过度优化？  
A：当你周账单或卡顿让你不爽时，就不是过度。

## 小结

最佳实践 = 预防（工具体积） + 节奏（60%） + 外置（文档/记忆） + 急救（九节/新会话）。把上下文管理当职业习惯，Tier3 会很少打扰你。



## 附录：本篇速查卡

Tier1: 无模型，清旧工具，留最近 5  
Tier2: ~87% 自动，3 连失败熔断  
Tier3: 九节 + CoT 剥离  
cache\_edits: 手术刀，保前缀  
API: compact-2026-01-12, Opus/Sonnet 4.6  
手动: /compact + 焦点  
成本: 30K \$0.09 vs 150K \$0.45 (5x 示意)  
介入: 60% 窗口占用

## 致谢与免责声明

本篇机制描述用于**教学**；具体阈值、头字段名与模型能力以你所用版本官方说明为准。

## 最后一图：把八篇知识叠成习惯

[Mermaid 图表 — 请在在线版本中查看]

## 第9篇：记忆系统

### 第 9 篇：记忆系统（Memory System）

Claude Code 完全指南 V2 · 会话会忘、仓库不会忘；记忆把「冷知识」从聊天里救出来。

#### 本篇学习目标

完成本篇后，你应能够：

1. 区分 三层记忆：会话记忆（当前对话）、项目记忆（`CLAUDE.md` 等由开发者维护）、自动记忆（`~/.claude/projects/memory/` 等由 Claude 观察写入）。
2. 解释 会话记忆的易失性：关闭后丢失，除非使用类似 `claude -c` 的延续机制（以你所用 CLI 为准）。
3. 绘制 `CLAUDE.md` 的层级拼接顺序：全局 → 项目根 → 目录级 → `.claude/CLAUDE.local.md`（个人、可 gitignore）。
4. 说明 自动记忆提取与 双模型检索（快速 Sonnet 扫标题/描述，最多注入 5 条）如何协作。
5. 贯彻 精确度优先：宁可少注入，也不塞无关记忆；并理解 记忆注入会增加上下文 token（与第 8 篇联动）。

#### 9.1 为什么需要「记忆系统」

##### 生活类比：三本笔记本

- 会话记忆像草稿纸：写满就扔，除非你把重要那页撕下来贴进档案袋。
- 项目记忆（`CLAUDE.md`）像团队手册：谁都能读，版本控制，长期有效。
- 自动记忆像私人助理的便签本：它观察你习惯，悄悄记下「咖啡半糖」，但你要能审查、纠正、删除。

##### 核心论断：上下文有限，记忆是「外置硬盘」

第 8 篇强调 200K 窗口与压缩；本篇强调：不被压缩卷走的东西，应落在记忆层。

| 载体                        | 谁写     | 持久性      | 典型内容     |
|---------------------------|--------|----------|----------|
| 会话                        | 用户+模型  | 低（关会话丢）  | 当下推理链    |
| <a href="#">CLAUDE.md</a> | 开发者    | 高（Git）   | 命令、约定、架构 |
| 自动记忆                      | Claude | 中高（本地文件） | 偏好、重复痛点  |

## Mermaid：三层记忆与 Claude Code 会话

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段（概念）：拼装时加载记忆

```
type MemoryLayer = "session" | "project" | "auto";

function buildPrompt(ctx: WorkspaceContext) {
  const projectBlock = loadClaudeMdStack(ctx.root); // 全局→根→目录→local
  const candidates = listAutoMemories(ctx.projectId);
  const picked = dualModelRetrieve(candidates, ctx.userQuery, { max: 5 });
  return concatSystemParts([projectBlock, formatInjections(picked)]);
}
```

## 与第 8 篇的交界：记忆不是免费

每注入一条记忆，都在吃掉 token 预算：

[Mermaid 图表 — 请在在线版本中查看]

因此 精确度优先（宁缺毋滥）同时是成本策略。

## 本篇路线图（10 节）

| 节    | 文件                                      | 主题                           |
|------|-----------------------------------------|------------------------------|
| 9.1  | <code>index.md</code>                   | 总览与三层记忆                      |
| 9.2  | <code>02-claude-md.md</code>            | <code>CLAUDE.md</code> 层级与篇幅 |
| 9.3  | <code>03-auto-extraction.md</code>      | 自动记忆提取                       |
| 9.4  | <code>04-dual-model-retrieval.md</code> | 双模型检索                        |
| 9.5  | <code>05-precision-first.md</code>      | 精确度优先                        |
| 9.6  | <code>06-kairos-dreaming.md</code>      | KAIROS 做梦模式                  |
| 9.7  | <code>07-dream-distillation.md</code>   | 做梦蒸馏                         |
| 9.8  | <code>08-memory-context.md</code>       | 记忆与上下文交互                     |
| 9.9  | <code>09-persistence.md</code>          | 持久化与 <code>claude -c</code>  |
| 9.10 | <code>10-practice.md</code>             | 练习与检查清单                      |

## KAIROS 做梦模式（预告）

长会话中，自动记忆可先当流水账；在低活跃、夜间或触发 `dream` 技能时，AI「睡觉」把日志蒸馏为结构化的「用户偏好」与「项目背景」。详见 9.6～9.7。

## 常见误解

| 误解                             | 纠正                          |
|--------------------------------|-----------------------------|
| 「记忆越多越好」                       | 注入有上限与成本， <b>精确度优先</b>      |
| 「自动记忆永远正确」                     | 需审计；可删可改                    |
| 「 <code>CLAUDE.md</code> 越长越好」 | 建议 <b>50～200 行</b> 甜区，超长难维护 |

## 练习（开篇）

- 列出三条你只愿意放在 `CLAUDE.local.md` 而不愿意提交 Git 的信息。
- 画一张图：你的项目里「会话 vs 仓库文档」各自负责什么。

## 术语表

| 术语           | 含义           |
|--------------|--------------|
| Injection    | 将记忆块写入当轮上下文  |
| Retrieval    | 从候选集中挑选相关记忆  |
| Distillation | 从流水账压缩成结构化偏好 |

## 小结

记忆系统是 Claude Code 的**第二存储层次**：会话负责当下，[CLAUDE.md](#) 负责团队真相，自动记忆负责个性化沉淀。下一步深入 [02-claude-md.md](#)，把层级与拼接规则写进你的肌肉记忆。

## 附录：目录级 [CLAUDE.md](#) 示意图（文字版）

~/claude/CLAUDE.md

repo/CLAUDE.md

repo/services/api/CLAUDE.md

repo/.claude/CLAUDE.local.md

← 全局

← 项目

← 子目录（若存在）

← 个人 (gitignore)

拼接时按**由广到窄**或产品定义顺序叠加；冲突时**窄范围覆盖常见**（以实现为准）。

## 安全提示

自动记忆可能记录路径、命令习惯；**敏感凭证**应走密钥管理，不进记忆文件。

## Mermaid：本篇阅读顺序

[Mermaid 图表 — 请在在线版本中查看]

## 与生态位对照

| 系统                         | 类比功能             |
|----------------------------|------------------|
| <code>.cursor/rules</code> | 项目规则（另一产品）       |
| <code>AGENTS.md</code>     | 代理说明（社区约定）       |
| <code>CLAUDE.md</code>     | Claude Code 一等公民 |

## 延伸阅读钩子

- 第 8 篇：`08-memory-context.md` 将用预算表量化注入成本。
- 第 5 篇：动态边界决定记忆块落在 system 的哪一段。

## 版本说明

具体路径 `~/.claude/projects/memory/`、命令 `claude -c`、技能名 `dream` 以你所安装版本为准；本篇以机制教学为主。

## 自测（简答）

- 为什么「最多 5 条」与精确度优先一致？
- `CLAUDE.local.md` 为什么要 gitignore？

下一节：`02-claude-md.md` —— 从全局到本地的拼接艺术，以及 50~200 行的维护纪律。

## 9.1 补充：一张表看懂「谁写、谁读、谁丢」

| 层                         | 典型写入者     | 典型读者   | 何时可能丢                            |
|---------------------------|-----------|--------|----------------------------------|
| 会话                        | 你 + 模型    | 当前线程   | 关窗口、强压缩、未 <code>claude -c</code> |
| <a href="#">CLAUDE.md</a> | 开发者/PR    | 所有贡献者  | 仅当误删 Git 提交                      |
| 自动记忆                      | Claude 观察 | 未来会话检索 | 你清目录、换机未备份                       |

## Mermaid：记忆写入与读取方向

[Mermaid 图表 — 请在在线版本中查看]

## 与「精确度优先」的一句话

记忆系统的 KPI 不是条数，而是「注入后更少纠正」。第 9.5 节展开。

## 与「KAيروس」的一句话

流水账 + 择机做梦蒸馏 = 低成本个性化。第 9.6~9.7 节展开。

## 快速对照：第 8 篇链接

| 第 8 篇概念               | 第 9 篇关联                           |
|-----------------------|-----------------------------------|
| 60% 介入                | 记忆与 CLAUDE 文本也占该 60%              |
| Tier2 87%             | 长描述记忆加速触线                         |
| <code>/compact</code> | 压会话，不替代 <a href="#">CLAUDE.md</a> |

## 自测答案提示

1. 「最多 5 条」限制噪声与 token，与「宁缺毋滥」一致。

2. `CLAUDE.local.md` gitignore 以免个人路径、口味、本地密钥风格描述进入团队仓库。

## 9.2 CLAUDE.md：层级、拼接与 50~200 行纪律

像建筑规范：总规在市里，细则在地块，装修备注在自己抽屉——叠在一起才好用，但不能互相打架。

### 本节学习目标

1. 默写 四层来源（教学顺序）：`~/ .claude/CLAUDE.md`（全局）→ 项目根 `CLAUDE.md` → 目录级 `CLAUDE.md` → `.claude/CLAUDE.local.md`（个人，建议 gitignore）。
2. 解释「按位置拼接」：越靠近当前工作目录的规则，越具体。
3. 遵守 篇幅建议 50~200 行：可检索、可审查、可 diff。
4. 区分 应提交仓库的内容 vs 应留在 local 的个人偏好。
5. 撰写 一份最小可用模板，覆盖：运行、测试、风格、禁区。

### 生活类比：俄罗斯套娃式的「家规」

- 全市物业管理条例（`~/ .claude/CLAUDE.md`）：所有项目默认遵守。
- 小区公约（仓库根 `CLAUDE.md`）：这个仓库特有。
- 楼栋通知（子目录 `CLAUDE.md`）：例如 `services/payments/` 下的额外约束。
- 自家便签（`CLAUDE.local.md`）：只有你喜欢的快捷键、本地路径——别贴到电梯里（别 commit）。



## 层级表

| 层级   | 路径（典型）                                            | 版本控制      | 受众     |
|------|---------------------------------------------------|-----------|--------|
| 全局   | <code>~/.claude/CLAUDE.md</code>                  | 个人机器      | 你所有项目  |
| 项目   | <code>&lt;repo&gt;/CLAUDE.md</code>               | Git       | 团队     |
| 目录   | <code>&lt;repo&gt;/foo/CLAUDE.md</code>           | Git       | 子模块贡献者 |
| 个人本地 | <code>&lt;repo&gt;/.claude/CLAUDE.local.md</code> | gitignore | 仅你     |

## Mermaid：拼接管线（概念）

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：解析栈（伪代码）

```
function loadClaudeMdStack(cwd: string): string {
  const parts: string[] = [];

  parts.push(readIfExists("~/.claude/CLAUDE.md"));
  parts.push(readIfExists(`${repoRoot(cwd)}/CLAUDE.md`));

  for (const dir of ancestorsFromRootTo(cwd)) {
    const p = `${dir}/CLAUDE.md`;
    if (exists(p)) parts.push(read(p));
  }

  parts.push(readIfExists(`${repoRoot(cwd)}/.claude/CLAUDE.local.md`));

  return parts.filter(Boolean).join("\n\n---\n\n");
}
```

## 为什么 50~200 行

| 行数区间   | 评价                    |
|--------|-----------------------|
| < 30   | 可能缺关键命令               |
| 50~200 | 甜区：人类可读、模型可抓重点        |
| > 300  | 易重复、易陈旧、难 code review |

## Mermaid：维护生命周期

[Mermaid 图表 — 请在在线版本中查看]

## 模板 A：最小可用 CLAUDE.md

```
# 项目: example-api

## 运行
- `pnpm i && pnpm dev` - 本地 API :8080

## 测试
- `pnpm test` - 单元
- `pnpm test:e2e` - 需 Docker

## 代码风格
- TypeScript strict; 禁止 `any` 除非附理由

## 禁区
- 勿改 `legacy/v1` 除非 ticket LEGACY-xxx

## 架构速览
- `src/domain` 纯逻辑; `src/http` 适配层
```

## 模板 B： CLAUDE.local.md (个人)

```
# 本地偏好 (勿提交)

- 默认使用简体中文回复我
- 本地 DB 连接串见 `~/config/...` (勿写入仓库)
```

`.gitignore` 追加：

```
.claude/CLAUDE.local.md
```

### 表：内容放置决策

| 内容       | 放根 <a href="#">CLAUDE.md</a> | 放目录 <a href="#">CLAUDE.md</a> | 放 local   |
|----------|------------------------------|-------------------------------|-----------|
| 团队测试命令   | 是                            | 若仅子包                          | 否         |
| 个人语气偏好   | 否                            | 否                             | 是         |
| 支付模块额外规范 | 可摘要                          | 是                             | 否         |
| 秘密密钥     | 永不                           | 永不                            | 永不（用密钥管理） |

### 冲突解决心智模型

| 情况          | 建议                  |
|-------------|---------------------|
| 根与目录矛盾      | 更具体目录优先（常见）         |
| 全局与项目矛盾     | 项目优先（团队真相）          |
| local 与团队矛盾 | local 仅个人；团队规则仍以根为准 |

精确优先级以实现为准；文档应显式写「本仓库优先级约定」。

### 源码片段：目录级覆盖示例

`services/billing/CLAUDE.md`：

```
# Billing 子系统

- 所有价格以「分」整数存储
- 禁止直接调用第三方支付 SDK；走 `BillingPort`
```

## 与自动记忆的分工

| <u>CLAUDE.md</u> | 自动记忆    |
|------------------|---------|
| 显式、可 PR          | 模型观察生成  |
| 团队共识             | 个人化沉淀   |
| 稳定条款             | 可能随习惯演变 |

## 练习

- 给你当前仓库写一版 80 行内 `CLAUDE.md` 草稿。
- 列出应放进 `CLAUDE.local.md` 的三项。

## FAQ

- Q：每个子目录都要 `CLAUDE.md` 吗？**  
A：不必须；只在边界清晰的子系统加，避免碎片化。
- Q：能用中文写吗？**  
A：可以；团队统一即可。

## 小结

`CLAUDE.md` 是项目记忆的脊柱：从全局到本地按位置拼接，用 50~200 行保持可读与可维护，把个人项隔离到 `.claude/CLAUDE.local.md` 并 gitignore。

## 附录：目录布局示例

```
myapp/
  CLAUDE.md
  .claude/
    CLAUDE.local.md      # gitignored
  packages/
    web/
      CLAUDE.md          # 前端特有
    api/
      CLAUDE.md          # 后端特有
```

## Mermaid：多包 monorepo 的信息流

[Mermaid 图表 — 请在在线版本中查看]

## 审查清单（Code Review）

- [ ] 无密钥与内部 URL 泄露
- [ ] 命令可在 CI/新同事机器复现
- [ ] 与 README 不重复冗长（可互链）
- [ ] 过期段落已删或标注日期

## 与 AGENTS.md 共存策略

若仓库同时存在：

| 文件        | 建议               |
|-----------|------------------|
| CLAUDE.md | Claude Code 专用细节 |
| AGENTS.md | 跨工具通用代理说明        |
| 重复内容      | 单点维护 + 另一方链接     |

## 反模式

| 反模式                               | 后果      |
|-----------------------------------|---------|
| 把聊天全文贴进 <a href="#">CLAUDE.md</a> | 不可维护    |
| 复制粘贴大段第三方文档                       | 版权与更新问题 |
| local 提交进 Git                     | 泄漏个人路径  |

## 术语

| 英文            | 中文    |
|---------------|-------|
| stack / merge | 拼接栈   |
| scoped rules  | 目录级规则 |

# 9.3 自动记忆提取：观察、写入与审计

像称职的秘书：记住你爱喝半糖，但若记错了，你要能翻开笔记本划掉那一行。

## 本节学习目标

1. **定义** 自动记忆：Claude 在会话中**观察**稳定重复的用户偏好与项目事实，并写入 `~/.claude/projects/memory/`（路径以版本为准）。
2. **列举** 适合自动沉淀的信息与**不适合**的信息（密钥、一次性试错）。
3. **设计** 个人审计节奏：定期查看、删除、纠正错误记忆。
4. **解释** 自动记忆与 `CLAUDE.md` 的互补：前者偏「个性化」，后者偏「团队契约」。
5. **预判** 错误记忆注入的后果：模型行为偏移、上下文噪声。

## 生活类比：Spotify 的「猜你喜欢」

流媒体会记录你切歌行为来调整推荐：

- **好处**：少搜几次。
- **风险**：某次借账号给朋友听了一晚重金属，推荐就歪了——你需要「重置口味」或删历史。

自动记忆同理：**高效但有偏，需要人类把关。**

## 自动记忆可能记录什么（示例域）

| 类型   | 示例                     |
|------|------------------------|
| 语言偏好 | 「回复用简体中文」              |
| 工具偏好 | 「优先 pnpm 不用 npm」       |
| 项目习惯 | 「此仓库 E2E 用 playwright」 |
| 重复踩坑 | 「勿改某 legacy 目录」        |

## Mermaid：从观察到落盘

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：memory 文件形态（伪 JSON）

```
{
  "id": "mem_7f3a",
  "title": "包管理器偏好",
  "description": "用户要求使用 pnpm；避免 npm 命令示例",
  "scope": "project:acme/web",
  "created_at": "2026-04-01T12:00:00Z",
  "confidence": 0.82
}
```

实际格式以实现为准；关键是**可检索的标题/描述**。

## 表：适合 vs 不适合自动记忆

| 适合         | 不适合       |
|------------|-----------|
| 重复 ≥3 次的偏好 | 临时 A/B 尝试 |
| 可公开的项目习惯   | 密码、token  |
| 与仓库文档一致的事实 | 未验证的猜测    |
| 个人效率设置     | 法务敏感结论    |

## 审计 SOP（建议每月）

1. 打开 `~/.claude/projects/memory/`（或设置页等价 UI）。
2. 删除低置信度或已过时条目。
3. 把应团队共享的条目升格为 `CLAUDE.md` PR。
4. 对错误条目写一句「纠正指令」给模型，观察是否更新。

## Mermaid：错误记忆的放大回路

[Mermaid 图表 — 请在在线版本中查看]

## 与双模型检索的衔接

自动记忆先广泛沉淀；检索阶段由 Sonnet 快速扫标题/描述，最多 5 条注入（见下一节）。提取与检索是流水线前后级。

## 源码片段：安全过滤（伪代码）

```
function safeToAutowrite(observation: Observation): boolean {
  if (observation.containsSecretPattern) return false;
  if (observation.confidence < 0.7) return false;
  if (observation.isOneOff) return false;
  return true;
}
```



## 练习

1. 写出你希望永远不要自动记录的两类信息。
2. 设计一条「团队政策」：自动记忆默认关闭还是开启。

## FAQ

- Q：自动记忆会同步到云端吗？

A：以实现与账号设置为准；敏感项目建议查隐私说明。
- Q：能否按项目隔离？

A：教学路径 `projects/memory/` 暗示按项目隔离；确认本地目录结构。

## 小结

自动记忆提取让 Claude 从「复读你的偏好」进化到「记住你的偏好」，但必须配套**审计与升格策略**：可靠的进 `CLAUDE.md`，个人的留记忆文件，**机密永不落盘**。

## 附录：观察信号表

| 信号强度 | 说明       |
|------|----------|
| 弱    | 说过一次     |
| 中    | 同一项目多次   |
| 强    | 纠正模型后仍坚持 |

仅中强适合自动写入（示意规则）。

## 与合规

- 若你在受监管行业：
- 记录**数据保留策略**；
  - 记忆目录纳入**退出权流程**（删除用户数据时一并清）。

# Mermaid: [CLAUDE.md](#) 升格路径

[Mermaid 图表 — 请在在线版本中查看]

## 反模式

| 反模式      | 修复                           |
|----------|------------------------------|
| 从不审计     | 设日历提醒                        |
| 把机密写进记忆  | 换用 vault                     |
| 团队靠记忆传规范 | 应用 <a href="#">CLAUDE.md</a> |

## 术语

| 英文          | 中文 |
|-------------|----|
| extraction  | 提取 |
| observation | 观察 |

## 扩展：写入节流（概念）

避免同一偏好 10 分钟内重复落盘：

```
const lastWrite = new Map<string, number>();

function throttleWrite(key: string, minIntervalMs: number): boolean {
  const now = Date.now();
  const prev = lastWrite.get(key) ?? 0;
  if (now - prev < minIntervalMs) return false;
  lastWrite.set(key, now);
  return true;
}
```

## Mermaid：多项目隔离

[Mermaid 图表 — 请在在线版本中查看]

防止 跨仓污染 是自动提取与检索的共同前提。

表：提取置信度与人工确认

| 置信度     | 默认行为     | 企业可选 |
|---------|----------|------|
| < 0.5   | 不写       | 同上   |
| 0.5~0.8 | 写「待确认」标记 | 需人审  |
| > 0.8   | 直接写入     | 审计抽样 |

### 练习补充

3. 为「自动提取默认关闭」写一条用户可见的产品说明（不超过 100 字）。

## 9.4 双模型检索：快速 Sonnet 扫描与「最多 5 条」注入

像图书馆检索台：先由管理员快速扫书名目录挑出五本书，再把书送到阅览室，而不是把整个书库搬进去。

### 本节学习目标

- 描述** 双模型检索管线：快速 Sonnet 负责扫描记忆条目的**标题/描述**，做相关性打分与筛选。
- 解释** 注入上限 **最多 5 条**：平衡「有用」与「上下文成本」。
- 对比** 单阶段大模型全量读取所有记忆的反例：延迟高、费用高、噪声大。
- 说明** 筛选结果如何进入 system 动态段（与第 5 篇边界呼应）。

5. 调试 检索失败：标题含糊、描述过短、跨项目污染。

---

## 生活类比：急诊分诊

---

到医院：

- 分诊护士（快速 Sonnet）看你主诉关键词，决定你去哪科——快。
- 专科医生（主模型）再深度处理——贵且慢不能用来做海量初筛。

记忆检索同样：快模型筛目录，主模型用结果。

---

## Mermaid：双阶段检索

---

[Mermaid 图表 — 请在在线版本中查看]

---

## 源码片段：检索接口（伪代码）

---

```
type MemoryCard = {
  id: string;
  title: string;
  description: string;
};

async function dualModelRetrieve(
  candidates: MemoryCard[],
  userQuery: string,
  opts: { max: number }
): Promise<MemoryCard[]> {
  const scored = await fastSonnetScore(candidates, userQuery); // 只看 title/desc
  return scored.slice(0, opts.max); // max 默认 5
}
```

## 表：为什么限制 5 条

| 若 K 太大   | 结果        |
|----------|-----------|
| token 膨胀 | 费用 ↑ 压缩 ↑ |
| 不相关混入    | 行为偏移      |
| 延迟       | 注入块解析变慢   |

| 若 K 太小 | 结果   |
|--------|------|
| 丢偏好    | 体验下降 |

5 是产品与成本的折中（教学数）。

## Mermaid：候选池来源

[Mermaid 图表 — 请在在线版本中查看]

## 写好标题/描述的技巧

| 差    | 好                                  |
|------|------------------------------------|
| 「偏好」 | 「包管理：优先 pnpm」                      |
| 「注意」 | 「勿修改 legacy/v1 路由」                 |
| 「项目」 | 「E2E: playwright + docker compose」 |

可检索性直接决定 Sonnet 扫描质量。

## 源码片段：注入格式化

```
function formatInjections(picked: MemoryCard[]): string {
  return [
    "## Retrieved memories (max 5)",
    ...picked.map(
      (m) => `### ${m.title}\n${m.description}\n(ref: ${m.id})`
    ),
  ].join("\n\n");
}
```

## 调试清单

| 症状   | 检查                               |
|------|----------------------------------|
| 从未注入 | 候选池是否空                           |
| 注入无关 | 标题是否太泛                           |
| 注入过时 | 审计记忆文件                           |
| 注入重复 | 与 <a href="#">CLAUDE.md</a> 是否重复 |

## 与精确度优先（下一节预告）

双模型检索是工程手段；产品哲学是「宁缺毋滥」——若不确定相关，少注入。

## 练习

- 给你三条真实偏好写「标题+描述」各一行。
- 假设候选池有 20 条，手写你认为应对某查询返回的 5 条 id。

## FAQ

- Q：能否全用 Opus 检索？

A：可以但贵；快速 Sonnet 是性价比常见的初筛角色。
- Q：5 能改吗？

A：可能为配置项；默认体现产品保守策略。

## 小结

双模型检索用 **Sonnet 快速扫标题/描述** 从候选池挑出 **最多 5 条** 记忆注入，避免把记忆系统做成「第二个无限上下文」。写好记忆卡片，比盲目增加条数更重要。

## 附录：打分特征（概念）

| 特征       | 用途    |
|----------|-------|
| 关键词重叠    | 粗匹配   |
| 项目 scope | 防跨仓污染 |
| 时间衰减     | 降权旧记忆 |

## Mermaid：失败路径

[Mermaid 图表 — 请在在线版本中查看]

## 性能直觉

| 阶段          | 相对成本 |
|-------------|------|
| 扫描 50 条标题   | 低    |
| 主模型读 5 段描述  | 中    |
| 主模型读 50 段全文 | 高    |

## 反模式

| 反模式           | 后果                 |
|---------------|--------------------|
| 描述写成长文        | 扫描模型仍只看摘要字段，浪费维护成本 |
| 同一事实复制 10 张卡片 | 检索抖动               |
| 跨项目同名标题       | 误注入                |

## 团队规范示例

```
## 记忆卡片规范
- title: 动宾结构, <= 40 字
- description: 1~3 句, 可执行
- 禁止放密钥
```

# 9.5 精确度优先：宁缺毋滥的注入哲学

像配菜：宁可少一道清爽小菜，也不要堆一盘互相串味的剩菜。

## 本节学习目标

- 1. 陈述 精确度优先原则：不确定相关则不注入，优于「可能有用就塞」。
- 2. 解释 该原则与 最多 5 条、上下文成本、模型注意力 的一致性。
- 3. 识别 误注入的典型模式：标题泛化、跨项目 scope 泄露、过期偏好。
- 4. 制定 团队评审：新增记忆卡片需 title/description 检查表。
- 5. 度量 主观指标：注入后用户纠正次数是否下降。

## 生活类比：导航语音

开车导航：

- 精确：「前方 200 米右转」——你立刻行动。



- **不精确**：同时播报五条备选路线历史、上周路况、朋友推荐餐厅——你**认知超载**，反而走错。

记忆注入亦然：**少而准胜过多而杂**。

## Mermaid：噪声注入的注意力稀释

[Mermaid 图表 — 请在在线版本中查看]

## 表：注入决策矩阵

| 相关性 | 置信度 | 动作          |
|-----|-----|-------------|
| 高   | 高   | 注入          |
| 高   | 中   | 可注入 + 标记待确认 |
| 中   | 任意  | 默认不注入       |
| 低   | 任意  | 不注入         |

## 源码片段：保守阈值（伪代码）

```
function shouldInject(card: MemoryCard, score: number): boolean {
  if (score < 0.78) return false; // 宁缺毋滥
  if (card.scope !== currentProjectScope) return false;
  if (card.stale) return false;
  return true;
}
```

## 与第 8 篇联动：token 账簿

每条注入：

+ title + description + 格式开销 ≈ 数十~数百 tokens（视长度）  
× 5 条 ≈ 可观

**精确度优先 = 成本优先。**

# Mermaid：检索—注入—反馈闭环

[Mermaid 图表 — 请在在线版本中查看]

## 反模式表

| 反模式                             | 症状      |
|---------------------------------|---------|
| 「万能卡片」                          | 任何查询都命中 |
| 复制粘贴 <code>CLAUDE.md</code> 进记忆 | 重复与冲突   |
| 把实验性偏好固化                        | 长期误行为   |

## 团队检查表（PR Review）

- ☐ title 是否唯一且具体
- ☐ description 能否单读自明
- ☐ 是否与现有 `CLAUDE.md` 重复
- ☐ scope 是否正确项目
- ☐ 是否含敏感信息

## 练习

- 写一条「应被拒绝注入」的虚构记忆，并说明原因。
- 把同一事实分别写成「泛标题」与「精确标题」对比。

## FAQ

- Q：注入 0 条是不是浪费检索？**  
A：检索成本低；错误注入成本高。
- Q：用户明确说「记住」就一定对？**  
A：仍应可审计；用户也会变心。

## 小结

精确度优先是记忆系统的价值观：宁缺毋滥保护模型注意力与 token 预算。工程上靠 Sonnet 扫描 + 阈值 + 上限 5 落实，文化上靠 卡片质量与审计 巩固。

## 附录：A/B 思想实验

| 策略 A     | 策略 B        |
|----------|-------------|
| 总是注满 5 条 | 只注入 >0.85 分 |
| 用户纠正率？   | 预期更低        |

## Mermaid：分数分布示意（条形抽象）

[Mermaid 图表 — 请在在线版本中查看]

注：阈值线（如 0.78）之上才注入；具体数以实现为准。

## 与伦理

滥注入可能强化错误刻板印象（例如错误归因用户能力）。精确度优先也是公平性策略。

## 术语

| 英文                       | 中文    |
|--------------------------|-------|
| precision-first          | 精确度优先 |
| false positive injection | 误注入   |

## 升级路径

当产品允许用户调「记忆敏感度」：

| 档位 | 描述         |
|----|------------|
| 保守 | 默认；精确度优先   |
| 平衡 | 略降阈值       |
| 激进 | 易误注入；不推荐默认 |

## 案例简析

**场景：**前端项目记忆注入了一条「后端 Java 风格偏好」。

**原因：**scope 未收紧或标题过泛。

**修复：**删卡 + 为前后端分项目 + 标题加栈前缀。

## 与监控

若可埋点：

- injected\_count\_per\_turn
- user\_correction\_after\_injection\_rate

用数据验证「宁缺毋滥」是否改善体验。

# 9.6 KAIROS 做梦模式：流水账、低活跃与夜间触发

像睡眠整理白天碎片：白天把经历先记在草稿本上，夜里大脑做重组，醒来留下几条清晰结论。

## 本节学习目标

1. **定义** KAIROS 做梦模式（教学名）：在长会话中，把自动记忆先当作**时间顺序流水账**，再在**低活跃、夜间或 dream 技能**触发时进入「睡眠」整理。
2. **解释** 为何需要两阶段：实时写入**求快**，做梦**求结构化**。
3. **列举** 适合触发做梦的信号：会话时长、无输入间隔、用户显式调用技能。
4. **对比** 做梦前后记忆文件形态：从日志到「用户偏好」「项目背景」分类。
5. **评估** 风险：做梦摘要错误时的回滚与人工审计。

## 生活类比：旅行日记与游记

- **白天**：在高铁上随手记「3:15 误车」「司机电话 138xxxx」——**流水账**。
- **晚上**：酒店里把一天整理成「行程」「花费」「明日计划」——**做梦蒸馏**。

KAIROS 把自动记忆从「草稿」变成「游记」。

## Mermaid：KAIROS 两阶段

[Mermaid 图表 — 请在在线版本中查看]

## 触发条件表（教学归纳）

| 条件       | 说明                   |
|----------|----------------------|
| 低活跃      | 例如 30~120 分钟无新消息（示意） |
| 夜间       | 本地时间窗口，如 23:00~07:00 |
| dream 技能 | 用户或策略显式触发            |
| 会话长度     | token 或轮次超阈          |

精确参数以实现为准。

## 流水账记忆示例（虚构）

```
[10:01] 用户拒绝使用 npm, 要求 pnpm
[10:18] 用户说测试要 Docker
[11:03] 用户纠正: E2E 在 packages/web
```

做梦后应变为结构化条目（见 9.7）。

## Mermaid：与主会话的关系

[Mermaid 图表 — 请在在线版本中查看]

## 为什么叫「KAIROS」（教学）

| 词源联想   | 含义                |
|--------|-------------------|
| Kairos | 古希腊「恰当时机」         |
| 做梦     | 在对的时机做整理，而非每轮打断用户 |

名称帮助记忆：不是每时每刻蒸馏，而是择机。

## 与 `dream` 技能

技能层可封装：

```
# 概念示意，非严格 YAML
skill: dream
triggers:
  - manual
  - idle_night
actions:
  - distill_memories
  - prune_duplicates
```

## 风险表

| 风险      | 缓解       |
|---------|----------|
| 做梦时摘要错  | 保留原流水账归档 |
| 夜间任务耗电  | 笔记本可关策略  |
| 用户不想自动跑 | 设置里关闭    |

## 练习

1. 写三条你认为应留在「流水账」而非立即结构化的事件。
2. 设计一个「仅手动 dream」的团队政策理由。

## FAQ

- Q：做梦会调用模型吗？

A：蒸馏通常需要模型参与；成本低于把流水账全量注入每轮对话。
- Q：和 /compact 什么关系？

A： /compact 压会话上下文；KAIROS 整理记忆文件——层次不同。

## 小结

KAIROS 做梦模式解决「实时记忆只能草草记」的问题：先流水账沉淀，再在低活跃/夜间/dream 把记忆重组为可用卡片。它是连接 9.3 提取 与 9.7 蒸馏 的桥梁。

## 附录：时间线示意图

|---活跃对话--->| idle |---dream--->| 结构化记忆 |

## Mermaid：策略开关

[Mermaid 图表 — 请在在线版本中查看]

## 与隐私

夜间任务读取本地记忆文件：

- 确保磁盘加密；
- 企业环境评估 **DLP** 策略。

## 反模式

| 反模式          | 后果   |
|--------------|------|
| 每轮都全量蒸馏      | 费用爆炸 |
| 不做梦只堆流水账     | 检索差  |
| dream 失败静默忽略 | 记忆腐化 |

## 术语

| 英文            | 中文   |
|---------------|------|
| dreaming mode | 做梦模式 |
| idle          | 低活跃  |

## 与第 8 篇

会话压缩丢细节；**做梦蒸馏**把可保留的长期项写入记忆，形成互补。



## 场景推演

---

周一长会话 产生 40 条流水账；周一晚 低活跃触发 dream → 合并为 6 张卡片；周二 检索只扫 6 张标题 → 更快更准。

---

## 监控建议

---

- dream\_runs\_count
  - distilled\_cards\_count
  - dream\_fail\_rate
- 

## 文化注记

---

「AI 睡觉」是隐喻：实际是后台批处理；避免用户以为模型真意识。

---

## 9.7 做梦蒸馏：从流水账到「用户偏好」与「项目背景」

---

像熬高汤：一大锅骨头和香料，最后只要那一碗清澈鲜甜的汤底。

---

## 本节学习目标

---

1. **描述** 蒸馏输出两类结构化块：**用户偏好**（如何与我协作）与 **项目背景**（此仓库的事实与约束）。
  2. **解释** 蒸馏与 **双模型检索** 的衔接：蒸馏提升卡片 **title/description** 质量。
  3. **对比** 蒸馏前后 token：流水账总长大于结构化卡片之和。
  4. **编写** 蒸馏提示的**结构骨架**（教学用），便于团队自建批处理。
  5. **处理** 冲突：新蒸馏与旧卡片矛盾时的合并策略。
-

## 生活类比：周报

员工每天写工作日志（流水账），周五写周报（蒸馏）：

- 日志：过程
- 周报：结果 + 风险 + 下周计划

老板只看周报也能掌握大局——类似模型只看蒸馏卡片。

## Mermaid：输入—输出形态

[Mermaid 图表 — 请在在线版本中查看]

## 两类结构化块定义

### 用户偏好（User Preferences）

| 字段 | 示例                              |
|----|---------------------------------|
| 沟通 | 简体中文；先给结论再细节                    |
| 工具 | pnpm；禁止擅自 <code>git push</code> |
| 审查 | PR 需附测试结果摘要                     |

### 项目背景（Project Context）

| 字段 | 示例                                  |
|----|-------------------------------------|
| 架构 | monorepo；API 在 packages/api         |
| 约束 | legacy/v1 冻结                        |
| 命令 | <code>pnpm test:e2e</code> 需 Docker |

## 源码片段：蒸馏提示骨架（伪）

你将阅读一段按时间排序的会话记忆流水账。  
请输出 JSON 数组，每项含：type ("preference"|"project"), title, description, evidence\_ids。

规则：

- 合并重复事件
- 丢弃一次性试错
- 描述需可执行、无敏感密钥

## Mermaid：合并去重

[Mermaid 图表 — 请在在线版本中查看]

## 表：蒸馏质量检查

| 检查项   | 通过标准  |
|-------|-------|
| 无密钥   | 正则扫描  |
| 有标题   | 非空    |
| 描述长度  | 1~5 句 |
| scope | 正确项目  |

## 与 CLAUDE.md 的升格（再次强调）

| 蒸馏结果  | 动作                      |
|-------|-------------------------|
| 团队应共识 | 提 PR 到 <u>CLAUDE.md</u> |
| 纯个人   | 保留记忆                    |
| 重复    | 删重复卡片                   |

## 练习

---

1. 给一段虚构流水账（5 行），手写蒸馏后的 2 张卡片。
  2. 指出哪一行流水账应被**丢弃**而非蒸馏。
- 

## FAQ

---

**Q：蒸馏会删证据吗？**

A：会；证据应已在 Git/文档；记忆只保留**指针级**描述。

**Q：能否人工触发蒸馏？**

A：`dream` 技能/命令（以实现为准）。

---

## 小结

---

做梦蒸馏把 KAIROS 的流水账变为 **用户偏好** 与 **项目背景** 两类高质量卡片，降低检索噪声与上下文注入量，是记忆系统「**由乱到治**」的关键一步。

---

## 附录：JSON 输出示例

---

```
[
  {
    "type": "preference",
    "title": "沟通：中文与结构",
    "description": "用户偏好简体中文；回复先结论后步骤。",
    "evidence_ids": ["log_12", "log_18"]
  },
  {
    "type": "project",
    "title": "E2E 运行方式",
    "description": "在 packages/web 运行 playwright; 需 docker compose up testdeps。",
    "evidence_ids": ["log_22"]
  }
]
```

---

## Mermaid：失败重试

[Mermaid 图表 — 请在在线版本中查看]

## Token 直觉表

| 阶段             | 相对体积 |
|----------------|------|
| 流水账 10K tokens | 100% |
| 蒸馏后卡片 1.5K     | ~15% |

示意比例；以实测为准。

## 冲突解决策略

| 策略              | 何时   |
|-----------------|------|
| 时间新覆盖旧          | 偏好变更 |
| 合并描述            | 互补事实 |
| 标记 human_review | 不确定  |

## 反模式

| 反模式      | 后果   |
|----------|------|
| 蒸馏输出过长   | 检索退化 |
| 把流水账原样复制 | 未蒸馏  |
| 无校验直接写入  | 脏数据  |

## 与 Opus / Sonnet

| 角色   | 可用模型      |
|------|-----------|
| 蒸馏整理 | 较强模型更稳    |
| 扫描检索 | 快速 Sonnet |

## 术语

| 英文                | 中文    |
|-------------------|-------|
| distillation      | 蒸馏    |
| structured memory | 结构化记忆 |

## 企业扩展

- 蒸馏任务走审计日志；
- 输出经 PII 扫描 再落盘。

## 与 9.5 精确度优先

蒸馏时应主动丢弃弱相关流水行，预先践行「宁缺毋滥」。

## 复盘模板

```
## Dream run 2026-04-02
- input lines: 42
- output cards: 6
- pruned: 36
- manual fixes: 1 (see issue #...)
```

## 9.8 记忆与上下文交互：注入如何吃掉窗口预算

记忆不是「额外附赠的超能力」，而是从同一只钱包里掏钱——掏的是上下文 token。

### 本节学习目标

1. **量化直觉**：每条记忆注入增加 **system 动态段** token，从而推高总输入与**压缩概率**（第 8 篇）。
2. **解释** 记忆与 **prompt caching** 的交界：注入块通常在**动态边界之后**，影响后缀长度。
3. **优化** 注入块格式：**短标题 + 精炼描述**，避免 Markdown 花饰。
4. **权衡** 「多记」vs 「少注入」：仓库文档承担部分冷知识。
5. **观测** 一轮对话中 **projectBlock + injections** 的占比（概念）。

### 生活类比：行李箱里的「必读小册子」

你登机箱总容量固定：

- 每塞一本「旅行必读小册子」（记忆注入），就要拿掉一件 T 恤（原始对话细节）。
- 小册子越薄，你越能多带真正要穿的衣服。

### Mermaid：上下文拼装中的记忆位

[Mermaid 图表 — 请在在线版本中查看]

## 表：组成部分与典型可压缩性

| 块                         | 是否常被 Tier1 动 | token 弹性 |
|---------------------------|--------------|----------|
| 静态 system                 | 否            | 低（稳定）    |
| <a href="#">CLAUDE.md</a> | 否            | 中（随文件变）  |
| 记忆注入                      | 否            | 完全由你控制   |
| 工具输出                      | 是            | 高        |

## 源码片段：估算注入开销（伪代码）

```
function estimateInjectionTokens(cards: MemoryCard[]): number {
  const text = formatInjections(cards);
  return countTokens(text);
}

function budgetCheck(ctx: Context) {
  const inj = estimateInjectionTokens(ctx.injectedMemories);
  const msgs = countTokens(ctx.messages);
  const budget = 200_000;
  if (inj + msgs > 0.6 * budget) {
    warn("记忆+消息已超 60%: 考虑缩短 CLAUDE.md 或记忆描述");
  }
}
```

## Mermaid：记忆长度与压缩触发

[Mermaid 图表 — 请在在线版本中查看]



## 优化技巧表

| 技巧         | 效果     |
|------------|--------|
| 删冗余修饰语     | -token |
| 用指针代替粘贴    | -一大段   |
| 重复事实只存一处   | -重复注入  |
| 升仓到 README | 记忆卡片变薄 |

## 与第 5 篇：SYSTEM\_PROMPT\_DYNAMIC\_BOUNDARY

记忆注入应落在边界之后，避免：

- 污染可缓存前缀；
- 让静态段频繁失效。

## 练习

1. 取一条你的记忆描述，尝试压缩到 **50% 字数**而不失语义。
2. 解释为何「注入 5 条长文」比「注入 2 条短文」更易触发 Tier2。

## FAQ

- Q：能不注入记忆吗？**  
A：可关闭或清空候选；但会损失个性化。
- Q：记忆和 CLAUDE.md 会重复计费吗？**  
A：二者都在输入中；**重复内容 = 双倍浪费**。

## 小结

记忆与上下文是**零和博弈**：注入越多，留给工具输出与对话历史的余量越少。**精确度优先 + 短文描述 + 文档外置** 是正统解法。

## 附录：示例对比

### 冗长注入（反例）

### 关于包管理器的一些说明  
在之前的对话中我们多次讨论过，用户似乎更喜欢.....

### 精简注入（正例）

### 包管理：pnpm  
使用 pnpm；勿生成 npm 命令。

## Mermaid：与 Tier1 的并行关系

[Mermaid 图表 — 请在在线版本中查看]

记忆块通常不由 Tier1 当垃圾删——更需源头控制长度。

## 监控字段建议

| 字段               | 用途     |
|------------------|--------|
| injection_tokens | 成本归因   |
| claude_md_tokens | 是否文档过长 |
| messages_tokens  | 对话主体   |

## 与多会话

每开新会话：

- 消息清空重来；
- 记忆仍可能被检索注入 → 跨会话成本基线存在。

## 反模式

| 反模式                                | 后果             |
|------------------------------------|----------------|
| 把日志贴进记忆描述                          | 注入像 Tier2 失败现场 |
| 记忆与 <a href="#">CLAUDE.md</a> 双份维护 | 同步地狱           |
| 依赖记忆保存密钥                           | 安全与体积双输        |

## 术语

| 英文               | 中文   |
|------------------|------|
| injection budget | 注入预算 |
| dynamic tail     | 动态后缀 |

## 与产品演进

若未来支持「按模型上下文上限自适应 K」：

- 小窗口模型 → K 从 5 降到 3；
- 仍应坚持 **精确度优先**。

## 速算

假设每条记忆 ~150 tokens, 5 条 ≈ 750  
750 / 200000 ≈ 0.375% 窗口 - 看似少，  
但与会话消息相加后，60% 警戒线很快到达。

## 团队政策示例

单条记忆 description ≤ 120 英文词或等价中文；超出请写文档链接。

## 与 8.9 成本篇

记忆注入推高有效输入，在可比单价下费用随体积上升——与 30K vs 150K 的直觉一致。

## 反思

你愿意为「个性化」支付多少每轮固定 token 税？写入 `CLAUDE.md` 共享是否更划算？

## 9.9 持久化：`claude -c`、会话边界与三层记忆的寿命

聊天窗口像沙滩写字，潮水一来就淡；`CLAUDE.md` 像石碑；自动记忆像你家抽屉里的便签——各有时限，各守其位。

## 本节学习目标

- 1. 说明 会话记忆的默认寿命：关闭/新会话后丢失，除非使用延续标志（教学：`claude -c`，以 CLI 文档为准）。
- 2. 对比 三层记忆的持久化介质：RAM/会话文件 vs Git vs `~/.claude/...`。
- 3. 规划 关键信息迁移：会话 → 文件或 `CLAUDE.md` 或审后记忆。
- 4. 解释 `.gitignore` 对 `CLAUDE.local.md` 与本地记忆目录的意义。
- 5. 制定 备份策略：换机器时迁移哪些目录。

## 生活类比：三种笔记本的存档方式

| 类型    | 存档        |
|-------|-----------|
| 草稿纸会话 | 用完即弃      |
| 团队手册  | Git 远程仓库  |
| 私人便签  | 本机备份/加密磁盘 |

## Mermaid：数据寿命对比

[Mermaid 图表 — 请在在线版本中查看]

claude -c

(教学说明)

| 维度   | 说明                   |
|------|----------------------|
| 意图   | 延续或携带上下文/会话状态（以实现为准） |
| 用户价值 | 减少「新开窗口从头讲」的摩擦       |
| 注意   | 不等于无限磁盘；仍可能被压缩       |

### 伪 CLI 帮助

```
claude -c  # continue / carry (具体语义见官方 --help)
```

务必阅读你所安装版本的官方文档；此处为**机制占位**。

## 表：信息应落何处

| 信息     | 会话 | 自动记忆 | CLAUDE.md | Git 其他文档 |
|--------|----|------|-----------|----------|
| 当日调试细节 | 是  | 可    | 否         | 可选       |
| 团队命令   | 可  | 否    | 是         | 可        |
| 个人语气   | 可  | 是    | local     | 否        |
| 架构决策   | 摘要 | 否    | 是         | ADR      |

## 源码片段：会话导出（用户侧脚本思路）

```
#!/usr/bin/env bash
# 将重要结论人工粘贴到 SESSION.md 后执行
git add SESSION.md && git commit -m "chore: archive session notes"
```

自动化导出取决于客户端是否提供 API；人工+文件永远可用。

## Mermaid：迁移决策树

[Mermaid 图表 — 请在在线版本中查看]

## 换机清单

| 路径                                      | 是否迁移    |
|-----------------------------------------|---------|
| <code>~/.claude/CLAUDE.md</code>        | 通常要     |
| <code>~/.claude/projects/memory/</code> | 若要保留个性化 |
| 仓库内 <code>CLAUDE.md</code>              | 已随 Git  |
| <code>CLAUDE.local.md</code>            | 私人；自行拷贝 |

## 练习

- 列出你本周应升格到 `CLAUDE.md` 的三条信息。
- 写一条「团队规范」：会话结束前必须执行什么动作。

## FAQ

- Q：Git 会提交自动记忆吗？

A：默认在 home 目录，不在仓库；除非误配。
- Q：`-c` 能恢复工具输出吗？

A：不一定；Tier1 仍可能清理旧工具结果。

## 小结

持久化策略决定你不会「每次重来」：会话负责当下，`-c` 类机制缓解断裂，`CLAUDE.md` 固化团队真相，自动记忆保存个人偏好——三者分工明确，关键事实最终应上 Git。

## 附录：威胁模型简表

| 威胁    | 缓解      |
|-------|---------|
| 笔记本丢失 | 磁盘加密    |
| 记忆含敏感 | 审计+删除   |
| 仓库泄漏  | 密钥不进任何层 |

## 与合规删除

用户行使删除权时：

- 清 `~/.claude/projects/<id>/memory/*`
- 审查是否有云同步副本

## Mermaid：备份频率建议

[Mermaid 图表 — 请在在线版本中查看]

## 反模式

| 反模式                            | 后果    |
|--------------------------------|-------|
| 唯一真相只在聊天                       | 人走茶凉  |
| local 提交进远程                    | 隐私泄漏  |
| 从不 push <code>CLAUDE.md</code> | 团队不同步 |

## 术语

| 英文               | 中文   |
|------------------|------|
| persistence      | 持久化  |
| continue session | 延续会话 |

## 与 9.6 KAIROS

流水账持久化在磁盘：**做梦**读写同一介质——理解文件布局有助于排障。

## 多设备同步警示

若记忆目录被云盘双向同步：

- 可能冲突；
- 优先单写者或最后写入 wins 策略。

## 企业 SSO 场景

换账号可能导致 `~/.claude` 路径变化——确认**租户**隔离文档。

## 快速命令备忘

```
# 查看记忆目录（示意）
ls -la ~/.claude/projects/

# 搜索敏感词自检
rg -n "sk-|BEGIN RSA|password=" ~/.claude/projects/
```

## 与 9.10 练习篇衔接

在练习中你将**模拟一次换机**，列出必须拷贝的路径。



## 9.10 综合练习：从空白仓库到「可记忆」协作

像考驾照前的场地练习：把本篇机制跑一遍，才算真会。

### 本节学习目标

- 1. 完成 一套最小演练：创建 `CLAUDE.md`、模拟 `CLAUDE.local.md`、书写记忆卡片。
- 2. 自查 精确度优先：为同一查询设计 0/3/5 条注入的预期。
- 3. 演练 蒸馏：把流水账手工折叠成偏好/背景两类。
- 4. 对照 第 8 篇：估算记忆注入对 60% 警戒线 的影响。
- 5. 输出 团队规范草案（半页纸）。

### 练习 A：分层文件搭建（45 分钟）

#### 步骤

- 1. 新建仓库 `memory-lab`。
- 2. 添加根 `CLAUDE.md`（50 行内）：运行、测试、目录说明。
- 3. 添加 `.claude/CLAUDE.local.md`（3~10 行）个人偏好；`gitignore` 之。
- 4. 可选：添加 `packages/demo/CLAUDE.md` 目录级规则。

#### 验收标准

| 项     | 标准                            |
|-------|-------------------------------|
| 根文件   | <code>pnpm / npm</code> 命令可复现 |
| local | 不出现在 <code>git status</code>  |
| 目录级   | 仅对子包生效的描述清晰                   |

### 练习 B：记忆卡片写作（30 分钟）

写 5 张 JSON 卡片（虚构即可），其中 2 张 应被精确度优先策略拒绝注入。

```
[
  {
    "title": "具体可检索标题",
    "description": "1~3 句可执行描述",
    "should_inject_for_query_auth": true
  }
]
```

## 讨论题

- 哪两张太泛？
- 如何把泛标题改到可注入？

## Mermaid：练习 A 目录结构

[Mermaid 图表 — 请在在线版本中查看]

## 练习 C：双模型检索桌游（20 分钟）

三人一组：

1. 法官给出用户查询。
2. **Sonnet 扮演者**快速从 20 张卡片标题中选 ≤5。
3. **主模型扮演者**仅基于 5 张回复。
4. 复盘：漏了哪张关键卡？为什么标题没帮到？

## 练习 D：KAيروس 蒸馏（30 分钟）

给定流水账：

```
[09:10] 用户讨厌 any
[09:40] 用户说先别写测试（临时）
[10:05] 用户要求 pnpm
[10:06] 用户要求 pnpm（重复强调）
[11:00] 用户说 E2E 要 docker
```

**任务：**输出蒸馏后的 **preference / project** 卡片；标注丢弃了哪行及原因。

## 练习 E：上下文账簿（15 分钟）

假设：

- `CLAUDE.md` 折合 2.5K tokens
- 注入 5 条记忆各 200 tokens
- 消息 90K tokens

填写下表：

| 块                      | tokens |
|------------------------|--------|
| <code>CLAUDE.md</code> | 2500   |
| 记忆                     | ?      |
| 消息                     | 90000  |
| 合计                     | ?      |
| 占 200K 比例              | ?      |
| 是否过 60%?               | ?      |

## 练习 F：持久化剧本

描述场景：笔记本送修一周，你用备用机。

列出你会：

1. 拷贝哪些目录？
2. 无法恢复什么（若没 Git）？
3. `-c` 能否救你？

## 评分 rubric（自学用）

| 等级 | 标准                                   |
|----|--------------------------------------|
| 及格 | 分清三层记忆                               |
| 良好 | <code>CLAUDE.md</code> 分层正确 + 5 卡片规范 |
| 优秀 | 蒸馏+检索+token 账簿全对                     |

## 参考答案节选（练习 D）

| 流水行        | 蒸馏命运                      |
|------------|---------------------------|
| 讨厌 any     | preference: TypeScript 严格 |
| 先别写测试      | 丢弃（临时）                    |
| pnpm×2     | 合并一条 preference           |
| E2E docker | project 背景                |

## Mermaid：全篇知识串联

[Mermaid 图表 — 请在在线版本中查看]

## 团队规范草案模板

```
# 记忆与 CLAUDE 规范 v0.1
1. 团队真相只进 Git (CLAUDE.md / ADR)
2. 个人偏好进 local 或自动记忆
3. 记忆卡片 title 必须可检索
4. 单条 description ≤ 120 词
5. 每月审计自动记忆
6. 密钥永不写入任何层
```

## FAQ

**Q：必须全做吗？**

**A：**至少 A+B+D+E；C 适合工作坊。

**Q：没有 Claude Code 环境？**

**A：**纸面推演同样有效。

## 小结

---

练习把 **CLAUDE.md** 层级、自动记忆、双模型检索、精确度优先、KAIROS 蒸馏、上下文成本、持久化 穿成一条可执行链。完成本篇，你应具备搭建与治理记忆系统的基础能力。

---

## 附录：打卡清单

---

- [ ] 根 **CLAUDE.md**
  - [ ] **.gitignore** 含 local
  - [ ] 5 张规范卡片
  - [ ] 1 次蒸馏手写
  - [ ] 1 次 token 估算
  - [ ] 半页团队规范
- 

## 延伸阅读

---

- 第 8 篇全文：上下文与压缩。
  - 第 5 篇：动态边界与拼装。
- 

## 最后寄语

---

记忆系统的敌人不是「忘」，而是脏、重、乱。保持短、准、审，比追求「全记住」更接近专业用法。

---

## 附加挑战（可选）

---

用脚本统计你的 **CLAUDE.md** 行数：若在 200+，尝试删到 120 并保证信息不丢（通过链接到 **docs/**）。

---

## 与开源协作

---

对外开源前：

```
git check-ignore -v .claude/CLAUDE.local.md
rg -n "SECRET|TOKEN" CLAUDE.md
```

## 术语收官

| 英文     | 中文   |
|--------|------|
| rubric | 评分量规 |
| drill  | 演练   |

## 第10篇：多Agent系统

### 第 10 篇：多 Agent 系统 · 10.1 蜂群作战

路径：docs/part10-multi-agent/index.md

系列：Claude Code 完全指南 V2

#### 学习目标

完成本节学习后，你应该能够：

1. 解释为何在复杂工程任务中，单一大上下文 Agent 往往不如「蜂群式」多 Agent 分工。
2. 区分「一个人扛全场」与「项目经理 + 工人」两种心智模型，并对应到 Coordinator / Worker 等角色。
3. 说出 Claude Code 中通过 Task 工具 派生子 Agent 的基本机制，以及子 Agent 独立上下文、向父 Agent 返回蒸馏结果 的含义。
4. 建立对后续小节（六个内建角色、只读 Explore/Plan、Verification、缓存前缀、防递归）的整体路线图。

#### 生活类比：从「全能厨师」到「中央厨房」

想象一家餐厅在晚高峰要出两百份套餐。

- 单打独斗：一位厨师同时接单、洗菜、切配、炒菜、摆盘——短期可能撑住，但错误率上升、节奏混乱、无法并行。
- 蜂群作战：主厨（类似 Coordinator / 主 Agent）拆单：冷菜台、热灶、面点并行；需要改同一口锅里的配方时则串行排队，避免两人同时加盐。

软件仓库里的「改 Bug、跑测试、搜全库引用」同理：搜索与验证可以并行，改同一文件往往需要串行或明确分工，否则冲突与「口头交代不清」会指数级放大。

## 为什么需要多 Agent?

| 维度    | 单 Agent                 | 多 Agent（合理编排）                     |
|-------|-------------------------|-----------------------------------|
| 上下文   | 所有探索、计划、实现挤在同一窗口，易被噪声淹没 | 子任务独立窗口，父 Agent 只收摘要              |
| 并行度   | 本质串行思考                  | 可 Phase 并行 多路探索（受工具与策略约束）         |
| 专业度   | 同一套提示承担所有角色             | Explore / Plan / Verify 等角色专用系统提示 |
| 风险    | 「又写又评」易自我确认偏误           | Verification 与写代码方利益隔离            |
| 成本与缓存 | 长对话重复多                  | 统一 Fork 前缀 利于前缀缓存命中（见 10.8）       |

## 从「单打独斗」到「蜂群」：心智模型切换

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## Task 工具：生成子 Agent 的机制（概念层）

在 Claude Code 的抽象模型中，**Task 工具**是「生成子 Agent」的入口：父 Agent 不直接在自身上下文里完成所有探索，而是\*\* Fork 出具有独立上下文窗口的子 Agent；子 Agent 运行结束后，将蒸馏后的结果\*\*（例如文件路径列表、结论段落、测试输出摘要）返回给父 Agent 合并。

**源码/配置片段（示意）**——不同版本字段名可能略有差异，核心语义一致：



```
{
  "tool": "Task",
  "subagent_type": "explore",
  "description": "3-5 词摘要",
  "prompt": "给子 Agent 的完整任务说明（须含具体路径/行号约束时见 10.6）",
  "readonly": true
}
```

要点：

- `subagent_type` 选择 `explore / plan / generalPurpose / shell / ...` 等，决定能力与系统提示。
- `readonly: true` 时对应 `Explore / Plan` 一类只读子 Agent（详见 10.3、10.4）。
- 子 Agent 不能再嵌套生成子 Agent（防无限递归，见 10.9）。

## 子 Agent 独立上下文与「蒸馏返回」

| 概念    | 含义                                                               |
|-------|------------------------------------------------------------------|
| 独立上下文 | 子 Agent 的对话与工具轨迹与父级隔离，避免父窗口被中间步骤塞爆                               |
| 蒸馏结果  | 父 Agent 需要的是决策级信息（改哪些文件、证据链、测试是否红），而非每一次 <code>grep</code> 的原始输出 |
| 父级职责  | 合并多路子结果、解决冲突、安排串行/并行阶段                                           |

## 何时不该用多 Agent？

多 Agent 不是银弹。以下情况优先单 Agent或小步迭代：

- 改动范围极小（单文件数行）。
- 任务无法被拆成可验证的子目标（拆了就丢上下文）。
- 过度 Fork 导致汇总成本高于直接动手。

## 本篇路线图（第 10 篇 12 节）

| 小节    | 文件                                      | 主题                            |
|-------|-----------------------------------------|-------------------------------|
| 10.1  | <code>index.md</code>                   | 蜂群作战与 Task 机制总览               |
| 10.2  | <code>02-six-agents.md</code>           | 六个内建角色与能力/限制表                 |
| 10.3  | <code>03-explore-agent.md</code>        | Explore 只读探索                  |
| 10.4  | <code>04-plan-agent.md</code>           | Plan 只读规划                     |
| 10.5  | <code>05-coordinator.md</code>          | Coordinator 调度与并行/串行          |
| 10.6  | <code>06-anti-lazy.md</code>            | 反偷懒与工人意识注入                    |
| 10.7  | <code>07-verification-agent.md</code>   | Verification: Try to break it |
| 10.8  | <code>08-cache-optimization.md</code>   | Fork 前缀与缓存                    |
| 10.9  | <code>09-anti-recursion.md</code>       | 防无限递归                         |
| 10.10 | <code>10-message-routing.md</code>      | 父子消息路由                        |
| 10.11 | <code>11-swarm-vs-coordinator.md</code> | Swarm vs Coordinator          |
| 10.12 | <code>12-practice.md</code>             | 实践设计工作流                       |

## 小结

- 蜂群作战的本质是：分工、并行、可验证、可汇总。
- Task 工具负责 Fork 子 Agent；子 Agent 有独立上下文，向父 Agent 返回蒸馏结果。
- 主 Agent 应像项目经理一样：明确子任务边界，并为 Verification 与实现方留出利益隔离空间（第 10.7 节展开）。

## 自测 checklist

- [ ] 能否用「中央厨房」类比解释并行与串行的取舍？
- [ ] 能否说明「独立上下文」解决了什么问题？
- [ ] 是否理解「子 Agent 不能生成子 Agent」在架构上的必要性？

## 深入：复杂任务中的「证据链」

蜂群模式的一个隐藏收益是：每一类子 Agent 产出不同形态的证据。

| 证据类型 | 典型产出者        | 父 Agent 合并时检查什么             |
|------|--------------|-----------------------------|
| 存在性  | Explore      | 路径是否真实、引用是否完整               |
| 可行性  | Plan         | 阶段是否可执行、依赖是否闭环              |
| 正确性  | Worker       | 补丁是否落在约定行号/文件               |
| 外部性  | Verification | Build/Test/Lint/HTTP 是否客观失败 |

父 Agent 若跳过某一类证据（例如只有代码 diff 没有测试），系统整体置信度应主动降级，并在对用户汇报时标明 **PARTIAL**（与 10.7 对齐）。

## 与「子 Agent 缓存」的前瞻关系（10.8 预告）

所有 Fork 任务若统一以固定前缀开头（例如 `Fork started - processing in background`），推理服务侧可对该前缀做字节级前缀匹配，使大量结构相似的子任务在首 token 段复用缓存。你在编写 `description` 或系统级模板时，应刻意统一该字符串，而不是每条任务换一种寒暄语。

```
# 推荐（统一前缀，利于缓存）
Fork started - processing in background: 搜索 auth 模块中的 refresh token 用法

# 不推荐（前缀碎片化）
【子任务】帮我看看 auth ...
后台任务开始: auth ...
```

## 防递归与组织边界（10.9 预告）

若允许「子 Agent 再派生子 Agent」，会出现：

- 1. 深度爆炸：每层都复制一套工具与提示，成本不可控。
- 2. 责任模糊：失败时难以判定是哪一层指令含糊。
- 3. 死锁风险：循环委派「你再去派一个人看看」。

因此平台层通常硬约束：子 Agent 不得再调用 Task 生成孙 Agent。父 Agent 必须扁平化拆分任务。

[Mermaid 图表 — 请在在线版本中查看]

## Coordinator 模式的直观结构（10.5 预告）

在 Coordinator 心智下，主 Agent 不急于亲自改代码，而是：

- **Phase 1**：并行派出多只 **Worker**（或 Explore）做只读/宽搜索，收敛候选文件集合。
- **Phase 2**：主 Agent 根据蒸馏结果锁定修改列表与顺序。
- **Phase 3**：对不同文件可并行；对同一热点文件应串行派工，避免冲突。

[Mermaid 图表 — 请在在线版本中查看]

## 「反偷懒」在总览层的含义（10.6 预告）

主 Agent 常犯的一类错误是：给子 Agent 一句「根据你的发现修 Bug」。这在工程上等于把责任推给子上下文，且子 Agent 没有你的全局产品约束。正确做法是：文件路径 + 行号范围 + 期望行为 + 复现步骤（或测试名）。工人意识注入则强调：Fork 出的 Worker 不是经理，禁止再委派、少问多做。

## Verification 为何是「全场最佳设计」之一（10.7 预告）

Verification 的核心口号是：Try to break it。它被要求：

- 严禁「只读代码就认为 OK」。
- 强制跑 Build / Test / Lint；前端可走浏览器自动化；后端可用 curl 实测。
- 使用 adversarial probes（对抗性探测）挑战边界条件。
- 输出 PASS / FAIL / PARTIAL，且与写代码 Agent 利益隔离（不能互相「给面子」）。

## 术语速查表

| 术语   | 解释                               |
|------|----------------------------------|
| Fork | 通过 Task 创建子 Agent，独立上下文          |
| 蒸馏结果 | 压缩后的结论文本/结构化要点，回传父级              |
| 只读模式 | 不可创建/修改/移动文件；Bash 受限             |
| 利益隔离 | Verification 与实现方目标对立统一：找茬 vs 修码 |
| 前缀缓存 | 相同起始字节序列命中 KV 缓存，降低时延与费用         |

## 常见误解

- 误解 1：**「子 Agent 越多越好。」

**正解：**子 Agent 数量上升会带来**汇总与一致性成本**；应有**阶段门控**。
- 误解 2：**「Explore 可以顺便改个小文件。」

**正解：**Explore/Plan 只读是**安全与职责设计**，不是建议项（10.3、10.4）。
- 误解 3：**「Verification 看一下 diff 就行。」

**正解：**必须以**可执行检查**为主，代码审查为辅（10.7）。

## 迷你案例：登录异常流量上升

- Phase 1（并行）：**Worker A 搜 `login` 相关路由；Worker B 搜限流中间件；Explore 列 `auth` 包目录与最近改动。
- Phase 2（主 Agent）：**合并路径清单，锁定 `middleware/rate_limit.go:88-120` 与 `handlers/login.go:40-55`。
- Phase 3（串行改不同文件或同文件排队）：**Worker1 改限流；Worker2 改 handler（若同文件则必须串行）。
- Verification：**跑单测 + `curl` 登录接口 + adversarial 探测（错误密码、空 body、超大 payload）。
- 判决：**若 Lint 红但功能绿 → **PARTIAL**，要求修复 Lint 后再 PASS。

## 与 Swarm 模式的对照（10.11 预告）

Swarm 更强调对等与自治的多路探索；Coordinator 更强调中心化调度与阶段门。二者可混合：早期 Swarm 宽搜，后期 Coordinator 收口。详见 `11-swarm-vs-coordinator.md`。

## 延伸阅读（本仓库内）

- [10.2 六个 Agent 角色图](#)
- [10.10 消息路由](#)
- [10.12 实践](#)

## 参考文献与实现提示（抽象）

以下名称随产品迭代可能调整，但语义稳定：

- **Task**：子 Agent 启动器。
- **subagent\_type**：角色选择器。
- **readonly**：只读沙箱开关（Explore/Plan）。

集成到你的内部文档时，请绑定**当前版本**的 JSON Schema 或官方说明链接。

下一节：[10.2 六个 Agent 角色图](#)

# 10.2 六个内建 Agent 角色全图

系列：Claude Code 完全指南 V2 · 第 10 篇

## 学习目标

1. 默写六个内建 Agent 的名称及其**主要职责**。
2. 对照表格说明每个角色在文件系统、**Bash**、子 Agent 上的能力边界。
3. 解释为何 **Explore / Plan** 必须 **只读**，以及 **Verification** 为何要与写代码路径隔离。

4. 选择在真实任务中应启用哪个 `subagent_type`，避免「用 Plan 去改代码」这类错配。

## 生活类比：剧组分工

一部电影需要：采景员（**Explore**，只看不拆景）、编剧统筹（**Plan**，写分镜不动摄影机）、执行导演（**Coordinator**）、场工搬运搭景（**Worker**）、替身试险（**Verification**，专找穿帮）、以及什么都能干一点的通用 **Agent**。没人会要求采景员当场改搭建——那是职责越界。

## 六角色一览表

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## 总表：角色 × 能力 × 限制

| 角色              | 英文标识                              | 核心职责                    | 文件创建/修改/移动       | Bash 典型范围                                        | 能否再派子 Agent             |
|-----------------|-----------------------------------|-------------------------|------------------|--------------------------------------------------|-------------------------|
| 通用 Agent        | <code>generalPurpose</code>       | 研究、实现、执行多步任务            | 按策略通常允许          | 较宽（依版本策略）                                        | 否（子 Agent 禁止递归，见 10.9)  |
| Explore 探索      | <code>explore</code>              | 快速定位文件、符号、调用链           | 禁止（只读模式）         | 仅 <code>ls</code> 、 <code>git status</code> 等白名单 | 否                       |
| Plan 规划         | <code>plan</code>                 | 架构权衡、阶段划分、风险清单          | 禁止（只读模式）         | 仅白名单只读命令                                         | 否                       |
| Verification 验证 | <code>verification</code> （若版本提供） | Build/Test/Lint、对抗探测、判决 | 依产品：常为读+执行测试     | 测试、curl、自动化脚本等                                   | 否                       |
| Coordinator 协调器 | <code>coordinator</code>          | 拆分阶段、并行/串行派工、汇总         | 可调度他人改；自身策略因版本而异 | 调度为主                                             | 通过 Task 派子 Agent（仅父级语境） |
| Worker 工人       | <code>worker</code> / 派工语境        | 被指派的具体活：搜/改/跑           | 通常允许在指派范围内修改     | 任务相关命令                                           | 否                       |

**注意：**上表为**教学抽象**。实际以你所用 Claude Code 版本的 **工具策略与 JSON Schema** 为准；尤其是 Verification 的文件写入权限可能收紧为「仅报告」。

## 各角色详解（简明版）

### 1. 通用 Agent (generalPurpose)

- **定位：**默认「全能型」子 Agent，适合**中等复杂度**、边界不那么极端的任务。
- **适用：**单路径功能开发、重构局部模块、联调脚本。



- **风险**：若任务含糊，容易与主 Agent **重复劳动**；应配合 10.6 的**明确派工**。

示意调用（字段名随版本调整）：

```
{
  "tool": "Task",
  "subagent_type": "generalPurpose",
  "description": "实现订单取消 API",
  "prompt": "在 services/order/cancel.go 第 40-95 行附近实现... (须含验收标准)"
}
```

---

## 2. Explore 探索 (explore · 纯只读)

- **定位**：代码库 GPS，不负责落地修改。
- **硬约束**：不能创建/修改/移动文件；Bash 只允许 `ls`、`git status` 等只读白名单。
- **典型产出**：受影响文件列表、相关符号、目录结构、Git 状态摘要。

---

## 3. Plan 规划 (plan · 纯只读)

- **定位**：调研与方案，强调「想清楚再动手」。
- **硬约束**：与 Explore 同属 **readonly** 阵营：不动文件系统；Bash 只读。
- **典型产出**：分阶段计划、风险、回滚策略、测试清单。

---

## 4. Verification 验证

- **定位**：Try to break it 的独立裁判。
- **核心**：严禁只看代码；强制 Build/Test/Lint；前端浏览器自动化；后端 `curl`；`adversarial probes`。
- **输出**：PASS / FAIL / PARTIAL；与写代码 Agent **利益隔离** (10.7)。

---

## 5. Coordinator 协调器

- **定位**：主 Agent 进入 **项目经理** 模式：拆阶段、派并行/串行 Worker。
  - **典型模式**：Phase1 三只 Worker 并行搜索 → Phase3 两只 Worker 串行改不同文件防冲突 (10.5)。
-

## 6. Worker 工人

- 定位：Fork 出来的执行者，不是经理。
- 意识注入：不要向用户/父级反复提问；直接工具干活；严禁再生成子 Agent（10.6、10.9）。
- 输入：必须是可执行说明：路径、行号、期望行为、完成定义。

### 角色选择决策表

| 用户/主 Agent 的意图  | 首选子 Agent                | 次选                             |
|-----------------|--------------------------|--------------------------------|
| 「这仓库里谁调用了 X? 」  | Explore                  | generalPurpose（仅当 Explore 不可用） |
| 「给我三期迭代方案和风险」   | Plan                     | 主 Agent 自规划（小任务）               |
| 「实现并自测」         | Worker + 末尾 Verification | generalPurpose                 |
| 「专门挑刺、跑 CI 等价物」 | Verification             | 外部 CI                          |
| 「大型改造分阶段推进」     | Coordinator              | 主 Agent 强自制                    |

### 源码片段：只读标志在调用层的表达

```
{
  "tool": "Task",
  "subagent_type": "explore",
  "readonly": true,
  "description": "Fork started – processing in background: 定位 payment 引用",
  "prompt": "只读搜索：列出所有引用 payment.Service 的 Go 文件路径..."
}
```

```
{
  "tool": "Task",
  "subagent_type": "plan",
  "readonly": true,
  "description": "Fork started – processing in background: 规划迁移步骤",
  "prompt": "在只读前提下阅读目录结构与接口定义，输出三阶段迁移计划..."
}
```

`readonly: true` 与 Explore/Plan 的系统提示共同保证：工具层拒绝写操作（具体错误信息因实现而异）。

## 错误配型示例（反模式）

| 反模式                          | 为何糟糕          | 应改为                      |
|------------------------------|---------------|--------------------------|
| Plan 里写「顺便改掉这个 Bug」          | 违反只读职责        | Worker + 明确行号            |
| Explore 里要求「重构 utils」        | Explore 不能改文件 | 先 Explore 列点，再派 Worker   |
| Verification 与 Worker 同上下文合并 | 利益不隔离         | 分两次 Task，Verification 独立 |
| Worker 收到「基于发现修」             | 指令模糊（10.6）    | 父 Agent 先合并 Explore 输出   |

## 与 Task 工具的关系（复习）

[Mermaid 图表 — 请在在线版本中查看]

子 Agent 独立上下文：每个 Task 调用相当于新会话，父会话只保留摘要边。

## 安全与合规视角

| 维度     | Explore/Plan | Worker/通用             |
|--------|--------------|-----------------------|
| 误删文件风险 | 低（写拒绝）       | 需代码审查与测试              |
| 秘密泄露   | 仍可能打印环境变量    | 同左；Verification 日志需脱敏 |
| 供应链    | 只读降低篡改       | 修改依赖需额外门禁             |

## 小结

- 六角色覆盖 搜、想、干、验、调度、通用 全链路。
- Explore + Plan = 只读双雄，Bash 白名单（ls / git status）。
- Verification = 外部化质检，PASS/FAIL/PARTIAL。
- Worker = 派工执行体，配合 反偷懒 与 防递归 使用。

## 自测

- 1. 为何 Plan 不能改文件？从**职责分离与安全**各答一点。
- 2. 何时选 Coordinator 而非 generalPurpose?
- 3. Verification 与「写代码子 Agent」为何要**利益隔离**？

上一节: [10.1 蜂群作战](#) · 下一节: [10.3 Explore](#)

## 10.3 Explore 探索专家（只读）

系列: Claude Code 完全指南 V2 · 第 10 篇

### 学习目标

- 1. **陈述** Explore Agent 的 **只读** 语义：不得创建、修改、移动文件。
- 2. **列举** Bash 允许的**白名单**命令（如 `ls`、`git status`）及设计原因。
- 3. **设计**适合派给 Explore 的任务描述模板，使父 Agent 能**合并蒸馏**结果。
- 4. **对比** Explore 与 generalPurpose：何时必须**强制只读**以避免副作用。

### 生活类比：博物馆导览员

Explore 像**导览员**：可以带你走遍展厅、指给你看展品编号与动线，但**不能触碰展品**、更不能把展品换位置。代码库就是展馆：**读**是本职工作，**写**属于「布展队」(Worker)的事。

### Explore 在蜂群中的位置

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## 能力/限制总表

| 类别      | Explore 行为                                         | 说明                          |
|---------|----------------------------------------------------|-----------------------------|
| 读文件     | 允许（通过 Read 等工具）                                    | 核心能力                        |
| 创建文件    | 禁止                                                 | 只读模式硬约束                     |
| 修改文件    | 禁止                                                 | 同上                          |
| 移动/重命名  | 禁止                                                 | 同上                          |
| Bash    | 仅 <code>ls</code> 、 <code>git status</code> 等只读白名单 | 防 <code>rm -rf</code> 、防意外写 |
| 子 Agent | 禁止再 Task                                           | 防递归（10.9）                   |

## 典型用途清单

| 场景        | 派 Explore 的价值 | 父 Agent 下一步         |
|-----------|---------------|---------------------|
| 新人接手仓库    | 快速得到模块地图      | Plan 或 Worker 分阶段切入 |
| Bug 定位前宽搜 | 收敛相关文件集合      | 明确行号后派 Worker       |
| 重命名影响分析   | 列举引用点         | 设计迁移顺序              |
| 合规审计只读    | 不产生写审计噪音      | 输出报告                |

## 任务描述模板（推荐给父 Agent）

Fork started – processing in background: Explore 只读任务

【目标】列出所有引用 `package auth` 中 `ValidateToken` 的位置。

【约束】

- 只读：不得创建/修改/移动任何文件。
- Bash 仅允许：ls、git status（如需）。

【输出格式】

1. 文件路径列表（按目录分组）
2. 每文件一行摘要：函数名或调用上下文
3. 若不确定，标注「需人工确认」而非猜测修改

统一前缀 Fork started — processing in background 用于 10.8 缓存优化。

## 源码片段：Task 调用示例

```
{
  "tool": "Task",
  "subagent_type": "explore",
  "readonly": true,
  "model": "fast",
  "description": "Fork started – processing in background: 搜 ValidateToken",
  "prompt": "只读探索：...（粘贴上文模板）"
}
```

`model: "fast"` 为可选：宽搜类任务可用更快模型降低成本（以产品支持为准）。

## Explore 不应做的事（反模式）

| 反模式                      | 后果           | 修复                         |
|--------------------------|--------------|----------------------------|
| 「顺手修一下格式」                | 违反只读         | 另开 Worker                  |
| 运行 <code>npm test</code> | 可能非白名单/非预期   | 交给 Verification 或 shell 专类 |
| 没有输出结构                   | 父 Agent 难以合并 | 强制表格/Markdown 结构           |
| 猜测不存在路径                  | 误导后续         | 标注置信度与搜索范围                 |

## 与 Plan 的边界

| 维度   | Explore   | Plan      |
|------|-----------|-----------|
| 重心   | 在哪、有哪些    | 怎么拆、风险是啥  |
| 典型工具 | 搜索、目录、读片段 | 读 + 结构化推理 |
| 产出   | 清单、地图     | 阶段、里程碑、回滚 |

正确流水线：Explore 给地图 → Plan 给路线图 → Worker 施工 → Verification 验收。

## 深入：为何 Bash 收紧到 ls / git status?

1. 最小权限：Explore 的目标是观测，不是操作环境。
2. 可复现：`git status` 提供变更面快照，利于与 CI 对齐。
3. 防偷懒：避免 Explore 用一行脚本「代替思考」，绕过结构化搜索。

[Mermaid 图表 — 请在在线版本中查看]

## 父 Agent 合并 Explore 结果的检查表

- [ ] 每条路径是否真实存在（抽样 Read 验证）
- [ ] 是否混入了需要写操作才能验证的假设
- [ ] 是否遗漏 测试目录 / 生成代码 目录（应要求 Explore 显式说明搜索范围）
- [ ] 输出是否可直接转化为 Worker 派工单（路径 + 行号 + 期望）

## 案例简叙：「支付回调重复入账」

1. Explore：只读列出 `webhook`、`ledger`、`idempotency` 相关文件与调用边。
2. 父 Agent：从报告中圈定 `internal/payment/webhook.go:120-180`。
3. Worker：在圈定范围内实现幂等键校验。
4. Verification：curl 重放 + 对抗性重复 POST。

若在步骤 1 就让 Explore 「试着改一下」，会破坏证据链且可能违反工具策略。

## 小结

- Explore = 只读 GPS，不能动文件，Bash 白名单。
- 产出应结构化，便于父 Agent 派工与 Verification 对照。
- 与 Plan 分工：地图 vs 路线图。

## 自测

1. 为何 Explore 禁止 `echo > file` ？
2. 给出两个适合 Explore 的任务标题（description）。
3. Explore 输出中最重要的三列应是什么？

## 附录：Explore 输出字段示例（可复制）

| 文件路径                              | 行号范围  | 符号/上下文                  | 备注           |
|-----------------------------------|-------|-------------------------|--------------|
| <code>internal/auth/jwt.go</code> | 45-78 | <code>ParseToken</code> | 核心解析         |
| <code>cmd/api/main.go</code>      | 12-30 | 中间件挂载                   | 需与 Worker 对齐 |

上一节: [10.2 六角色](#) · 下一节: [10.4 Plan](#)

# 10.4 Plan 规划专家（只读 · 调研不动手）

系列：Claude Code 完全指南 V2 · 第 10 篇

## 学习目标

1. 说明 Plan Agent 与 Explore 同为 **readonly**，但在产出形态上偏 **阶段、风险、依赖**。
2. 运用「调研不动手」原则：Plan 只输出可执行计划，不直接改仓库。
3. 编写可被 Worker/Verification 消费的测试清单与完成定义（DoD）。
4. 识别何时应用 Plan 而非让主 Agent 边想边改。



## 生活类比：建筑设计师与施工队

Plan 像**建筑师画施工图**：可以现场勘测、读规范、标尺寸，但**不会**在图纸上把墙砌起来。砌墙是 **Worker**；验收是 **Verification**。若建筑师边画边拆墙，图纸与现场将永远对不上——对应到软件就是计划与 diff 纠缠，难以复盘。

## Plan 在流水线中的角色

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## Plan vs Explore 对照表

| 维度    | Explore                                     | Plan                      |
|-------|---------------------------------------------|---------------------------|
| 主要问题  | 「东西在哪？」                                     | 「先做什么、后做什么？」              |
| 典型输出  | 路径列表、调用图文字描述                                | 分阶段计划、依赖排序                |
| 只读    | 是                                           | 是                         |
| Bash  | <code>ls</code> 、 <code>git status</code> 等 | 同左（教学抽象）                  |
| 是否给行号 | 常给                                          | 可引用 Explore 行号并 <b>复核</b> |

## Plan 的标准输出结构（推荐）

父 Agent 应要求子 Plan 按固定 Markdown 骨架返回，便于机器与人阅读：

```
## 目标重述
(一句话)

## 现状摘要
- 相关模块: ...
- 已知约束: ...

## 阶段划分
### Phase 1 - ...
- 输入: ...
- 输出: ...
- 风险: ...

### Phase 2 - ...
...

## 测试与验收
- 单元测试: ...
- 集成: curl / 浏览器场景 ...
- adversarial: ...

## 回滚策略
...

## 开放问题
(需用户决策的点, Plan 不擅自假设)
```

---

## 源码片段: Task 调用

---

```
{
  "tool": "Task",
  "subagent_type": "plan",
  "readonly": true,
  "description": "Fork started - processing in background: 规划缓存迁移",
  "prompt": "在只读前提下阅读 docs/ 与 pkg/cache/, 输出三阶段迁移计划, 不得修改任何文件。必须包含测试计划"
}
```

---

## 「调研不动手」的纪律

| 行为                     | 是否允许 | 说明       |
|------------------------|------|----------|
| Read 多个文件归纳接口          | 允许   | 调研       |
| 建议重命名符号                | 允许   | 文字建议     |
| ApplyPatch 修改          | 禁止   | Worker 事 |
| 写临时脚本落盘                | 禁止   | 破坏只读     |
| 输出「应由 Worker 执行的精确指令块」 | 允许   | 移交施工     |

[Mermaid 图表 — 请在在线版本中查看]

## Plan 与 Coordinator 的协同

Coordinator 可把 Plan 的输出当作 Gantt 的逻辑版：

| Plan 输出          | Coordinator 动作         |
|------------------|------------------------|
| Phase 1 仅只读搜索    | 并行派多个 Explore/Worker 搜 |
| Phase 3 改多个不相交文件 | 可并行 Worker             |
| Phase 3 改同一文件    | 串行或单 Worker            |

## 反模式

| 反模式                  | 为何有害              | 替代            |
|----------------------|-------------------|---------------|
| Plan 内「我帮你改了 README」 | 违反只读              | Worker        |
| 计划过于宏大不可验收           | Worker 迷失         | 每 Phase 附 DoD |
| 无测试计划                | Verification 无法对齐 | 显式 curl/用例名   |
| 与 Explore 重复劳动       | 浪费 token          | Explore 先收敛范围 |

## 深入：Plan 如何服务 Verification?

---

在计划中提前写明：

1. 必须绿的命令：`go test ./...`、`pnpm lint` 等。
2. 对抗性场景：空输入、超大 body、重复请求。
3. PARTIAL 的允许条件（例如仅文档未更新但核心绿）。

这样 Verification 的 PASS/FAIL/PARTIAL 与 Plan 对齐，减少扯皮。

---

## 案例骨架：API 版本化

---

1. **Explore**：列出现有路由与中间件文件。
2. **Plan**：Phase1 加兼容层；Phase2 切换默认版本；Phase3 删废弃；每阶段附测试与监控指标。
3. **Worker**：严格按 Phase 派工。
4. **Verification**：Try to break it + curl 新旧路径。

Plan 全程只读；若 Phase 需要样例 JSON，用文字给出，不写文件。

---

## 父 Agent 审核 Plan 的检查表

---

- [ ] 每个 Phase 是否有明确输出物（文件/接口/配置键）？
  - [ ] 依赖是否无环？
  - [ ] 测试是否覆盖 happy path + 至少两条边？
  - [ ] 是否把「应派给 Worker 的行号」从 Explore 接力过来？
- 

## 与「反偷懒」的关系（10.6 预告）

---

Plan 可以写得很漂亮，但若父 Agent 给 Plan 的输入是「你看着办」，Plan 会泛化。父 Agent 应提供：业务目标 + Explore 的硬证据 + 约束（时间/兼容）。

---

## 小结

---

- Plan = 只读战略家；Explore = 只读侦察兵。

- 产出聚焦 阶段、风险、测试、回滚，不落地修改。
- 与 Coordinator / Worker / Verification 形成 想→干→验 闭环。

---

## 自测

---

1. Plan 与 Explore 各回答哪类问题？
2. 为什么 Plan 不应写临时文件？
3. 写出你心目中 Phase 的「完成定义」三要素。

---

上一节: [10.3 Explore](#) · 下一节: [10.5 Coordinator](#)

---

## 10.5 Coordinator 协调器（项目经理调度）

---

系列: Claude Code 完全指南 V2 · 第 10 篇

---

### 学习目标

---

1. 描述 Coordinator 模式下主 Agent 的项目经理心智：拆阶段、派工、汇总。
2. 设计 Phase1 并行宽搜与 Phase3 串行修改策略，降低文件冲突概率。
3. 解释为何「改同一热点文件」应排队，而「改不相交文件」可并行。
4. 对接 Explore/Plan/Worker/Verification 在协调流中的时间序。

---

### 生活类比：工地总包

---

Coordinator 像总包项目经理：同时安排水电工、木工并行勘测（Phase1），但在同一面墙上不能两人同时开槽——必须串行（Phase3）。验收员（Verification）独立巡检，不站队某工种。

---

## Coordinator 模式总览

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## Phase 设计模板（推荐）

| 阶段     | 目的        | 并行度            | 典型子 Agent            |
|--------|-----------|----------------|----------------------|
| Phase1 | 宽搜、建候选集   | 高（3+ Worker）   | Explore / Worker 只读搜 |
| Phase2 | 锁定修改列表与顺序 | 单线程思考          | 主 Coordinator        |
| Phase3 | 落地修改      | 按文件不相交并行；相交则串行 | Worker               |
| Phase4 | 质检        | 独立             | Verification         |

## 典型调度：Phase1 三只 Worker 并行搜索

目标：尽快获得「全库与 Bug 相关的触点」。

Coordinator 指令（示意）

Phase1 – 并行（3 Task）：

- Worker A：搜索关键字 `timeout` 于 `services/` 下，返回路径+行号。
- Worker B：搜索 `retry` 于 `pkg/net/` 下。
- Worker C：Explore 只读列 `infra/` 与 `deploy/` 中与外部调用相关文件。

统一 description 前缀：Fork started – processing in background

合并规则：Coordinator 去重路径，标注冲突假设，再进入 Phase2。

## Phase3：两只 Worker 串行修改不同文件（防冲突）

当候选集中有两个不相交文件需修改时：

| 顺序 | Worker           | 文件                      | 说明                   |
|----|------------------|-------------------------|----------------------|
| 1  | Worker- $\alpha$ | api/handlers/foo.go     | 先完成并简短摘要             |
| 2  | Worker- $\beta$  | worker/processor/bar.go | 依赖 $\alpha$ 的接口契约时再派 |

若两文件无依赖，可并行两只 Worker；Coordinator 仍需在汇总时做接口一致性检查。

[Mermaid 图表 — 请在在线版本中查看]

若 同一文件：

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：并行 Task 伪代码（概念）

```
# 非真实语言，仅表达协调逻辑

parallel_tasks = [
    Task(subagent_type="worker", prompt="...搜 services/..."),
    Task(subagent_type="worker", prompt="...搜 pkg/net/..."),
    Task(subagent_type="explore", readonly=true, prompt="...列 infra/..."),
]
results = gather(parallel_tasks)
merge_plan = coordinator_merge(results)
serial_tasks = [
    Task(subagent_type="worker", prompt="改 api/handlers/foo.go 第 40-80 行..."),
    Task(subagent_type="worker", prompt="改 worker/processor/bar.go 第 10-55 行..."),
]
```

实际在 Claude Code 中由主 Agent 通过多次 Task 工具调用实现；是否真并行取决于运行时调度（教学上按「可同时发起」理解）。

## Coordinator 与 Plan/Explore 的分工

| 角色           | 贡献                |
|--------------|-------------------|
| Explore      | Phase1 只读候选，轻量    |
| Plan         | Phase2 前给出风险与阶段蓝图 |
| Coordinator  | 拍板并行度与顺序          |
| Worker       | 执行                |
| Verification | 判决                |

## 冲突类型与策略表

| 冲突类型 | 现象             | 策略                              |
|------|----------------|---------------------------------|
| 文件级  | 两 Worker 同改一文件 | 串行或单 Worker                     |
| 逻辑级  | A 改接口 B 未跟进    | Phase2 锁定契约再派工                  |
| 测试级  | 单元绿、集成红        | Verification <b>FAIL</b> ，回滚或补测 |
| 过程级  | 子任务描述含糊        | 反偷懒 (10.6) 重写 prompt            |

## 反模式

| 反模式                     | 后果           |
|-------------------------|--------------|
| Phase1 只派一只 Worker 全库盲跑 | 时延大、上下文噪声    |
| Phase3 并行改同一文件          | diff 冲突、逻辑撕裂 |
| 跳过合并直接派下一波              | 重复劳动/矛盾补丁    |
| Coordinator 亲自写大量代码     | 失去调度视角       |

## 与消息路由的关系 (10.10)

Coordinator 应要求子 Agent 返回**固定字段**：`summary` / `touched_files` / `open_questions`，便于父级**路由**到下一 Task。



## 案例简叙：配置热更新

1. Phase1: 三 Worker 并行搜 `config`、`viper`、`watch`。
2. Phase2: 合并后锁定 `internal/config/loader.go` 与 `cmd/agent/main.go`。
3. Phase3: 两文件可并行两 Worker（若团队策略允许）；若共享锁逻辑在同一函数，则串行。
4. Verification: `go test` + 对抗性「坏 YAML」输入。

## 小结

- Coordinator = 项目经理：并行宽搜、串行热点、合并蒸馏。
- Phase1 多路并行，Phase3 按文件相交关系决定并行或排队。
- 与 Verification 配合形成调度→执行→判决闭环。

## 自测

1. 何时 Phase3 必须串行？
2. Phase1 并行三只 Worker 的收益与成本各是什么？
3. Coordinator 输出合并时最少应检查哪三类信息？

上一节: [10.4 Plan](#) · 下一节: [10.6 反偷懒](#)

## 10.6 反偷懒机制与工人意识注入

系列: Claude Code 完全指南 V2 · 第 10 篇

## 学习目标

1. 定义主 Agent 分配任务时的模糊指令反模式（如「基于发现修 Bug」）及其危害。
2. 撰写合格派工单：路径 + 行号 + 期望行为 + 验收，禁止把「想清楚」甩给子 Agent。
3. 背诵工人意识注入要点：Fork 出来的工人，不是经理；不要交流提问；直接工具干活；严禁再生成子 Agent。

4. 关联反偷懒与 **Verification**：模糊输入 → 模糊实现 → **FAIL/PARTIAL** 概率上升。

生活类比：装修工单

若你对工人说「看着弄」，你会得到**随机风格的墙漆**。若你说「客厅东墙，色号 NN-01，两遍面漆，验收照 CIEC 标准拍照」，工人才能**一次性做对**。主 Agent 写子任务同理：**模糊指令 = 授权偷懒**。

反偷懒：主 Agent 的责任边界

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

禁止的模糊指令（示例表）

| 模糊说法          | 问题       | 改写方向                            |
|---------------|----------|---------------------------------|
| 「基于你的发现修 Bug」 | 发现不在父上下文 | 父 Agent <b>先合并 Explore</b> 再写行号 |
| 「优化一下性能」      | 无基线无指标   | 指定函数/接口 + pprof 场景              |
| 「重构这块」        | 范围不明     | 文件列表 + 禁止行为 + 允许行为              |
| 「对齐业界最佳实践」    | 不可测试     | 给出具体规范链接或规则条目                   |
| 「顺便修一下测试」     | 无测试名     | <code>TestFoo_Bar</code> + 期望断言 |

## 合格派工单模板（Worker / generalPurpose）

Fork started – processing in background: Worker 派工

【身份】你是 Fork 出来的工人，不是经理。不要向用户或父级反复提问；  
直接调用工具完成工作；严禁再生成子 Agent（Task）。

【范围】仅修改 `internal/billing/invoice.go` 第 142-201 行及同文件内直接必要的 import。

【问题】当 `TaxRate` 为 nil 时当前会 panic。

【期望】在计算总价前使用默认税率 0；记录 debug 日志一行（已有 logger）。

【禁止】不得修改 `go.mod`；不得重命名导出符号。

【验收】`go test ./internal/billing/...` 全绿；若红，自行修复至绿或回报精确错误块。

【输出】1) 变更摘要 2) 触及文件列表 3) 测试输出摘要

## 工人意识注入（原文可贴入 prompt）

以下段落建议由 父 Agent 在每次派 Worker 时原样或略作裁剪附带：

你是 Fork 出来的工人，不是经理。  
不要交流提问；在信息不足时基于仓库可见证据做最小合理假设并继续。  
直接工具干活：读、搜、改、跑测试。  
严禁再生成子 Agent（不要调用 Task 创建孙 Agent）。

## 为何「禁止子 Agent 再提问」？

| 原因    | 说明               |
|-------|------------------|
| 延迟    | 问答往返拉长 wall time |
| 上下文漂移 | 子窗口与父窗口易不一致      |
| 责任回滚  | 失败时难以判定「谁没问清」    |

例外：致命缺失（如密钥、产品二选一）可由子 Agent 单次在摘要中标注 **BLOCKED：需人类确认 X**，  
但不应展开多轮闲聊。

## 与 Explore/Plan 的衔接

| 步骤 | 负责方     | 产出      |
|----|---------|---------|
| 宽搜 | Explore | 路径+行号候选 |
| 战略 | Plan    | 阶段与风险   |
| 合并 | 父 Agent | 去模糊后的工单 |
| 施工 | Worker  | diff    |

父 Agent 禁止把「合并」甩给 Worker：那是经理活。

## 源码片段：父 Agent 内部 checklist（伪代码）

```
function dispatch_worker(explore_result, bug_report):
    require explore_result.paths.non_empty
    for each change:
        require change.file
        require change.line_range
        require change.expected_behavior
        require change.verification_command
    prompt = render_template(WORKER_TEMPLATE, change)
    Task(subagent_type="worker", prompt=prompt)
```

## Verification 对模糊派的「报复链」

[Mermaid 图表 — 请在在线版本中查看]

模糊派工 → 子 Agent 过度猜测 → 测试未覆盖分支 → adversarial probes 抓穿 → FAIL 或 PARTIAL。

## 反模式对照表（经理 vs 工人）

| 经理（父/Coordinator）应做 | 工人（Worker）应做 |
|---------------------|--------------|
| 合并 Explore 输出       | 按行号改         |
| 决定并行/串行             | 不调度他人        |
| 写清验收命令              | 跑命令并贴摘要      |
| 对接 Verification 结果  | 不「自评满分」      |

## 案例：从模糊到明确

模糊：「Worker：根据前面讨论修登录。」

明确：

- 文件： `auth/login.go`
- 行号： `88-112` （ `VerifyPassword` ）
- 期望：连续失败 5 次后锁定 15 分钟；错误信息不区分用户不存在与密码错误
- 验收： `go test ./auth -run TestLogin` + `curl` 样例两条

## 小结

- 反偷懒核心：主 Agent 去模糊，子 Agent 执行。
- 工人意识注入减少无效对话，硬禁子 Agent 再 Fork。
- 与 Verification 形成压力：含糊工单 → 更易 FAIL。

## 自测

1. 写出两句「模糊指令」并改写为「明确工单」。
2. 工人意识注入三条禁令是什么？
3. 为什么合并 Explore 结果不该由 Worker 代劳？

上一节： [10.5 Coordinator](#) · 下一节： [10.7 Verification](#)

# 10.7 Verification 验证专家 (Try to break it)

系列：Claude Code 完全指南 V2 · 第 10 篇

## 学习目标

- 1. 阐述 Verification 的核心原则：**Try to break it**，禁止「只读代码就认为 OK」。
- 2. 列举强制检查类型：**Build / Test / Lint**、前端浏览器自动化、后端 **curl 实测**、**adversarial probes**。
- 3. 解释 **PASS / FAIL / PARTIAL** 三种判决的适用条件。
- 4. 论证 Verification 与写代码 Agent **利益隔离**的必要性及落地方式。

## 生活类比：第三方质检与施工队

施工队 (**Worker**) 希望尽快交工；第三方质检 (**Verification**) 按规范泼水、加压、拆检样品。若质检员是施工队队友，「全是合格」就不可信——对应 **利益隔离**：验的人不能同时是主要改代码的人。

## Verification 在系统中的位置

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## 核心口号：Try to break it

| 层次    | 做什么              | 不做什么          |
|-------|------------------|---------------|
| 静态    | Lint、类型检查、安全扫描   | 只看风格不跑测       |
| 单元/集成 | 测试套件             | 假设「应该能过」      |
| 运行时   | curl、浏览器脚本       | 只读路由定义        |
| 对抗    | 畸形输入、重复请求、竞态简易模拟 | 只测 happy path |

## 强制检查清单（教学模板）

| 检查项         | 命令/手段示例                                               | 失败时                 |
|-------------|-------------------------------------------------------|---------------------|
| Build       | <code>go build ./...</code> 、 <code>pnpm build</code> | FAIL（若任务要求可构建）      |
| Test        | <code>go test ./...</code> 、 <code>pytest</code>      | FAIL                |
| Lint        | <code>golangci-lint</code> 、 <code>eslint</code>      | 依策略 FAIL 或 PARTIAL  |
| 前端 E2E      | Playwright / Cypress                                  | FAIL 若关键路径断         |
| 后端 HTTP     | <code>curl</code> + 断言状态码/JSON 字段                     | FAIL                |
| adversarial | 空 body、超大 JSON、非法编码                                   | 未处理则 FAIL 或 PARTIAL |

## 前端：浏览器自动化

Verification 应描述可重复的步骤，而非「我点了一下没问题」：

1. 启动本地 dev server（若已有则复用端口 3000）
2. 浏览器自动化：打开 `/login`
3. 输入测试账号（来自 `.env.test` 或文档约定）
4. 断言：URL 跳转到 `/dashboard`；控制台无 `error` 级日志
5. 截图或保存 trace（摘要写入报告）

## 后端：curl 实测

```
# 健康检查
curl -sS -o /tmp/h.json -w "%{http_code}" http://127.0.0.1:8080/health

# 对抗性：缺 Authorization
curl -sS -o /tmp/u.json -w "%{http_code}" -X POST http://127.0.0.1:8080/api/v1/orders

# 对抗性：重复幂等键（若实现要求）
curl -sS -H "Idempotency-Key: dup" -H "Content-Type: application/json" \
  -d '{"sku":"x","qty":1}' http://127.0.0.1:8080/api/v1/orders
```

Verification 输出应包含：HTTP 状态码、关键字段、错误体摘要。

## adversarial probes（对抗性探测）示例表

| 探针              | 目的     | 期望          |
|-----------------|--------|-------------|
| 空 JSON body     | 解析健壮性  | 400 + 明确错误码 |
| 字段类型错误          | schema | 400         |
| 超大 payload      | DoS 边界 | 413 或安全拒绝   |
| 重复提交            | 幂等     | 不重复记账       |
| 错误 Content-Type | 中间件    | 415/400     |

## 判决：PASS / FAIL / PARTIAL

| 判决      | 条件                                                         |
|---------|------------------------------------------------------------|
| PASS    | Build/Test（约定范围）全绿；关键 Lint 绿；约定的 curl/E2E/adversarial 探针通过 |
| FAIL    | 测试红、构建失败、关键 HTTP 错误、严重 Lint（策略定义）、安全探针击穿                   |
| PARTIAL | 核心功能绿但文档未更新、非关键 Lint 黄、部分探针未覆盖但已声明范围                       |

[Mermaid 图表 — 请在在线版本中查看]



## 利益隔离：为何是「全场最佳设计」之一？

| 风险   | 无隔离时         | 有隔离时                  |
|------|--------------|-----------------------|
| 确认偏误 | 实现者「看自己代码顺眼」 | 验证者专职找茬               |
| 测试偷懒 | 只写能过的用例      | Verification 独立跑全量与探针 |
| 报告失真 | 「应该没问题」      | 命令输出为证据               |

落地方式：

- 独立 Task：实现合并后，再派 **Verification** 子 Agent（或独立会话）。
- 提示词隔离：Verification 系统提示强调 **Try to break it** 与 **禁止盲信实现者摘要**。
- 证据优先：判决必须附 **可复现命令** 与 **输出片段**。

## 源码片段：父 Agent 两段式调度

```
# 阶段 A：实现
Task(subagent_type="worker", prompt="...实现...")

# 阶段 B：验证（独立）
Task(subagent_type="verification", prompt="
  对当前工作区做 Verification：严禁只读代码下结论。
  必须运行：go test ./...、golangci-lint run ./...
  必须 curl：/health 与 POST /api/v1/orders 的对抗性用例（见附录）
  输出：PASS/FAIL/PARTIAL + 证据
")
```

`subagent_type` 实际枚举以版本为准；若无专用 verification，可用 **shell** 专类 + 强提示词模拟，但仍须独立会话。

## Verification 报告模板

```
## 判决: FAIL

## 执行环境
- OS / Go 版本 / Node 版本: ...

## 命令与结果
1. `go test ./...` → 退出码 1; 失败包 `internal/billing`; 摘要: ...
2. `curl ...` → 500; 响应体摘要: ...

## adversarial
- 空 body → 500 (期望 400) → **FAIL 理由**

## 建议后续动作
- Worker 应修改 `internal/billing/invoice.go:...`
```

## 与 Coordinator 的衔接

Coordinator 在 Phase4 固定派出 Verification，并禁止与 Phase3 Worker 同 prompt 混写，以确保隔离。

## 反模式

| 反模式               | 后果   |
|-------------------|------|
| 「我看了 diff，逻辑对」    | 无效验证 |
| 只跑单测不跑集成          | 生产仍炸 |
| 实现 Agent 自报「测试过了」 | 不可采信 |
| PASS 但无命令输出       | 无法复盘 |

## 深入：PARTIAL 何时可接受？

在时间盒或范围裁剪下，团队可预先约定：

- 文档可后补 → **PARTIAL** + 列出缺失页
- 仅 Windows 环境未测 → **PARTIAL** + 声明

禁止用 PARTIAL 掩盖核心功能红测。

## 案例简叙：支付回调

1. Worker 实现幂等。
2. Verification: `go test` + `curl` 重放同一 `Idempotency-Key` + 乱序到达简易探针。
3. 若 Lint 红：依团队策略 `FAIL` 或 `PARTIAL`。
4. 输出 `PASS` 仅当全部约定绿。

## 小结

- Try to break it = 可执行检查 + 对抗探针 + 证据。
- `PASS/FAIL/PARTIAL` 必须可辩护。
- 利益隔离是可信度基石：验与写分身。

## 自测

1. 为何「只读代码」不能判 `PASS`？
2. 给出两个 `adversarial` 探针例子。
3. `PARTIAL` 与 `FAIL` 的边界如何界定？

上一节: [10.6 反偷懒](#) · 下一节: [10.8 缓存](#)

## 10.8 子 Agent 缓存优化（Fork 前缀匹配）

系列：Claude Code 完全指南 V2 · 第 10 篇

## 学习目标

1. 说明为何所有 Fork 任务应使用统一前缀 `Fork started - processing in background`。
2. 解释「字节级前缀匹配」如何提升缓存命中（KV 缓存 / 前缀复用）。
3. 设计在 `description` 与系统模板中稳定前缀 + 可变后缀的命名习惯。

4. 规避因寒暄语随机化导致的缓存碎片化。

生活类比：统一报工口令

工厂班组向调度台报到时，若每人用不同口头禅：「开工了老铁」「俺来了」「Go」——调度系统难以批量识别。若统一喊「Fork started — processing in background: {具体任务}」，前半完全重复，自动分拣与缓存预热都更便宜。推理侧同理：前缀一致才好「白嫖」缓存。

前缀匹配直观图

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

推荐格式

Fork started — processing in background: {3-5 词或短句描述具体子任务}

| 部分                                      | 是否固定 | 作用            |
|-----------------------------------------|------|---------------|
| Fork started — processing in background | 固定   | 前缀匹配、用户可见状态一致 |
| : 后短描述                                  | 可变   | 区分任务语义        |

反例：导致缓存碎片

[BG] 搜索 auth ...  
后台: 搜索 auth ...  
(Subagent) auth search ...  
🔧 正在 fork: auth

以上首字节即不同，前缀层难以合并。

## 在 Task 调用中的位置

```
{
  "tool": "Task",
  "subagent_type": "worker",
  "description": "Fork started – processing in background: 修改 invoice 空指针",
  "prompt": "...详细工单..."
}
```

| 字段          | 缓存相关性             |
|-------------|-------------------|
| description | 常出现在子会话首条或元数据，高相关 |
| prompt 首段   | 若模板统一前缀，同样有益      |

建议在 prompt 顶部也重复一行相同前缀（若产品不 dedupe）：

```
Fork started – processing in background: 修改 invoice 空指针

(以下正文...)
```

## 源码片段：批量派工模板

```
Coordinator 批量 Phase1:

- Task 1 description: Fork started – processing in background: 搜 services/payment
- Task 2 description: Fork started – processing in background: 搜 pkg/retry
- Task 3 description: Fork started – processing in background: Explore 列 deploy 清单
```

## 与模型路由的关系（概念）

部分部署会对 首 token 分布做 KV cache 复用：

| 实践         | 效果              |
|------------|-----------------|
| 统一系统提示     | 前缀稳定            |
| 统一 Fork 前缀 | 用户/工具注入段稳定      |
| 统一语言（中/英）  | 减少 tokenizer 差异 |

## 监控与 A/B（团队向）

| 指标        | 说明        |
|-----------|-----------|
| 子任务平均首包延迟 | 前缀优化后期望下降 |
| 重复前缀任务占比  | 应接近 100%  |
| 成本/token  | 间接指标      |

## 边界：前缀不是万能药

| 限制      | 说明                 |
|---------|--------------------|
| 后缀极长且迥异 | 仅前缀段复用             |
| 模型/版本切换 | tokenizer 与缓存策略可能变 |
| 隐私与合规   | 勿把密钥写进 description |

## 与防递归、消息路由的协同

- 防递归（10.9）：前缀统一不意味着可嵌套 Task；仍扁平派工。
- 消息路由（10.10）：返回结构固定 + 前缀固定，父 Agent 解析成本更低。

## 反模式表

| 反模式            | 后果      |
|----------------|---------|
| 每条任务换 emoji 前缀 | 命中差     |
| 把长 JSON 放在前缀前  | 前缀被污染   |
| 仅人眼可读、机器不稳定的文案 | 无法批量脚本化 |

## 案例：Phase1 三并行

前：三个 description 分别以「任务1」「帮我」「search」开头 → 缓存三路。

后：统一 `Fork started – processing in background:` → 共享前缀。

## 进阶：与「蒸馏返回」的衔接

子 Agent 返回父 Agent 时，若首段摘要也采用相同前缀族（例如 `Fork completed – summary:`），部分栈可对输出侧做类似优化——具体以服务商实现为准。教学重点是：结构化 + 可预测的起始 token 总优于随机散文。

| 阶段 | 推荐起始样式                                                |
|----|-------------------------------------------------------|
| 启动 | <code>Fork started – processing in background:</code> |
| 结束 | <code>Summary –</code> 或固定 JSON 键序                    |

## 小结

- 统一前缀 `Fork started – processing in background` 利用字节级前缀匹配提升缓存命中。
- description + prompt 顶部保持一致习惯。
- 避免碎片化寒暄。

## 自测

1. 写出正确与错误各一条 description。
2. 前缀固定、后缀可变各自解决什么问题？

3. 为何 emoji 不适合做前缀?

上一节: [10.7 Verification](#) · 下一节: [10.9 防递归](#)

# 10.9 防无限递归（子 Agent 不得生成子 Agent）

系列: Claude Code 完全指南 V2 · 第 10 篇

## 学习目标

- 1. 陈述平台约束: 子 Agent 不能生成子 Agent (无「孙 Agent」)。
- 2. 分析若允许递归委派, 会在成本、责任、死锁上产生哪些问题。
- 3. 掌握扁平化拆分: 由父 Agent 统一持有 Task 派发权。
- 4. 关联工人意识注入 (10.6): Worker 严禁再调用 Task。

## 生活类比: 俄罗斯套娃式派活

若经理派组长, 组长再派副组长, 副组长再派「临时负责人」, 最后没人真正搬砖, 只剩开会链。工程上对应 token 与 wall time 爆炸。正确做法是: 一个调度平面 (父/Coordinator) 扁平派工到执行面 (Worker)。

## 递归禁令示意图

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]



## 为何必须防无限递归？

| 问题   | 说明                        |
|------|---------------------------|
| 成本   | 每层独立上下文， <b>指数级</b> token |
| 延迟   | 串行等待子子任务， <b>墙钟时间</b> 不可控 |
| 责任   | 「谁的理解错了」在深层栈中 <b>不可审计</b> |
| 死锁   | A 等 B、B 等 A 的委派环          |
| 工具风暴 | 并发工具调用在深层放大               |

## Task 工具是「生成子 Agent 的机制」——但仅一层

父 Agent 调用 Task → 创建 子 Agent。  
子 Agent 侧 Task 应被策略 **拒绝** 或 **系统提示禁止**（以实际产品为准）。

正确：

Parent --Task--> Child --tools--> done --summary--> Parent

错误：

Parent --Task--> Child --Task--> GrandChild --> ...

## 扁平化拆分模式

| 原错误思路              | 扁平替代                                      |
|--------------------|-------------------------------------------|
| 「子 Agent 你再派一个人搜」  | 父 Agent <b>直接发</b> 第二个 Task               |
| 「让孙 Agent 写测试」     | 父 Agent 派 <b>Verification</b>             |
| 「子 Agent 内部再 plan」 | 父 Agent 先 <b>Plan</b> 只读，再派 <b>Worker</b> |

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：系统提示层禁令（示意）

```
You are a subagent. You MUST NOT invoke the Task tool or spawn subagents.  
If the task is too large, return a structured decomposition request for the parent agent.
```

工人模板（10.6）应与此一致。

## 与 Explore/Plan 只读的关系

只读角色更不能通过 Task 「找帮手改文件」——否则绕过只读。递归禁令与只读共同构成安全边界。

## 误用场景与纠正

| 误用                              | 纠正                           |
|---------------------------------|------------------------------|
| Worker prompt 写「你再开个子任务」        | 删除；改由父 Agent 拆               |
| 子 Agent 发现任务太大就 Task            | 返回 <b>NEEDS_SPLIT</b> 结构化块给父 |
| Coordinator 在子会话里再当 Coordinator | <b>单平面</b> ：仅父会话调度           |

## 边界：父 Agent 连续多次 Task 是否递归？

否。同一 父上下文 连续 **Task** 调用属于 **广度扁平**，不是深度递归。

[Mermaid 图表 — 请在在线版本中查看]

## 监控与排障

| 征兆      | 可能原因                             |
|---------|----------------------------------|
| 极深对话标题链 | 误用多会话代替单父调度                      |
| 重复搜索    | 子 Agent 无法委派，重复劳动 → 父应合并 Explore |
| 费用暴增    | 多个并行子任务无门控                       |

---

## 与缓存前缀（10.8）的协同

---

扁平化派工使 Fork 前缀统计更稳定；深层递归会导致难以批量优化且前缀语义混乱。

---

## 案例

---

错误：Worker 被提示「若不确定就再 spawn explore」。

正确：Worker 返回 **UNCERTAIN**：需在 pkg/foo 全量 grep，父 Agent 再派 Explore。

---

## 小结

---

- 子 Agent 不得生成子 Agent → 防无限递归。
  - Task 是子 Agent 创建机制，但应 单层使用。
  - 父/Coordinator 负责 广度拆分 与 汇总。
- 

## 自测

---

1. 为何「孙 Agent」会放大成本？
  2. 子 Agent 发现任务过大时应如何响应？
  3. 父 Agent 连发 3 个 Task 是否违规？
- 

## FAQ

---

Q：若产品 UI 仍显示「子 Agent 启动了子 Agent」怎么办？

A：以实际工具策略为准；教学上仍应按单层设计工作流，并在审计日志中区分「父连续 Task」与「嵌套 Task」。

Q：能否用「同一个会话多角色扮演」代替多 Task？

A：那是单上下文技巧，不是 Fork；缺少独立窗口与角色系统提示的隔离，不等价于子 Agent。

---

## 与六个内建角色的交叉表

| 角色                   | 递归风险点       | 缓解                            |
|----------------------|-------------|-------------------------------|
| Worker               | 想再派 Explore | 父扁平派 Explore                  |
| 通用                   | 任务过大想分包     | 返回 <code>SPLIT_REQUEST</code> |
| Coordinator（若跑在子上下文） | 误以为可继续嵌套    | 仅父会话持有调度权                     |
| Verification         | 不应再派实现者     | 只出判决与证据                       |

## 伪代码：检测非法嵌套（概念实现）

```
on_tool_call(agent, tool):
    if agent.depth >= 1 and tool == "Task":
        reject("subagents cannot spawn subagents")
```

真实系统可能以 策略引擎 或 提示词 实现；语义一致即可。

## 延伸阅读：与消息蒸馏的衔接

子 Agent 若试图用 Task 「甩锅」，会破坏 10.10 的星型拓扑。正确返回：

```
## 需要父 Agent 代为派发的子任务
- 类型：explore
- 范围：pkg/net/
- 理由：超出当前 token/时间盒
```

上一节：[10.8 缓存](#) · 下一节：[10.10 消息路由](#)

# 10.10 消息路由（父子 Agent 通信模式）

系列：Claude Code 完全指南 V2 · 第 10 篇

## 学习目标

- 1. 描述子 Agent 独立上下文窗口与父 Agent 的关系。
- 2. 解释「蒸馏结果」如何回传父 Agent 并进入下一轮决策。
- 3. 设计结构化返回字段，便于 Coordinator 做 merge。
- 4. 区分同步等待子任务与后台（视产品 UI）在用户感知上的差异。

## 生活类比：专案组与指挥部

子 Agent 像一线专案组：自己翻卷宗、取证、写笔录（工具调用）。指挥部（父 Agent）不看每一份原始复印件，只看摘要卷（蒸馏结果），再决定下一组警力派往何处。若专案组每天把全部复印件堆到指挥部，信息流会瘫痪——对应 上下文爆炸。

## 父子通信总览

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## 蒸馏结果应包含什么？

| 字段（建议）         | 用途                                   |
|----------------|--------------------------------------|
| summary        | 3-8 句结论                              |
| evidence       | 关键路径/行号/命令输出片段                       |
| touched_files  | 若子 Agent 可写                          |
| commands_run   | 便于 Verification 复现                   |
| open_questions | 需人类/父级决策                             |
| verdict        | 若子角色为 Verification：PASS/FAIL/PARTIAL |

父 Agent 不应要求子 Agent 粘贴 全文 `grep` 结果，除非调试需要。

## Task 工具与上下文隔离

子 Agent 独立上下文窗口 意味着：

- 子会话的 `system` + 前几轮 可能含 角色专用提示 (Explore/Plan/Verify)。
- 父会话 看不到 子会话中间工具细节，除非子 Agent 主动写入摘要。
- 返回蒸馏结果是协议上的必选义务（教学规范）。

```
{
  "child_return_shape": {
    "summary": "string",
    "evidence": ["path:line - snippet"],
    "open_questions": []
  }
}
```

## 路由模式表

| 模式      | 描述                 | 适用                 |
|---------|--------------------|--------------------|
| 单跳      | 父 → 一子 → 回父        | 小任务                |
| 星型      | 父 → 多子并行 → 父 merge | Coordinator Phase1 |
| 链式（仍扁平） | 父依次派子，非子再派子        | 串行改文件              |
| 验证旁路    | 父 → 实现子 → 父 → 验证子  | 利益隔离               |

## 源码片段：父 Agent merge 伪代码

```
results = []
for task in phase1_tasks:
    results.append(await Task(...))

merged_paths = dedupe(flatten(r.evidence.paths for r in results))
conflicts = detect_overlap(merged_paths)
dispatch_phase3(merged_paths, conflicts)
```

## UI/产品层：「Fork started — processing in background」

用户侧常看到统一前缀（10.8），提示子任务在后台处理；父 Agent 仍应在逻辑上 `await` 结果或显式分段汇报，避免用户以为「没反应」。

## 反模式

| 反模式              | 后果           |
|------------------|--------------|
| 子 Agent 返回整屏日志   | 父上下文污染       |
| 父 Agent 不读摘要直接再派 | 重复/矛盾        |
| 把用户直接 @ 给子 Agent | 责任链混乱；应父统一对口 |
| 子 Agent 提问循环     | 违反工人意识（10.6） |

## 与防递归（10.9）的关系

子 Agent 若试图 Task 转发给「同事」，会破坏星型路由，形成深度链。正确做法是 `NEEDS_PARENT_DISPATCH` 结构化回报。

## 深入：多子 Agent 结果冲突时如何路由？

| 冲突类型    | 父 Agent 动作             |
|---------|------------------------|
| 路径列表不一致 | 抽样 Read 或再派 Explore 仲裁 |
| 技术方案相反  | 抬升给用户或 Plan 只读裁决       |
| 测试结论矛盾  | Verification 独立跑       |

[Mermaid 图表 — 请在在线版本中查看]

## 安全与隐私路由

| 注意点  | 建议     |
|------|--------|
| 密钥   | 子摘要 脱敏 |
| 客户数据 | 最小必要片段 |
| 日志   | 截断与哈希  |

## 案例：三路搜索合并

- 子 A 返回 `services/a` 下 4 文件。
- 子 B 返回 `pkg/b` 下 2 文件。
- 子 C 返回 `deploy` 目录树。
- 父 Agent **去重** → Phase3 派 **Worker** 仅触及 **不相交集**；相交则 **串行** (10.5)。

## 小结

- 子 Agent 独立上下文；父 Agent 依赖 **蒸馏结果** 决策。
- 结构化返回 = 可 merge、可验证、可缓存。
- 星型路由** + **扁平 Task** = 可扩展蜂群。

## 自测

- 为何不能把子 Agent 的全量工具日志都塞进父上下文？
- 写出你建议的五字段返回结构。
- 星型与链式（扁平）有何区别？



## 与统一 Fork 前缀的衔接 (10.8)

| 环节               | 建议                                                        |
|------------------|-----------------------------------------------------------|
| Task.description | <code>Fork started – processing in background: ...</code> |
| 子 Agent 最终答复首行   | 可用固定 <code>Summary –</code> 便于父解析                         |
| 日志归档             | 按 <code>task_id</code> 分文件，避免混在父会话                        |

## 失败模式诊断表

| 症状                | 可能根因                              | 修复         |
|-------------------|-----------------------------------|------------|
| 父 Agent 重复问用户同样问题 | 子摘要未含 <code>open_questions</code> | 强制模板       |
| merge 后路径仍错       | evidence 无行号                      | 反偷懒 (10.6) |
| 用户看到乱序回复          | 多 Task 完成顺序非提交顺序                  | 父侧排序再呈现    |

## 示例：结构化 JSON 风格摘要（可选）

若你的编排层支持 JSON mode：

```
{
  "summary": "找到 3 处引用",
  "evidence": [
    {"path": "a.go", "lines": "10-40", "note": "调用 Foo"}
  ],
  "open_questions": []
}
```

父 Agent `parse` → `merge` → 派下一 Task，比散文更稳。

上一节：[10.9 防递归](#) · 下一节：[10.11 Swarm vs Coordinator](#)

# 10.11 Swarm vs Coordinator（两种多 Agent 模式对比）

系列：Claude Code 完全指南 V2 · 第 10 篇

## 学习目标

- 1. 对比 Swarm（蜂群/对等自治探索）与 Coordinator（中心调度）的适用场景。
- 2. 选择在宽搜、收敛、落地、验证各阶段分别偏向哪种模式。
- 3. 解释二者如何混合：早期 Swarm 宽搜 + 晚期 Coordinator 收口。
- 4. 关联反偷懒、防递归、Verification 在两种模式下的**共性要求**。

## 生活类比：跳蚤市场 vs 连锁总部

**Swarm 像跳蚤市场**：多个摊主各自吆喝、各自发现顾客，没有统一麦克风，但覆盖面大。  
**Coordinator 像连锁总部**：分区经理听汇报、排班、决定谁先补货——强中心、强阶段门。软件里：未知问题常先 Swarm；强交付常收 Coordinator。

## 双模式概念图

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## 对照总表

| 维度             | Swarm        | Coordinator    |
|----------------|--------------|----------------|
| 决策中心           | 弱 / 多路并行各自为战 | 强 / 父 Agent 拍板 |
| 适用阶段           | 未知问题、发散      | 已知路线、收敛交付      |
| 汇总成本           | 可能高（重复）      | 中等（显式 merge）   |
| 冲突处理           | 事后发现         | Phase2 预先排序    |
| 风险             | 上下文碎片化、重复劳动  | 中心瓶颈、调度失误      |
| 与 Verification | 仍需独立         | 仍需独立           |

## 何时偏 Swarm?

| 场景             | 理由           |
|----------------|--------------|
| 新仓库 onboarding | 多视角快速扫       |
| 根因不明 incident  | 并行假设验证（只读为主） |
| 技术调研           | 广撒网收集替代方案    |

注意：Swarm 不是允许子 Agent 再派子 Agent；仍 扁平（10.9）。

## 何时偏 Coordinator?

| 场景       | 理由                               |
|----------|----------------------------------|
| 明确里程碑发布  | 阶段门 + 责任到人                       |
| 多文件改造需顺序 | 串行 Worker（10.5）                  |
| 大团队仿真    | 角色分明（Explore/Plan/Worker/Verify） |

## 混合模式（推荐）

Phase 0: 用户目标  
Phase 1 (Swarm 向): 3× Explore/Worker 并行宽搜  
Phase 2 (Coordinator): 父 merge → 锁定路径与顺序  
Phase 3 (Coordinator): Worker 并行/串行修改  
Phase 4 (强制): Verification 独立判决

[Mermaid 图表 — 请在在线版本中查看]

## 与「反偷懒」的共性

无论 Swarm 还是 Coordinator，**模糊指令都会放大无效探索**：

| 模式          | 模糊指令的后果       |
|-------------|---------------|
| Swarm       | 多子 Agent 重复跑偏 |
| Coordinator | 错误并行浪费        |

父 Agent 必须在进入 Phase1 前给出**搜索边界**（目录/关键字/时间范围）。

## 与缓存前缀（10.8）

两种模式都应使用 **统一 Fork 前缀**；Swarm 下任务数更多，**前缀收益更大**。

## 与消息路由（10.10）

- Swarm：父 merge 负担重 → 更需要 **结构化返回**。
- Coordinator：路由 **显式**，但仍依赖 **蒸馏** 质量。

## 案例对照

**场景**：全库「慢查询」优化。

| 模式          | 做法                                                                          |
|-------------|-----------------------------------------------------------------------------|
| Swarm 向     | 三子分别搜 ORM、索引、日志配置                                                           |
| Coordinator | 父合并后指定 只改 <code>queries/report.sql</code> 与 <code>index migration</code> 顺序 |

若全程 Swarm 无收口 → 可能三人各改一处冲突。若全程无 Swarm → 搜索覆盖不足。

## 反模式

| 反模式                     | 说明       |
|-------------------------|----------|
| 假 Swarm：子 Agent 再 Task  | 递归（禁）    |
| 假 Coordinator：父从不 merge | 并行失控     |
| 无 Verification          | 两种模式都不可信 |

## 选型速查

[Mermaid 图表 — 请在在线版本中查看]

## 小结

- Swarm = 发散、覆盖；Coordinator = 收敛、调度。
- 混合最常见：宽搜 Swarm → 收口 Coordinator → Verification。
- 防递归、反偷懒、利益隔离 两种模式都要。

## 自测

1. 何时不应使用纯 Swarm？
2. Coordinator 的最大风险是什么？
3. 写出一个你项目里的三阶段混合流程。

## 与 Verification 的「模式无关」原则

无论 Swarm 还是 Coordinator，Verification 都必须：

- Try to break it (10.7)
- PASS/FAIL/PARTIAL 附命令证据
- 与实现子 Agent 会话隔离

[Mermaid 图表 — 请在在线版本中查看]

## 团队规模与模式倾向（经验法则）

| 团队/context | 倾向                      |
|------------|-------------------------|
| 单人 + 超大仓   | 先 Swarm 宽搜再 Coordinator |
| 强发布火车      | Coordinator 主导          |
| 研究型 spike  | Swarm 向                 |

## 术语对照

| 口语  | 本文含义                  |
|-----|-----------------------|
| 蜂群  | 多子 Agent 并行探索（扁平）     |
| 协调器 | 强中心 merge 与派工         |
| 混合  | Swarm→Coordinator 流水线 |

上一节：[10.10 消息路由](#) · 下一节：[10.12 实践](#)

## 10.12 实践：设计多 Agent workflows

系列：Claude Code 完全指南 V2 · 第 10 篇

## 学习目标

1. 从零设计一条含 **Explore / Plan / Coordinator / Worker / Verification** 的完整工作流。
2. 落地 Phase1 并行、Phase3 串行/并行决策与 **Fork 统一前缀**。
3. 写出可执行的 Worker 工单与 Verification 探针列表。
4. 复盘失败模式：递归、模糊指令、无隔离验证。

## 生活类比：筹办婚礼

**Explore** 看场地与供应商目录；**Plan** 排出流程表；**Coordinator** 协调并行预定与串行签约；**Worker** 执行每一项采购与布置；**Verification** 像闺蜜团挑刺（音响测试、座位表冲突检查）——**不能**由新郎新娘自评「一切完美」。

## 实践总流程图

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## 练习题 A：为「API 速率限制误伤」设计工作流

背景：用户报告正常流量被 429。

| 阶段     | 子 Agent      | 关键动作                                                           | description 前缀                                        |
|--------|--------------|----------------------------------------------------------------|-------------------------------------------------------|
| Phase1 | Explore x2   | 搜 <code>rate</code> 、 <code>limit</code> 、 <code>429</code>    | <code>Fork started – processing in background:</code> |
| Phase1 | Worker       | 搜中间件注册点                                                        | 同左                                                    |
| Phase2 | 父            | 合并路径，锁定<br><code>middleware/rate.go:??</code>                  | —                                                     |
| Phase2 | Plan（可选）     | 输出两阶段修复+回滚                                                     | 同左                                                    |
| Phase3 | Worker       | 按行号调整阈值与 bypass 规则                                             | 同左                                                    |
| Phase4 | Verification | <code>go test</code> + <code>curl</code> 正常/越界流量 + adversarial | 同左                                                    |

Verification 探针示例：

| 探针        | 期望                             |
|-----------|--------------------------------|
| 正常 Header | 200                            |
| 超频        | 429 + <code>Retry-After</code> |
| 空 UA      | 依策略 400 或 200（需文档）             |

判决：Lint 若红 → 依团队策略 **FAIL** 或 **PARTIAL**（10.7）。





## 练习题 B：模板填空（请手写）

```
## 用户目标
(一句话)

## Phase1 并行 (3 Task)
1. description: Fork started – processing in background: _____
   subagent_type: _____
   prompt 要点: _____

2. ...

3. ...

## Phase2 merge 产出
- 锁定文件与行号: _____

## Phase3 Worker 工单
- 工人意识注入（粘贴 10.6 段落）: _____
- 禁止事项: _____

## Phase4 Verification
- 必跑命令: _____
- adversarial: _____
- 判决标准 PASS/FAIL/PARTIAL: _____

## 防递归自检
- [ ] 子 Agent prompt 无「再 Task」字样
```

## 练习题 C：找出错误 workflow

下面 workflow 错在哪？

- 1. Worker prompt: 「不确定就再 spawn 一个 explore。」
- 2. Plan: 「我会直接改 `go.mod` 升级依赖。」
- 3. Verification: 「我阅读了 Worker 的总结, PASS。」
- 4. Phase3: 两 Worker 同时改 `config.yaml` 不同键但同一 `merge` 热点且无锁。

| 序号 | 错误类型  | 改正              |
|----|-------|-----------------|
| 1  | 递归/委派 | 扁平 + 父派 Explore |
| 2  | 只读违反  | Worker          |
| 3  | 未执行检查 | 跑命令 + 探针        |
| 4  | 冲突风险  | 串行或单 Worker     |

## 评分 rubric（自评用）

| 项目   | 满分标准                                 |
|------|--------------------------------------|
| 角色覆盖 | 至少用到 Explore + Worker + Verification |
| 工单质量 | 路径+行号+期望+验收命令                        |
| 并行策略 | Phase1 并行；热点文件不盲目并行                  |
| 隔离   | Verification 独立 Task                 |
| 前缀   | 所有 Task description 统一前缀             |
| 防递归  | 明示禁止子 Task                           |

## 进阶：与 CI/CD 对齐

| 层级              | 多 Agent     | CI        |
|-----------------|-------------|-----------|
| 本地 Verification | 快速探针        | 全量矩阵      |
| 证据格式            | Markdown 摘要 | JUnit/XML |

设计 workflow 时，注明：**Agent 内 Verification  $\subset$  CI**，避免误以为本地 PASS 等于生产免疫。

## 复盘清单（项目结束后）

- [ ] 是否出现 **重复搜索**？→ Phase1 提示更窄或加 Explore-only
- [ ] 是否 **FAIL** 多？→ 反查是否 **模糊派工**
- [ ] token 成本？→ 蒸馏是否过冗长；前缀是否统一
- [ ] 人为介入点是否前置？→ Plan **开放问题**是否提前抛出

## 参考速查链接（本篇内）

- [10.2 六角色](#)
- [10.5 Coordinator](#)
- [10.6 反偷懒](#)
- [10.7 Verification](#)

- [10.8 缓存](#)
- [10.9 防递归](#)
- [10.10 消息路由](#)
- [10.11 Swarm vs Coordinator](#)

---

## 小结

---

- 实践 = 角色 × 阶段 × 结构化协议 × 独立验证。
- 用 模板 与 rubric 固化习惯。
- 混合 Swarm→Coordinator 是默认优解。

---

## 最后一题

---

为你当前真实项目写一条 Phase1 三个并行 Task 的 `description` 行（含统一前缀），主题自选。

---

上一节: [10.11 Swarm vs Coordinator](#) · 返回: [10.1 索引](#)

---

## 第11篇：终端UI

### 第 11 篇：终端 UI · 11.1 架构总览与自研渲染栈

路径：docs/part11-terminal-ui/index.md

系列：Claude Code 完全指南 V2

#### 学习目标

完成本节学习后，你应该能够：

1. **说明** Claude Code 终端界面为何采用自研 **React/Ink 风格渲染器**，而非直接依赖标准 npm **ink** 包。
2. **描述** 终端 UI 的四层心智模型：**React 组件树** → **Fiber 协调器** → **Yoga 布局** → **字符栅格绘制**。
3. **列举** 终端交互能力边界：鼠标、文本选择、OSC 8 超链接、剪贴板、Kitty 协议与能力探测。
4. **建立** 与后续小节（Yoga、Fiber、流式输入、虚拟滚动、Vim、Diff、设计系统）的**阅读路线图**。

#### 生活类比：剧院后台 vs 观众席

把终端想成一座小剧场：

- **剧本（React 组件）** 描述「谁站哪、说什么」——但观众看不见 JSX，只看**最终灯光与台词**。
- **导演（Reconciler / Fiber）** 决定「这一场改台词还是换演员」，**最小化重排**。
- **舞台机械（Yoga）** 按 Flex 规则把演员摆到格子上：前后顺序、对齐、留白。
- **灯光师（渲染器）** 把二维布局刷成 **ANSI 转义序列**，送到 stdout。

标准 npm Ink 像是租来的整套舞美；Claude Code 选择**自研**，是为了把**流式输出**、**389 个组件**、**Diff**、**Vim**、**虚拟滚动**等需求焊进同一条流水线，而不是在第三方抽象缝隙里打补丁。

## 自研渲染栈：为何不是「npm install ink」

| 对比维度 | 典型 npm Ink 用法     | Claude Code 自研方向                        |
|------|-------------------|-----------------------------------------|
| 包边界  | 黑盒 API，升级与定制受上游约束 | 可控演进：协议、性能、交互与产品节奏对齐                    |
| 布局引擎 | 常依赖既有绑定或简化布局      | 纯 TS 移植 Meta Yoga，无 C++ 绑定，便于同仓调试       |
| 流式场景 | 通用 TUI，流式长文需额外胶水  | async generator 逐词、虚拟滚动、与 Agent 输出管线一体化 |
| 交互深度 | 基础键盘为主            | 鼠标追踪、选择、OSC 8、剪贴板、Kitty、Vim 模式          |
| 组件规模 | 中小项目              | 约 389 个 React 组件，需要统一设计系统与主题            |

## 架构鸟瞰：从 React 到像素（字符）

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## 源码心智模型（示意片段）

以下伪代码展示「自定义渲染器」在概念上如何挂接 React 与终端，真实仓库会有更多类型与边界处理：

```
// 概念示意: 自定义 reconciler 的“宿主配置”一角
// 真实实现会包含 createInstance、appendChild、commitUpdate 等完整宿主方法
type TerminalHostConfig = {
  // 将 React 元素类型映射为终端节点描述
  createInstance: (type: string, props: Record<string, unknown>) => TermNode;
  // 布局阶段: 把子树交给 Yoga 计算
  finalizeInitialChildren: (node: TermNode) => void;
};

// 流式渲染常与 async generator 结合: 逐词/逐块提交到子树
async function* streamTokens(source: AsyncIterable<string>) {
  for await (const token of source) {
    yield token; // UI 层消费后触发局部 reconciler 更新
  }
}
```

## 核心能力速查表

| 能力      | 用户可感知效果       | 技术要点 (后文展开)                                                                                             |
|---------|---------------|---------------------------------------------------------------------------------------------------------|
| Flex 布局 | 面板对齐、自适应宽度    | Yoga TS: <code>flex-direction</code> 、 <code>align*</code> 、 <code>margin</code> / <code>padding</code> |
| Fiber   | 流畅刷新、局部更新     | 自定义 reconciler 桥接「终端 DOM」                                                                               |
| 流式输出    | 打字机式 Agent 回复 | async generator + 受控提交频率                                                                                |
| 输入      | 各终端差异缩小       | Kitty 协议、能力检测 (iTerm2 / xterm.js / multiplexer)                                                         |
| 长列表     | 不卡顿           | 虚拟滚动: 只布局可见窗口                                                                                           |
| 编辑体验    | hjkl 党友好      | Vim 模式键位绑定                                                                                              |
| 审阅代码    | 并排/高亮变更       | Diff 展示子系统                                                                                              |
| 主题      | 暗色/亮色一致       | 设计系统与令牌                                                                                                 |

## 389 个组件：如何不迷路

建议按**职责分桶**阅读源码 (名称以仓库为准):

- 1. **Primitives**: 文本、盒子、分隔、图标位。
- 2. **Layout shells**: 侧边栏、面板、堆叠、滚动容器。
- 3. **Agent UI**: 消息气泡、工具调用卡片、状态条。
- 4. **Editor-like**: 选择、光标、Vim 状态、命令面板。
- 5. **Theming**: Provider、令牌解析、暗/亮切换。

[Mermaid 图表 — 请在在线版本中查看]

## 与本篇其他小节的关系

| 小节    | 文件                                     | 重点                   |
|-------|----------------------------------------|----------------------|
| 11.2  | <a href="#">02-yoga-layout.md</a>      | Yoga 纯 TS、flex、盒模型   |
| 11.3  | <a href="#">03-react-fiber.md</a>      | 自定义 reconciler 与终端节点 |
| 11.4  | <a href="#">04-streaming-render.md</a> | async generator 流式渲染 |
| 11.5  | <a href="#">05-input-handling.md</a>   | 输入解码与能力检测            |
| 11.6  | <a href="#">06-virtual-scroll.md</a>   | 长输出性能                |
| 11.7  | <a href="#">07-vim-mode.md</a>         | 键位绑定与模式状态机           |
| 11.8  | <a href="#">08-diff-display.md</a>     | 代码对比呈现               |
| 11.9  | <a href="#">09-mouse-hyperlinks.md</a> | 鼠标、OSC 8、剪贴板         |
| 11.10 | <a href="#">10-design-system.md</a>    | 主题与设计系统              |

## 常见误解

| 误解                   | 澄清                                     |
|----------------------|----------------------------------------|
| 「用了 Ink 就等于用了 npm 包」 | 此处是 Ink 风格/理念 (React 驱动 TUI)，实现为自研渲染栈。 |
| 「Yoga 一定要原生绑定」       | 本栈为 TypeScript 移植，避免 C++ 绑定与跨平台分发成本。   |
| 「终端只能键盘」             | 现代终端 + 协议可支持鼠标、链接、剪贴板；需降级路径。           |

## 小结

终端 UI 是 Claude Code 的「面孔」：它必须把 Agent 流式智能、代码审阅、类 IDE 交互 压缩进纯文本管线。自研 React 渲染器 + Yoga 布局 + Fiber 协调，是为了在 389 个组件 的规模下仍保持可演进、可调试、可主题化。下一节深入 Yoga 纯 TypeScript 布局引擎 如何服务 flex 与盒模型。

## 延伸阅读建议

- 先通读 11.2 Yoga 与 11.3 Fiber，再读 11.4 流式，可建立「布局—提交—输出」闭环。
- 若你维护终端兼容性，优先 11.5 输入 与 11.9 鼠标与链接。

## 与 Claude Code 其他子系统的接口（概念）

| 邻域             | 终端 UI 承担的角色                 |
|----------------|-----------------------------|
| Agent 输出       | 流式文本与工具卡片的呈现与合批             |
| 权限与确认          | 对话框、列表选择在 TUI 层的交互外壳        |
| 记忆与上下文         | 通常不在 UI 层持久化，只负责展示摘要        |
| Bridge（第 12 篇） | 同一核心能力可走 IDE 原生 UI，终端为 另一前端 |

## 读者画像与阅读路径

| 读者   | 建议路径                                  |
|------|---------------------------------------|
| 前端背景 | 先 11.3 Fiber → 11.10 设计系统 → 11.2 Yoga |
| 基础设施 | 先 11.5 输入 → 11.9 鼠标链接 → 11.6 滚动       |
| 技术写作 | 按目录 11.1→11.10 保持叙事连贯                 |

## 自研渲染器的「非目标」

以下通常不是终端 UI 栈的一等目标（避免误读）：

- 完整 CSS Grid 与任意选择器（终端以 布局子集 为主）。
- 与浏览器 DOM API 级兼容。
- 像素级 WYSIWYG 打印（ANSI 与终端字体差异巨大）。

明确 非目标 有助于在源码中理解为何某些 Web 模式被刻意省略。



## 11.2 Yoga 布局：纯 TypeScript 中的 Flexbox

路径：`docs/part11-terminal-ui/02-yoga-layout.md`  
系列：Claude Code 完全指南 V2 · 第 11 篇

### 学习目标

完成本节学习后，你应该能够：

- 1. **解释** 为何在终端 UI 中引入 **Meta Yoga** 的语义，并以 **纯 TypeScript** 实现而非 C++ 绑定。
- 2. **列举** 已覆盖的关键能力：`flex-direction`、主轴/交叉轴对齐、`margin` 与 `padding`。
- 3. **对比** Web CSS 与「字符栅格」布局：像素变单元格、行高与折行约束。
- 4. **将** Yoga 计算结果映射到**行/列索引**，为渲染器提供确定性坐标。

### 生活类比：停车场划线 vs 方格纸

Web 页面像在无限延展的停车场：车位可以毫米级微调。终端更像**方格纸**：每个字符是一个不可分割的格子。

Yoga 负责在「**逻辑盒子**」层面说清：A 在 B 左边还是下面、剩余空间怎么分、外边距推开多少。渲染器再把盒子**量化**为整数列宽与行数——就像把设计图**对齐到网格**，避免「半格车」停不进去。

### 为什么选 Yoga 语义？

| 需求            | Yoga/Flex 模型的收益                                                 |
|---------------|-----------------------------------------------------------------|
| 侧栏 + 主内容      | <code>flex-direction: row</code> + <code>flex-grow</code> 分配剩余列 |
| 顶栏固定、正文滚动     | 交叉轴对齐 + 子树独立测量                                                  |
| 工具卡片栅格        | wrap 与 gap（若实现层支持）类语义可渐进补齐                                      |
| 与 React 组件树同构 | 每个组件节点挂 <b>layout props</b> ，树即布局树                              |

## 无 C++ 绑定：工程权衡

[Mermaid 图表 — 请在在线版本中查看]

| 维度  | C++ 绑定 Yoga   | 纯 TS Yoga   |
|-----|---------------|-------------|
| 分发  | 多架构二进制        | 仅 JS/TS 产物  |
| 调试  | LLDB + JS 混合栈 | 同语言断点       |
| 性能  | 通常更快          | 终端布局规模下常足够快 |
| 一致性 | 与上游 ABI 版本锁   | 与产品迭代同频     |

## 布局管线（概念）

[Mermaid 图表 — 请在在线版本中查看]

## 核心属性在终端中的含义

| Yoga/CSS 概念                         | 终端侧直觉                 |
|-------------------------------------|-----------------------|
| <code>flex-direction: row</code>    | 子节点横向排列在字符列上          |
| <code>flex-direction: column</code> | 子节点纵向占行               |
| <code>justify-content</code>        | 主轴剩余空间分配（左/中/右或顶/中/底） |
| <code>align-items</code>            | 交叉轴对齐（例如多行面板在同一高度对齐）  |
| <code>margin</code>                 | 在盒子外扩空格或留白行           |
| <code>padding</code>                | 盒子内缘与内容之间的内边距         |

## 源码片段（示意 API）

以下展示 TypeScript 侧 可能如何声明布局 props（真实仓库字段名可能更细）：

```
type FlexDirection = 'row' | 'column' | 'row-reverse' | 'column-reverse';

type TerminalBoxStyle = {
  flexDirection?: FlexDirection;
  flexGrow?: number;
  flexShrink?: number;
  flexBasis?: number | 'auto';
  justifyContent?: 'flex-start' | 'center' | 'flex-end' | 'space-between';
  alignItems?: 'stretch' | 'flex-start' | 'center' | 'flex-end';
  margin?: { top?: number; right?: number; bottom?: number; left?: number };
  padding?: { top?: number; right?: number; bottom?: number; left?: number };
};

interface LayoutNode {
  id: string;
  style: TerminalBoxStyle;
  children: LayoutNode[];
  /** 测量：给定最大宽度，返回所需行数与宽度 */
  measure: (maxWidth: number) => { width: number; height: number };
}
```

测量函数通常与 **Unicode 宽度**（全角字符）、**ANSI 脱敏后的可见宽度**、以及 **折行策略** 绑定。

## 与 Fiber 的协作点

Yoga 不直接懂 React。协作方式是：

- 1. Reconciler 为每个宿主节点创建 **layout 影子节点**。
- 2. `style` 变更标记 **dirty**，在提交阶段 **自顶向下** 或 **脏路径** 重算。
- 3. 布局结果写入 **终端节点** 的 `frame: { x, y, w, h }`（单位：单元格）。

[Mermaid 图表 — 请在在线版本中查看]

## 调试建议

| 现象    | 可能原因                            | 排查方向                             |
|-------|---------------------------------|----------------------------------|
| 右侧被裁切 | <code>flexShrink</code> 或最大宽度过小 | 检查父链 <code>maxWidth</code> 与测量   |
| 垂直错位  | 交叉轴对齐与行高不一致                     | <code>alignItems</code> 与字体/行高常量 |
| 多余空行  | <code>margin</code> 叠加或折行测量偏大   | 单节点最小高度与 padding                 |
| 抖动    | 每次流式更新全量布局                      | 增量脏区与布局缓存                        |

## 小结

Yoga 语义 + 纯 TS 实现 让终端 UI 在不引入原生绑定的前提下获得 Web 级布局表达力。关键是把 flex-direction / 对齐 / margin / padding 落到 字符网格，并与 测量（文本折行）形成闭环。下一节聚焦 自定义 React Fiber reconciler 如何把组件树接到这套布局之上。

## 自测清单

- 说出至少三种 `justify-content` 在终端宽面板中的视觉效果。
- 解释 `margin` 与 `padding` 在「方格纸」模型中的差异。
- 画一条从「style 变更」到「ANSI 输出」的数据路径。

## Flex 子项常见组合（终端侧）

| 组合                                               | 典型界面                       |
|--------------------------------------------------|----------------------------|
| <code>row</code> + <code>flex-grow</code> 主区域    | 左窄侧栏 + 右宽对话流               |
| <code>column</code> + <code>space-between</code> | 顶工具条 + 底状态条夹中间内容           |
| <code>align-items: center</code>                 | 在固定高度面板内 垂直居中 短文本          |
| <code>margin: { left: 2 }</code>                 | 与左侧边框形成 缩进栏（类似 IDE gutter） |

## 「最小可行布局树」练习

给定三栏： `Sidebar | Main | Inspector`，要求在中等宽度下 `Inspector` 折到 `Main` 下方，在宽屏恢复三列。用 Yoga 语义思考：

- 外容器 `flexDirection` 由 `columns` 断点驱动。
- 各子项 `flexBasis` / `flexGrow` 的相对关系。

此练习帮助把 响应式 从 Web 直觉迁移到 字符宽度断点。

## 与 Unicode 测量的耦合点

Yoga 询问子节点「你需要多大？」时，终端文本节点必须返回 **显示宽度** 而非 **code point 数**。

| 话题               | 说明                       |
|------------------|--------------------------|
| East Asian Width | 全角字符常占 <b>2 列</b>        |
| 组合字符             | 可能 <b>1 列</b> 但多码点       |
| Emoji            | 宽度因终端字体而异，常需 <b>保守估计</b> |

布局 **确定性** 来自团队内固定的 **测量表版本** 与 **降级规则**。

## 参考阅读（外部）

- Yoga 与 Flexbox 命名对照（理解字段即可）。
- Unicode TR11（East Asian Width）——仅深潜文本测量时必要。

# 11.3 React Fiber：自定义协调器与「终端 DOM」

路径： docs/part11-terminal-ui/03-react-fiber.md

系列： Claude Code 完全指南 V2 · 第 11 篇

## 学习目标

完成本节学习后，你应该能够：

1. 说明 何为 **自定义 React reconciler**，以及它如何把 React 的更新语义映射到终端宿主树。
2. 区分 Render 阶段与 Commit 阶段在 TUI 中的职责划分。
3. 理解 Fiber 节点如何同时承载 **React 语义** 与 **终端绘制属性**。
4. 联系 上一节 Yoga：布局计算发生在协调管线的哪一步。

## 生活类比：总机接线员

把 `Reconciler` 想成公司总机：

- 外线（`setState` / `props` 变更）不断打进来。
- 接线员（`Fiber` 调度）决定先接哪通、是否挂断重拨（中断与恢复）。
- 最后才把通话转接到分机（宿主提交：创建/更新/删除终端节点）。

若没有 `Fiber` 级别的控制，每次小改动都像重播整本电话簿——终端会闪屏、CPU 飙高、流式场景无法增量拼接。

## 自定义 reconciler 在栈中的位置

[Mermaid 图表 — 请在在线版本中查看]

## 宿主方法族（概念表）

真实 `react-reconciler` 需要实现一整张 `host config`。下表是教学用裁剪版：

| 宿主方法                                                       | 典型职责（终端）                                                   |
|------------------------------------------------------------|------------------------------------------------------------|
| <code>createInstance</code>                                | 根据 <code>type</code> 创建 <code>TermNode</code> （文本/容器/滚动区等） |
| <code>createTextInstance</code>                            | 纯文本叶子                                                      |
| <code>appendInitialChild</code> / <code>appendChild</code> | 维护子树链接                                                     |
| <code>prepareUpdate</code>                                 | diff props，生成 payload                                      |
| <code>commitUpdate</code>                                  | 把新 props 写入节点，标记绘制脏                                        |
| <code>removeChild</code>                                   | 卸载子树，释放布局与缓冲                                               |
| <code>getRootHostContext</code>                            | 根上下文（主题、宽度、能力位）                                            |

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段（伪 host config 骨架）

```
import Reconciler from 'react-reconciler';

const HostConfig = {
  supportsMutation: true,

  createInstance(
    type: string,
    props: Record<string, unknown>,
    rootContainer: TermRoot,
    hostContext: HostCtx,
    internalHandle: unknown
  ) {
    return rootContainer.factory.createElement(type, props, hostContext);
  },

  createTextInstance(text: string, root: TermRoot, ctx: HostCtx) {
    return root.factory.createText(text, ctx);
  },

  appendChildToContainer(container: TermRoot, child: TermNode) {
    container.document.appendChild(child);
  },

  prepareUpdate(
    instance: TermNode,
    type: string,
    oldProps: Record<string, unknown>,
    newProps: Record<string, unknown>
  ) {
    return shallowDiff(oldProps, newProps);
  },

  commitUpdate(
    instance: TermNode,
    updatePayload: unknown,
    type: string,
    prev: Record<string, unknown>,
    next: Record<string, unknown>
  ) {
    instance.applyProps(next, updatePayload);
    instance.markLayoutDirty();
  },

  // ...removeChild、insertBefore、finalizeInitialChildren 等
};

export const TermRenderer = Reconciler(HostConfig);
```

## Fiber 与 Yoga 的衔接时机

| 阶段     | Fiber 在做什么             | Yoga 是否参与                 |
|--------|------------------------|---------------------------|
| Render | 生成更新计划，可被打断            | 通常不做完整布局                  |
| Commit | 应用 DOM（此处为 TermNode）变更 | 标记 dirty，触发布局遍历           |
| Layout | 计算坐标与尺寸                | Yoga 全量或增量                |
| Paint  | 生成字符缓冲与 ANSI           | 读取 <code>frame</code> 与样式 |

## 根容器与上下文

终端根容器常携带：

- 终端列宽/行高（随窗口变化触发根更新）。
- 能力位：真彩色、鼠标、Kitty、Unicode 宽度策略。
- 主题令牌：暗/亮与对比度。

```
type HostCtx = {
  columns: number;
  rows: number;
  capabilities: {
    trueColor: boolean;
    mouse: boolean;
    kittyKeyboard: boolean;
  };
  theme: 'dark' | 'light';
};
```

## 流式场景下的 Fiber 策略

流式输出时，频繁小更新若每次都 **全树 commit**，会浪费在布局与绘制上。常见优化思路：

| 策略   | 思路                             |
|------|--------------------------------|
| 子树隔离 | 将流式文本封在 <code>memo 边界</code> 内 |
| 批量提交 | 微任务合并多次 token 到达               |
| 脏矩形  | 仅重绘变更行的区间（与虚拟滚动协同）             |



## 常见坑

| 症状    | 可能原因                                                                |
|-------|---------------------------------------------------------------------|
| 焦点丢失  | 宿主未实现 <code>prepareForCommit</code> / <code>resetAfterCommit</code> |
| 文本重复  | <code>textInstance</code> 与 <code>children</code> 双重渲染路径冲突          |
| 布局不更新 | <code>finalizeInitialChildren</code> 未触发首次 Yoga                     |
| 内存涨   | 卸载路径 <code>removeChild</code> 未断开环引用                                |

## 小结

自定义 **Fiber reconciler** 是「React 组件」与「终端节点」之间的**契约层**：它把声明式 UI 变成可增量提交的宿主操作，并为 **Yoga 布局** 提供稳定的**节点生命周期**。下一节进入 **async generator 流式渲染**，看 Fiber 如何在「逐词到达」时保持顺滑。

## 与 11.2 的衔接练习

- 标出 `commitUpdate` 之后、`ANSI 写出` 之前必须经过的两个子阶段。
- 解释为何 `supportsMutation: true` 更贴近多数 TUI 的更新模式。

# 11.4 流式渲染：Async Generator 与逐词输出

路径：`docs/part11-terminal-ui/04-streaming-render.md`

系列：Claude Code 完全指南 V2 · 第 11 篇

## 学习目标

完成本节学习后，你应该能够：

- 描述** Claude Code 如何将 **LLM 流式 token** 映射到终端 UI 的**增量更新**。
- 使用** `async generator` 组织「读一块 → 合批 → 提交 reconciler」的流水线。

3. 权衡 刷新频率与 CPU/闪屏：批量、节流、`requestAnimationFrame` 类比。
4. 关联 虚拟滚动 (11.6)：长流式文本如何避免全量布局。

## 生活类比：听写与板书

老师口述课文，学生往黑板上写：

- 若每个字都立刻擦了重写，黑板会闪、手臂会酸——对应终端 过度重绘。
- 合理做法是一小句一停顿（批量），或每秒最多擦几次（节流）。

`async generator` 就像分段听写：上游说「下一段」，下游决定何时抄到板上。

## 流式数据路径（鸟瞰）

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## 为何选 Async Generator?

| 方案              | 优点                | 缺点        |
|-----------------|-------------------|-----------|
| 回调地狱            | 简单                | 组合难、背压不清晰 |
| Promise 链       | 扁平                | 难表达「无限流」  |
| Async generator | for await、可中断、可组合 | 需理解迭代器协议  |
| Observable      | 丰富算子              | 依赖与团队心智成本 |

## 源码片段：合批与背压（示意）

```
type StreamChunk = { text: string; done?: boolean };

async function* parseModelStream(
  body: ReadableStream<Uint8Array>
): AsyncGenerator<StreamChunk> {
  const reader = body.getReader();
  const decoder = new TextDecoder();
  while (true) {
    const { value, done } = await reader.read();
    if (done) break;
    yield { text: decoder.decode(value, { stream: true }) };
  }
}

function createBatchedAppender(
  flush: (slice: string) => void,
  opts: { maxChars: number; maxMs: number }
) {
  let buf = '';
  let timer: ReturnType<typeof setTimeout> | null = null;

  const flushNow = () => {
    if (!buf) return;
    flush(buf);
    buf = '';
    if (timer) clearTimeout(timer);
    timer = null;
  };

  return (piece: string) => {
    buf += piece;
    if (buf.length >= opts.maxChars) flushNow();
    else if (!timer) {
      timer = setTimeout(flushNow, opts.maxMs);
    }
  };
}
```

## 与 React 更新的边界

| 实践       | 说明                                          |
|----------|---------------------------------------------|
| 单一文本宿主节点 | 长消息尽量挂在一个 <code>text</code> 或容器，减少 Fiber 深度 |
| key 稳定   | 流式段落用稳定 key，避免整段卸载重建                        |
| 受控高度     | 外层滚动容器固定高度，配合 11.6 虚拟滚动                     |
| 取消与竞态    | 新会话开始时 <code>abort</code> 旧流，丢弃过期 chunk     |

## 节流与「伪 rAF」

浏览器有 `requestAnimationFrame`；Node 终端常用：

- `setImmediate` / `queueMicrotask` 微批
- `setTimeout(0)` 合并宏任务
- 与 屏幕刷新 无关，但与 `stdin` 事件循环协同

[Mermaid 图表 — 请在在线版本中查看]

## 错误与降级

| 场景        | 策略                                                       |
|-----------|----------------------------------------------------------|
| 解析半包 JSON | 增量解析器或状态机，不 <code>JSON.parse</code> 全串                   |
| 编码异常      | <code>TextDecoder</code> <code>fatal: false</code> ，替换字符 |
| 用户 Ctrl+C | <code>AbortSignal</code> 贯通 generator                    |
| 极长单条消息    | 强制切块显示或折叠，避免 O(n) 全量 diff                                |

## 性能检查清单

- 每帧（每次 flush）是否触发布局子树过大？
- 是否在 flush 中做 同步文件 IO？（应移出热路径）
- ANSI 拼接是否可缓存样式前缀？

4. 与 Diff（11.8）同时开启时，是否互斥重绘？

小结

Async generator 为模型输出提供自然的异步迭代抽象；合批把细粒度 token 变成粗粒度 UI 提交，是终端流畅度的关键。下一节 11.5 讲解 输入处理：Kitty 协议、能力检测与多路复用器差异。

自测

- 1. 写出一个会导致「竞态重复段落」的流式 bug 场景及修复思路。
- 2. 解释 maxChars 与 maxMs 两个维度如何共同约束刷新。

附录：与 Ink 理念的差异点

| Ink 典型模式 | 自研栈可增强点        |
|----------|----------------|
| 静态更新为主   | 无限流一等公民        |
| 通用组件     | Agent 专用卡片与工具态 |
| 布局黑盒     | Yoga TS 白盒调试   |

术语对照

| 英文           | 中文表述      |
|--------------|-----------|
| backpressure | 背压        |
| flush        | 刷盘/刷新到 UI |
| chunk        | 数据块       |
| reconciler   | 协调器       |

延伸阅读（概念）

- 迭代器与异步迭代器：MDN AsyncGenerator

- React 18 并发特性在自定义渲染器中的**可选**启用策略（视宿主支持而定）

---

## 实战思考题

---

某用户在 SSH + tmux 中运行，网络延迟高，chunk 稀疏。若仅按 `maxChars` 合批，会出现「半天无输出」的错觉。应如何调整策略？

提示：引入 **时间优先** flush、或 **首个 token 立即显示** 的「冷启动例外」。

---

## 与虚拟滚动的接口

---

流式追加时，虚拟列表需获知：

- 总行数变化
- 锚定行（用户是否手动上滚阅读历史）

建议在滚动容器上维护 `scrollTop` 与 `contentHeight` 的**单一数据源**，避免双状态打架。

---

## 伪代码：锚定策略

---

```
type ScrollModel = {
  contentHeight: number;
  viewportHeight: number;
  scrollTop: number;
  stickToBottom: boolean;
};

function onStreamAppend(m: ScrollModel, deltaLines: number) {
  m.contentHeight += deltaLines;
  if (m.stickToBottom) {
    m.scrollTop = Math.max(0, m.contentHeight - m.viewportHeight);
  }
}
```

---

## 版本演进备忘

---

当产品增加「工具调用中间态」时，流式管线需插入 **结构化事件**（非纯文本），generator 可产出 **discriminated union**：

```
type UiEvent =  
  | { kind: 'text'; value: string }  
  | { kind: 'tool'; name: string; status: 'start' | 'end' };
```

UI 层 `switch (e.kind)` 分别更新子树——这是 389 个组件 协同的典型扩展点。

## 11.5 输入处理：Kitty 协议与终端能力检测

路径: `docs/part11-terminal-ui/05-input-handling.md`

系列: Claude Code 完全指南 V2 · 第 11 篇

### 学习目标

完成本节学习后，你应该能够：

1. 列举 终端输入源：键盘、鼠标、粘贴、焦点 等在不同仿真器中的表示差异。
2. 说明 Kitty 键盘协议 的价值：扩展键、组合键、释放事件的统一表达。
3. 设计 一层 能力检测：区分 iTerm2、xterm.js、tmux/screen 等多路复用场景。
4. 将 原始转义序列解析为语义化输入事件，供 Fiber 事件系统消费。

### 生活类比：多国插座与转接头

你的笔记本插头（物理按键）到了不同国家（终端仿真器），插孔形状不同。Kitty 协议像 USB-C 统一标准：尽量一种转义格式描述更多键位。

能力检测则是旅行前查电压：能不能用 110V？要不要变压器？——对应「支不支持修饰键报告」「是否在 multiplexer 内」。

### 输入栈分层

[Mermaid 图表 — 请在在线版本中查看]

## Kitty 键盘协议（要点）

| 能力         | 无扩展时     | Kitty 扩展后             |
|------------|----------|-----------------------|
| 组合键        | 常丢失或映射混乱 | 显式修饰位                 |
| 功能键 F1–F12 | 各终端不一    | 参数化 CSI               |
| 释放事件       | 常无       | press/release 区分（若启用） |

启用通常通过 特定 DSR/模式序列 与终端握手；具体序列以实现与版本为准，代码中常集中在一处 `capability negotiation` 模块。

## 能力检测矩阵（教学用）

| 环境                  | 典型特征       | 注意事项                                    |
|---------------------|------------|-----------------------------------------|
| iTerm2              | 丰富键鼠、真彩色   | 部分序列与 xterm 兼容子集                        |
| xterm.js（VS Code 等） | Web 技术栈    | 粘贴、焦点与桌面终端不同                            |
| tmux                | 包裹序列       | 外层与内层能力可能剥离                             |
| screen              | 老协议        | 高级键位常降级                                 |
| 纯 SSH               | 取决于远端 TERM | <code>TERM=xterm-256color</code> 仍非能力全集 |

[Mermaid 图表 — 请在在线版本中查看]



## 源码片段：输入状态机（示意）

```
type KeyEvent = {
  type: 'key';
  key: string;
  ctrl: boolean;
  alt: boolean;
  shift: boolean;
  meta: boolean;
  action: 'press' | 'repeat' | 'release';
};

type ParseState =
  | { kind: 'ground' }
  | { kind: 'escape' }
  | { kind: 'csi'; buf: string }
  | { kind: 'osc'; buf: string };

class InputParser {
  private state: ParseState = { kind: 'ground' };

  feed(byte: number, emit: (e: KeyEvent) => void) {
    // 真实实现需处理 UTF-8 多字节与非法续字节
    const ch = String.fromCharCode(byte);
    // ... CSI/SS3 分支、Kitty 解析、退格与回车 ...
  }
}
```

## 窗口尺寸与 SIGWINCH

| 信号/事件                 | 行为                                              |
|-----------------------|-------------------------------------------------|
| <code>SIGWINCH</code> | 终端尺寸变化，需重读 <code>ioctl</code>                   |
| 备用                    | 部分环境用 <code>CSI t</code> 报告                     |
| UI                    | 触发 根容器 <code>columns/rows</code> 更新 → Yoga 重新布局 |

## Bracketed Paste 与安全性

括号粘贴模式把粘贴内容包在 转义序列 内，应用可区分「用户手打」与「整段粘贴」——对 Vim 模式与 命令行注入 防护有意义。

## 与鼠标追踪协同

鼠标事件常以 SGR 1006 等格式上报。输入层解析后产出：

```
type MouseEvent = {
  type: 'mouse';
  button: 'left' | 'middle' | 'right' | 'wheelUp' | 'wheelDown';
  x: number;
  y: number;
  modifiers: { shift: boolean; alt: boolean; ctrl: boolean };
};
```

上层 11.9 负责命中测试与 OSC 8 超链接。

## 多路复用器下的陷阱

| 问题    | 成因          | 缓解             |
|-------|-------------|----------------|
| 修饰键丢失 | tmux 过滤     | 检测后降级快捷键       |
| 颜色不对  | palette 重映射 | 能力位 + 主题回退     |
| 延迟感   | 网络 + 缓冲     | 本地 echo 策略（慎用） |

## 测试建议

1. **矩阵测试**：同一构建在 iTerm2、Terminal.app、Windows Terminal、web 终端跑**按键表**。
2. **录制 stdin**：捕获字节 golden file，解析器回归。
3. **fuzz**：随机字节流不崩溃。

## 小结

输入层是终端 UI 的「耳朵」：把混乱的字节流变成 **Key/Mouse/Paste/Resize** 事件。Kitty 协议与**能力检测** 让高级交互在支持的环境发光，同时在不支持时 **优雅降级**。下一节 11.6 **虚拟滚动** 解决长输出的性能问题。

## 快捷键路由表（示例）

| 上下文 | 按键     | 行为               |
|-----|--------|------------------|
| 全局  | Ctrl+L | 清屏或滚动到底（产品定义）    |
| 全局  | Ctrl+C | 中断当前流            |
| 输入框 | Ctrl+V | 粘贴（尊重 bracketed） |
| Vim | j / k  | 上下移动（见 11.7）     |

## 与 Reconciler 的事件挂钩

宿主可把语义事件投递到 根 Fiber 的 `onKeyDown` 等价钩子，或走命令总线：

[Mermaid 图表 — 请在在线版本中查看]

## 自测

- 解释为何在 tmux 内 `DA` 查询结果可能与「直连终端」不同。
- Bracketed Paste 如何帮助防止「粘贴即执行」类事故？

## 术语

| 英文          | 中文                          |
|-------------|-----------------------------|
| CSI         | Control Sequence Introducer |
| OSC         | Operating System Command    |
| multiplexer | 多路复用器                       |

## 实现清单（供读者对照源码）

- [ ] UTF-8 解码与非法序列恢复
- [ ] CSI 参数解析（分号分隔数字）

- [ ] Kitty 键盘扩展开关与版本协商
- [ ] 鼠标 SGR 模式启用/关闭配对
- [ ] `SIGWINCH` 与初始 `ioctl` 尺寸读取
- [ ] 能力位写入 `HostCtx` 供子组件只读

---

## 延伸阅读

---

- xterm 控制序列参考（对照阅读，勿死记硬背）
- Kitty 文档：键盘协议章节

---

## 常见问题

---

问：为何不全用 `readline` ？

答：全屏 TUI 需要非规范模式、原始字节与自定义焦点，`readline` 面向行编辑而非画布。

问：Web 终端与本地终端输入最大差异？

答：焦点模型、浏览器快捷键抢占、以及 `WebSocket` 延迟 对合批策略的影响。

---

## 11.6 虚拟滚动：长输出的性能护栏

---

路径： `docs/part11-terminal-ui/06-virtual-scroll.md`

系列： Claude Code 完全指南 V2 · 第 11 篇

---

## 学习目标

---

完成本节学习后，你应该能够：

1. 解释 虚拟滚动在终端 UI 中的目标： $O(\text{可见行})$  而非  $O(\text{总行数})$  的布局与绘制。
  2. 描述 窗口、项高、偏移量、缓冲区 四个核心变量如何联动。
  3. 关联 11.4 流式：追加内容时如何维护 `stick-to-bottom` 与 用户上翻阅读 的冲突。
  4. 识别 与 Yoga 布局的切分点：仅对可见子树做精细测量或使用估算高度。
-

## 生活类比：超市货架可视窗口

仓库里有一万件货（十万行日志），但顾客只能透过一个橱窗看十几件。店员不必把一万件全摆进橱窗——只需根据你站的位置换上该段的样品。

虚拟滚动就是**换样品**：滚动时更换「当前窗口对应的数据切片」，而不是渲染全部行。

## 核心公式（行定向）

设：

- `totalLines`：逻辑总行数（或消息条目的累计行数）
- `viewportHeight`：可视行数
- `scrollTop`：第一条可见逻辑行的索引
- `end = min(totalLines, scrollTop + viewportHeight)`

则可见范围为 `[scrollTop, end)`，仅这些行参与 `TermNode` 实例化 或 字符串拼接。

[Mermaid 图表 — 请在在线版本中查看]

## 架构序列

[Mermaid 图表 — 请在在线版本中查看]

## 变高行与估算

终端中每行高度常为 1，但若支持 换行消息 或 富文本块，条目高度 不等。

| 策略       | 适用               | 代价       |
|----------|------------------|----------|
| 固定行高     | 纯日志              | 最简单      |
| 测量缓存     | Markdown 渲染后高度已知 | 需缓存每条目高度 |
| 二分 + 前缀和 | 快速定位 scrollTop   | 实现复杂     |
| 懒测量      | 首次进入视口才测         | 滚动时可能 跳动 |

## 源码片段：可见切片（示意）

```
type LineRef = { id: string; height: number; render: () => string };

function visibleSlice(
  lines: LineRef[],
  scrollTopPx: number,
  viewportH: number
): LineRef[] {
  let acc = 0;
  const out: LineRef[] = [];
  for (const line of lines) {
    const start = acc;
    const end = acc + line.height;
    if (end > scrollTopPx && start < scrollTopPx + viewportH) {
      out.push(line);
    }
    acc = end;
    if (start > scrollTopPx + viewportH) break;
  }
  return out;
}
```

## 与流式输出的协同

| 用户状态     | 策略                                               |
|----------|--------------------------------------------------|
| 在底部「跟随」  | 新行追加后 自动 <code>scrollTop = max - viewport</code> |
| 向上阅读历史   | 不自动滚动；可显示「新消息 N 条」提示                             |
| 跳转到某工具结果 | <code>scrollToLineId</code> 程序化滚动                |

[Mermaid 图表 — 请在在线版本中查看]

## 与 Fiber 更新粒度

| 做法   | 说明           |
|------|--------------|
| 整屏重绘 | 实现简单，CPU 可能高 |
| 脏行合并 | 仅变更行写缓冲      |
| 双缓冲  | 前台缓冲展示，后台合成  |

## 内存与对象池

长会话中，反复创建字符串与数组会加压 GC。可考虑：

- 行对象池（注意内容失效）
- Rope/大块缓冲（降低拼接次数）
- 淘汰策略：仅保留最近 K 条 **完整** 内容，更早的只保留 **摘要**（产品决策）

## 无障碍与可搜索性

虚拟滚动可能导致「屏幕阅读器/复制全文」困难。可选：

- 「导出完整会话」命令
- 专用 **全缓冲视图** 模式（性能换功能）

## 小结

虚拟滚动把终端 UI 从「无限向下增长」变成「恒定窗口成本」，是 Agent 长输出 的必要护栏。实现要点是 scrollTop 单一数据源、变高测量缓存、以及与 流式 stick-to-bottom 的协同。下一节 11.7 Vim 模式。

## 性能预算（经验值）

| 指标    | 目标直觉                   |
|-------|------------------------|
| 单次滚动帧 | 布局+绘制 < 16ms（理想）       |
| 可见行数  | 通常 < 终端行数，留 status bar |
| 缓存命中率 | 高度前缀和 O(log n) 查询      |

## 与 Diff 视图冲突时的策略

同时展示 Diff（11.8）与 对话流 时，建议 分面板 各自滚动模型，避免一个 `scrollTop` 驱动两套逻辑。

## 自测

- 变高条目下，为何「仅按行号滚动」会错位？
- `stickToBottom` 与「用户轻微上滚」如何区分（阈值像素/行）？

## 术语表

| 英文              | 中文              |
|-----------------|-----------------|
| viewport        | 视口              |
| overscan        | 过扫描（上下多渲染几行防闪烁） |
| stick to bottom | 粘底              |

## overscan 示意

多渲染上下各 `overscan=2` 行，滚动快速时减少 空白闪动：

```
const start = Math.max(0, scrollTop - overscan);
const end = Math.min(totalLines, scrollTop + viewportHeight + overscan);
```



## 调试技巧

- 在 dev 模式打印 `visibleCount / totalLines` 比例。
- 人为注入 1e5 行假数据压测。
- 与 389 组件 中列表容器联调，确认 `key` 不抖动。

## 常见 bug

| 现象   | 原因                  |
|------|---------------------|
| 滚动条跳 | 高度测量前后不一致           |
| 空白洞  | slice 区间 off-by-one |
| 内存仍涨 | 逻辑缓冲无限增长无淘汰         |

## 与 Yoga 的边界

虚拟滚动是 **数据面** 优化；Yoga 仍可对**可见容器**做 flex。避免对不可见子树调用昂贵 `measure` 。

## 延伸阅读

- 前端虚拟列表经典文章（概念迁移到终端）
- Rope 数据结构简介

## 实战题

设计一个策略：当 `totalLines` 超过 100 万时，如何在**不丢失可检索性**的前提下控制内存？

**提示：**分段归档文件 + 索引 + 视窗只映射文件 offset。

## 11.7 Vim 模式：键位绑定与状态机

路径：`docs/part11-terminal-ui/07-vim-mode.md`

系列：Claude Code 完全指南 V2 · 第 11 篇

### 学习目标

完成本节学习后，你应该能够：

1. **描述** Vim 风格交互在终端 UI 中的 **模式状态机**：Normal / Insert / Visual 等（以产品实现为准）。
2. **解释** 为何键位路由需在 **输入解析之后**、**组件焦点链之前** 做统一仲裁。
3. **列举** 与全局快捷键冲突时的 **优先级** 设计思路。
4. **关联** 11.5：修饰键与 Kitty 协议如何影响 `hjkl` 可靠性。

### 生活类比：汽车档位

Insert 模式像 D 档：踩油门就走（字符进缓冲区）。

Normal 模式像 N 档或 P 档：同样踩「字母键」，车不会前进，而是触发其他机构（命令）。

Vim 模式就是把键盘从「打字」切换成「导航与编辑命令」——终端 TUI 同样需要显式档位，否则会 Agent 输入框 抢键。

### 状态机鸟瞰

[Mermaid 图表 — 请在在线版本中查看]

（状态命名以实际产品为准；上图为教学抽象。）

## 键位路由管线

[Mermaid 图表 — 请在在线版本中查看]

| 层级  | 职责         | 示例          |
|-----|------------|-------------|
| 全局  | 不随模式变化的安全键 | Ctrl+C 中断   |
| Vim | 模式敏感导航     | gg G Ctrl+u |
| 组件  | 文本框内编辑     | 普通字符输入      |

## 源码片段：简单路由（示意）

```
type Mode = 'normal' | 'insert' | 'visual';

type VimContext = {
  mode: Mode;
  buffer: string; // 待处理多键序列如 "di"
};

function routeKey(
  ctx: VimContext,
  ev: KeyEvent,
  sink: { vim?: (e: KeyEvent) => boolean; widget?: (e: KeyEvent) => boolean }
): void {
  if (ev.ctrl && ev.key === 'c') {
    /* 全局中断 */ return;
  }
  if (ctx.mode !== 'insert') {
    if (sink.vim?.(ev)) return;
  }
  sink.widget?.(ev);
}
```

## 绑定表数据结构

常用 Trie 或 前缀表 处理多键命令：

| 方案       | 优点                   | 缺点     |
|----------|----------------------|--------|
| 大 switch | 易读                   | 组合爆炸   |
| Trie     | <span>d</span> 等待第二键 | 需超时取消  |
| 声明式 JSON | 非程序员可配               | 运行时构建表 |

```
type BindingLeaf = { action: string };
type BindingNode = Map<string, BindingNode | BindingLeaf>;

// 例: "gg" -> jumpTop
```

## 与文本选择的协同

Visual 模式常与 鼠标拖拽 (11.9) 共享选择模型：

- 起始 offset
- 结束 offset
- 高亮属性（主题令牌）

## 超时与「前缀等待」

用户按 d 后停顿，系统应：

1. 进入 **pending** 状态，显示状态栏提示
2. 超时后把 d 当作普通命令或丢弃（策略）
3. 与 流式输出 同时发生时，避免 状态栏被冲掉 不可见

## 可发现性

| 手段                  | 说明                      |
|---------------------|-------------------------|
| <span>:</span> 命令面板 | 列出可用动作                  |
| 内置帮助                | <span>?</span> 显示当前模式键位 |
| 与 IDE Bridge        | 侧栏展示快捷键表（第 12 篇）        |

## 测试矩阵

| 场景                      | 期望                          |
|-------------------------|-----------------------------|
| Normal + <code>j</code> | 光标下移，不插入字符                  |
| Insert + <code>j</code> | 插入字符 <code>j</code>         |
| tmux 下修饰键               | 能力检测降级仍可用 <code>hjkl</code> |
| Bracketed Paste         | 粘贴不进 Normal 命令解析            |

## 小结

Vim 模式本质是 模式敏感的路由层 + 多键绑定表 + 与焦点/选择/流式 UI 的协同。实现质量取决于 11.5 输入 的语义是否干净。下一节 11.8 Diff 展示。

## 与 React 组件的关系

建议在根 Provider 注入：

```
type VimApi = {
  mode: Mode;
  dispatch: (cmd: VimCommand) => void;
};
```

子组件用 `useContext` 只读模式，避免 `props` 钻透。

## 常见命令映射（示例）

| 按键                   | Normal 行为       |
|----------------------|-----------------|
| <code>h j k l</code> | 左下上右            |
| <code>0 ^ \$</code>  | 行首/首非空/行尾       |
| <code>gg G</code>    | 顶/底             |
| <code>v</code>       | 进入可视            |
| <code>y</code>       | yank（与剪贴板 11.9） |

## 自测

1. 为何 `:` 开头的命令需要独立 **Commandline** 状态？
2. 设计一个避免 `d` 前缀永久挂起的超时策略。

## 术语

| 英文             | 中文    |
|----------------|-------|
| chord          | 组合键序列 |
| pending prefix | 前缀等待  |

## 安全注意

Vim 宏与「执行寄存器」类能力在 AI 终端中可能 **放大误操作**。产品可 **默认关闭** 或 **二次确认**。

## 性能注意

每次按键全树 reconciler 更新状态栏会贵；优先 **局部 context** 或 **命令总线**。

## 与 389 组件的落点

典型会有：`VimStatusBar`、`ScrollableBuffer`、`InputPrompt`、`SelectionLayer` 协同——阅读源码时从 `VimController` 入口 grep。

## 扩展阅读

- Neovim 键位哲学（对比差异）
- Readline Emacs 模式（互斥用户预期）

## 模式冲突解决

---

若用户同时开启 IDE 快捷键 与 终端 Vim，Bridge 层（12 篇）可把某些键 转发给 IDE，需在文档中标明 优先级。

---

## 伪代码：pending 超时

---

```
let pendingTimer: ReturnType<typeof setTimeout> | null = null;

function onKey(k: string) {
  if (pendingPrefix) {
    clearTimeout(pendingTimer!);
    pendingTimer = setTimeout(() => {
      flushPendingAsLiteral();
    }, 800);
  }
}
```

## 复盘清单

---

- [ ] Normal 下不向 stdin 发送可打印字符到错误目标
  - [ ] Insert 与中文输入法兼容（IME 事件若存在）
  - [ ] 大小写与 `shift` 组合在 Kitty 下正确
- 

## 结语

---

好的 Vim 模式不是「照搬 Neovim」，而是在 Agent 工作流 中提供 可预测的导航与编辑肌肉记忆。

---

## 11.8 Diff 展示：代码修改对比子系统

---

路径： `docs/part11-terminal-ui/08-diff-display.md`

系列： Claude Code 完全指南 V2 · 第 11 篇

---

## 学习目标

完成本节学习后，你应该能够：

- 1. 说明 Diff 子系统在终端 UI 中的职责：把 unified / side-by-side 等结构转成带 ANSI 的区块。
- 2. 区分 语法高亮、行级高亮、字符级高亮 三层的成本与效果。
- 3. 关联 主题系统 (11.10)： `added` / `removed` / `context` 令牌颜色 必须可主题化。
- 4. 理解 与 虚拟滚动 (11.6) 结合时，如何按 hunk 切片而非按原始文件全长渲染。

## 生活类比：批改作文的红笔

老师用红笔划掉旧句、绿笔写上 新句——Diff 就是 红绿标注 的机器版本。终端没有 Word 的修订视图，只能用 前景色、背景色、前缀符号（`+` `-` ）表达。

## 数据流

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## 行类型与主题令牌

| 行类型    | 常见前缀            | 主题令牌示例                    |
|--------|-----------------|---------------------------|
| 上下文    | 空格              | <code>diff.context</code> |
| 删除     | <code>-</code>  | <code>diff.removed</code> |
| 新增     | <code>+</code>  | <code>diff.added</code>   |
| hunk 头 | <code>@@</code> | <code>diff.meta</code>    |



## 源码片段：行对象（示意）

```
type DiffLineKind = 'context' | 'add' | 'del' | 'hunk' | 'eof';

type DiffLine = {
  kind: DiffLineKind;
  raw: string; // 不含前缀或含前缀，按渲染器约定
  oldNo?: number;
  newNo?: number;
};

function paintLine(line: DiffLine, theme: ThemeTokens): string {
  const style = theme.diff[line.kind];
  return ansiApply(style, formatPrefix(line) + line.raw);
}
```

## Unified vs Side-by-side

| 模式           | 终端友好度   | 信息密度  |
|--------------|---------|-------|
| Unified      | 高，单列    | 顺序阅读好 |
| Side-by-side | 需宽列，易裁切 | 对照直观  |

在窄终端自动 **降级 unified** 是常见产品策略。

## 与语法高亮的组合

| 策略             | CPU | 效果     |
|----------------|-----|--------|
| 仅 diff 色       | 低   | 快      |
| 整行词法高亮         | 中   | 更像 IDE |
| 字符级 intra-line | 高   | 最细     |

建议：可见 **hunk** 内才跑高成本高亮（与虚拟滚动一致）。

## Hunk 虚拟化

[Mermaid 图表 — 请在在线版本中查看]

## 制表符与对齐

代码 diff 常含 **Tab**。终端渲染需统一：

- 展开宽度（如 `tab=2` 或 `tab=4`）
- 与 `side-by-side` 列分割对齐

否则会出现「竖线对不齐」的错觉。

## 与 Agent 工具输出对接

工具可能返回：

- Git unified diff
- 结构化 JSON patch

UI 层应 归一化 为 `DiffLine[]`，避免组件感知多种上游格式。

## 无障碍与色弱

| 手段       | 说明                                 |
|----------|------------------------------------|
| 非仅依赖颜色   | 前缀符号保留                             |
| 高对比主题    | 11.10 提供 <code>highContrast</code> |
| 蓝/黄替代红/绿 | 主题变量可换                             |

## 小结

Diff 展示是「审阅 Agent 改代码」的关键屏。把 **hunk 行模型**、**主题令牌**、**可选词法高亮** 与 **虚拟滚动** 组合，才能在长补丁下保持 **流畅与可读**。下一节 11.9 鼠标与超链接。

## 性能清单

- `[]` 大文件 diff **流式解析** 或 `worker`（若架构允许）

- [ ] 避免在每次 `paint` 重新 diff
- [ ] 缓存 `ansi` 字符串 至内容哈希变化

## 测试用例建议

1. 仅换行变更的补丁
2. 重命名 + 内容变更混合
3. 极大列宽行（终端折行 vs 横向滚动策略）

## 自测

1. 解释为何 side-by-side 在 80 列终端可能失败。
2. `diff.meta` 与 `diff.context` 颜色过于接近时如何解决？

## 术语

| 英文           | 中文     |
|--------------|--------|
| hunk         | 差异块    |
| unified diff | 统一差异格式 |

## 与剪贴板

用户可能想复制 `纯文本 diff` 而非 ANSI。提供 `Copy as plain` 动作，剥离转义序列。

## 错误处理

若 `patch` 解析失败，降级为 `原始文本块 + 警告条`，避免白屏。

## 组件拆分（389 中可能命名）

- `DiffView` 容器

- `DiffLineRow` 行渲染
- `DiffGutter` 行号槽
- `DiffHeader` 文件路径与模式切换

---

## Mermaid：用户操作流

---

[Mermaid 图表 — 请在在线版本中查看]

---

## 与权限系统

---

若 diff 包含 密钥片段，需在 上游脱敏；UI 层只做展示，不信任为安全边界。

---

## 扩展

---

支持 inline word diff 时，可在 `DiffLine` 内嵌 `Span[]`：

```
type Span = { text: string; kind: 'equal' | 'ins' | 'del' };
```

---

## 实战题

---

实现「忽略空白」切换时，如何 复用 同一视图组件而 不泄漏 旧高亮缓存？

---

## 结语

---

Diff 子系统是工程师对 Agent 的信任界面：快、准、清晰 比花哨动画更重要。

---

# 11.9 鼠标、文本选择与 OSC 8 超链接

---

路径：`docs/part11-terminal-ui/09-mouse-hyperlinks.md`

## 学习目标

完成本节学习后，你应该能够：

- 1. 说明 终端中 鼠标追踪（SGR 等）如何启用，以及事件如何映射到 字符坐标。
- 2. 描述 文本选择 模型：锚点、拖曳、释放、与 复制 动作。
- 3. 解释 OSC 8 超链接格式如何把 URL 附着到一段可见文本。
- 4. 列举 剪贴板集成：OSC 52、桌面集成回退、权限与安全注意。

## 生活类比：触摸屏上的链接

手机屏幕上，文字块下面可以藏 超链接——你看到的是字，点的是链。终端里 OSC 8 类似：在 纯文本画布 上为一段范围打上「可点的 URL 元数据」。

鼠标追踪则像 触摸屏坐标：系统告诉你用户点在第几行第几列。

## 能力依赖关系

[Mermaid 图表 — 请在在线版本中查看]

## 鼠标事件到网格

解析层（11.5）产出 `x,y` 后，渲染器维护：

| 结构     | 作用                                      |
|--------|-----------------------------------------|
| 字符单元矩阵 | 当前帧每个格子的样式与字符                           |
| 链接索引   | <code>Rect</code> 或 run-length 区间 → URL |
| 选择区间   | <code>[start,end)</code> 逻辑 offset      |

[Mermaid 图表 — 请在在线版本中查看]

## OSC 8 超链接（概念）

格式思想：开启链 → 可见文本 → 关闭链。具体字节序列遵循规范；实现上常封装：

```
type HyperlinkSpan = { url: string; text: string; id?: string };

function osc8Wrap(span: HyperlinkSpan): string {
  // 伪代码: 参数需 URL 编码与 id 可选
  const params = `id=${encodeURIComponent(span.id ?? '')}`;
  const open = `\x1b]8;${params};${span.url}\x1b\\`;
  const close = `\x1b]8;;\x1b\\`;
  return open + span.text + close;
}
```

## 安全与 OSC 8

| 风险     | 缓解                  |
|--------|---------------------|
| 钓鱼链接   | 状态栏显示 真实 URL，点击二次确认 |
| 超长 URL | 截断显示与提示             |
| 不可信会话  | 可配置关闭 OSC 8 点击      |

## 文本选择

| 阶段          | 行为                        |
|-------------|---------------------------|
| pointerDown | 记录 <code>anchor</code>    |
| pointerMove | 更新 <code>active</code>    |
| pointerUp   | 固化 <code>selection</code> |
| doubleClick | 选词（需分词表或启发式）              |
| tripleClick | 选行                        |

与 Vim Visual（11.7）共享 `selection model` 可减少重复。

## 剪贴板：OSC 52 与回退

| 路径           | 环境                   |
|--------------|----------------------|
| OSC 52       | 许多现代终端允许 base64 传剪贴板 |
| Bridge 到 IDE | 第 12 篇：由 IDE 写入系统剪贴板 |
| 无能力          | 显示「请手动复制」或写入临时文件     |

[Mermaid 图表 — 请在在线版本中查看]

## 与流式输出的竞态

流式刷新时 行内容平移 会导致选择区间失效策略：

- 锁定选择：刷新跳过选中区域合并（复杂）
- 清除选择：新内容到达即清空（简单）
- 逻辑锚定：用 消息 id + offset 而非纯坐标

## 源码片段：命中测试（示意）

```
type Cell = { char: string; linkId?: string };

function hitTest(grid: Cell[][], x: number, y: number): Cell | null {
  const row = grid[y];
  if (!row) return null;
  return row[x] ?? null;
}
```

真实实现需处理 双倍宽字符： x 与 逻辑列 非线性。

## iTerm2 / xterm.js 差异

| 点           | 备注                         |
|-------------|----------------------------|
| 焦点          | Web 终端 <b>blur</b> 行为不同    |
| 鼠标追踪启用序列    | 需查各自兼容矩阵                   |
| OSC 52 长度限制 | 大块 base64 可能被截断——分片协议（若支持） |

## 小结

鼠标 + 选择 + OSC 8 + 剪贴板 把终端从「纯键盘」推进到「可点的控制台应用」。实现关键是 坐标 ↔ 逻辑偏移 映射、链接索引、以及 能力降级链。下一节 11.10 设计系统与主题。

## 自测

- 双倍宽字符下 `x++` 命中为何会跳过？
- OSC 8 关闭序列遗漏会导致什么 UI 事故？

## 表格：事件到动作

| 事件   | 无修饰       | Ctrl   |
|------|-----------|--------|
| 左键点击 | 移动光标/打开链接 | 块选（示例） |
| 拖曳   | 文本选择      | —      |
| 滚轮   | 滚动缓冲      | 加速滚动   |

## 术语

| 英文             | 中文   |
|----------------|------|
| hit testing    | 命中测试 |
| wide character | 宽字符  |



## 与 389 组件

---

常见分层： `MouseLayer` 、 `SelectionOverlay` 、 `LinkHintBar` 、 `ScrollView` 。

---

## 调试

---

- 打开 `cell` 网格调试模式（dev）显示每个逻辑列边界。
  - 记录 OSC 8 开启/关闭 是否配对（栈深度）。
- 

## 无障碍

---

纯鼠标交互对键盘用户不友好：`Enter` 打开当前聚焦链接、`Shift+箭头` 扩展选择应保留。

---

## 扩展阅读

---

- iTerm2 文档：复制模式与报告
  - xterm.js 输入文档
- 

## 实战题

---

设计当 `Bridge` 断开 时，复制操作如何 仅依赖本地终端能力 而不静默失败。

---

## 伪代码：链接栈

```
const stack: string[] = [];  
  
function onOpen(url: string) {  
  stack.push(url);  
}  
  
function onClose() {  
  stack.pop();  
}  
  
function currentUrl() {  
  return stack[stack.length - 1];  
}
```

## 性能

命中测试 O(1) 每格理想；每帧重建链接映射可从 **渲染遍历** 同步生成，避免二次扫描。

## 结语

当 Agent 输出里充满 **路径**、**Issue**、**文档链接**，OSC 8 让终端 **像网页一样可导航**——这是现代 CLI 产品体验的分水岭。

# 11.10 设计系统与主题：暗色、亮色与 389 组件一致性

路径： docs/part11-terminal-ui/10-design-system.md

系列： Claude Code 完全指南 V2 · 第 11 篇

## 学习目标

完成本节学习后，你应该能够：

1. 描述 终端 UI 设计系统 的组成：设计令牌、语义色、排版刻度、间距与圆角（字符近似）。
2. 解释 暗色 / 亮色 切换如何流经 **HostCtx**、Provider 与 ANSI 调色。

3. 将 389 个 React 组件 的样式来源归为「令牌优先」而非散落的魔法数字。
4. 关联 Diff（11.8）、超链接（11.9）：语义色在主题切换下保持 对比度。

## 生活类比：城市路灯与路牌

设计令牌像 交通系统标准：路牌颜色、字体大小、反光材料统一，司机 不用重新学习 每个路口。

若每条街自制路牌（魔法颜色），夜间（暗色主题）就会有人看不清——对应终端里 对比度崩溃 与 品牌感消失。

## 令牌层级

[Mermaid 图表 — 请在在线版本中查看]

| 层级   | 稳定性      | 例                         |
|------|----------|---------------------------|
| 原始   | 可随品牌重塑调整 | <code>palette.cyan</code> |
| 语义   | 组件应只依赖这层 | <code>text.primary</code> |
| 组件局部 | 极少数例外    | 一次性实验                     |

## 暗色与亮色映射

[Mermaid 图表 — 请在在线版本中查看]

| 语义   | 暗色直觉   | 亮色直觉  |
|------|--------|-------|
| 主文字  | 近白     | 近黑    |
| 面板背景 | 深灰     | 浅灰    |
| 次要文字 | 灰 400  | 灰 600 |
| 链接   | 高饱和蓝/青 | 深蓝    |

## 源码片段：令牌解析（示意）

```
type ThemeName = 'dark' | 'light';

type TokenTable = Record<string, string>; // '#RRGGBB' 或逻辑名

const dark: TokenTable = {
  'text.primary': '#e6e6e6',
  'diff.added.fg': '#56d364',
  'diff.removed.fg': '#ff7b72',
};

function resolveToken(theme: ThemeName, key: string): string {
  const table = theme === 'dark' ? dark : light;
  return table[key] ?? 'ffffff';
}

function toAnsiFg(hex: string, trueColor: boolean): string {
  // 真彩色 CSI 或 256 色降级
  return trueColor ? ansiTrueColorFg(hex) : ansi256Fallback(hex);
}
```

## 与能力位联动

| 能力     | 主题策略    |
|--------|---------|
| 真彩色    | 直接用 hex |
| 256 色  | 最近色映射表  |
| 仅 16 色 | 粗粒度语义桶  |

## 389 组件治理

| 实践                           | 说明                    |
|------------------------------|-----------------------|
| 禁止组件内写死 <code>#rrggbb</code> | 除调试开关                 |
| lint                         | CI 检查裸色值              |
| Storybook 式快照                | 若仓库有，跑 dark/light 双主题 |

[Mermaid 图表 — 请在在线版本中查看]

## 排版与间距（终端特化）

| Web 概念                     | 终端近似             |
|----------------------------|------------------|
| <code>px</code>            | 字符单元             |
| <code>rem</code>           | 行高倍数             |
| <code>border-radius</code> | ASCII 边框或 色块背景模拟 |

## 状态色与工具态

Agent UI 常见：

| 状态   | 语义令牌示例                            |
|------|-----------------------------------|
| 运行中  | <code>agent.status.running</code> |
| 成功   | <code>agent.status.success</code> |
| 失败   | <code>agent.status.error</code>   |
| 只读探索 | <code>agent.mode.explore</code>   |

## 可访问性

| 检查     | 方法                                                                                                                                                                                   |
|--------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| 对比度    | WCAG 公式对 hex 校验                                                                                                                                                                      |
| 色盲     | 非颜色线索（图标前缀   ） |
| 终端背景透明 | 主题需 <b>alpha</b> 合成假设或禁用混色                                                                                                                                                           |

## 小结

设计系统让 389 个组件 在迭代中仍 视觉一致、主题可切换、Diff/链接/状态 语义稳定。令牌化 + 能力降级是终端产品的 工程纪律。第 11 篇至此从 Yoga → Fiber → 流式 → 输入 → 滚动 → Vim → Diff → 交互 → 主题 形成闭环。

## 本篇复盘表

| 小节    | 关键词                |
|-------|--------------------|
| 11.1  | 自研渲染栈、389 组件       |
| 11.2  | Yoga TS、flex、盒模型   |
| 11.3  | 自定义 reconciler     |
| 11.4  | async generator、合批 |
| 11.5  | Kitty、能力检测         |
| 11.6  | 虚拟滚动               |
| 11.7  | Vim 状态机            |
| 11.8  | Diff 层次            |
| 11.9  | 鼠标、OSC 8、剪贴板       |
| 11.10 | 令牌与主题              |

## 自测

- 为什么组件应消费 **语义令牌** 而非 **palette 原始色**？
- 真彩色不可用时，如何保证 **diff.added** 仍可辨？

## 术语

| 英文             | 中文   |
|----------------|------|
| design token   | 设计令牌 |
| semantic color | 语义色  |

## 与第 12 篇衔接

IDE Bridge 可把 **主题偏好** 从编辑器同步到 CLI，或反向——见 [docs/part12-bridge/](#)。

## 实战题

---

为 `highContrast` 主题增加一套映射，要求 **不改动** 组件代码，仅增 **主题 JSON** ——如何设计令牌继承？

---

## 伪代码：主题继承

---

```
function mergeTheme(base: TokenTable, override: TokenTable): TokenTable {
  return { ...base, ...override };
}
```

## 组件作者清单

---

发布新组件前：

- [ ] 所有颜色来自 `resolveToken`
  - [ ] 焦点环可见（键盘导航）
  - [ ] dark/light 快照通过
  - [ ] diff/代码块使用 **等宽** 令牌
- 

## 扩展阅读

---

- Material Design Token 概念（类比）
  - ANSI 颜色历史与限制
- 

## 结语

---

终端 UI 的「美」来自 **约束中的秩序**：在字符网格里，**令牌化**比「随意调色」更接近 **专业工具** 的长期可维护性。

---

## 第12篇：Bridge桥接

### 第 12 篇：Bridge 桥接 · 12.1 总览：CLI 与 IDE 的双向通道

路径：docs/part12-bridge/index.md

系列：Claude Code 完全指南 V2

#### 学习目标

完成本节学习后，你应该能够：

1. 定义 Bridge：在 CLI 进程与 IDE（VS Code / Cursor / JetBrains）之间的双向通信通道。
2. 说出 Bridge 解决的痛点：共享上下文、统一认证、多会话并发、传输可插拔。
3. 列举仓库中与 Bridge 相关的约 31 个文件可能承担的职责分层（协议、传输、会话、安全）。
4. 建立本篇路线图：IPC → bridgeMain → 协议 → JWT → sessionRunner → 传输 → IDE 实践 → BoundedUUIDSet → 总结。

#### 生活类比：机场廊桥

飞机（CLI 核心逻辑）停在远机位时，乘客要坐摆渡车，慢且易脱节。廊桥让机舱门直接对准航站楼（IDE 窗口）——双向通行：你可以从 IDE 看 CLI 状态，也可以让 CLI 回调 IDE 打开文件。

Bridge 就是这座双向廊桥，而不是单向广播喇叭。



## Bridge 在系统中的位置

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## 为何需要 Bridge?

| 无 Bridge      | 有 Bridge                    |
|---------------|-----------------------------|
| CLI 与编辑器 各说各话 | 结构化消息对齐能力                   |
| 多窗口会话 状态打架    | sessionRunner 管理并发          |
| 任意进程可冒充 IDE   | JWT 校验连接合法性                 |
| 仅适合本地管道       | stdio / WebSocket / TCP 可切换 |

## 31 个 Bridge 相关文件：如何分层阅读

建议从入口 `grep bridgeMain`、`sessionRunner`、`Transport` 等符号，按层归档：

| 分层     | 可能包含                               |
|--------|------------------------------------|
| 入口与主循环 | <code>bridgeMain</code> 、生命周期、错误边界 |
| 协议     | 请求/响应/通知类型、序列化                     |
| 认证     | JWT 签发、校验、时钟偏移                     |
| 会话     | 多会话映射、取消、隔离                        |
| 传输     | stdio、WebSocket、TCP 适配             |
| 工具类    | <code>BoundedUUIDSet</code> 等有界结构  |

(具体文件名以仓库为准；本节提供 心智地图。)

## 源码片段：概念性桥接门面（伪代码）

```
type BridgeFacade = {
  start(): Promise<void>;
  stop(): Promise<void>;
  /** IDE -> CLI */
  onRequest(handler: (req: RpcRequest) => Promise<RpcResponse>): void;
  /** CLI -> IDE */
  notify(event: RpcNotification): Promise<void>;
};
```

## 与第 11 篇终端 UI 的关系

| 终端 UI（11 篇） | Bridge（12 篇）      |
|-------------|-------------------|
| stdout 绘制   | 结构化侧信道（可并存）       |
| 键盘鼠标交互      | IDE 编辑器焦点、diff 内联 |
| 主题本地令牌      | 可从 IDE 同步主题       |

## 安全心智

| 话题       | 要点                       |
|----------|--------------------------|
| JWT      | 验签、过期、受众 claim           |
| 传输       | TLS（WebSocket/TCP）与 证书校验 |
| 本地 stdio | 依赖 OS 管道权限，非网络暴露         |

## 本篇目录

| 小节    | 文件                     | 主题                              |
|-------|------------------------|---------------------------------|
| 12.2  | 02-ipc.md              | stdin/stdout、Unix domain socket |
| 12.3  | 03-bridge-main.md      | 主循环：监听→分发→执行→返回                 |
| 12.4  | 04-protocol.md         | JSON-RPC 风格三类消息                 |
| 12.5  | 05-jwt-auth.md         | JWT 认证                          |
| 12.6  | 06-session-runner.md   | 并发会话                            |
| 12.7  | 07-transport.md        | stdio / WebSocket / TCP         |
| 12.8  | 08-ide-integration.md  | VS Code、Cursor、JetBrains        |
| 12.9  | 09-bounded-uuid-set.md | 有界 UUID 集合                      |
| 12.10 | 10-summary.md          | 总结与检查表                          |

## 常见误解

| 误解                | 澄清                       |
|-------------------|--------------------------|
| Bridge 等于 LSP     | 不同协议与目标，可共存但不要混为一谈       |
| 有 Bridge 就不要终端 UI | 很多用户 仅终端；Bridge 是 增强轨道   |
| JWT 越长越安全         | 短寿命 + 旋转 比冗长 payload 更重要 |

## 小结

Bridge 把 Claude Code 从「孤立的 TTY 应用」提升为「IDE 共生体」：靠 IPC 传血，靠 协议 说话，靠 JWT 验身，靠 sessionRunner 管多开，靠 传输抽象 适配环境。下一节 12.2 夯实 跨进程通信 基础。

## 自测

- 画一张 IDE 发起打开文件 请求的往返箭头图。
- 说出 Bridge 与 纯 stdio Agent 输出 正交的一处证据。

## 延伸阅读建议

先读 02-ipc 与 04-protocol，再读 03-bridge-main，可最快建立「字节如何变消息」的全景。

## 术语表

| 英文  | 中文             |
|-----|----------------|
| IPC | 进程间通信          |
| RPC | 远程过程调用（此处多为本地） |

## 运维视角

| 场景       | 提示                      |
|----------|-------------------------|
| CI 无 IDE | Bridge 可禁用或 mock 传输     |
| 远程开发     | WebSocket/TCP 穿透与 认证更重要 |

## 与多 Agent（第 10 篇）的边界

sessionRunner 管 IDE↔CLI 会话，多 Agent 是 CLI 内部编排；二者可在 会话 id 上对齐，但职责不同。

## 实现清单（读者对照）

- [ ] 启动时选择传输实现
- [ ] JWT 中间件挂载点
- [ ] 请求 id 与响应关联
- [ ] 通知 无响应 的 fire-and-forget 路径
- [ ] 进程退出时 flush 与 关闭套接字

## 质量属性

| 属性  | Bridge 相关手段        |
|-----|--------------------|
| 可靠  | 超时、重试策略（谨慎）、幂等 key |
| 性能  | 批量通知、有界集合          |
| 可观测 | 结构化日志、trace id     |

## 结语

把 Bridge 想成 **产品的一体两面**：终端用户看到 **字符画布**，IDE 用户看到 **编辑器原生体验**——**同一核心，两条前端**。

# 12.2 跨进程通信基础：stdin/stdout 与 Unix Domain Socket

路径： docs/part12-bridge/02-ipc.md  
系列： Claude Code 完全指南 V2 · 第 12 篇

## 学习目标

完成本节学习后，你应该能够：

1. 对比 **标准流（stdin/stdout）** 与 **Unix domain socket（UDS）** 在 Bridge 场景下的优缺点。
2. 解释 为何 **JSON 行协议** 常与 **newline-delimited JSON** 结合出现在管道传输中。
3. 识别 **背压**：读端慢时写端行为、以及 **缓冲区膨胀** 风险。
4. 为 后续 **Transport** 抽象（12.7）建立 **操作系统级直觉**。

## 生活类比：水管 vs 专用电话线

`stdin/stdout` 像 住宅里的通用上下水：装修简单、到处可用，但你要自己约定 水里流的是茶还是油（应用层协议）。

Unix domain socket 像 楼宇内部对讲专线：不经过网络栈、延迟更低、可传递 文件描述符（能力视 OS），适合 本机高吞吐 的 Bridge。

## stdin/stdout 管道模型

[Mermaid 图表 — 请在在线版本中查看]

| 优点         | 缺点               |
|------------|------------------|
| 跨平台（概念上）易用 | 半双工心智易混（实际两路各一根） |
| 与 CI/脚本 友好 | 二进制与多帧需额外分帧      |
| 权限模型简单     | 难做多客户端（一对一管道）    |

## Unix Domain Socket 模型

[Mermaid 图表 — 请在在线版本中查看]

| 优点           | 缺点                   |
|--------------|----------------------|
| 面向连接，可多路复用架构 | 路径/权限要管理             |
| 性能常优于管道突发场景  | Windows 命名管道是另一套 API |
| 可做 凭证（部分 OS） | 调试工具链与 stdout 不同     |

## 分帧：为何不能「裸 JSON 粘包」

TCP/流式 socket 上 无消息边界；stdout 管道同理。常见策略：

| 策略        | 说明                                                     |
|-----------|--------------------------------------------------------|
| NDJSON    | 每行一个 JSON, <code>\n</code> 为界                          |
| 长度前缀      | <code>u32 be length</code> + <code>utf8 payload</code> |
| WebSocket | 帧边界由协议提供                                               |

Bridge 若采用 **JSON-RPC 风格**，传输层必须 **定义分帧**，否则解析器会在 **半个对象** 处失败。

## 背压与流控

[Mermaid 图表 — 请在在线版本中查看]

| 现象       | 解释                               |
|----------|----------------------------------|
| CLI 处理慢  | stdout 缓冲涨，内存压力                  |
| IDE 发送洪泛 | 需 <b>限流</b> 或 <b>队列丢弃策略</b> （谨慎） |

## 源码片段：按行解码（示意）

```
import { createInterface } from 'node:readline';

async function* ndjsonLines(stdin: NodeJS.ReadableStream) {
  const rl = createInterface({ input: stdin, crlfDelay: Infinity });
  for await (const line of rl) {
    if (!line.trim()) continue;
    yield JSON.parse(line);
  }
}
```

生产代码需 **try/catch**、**最大行长度**、**JSON.parse 安全预算**。

## 安全注意

| 面            | 建议                           |
|--------------|------------------------------|
| 本地 socket 文件 | 文件权限 <code>0600</code> 级别视场景 |
| 任意本地连接       | 必须叠 JWT (12.5) 或等价           |
| 环境注入         | 小心 LD_PRELOAD 等 (运维向)        |

## 与 Windows 的映射

| Unix                    | Windows 思路                     |
|-------------------------|--------------------------------|
| UDS                     | 命名管道 Named Pipe 或 loopback TCP |
| <code>fork/spawn</code> | CreateProcess 与句柄继承            |

Bridge 的 Transport 抽象 (12.7) 常为此存在。

## 小结

IPC 基础决定 Bridge 的 延迟、吞吐、安全面。stdin/stdout 简单普适，UDS 强本机连接；二者都需 分帧 与 背压意识。下一节 12.3 bridgeMain 主循环。

## 自测

- 说明 粘包 在 stdout JSON 中的成因。
- 何时 UDS 优于匿名管道？

## 调试技巧

| 工具                                        | 用途               |
|-------------------------------------------|------------------|
| <code>socat</code>                        | 探针 socket        |
| <code>strace</code> / <code>dtruss</code> | 看 read/write 阻塞点 |
| 代理日志                                      | 在 解码前后 打印长度与类型   |



---

## 术语

---

| 英文           | 中文 |
|--------------|----|
| backpressure | 背压 |
| framing      | 分帧 |

---

## 性能数量级（直觉）

---

本机 UDS 常 **微秒级** RTT；真正瓶颈多在 **JSON 序列化** 与 **上层业务**。

---

## 与 12.4 协议衔接

---

分帧之上才有 **id**、**method**、**params** 等字段；否则协议只是字符串约定。

---

## 常见问答

---

问：能用 gRPC 吗？  
答：可以，但需评估 **浏览器/扩展侧** 生态与 **调试成本**；JSON-RPC 风格在脚本与日志上更轻。

---

## 实战题

---

设计 **最大消息大小** 为 8MB 时，如何在解码前 **拒绝** 恶意前缀长度攻击？

---

## 伪代码：长度前缀读取

```
async function readExact(rs: Readable, n: number): Promise<Buffer> {
  const chunks: Buffer[] = [];
  let got = 0;
  while (got < n) {
    const buf = await readChunk(rs, n - got);
    if (!buf) throw new Error('EOF');
    chunks.push(buf);
    got += buf.length;
  }
  return Buffer.concat(chunks, n);
}
```

## 清单：选择传输前的问题

- [ ] 是否 仅本机？
- [ ] 是否需要 多客户端？
- [ ] 是否需要 双向同时 高频？
- [ ] 防火墙/NAT 是否参与？

## 结语

理解 IPC 像理解 **Bridge** 的地基：协议是 **钢筋**，传输是 **地基土质**。

## 12.3 bridgeMain 主循环：监听、分发、执行、返回

路径： `docs/part12-bridge/03-bridge-main.md`

系列：Claude Code 完全指南 V2 · 第 12 篇

## 学习目标

完成本节学习后，你应该能够：

1. 描述 `bridgeMain` 风格主循环的四个阶段：监听 IDE 消息 → 分发 → 执行 → 返回结果。

- 2. 区分 同步处理、异步任务、后台通知 在循环中的调度差异。
- 3. 解释 错误如何映射为 RPC 错误对象 而非裸异常泄漏。
- 4. 关联 `sessionRunner` (12.6)：主循环如何把消息路由到正确会话。

## 生活类比：医院分诊台

分诊台护士 (`bridgeMain`) 不停听病人诉求 (IDE 消息)：

- 1. 听：挂号机吐票 (读取一帧)
- 2. 分：内科还是外科 (`method` 路由)
- 3. 办：医生问诊 (调用核心能力)
- 4. 回：打印回执 (写响应)

若护士把多张票粘在一起读，就会误诊——对应 协议分帧 (12.2)。

## 主循环鸟瞰

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## 处理表 (概念)

| method 类型                                  | 典型行为         |
|--------------------------------------------|--------------|
| <code>ping</code> / <code>handshake</code> | 健康检查、能力交换    |
| <code>runTool</code>                       | 进入沙箱与权限 (他篇) |
| <code>openFile</code>                      | 回调 IDE 或记录意图 |
| <code>subscribe</code>                     | 建立通知频道       |

真实方法名以仓库为准；此处强调 注册表 + 类型安全。

## 源码片段：极简循环骨架（伪代码）

```
async function bridgeMain(tr: Transport, deps: Deps) {
  for await (const frame of tr.incomingFrames()) {
    const msg = decode(frame);
    const auth = await deps.jwt.verify(msg.auth);
    if (!auth.ok) {
      await tr.send(encode(errorResponse(msg.id, 'unauthorized')));
      continue;
    }

    try {
      const session = deps.sessions.resolve(msg.sessionId);
      const result = await dispatch(msg.method, msg.params, session, deps);
      if (msg.kind === 'request' && msg.id !== null) {
        await tr.send(encode({ type: 'response', id: msg.id, result }));
      }
    } catch (e) {
      await tr.send(encode(rpcError(msg.id, e)));
    }
  }
}
```

## 并发模型

| 模型                  | 说明           |
|---------------------|--------------|
| 单线程循环 + worker pool | 常见 Node 风格   |
| 每会话队列               | 避免同会话重入      |
| 全局并发限制              | 保护 CPU 与文件句柄 |

[Mermaid 图表 — 请在在线版本中查看]

## 通知路径

通知无 `id` 或无需响应：主循环可能 单向 `tr.send`。注意 与请求的交错顺序 是否被 IDE 接受。

## 关闭与优雅退出

| 阶段      | 动作                                          |
|---------|---------------------------------------------|
| SIGINT  | 停止接收新请求                                     |
| drain   | 等待在途响应                                      |
| cleanup | 关闭 transport、 <code>sessionRunner</code> 释放 |

## 可观测性

| 字段                     | 作用   |
|------------------------|------|
| <code>traceId</code>   | 关联日志 |
| <code>method</code>    | 指标聚合 |
| <code>latencyMs</code> | SLO  |

## 小结

`bridgeMain` 是 Bridge 的 心脏：把 传输字节 变成 可靠的消息处理管线，并把 会话 与 认证 放在最外层。下一节 12.4 消息协议 细化三类消息。

## 自测

- 为何错误要编码为 **RPC 错误** 而非直接 `throw` 到传输层？
- 同会话串行 vs 全局并行各解决什么 bug？

## 与 JWT 顺序

常见顺序：先验 JWT，再解析业务体；避免 **重 CPU 路由** 在非法连接上浪费。

## 术语

| 英文                | 中文   |
|-------------------|------|
| dispatch          | 分发   |
| graceful shutdown | 优雅退出 |

## 常见坑

| 坑                    | 后果       |
|----------------------|----------|
| 忘记响应 <code>id</code> | IDE 侧 挂起 |
| 通知当请求回               | 协议状态机错乱  |
| 未捕获 Promise          | 静默失败     |

## 测试策略

- 假 transport：内存双端队列
- 属性测试：随机 method 与 id 交错
- 混沌：半关 socket 看恢复

## 实战题

设计 method 未注册 时错误码与 IDE 可恢复 策略。

## 伪代码：注册表

---

```
type Handler = (p: unknown, s: Session) => Promise<unknown>;
const registry = new Map<string, Handler>();

function register(method: string, fn: Handler) {
  registry.set(method, fn);
}

async function dispatch(method: string, params: unknown, session: Session) {
  const fn = registry.get(method);
  if (!fn) throw rpcMethodNotFound(method);
  return fn(params, session);
}
```

## 与 31 文件关系

---

`bridgeMain` 常是 单文件或单模块 焦点；周边为 `protocol` 编解码、`transport` 适配、`session` 存储。

---

## 性能提示

---

热路径避免同步 大 `JSON.stringify`；可考虑 结构化克隆 或 共享缓冲（高级）。

---

## 结语

---

主循环代码往往 不长，但决定 系统性格：稳健、可调试、可测试 —— 比「功能多」更重要。

---

## 12.4 消息协议：JSON-RPC 风格的请求、响应与通知

---

路径： `docs/part12-bridge/04-protocol.md`

系列： Claude Code 完全指南 V2 · 第 12 篇

## 学习目标

完成本节学习后，你应该能够：

- 1. 区分 三种消息类型：请求 (request)、响应 (response)、通知 (notification)。
- 2. 解释 `id` 字段在 异步 环境下的 关联 作用。
- 3. 列举 常见 错误对象 字段：`code`、`message`、`data`。
- 4. 设计 版本协商：`protocolVersion` 或 `capabilities` 交换。

## 生活类比：快递单号

- 请求像 寄件：你拿到 单号 `id`。
- 响应像 签收：必须带回 同一单号，才知道对应哪包裹。
- 通知像 小广告传单：不需要回执，收件人看了就行。

## 三类消息（教学模型）

[Mermaid 图表 — 请在在线版本中查看]

| 类型 | 是否有 <code>id</code> | 是否期待对端回包 |
|----|---------------------|----------|
| 请求 | 有（推荐）               | 是        |
| 响应 | 同请求 <code>id</code> | 否（本身是回包） |
| 通知 | 无 或约定忽略             | 否        |

## JSON-RPC 2.0 对齐点

Bridge 常 借鉴 JSON-RPC 2.0，但可能 扩展 头字段：



| JSON-RPC                    | Bridge 可能扩展                  |
|-----------------------------|------------------------------|
| <code>jsonrpc: "2.0"</code> | 可保留或省略                       |
| <code>method</code>         | 命名空间 <code>claude.xxx</code> |
| <code>params</code>         | 对象优于数组（自描述）                  |
| <code>id</code>             | string/number, 需 稳定序列化       |

## 源码片段：类型判别（示意）

```
type RpcRequest = {
  type: 'request';
  id: string;
  method: string;
  params?: unknown;
};

type RpcResponse = {
  type: 'response';
  id: string;
  result?: unknown;
  error?: RpcError;
};

type RpcNotification = {
  type: 'notification';
  method: string;
  params?: unknown;
};

type RpcError = { code: number; message: string; data?: unknown };
```

## 错误码分层

| 区间              | 含义示例               |
|-----------------|--------------------|
| -32700 ~ -32600 | JSON-RPC 保留（解析错误等） |
| 40xx            | 认证                 |
| 41xx            | 会话                 |
| 42xx            | 方法参数               |
| 43xx            | 业务不可用              |

具体表以产品为准；关键是 稳定文档化。

## 能力协商

[Mermaid 图表 — 请在在线版本中查看]

| 能力项                           | 例                                            |
|-------------------------------|----------------------------------------------|
| <code>maxMessageBytes</code>  | 8192                                         |
| <code>supportsProgress</code> | true                                         |
| <code>editorKinds</code>      | <code>["vscode","cursor","jetbrains"]</code> |

## 版本演进策略

| 策略          | 说明                  |
|-------------|---------------------|
| 向后兼容 加可选字段  | 首选                  |
| method 版本后缀 | <code>foo.v2</code> |
| 特性开关        | capabilities        |

## 与传输分帧关系

协议层假设拿到 完整一条消息；分帧由 12.2/12.7 负责。

## 安全字段放置

| 字段                     | 位置                                           |
|------------------------|----------------------------------------------|
| JWT                    | Authorization 头 或 <code>auth</code> 顶层（团队约定） |
| <code>sessionId</code> | params 或顶层                                   |

避免把密钥塞进 可日志化的 method 名。

## 小结

JSON-RPC 风格 让 IDE 与 CLI 对齐心智：`id` 关联异步、通知减轻握手负担、错误对象可机读。  
下一节 12.5 JWT 认证。

## 自测

- 1. 通知若误带 `id`，接收端应如何处理？（策略）
- 2. `error.data` 适合放什么不适合放什么？

## 与 IDE 生成代码

强类型语言可为 `method` 建立 联合类型 + discriminated union：

```
type Methods =  
  | { method: 'ping'; params?: undefined }  
  | { method: 'open'; params: { path: string } };
```

## 术语

| 英文              | 中文    |
|-----------------|-------|
| correlation id  | 关联 id |
| fire-and-forget | 发后即忘  |

## 调试输出注意

日志打印 截断 params、脱敏 token。

## 实战题

设计 批量请求 `batch` 时，`id` 与 子响应顺序 的约束。

## 伪代码：多路复用等待

```
const pending = new Map<string, Deferred<RpcResponse>>();

function onInbound(msg: RpcResponse) {
  const d = pending.get(msg.id);
  d?.resolve(msg);
  pending.delete(msg.id);
}
```

## 与 bridgeMain

主循环 decode → switch kind → invoke → encode；类型收窄靠 TypeScript 或 zod 校验。

## 常见扩展：Progress

长任务可发 通知 `progress`：

```
{ "type": "notification", "method": "progress", "params": { "token": "t1", "pct": 0.4 } }
```

## 结语

协议是 契约；契约清晰，31 个文件才不会变成 31 种方言。

## 12.5 JWT 认证：验证 IDE 连接合法性

路径：`docs/part12-bridge/05-jwt-auth.md`

系列：Claude Code 完全指南 V2 · 第 12 篇

## 学习目标

完成本节学习后，你应该能够：

1. 说明 Bridge 使用 JWT 的目的：声明式凭证、无会话服务器（视架构）下的快速验签。
2. 列举 必须校验的字段：`exp`、`nbf`、`iss`、`aud`、签名算法 禁止 `none`。
3. 区分 对称（HS256）与 非对称（RS256）在本机 Bridge 中的取舍。
4. 描述 密钥轮换 与 时钟偏移 对认证稳定性的影响。

## 生活类比：游乐园手带

JWT 像 一次性手带：门口盖章（签名）后，园内各摊位（`bridgeMain` 各方法）只看手带不必再问你身份证——但手带 有过期时间，而且 伪造要成本。

## JWT 在 Bridge 中的位置

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## Claims 教学表

| Claim            | 含义    | 校验                              |
|------------------|-------|---------------------------------|
| <code>iss</code> | 签发者   | 白名单                             |
| <code>aud</code> | 受众    | 固定 <code>claude-bridge</code> 等 |
| <code>sub</code> | 主体    | 用户或设备 id                        |
| <code>exp</code> | 过期    | 必须                              |
| <code>nbf</code> | 生效时间  | 可选                              |
| <code>jti</code> | 令牌 id | 防重放（若配合存储）                      |

## 源码片段：验签伪代码

```
import { createRemoteJWKSet, jwtVerify } from 'jose';

type VerifyResult = { ok: true; claims: Claims } | { ok: false; reason: string };

async function verifyJwt(token: string, expectedAud: string): Promise<VerifyResult> {
  try {
    const { payload } = await jwtVerify(token, jwksOrSecret, {
      audience: expectedAud,
      issuer: ALLOWED_ISS,
      algorithms: ['RS256', 'ES256'],
    });
    return { ok: true, claims: payload as Claims };
  } catch (e) {
    return { ok: false, reason: String(e) };
  }
}
```

禁止接受 `alg: none` 或未声明算法的 `kid` 混淆 攻击面——使用成熟库并 固定算法白名单。

## 对称 vs 非对称

| 方案          | 场景       | 备注        |
|-------------|----------|-----------|
| HS256       | 本机共享密钥   | 轮换需 同步两端  |
| RS256/ES256 | IDE 只持公钥 | 私钥集中在签发服务 |

本机 stdio Bridge 可能用 短期 HS256 简化；企业部署倾向 非对称 + JWKS。

## 密钥轮换

[Mermaid 图表 — 请在在线版本中查看]

| 实践                 | 说明             |
|--------------------|----------------|
| <code>kid</code> 头 | 选择正确公钥         |
| 重叠窗口               | 旧 JWT 仍可验证     |
| 紧急吊销               | 需 黑名单 或 极短 TTL |

## 时钟偏移

| 问题         | 缓解                  |
|------------|---------------------|
| IDE 快 5 分钟 | <code>nbf</code> 误判 |
| NTP 未同步    | 运维层面修               |

## 与传输安全关系

| 传输     | JWT 是否足够             |
|--------|----------------------|
| TLS    | 防窃听；JWT 防 伪造         |
| 明文 TCP | 必须 TLS；否则 JWT 泄露即被盗用 |
| 本机 UDS | 依赖 OS 权限 + JWT 纵深防御  |

## 错误映射到 RPC

| 失败      | <code>code</code> 例 | <code>message</code> |
|---------|---------------------|----------------------|
| 过期      | 40101               | token expired        |
| 签名校验失败  | 40102               | invalid signature    |
| aud 不匹配 | 40103               | invalid audience     |

## 小结

JWT 让 Bridge 在 多客户端 与 多会话 场景下仍能 快速拒绝非法连接。正确性来自 严格验签 + claims + 轮换 + 传输层配合。下一节 12.6 sessionRunner。

## 自测

- 为何 仅 Base64 解码 payload 不够？
- `jti` 在无服务器状态下如何实现吊销？

## 威胁建模简表

| 威胁   | 缓解               |
|------|------------------|
| 令牌窃取 | 短 TTL、绑定机器指纹（可选） |
| 重放   | jti + 短期缓存拒绝     |
| 算法降级 | 固定 algorithms    |

## 术语

| 英文    | 中文               |
|-------|------------------|
| claim | 声明/载荷字段          |
| JWKS  | JSON Web Key Set |

## 与 IDE 集成

扩展启动时向 本地服务 换 JWT，或 用户登录 OAuth 后注入——见 12.8。

## 实战题

设计 开发模式 CLAUDE\_BRIDGE\_INSECURE=1 的风险与 编译期剔除 策略。

## 伪代码：clockTolerance

```
await jwtVerify(token, key, {
  audience: AUD,
  clockTolerance: '60s',
});
```

## 日志纪律

永不 console.log(token)；出错只打印 jti 前缀 或 hash。



---

## 合规注意

---

若 JWT 含 PII，日志与崩溃上报需 脱敏。

---

## 结语

---

JWT 不是「万能钥匙」，而是 时间盒凭证：短、可验、可轮换，才配得上 Bridge 这种高频通道。

---

# 12.6 sessionRunner：多会话并发与隔离

---

路径： docs/part12-bridge/06-session-runner.md

系列： Claude Code 完全指南 V2 · 第 12 篇

---

## 学习目标

---

完成本节学习后，你应该能够：

1. 解释 `sessionRunner` 如何管理 多个并发 IDE↔CLI 会话 的生命周期。
  2. 描述 会话隔离：上下文、取消令牌、资源上限。
  3. 关联 `BoundedUUIDSet` (12.9)：限制 活跃会话 id 集合防止 内存泄漏。
  4. 列举 会话结束条件：显式 `close`、进程退出、超时闲置。
- 

## 生活类比：银行窗口叫号

---

每个 会话像 一个窗口客户：柜员系统（Runner）要知道 当前叫到几号、哪个窗口空闲、客户走了要释放柜台。

若只用一个全局计数器，A 客户和 B 客户的 业务单据会混在一起——对应 Agent 上下文串线 的严重 bug。

---

## 会话模型

[Mermaid 图表 — 请在在线版本中查看]

| 对象                           | 职责                    |
|------------------------------|-----------------------|
| <code>SessionId</code>       | 稳定关联 IDE 侧 tab/window |
| <code>SessionState</code>    | 运行中/结束/错误             |
| <code>AbortController</code> | 中断工具调用与流              |

## 并发策略

[Mermaid 图表 — 请在在线版本中查看]

| 策略    | 说明           |
|-------|--------------|
| 会话内串行 | 避免同上下文重入     |
| 会话间并行 | 提高吞吐         |
| 全局上限  | 保护文件描述符与 CPU |

## 源码片段：会话注册（示意）

```
type Session = {
  id: string;
  abort: AbortController;
  createdAt: number;
  lastActiveAt: number;
};

class SessionRunner {
  private sessions = new Map<string, Session>();

  create(id: string): Session {
    if (this.sessions.has(id)) return this.sessions.get(id)!;
    const s = { id, abort: new AbortController(), createdAt: Date.now(), lastActiveAt: Date.now() };
    this.sessions.set(id, s);
    return s;
  }

  dispose(id: string) {
    const s = this.sessions.get(id);
    s?.abort.abort();
    this.sessions.delete(id);
  }
}
```

## 与 bridgeMain 集成

- 1. 解码消息后读取 `sessionId`。
- 2. `runner.create` / `get`。
- 3. 将 `abort.signal` 传入 下游 `fetch` / 子进程。
- 4. 响应后 `touch(sessionId)` 更新活跃时间。

## 闲置超时

| 参数                           | 作用         |
|------------------------------|------------|
| <code>idleTtlMs</code>       | 回收长时间无消息会话 |
| <code>sweepIntervalMs</code> | 周期扫描       |

需 通知 IDE 会话已关闭，避免 幽灵 UI。

## 错误隔离

| 原则           | 说明            |
|--------------|---------------|
| 会话 A 抛错      | 不关闭 全局 Bridge |
| 记录 sessionId | 日志可过滤         |

## 与 BoundedUUIDSet

活跃 sessionId 若无限增长（恶意或 bug），可用 有界集合统计最近 id，配合 LRU 淘汰策略——详见 12.9。

## 小结

sessionRunner 是多窗口 IDE 工作流 的地基：隔离、取消、生命周期 三角缺一不可。下一节 12.7 传输层抽象。

## 自测

- 同会话并发两个 runAgent 请求会发生什么？如何选策略？
- abort 应传播到哪些下游？

## 资源表

| 资源   | 每会话         |
|------|-------------|
| 临时目录 | 可选隔离        |
| 子进程  | 绑定 signal   |
| 打开文件 | ulimit 级别保护 |

## 术语

| 英文         | 中文    |
|------------|-------|
| reentrancy | 重入    |
| teardown   | 拆卸/清理 |

## 测试用例

- 创建 1000 会话后 批量 dispose 不泄漏句柄。
- abort 后 无新 chunk 到达 UI。

## 实战题

设计 会话迁移：IDE 崩溃重启后 恢复 同 `sessionId` 的风险与 新 session 策略。

## 伪代码：每会话队列

```
async function enqueue<T>(sessionId: string, job: () => Promise<T>): Promise<T> {
  return withSessionLock(sessionId, job);
}
```

## 与多 Agent

会话可包含 多个子 Agent（第 10 篇），但 Runner 不直接编排 Agent——只提供 边界与信号。

## 监控指标

| 指标                                  | 含义   |
|-------------------------------------|------|
| <code>sessions_active</code>        | 当前数  |
| <code>sessions_created_total</code> | 累计   |
| <code>session_job_latency</code>    | 处理耗时 |

---

## 结语

---

会话管理是 分布式系统的微缩版：哪怕只有 本机两进程，也要 正经对待隔离。

---

## 12.7 传输层抽象：stdio、WebSocket 与 TCP 可切换

---

路径： [docs/part12-bridge/07-transport.md](#)

系列：Claude Code 完全指南 V2 · 第 12 篇

---

### 学习目标

---

完成本节学习后，你应该能够：

1. 定义 Transport 接口：双向帧流、连接/断开、背压 语义。
  2. 对比 stdio、WebSocket、TCP loopback 的部署场景。
  3. 解释 为何 同一协议（12.4）可复用在多种传输之上。
  4. 列举 WebSocket 下 ping/pong、心跳、重连 的注意点。
- 

### 生活类比：同一种语言，不同交通工具

---

协议像 普通话；传输像 高铁/飞机/公交。你可以 换交通工具 仍说同一种话——Transport 抽象让 Bridge 换连接方式 而不重写 RPC 语义。

---

## 接口形状（示意）

```
type Frame = Uint8Array | string;

type Transport = {
  name: 'stdio' | 'websocket' | 'tcp';
  connect(): Promise<void>;
  close(): Promise<void>;
  readonly incomingFrames: AsyncIterable<Frame>;
  send(frame: Frame): Promise<void>;
};
```

真实实现可能分 二进制/文本 两轨，或统一 `Uint8Array`。

## 三种传输对比

| 传输        | 典型场景                        | 优点              | 风险        |
|-----------|-----------------------------|-----------------|-----------|
| stdio     | 本地扩展 <code>spawn</code> CLI | 简单、权限跟随进程       | 一对一、难远程   |
| WebSocket | 远程/浏览器侧                     | 双向、穿越 HTTP 基础设施 | 代理与心跳     |
| TCP       | 高性能本机或内网                    | 灵活              | 需 TLS、防火墙 |

[Mermaid 图表 — 请在在线版本中查看]

## stdio 实现要点

| 点           | 说明                         |
|-------------|----------------------------|
| stdin 非 tty | 与交互 CLI 抢 stdin 时要协调       |
| stdout 与日志  | 结构化输出与 人类日志 分离（文件或 stderr） |
| Windows     | 换行与编码 UTF-8 统一             |

## WebSocket 实现要点

[Mermaid 图表 — 请在在线版本中查看]

| 点    | 说明                               |
|------|----------------------------------|
| 子协议  | 约定 <code>claude-bridge.v1</code> |
| 消息类型 | text vs binary（与帧格式一致）           |
| 重连   | 恢复会话策略与 id 幂等                    |

## TCP 实现要点

| 点   | 说明                           |
|-----|------------------------------|
| TLS | <code>ca</code> 校验、证书固定（可选）  |
| 粘包  | 长度前缀（12.2）                   |
| 端口  | <code>localhost</code> 绑定 优先 |

## 源码片段：适配器组合（伪代码）

```
class FramingTransport implements Transport {
  constructor(private inner: Transport, private codec: Codec) {}

  async *incomingFrames() {
    for await (const f of this.inner.incomingFrames()) {
      for (const msg of this.codec.decodeChunk(f)) {
        yield msg;
      }
    }
  }

  async send(frame: Frame) {
    for (const chunk of this.codec.encode(frame)) {
      await this.inner.send(chunk);
    }
  }
}
```



## 选择矩阵（运维）

| 环境         | 首选                     |
|------------|------------------------|
| 本地 VS Code | stdio                  |
| Remote SSH | WebSocket 隧道或 stdio 转发 |
| 企业零信任      | mTLS TCP               |

## 小结

**Transport 抽象**让 **Bridge 部署**可选：开发用 **stdio**，远程用 **WS**，性能实验用 **TCP**。协议层保持稳定。下一节 **12.8 IDE 集成实践**。

## 自测

- 为何 **日志** 不应与 **协议 stdout** 混用？
- WebSocket 断线时 **进行中的 RPC** 如何处理？

## 术语

| 英文           | 中文   |
|--------------|------|
| multiplexing | 多路复用 |
| keepalive    | 保活   |

## 与 JWT

WS 可在 **首帧** 发送 **auth** 或在 **子协议+Header**（视客户端能力）。

## 实战题

设计 **双通道**：一条 **低延迟控制面**、一条 **大块日志面**，是否值得？

## 伪代码：stdio 双工

```
const incoming = ndjsonLines(process.stdin);
async function send(obj: unknown) {
  process.stdout.write(JSON.stringify(obj) + '\n');
}
```

## 性能

批量 `send` 减少 `syscall`；注意 `writew` 或缓冲合并。

## 调试

为每种 Transport 提供 内存 `fake`，可在 单元测试 跑全链路。

## 常见坑

| 坑           | 修复                                                 |
|-------------|----------------------------------------------------|
| stdout 缓冲全块 | <code>setEncoding</code> + <code>flush</code> 或换行帧 |
| WS 最大消息     | 对齐 协议 <code>max</code>                             |

## 结语

传输是可替换的 齿轮；不要在齿轮里写业务——这是 31 文件 能维护住的关键。

# 12.8 IDE 集成实践：VS Code 扩展、Cursor 内置与 JetBrains 插件

路径： docs/part12-bridge/08-ide-integration.md  
系列： Claude Code 完全指南 V2 · 第 12 篇

## 学习目标

完成本节学习后，你应该能够：

- 1. 对比 VS Code 扩展、Cursor 内置、JetBrains 插件 三种集成路径的 进程模型 差异。
- 2. 描述 扩展侧如何 spawn 或连接 CLI 并建立 Transport（12.7）。
- 3. 列举 常见 UX 能力：内联 diff、文件树跳转、状态栏、WebView 面板。
- 4. 解释 JWT 获取 在 IDE 内的典型流程（登录/配对/本机密钥）。

## 生活类比：同一台车，三种车钥匙

核心引擎仍是 CLI（或共享库），但 钥匙形状不同：

- VS Code：标准扩展 API 钥匙
- Cursor：内置通道 钥匙（更紧耦合）
- JetBrains：Platform SDK 钥匙

Bridge 协议是 点火线路——钥匙不同，但 线路电压应一致。

## 集成架构对比

[Mermaid 图表 — 请在在线版本中查看]

| 维度   | VS Code                   | Cursor  | JetBrains         |
|------|---------------------------|---------|-------------------|
| API  | <code>vscode.*</code>     | 产品私有/混合 | IntelliJ Platform |
| 终端集成 | <code>Terminal</code> API | 内置终端    | Run Configuration |
| 分发   | Marketplace               | 应用内置    | Plugin Repo       |

## VS Code 扩展路径（概念步骤）

- 1. `activate()` 中注册命令 `claude.start`。
- 2. `child_process.spawn` 启动 CLI，`stdio` Transport。
- 3. `ExtensionContext.secrets` 存 refresh token（若适用）。

4. `window.showTextDocument` 响应 `openFile` 通知/响应。

[Mermaid 图表 — 请在在线版本中查看]

## Cursor 内置路径

| 特点    | 说明                   |
|-------|----------------------|
| 更深集成  | 编辑器事件可直接映射 Bridge 通知 |
| 版本齐步走 | 协议与 IDE 同发版          |
| 调试    | 内部通道可能对 外部插件 不可见     |

## JetBrains 插件路径

| 特点             | 说明                                     |
|----------------|----------------------------------------|
| VirtualFile 体系 | <code>openFile</code> 需映射路径与模块根        |
| 后台任务           | <code>ProgressIndicator</code> 展示长 RPC |
| 多 IDE          | Android Studio 等 兼容矩阵                  |

## 共享：协议客户端 SDK

建议在仓库维护 TypeScript 客户端：

```
export class BridgeClient {
  constructor(private tr: Transport) {}
  async call<T>(method: string, params: unknown): Promise<T> {
    const id = crypto.randomUUID();
    await this.tr.send(encode({ type: 'request', id, method, params }));
    return waitForId(id) as Promise<T>;
  }
}
```

各 IDE 薄适配 即可。

## UX 映射表

| Bridge 消息               | IDE 动作               |
|-------------------------|----------------------|
| <code>openFile</code>   | 打开编辑器并定位行            |
| <code>showDiff</code>   | diff 编辑器或 inline     |
| <code>setStatus</code>  | 状态栏文本                |
| <code>focusPanel</code> | WebView / ToolWindow |

## 安全与权限

| 点     | 说明               |
|-------|------------------|
| 工作区信任 | VS Code Trust 闸门 |
| 路径校验  | 防 目录穿越           |
| 命令执行  | 与权限系统（他篇）对齐      |

## 小结

IDE 集成是 Bridge 的 产品面：三种平台 共享协议，在 spawn/连接/密钥存储/UI API 上分叉。下一节 12.9 BoundedUUIDSet。

## 自测

- 为何扩展侧宜用 `secrets` 而非 `settings.json` 存 JWT？
- `openFile` 应 同步阻塞 还是 异步 fire？

## 分发与版本

| 策略      | 说明            |
|---------|---------------|
| 协议版本 握手 | 防止 新旧混用       |
| CLI 捆绑  | 扩展内置 固定 minor |

## 术语

| 英文             | 中文     |
|----------------|--------|
| extension host | 扩展宿主进程 |

## 调试清单

- [ ] `--bridge-debug` 打印帧类型
- [ ] MITM 本地 loopback (仅 dev)
- [ ] 记录 `method` 直方图

## 实战题

Remote SSH 场景：扩展跑 远端，CLI 应跑 远端还是本地？权衡 文件系统 与 延迟。

## 与终端 UI 第 11 篇

IDE 用户可能 不打开 TUI；Bridge 独立提供 面板与 diff——双前端 并存。

## 用户文档建议

为每种 IDE 提供 三步上手：安装 → 登录/配对 → 打开面板。

## 结语

集成工作 琐碎但关键：用户感知的是 按钮与光标，不是 JSON 帧。

## 发布与遥测（产品向）

| 话题   | 建议                                         |
|------|--------------------------------------------|
| 崩溃报告 | 脱敏 RPC method 与路径                          |
| 功能开关 | <code>settings.json</code> / Registry 统一前缀 |
| A/B  | 协议能力位协商后再启用新 method                        |

## 本地化与可访问性

- 状态栏字符串走 `i18n` 资源，避免硬编码英文。
- 关键操作提供 **键盘快捷方式** 与 **屏幕阅读器** 标签（平台允许范围内）。

# 12.9 BoundedUUIDSet：有界 UUID 集合与防内存泄漏

路径：`docs/part12-bridge/09-bounded-uuid-set.md`

系列：Claude Code 完全指南 V2 · 第 12 篇

## 学习目标

完成本节学习后，你应该能够：

- 解释 为何 Bridge 需要 **有界** 的 UUID 集合：**长时间运行 + 高频 id** → **Set 无限增长** 风险。
- 描述 `BoundedUUIDSet` 的典型语义：`add`、`has`、**淘汰策略**（FIFO、LRU）。
- 权衡 **误判**（Bloom 过滤） vs **精确**（纯 Set + 上限）。
- 关联 `sessionRunner`（12.6）：**活跃会话** 与 **历史 id 去重** 的不同需求。

## 生活类比：门禁卡回收桶

公司发卡（UUID）记录 谁进过门。若只进不出，办公室堆满 废卡盒（内存泄漏）。有界桶策略：最多保留最近 N 张卡复印件——旧复印件扔掉，但 仍在有效期 的卡 另有权威登记（会话表）。

## 问题场景

[Mermaid 图表 — 请在在线版本中查看]

| 场景       | 需要 Bounded?      |
|----------|------------------|
| 已处理通知 id | 是                |
| 当前会话 Map | 用 dispose 而非盲目增长 |
| 调试日志索引   | 可落盘而非内存          |

## API 形状（示意）

```
class BoundedUUIDSet {
  constructor(private readonly maxSize: number) {}

  add(id: string): boolean {
    // 返回是否首次出现
  }

  has(id: string): boolean {}

  get size(): number {}
}
```

## 实现策略对比

| 策略           | 优点      | 缺点               |
|--------------|---------|------------------|
| Set + 队列     | 精确、实现简单 | has O(1)，淘汰 O(1) |
| Map + LRU    | 可刷新热点   | 稍复杂              |
| Bloom Filter | 省内存     | 假阳性              |



Bridge 防重放若不能容忍假阳性，优先 精确结构。

## 源码片段：Set + 队列（示意）

```
class BoundedUUIDSet {
  private readonly set = new Set<string>();
  private readonly q: string[] = [];

  constructor(private readonly maxSize: number) {}

  add(id: string): boolean {
    if (this.set.has(id)) return false;
    this.set.add(id);
    this.q.push(id);
    while (this.q.length > this.maxSize) {
      const old = this.q.shift()!;
      this.set.delete(old);
    }
    return true;
  }

  has(id: string): boolean {
    return this.set.has(id);
  }
}
```

## 与 sessionRunner 的边界

| 结构                                         | 职责               |
|--------------------------------------------|------------------|
| <code>Map&lt;SessionId, Session&gt;</code> | 权威活跃会话           |
| <code>BoundedUUIDSet</code>                | 短期去重（通知 id / 重试） |

勿用 Bounded 结构代替会话释放逻辑。

## 参数选择

| 参数                   | 经验直觉                                     |
|----------------------|------------------------------------------|
| <code>maxSize</code> | 与 峰值 QPS × TTL窗口 相关                      |
| 监控                   | <code>bounded_set_evictions_total</code> |

## 并发

多线程语言需 锁；Node 单线程 也要注意 async 交错 —— 批量 add 时在 同一 tick 内完成。

## 小结

BoundedUUIDSet 是 小工具、大收益：防止 边缘路径 把 长期进程 拖死。最后一节 12.10 总结。

## 自测

1. 队列淘汰后 旧 id 再来 会被如何判断？是否符合你的 防重放 语义？
2. 为何 LRU 有时优于 FIFO？

## 术语

| 英文       | 中文     |
|----------|--------|
| eviction | 淘汰     |
| LRU      | 最近最少使用 |

## 测试用例

- maxSize=3 序列 a b c d 后 has(a) 期望？
- 并发 1e5 随机 uuid 内存平台稳定。

## 实战题

若 jti 需要 强吊销 (12.5)，BoundedSet 不够——应补充什么存储？

## 伪代码：LRU Map

```
// 使用 Map 迭代顺序：最近插入在末尾
function touch(m: Map<string, true>, id: string, max: number) {
  if (m.has(id)) m.delete(id);
  m.set(id, true);
  while (m.size > max) {
    const first = m.keys().next().value;
    m.delete(first);
  }
}
```

## 与 31 文件

该结构常是 独立 util，被 bridgeMain 与 通知子系统 引用。

## 监控告警

若 淘汰速率 持续高于预期，可能 maxSize 过小 或 攻击重放。

## 结语

工程魅力常在 边界条件：有界数据结构是 成熟后端本能 在 Bridge 中的投影。

## 与其他「有界」结构类比

| 结构             | 用途              |
|----------------|-----------------|
| 有界队列           | 限制待处理 RPC 深度    |
| Ring buffer 日志 | 限制内存中 最近 N 条 诊断 |
| BoundedUUIDSet | 限制 去重/防重放 集合    |

统一哲学：长期进程中的任何无限集合都要么落盘，要么有界。

## 复杂度小结

| 操作               | Set+队列实现            |
|------------------|---------------------|
| <code>add</code> | 均摊 $O(1)$           |
| <code>has</code> | $O(1)$              |
| 空间               | $O(\text{maxSize})$ |

在 `maxSize` 为几千到几万量级时，对 Bridge 微不足道；换来 可预测的内存上界。

## 12.10 总结：Bridge 桥接全链路检查表

路径：`docs/part12-bridge/10-summary.md`  
系列：Claude Code 完全指南 V2 · 第 12 篇

## 学习目标

完成本节学习后，你应该能够：

- 1. 复述 Bridge 端到端链路：IPC → 分帧 → JWT → `bridgeMain` → 协议 → `sessionRunner` → Transport → IDE。
- 2. 使用 检查表 自检 实现或阅读源码时的遗漏点。
- 3. 关联 第 11 篇终端 UI：理解 双前端 产品形态。
- 4. 说出 约 31 个 Bridge 文件 在你团队分工中的 模块边界 建议。

## 生活类比：收工前的安检清单

飞行员 降落前 有 检查表，不按表走 容易漏襟翼。Bridge 交付也同理：协议、安全、资源、可观测 一条条勾掉，才能 长期飞行。

## 全链路一张图

[Mermaid 图表 — 请在在线版本中查看]

[Mermaid 图表 — 请在在线版本中查看]

## 模块→文件分工建议（31 文件级）

| 模块          | 职责边界             |
|-------------|------------------|
| transport/* | 仅字节与连接，不知业务      |
| protocol/*  | 编解码、类型、错误码表      |
| auth/*      | JWT、密钥、轮换        |
| bridgeMain* | 循环、路由、生命周期       |
| session/*   | Runner、队列、abort  |
| ide/*       | 可选：共享客户端 SDK     |
| util/*      | BoundedUUIDSet 等 |

## 检查表：安全

| 项         | 通过标准           |
|-----------|----------------|
| JWT 算法白名单 | 拒绝 none        |
| TLS       | 非本机 TCP 必须     |
| 日志脱敏      | 无裸 token       |
| 路径校验      | openFile 根目录限制 |
| 重放        | jti 或短 TTL     |

## 检查表：正确性

| 项                  | 通过标准                 |
|--------------------|----------------------|
| 请求 <code>id</code> | 响应 可关联               |
| 通知                 | 不阻塞等待响应              |
| 错误对象               | 稳定 <code>code</code> |
| 版本握手               | 不兼容则拒绝               |

## 检查表：性能与资源

| 项          | 通过标准                         |
|------------|------------------------------|
| 背压         | 慢消费者 不无限缓冲                   |
| 会话上限       | 拒绝或淘汰策略明确                    |
| Bounded 集合 | 淘汰指标可观测                      |
| JSON       | 大 <code>payload</code> 流式或分块 |

## 检查表：可观测性

| 项                    | 通过标准                   |
|----------------------|------------------------|
| <code>traceId</code> | 跨 IDE-CLI              |
| method 指标            | P95 延迟                 |
| 错误率                  | 按 <code>code</code> 聚合 |

## 与第 11 篇关系小结

| 11 篇      | 12 篇              |
|-----------|-------------------|
| stdout 绘制 | 结构化侧信道            |
| 主题令牌      | 可从 IDE 同步         |
| 鼠标 OSC8   | IDE 原生超链接 可替代部分场景 |

## 31 文件阅读顺序建议

1. 04-protocol 类型定义
2. 07-transport 实现之一
3. 03-bridge-main
4. 06-session-runner
5. 05-jwt-auth
6. 09-bounded-uuid-set
7. 08-ide-integration (产品向)

## 源码片段：启动开关（示意）

```
type BridgeConfig = {
  enabled: boolean;
  transport: 'stdio' | 'websocket' | 'tcp';
  jwt: { audience: string; issuer: string };
  sessions: { max: number; idleMs: number };
  dedupe: { maxIds: number };
};

function assertBridgeConfig(c: BridgeConfig) {
  if (c.transport !== 'stdio' && !c.jwt) {
    throw new Error('JWT required for network transports');
  }
}
```

## 常见反模式

| 反模式              | 后果      |
|------------------|---------|
| 业务逻辑塞进 Transport | 不可测试    |
| 全局单会话            | 多窗口 串线  |
| 无限缓存 uuid        | 内存泄漏    |
| 日志打印帧全文          | 密钥与隐私泄露 |

## 本篇一句提纲

Bridge = 传输上的 JSON-RPC 风格协议 + JWT 信任 + 会话隔离 + IDE 适配。

## 自测总复习

- 1. 画出 粘包 与 分帧 关系。
- 2. 解释 通知 与 请求 在 `bridgeMain` 中的分支。
- 3. `SessionRunner` 与 `BoundedUUIDSet` 各防什么？

## 术语总表

| 英文        | 中文             |
|-----------|----------------|
| Bridge    | 桥接             |
| IPC       | 进程间通信          |
| RPC       | 远程过程调用         |
| JWT       | JSON Web Token |
| Transport | 传输层            |
| session   | 会话             |

## 团队落地建议

| 角色     | 关注点              |
|--------|------------------|
| 后端/CLI | 协议、会话、安全         |
| 扩展开发   | UX 映射、spawn 生命周期 |
| SRE    | TLS、端口、日志        |



## 与系列其他篇的接口

| 系列篇     | 接口                        |
|---------|---------------------------|
| 权限      | <code>runTool</code> 鉴权下游 |
| 工具系统    | 方法 → 工具编排                 |
| 多 Agent | 会话内子 Agent                |

## 结语

Bridge 把 Claude Code 从 终端单形态 扩展到 IDE 共生。掌握 12.2–12.7 即可 读懂大部分故障；12.5/12.6/12.9 决定 能不能长期跑。

## 附录：最小可行演示（伪）

- `socat` 建 TCP 回显（仅实验）。
- `ndjson` CLI 读写 `handshake`。
- 打印 `capabilities`。

## 下一步学习

回到 第 11 篇 看 主题同步，或回到 第 10 篇 看 会话内多 Agent 编排。

## 质量门禁建议

- [ ] 协议 JSON Schema CI 校验
- [ ] 模糊测试解码器
- [ ] 渗透：伪造 JWT、目录穿越

## 致谢读者

若你 逐篇读到 12.10，已具备 通读 Bridge 源码 的 导航能力；剩下是 在仓库里对号入座文件名。

# 第13篇：状态管理

## 第13篇：状态管理 · 第1节 createStore — 极简状态内核

Claude Code 完全指南 V2 · 本篇共 8 节。本节讲解不依赖外部库的轻量状态容器，其设计思想与 Redux 一脉相承，但体积与心智负担远小于完整 Redux 生态。

### 学习目标

完成本节学习后，你应能够：

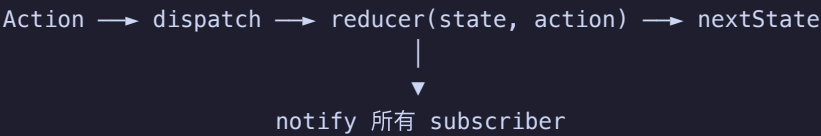
| 能力项 | 说明                                                               |
|-----|------------------------------------------------------------------|
| 理解  | 说明「单一数据源 + 纯 reducer + 订阅通知」与大型状态库的差异与取舍                         |
| 设计  | 手绘或口述 createStore 的最小 API 表面 ( getState / dispatch / subscribe ) |
| 实现  | 在 TypeScript 中实现一个线程安全的简易 createStore，并处理 reducer 组合             |
| 迁移  | 将「组件内 useState 泛滥」重构为「模块级 store + 选择器」的可行路径                      |
| 排错  | 识别「在 reducer 中写副作用」导致的不可复现 bug                                   |

### 生活类比：社区公告栏

想象小区里只有一块电子公告栏（单一 state 树）。任何住户想贴通知，不能直接撕别人的纸，必须把诉求交给物业值班室（dispatch(action)）。值班室按既定章程（reducer）决定公告栏怎么改——章程里只描述「若收到某类消息则新公告栏长什么样」，不负责顺便去帮谁家通下水道（无副作用）。改完后，所有订阅了「公告变更」的住户（subscribe）会收到推送。

Claude Code 里的 createStore 就像这块公告栏 + 章程：状态可预测、变更有迹可循，适合工具型 CLI / Agent 这种需要可测试、可回放心智模型的场景。

## 核心概念速览



- **Store**: 持有 `state` 引用, 对外暴露最小 API。
- **Reducer**: `(state, action) => newState`, 必须**纯函数** (同输入同输出, 不碰 I/O)。
- **Subscriber**: `() => void`, 在 `state` 引用替换后依次调用。

## 源码片段: 极简 createStore (教学示意)

下列代码为**教学抽象**, 与 Claude Code 仓库中具体文件名、导出方式可能不同, 但**模式**一致: 零第三方依赖、几十行量级可维护。

```
// store/createStore.ts - 教学示意: mini Redux 风格
export type Action = { type: string; payload?: unknown };

export type Reducer<S> = (state: S | undefined, action: Action) => S;

export type Listener = () => void;

export interface Store<S> {
  getState: () => S;
  dispatch: (action: Action) => Action;
  subscribe: (listener: Listener) => () => void;
  replaceReducer: (next: Reducer<S>) => void;
}

export function createStore<S>(
  reducer: Reducer<S>,
  preloadedState?: S
): Store<S> {
  let state = preloadedState as S;
  let currentReducer = reducer;
  const listeners = new Set<Listener>();
  let isDispatching = false;

  function getState(): S {
    if (isDispatching) {
      throw new Error("Reducers may not dispatch.");
    }
    return state;
  }

  function subscribe(listener: Listener): () => void {
    listeners.add(listener);
    return () => listeners.delete(listener);
  }

  function dispatch(action: Action): Action {
    if (isDispatching) {
      throw new Error("Reducers may not dispatch.");
    }
    try {
      isDispatching = true;
      state = currentReducer(state, action);
    } finally {
      isDispatching = false;
    }
    listeners.forEach((l) => l());
    return action;
  }

  function replaceReducer(next: Reducer<S>): void {
    currentReducer = next;
    dispatch({ type: "@@/REPLACE" });
  }

  dispatch({ type: "@@/INIT" });
}
```

```
    return { getState, dispatch, subscribe, replaceReducer };
  }
```

## combineReducers 示意

多模块状态常按 key 拆分 reducer，最后在根上合并：

```
export function combineReducers<S extends Record<string, unknown>>(  
  map: { [K in keyof S]: Reducer<S[K]> }  
) : Reducer<S> {  
  return (state, action) => {  
    const next = {} as S;  
    let changed = false;  
    (Object.keys(map) as (keyof S)[]).forEach((key) => {  
      const prevSlice = state?.[key];  
      const nextSlice = map[key](prevSlice, action);  
      next[key] = nextSlice;  
      if (nextSlice !== prevSlice) changed = true;  
    });  
    return changed ? next : (state as S);  
  };  
}
```

---

## Mermaid：数据流与订阅关系

### 图1: dispatch 闭环

[Mermaid 图表 — 请在在线版本中查看]

### 图2: 与「副作用同步」节的衔接（预告）

[Mermaid 图表 — 请在在线版本中查看]

## 对比表：createStore vs 完整 Redux

| 维度   | 本教程 mini store          | Redux + 中间件生态             |
|------|-------------------------|---------------------------|
| 依赖   | 0                       | react-redux、redux-thunk 等 |
| 时间旅行 | 需自实现                    | devtools 成熟               |
| 异步   | 放在 subscriber 或 saga 外层 | thunk / observable        |
| 适用   | CLI、单进程工具内核             | 大型 Web 前端                 |
| 测试   | reducer 单测极简单           | 需 mock 中间件链               |

## 在 Claude Code 语境中的落点

| 场景      | 建议                                         |
|---------|--------------------------------------------|
| 会话内工具开关 | action 粒度小、类型字符串集中管理                       |
| 配置热更新   | <code>replaceReducer</code> 或整树替换需审慎，避免丢订阅 |
| 与文件系统   | 不要在 reducer 里读写磁盘；见第3节「副作用同步」              |
| 与 UI 渲染 | TUI 层 subscribe 后 diff 再重绘，避免全屏闪烁          |

## 常见反模式

| 反模式                                                      | 后果             | 修正                              |
|----------------------------------------------------------|----------------|---------------------------------|
| reducer 内 <code>fetch</code> / <code>fs.writeFile</code> | 状态不可预测、难测      | 纯 reducer + 外层 effect           |
| 可变修改 <code>state.x = 1</code>                            | 订阅方无法感知引用变化    | 返回新对象或 Immer（若引入）               |
| 巨型单一 reducer                                             | 合并冲突、review 困难 | <code>combineReducers</code> 分域 |

## 小结

`createStore` 用最小 API 换来可推理的数据流：所有变更经 `dispatch`，经 `reducer` 折叠为下一状态，再通知监听者。它是 Claude Code 风格应用中全局一致性与可测试性的基石；副作用则刻意留在 store 边界之外，由第3节统一编排。

## 自测清单

1. 为什么 `dispatch` 过程中禁止嵌套 `dispatch` ?
2. `subscribe` 返回的函数是什么设计模式?
3. 若要在测试中断言「某 action 后 state 形状」, 应测 reducer 还是整 store?

下一节: [02-app-state.md](#) — `AppState` 全局应用状态组织。

# 第13篇：状态管理 · 第2节 AppState — 全局应用状态组织

本节将「会话 / 工具 / UI / 配置」四类关注点收拢到一棵可类型化的状态树, 并说明与 `createStore` 的衔接方式。

## 学习目标

| 能力项 | 说明                                                                                                        |
|-----|-----------------------------------------------------------------------------------------------------------|
| 建模  | 为 CLI/Agent 应用划分 <code>session</code> 、 <code>tools</code> 、 <code>ui</code> 、 <code>config</code> 四大域及边界 |
| 类型  | 用 TypeScript 表达嵌套 <code>AppState</code> , 并定义各 slice 的 action 命名空间                                        |
| 选择器 | 编写 memo 友好的 selector, 避免无关重渲染或重复计算                                                                        |
| 安全  | 识别哪些字段属于敏感数据 (令牌、路径), 在日志与快照中脱敏                                                                           |
| 演进  | 预留 <code>version</code> 与 <code>meta</code> 字段, 为迁移节铺路                                                    |

## 生活类比：移动指挥所里的四张桌子

一次野外救援, 指挥所有四张桌子: **人事签到本** (谁在场上)、**工具清单** (铲子无人机是否可用)、**大屏 UI** (当前展示哪块地图)、**操作手册与规章** (配置)。四张桌子上的信息彼此有关但不能混成一本乱账: 人事变动可能影响 UI 上显示的队长名, 但**规章本子**不该被临时涂改——对应**配置与运行时**

态分离。 `AppState` 就是把四张桌子的台账结构印在一张总表上，值班长（reducer）按规矩更新，任何人抬头都能看到一致的全局图景。

---



## AppState 形状（教学示意）

```
// state/appState.ts - 教学示意
export interface SessionSlice {
  sessionId: string;
  cwd: string;
  startedAt: number;
  lastUserMessageAt?: number;
  /** 续接会话时由 History 注入 */
  resumedFromCheckpoint?: string;
}

export interface ToolInvocation {
  id: string;
  name: string;
  status: "pending" | "running" | "success" | "error";
  startedAt: number;
  endedAt?: number;
  /** 仅调试；生产日志应脱敏 */
  argsDigest?: string;
}

export interface ToolsSlice {
  registryVersion: string;
  active: ToolInvocation[];
  lastError?: { code: string; message: string };
}

export interface UiSlice {
  theme: "dark" | "light" | "system";
  layout: "compact" | "comfortable";
  modalStack: string[];
  /** TUI: 当前焦点面板 id */
  focusPaneId?: string;
}

export interface ConfigSlice {
  /** 与磁盘 profile 对齐的版本号 */
  schemaVersion: number;
  model: string;
  approvalPolicy: "off" | "ask" | "strict";
  experimental: Record<string, boolean>;
}

export interface AppState {
  session: SessionSlice;
  tools: ToolsSlice;
  ui: UiSlice;
  config: ConfigSlice;
}
```

## 根 reducer 组合

```
import { combineReducers, Action, Reducer } from "../store/createStore";

const appReducer: Reducer<AppState> = combineReducers({
  session: sessionReducer,
  tools: toolsReducer,
  ui: uiReducer,
  config: configReducer,
});

export function createAppStore(preloaded?: Partial<AppState>) {
  const initial: AppState = {
    session: defaultSession,
    tools: defaultTools,
    ui: defaultUi,
    config: defaultConfig,
    ...preloaded,
  };
  return createStore(appReducer, initial);
}
```

## Action 命名空间约定

| 前缀       | 含义      | 示例 <b>type</b>       |
|----------|---------|----------------------|
| session/ | 会话生命周期  | session/SET_CWD      |
| tools/   | 工具注册与调用 | tools/INVOCATION_END |
| ui/      | 界面与交互   | ui/PUSH_MODAL        |
| config/  | 持久化相关配置 | config/PATCH         |

统一前缀便于日志过滤与遥测聚合（见第14篇）。

## Mermaid：四域依赖（只读视角）

[Mermaid 图表 — 请在在线版本中查看]

图2: dispatch 在各 slice 间的广播

[Mermaid 图表 — 请在在线版本中查看]

选择器与派生状态

```
export const selectActiveTool = (s: AppState) =>
  s.tools.active.find((t) => t.status === "running");

export const selectIsModalOpen = (name: string) => (s: AppState) =>
  s.ui.modalStack.includes(name);

/** 派生: 是否处于「需要用户注意」状态 */
export const selectNeedsAttention = (s: AppState) =>
  !!s.tools.lastError || s.ui.modalStack.length > 0;
```

| 实践              | 说明                     |
|-----------------|------------------------|
| 派生放 selector    | 避免在 reducer 存重复字段导致双写  |
| 昂贵计算 memo       | 大列表 + 过滤时引入 LRU 或显式缓存键 |
| 禁止 selector 副作用 | 与 reducer 同样保持纯函数      |

敏感字段与脱敏策略

| 字段示例                                   | 风险      | 处理               |
|----------------------------------------|---------|------------------|
| <code>sessionId</code>                 | 关联用户轨迹  | 日志只打 hash 前缀     |
| <code>tools.active[].argsDigest</code> | 泄露路径/密钥 | 默认关闭; 开启时加盐 hash |
| <code>config</code> 内 API 相关           | 合规      | 从不进入匿名遥测 payload |

## 与 createStore / 副作用 / 持久化的衔接

| 组件          | 关系                                                                            |
|-------------|-------------------------------------------------------------------------------|
| createStore | 承载整棵 <code>AppState</code>                                                    |
| 副作用同步       | <code>subscribe</code> 监听 <code>config</code> 或 <code>session</code> 变更触发 I/O |
| Memdir      | 用户偏好可能映射到 <code>config.experimental</code> 或独立 mem 文件                         |
| History     | <code>session.resumedFromCheckpoint</code> 与检查点元数据                            |
| Migrations  | <code>config.schemaVersion</code> 驱动升级脚本                                      |
| 持久化         | 仅部分 slice 写入 <code>~/.claude/</code> （见第7节）                                   |

## 表：slice 职责边界

| Slice   | 典型 action 来源 | 是否默认持久化 |
|---------|--------------|---------|
| session | 进程启动、用户输入    | 部分元数据   |
| tools   | MCP/内置工具回调   | 通常不     |
| ui      | 键盘/配置切换      | 部分      |
| config  | 设置页、环境变量合并   | 是       |

## 小结

`AppState` 把运行时真相结构化：会话提供上下文，工具承载 Agent 能力状态，UI 承载人机交互壳层，配置承载策略与模型选择。四域通过 `action 广播 + selector 派生` 协作，而敏感数据与持久化边界需在类型与文档层显式标出。

## 自测

- 为何 `ui` 不应直接写 `session.sessionId`？应通过哪种 action？
- `tools.lastError` 清除动作应放在哪个 reducer？是否可能跨 slice？
- 若新增 `permissions` slice，对 `combineReducers` 与迁移脚本各有什么影响？

上一节: [index.md](#) · 下一节: [03-side-effects.md](#)

# 第13篇：状态管理 · 第3节 副作用同步 — 状态变更触发外部世界

reducer 保持纯粹；副作用在状态提交之后运行。本节讲解如何把「写文件、刷新 TUI、打点」编排成可测试、可取消、可去重的管线。

## 学习目标

| 能力项 | 说明                                                                               |
|-----|----------------------------------------------------------------------------------|
| 分层  | 严格区分「计算下一状态」与「对外界产生影响」                                                           |
| 模式  | 实现 <code>runEffects(prev, next, action)</code> 或中间层 <code>effectScheduler</code> |
| 幂等  | 对同一 state 指纹不重复写盘或重复上报                                                           |
| 顺序  | 处理 effect 之间的先后依赖（如先持久化再通知 UI）                                                   |
| 故障  | effect 失败不反滚已提交 state 时的用户可见策略                                                   |

## 生活类比：法院判决书与执行局

法官敲锤定下判决（新 state 成立），但不会亲自去查封账户。执行局在判决生效后启动（副作用）：冻结、划扣、登记公示。若执行失败，判决书本身通常不会自动作废——而是走「执行异议 / 重试」流程。同理：store 已更新代表应用内部真相已变；写配置文件失败时，可提示用户、重试或回写「脏标记」，而不是悄悄把内存 state 改回旧值造成双真相。

## 核心模式：订阅 + 差异驱动

```
// effects/sync.ts - 教学示意
export type EffectContext = {
  action: { type: string; payload?: unknown };
  prev: AppState;
  next: AppState;
};

export type Effect = (ctx: EffectContext) => void | Promise<void>;

export function attachSideEffects(store: Store<AppState>, effects: Effect[]) {
  let prev = store.getState();
  return store.subscribe(() => {
    const next = store.getState();
    const action = (store as unknown as { __lastAction?: EffectContext["action"] })
      .__lastAction ?? { type: "@@/UNKNOWN" };
    const ctx = { action, prev, next };
    for (const fx of effects) {
      void Promise.resolve(fx(ctx)).catch((e) => {
        console.error("[effect]", e);
      });
    }
    prev = next;
  });
}
```

生产实现中应在 `dispatch` 包装器里传入**真实 lastAction**，上例仅展示结构。

### 带 lastAction 的 dispatch 包装

```
export function withEffectContext(store: Store<AppState>) {
  const raw = store.dispatch.bind(store);
  let last: { type: string; payload?: unknown } = { type: "@@/INIT" };
  store.dispatch = (a) => {
    last = a;
    return raw(a);
  };
  return { ...store, getLastAction: () => last };
}
```

## 典型 effect 表

| Effect        | 触发条件                                        | 行为                                 |
|---------------|---------------------------------------------|------------------------------------|
| persistConfig | <code>next.config !== prev.config</code>    | 写入 <code>~/.claude/settings</code> |
| flushMemdir   | <code>action.type === 'memdir/DIRTY'</code> | 同步偏好目录                             |
| repaintTUI    | <code>ui</code> 任意浅变                        | 局部 diff 渲染                         |
| telemetry     | 关键 action                                   | 匿名事件队列                             |
| auditLog      | 工具调用结束                                      | 追加本地只读日志                           |

## Mermaid：effect 管线

[Mermaid 图表 — 请在在线版本中查看]

### 图2：幂等指纹

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：去重写盘

```
let lastConfigFingerprint = "";

function fingerprintConfig(c: AppState["config"]): string {
  return JSON.stringify({
    v: c.schemaVersion,
    model: c.model,
    policy: c.approvalPolicy,
    exp: c.experimental,
  });
}

const persistConfigEffect: Effect = async ({ prev, next }) => {
  const fp = fingerprintConfig(next.config);
  if (fp === lastConfigFingerprint) return;
  if (fp === fingerprintConfig(prev.config)) return;
  lastConfigFingerprint = fp;
  await atomicWriteJson("~/claude/settings.json", next.config);
};
```

## 与 Memdir / History 的协作

| 模块         | effect 角色                                           |
|------------|-----------------------------------------------------|
| Memdir     | <code>memdir/DIRTY</code> 或定时 flush；避免每次按键写盘        |
| History    | checkpoint 前 snapshot <code>session</code> + 压缩消息列表 |
| Migrations | 在 hydrate 之后、业务 dispatch 之前跑一次性升级 effect            |

## 故障与一致性

| 场景        | 推荐策略                                                     |
|-----------|----------------------------------------------------------|
| 写盘失败      | <code>ui</code> 显示「设置未保存」；内存 state 保持新值                  |
| 遥测失败      | 队列落本地，下次启动重发                                             |
| UI 重绘抛错   | 边界组件捕获，不影响 store                                         |
| 高频 action | <code>requestAnimationFrame</code> 或 debounce 合并 repaint |

## 表：禁止 vs 允许

| 位置       | 禁止                                        | 允许                            |
|----------|-------------------------------------------|-------------------------------|
| reducer  | <code>fs</code> 、 <code>fetch</code> 、随机数 | 纯计算                           |
| selector | I/O、Date.now 用于缓存键                        | 纯派生                           |
| effect   | 修改 reducer 输入的旧对象                         | 读 <code>next</code> 、调用外部 API |

## 小结

副作用同步的本质是：**先承认新状态，再与世界对齐**。通过 `subscribe` + `prev/next` 对比 + 指纹去重，可以在不引入重型中间件的前提下，保持 Claude Code 风格应用的**可测性与可恢复性**。



## 自测

1. 若 effect 内再次 `dispatch`，需注意哪些死锁或顺序问题？
2. 如何把「仅会话结束写一次」与「配置立即写」放在同一 runner？
3. `atomicWriteJson` 在实现上通常用什么策略避免半写文件？

## 进阶：按 action 类型的 effect 注册表

将 effect 从线性数组改为模式匹配，可降低无关调用的空转成本（大状态树时尤其明显）：

```
type EffectMap = Partial<Record<string, Effect[]>>;

export function attachKeyedEffects(
  store: Store<AppState>,
  map: EffectMap,
  getLastAction: () => { type: string }
) {
  let prev = store.getState();
  return store.subscribe(() => {
    const next = store.getState();
    const action = getLastAction();
    const list = map[action.type] ?? [];
    const ctx = { action, prev, next };
    for (const fx of list) void Promise.resolve(fx(ctx)).catch(console.error);
    prev = next;
  });
}
```

| 设计点                | 说明                                                |
|--------------------|---------------------------------------------------|
| 默认桶 <code>*</code> | 可放「任意 action 都跑」的遥测                               |
| 顺序                 | 同 type 多 effect 时数组顺序即执行序                         |
| 测试                 | 对 <code>map['tools/INVOCATION_END']</code> 单测即可隔离 |

## 与架构全景（第8节）的对照

副作用层是有向无环图上的叶节点：只读 `next`、写外部世界，禁止再经隐蔽通道改写 reducer 输入。这样在全链路调试时，只需比对「action 序列 + 最终 state」，不必猜测某个定时器是否偷偷改了内存。

上一节: [02-app-state.md](#) · 下一节: [04-memdir.md](#)

## 第13篇：状态管理 · 第4节 Memdir — 基于文件系统的记忆目录

**Memdir** 指一类「把结构化偏好与长期记忆落在磁盘目录」的方案：人类可读、可 diff、可备份，并与 **AppState** / 副作用层松耦合。

### 学习目标

| 能力项 | 说明                                                           |
|-----|--------------------------------------------------------------|
| 布局  | 设计 <code>~/.claude/mem/</code> 下多级目录与文件命名规范                  |
| 原子性 | 使用临时文件 + rename 保证崩溃下不损坏主文件                                  |
| 模式  | 区分「键值偏好」「片段记忆」「索引清单」三类文档                                     |
| 同步  | 与 store 的 <code>subscribe</code> 或显式 <code>flush()</code> 协调 |
| 隐私  | 用户可整体打包导出或 <code>.gitignore</code> 式排除                       |

### 生活类比：书房抽屉里的分类便签

有人喜欢把所有备忘写在一个巨型 Word 里——**难找、难合并**。Memdir 像是在书房设了**多个贴好标签的抽屉**：「常用联系人」「菜谱」「项目 A 灵感」。每张便签是一张**独立小文件**，搬家时整抽屉端走即可。Agent 读记忆时不必一次加载全部，可按**路径约定**只打开相关抽屉，**降低 token 与延迟**。

# 目录结构示意

```
~/ .claude/
├── settings.json          # 配置 (非 memdir 本体, 常与之并列)
├── mem/
│   ├── preferences/
│   │   ├── editor.json
│   │   └── shortcuts.yaml
│   ├── projects/
│   │   └── acme-app/
│   │       ├── context.md
│   │       └── decisions/
│   │           └── 2026-04-02-auth.md
│   └── index.json        # 可选: 轻量索引, 指向热点文件
```

## 读写 API 示意

```
// memdir/Memdir.ts - 教学示意
import fs from "node:fs/promises";
import path from "node:path";

export class Memdir {
  constructor(private root: string) {}

  private resolve(rel: string): string {
    const full = path.join(this.root, rel);
    if (!full.startsWith(this.root)) throw new Error("path traversal");
    return full;
  }

  async readText(rel: string): Promise<string | null> {
    try {
      return await fs.readFile(this.resolve(rel), "utf8");
    } catch {
      return null;
    }
  }

  async writeTextAtomic(rel: string, body: string): Promise<void> {
    const target = this.resolve(rel);
    await fs.mkdir(path.dirname(target), { recursive: true });
    const tmp = `${target}.${process.pid}.tmp`;
    await fs.writeFile(tmp, body, "utf8");
    await fs.rename(tmp, target);
  }

  async mergeJson(rel: string, patch: Record<string, unknown>): Promise<void> {
    const raw = (await this.readText(rel)) ?? "{}";
    const cur = JSON.parse(raw) as Record<string, unknown>;
    const next = { ...cur, ...patch };
    await this.writeTextAtomic(rel, JSON.stringify(next, null, 2));
  }
}
```

## 与 AppState 的映射

| AppState 路径                      | Memdir 文件                                         | 说明                  |
|----------------------------------|---------------------------------------------------|---------------------|
| <code>config.experimental</code> | <code>mem/preferences/flags.json</code>           | 可与 feature flags 叠加 |
| 项目级上下文                           | <code>mem/projects/&lt;slug&gt;/context.md</code> | 长文，按需加载             |
| 用户显式「记住」                         | <code>mem/preferences/user-notes.md</code>        | 人工可编辑               |

## Mermaid：加载与脏写回

[Mermaid 图表 — 请在在线版本中查看]

### 图2：多进程与文件锁（概念）

[Mermaid 图表 — 请在在线版本中查看]

## 结构化存储：Markdown + frontmatter

```
---
id: decision-auth-20260402
project: acme-app
tags: [auth, oauth]
---

# 决策：采用 OAuth2+PKCE

原因：.....
```

解析时先读 YAML 头，正文供人类与 Agent 共读。

## 表：Memdir vs 单一大 JSON

| 维度   | Memdir 多文件 | 单文件 JSON |
|------|------------|----------|
| 合并冲突 | 文件级，较易手工解决 | 整文件冲突    |
| 部分读取 | 易          | 需解析全量    |
| 备份   | rsync 友好   | 单点       |
| 一致性  | 需索引或约定     | 天然单快照    |

## 副作用集成片段

```
let dirty = false;
let timer: NodeJS.Timeout | null = null;

export function scheduleMemdirFlush(mem: Memdir, getSlice: () => object) {
  dirty = true;
  if (timer) clearTimeout(timer);
  timer = setTimeout(async () => {
    if (!dirty) return;
    dirty = false;
    await mem.writeTextAtomic(
      "preferences/editor.json",
      JSON.stringify(getSlice(), null, 2)
    );
  }, 400);
}
```

## 隐私与合规

| 项  | 建议                                                  |
|----|-----------------------------------------------------|
| 路径 | 默认在用户 home，不进入项目仓库                                  |
| 导出 | 提供「导出记忆包」显式勾选范围                                     |
| 清理 | <code>claude mem purge --older-than</code> 类命令（教学名） |

## 小结

Memdir 用文件系统当数据库：扩展性好、可人工审计、与 Git 式工作流相容。关键是 原子写、路径净化、脏标记 + debounce，以及与 AppState 的单向数据流（state 为源，磁盘为投影）。

## 自测

- 为何 `writeTextAtomic` 要先写临时文件再 `rename` ？
- `index.json` 与真实文件不同步时如何修复？
- 多机器同步 Memdir 时有哪些冲突解决策略？

上一节：[03-side-effects.md](#) · 下一节：[05-history.md](#)

# 第13篇：状态管理 · 第5节 History — 会话历史与续接 (claude -c)

会话历史不仅是聊天记录：**检查点、元数据、工具轨迹**共同决定能否可靠 **resume**。本节以 `claude -c` 心智模型讲解 History 的存储与回放。

## 学习目标

| 能力项 | 说明                                                       |
|-----|----------------------------------------------------------|
| 模型  | 区分 transcript、checkpoint、session manifest                |
| 续接  | 解释 <code>-c</code> / <code>--continue</code> 如何定位最近可恢复会话 |
| 压缩  | 长会话下载剪与摘要策略对 state 的影响                                   |
| 一致  | 恢复后 <code>sessionId</code> 、cwd、工具注册版本的对齐                |
| 排错  | 损坏历史文件时的降级路径（新建会话、只读模式）                                  |

## 生活类比：电视剧的「上一集回顾」

追剧时点「继续观看」，播放器要知：哪一季哪一集、上次看到第几分钟、字幕语言是否照旧。若云端记录损坏，至少从头播——对应 **无法续接则新开 session**。History 就是这份**播放进度 + 剧情摘要**：全量台词 (transcript) 太大时，用「前情提要」(摘要 checkpoint) 换体积。

# 历史数据分层

```
~/.claude/
├── projects/
│   └── <project-hash>/
│       ├── sessions/
│       │   └── <session-id>/
│       │       ├── transcript.jsonl
│       │       ├── checkpoints/
│       │       │   └── cp-0007.json
│       │       └── meta.json
│       └── ...
└── cache/ # 可选：嵌入向量等，续接时非必须
```

| 文件                 | 用途                             |
|--------------------|--------------------------------|
| transcript.jsonl   | 追加式消息流，一行一条事件                  |
| checkpoints/*.json | 周期性快照：消息游标 + 摘要 + 关键 tool 结果引用 |
| meta.json          | startedAt、cwd、model、cliVersion |



## 元数据类型示意

---

```
// history/types.ts - 教学示意
export interface SessionMeta {
  sessionId: string;
  projectPath: string;
  cwd: string;
  cliVersion: string;
  model: string;
  createdAt: string;
  updatedAt: string;
  /** 续接链: 父 session (fork 场景) */
  parentSessionId?: string;
}

export interface TranscriptLine {
  ts: string;
  role: "user" | "assistant" | "tool" | "system";
  /** 工具事件可含 invocation id */
  payload: Record<string, unknown>;
}

export interface Checkpoint {
  id: string;
  atMessageIndex: number;
  summary: string;
  /** 大 payload 仅存引用, 不内联全文 */
  artifactRefs?: string[];
}
```

## 续接流程（`claude -c`）

```
// history/resume.ts - 教学示意
export async function findLatestResumableSession(
  projectRoot: string
): Promise<SessionMeta | null> {
  const sessions = await listSessions(projectRoot);
  const sorted = sessions.sort(
    (a, b) => Date.parse(b.updatedAt) - Date.parse(a.updatedAt)
  );
  for (const m of sorted) {
    if (await isSessionHealthy(m)) return m;
  }
  return null;
}

export async function continueSession(meta: SessionMeta): Promise<LoadResult> {
  const cp = await loadLatestCheckpoint(meta.sessionId);
  const tail = await readTranscriptTail(meta.sessionId, cp?.atMessageIndex);
  return { meta, checkpoint: cp, tail };
}
```

## Mermaid：续接时序

[Mermaid 图表 — 请在在线版本中查看]

### 图2：消息流与检查点

[Mermaid 图表 — 请在在线版本中查看]

## 与 AppState 的映射

| History 字段                 | AppState                                   |
|----------------------------|--------------------------------------------|
| <code>sessionId</code>     | <code>session.sessionId</code>             |
| <code>cwd</code>           | <code>session.cwd</code>                   |
| <code>checkpoint.id</code> | <code>session.resumedFromCheckpoint</code> |
| 工具版本漂移                     | <code>tools.registryVersion</code> 与警告横幅   |

## 压缩与裁剪策略

| 策略                         | 优点           | 风险      |
|----------------------------|--------------|---------|
| 固定窗口保留 N 条                 | 实现简单         | 丢早期上下文  |
| 摘要 checkpoint              | 体积小          | 摘要失真    |
| 工具结果外置                     | transcript 轻 | 引用失效需检测 |
| 用户手动 <code>/compact</code> | 可控           | 需产品暴露   |

## 表：健康检查 isSessionHealthy

| 检查项                       | 失败处理            |
|---------------------------|-----------------|
| <code>meta.json</code> 可读 | 跳过              |
| transcript 尾部 JSON 合法     | 尝试截断修复或跳过       |
| checkpoint 索引 $\leq$ 消息数  | 丢弃无效 checkpoint |
| 磁盘只读                      | 提示无写权限，禁止续接写模式  |

## 源码片段：追加 transcript

```
export async function appendTranscriptLine(
  sessionId: string,
  line: TranscriptLine
): Promise<void> {
  const p = transcriptPath(sessionId);
  await fs.appendFile(p, JSON.stringify(line) + "\n", "utf8");
}
```

生产环境可配合 文件锁 或 单写队列 避免并发 CLI 实例撕裂 jsonl。

## 多实例与锁

---

```
// 简化: advisory lock 文件
export async function withSessionLock<T>(  
  sessionId: string,  
  fn: () => Promise<T>  
) : Promise<T> {  
  const lockPath = path.join(sessionDir(sessionId), ".lock");  
  const fd = await fs.open(lockPath, "wx").catch(() => null);  
  if (!fd) throw new Error("session busy");  
  try {  
    return await fn();  
  } finally {  
    await fd.close();  
    await fs.rm(lockPath).catch(() => {});  
  }  
}
```

## 小结

---

History 让 **时间维度** 成为一等公民: **transcript** 保真、**checkpoint** 换空间、**meta** 保上下文。  
**claude -c** 是「从磁盘hydrate 到内存 AppState」的用户可见入口; **健康检查与原子追加** 决定体验是否可靠。

## 自测

---

1. 续接时若模型从 A 换成 B, 应在哪一层提示用户?
2. checkpoint 的 **atMessageIndex** 与 jsonl 行号如何对齐 (含 system 行)?
3. 为何工具大结果适合外置 artifact 而非内联 transcript?

---

上一节: [04-memdir.md](#) · 下一节: [06-migrations.md](#)

---

# 第13篇：状态管理 · 第6节 Migrations — 配置文件版本迁移

用户目录里的 `settings.json`、`feature` 清单、Memdir 索引会随 CLI 版本演进。  
**Migration** 在启动早期把旧格式抬升到当前 `schemaVersion`，避免「新代码读旧文件」静默出错。

## 学习目标

| 能力项 | 说明                                               |
|-----|--------------------------------------------------|
| 版本  | 在配置根上维护单调递增 <code>schemaVersion</code>           |
| 管线  | 实现 <code>runMigrations(from, to)</code> 链式或阶梯式升级 |
| 幂等  | 同一版本重复运行不产生差异                                    |
| 回滚  | 区分「不可回滚」字段与用户显式备份                                |
| 测试  | fixture 覆盖 v1→v2→v3 全路径                          |

## 生活类比：户籍系统升级

一代身份证换二代，不是让办事员脑内兼容两种底卡，而是集中换发：旧数据读入 → 按规则补全字段 → 写入新卡 → 更新「档案版本号」。若某人已经二代证，窗口不应再打孔换一遍——对应迁移的幂等。若新系统发现字段非法，应拒绝启动并提示备份路径，而不是写半个文件。

## 迁移记录结构

```
// migrations/types.ts - 教学示意
export type Migration = {
  /** 应用此迁移后应达到的版本 */
  version: number;
  /** 人类可读说明 */
  description: string;
  up: (raw: unknown) => unknown;
};

export const CURRENT_SCHEMA_VERSION = 7;

export const migrations: Migration[] = [
  {
    version: 2,
    description: "rename approvalMode -> approvalPolicy",
    up: (raw) => {
      const o = raw as Record<string, unknown>;
      if (typeof o.approvalMode === "string" && o.approvalPolicy == null) {
        o.approvalPolicy = o.approvalMode;
        delete o.approvalMode;
      }
      return o;
    },
  },
  {
    version: 3,
    description: "nest experimental flags under experimental{}",
    up: (raw) => {
      const o = raw as Record<string, unknown>;
      const legacy = o.experimental;
      if (legacy !== null && typeof legacy === "object") return o;
      const flags = { ...o } as Record<string, unknown>;
      const experimental: Record<string, boolean> = {};
      for (const k of Object.keys(flags)) {
        if (k.startsWith("feat.")) {
          experimental[k.slice(5)] = Boolean(flags[k]);
          delete flags[k];
        }
      }
      flags.experimental = experimental;
      return flags;
    },
  },
  // ... 递增
];
```

## 运行器

```
export function runMigrations(
  raw: unknown,
  migrationsSorted: Migration[],
  targetVersion: number
): { value: unknown; from: number; to: number } {
  const obj =
    raw && typeof raw === "object"
      ? (raw as Record<string, unknown>)
      : ({} as Record<string, unknown>);
  let from = Number(obj.schemaVersion ?? 1);
  if (!Number.isFinite(from)) from = 1;

  let cur = obj;
  for (const m of migrationsSorted) {
    if (m.version <= from) continue;
    if (m.version > targetVersion) break;
    cur = m.up(cur) as Record<string, unknown>;
    cur.schemaVersion = m.version;
    from = m.version;
  }
  return { value: cur, from, to: targetVersion };
}
```

## Mermaid：启动时迁移顺序

[Mermaid 图表 — 请在在线版本中查看]

## 图2：多文件迁移（Memdir + settings）

[Mermaid 图表 — 请在在线版本中查看]

## 表：迁移 vs 运行时 patch

| 维度   | Migration（启动） | dispatch <span>config/PATCH</span> |
|------|---------------|------------------------------------|
| 触发   | 进程启动一次        | 任意时刻                               |
| 用户感知 | 可打日志「已升级配置」   | 即时                                 |
| 回滚   | 备份旧文件         | 内存级 undo 栈（可选）                     |
| 失败   | 阻断启动或降级默认     | 提示错误，保持旧 config                    |

## 备份策略

```
export async function migrateWithBackup(
  filePath: string,
  targetVersion: number
): Promise<void> {
  const raw = JSON.parse(await fs.readFile(filePath, "utf8"));
  const backup = `${filePath}.bak.${Date.now()}`;
  await fs.copyFile(filePath, backup);
  const { value } = runMigrations(raw, migrations, targetVersion);
  await atomicWriteJson(filePath, value);
}
```

| 策略                    | 说明                                              |
|-----------------------|-------------------------------------------------|
| 时间戳备份                 | 简单；磁盘增长需轮转                                      |
| 单备份 <span>.bak</span> | 覆盖；节省空间                                         |
| 用户环境变量跳过              | <span>CLAUDE_NO_CONFIG_MIGRATION=1</span> （教学名） |

## 测试 fixture 示例

```
describe("migrations", () => {
  it("v1 -> v2 renames approvalMode", () => {
    const raw = { approvalMode: "ask" };
    const { value } = runMigrations(raw, migrations, 2);
    expect((value as any).approvalPolicy).toBe("ask");
    expect((value as any).approvalMode).toBeUndefined();
  });
});
```



## 与 Feature Flags 的交界

| 情况           | 处理                                                  |
|--------------|-----------------------------------------------------|
| 旧文件含已删除 flag | 迁移 <code>up</code> 里 <code>strip</code> ，并记录日志      |
| 新 flag 默认值   | 在 reducer <code>initialState</code> 合并，而非迁移硬编码业务默认值 |
| A/B 与 schema | schema 只描述形状；实验由 flags 层控制                          |

## 小结

Migrations 是时间旅行中的护栏：通过 `schemaVersion` 与有序 `up` 函数，把「用户磁盘上的过去」安全映射到「当前代码的现在」。备份 + 原子写 + 幂等是生产三件套。

## 自测

- 为何 `up` 不应依赖网络或读取全局单例？
- 若用户手动把 `schemaVersion` 改大，会发生什么？如何防御？
- 分支版本 hotfix 与 migration 版本号如何协调（git vs schema）？

上一节：[05-history.md](#) · 下一节：[07-persistence.md](#)

# 第13篇：状态管理 · 第7节 持久化策略 — 什么该留下、什么该随风

并非所有 `AppState` 都值得写入磁盘。**持久化策略**在隐私、性能与可恢复性之间划界：  
`~/.claude/` 是默认大本营，但子目录职责各不相同。

## 学习目标

| 能力项 | 说明                                   |
|-----|--------------------------------------|
| 分类  | 列举应持久化 / 会话级 / 纯内存三类状态               |
| 位置  | 默认识别 <code>~/.claude/</code> 下各子路径职责 |
| 粒度  | 决定同步写、debounce、退出时 flush             |
| 清理  | 缓存与日志轮转策略                            |
| 跨机  | 哪些目录可安全同步（Syncthing、dotfiles 仓库）     |

## 生活类比：行李箱与酒店房卡

托运大箱 = 持久化配置与长期记忆（回家还要用）。随身小包 = 会话内临时状态（退房即弃）。酒店房卡 = 纯内存 token（丢卡可补办，不应复印一份贴墙上）。错误地把房卡复印件存进托运箱，等于把短期凭证长期化——安全风险。持久化策略就是：什么进托运、什么只揣兜里。

## ~/.claude/ 布局总表

| 路径                                   | 内容类型         | 典型持久化   | 备注           |
|--------------------------------------|--------------|---------|--------------|
| <code>settings.json</code>           | 用户配置         | 是       | 经 migrations |
| <code>mem/**</code>                  | Memdir 偏好与笔记 | 是       | 用户可读         |
| <code>projects/**/sessions/**</code> | History      | 是       | 可体积大         |
| <code>cache/**</code>                | 嵌入、索引        | 可丢      | 可重建          |
| <code>logs/**</code>                 | 诊断日志         | 轮转      | 敏感需脱敏        |
| <code>plugins/**</code>              | 扩展           | 视安装方式   | 版本化          |
| <code>.credentials</code> 类          | OAuth token  | 是（加密最佳） | 权限严格         |

## AppState slice 持久化矩阵

| Slice                 | 持久化      | 理由                            |
|-----------------------|----------|-------------------------------|
| config                | 是        | 用户显式选择应跨会话保留                  |
| session               | 部分       | sessionId /cwd 写入 meta；瞬时指针不必 |
| tools.active          | 否        | 进程内队列；续接时重建                   |
| tools.registryVersion | 可写入 meta | 续接一致性                         |
| ui.theme              | 是        | 用户偏好                          |
| ui.modalStack         | 否        | 会话 UI 瞬态                      |
| tools.lastError       | 否        | 提示后即消                         |

## 策略代码示意

```
// persistence/policy.ts - 教学示意
export const PERSIST_DEBOUNCE_MS = 400;

export function shouldPersistConfig(prev: AppState, next: AppState): boolean {
  return prev.config !== next.config;
}

export function shouldTouchSessionMeta(prev: AppState, next: AppState): boolean {
  return (
    prev.session.sessionId !== next.session.sessionId ||
    prev.session.cwd !== next.session.cwd
  );
}

export function ephemeralSlices(): (keyof AppState)[] {
  return []; // 不持久化整 slice；仅持久化投影
}
```

## Mermaid：数据落盘决策树

[Mermaid 图表 — 请在在线版本中查看]

## 图2：写入时序

[Mermaid 图表 — 请在在线版本中查看]

## 丢弃策略

| 数据         | 丢弃时机        | 实现        |
|------------|-------------|-----------|
| 内存 tool 队列 | 进程退出        | 自然 GC     |
| cache      | 磁盘配额 / TTL  | LRU       |
| 旧 session  | 用户 prune 命令 | 按日期归档     |
| 实验性 UI 状态  | 每次启动        | 不 hydrate |

## 环境变量与覆盖（教学名）

| 变量                | 作用                  |
|-------------------|---------------------|
| CLAUDE_CONFIG_DIR | 替换 ~/.claude 根      |
| XDG_CONFIG_HOME   | 与 Linux FHS 对齐时的基路径 |
| CLAUDE_NO_PERSIST | 干跑模式，不写盘（测试）        |

## 安全基线

```
export async function ensurePrivateDir(dir: string) {
  await fs.mkdir(dir, { recursive: true, mode: 0o700 });
}
```

| 项        | 建议                  |
|----------|---------------------|
| settings | chmod 600           |
| token 文件 | 避免 world-readable   |
| 日志       | 默认不含 API key；过滤 env |

## 与 History / Memdir 的协同

| 模块       | 持久化角色                       |
|----------|-----------------------------|
| History  | 大块: transcript + checkpoint |
| Memdir   | 中块: 偏好与笔记                   |
| settings | 小块: 结构化配置                   |
| 冲突       | Memdir 与 settings 勿重复存同一真相源 |

## 表：同步到多台机器

| 目录            | 适合 dotfiles | 风险           |
|---------------|-------------|--------------|
| settings.json | 高           | 模型默认值可能不同机不同 |
| mem/          | 中           | 冲突需人工合并      |
| cache/        | 否           | 体积与机器相关      |
| sessions/     | 低-中         | 可同步但体积大      |

## 小结

持久化策略回答三个问题：要不要存、存哪、何时写。config 与 ui 偏好进 settings/Memdir；会话真相进 projects sessions；工具运行时队列留在内存；缓存随时可牺牲。统一根目录 ~/.claude/ 降低支持成本。

## 自测

- 为何 transcript 适合 append，而 settings 适合全量原子写？
- debounce 与「进程被 kill -9」如何折中（退出钩子）？
- CLAUDE\_CONFIG\_DIR 改变后 History 旧路径如何迁移或提示？

## 退出钩子与 fsync 策略

| 信号                                         | 行为                                  |
|--------------------------------------------|-------------------------------------|
| <code>SIGINT</code> / <code>SIGTERM</code> | 清空 debounce 队列并同步 flush             |
| <code>uncaughtException</code>             | 尽力写 <code>crash.json</code> ；避免复杂逻辑 |
| <code>kill -9</code>                       | 无法捕获；依赖原子写与下次启动修复                   |

```
export function registerExitFlush(flush: () => Promise<void>) {
  const run = () => {
    void flush().finally(() => process.exit(0));
  };
  process.on("SIGINT", run);
  process.on("SIGTERM", run);
}
```

生产环境可酌情对关键文件 `fsync` ，在延迟与崩溃丢失窗口之间取舍。

## 配额与轮转（日志 / cache）

| 类型             | 建议上限     | 动作                 |
|----------------|----------|--------------------|
| 单会话 transcript | 例如 512MB | 强制 checkpoint + 截断 |
| 全局 cache       | 例如 2GB   | LRU                |
| 诊断日志           | 按日 + 总大小 | 压缩归档               |

上一节：[06-migrations.md](#) · 下一节：[08-overview.md](#)

# 第13篇：状态管理 · 第8节 架构全景 — 状态流转总图

本节串联第1—7节：`createStore` → `AppState` → 副作用 → `Memdir / History` → `Migrations` → 持久化，形成一张可贴在墙上的状态架构全景。

## 学习目标

| 能力项 | 说明                                 |
|-----|------------------------------------|
| 口述  | 三分钟讲清「一次用户输入到磁盘落点」全链路              |
| 绘图  | 独立画出主数据流与次要 I/O 流                  |
| 权衡  | 解释为何 reducer 保持纯、副作用外置             |
| 运维  | 指出备份 <code>~/.claude</code> 时最小必要集 |
| 扩展  | 新增 slice 时检查清单                     |

## 生活类比：城市供水排水一张图

水厂（配置与长期记忆）经主干管（`dispatch`）送到小区水箱（`AppState`）。住户开水龙头（UI）只是末端读取。用水后污水经另一套管网排出（副作用：日志、遥测、写盘）——**绝不能**让污水倒灌进水厂处理前的清水池（`reducer`）。全景图就是清水系统与污水系统分色标注，应急时知道关哪道阀。

## 一张总表：模块 × 职责

| 模块                       | 职责       | 典型 I/O           |
|--------------------------|----------|------------------|
| <code>createStore</code> | 状态容器 API | 无                |
| <code>AppState</code>    | 四域结构     | 无                |
| 副作用同步                    | 投影到世界    | FS、TUI、网络        |
| <code>Memdir</code>      | 结构化用户记忆  | FS 目录树           |
| <code>History</code>     | 会话续接     | jsonl、checkpoint |
| <code>Migrations</code>  | 配置版本     | 读改写 settings     |
| 持久化策略                    | 决策何时写    | 调度 debounce      |

## Mermaid：状态流转全景（主图）

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid：续接与日常双模式（辅图）

[Mermaid 图表 — 请在在线版本中查看]

## 端到端序列：用户改主题

```
// 仅叙事用伪代码串联
dispatch({ type: "ui/SET_THEME", payload: "dark" });
// 1) uiReducer 返回新 state
// 2) subscriber: repaintTUI
// 3) subscriber: persistConfig 若 theme 镜像在 config (依产品设计)
// 4) debounce 写 ~/.claude/settings.json
```

| 步骤         | 纯 / 副作用 |
|------------|---------|
| reducer 计算 | 纯       |
| TUI 绘制     | 副作用     |
| 写 settings | 副作用     |



## 扩展检查清单（新增 slice）

| # | 检查项                                             |
|---|-------------------------------------------------|
| 1 | 是否属于 session/tools/ui/config 之一？ 否则新建顶层 key 需迁移 |
| 2 | reducer 是否纯？                                    |
| 3 | 是否需要 Memdir 投影？ 避免与 settings 双写                 |
| 4 | 是否进入 transcript？ 工具类事件注意脱敏                      |
| 5 | 默认持久化矩阵中打勾                                      |
| 6 | 增加 migration 若磁盘形状变化                            |

## 备份最小集（用户视角）

| 包含                                 | 说明           |
|------------------------------------|--------------|
| <code>settings.json</code>         | 恢复偏好         |
| <code>mem/</code>                  | 恢复长期笔记       |
| <code>projects/**/sessions/</code> | 续接对话（可选，体积大） |
| 不含 <code>cache/</code>             | 可重建          |

## 与第14篇的接口

| 本篇输出           | 第14篇消费                                    |
|----------------|-------------------------------------------|
| 稳定 AppState 形状 | API 客户端错误态写入 <code>tools.lastError</code> |
| effect 管线      | 遥测 batch 发送                               |
| config         | OAuth token 路径、feature flags 默认值          |

## 表：读本篇各节顺序建议

| 读者背景        | 顺序                            |
|-------------|-------------------------------|
| 前端 Redux 经验 | 1 → 2 → 3 → 8 → 4-7           |
| 运维 / 用户     | 7 → 5 → 4 → 8                 |
| 贡献者         | 1 → 2 → 3 → 6 → 5 → 4 → 7 → 8 |

## 小结

Claude Code 风格应用的状态架构可概括为：**纯内核（store + reducer） + 脏边缘（effects） + 分层磁盘（settings / mem / sessions） + 启动护栏（migrations）**。记住一句：**状态先真，世界后追**；全景图即是这句的工程展开。

## 自测

- 1. 画出「API 错误」从第14篇进入本篇 state 再反映到 UI 的路径。
- 2. 若移除所有副作用，应用还能否「技术上运行」？缺什么？
- 3. `claude -c` 在全景图中跨越了哪些子系统？

上一节: [07\\_persistence.md](#) · 返回索引: [index.md](#)

## 第13篇附录：术语索引

| 术语                      | 见节 |
|-------------------------|----|
| dispatch / reducer      | 1  |
| session/tools/ui/config | 2  |
| subscribe / effect      | 3  |
| Memdir 原子写              | 4  |
| transcript / checkpoint | 5  |
| schemaVersion / up      | 6  |
| ~/.claude 布局            | 7  |
| 全景图                     | 8  |

## 反模式速查（全景视角）

| 反模式                      | 症状            | 修复方向                            |
|--------------------------|---------------|---------------------------------|
| reducer 写盘               | 测试 flaky、顺序依赖 | 迁至 effect                       |
| 双源真相（settings + mem 同字段） | 合并冲突、随机覆盖     | 单一主源 + 投影                       |
| 无 schemaVersion          | 升级后静默坏数据      | 引入 migrations                   |
| 全量 state 持久化             | 慢、泄露瞬态        | 按第7节矩阵裁剪                        |
| 续接不校验工具版本                | 工具调用失败        | History meta 记录 registryVersion |

## 延伸阅读（概念对齐）

| 概念    | 经典参考                                                          |
|-------|---------------------------------------------------------------|
| 单向数据流 | Flux / Redux 文档                                               |
| 事件溯源  | transcript $\approx$ event log; checkpoint $\approx$ snapshot |
| CQRS  | 读模型（selector）与写模型（command/action）分离                           |

本节不提供外部链接绑定到具体版本，避免链接失效；读者可按关键词检索官方文档。

## 第14篇：服务与集成

# 第14篇：服务与集成 · 第1节 API 客户端 — Anthropic Messages 与流式 SSE

Claude Code 完全指南 V2 · 本篇共 8 节。本节聚焦 Messages API 的封装：流式 SSE、超时、重试与降级，以及与上层 Agent 循环的边界。

### 学习目标

| 能力项 | 说明                                                                                                                  |
|-----|---------------------------------------------------------------------------------------------------------------------|
| 协议  | 描述 Messages API 请求/响应中 <code>messages</code> 、 <code>system</code> 、 <code>tools</code> 、 <code>stream</code> 等核心字段 |
| SSE | 解析 <code>event:</code> / <code>data:</code> 帧，增量拼出 assistant 消息与 <code>tool_use</code>                              |
| 韧性  | 设计指数退避、可重试状态码、熔断与降级（模型/上下文截断）                                                                                       |
| 边界  | 区分「传输层客户端」与「业务编排」（谁负责多轮 tool loop）                                                                                  |
| 观测  | 在日志中安全打点 <code>requestId</code> 、 <code>latency</code> ，避免泄露正文                                                      |

### 生活类比：挂号窗口与叫号屏

医院挂号（发请求）后，你不会堵在窗口等医生把病历全写完——而是去大厅看叫号屏逐条刷新（SSE 流）。若网络抖动叫号屏卡了，你会过会儿抬头再看（重试）；若全院系统挂了，护士可能给你纸质临时号（降级：换模型或缩短上下文）。API 客户端就是窗口柜员 + 叫号屏协议解析器：保证你拿到有序、完整的就诊序列，而不是一堆乱序纸条。

## 请求骨架（教学示意）

```
// api/messagesTypes.ts - 教学示意, 非官方 SDK 逐字段保证
export interface MessagesRequestBody {
  model: string;
  max_tokens: number;
  system?: unknown;
  messages: Array<{
    role: "user" | "assistant";
    content: unknown;
  }>;
  tools?: unknown[];
  tool_choice?: unknown;
  stream?: boolean;
  metadata?: Record<string, string>;
}
```

## 流式读取（fetch + ReadableStream）

```
// api/streamingClient.ts - 教学示意
export async function* streamSSE(
  response: Response
): AsyncGenerator<{ event: string; data: string }> {
  const reader = response.body!.getReader();
  const dec = new TextDecoder();
  let buf = "";
  while (true) {
    const { value, done } = await reader.read();
    if (done) break;
    buf += dec.decode(value, { stream: true });
    let idx: number;
    while ((idx = buf.indexOf("\n\n")) >= 0) {
      const block = buf.slice(0, idx);
      buf = buf.slice(idx + 2);
      let event = "message";
      let data = "";
      for (const line of block.split("\n")) {
        if (line.startsWith("event:")) event = line.slice(6).trim();
        if (line.startsWith("data:")) data += line.slice(5).trim();
      }
      if (data === "[DONE]") return;
      yield { event, data };
    }
  }
}
```

## 聚合流为最终消息（简化）

```
export async function collectStreamedMessage(
  resp: Response
): Promise<{ text: string; toolUses: unknown[] }> {
  let text = "";
  const toolUses: unknown[] = [];
  for await (const frame of streamSSE(resp)) {
    const chunk = JSON.parse(frame.data);
    // 伪代码：按 Anthropic SSE 事件类型分支 delta
    if (chunk.type === "content_block_delta") {
      if (chunk.delta?.type === "text_delta") text += chunk.delta.text;
    }
    if (chunk.type === "content_block_start" && chunk.content_block?.type === "tool_use") {
      toolUses.push(chunk.content_block);
    }
  }
  return { text, toolUses };
}
```

## 重试与降级策略表

| 场景               | 策略                | 注意                        |
|------------------|-------------------|---------------------------|
| 429 / Rate limit | Retry-After 或指数退避 | 尊重 header                 |
| 5xx              | 有限次重试 + jitter    | 避免惊群                      |
| 网络 reset         | 重试整请求             | 流式需从头或支持 resume（若 API 提供） |
| 上下文过长            | 降级：摘要、删旧轮、换小模型    | 产品提示用户                    |
| 401              | 不重试 token         | 刷新 OAuth（见第6节）            |

## Mermaid：单次流式请求

[Mermaid 图表 — 请在在线版本中查看]

图2：重试状态机

[Mermaid 图表 — 请在在线版本中查看]

## 客户端配置表

| 参数                                    | 典型值    | 用途    |
|---------------------------------------|--------|-------|
| <code>timeoutMs</code>                | 120000 | 首字节超时 |
| <code>maxRetries</code>               | 3      | 可恢复错误 |
| <code>baseUrl</code>                  | 官方或代理  | 企业网关  |
| <code>anthropic-version</code> header | API 版本 | 服务端路由 |

## 与错误分类（第2节）的衔接

传输层应抛出结构化错误（`kind`，`status`，`requestId`），由上层统一映射为用户可见文案与 `AppState.tools.lastError`。不要在底层 `console.error` 打印完整 prompt。

## 安全与合规

| 项       | 做法                      |
|---------|-------------------------|
| API Key | 环境变量或 keychain；禁止写入项目仓库 |
| 日志      | 哈希 message id；截断正文      |
| 代理 TLS  | 企业根证书信任链校验              |

## 小结

API 客户端负责 HTTP/SSE 细节 + 韧性策略，把「一坨字节流」变成「结构化的 assistant 输出 + tool 调用」。重试要分级，401 与 429 不同对待；流式解析要保持缓冲半包与 DONE 语义。

## 自测

- SSE 半包时为何不能按行 `split` 后立即丢弃 `buf`？
- 流式中途连接断开，若无 resume，业务层有哪些选择？
- `metadata` 适合传哪些可观测字段？

## 与 Agent 循环的边界

| 层次         | 职责                    | 非职责              |
|------------|-----------------------|------------------|
| API Client | 单次 HTTP/SSE、重试、解析     | 不决定下一轮流是否调用 tool |
| Agent Loop | 多轮 messages、tool 结果回填 | 不直接操作 TLS socket |
| State      | 记录 lastError、会话 id    | 不实现指数退避细节        |

```
// 边界示意：循环调用客户端，而非相反
export async function agentTurn(client: MessagesClient, state: ChatState) {
  const resp = await client.streamOnce(buildRequest(state));
  const parsed = await collectStreamedMessage(resp);
  return applyToolResultsToState(state, parsed);
}
```

## 参考阅读（关键词）

可在官方文档中检索：`Messages API`、`server-sent events`、`tool use`、`streaming`。具体 URL 以当前版本为准，避免教材链接失效。

下一节：[02-error-handling.md](#) — 错误分类与处理策略。

# 第14篇：服务与集成 · 第2节 错误分类 — API / 网络 / 认证 / 限流

统一 `try/catch` 往往掩盖可恢复性差异。本节建立四维错误taxonomy，并给出策略表 + 状态机，与第1节客户端、第13篇 `AppState` 对齐。



## 学习目标

| 能力项 | 说明                                                        |
|-----|-----------------------------------------------------------|
| 分类  | 将异常映射到 API / 网络 / 认证 / 速率限制                               |
| 策略  | 每类选择重试、刷新凭证、退避或用户介入                                       |
| 类型  | 用 TypeScript discriminated union 表达 <code>AppError</code> |
| UX  | 文案分层：开发者详情 vs 用户一句话                                       |
| 观测  | 错误码、requestId、重试次数进入遥测（第8节）                               |

## 生活类比：快递异常编码

同一句「没收到货」背后可能是：**地址写错**（你的问题 = 4xx）、**分拣中心火灾**（对方问题 = 5xx）、**小区门禁不让进**（认证）、**双十一爆仓**（限流）。客服话术与是否**自动再投**完全不同。软件错误分类就是**先贴对标签再办事**，避免对「门禁问题」疯狂**重试撞门**。

## 统一错误类型（教学示意）

---

```
// errors/AppError.ts - 教学示意
export type AppError =
  | {
    kind: "api";
    status: number;
    code?: string;
    message: string;
    requestId?: string;
    retryable: boolean;
  }
  | {
    kind: "network";
    code: "ECONNRESET" | "ETIMEDOUT" | "ENOTFOUND" | string;
    message: string;
    retryable: true;
  }
  | {
    kind: "auth";
    reason: "expired" | "invalid" | "missing";
    message: string;
    retryable: false;
  }
  | {
    kind: "rate_limit";
    retryAfterMs?: number;
    message: string;
    retryable: true;
  };

export function isRetryable(e: AppError): boolean {
  if (!e.retryable) return false;
  if (e.kind === "api" && e.status >= 400 && e.status < 500) return false;
  return true;
}
```

## 从 fetch Response 映射

```
export async function toAppError(resp: Response, bodyText: string): Promise<AppError> {
  const requestId = resp.headers.get("request-id") ?? undefined;
  if (resp.status === 401)
    return { kind: "auth", reason: "invalid", message: bodyText, retryable: false };
  if (resp.status === 429) {
    const ra = resp.headers.get("retry-after");
    const retryAfterMs = ra ? Number(ra) * 1000 : undefined;
    return { kind: "rate_limit", retryAfterMs, message: bodyText, retryable: true };
  }
  if (resp.status >= 500)
    return {
      kind: "api",
      status: resp.status,
      message: bodyText,
      requestId,
      retryable: true,
    };
  return {
    kind: "api",
    status: resp.status,
    message: bodyText,
    requestId,
    retryable: false,
  };
}
```

## 处理策略表

| kind         | 首选动作            | 次选          | 禁止           |
|--------------|-----------------|-------------|--------------|
| api (5xx)    | 退避重试            | 换模型/缩短上下文   | 无限循环         |
| api (4xx 业务) | 展示具体原因          | 修正输入        | 盲重试          |
| network      | 重试 + jitter     | 检查代理        | 立即判定密钥错误     |
| auth         | OAuth 刷新 / 重新登录 | 提示 settings | 重试同一 token   |
| rate_limit   | 等待 Retry-After  | 降并发         | 忽略 header 狂刷 |

## Mermaid：错误处理决策树

[Mermaid 图表 — 请在在线版本中查看]

## 图2：与状态管理联动

[Mermaid 图表 — 请在在线版本中查看]

## 用户文案 vs 日志

| 层级  | 内容                                                   |
|-----|------------------------------------------------------|
| 用户  | 「请求过于频繁，将在几秒后重试」                                     |
| 日志  | <code>rate_limit requestId=... retryAfter=...</code> |
| 支持包 | 完整堆栈 + 脱敏配置快照                                        |

## 与 MCP / LSP 错误的关系

| 子系统 | 额外错误源                            |
|-----|----------------------------------|
| MCP | 工具执行异常、协议版本不兼容                   |
| LSP | <code>initialize</code> 失败、服务器崩溃 |

统一建议：外层仍归并为上述四类之一，并附加 `subsystem: "mcp" | "lsp"` 字段便于遥测切片。

## 源码片段：重试包装器

```
export async function withRetries<T>(  
  fn: () => Promise<T>,  
  opts: { max: number; baseMs: number; classify: (e: unknown) => AppError }  
) : Promise<T> {  
  let attempt = 0;  
  // eslint-disable-next-line no-constant-condition  
  while (true) {  
    try {  
      return await fn();  
    } catch (e) {  
      const err = opts.classify(e);  
      attempt++;  
      if (!isRetryable(err) || attempt > opts.max) throw err;  
      const jitter = Math.random() * opts.baseMs;  
      const wait = opts.baseMs * 2 ** (attempt - 1) + jitter;  
      await new Promise((r) => setTimeout(r, wait));  
    }  
  }  
}
```

## 表：HTTP 状态速查

| 状态      | 归类              | retryable |
|---------|-----------------|-----------|
| 400     | api 业务          | false     |
| 401     | auth            | false     |
| 403     | api 业务/权限       | 视产品       |
| 404     | api 业务          | false     |
| 408     | network/timeout | true      |
| 429     | rate_limit      | true      |
| 500—504 | api 服务端         | true      |

## 小结

**先分类、再策略：**认证与业务 4xx 不应重试；网络与 5xx、429 可退避。用 **discriminated union** 表达错误可在编译期穷尽处理分支，并与 **UI 状态、遥测**统一。

自测

- 1. 403 与 401 在产品上应如何区分提示？
- 2. 若 `Retry-After` 为 HTTP-date 而非秒数，解析应注意什么？
- 3. `unknown` 错误如何防止泄露敏感信息到用户界面？

上一节: [index.md](#) · 下一节: [03-mcp-protocol.md](#)

第14篇：服务与集成 · 第3节 MCP — Model Context Protocol 概览

MCP 让模型以标准方式发现外部工具与资源：服务器注册能力，宿主按名调用。本节讲消息形态、生命周期、动态发现，与传输层（第4节）解耦。

学习目标

| 能力项 | 说明                                                                                             |
|-----|------------------------------------------------------------------------------------------------|
| 角色  | 区分 host / client / server 三角关系                                                                 |
| 方法  | 列举 <code>initialize</code> 、 <code>tools/list</code> 、 <code>tools/call</code> 等核心 JSON-RPC 意图 |
| 能力  | 解释 <code>capabilities</code> 协商与版本字段                                                           |
| 安全  | 工具调用的权限、用户确认与审计                                                                                |
| 扩展  | resources / prompts 与 tools 的并列关系                                                              |

生活类比：万能转接头上的设备枚举

你的笔记本是 `host`，USB 集线器是 `client`，插上移动硬盘、键盘、打印机是多个 `server`。插上后系统先问：你是什么设备、支持哪些指令 (`list`)；你点打印时系统发具体打印任务 (`call`)。MCP 就是这套问能力再派发任务的协议，只不过对话发生在 LLM 宿主与外部世界之间。

## JSON-RPC 消息壳（概念）

---

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "tools/call",
  "params": {
    "name": "read_file",
    "arguments": { "path": "./README.md" }
  }
}
```

## TypeScript 类型骨架

---

```
// mcp/types.ts - 教学示意
export interface McpTool {
  name: string;
  description?: string;
  inputSchema: Record<string, unknown>;
}

export interface InitializeParams {
  protocolVersion: string;
  capabilities: Record<string, unknown>;
  clientInfo: { name: string; version: string };
}

export interface InitializeResult {
  protocolVersion: string;
  capabilities: Record<string, unknown>;
  serverInfo: { name: string; version: string };
}
```

## 宿主侧：动态工具发现

---

```
export async function refreshTools(conn: McpConnection): Promise<McpTool[]> {
  const res = await conn.request("tools/list", {});
  return (res as { tools: McpTool[] }).tools;
}
```

发现后的工具需合并进 **Agent 可用工具表**，并 bump `AppState.tools.registryVersion`（与第13篇 History 续接校验呼应）。

---

## Mermaid：初始化与 listing

[Mermaid 图表 — 请在在线版本中查看]

### 图2：tools/call 与模型回合

[Mermaid 图表 — 请在在线版本中查看]

## 能力协商表

| 能力键                    | 含义           |
|------------------------|--------------|
| <code>tools</code>     | 支持 list/call |
| <code>resources</code> | 可读资源 URI     |
| <code>prompts</code>   | 预置提示模板       |
| <code>logging</code>   | 服务端日志通道      |

## 安全与权限

| 机制   | 说明                                     |
|------|----------------------------------------|
| 用户确认 | 高危工具需 TUI 二次确认                         |
| 沙箱   | 限制可访问路径 / 网络                           |
| 审计   | transcript 记录 tool name + digest       |
| 版本锁  | 不兼容的 <code>protocolVersion</code> 拒绝连接 |



## 与 Anthropic tool\_use 的映射

| 模型侧                         | MCP 侧                               |
|-----------------------------|-------------------------------------|
| <code>tool_use.name</code>  | <code>tools/call.params.name</code> |
| <code>tool_use.input</code> | <code>arguments</code>              |
| <code>tool_result</code>    | JSON-RPC 响应 → 消息块                   |

## 源码片段：合并多服务器工具

```
export function mergeToolNamespaces(
  servers: Array<{ prefix: string; tools: McpTool[] }>
): McpTool[] {
  const out: McpTool[] = [];
  for (const s of servers) {
    for (const t of s.tools) {
      out.push({
        ...t,
        name: `${s.prefix}__${t.name}`,
      });
    }
  }
  return out;
}
```

| 注意     | 说明                       |
|--------|--------------------------|
| 前缀     | 避免多 server 同名冲突          |
| schema | 合并后仍需满足 API tools 数组大小限制 |

## 故障模式

| 现象         | 可能原因                         |
|------------|------------------------------|
| list 为空    | server 未实现或 capabilities 未声明 |
| call 超时    | 下游 I/O；应可取消                  |
| schema 不匹配 | 模型幻觉参数；需校验后反馈 4xx 等价错误       |

## 小结

MCP 用 JSON-RPC 统一「发现 + 调用」：`initialize` 定调，`tools/list` 更新注册表，`tools/call` 执行。宿主负责 合并命名空间、权限与审计，传输细节交给第4节。

## 自测

- 1. `initialized` 通知与 `initialize` 请求响应的先后顺序为何重要？
- 2. `resources` 与 `tools` 在模型上下文注入上有何不同？
- 3. 多 `server` 同名工具不加前缀会发生什么？

## 调试清单（host 开发者）

| 步骤 | 检查                                                                          |
|----|-----------------------------------------------------------------------------|
| 握手 | <code>protocolVersion</code> 双方兼容                                           |
| 列举 | <code>tools/list</code> 返回非空且 <code>inputSchema</code> 为合法 JSON Schema      |
| 调用 | <code>tools/call</code> 的 <code>name</code> 与 <code>list</code> 完全一致（含前缀策略） |
| 并发 | 同一 <code>id</code> 不重复使用直至响应返回                                              |
| 日志 | JSON-RPC 与 <code>stderr</code> 分离                                           |

## JSON-RPC 通知与请求对照

| 类型           | 需 id       | 需 response |
|--------------|------------|------------|
| request      | 是          | 是          |
| response     | 是（与请求同 id） | —          |
| notification | 否          | 否          |

宿主应对未知 `method` 返回规范错误，避免 `server` 悬挂。

上一节：[02-error-handling.md](#) · 下一节：[04-mcp-transport.md](#)

# 第14篇：服务与集成 · 第4节 MCP 传输 — stdio / SSE / WebSocket

同一套 MCP 语义可跑在多种**传输**上。本节对比 **stdio**、**SSE**、**WebSocket** 的适用场景、进程模型与安全边界。

## 学习目标

| 能力项       | 说明                                    |
|-----------|---------------------------------------|
| stdio     | 解释父子进程、行/帧分帧、生命周期与僵尸进程防护              |
| SSE       | 理解 Server-Sent Events 单向流与 HTTP/2 注意点 |
| WebSocket | 全双工、心跳、重连与鉴权头                         |
| 选型        | 本地插件 vs 远程服务 vs 浏览器环境的决策表             |
| 排错        | 日志分离 stderr、避免污染 JSON-RPC 流           |

## 生活类比：对讲机、广播与电话

- stdio** 像**对讲机专线**：两人独占频道，简单可靠，但必须**站得近**（本机进程）。
- SSE** 像**电台广播**：主机只管播，听众单向接收；适合**服务器推事件**。
- WebSocket** 像**电话**：双方随时插话（全双工），适合**高频来回与长会话**。

选错介质就会出现：**电话里听广播的杂音**（把日志打进协议流）或**对讲机跨城**（误把 stdio 当远程）。

## stdio 传输要点

```
// transport/stdio.ts - 教学示意
import { spawn } from "node:child_process";

export function spawnMcpStdio(command: string, args: string[]) {
  const child = spawn(command, args, {
    stdio: ["pipe", "pipe", "pipe"],
  });
  child.stderr.on("data", (d) => {
    // 仅记录诊断，绝不混入 stdout 解析器
    console.error("[mcp stderr]", d.toString());
  });
  return {
    stdin: child.stdin,
    stdout: child.stdout,
    kill: () => child.kill("SIGTERM"),
  };
}
```

| 注意 | 说明                     |
|----|------------------------|
| 缓冲 | 使用流式解析器处理半包 JSON       |
| 结束 | 子进程退出需清理挂起 promise     |
| 并发 | 单连接多 in-flight 需 id 关联 |

## SSE 传输概念

| 项  | 说明                                                  |
|----|-----------------------------------------------------|
| 方向 | 服务器 → 客户端为主；请求仍用 HTTP POST                          |
| 帧  | <code>data:</code> 行承载事件负载                          |
| 代理 | 某些反向代理缓冲 SSE，需 <code>X-Accel-Buffering: no</code> 等 |
| 鉴权 | Cookie / Bearer 与长连接并存                              |

```
// 伪代码：打开 SSE 端点并喂入解析器
export function openSseTransport(url: string, headers: Record<string, string>) {
  const es = new EventSource(url, { fetch: /* 自定义带 header 的 fetch */ as any });
  es.onmessage = (ev) => handleJsonRpcLine(ev.data);
  return es;
}
```

浏览器原生 `EventSource` 不便带自定义 header；Node 环境常用 `fetch` 流式解析（见第1节 SSE）。

## WebSocket 传输概念

```
export function openWsTransport(
  url: string,
  onFrame: (s: string) => void
): { send: (s: string) => void; close: () => void } {
  const ws = new WebSocket(url);
  ws.on("message", (data) => onFrame(String(data)));
  const send = (s: string) => ws.send(s);
  const close = () => ws.close();
  const ping = setInterval(() => {
    if (ws.readyState === WebSocket.OPEN) ws.ping();
  }, 20000);
  ws.on("close", () => clearInterval(ping));
  return { send, close };
}
```

| 项   | 说明                        |
|-----|---------------------------|
| 心跳  | 防止 NAT 超时                 |
| 重连  | 指数退避 + 会话恢复（若协议支持）        |
| TLS | <code>wss://</code> 与证书校验 |

## Mermaid：三种拓扑

[Mermaid 图表 — 请在在线版本中查看]

### 图2：JSON-RPC 在双向通道上的复用

[Mermaid 图表 — 请在在线版本中查看]

## 选型表

| 场景        | 推荐            | 原因              |
|-----------|---------------|-----------------|
| 本地 CLI 插件 | stdio         | 零端口暴露、简单        |
| 企业内远程工具   | WS 或 SSE+POST | 防火墙友好程度因环境而异    |
| 浏览器宿主     | WS / HTTP 封装  | SSE 受 header 限制 |
| 极高交互调试    | WS            | 低延迟双向           |

## 安全对照

| 传输    | 风险        | 缓解                       |
|-------|-----------|--------------------------|
| stdio | 子进程命令注入   | 固定 allowlist、不用 shell 解析 |
| SSE   | CSRF / 跨域 | Cookie SameSite、Token    |
| WS    | 中间人       | TLS、证书锁定（谨慎）             |

## 与协议层（第3节）边界

传输只负责 可靠送达字节；方法名、params、id 由 MCP JSON-RPC 定义。换传输时不应改业务 `tools/call` 语义。

## 小结

**stdio** 最适合本地 `fork/exec` 模型；**SSE** 适合 HTTP 生态下的单向推送；**WebSocket** 适合 长连接双向 与远程 MCP。共性：**stderr** 与协议流分离、半包解析、心跳/退避（远程）。

## 自测

- 为何 MCP stdio 服务器不应向 stdout 打印调试日志？
- SSE 在反向代理后「延迟很久才收到」可能是什么原因？
- WebSocket 重连后 JSON-RPC 的 `id` 应如何与宿主状态关联？

## 背压与流控（进阶）

| 传输        | 背压手段                                  |
|-----------|---------------------------------------|
| stdio     | <code>pause()</code> / 高水位标记，避免内存暴涨   |
| SSE       | 客户端停止读触发 TCP 窗口                       |
| WebSocket | <code>bufferedAmount</code> 监控 + 分片发送 |

当 MCP 响应体极大（例如嵌入资源）时，应在协议层支持分块或引用，而非一次塞满管道。

## 兼容性说明（教学）

不同宿主实现对「单连接多多路复用」支持程度不一：集成测试应覆盖 串行请求 与 有限并行 两种模式，防止对端 server 仅实现半双工假设。

本节强调：传输只搬运帧，不解释业务；协议语义始终以 MCP JSON-RPC 为准。

上一节：[03-mcp-protocol.md](#) · 下一节：[05-lsp.md](#)

# 第14篇：服务与集成 · 第5节 LSP — Language Server Protocol 集成

**LSP** 把「语法树、补全、诊断」从编辑器搬到独立进程。Claude Code 类工具可用 LSP **增强上下文精度**：跳转定义、引用、工作区符号，而不仅靠全文搜索。

## 学习目标

| 能力项  | 说明                                                                                                |
|------|---------------------------------------------------------------------------------------------------|
| 生命周期 | <code>initialize</code> → <code>initialized</code> → <code>textDocument/*</code>                  |
| 能力   | <code>textDocumentSync</code> 、 <code>completionProvider</code> 、 <code>publishDiagnostics</code> |
| 上下文  | 如何把 <code>hover</code> / <code>definition</code> 结果注入 prompt                                      |
| 性能   | 按需启动语言服务器、项目根探测                                                                                   |
| 容错   | 服务器崩溃重启、降级为纯文本工具                                                                                  |

## 生活类比：专业翻译坐席

你是作家（模型），写外文小说时旁边有**专业翻译坐席（LSP）**：你问「这个词在合同里出现过吗」，翻译查术语库（符号表）再答。你不会自己背整本词典（不解析全 AST 在宿主）。LSP 就是随叫随到的语言专家进程。

## 初始化请求（概念 JSON-RPC）

```
{
  "jsonrpc": "2.0",
  "id": 1,
  "method": "initialize",
  "params": {
    "processId": 12345,
    "rootUri": "file:///path/to/project",
    "capabilities": {
      "textDocument": {
        "completion": { "dynamicRegistration": false },
        "publishDiagnostics": {}
      }
    }
  }
}
```



## TypeScript 宿主封装骨架

```
// lsp/client.ts - 教学示意
export class LspClient {
  private nextId = 1;
  private pending = new Map<number, (v: unknown) => void>();

  constructor(private send: (msg: unknown) => void) {}

  request(method: string, params: unknown): Promise<unknown> {
    const id = this.nextId++;
    const msg = { jsonrpc: "2.0", id, method, params };
    return new Promise((resolve) => {
      this.pending.set(id, resolve);
      this.send(msg);
    });
  }

  onMessage(raw: unknown) {
    const m = raw as { id?: number; method?: string; params?: unknown };
    if (m.id != null && this.pending.has(m.id)) {
      const cb = this.pending.get(m.id)!;
      this.pending.delete(m.id);
      cb(m);
      return;
    }
    if (m.method === "textDocument/publishDiagnostics") {
      this.onDiagnostics(m.params);
    }
  }

  onDiagnostics(_params: unknown) {
    /* 合并到 AppState 或本地缓存 */
  }
}
```

## 用于 Agent 的典型请求

| 方法                      | 用途                |
|-------------------------|-------------------|
| textDocument/hover      | 类型/文档字符串          |
| textDocument/definition | 跳转定义 URI+range    |
| textDocument/references | 引用列表              |
| workspace/symbol        | 粗粒度符号搜索           |
| textDocument/completion | 与 IDE 类似，Agent 少用 |

## Mermaid：启动与诊断流

[Mermaid 图表 — 请在在线版本中查看]

### 图2：与模型上下文拼接

[Mermaid 图表 — 请在在线版本中查看]

## 文档同步策略

| 策略          | 说明                                        |
|-------------|-------------------------------------------|
| Full        | 每次全量发 <code>textDocument/didChange</code> |
| Incremental | 发变更 <code>range</code> ；需与磁盘一致            |
| None        | 仅打开/关闭事件；服务器自读盘                           |

Agent 工具若已修改文件，宿主需 `didChange` 或 `didSave` 避免 LSP 与磁盘漂移。

## 表：LSP vs 朴素 grep

| 维度   | LSP        | grep/ripgrep |
|------|------------|--------------|
| 语义   | 强          | 弱            |
| 启动成本 | 高          | 低            |
| 语言覆盖 | 需对应 server | 通用           |
| 实时诊断 | 有          | 无            |

## 进程管理

```
export async function startServerForWorkspace(root: string): Promise<LspClient> {
  const proc = spawn("typescript-language-server", ["--stdio"], { cwd: root });
  const client = new LspClient((msg) => proc.stdin.write(JSON.stringify(msg) + "\n"));
  lineParser(proc.stdout, (obj) => client.onMessage(obj));
  proc.on("exit", (code) => {
    console.error("lsp exited", code);
  });
  await client.request("initialize", {
    processId: process.pid,
    rootUri: pathToUri(root),
    capabilities: minimalCaps,
  });
  await client.notify("initialized", {});
  return client;
}
```

## 容错与降级

| 故障       | 策略                    |
|----------|-----------------------|
| 启动失败     | 关闭 LSP 功能位，回退 ripgrep |
| 无 server | 按语言跳过                 |
| 超时       | 取消请求，提示「语言服务无响应」      |

## 小结

LSP 为 Claude Code 提供结构化代码理解：初始化协商能力，诊断可反馈用户，hover/definition 可压缩进上下文。集成关键是 同步模型 与 进程生命周期，失败时优雅降级。

## 自测

- `rootUri` 与单文件打开的关系？
- 为何增量同步对 Agent 自动改文件更危险？
- `publishDiagnostics` 是请求还是通知？

## 多工作区与多根（概念）

| 场景           | 策略                                                   |
|--------------|------------------------------------------------------|
| monorepo     | <code>workspaceFolders</code> 初始化                    |
| 单根           | 传统 <code>rootUri</code>                              |
| Agent 切换 cwd | 重新 <code>didChangeWorkspaceFolders</code> 或重启 server |

语言服务器若不支持多根，宿主应按主项目根启动，避免符号解析漂移。

## 与 ripgrep 的协同

典型流水线：`rg` 粗定位 → LSP 精修（hover/definition）。在超大仓库中可节省 LSP 负载，并减少无响应超时。

上一节：[04-mcp-transport.md](#) · 下一节：[06-oauth.md](#)

# 第14篇：服务与集成 · 第6节 OAuth 2.0 + PKCE — 认证与 Token 管理

控制台类应用常通过 **OAuth 2.0 授权码 + PKCE** 获取访问令牌，浏览器完成 **consent**（如 `consent.anthropic.com` 教学名），CLI 监听 **localhost 回调**或使用 **设备码流**（视产品）。本节讲流程、**Token 存储与刷新**。

## 学习目标

| 能力项 | 说明                                                      |
|-----|---------------------------------------------------------|
| 流程  | 描述授权码 + PKCE: code_verifier / code_challenge / redirect |
| 安全  | 为何 CLI 需要 PKCE; state 防 CSRF                            |
| 存储  | refresh token 加密-at-rest、文件权限                           |
| 刷新  | 在 401 前 proactive refresh vs reactive                   |
| 撤销  | 登出与 token 作废的产品语义                                       |

## 生活类比：酒店前台临时房卡

办理入住（授权）时，前台给你一张**限时房卡（access token）**，同时系统在后台给你**延期资格（refresh token）**。房卡丢了要**补办（刷新）**，离店**注销（revoke）**。PKCE 像是**当场对暗号**：即使有人偷听对话，也拿不到能开门的完整密钥。

## PKCE 参数生成（示意）

```
// oauth/pkce.ts - 教学示意
import crypto from "node:crypto";

export function generatePkcePair() {
  const verifier = base64Url(crypto.randomBytes(32));
  const challenge = base64Url(
    crypto.createHash("sha256").update(verifier).digest()
  );
  return { code_verifier: verifier, code_challenge: challenge };
}

function base64Url(buf: Buffer): string {
  return buf
    .toString("base64")
    .replace(/\+/g, "-")
    .replace(/\//g, "_")
    .replace(/=+$/, "");
}
```

## 授权 URL 组装

---

```
export function buildAuthorizeUrl(opts: {
  base: string;
  clientId: string;
  redirectUri: string;
  scope: string;
  state: string;
  codeChallenge: string;
}) {
  const u = new URL("/oauth/authorize", opts.base);
  u.searchParams.set("response_type", "code");
  u.searchParams.set("client_id", opts.clientId);
  u.searchParams.set("redirect_uri", opts.redirectUri);
  u.searchParams.set("scope", opts.scope);
  u.searchParams.set("state", opts.state);
  u.searchParams.set("code_challenge", opts.codeChallenge);
  u.searchParams.set("code_challenge_method", "S256");
  return u.toString();
}
```

## 用 code 换 token

---

```
export async function exchangeCode(params: {
  tokenEndpoint: string;
  clientId: string;
  code: string;
  redirectUri: string;
  codeVerifier: string;
}): Promise<{ access_token: string; refresh_token?: string; expires_in?: number }> {
  const body = new URLSearchParams({
    grant_type: "authorization_code",
    code: params.code,
    redirect_uri: params.redirectUri,
    client_id: params.clientId,
    code_verifier: params.codeVerifier,
  });
  const res = await fetch(params.tokenEndpoint, {
    method: "POST",
    headers: { "content-type": "application/x-www-form-urlencoded" },
    body,
  });
  if (!res.ok) throw new Error(`token ${res.status}`);
  return res.json();
}
```

## Mermaid：授权码 + PKCE

[Mermaid 图表 — 请在在线版本中查看]

### 图2：401 时刷新

[Mermaid 图表 — 请在在线版本中查看]

## Token 持久化

| 字段            | 存储建议        |
|---------------|-------------|
| access_token  | 内存优先；落盘则加密  |
| refresh_token | 必须限制权限位；加密  |
| expires_at    | 计算绝对时间，提前刷新 |

```
export async function saveTokenBundle(
  filePath: string,
  bundle: object,
  key: Buffer
) {
  const iv = crypto.randomBytes(12);
  const cipher = crypto.createCipheriv("aes-256-gcm", key, iv);
  const pt = Buffer.from(JSON.stringify(bundle), "utf8");
  const enc = Buffer.concat([cipher.update(pt), cipher.final()]);
  const tag = cipher.getAuthTag();
  const payload = { iv: iv.toString("base64"), tag: tag.toString("base64"), enc: enc.toString("base64") };
  await fs.writeFile(filePath, JSON.stringify(payload), { mode: 0o600 });
}
```

## consent 与 scope

| 概念         | 说明           |
|------------|--------------|
| consent UI | 用户显式同意 scope |
| scope 最小化  | 仅申请必要权限      |
| state      | 随机串，回调时校验    |

## 与错误处理（第2节）衔接

| 事件                         | 映射                        |
|----------------------------|---------------------------|
| <code>invalid_grant</code> | <code>auth</code> + 重新登录  |
| token 刷新网络错误               | <code>network</code> + 退避 |

## 表：localhost 回调 vs 设备码

| 模式        | 优点        | 缺点          |
|-----------|-----------|-------------|
| localhost | UX 顺滑     | 端口冲突/无浏览器环境 |
| 设备码       | 适合 SSH 远程 | 步骤多         |

## 小结

OAuth 2.0 + PKCE 让 CLI 安全代用户调用 API：`code_verifier` 保密、`challenge` 公开；token 加密存储、刷新与 401 路径打通 consent 与 API 客户端。

## 自测

- 为何隐式流（implicit）不适合现代 CLI？
- `state` 若省略会怎样？
- refresh 成功后是否要重试原请求而非让用户重试？

上一节：[05-lsp.md](#) · 下一节：[07-feature-flags.md](#)



# 第14篇：服务与集成 · 第7节 特性标志 — 90+

## Flags、渐进发布与实验

大型 CLI/Agent 产品常维护数十至上百个 **feature flags**：控制新工具、UI、模型路径的灰度。本节讲分层来源、合并规则、遥测绑定与 A/B 测试注意点。

### 学习目标

| 能力项 | 说明                            |
|-----|-------------------------------|
| 来源  | 区分默认值、远程配置、环境变量、本地 settings   |
| 合并  | 定义优先级：例如 env > 用户设置 > 远程 > 默认 |
| 类型  | bool / 枚举 / 数值阈值（采样率）         |
| 发布  | 渐进发布、kill switch、按用户哈希分桶      |
| 治理  | 命名规范、过期清理、与 migrations 关系     |

### 生活类比：游乐园快速通行证开关

游乐园有总控台（远程配置）：今天雷雨关掉过山车（kill switch），VIP 区试运行新项目（10% 用户）。你手里的年卡权益（本地 settings）可能覆盖普通规则；门口告示牌（环境变量）写着「今日检修」——告示优先。90+ 个开关就像每个游乐设施独立闸机，但优先级必须写清楚，否则员工与告示冲突时游客懵。

### Flag 定义表（示例）

| key                   | 类型     | 默认    | 说明     |
|-----------------------|--------|-------|--------|
| tools.mcp.v2Transport | bool   | false | 启用新传输  |
| ui.compactMode        | bool   | false | TUI 密度 |
| model.routing.variant | enum   | A     | A/B 路径 |
| telemetry.sampleRate  | number | 0.1   | 采样     |

## 合并优先级（教学实现）

```
// flags/merge.ts - 教学示意
export type FlagMap = Record<string, boolean | string | number>;

export function mergeFlags(layers: FlagMap[]): FlagMap {
  const out: FlagMap = {};
  // 后者覆盖前者: 默认 -> 远程 -> 用户 -> 环境
  for (const layer of layers) {
    for (const [k, v] of Object.entries(layer)) {
      if (v === undefined) continue;
      out[k] = v;
    }
  }
  return out;
}

export function envOverridePrefix(prefix = "CLAUDE_FLAG_"): FlagMap {
  const out: FlagMap = {};
  for (const [k, v] of Object.entries(process.env)) {
    if (!k.startsWith(prefix) || v == null) continue;
    const name = k.slice(prefix.length).replace(/__/g, ".");
    out[name] = parseEnvValue(v);
  }
  return out;
}

function parseEnvValue(v: string): boolean | string | number {
  if (v === "true" || v === "false") return v === "true";
  if (/^\d+$/g.test(v)) return Number(v);
  return v;
}
```

## 远程获取与缓存

```
export async function fetchRemoteFlags(url: string): Promise<FlagMap> {
  const res = await fetch(url, { headers: { "cache-control": "no-store" } });
  if (!res.ok) return {};
  return res.json() as Promise<FlagMap>;
}
```

| 策略   | 说明       |
|------|----------|
| ETag | 减少带宽     |
| 失败降级 | 使用上次缓存   |
| TTL  | 防止远程挂死启动 |

## Mermaid：合并管线

[Mermaid 图表 — 请在在线版本中查看]

### 图2：分桶（A/B）

[Mermaid 图表 — 请在在线版本中查看]

## 与 AppState.config.experimental 映射

| 层级           | 映射                               |
|--------------|----------------------------------|
| 用户显式 toggles | <code>config.experimental</code> |
| 远程实验         | hydrate 时写入只读投影，避免用户误改           |
| 冲突           | 以合并优先级为准，UI 显示「由组织策略锁定」          |

## 治理表

| 实践   | 说明                             |
|------|--------------------------------|
| 前缀   | <code>area.subarea.name</code> |
| 生命周期 | issue 关联移除日期                   |
| 文档   | 自动生成 flags 目录页                 |
| 测试   | 矩阵测 true/false 关键路径            |

## Kill switch 场景

| 场景         | flag 动作                                 |
|------------|-----------------------------------------|
| MCP 协议 bug | 关闭 <code>mcp.enabled</code>             |
| 新模型事故      | 路由回旧模型                                  |
| 遥测风暴       | <code>telemetry.enabled=false</code> 远程 |

## 与遥测（第8节）

将 `effectiveFlags` 子集（非敏感）附在事件上下文，便于分析「B 组崩溃率」。注意 **GDPR/匿名化**：不传邮箱，仅哈希 user id。

## 表：90+ flags 管理

| 问题        | 对策                            |
|-----------|-------------------------------|
| 组合爆炸      | 分类 + 集成测分层                    |
| 默认值漂移     | 单源 <code>defaults.json</code> |
| 远程与本地不一致  | UI 展示来源列                      |
| 旧 flag 残留 | CI 检查无引用键                     |

## 小结

特性标志把发布从「发版」解耦为「调参」：**多层合并、远程 kill、分桶实验**是三大支柱。务必维护**优先级文档与清理纪律**，否则 90+ 很快变债务。

## 自测

- 若 env 与 settings 同时设置同一 bool，谁应胜出？理由？
- 分桶用的 user id 为何需要稳定哈希盐？
- 远程 flag 拉取失败时，静默用空对象还是旧缓存更安全？

## 渐进发布数值示例

| 阶段      | 受众       | 典型做法                             |
|---------|----------|----------------------------------|
| dogfood | 内部员工     | <code>employee</code> 名单或 org id |
| canary  | 1%—5% 用户 | 哈希分桶                             |
| GA      | 100%     | 默认值改为 true，保留 kill switch        |

每个阶段应配置自动回滚条件（错误率、延迟），与遥测第8节联动。

## 与 migrations 的边界

| 维度 | migrations | feature flags |
|----|------------|---------------|
| 改变 | 磁盘形状       | 运行时分支         |
| 时机 | 启动一次       | 运行中可刷新        |
| 回滚 | 备份文件       | 远程关开关         |

勿用 flag 修补无效 JSON——应走第13篇迁移管线。

上一节：[06-oauth.md](#) · 下一节：[08-summary.md](#)

# 第14篇：服务与集成 · 第8节 遥测与全篇总结 — 观测、隐私与集成全景

最后一节聚焦匿名遥测、性能指标、错误报告的采集与隐私边界，并用一张集成全景回扣第1—7节。

## 学习目标

| 能力项 | 说明                                           |
|-----|----------------------------------------------|
| 事件  | 设计 schema: name、timestamp、版本、会话哈希、属性         |
| 采样  | 与 feature flags 的 <code>sampleRate</code> 协同 |
| PII | 明确禁止字段与脱敏规则                                  |
| 性能  | latency histogram、冷启动分段                      |
| 伦理  | 可选退出 (opt-out)、本地预览队列                        |

## 生活类比：城市公交刷卡统计

公交公司统计哪条线路拥挤（性能与使用分布），不需要知道你是谁——只需匿名卡号哈希、上下车站点。若有人要求永不统计，应提供关闭刷卡分析开关（opt-out）。遥测就是城市交通大脑：优化排班，但不把乘客日记公开。

## 事件 envelope（教学示意）

```
// telemetry/event.ts - 教学示意
export interface TelemetryEvent {
  name: string;
  ts: string;
  cliVersion: string;
  os: string;
  /** 不可逆会话指纹 */
  sessionFingerprint: string;
  props?: Record<string, string | number | boolean>;
}

export interface ErrorReport extends TelemetryEvent {
  name: "error";
  props: {
    kind: "api" | "network" | "auth" | "rate_limit";
    code?: string;
    retryCount?: number;
  };
}
```

## 批处理与发送

```
export class TelemetryQueue {
  private buf: TelemetryEvent[] = [];
  constructor(private max: number, private flushMs: number) {
    setInterval(() => void this.flush(), flushMs);
  }
  push(e: TelemetryEvent) {
    this.buf.push(e);
    if (this.buf.length >= this.max) void this.flush();
  }
  async flush() {
    if (!this.buf.length) return;
    const batch = this.buf.splice(0, this.buf.length);
    await fetch("https://telemetry.example/v1/batch", {
      method: "POST",
      body: JSON.stringify({ events: batch }),
      headers: { "content-type": "application/json" },
    }).catch(() => {
      // 失败: 可落盘重试 (与第13篇持久化呼应)
    });
  }
}
```

## 性能分段

| span                            | 说明            |
|---------------------------------|---------------|
| <code>boot.migrate</code>       | migrations 耗时 |
| <code>boot.hydrate</code>       | 读配置与 mem      |
| <code>api.first_token_ms</code> | 首 token 延迟    |
| <code>mcp.connect</code>        | 各 server 握手   |
| <code>lsp.init</code>           | 语言服务启动        |

```
export function span<T>(name: string, fn: () => Promise<T>): Promise<T> {
  const t0 = performance.now();
  return fn().finally(() => {
    const dt = performance.now() - t0;
    queue.push({ name: `perf.${name}`, ts: new Date().toISOString(), /* ... */ props: { ms:
  });
}
```

## Mermaid：遥测管线

[Mermaid 图表 — 请在在线版本中查看]

### 图2：第14篇集成全景

[Mermaid 图表 — 请在在线版本中查看]

## 隐私清单

| 禁止        | 替代          |
|-----------|-------------|
| 完整 prompt | 长度 + 哈希     |
| API Key   | 永不          |
| 绝对路径      | 哈希 basename |
| 邮箱        | 稳定匿名 id     |

## Opt-out 实现表

| 层级       | 机制                                    |
|----------|---------------------------------------|
| 环境变量     | <code>TELEMETRY_DISABLED=1</code>     |
| settings | <code>telemetry.enabled: false</code> |
| 企业策略     | 远程 flags 强制关                          |

## 与错误报告联动

| 数据                      | 用途                 |
|-------------------------|--------------------|
| <code>requestId</code>  | 与服务商日志关联（用户授权支持包时） |
| <code>retryCount</code> | 评估重试策略是否过激         |
| <code>subsystem</code>  | MCP/LSP/API 切片     |



## 全篇回顾表

| 节 | 关键词                          |
|---|------------------------------|
| 1 | SSE、重试、降级                    |
| 2 | API/网络/认证/限流                 |
| 3 | MCP JSON-RPC、tools/list/call |
| 4 | stdio、SSE、WebSocket          |
| 5 | LSP initialize、diagnostics   |
| 6 | PKCE、token 存储                |
| 7 | flags 合并、分桶                  |
| 8 | 遥测、隐私、全景                     |

## 小结

服务与集成层把 Claude Code 连到外部世界：**API** 负责模型对话；**错误系统**统一韧性；**MCP** 扩展工具；**LSP** 深化代码理解；**OAuth** 保安全认证；**Feature Flags** 控发布；**遥测**在**隐私约束**下闭环改进。与**状态管理**结合，形成可观测、可恢复、可灰度的整体架构。

## 自测

- 批处理失败落盘重试时，如何防止无限增长？
- perf span 与业务事件在 schema 上如何共用字段？
- 企业客户要求「零出站」，应关闭哪些子系统？

上一节：[07-feature-flags.md](#) · 返回索引：[index.md](#)

# 第14篇术语索引

| 术语                 | 节 |
|--------------------|---|
| SSE / stream       | 1 |
| AppError           | 2 |
| tools/list         | 3 |
| stdio / WS         | 4 |
| publishDiagnostics | 5 |
| PKCE               | 6 |
| mergeFlags         | 7 |
| TelemetryQueue     | 8 |

## 第15篇：隐藏功能

# 第十五部分 · 隐藏功能（15.1） — 源码中的 90+ Feature Flags 全景

**导航：**本部分共 6 节。本节从\*\*特性开关（Feature Flags）\*\*总览入手；后续各节分别讲解 Undercover Mode、Buddy 宠物、反作弊、深度规划与内部彩蛋分支。

### 学习目标

完成本节学习后，你应该能够：

1. **理解** Feature Flag 在 Claude Code 类客户端中的角色：灰度发布、实验能力、合规与差异化构建。
2. **归类** 在源码审计中常见的 Flag 类型（环境变量覆盖、构建期常量、远程配置、用户/会话派生状态）。
3. **估算** 「90+」量级开关在工程上的含义：并非全部对用户可见，多数服务于内部实验、遥测采样与 UI 微功能。
4. **建立** 阅读源码时的检索习惯：`FEATURE_`、`ENABLE_`、`BUDDY`、`UNDERCOVER` 等关键词与集中注册表（registry）模式。

### 生活类比：大楼里的「配电盘」

把 90+ Feature Flags 想象成一栋智能写字楼地下室的**配电盘**：

- 每一格开关对应一条**电路**（某段逻辑、某块 UI、某种模型策略）。
- 物业（发布系统）可以**只给某几层通电**（灰度用户），而不必拆掉整栋楼重装。
- 有些开关连着**消防与合规**（例如公开场景下去标识化），平时你看不到灯亮，但一直在工作。
- 偶尔有一两路是**装饰灯带**（彩蛋、宠物、内部演示），不影响承重结构，但让体验更有趣。

学 Flag 不是背 90 个名字，而是学会**找到总闸、读懂标签、推断电流走向**。

## 为何源码里会有「90+」个开关？

| 动因      | 说明                   | 对读者的启示                                    |
|---------|----------------------|-------------------------------------------|
| 持续交付    | 主干开发 + 按需点亮功能，避免长期分支 | 同一版本二进制可能包含未启用路径                          |
| 风险控制    | 高危能力默认关闭，事故时可快速切断    | 文档未写 ≠ 代码不存在                              |
| A/B 与实验 | 采样、对照组、内部员工差异        | 行为「不一致」有时是刻意的                             |
| 合规与场景   | 公开仓库、企业租户、地域策略       | 环境变量可强制某些模式（见 15.2）                       |
| 性能与缓存   | 延迟加载、工具注册、索引构建       | 与 <code>defer_loading</code> 等机制联动（第十六部分） |

## Mermaid：Feature Flag 决策流（概念模型）

下列流程图展示一次请求或会话初始化时，典型客户端如何叠加多层 Flag 决策。节点 ID 使用英文字母与下划线，标签用中文引号。

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid：Flag 分类思维导图（教学用）

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：集中注册与布尔判断（示意）

下列 TypeScript 片段为**教学示意**，展示工程中常见的「注册表 + 守卫函数」写法；真实仓库中的命名与文件路径以你本地检出为准。

```
// feature-flags.ts (示意)
export const FeatureRegistry = {
  BUDDY: 'buddy_pet_system',
  DEEP_PLANNING: 'deep_planning_mode',
  UNDERCOVER_HINT: 'undercover_mode',
  // ... 数十至上百项, 按域分组
} as const;

export type FeatureKey = (typeof FeatureRegistry)[keyof typeof FeatureRegistry];

export function isFeatureEnabled(
  key: FeatureKey,
  ctx: { env: NodeJS.ProcessEnv; session: SessionContext }
): boolean {
  // 1) 环境变量强制 (高优先级)
  if (ctx.env.CLAUDE_CODE_FORCE_FEATURES?.includes(key)) return true;
  // 2) 远程配置 / 本地覆盖
  if (ctx.session.remoteFlags?.[key] !== undefined) {
    return Boolean(ctx.session.remoteFlags[key]);
  }
  // 3) 默认值 + 构建期注入
  return DEFAULT_FLAGS[key] ?? false;
}
```

```
// usage-example.ts (示意)
if (isFeatureEnabled(FeatureRegistry.BUDDY, ctx)) {
  renderBuddyStatusLine(petRollup);
}
```

## 「90+」 Flag 如何分组阅读？（速查表）

下表为学习方法论，不是完整清单。完整枚举应以你检出的版本 `git grep -i feature` 结果为准。

| 分组    | 检索关键词示例                                                                     | 典型文件/符号模式                                             | 本节关联                            |
|-------|-----------------------------------------------------------------------------|-------------------------------------------------------|---------------------------------|
| 合规/隐私 | <code>undercover</code> , <code>public_repo</code> ,<br><code>redact</code> | <code>*privacy*</code> ,<br><code>*compliance*</code> | <a href="#">15.2 Undercover</a> |
| 彩蛋/UX | <code>buddy</code> , <code>pet</code> , <code>rarity</code>                 | <code>*easter*</code> , <code>*cosmetic*</code>       | <a href="#">15.3 Buddy</a>      |
| 完整性   | <code>session</code> , <code>checksum</code> , <code>recompute</code>       | <code>*anti*cheat*</code> , <code>*verify*</code>     | <a href="#">15.4 反作弊</a>        |
| 规划/任务 | <code>deep_plan</code> , <code>planner</code> , <code>timeout</code>        | <code>*planning*</code>                               | <a href="#">15.5 深度规划</a>       |
| 内部差异  | <code>USER_TYPE</code> , <code>ant</code> , <code>internal</code>           | <code>*employee*</code> , <code>*staff*</code>        | <a href="#">15.6 内部彩蛋</a>       |

## 审计清单：自己数一遍「是不是 90+」

| 步骤 | 命令/动作                                            | 目的            |
|----|--------------------------------------------------|---------------|
| 1  | <code>`rg -n "feature.?flag`</code>              | FeatureFlag   |
| 2  | <code>rg -n "process\\.env\\.\\.CLAUDE_"`</code> | 找环境变量型开关      |
| 3  | <code>`rg -n "UNDERCOVER`</code>                 | BUDDY         |
| 4  | 导出唯一键到文件 <code>sort -u`</code>                   | 去重后统计数量级      |
| 5  | 对照发行说明 / 配置 schema                               | 区分「存在」与「对外承诺」 |

## 常见误解澄清

| 误解                | 实际情况                                  |
|-------------------|---------------------------------------|
| 「Flag 多 = 产品不稳定」  | 多数开关用于渐进发布与实验，主干保持可回滚。                |
| 「我关了某选项，源码路径就不存在」 | 可能只是默认 <code>false`</code> ，代码仍在二进制中。 |
| 「公开文档没写 = 没有」     | 隐藏功能可能故意不宣传（合规、趣味、内部）。                |
| 「环境变量万能」          | 变量名与语义随版本变化，需以源码/变更为准。                |

## 与本指南其他部分的衔接

| 概念               | 延伸阅读（第十六部分及他处）                                                       |
|------------------|----------------------------------------------------------------------|
| 生命周期 Hook        | <code>SessionStart` / <code>PreToolUse` 可在 Flag 变化时做审计</code></code> |
| Skills / Plugins | 重型扩展与轻量工作流对「工具可用性」另有约束                                               |
| MCP              | 动态指令注入改变模型上下文，与 Flag 正交                                              |
| defer_loading    | 工具延迟加载与特性开关共同影响首包时延                                                  |

## 小结

- 90+ Feature Flags 是工程现实：把复杂度从「分支爆炸」迁移到「可配置矩阵」。
- 学习顺序建议：先找 registry → 再跟环境变量 → 最后读调用点。

- 本部分后续章节挑选与隐私、趣味、完整性、规划、内部差异强相关的子集深入，避免把文档写成不可维护的「开关电话簿」。

## 课后自测

1. 说出至少三种 Flag 来源（构建期、环境、远程、上下文）及其典型优先级思路。
2. 解释为何「公开仓库」可能成为自动触发某类模式的信号。
3. 列出你将如何在团队中建立「Flag 变更审查」规范（命名、文档、回滚）。

## 附录：Feature Flag 命名反模式（避坑表）

| 反模式                              | 症状                                                      | 改进                                 |
|----------------------------------|---------------------------------------------------------|------------------------------------|
| <code>enableNewThing</code> 无域前缀 | 半年后无人敢删                                                 | <code>ui_enable_new_sidebar</code> |
| 同一语义双开关                          | <code>USE_V2</code> 与 <code>DISABLE_LEGACY</code> 并存    | 合并为单一枚举状态                          |
| 远程与本地同名不同义                       | 预发与生产行为漂移                                               | 配置 schema 版本化                      |
| 字符串拼写敏感                          | <code>true</code> / <code>1</code> / <code>on</code> 混用 | 统一解析层                              |

## 附录：与本部分各节映射速查

| 节    | 文件                                      | 一句话      |
|------|-----------------------------------------|----------|
| 15.2 | <code>02-undercover-mode.md</code>      | 公开贡献去标识化 |
| 15.3 | <code>03-buddy-pet.md</code>            | 确定性宠物宇宙  |
| 15.4 | <code>04-anti-cheat.md</code>           | 会话级外观真相源 |
| 15.5 | <code>05-deep-planning.md</code>        | 长跑规划预算   |
| 15.6 | <code>06-internal-easter-eggs.md</code> | 内部用户类型分支 |

下一节：[15.2 Undercover Mode — 公开贡献的「隐身斗篷」](#)

## 第十五部分 · 15.2 Undercover Mode — 公开仓库中的「去标识化」隐身系统

导航: [← 15.1 Feature Flags](#) · [15.3 Buddy 宠物](#) →

### 学习目标

完成本节学习后，你应该能够：

1. **说明** Undercover Mode 的设计目标：Anthropic 员工向公开仓库贡献代码时，弱化或移除与雇主、产品与模型相关的可追溯痕迹。
2. **列举** 该模式典型处理的元数据类别：`Co-Authored-By` 头、模型名展示、文案中的「Claude Code」提及等（以具体实现为准）。
3. **区分** 自动激活（公开仓库上下文）与 强制开启（`CLAUDE_CODE_UNDERCOVER=1`）两种入口，并理解文档所述「无 force-off」的合规含义。
4. **评估** 在开源协作中的利弊：降低品牌噪声 vs. 贡献者身份披露策略需在团队规范中另行约定。

### 生活类比：便衣采访 versus 正装发布会

想象技术团队有两套对外沟通礼仪：

- **正装发布会**（默认产品模式）：名牌、话筒、背景板上写满品牌与产品线——对应日常 Claude Code 体验中的署名、模型名片段、产品文案。
- **便衣采访**（Undercover）：记者只呈现事实与补丁，不携带台标，避免公众把「某公司背书」误读为「官方立场」。

Undercover Mode 就是给公开 **Git 历史**准备的「便衣规则」：代码与 Issue 讨论仍专业，但**自动剥离**容易触发「这是某模型代笔」联想的元数据层。



## 触发条件与优先级（概念表）

| 条件                               | 行为倾向                | 备注                             |
|----------------------------------|---------------------|--------------------------------|
| 当前工作区被判定为公开仓库（可见性信号）             | 自动进入 Undercover 等策略 | 实现上可能组合远程 API、本地 git remote、缓存 |
| 环境变量<br>CLAUDE_CODE_UNDERCOVER=1 | 强制开启 Undercover     | 用于本地复现或 CI 对齐                  |
| 非公开 / 私有上下文且未设强制变量               | 通常走默认产品展示路径         | 具体以版本为准                        |
| 「force-off」开关                    | 文档口径：不提供            | 防止误关导致合规风险                     |

**教学提示：**「无 force-off」强调**安全默认**——宁可多隐藏，也不在公开场景误露标识。

## Mermaid：Undercover 决策管线

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid：元数据剥离的数据流

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：环境变量守卫（示意）

---

```
// undercover.ts (示意)
export function resolveUndercoverMode(ctx: {
  env: NodeJS.ProcessEnv;
  repository: { visibility: 'public' | 'private' | 'unknown' };
}): boolean {
  if (ctx.env.CLAUDE_CODE_UNDERCOVER === '1') {
    return true; // 强制开启, 无对称的 "force off" 环境闸
  }
  if (ctx.repository.visibility === 'public') {
    return true;
  }
  return false;
}
```

```
// commit-message-sanitizer.ts (示意)
export function sanitizeTrailers(
  message: string,
  undercover: boolean
): string {
  if (!undercover) return message;
  return message
    .split('\n')
    .filter((line) => !/^Co-Authored-By:/i.test(line))
    .join('\n');
}
```

```
// branding-strip.ts (示意)
const PATTERNS = [/Claude Code/gi, /Anthropic/gi /* 版本相关 */];

export function stripBranding(text: string, undercover: boolean): string {
  if (!undercover) return text;
  let out = text;
  for (const re of PATTERNS) {
    out = out.replace(re, '');
  }
  return out.replace(/\n{3,}/g, '\n\n').trim();
}
```

## 与 Git / Code Hosting 的协作注意点

| 场景             | 建议                                                       |
|----------------|----------------------------------------------------------|
| Squash merge   | trailer 可能在合并策略中丢失或重写；Undercover 仍应作用于最终消息模板。            |
| Signed commits | 签名证明密钥持有者，与 Undercover 的「品牌剥离」不同维度。                      |
| Bot 账户         | 组织可统一使用 <code>machine account</code> ，与员工 Undercover 互补。 |
| Issue / PR 模板  | 若模板含产品名，Undercover 可能只处理助手生成部分，不改动仓库静态文件。                |

## 合规与开源文化：不是「隐藏作者」

| 维度      | Undercover 通常做什么 | 通常不替代什么        |
|---------|------------------|----------------|
| 品牌与模型痕迹 | 弱化自动化生成标签        | 人工署名与法律要求的版权信息 |
| 公司政策    | 对齐雇主开源指南         | 各公司 CLA 与审批流   |
| 社区信任    | 减少「AI 公关」误解      | 技术审查与测试义务      |

## 调试与验证清单

| 步骤 | 操作                                               | 期望               |
|----|--------------------------------------------------|------------------|
| 1  | 在测试公开克隆上生成提交草稿                                   | 不出现指定 trailer    |
| 2  | <code>export CLAUDE_CODE_UNDERCOVER=1</code> 后重复 | 行为与公开仓一致或更严格     |
| 3  | 检索输出中的模型名关键词                                     | 命中率为 0（在实现承诺范围内） |
| 4  | 对照发行说明变更                                         | 变量名与语义是否迁移       |

## 常见问题 FAQ

| 问题             | 回答方向                           |
|----------------|--------------------------------|
| 为何不对私有仓默认开启？   | 私有协作常需完整遥测与产品内标识用于排障。          |
| 能否在公开仓「半开启」？   | 实现上多为布尔管线；半开需自定义 Hook（见第十六部分）。 |
| 会影响 Token 计费吗？ | 一般不影响；过滤在输出侧，输入系统提示可能仍含中性能力描述。 |

## 与本部分其他节的关系

| 关联                   | 说明                                                 |
|----------------------|----------------------------------------------------|
| <a href="#">15.1</a> | Undercover 常作为 Feature Flag 矩阵中的一支高优先级分支。          |
| <a href="#">15.6</a> | 内部 <code>USER_TYPE</code> 分支与公开合规分支正交：前者偏能力，后者偏展示。 |

## 延伸思考

- 若未来法规要求标注「AI 辅助」，Undercover 与披露义务如何共存？可能需要「技术性标注」与「品牌文案」分层。
- 多语言提交信息中，正则剥离是否足够？可能需要 AST/模板级处理。

## 小结

- Undercover Mode = 面向公开协作的输出消毒管线 + `CLAUDE_CODE_UNDERCOVER=1` 强制入口。
- 自动于公开仓库减少人为遗忘；无 `force-off` 体现「默认安全」。
- 学习重点在元数据层与模板层，而非替代工程流程与社区规范。

## 课后自测

- 画一张草图：从「模型生成」到 `git commit`，哪些环节应插入 sanitizer？
- 解释为何「不提供 `force-off`」在合规叙事上成立。
- 列举三种除 `Co-Authored-By` 外可能需要剥离的元数据类型。

上一节：[15.1 Feature Flags](#)

下一节：[15.3 Buddy 宠物系统](#)

# 第十五部分 · 15.3 Buddy Pet System — BUDDY 旗

# 标与 18 种宠物宇宙

导航: [← 15.2 Undercover](#) · [15.4 反作弊](#) →

## 学习目标

完成本节学习后，你应该能够：

1. 解释 `BUDDY` Feature Flag 如何门控整个宠物子系统，以及它在 UI/状态行中的典型呈现位置（版本相关）。
2. 枚举 18 种宠物角色：`duck`、`goose`、`blob`、`cat`、`dragon`、`octopus`、`owl`、`penguin`、`turtle`、`snail`、`ghost`、`axolotl`、`capybara`、`cactus`、`robot`、`rabbit`、`mushroom`、`chonk`。
3. 复述 五档稀有度及基准概率：`Common 60%`、`Uncommon 25%`、`Rare 10%`、`Epic 4%`、`Legendary 1%`，以及独立 1% 「闪光」层。
4. 理解 以 `userId` 为种子的 `Mulberry32` PRNG 如何实现跨会话稳定的宠物身份。
5. 描述 五维属性 `DEBUGGING` / `PATIENCE` / `CHAOS` / `WISDOM` / `SNARK` 及稀有度对属性下限的约束意义。

## 生活类比：固定星座 versus 每日抽签

把 Buddy 系统想象成「出生证上写死的守护动物」：

- 不是你的心情每天抽一次盲盒，而是身份证号（`userId`）决定你终身对应的那个小生灵。
- 稀有度像血统证书上的品级：传说级并不改变你写代码的能力，但影响文案趣味与属性条起点。
- 闪光像 Pokémon 的异色：基础角色不变，多一层极低概率的视觉/标签修饰。

因此 Buddy 是确定性娱乐系统，不是竞技排行榜。

## 18 宠物角色速查表

| 英文名      | 中文昵称（教学） | 气质联想         |
|----------|----------|--------------|
| duck     | 小鸭       | 划水但可爱        |
| goose    | 大鹅       | 攻击性幽默        |
| blob     | 软泥       | 情绪容器         |
| cat      | 猫        | 高冷监工         |
| dragon   | 龙        | 架构师脾气        |
| octopus  | 章鱼       | 多线程触手        |
| owl      | 猫头鹰      | 夜猫子 review   |
| penguin  | 企鹅       | 冷静排队 debug   |
| turtle   | 龟        | 慢速稳健         |
| snail    | 蜗牛       | 渐进式交付        |
| ghost    | 幽灵       | 隐式依赖猎手       |
| axolotl  | 美西螈      | 再生力（重写模块）    |
| capybara | 水豚       | 情绪稳定         |
| cactus   | 仙人掌      | 扎人的直评        |
| robot    | 机器人      | 规则至上         |
| rabbit   | 兔子       | 快速迭代         |
| mushroom | 蘑菇       | 在阴暗角落找 bug   |
| chonk    | 圆滚滚      | 重量级 refactor |

## 稀有度与概率（教学口径）

| 稀有度       | 基准概率                  | 属性下限倾向（概念）    |
|-----------|-----------------------|---------------|
| Common    | 60%                   | 下限较低，方差小      |
| Uncommon  | 25%                   | 略抬升           |
| Rare      | 10%                   | 明显抬升          |
| Epic      | 4%                    | 高起步           |
| Legendary | 1%                    | 顶配起点          |
| Shiny（闪光） | 独立 1%（与上表正交叠加语义，教学模型） | 视觉层，不改变种子的主物种 |

说明：「独立 1% 闪光」在实现上可建模为二次掷骰或高位掩码；本节强调与主稀有度滚动分离的趣味层。

## Mermaid：BUDDY 功能门控

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid：种子确定性管线

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：Mulberry32 与滚动（示意）

```
// mulberry32.ts (示意)
export function mulberry32(seed: number) {
  return function () {
    let t = (seed += 0x6d2b79f5);
    t = Math.imul(t ^ (t >>> 15), t | 1);
    t ^= t + Math.imul(t ^ (t >>> 7), t | 61);
    return ((t ^ (t >>> 14)) >>> 0) / 4294967296;
  };
}
```

```
// buddy-roll.ts (示意)
import { hashUserIdToSeed } from './hash';
import { mulberry32 } from './mulberry32';

export type Rarity = 'common' | 'uncommon' | 'rare' | 'epic' | 'legendary';

const SPECIES = [
  'duck',
  'goose',
  'blob',
  'cat',
  'dragon',
  'octopus',
  'owl',
  'penguin',
  'turtle',
  'snail',
  'ghost',
  'axolotl',
  'capybara',
  'cactus',
  'robot',
  'rabbit',
  'mushroom',
  'chonk',
] as const;

export function rollBuddy(userId: string) {
  const rng = mulberry32(hashUserIdToSeed(userId));
  const rarity = rollRarity(rng); // 60/25/10/4/1
  const speciesIndex = Math.floor(rng() * SPECIES.length);
  const species = SPECIES[speciesIndex];
  const shiny = rng() < 0.01; // 独立 1% 闪光
  const attrs = rollAttributes(rng, rarity);
  return { species, rarity, shiny, attrs };
}
```



```
// attributes.ts (示意)
export type BuddyAttrs = {
  debugging: number;
  patience: number;
  chaos: number;
  wisdom: number;
  snark: number;
};

const RARITY_FLOORS: Record<Rarity, number> = {
  common: 10,
  uncommon: 18,
  rare: 28,
  epic: 40,
  legendary: 55,
};

export function rollAttributes(
  rng: () => number,
  rarity: Rarity
): BuddyAttrs {
  const floor = RARITY_FLOORS[rarity];
  const pick = () => floor + Math.floor(rng() * (100 - floor));
  return {
    debugging: pick(),
    patience: pick(),
    chaos: pick(),
    wisdom: pick(),
    snark: pick(),
  };
}
```

## 五维属性与文案钩子

| 属性        | 含义（教学）    | UI 可能用法     |
|-----------|-----------|-------------|
| DEBUGGING | 找 bug 积极性 | 失败重试提示语气    |
| PATIENCE  | 对长任务的容忍   | 进度条旁白       |
| CHAOS     | 随机玩梗频率    | 彩蛋密度        |
| WISDOM    | 「架构师腔」浓度  | 建议条风格       |
| SNARK     | 吐槽锋利度     | 代码 smell 评论 |

## 外观系统预告（与 15.4 联动）

| 组件   | 规模  | 备注                 |
|------|-----|--------------------|
| 眼睛样式 | 6 种 | 与种子派生索引            |
| 帽子   | 8 种 | Common 稀有度无帽（教学口径） |
| 闪光   | 二元  | 独立 1%              |

具体会话级重算防篡改逻辑见 [15.4](#)。

## 产品伦理与体验边界

| 话题   | 建议                                   |
|------|--------------------------------------|
| 儿童向？ | 默认终端受众为开发者；仍应避免过度干扰主任务流。             |
| 付费墙？ | 教学上视为纯客户端彩蛋，不与 API 配额挂钩。             |
| 隐私   | userId 仅客户端派生展示，不应上传敏感画像（以正式隐私政策为准）。 |

## 调试开关建议

| 操作              | 目的            |
|-----------------|---------------|
| 固定测试 userId     | 验证同一用户跨机器一致   |
| 切换 BUDDY off/on | 确认门控与降级路径     |
| 快照 UI           | 回归眼睛/帽子贴图资源路径 |

## 常见问题 FAQ

| 问题         | 回答方向                        |
|------------|-----------------------------|
| 换账号会变吗？    | 会，种子来自 userId。              |
| 能「刷」传说宠吗？  | 确定性种子下不能靠重 Roll；除非换身份或实现变更。 |
| 宠物影响模型能力吗？ | 不应影响；属娱乐与 UI。               |

## 小结

- BUDDY 门控一整个**确定性宠物宇宙**：18 物种 × 稀有度 × 独立闪光 × 外观组件。
- Mulberry32(userId) 保证「你是谁，就养哪只」。
- 五维属性服务文案与个性，**稀有度**主要抬升下限，不是 PVP 数值。

## 课后自测

1. 手写伪代码：给定固定 seed，连续调用 `mulberry32` 10 次，输出应可复现。
2. 解释「闪光 1%」与「Legendary 1%」在概率论上的独立性含义。
3. 列出 Common 无帽对 UI 资源组织的两点工程影响（条件渲染、资产打包）。

上一节： [15.2 Undercover Mode](#)

下一节： [15.4 外观反作弊与会话重算](#)

# 第十五部分 · 15.4 Buddy 外观反作弊 — 会话级重算与配置一致性

导航： [← 15.3 Buddy](#) · [15.5 深度规划](#) →

## 学习目标

完成本节学习后，你应该能够：

1. **解释** 为何纯客户端彩蛋仍需要「反作弊」：防止用户通过编辑本地配置获得**非种子一致**的外观组合。
2. **描述** 每次会话重算外观的策略：在会话初始化阶段基于权威输入（如 `userId`、服务端/密钥链、会话盐）重新派生眼睛/帽子/闪光位。
3. **关联** [15.3](#) 中的组件规模：**6 眼睛**、**8 帽子**（Common 无帽）、物种与稀有度。
4. **对比** 「持久化缓存」与「会话派生」在 UX 与完整性上的权衡。

## 生活类比：游乐园手环 versus 自制贴纸

把 Buddy 外观想象成游乐园入场手环颜色：

- 手环颜色由售票系统根据**身份证哈希**决定，不是你进场后自己贴一张彩色贴纸就算数。
- 每次入园（**新会话**），闸机重新验证并**重新发放**当日有效手环——防止你昨晚把贴纸撕下来换给别人用。
- **Common** 无帽像「基础票不发遮阳帽」：不是歧视，是**稀有度分层**的一部分。

## 威胁模型（教学简化）

| 攻击意图     | 手段示例                 | 防御思路                  |
|----------|----------------------|-----------------------|
| 改本地 JSON | 手动把 <b>hat</b> 改成传奇款 | 会话重算覆盖                |
| 换皮资源     | 替换 png/svg 文件        | 哈希校验或打包签名（若实现）        |
| 内存改值     | 运行时 patch 状态         | 周期性 reconcile 或只信派生函数 |
| 共享配置     | 拷贝「他人宠物档案」           | 种子绑定 userId，拷贝无效      |

## Mermaid：会话重算时序

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid：外观派生状态机

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：派生与覆盖（示意）

```
// appearance.ts (示意)
export type BuddyAppearance = {
  eyeVariant: number; // 0..5
  hatVariant: number | null; // 0..7 or null if common without hat
  shiny: boolean;
};

export function deriveAppearance(input: {
  userId: string;
  sessionId: string;
  rarity: Rarity;
  species: string;
}): BuddyAppearance {
  const rng = mulberry32(
    hashCompositeSeed(input.userId, input.sessionId, input.species)
  );
  const eyeVariant = Math.floor(rng() * 6);
  const shiny = rng() < 0.01;
  const hatVariant =
    input.rarity === 'common' ? null : Math.floor(rng() * 8);
  return { eyeVariant, hatVariant, shiny };
}
```

```
// session-bootstrap.ts (示意)
export function bootstrapBuddyUiState(ctx: RuntimeContext) {
  const rolled = rollBuddy(ctx.userId); // 15.3
  const canonical = deriveAppearance({
    userId: ctx.userId,
    sessionId: ctx.sessionId,
    rarity: rolled.rarity,
    species: rolled.species,
  });
  // 本地文件若存在篡改字段，一律以 canonical 为准
  ctx.ui.buddy = { ...rolled, appearance: canonical };
}
```

```
// integrity-check.ts (示意)
export function reconcileStoredBuddyPrefs(
  disk: Partial<BuddyAppearance> | undefined,
  canonical: BuddyAppearance
): BuddyAppearance {
  if (!disk) return canonical;
  const mismatch =
    disk.eyeVariant !== canonical.eyeVariant ||
    disk.hatVariant !== canonical.hatVariant ||
    disk.shiny !== canonical.shiny;
  if (mismatch) {
    telemetryDebug('buddy_prefs_reset', { reason: 'tamper' });
    return canonical;
  }
  return canonical;
}
```

## 眼睛 × 帽子 × 稀有度（约束表）

| 稀有度       | 帽子                     | 眼睛    | 闪光    |
|-----------|------------------------|-------|-------|
| Common    | 无（ <code>null</code> ） | 6 选 1 | 独立 1% |
| Uncommon  | 8 选 1                  | 6 选 1 | 独立 1% |
| Rare      | 8 选 1                  | 6 选 1 | 独立 1% |
| Epic      | 8 选 1                  | 6 选 1 | 独立 1% |
| Legendary | 8 选 1                  | 6 选 1 | 独立 1% |

## 与「仅种子\_roll」方案的区别

| 方案                                           | 优点              | 缺点                                |
|----------------------------------------------|-----------------|-----------------------------------|
| 仅 <code>userId</code> 种子                     | 极简、可预测          | 换会话无法轮换会话盐；篡改本地更易「粘住」错误状态         |
| <code>userId</code> + <code>sessionId</code> | 防简单拷贝配置、鼓励每会话对齐 | 同用户每天外观可能变化（若实现如此）                |
| <code>userId</code> 唯一权威                     | 最强「身份一致」        | 需在文档声明 <code>session</code> 盐是否参与 |

本指南教学口径：强调「每次会话重算」——实现可选用 `session` 盐或仅校验和覆盖，但**核心**

是不以磁盘为最终真相源。

## 遥测与隐私

| 事件                | 是否应含 PII | 说明       |
|-------------------|----------|----------|
| buddy_prefs_reset | 否        | 用匿名会话 ID |
| 外观哈希              | 可        | 仅用于崩溃聚合  |

## 测试用例矩阵（节选）

| 用例 | 输入                | 期望                            |
|----|-------------------|-------------------------------|
| T1 | 删配置文件             | 回退 canonical                  |
| T2 | hat 改非法索引         | 重置为派生值                        |
| T3 | common + 非 null 帽 | 强制 null                       |
| T4 | 连续两次 SessionStart | 若含 session 盐则 eye/hat 可变；否则稳定 |

## 与 Feature Flag 的交互

| Flag         | 影响              |
|--------------|-----------------|
| BUDDY off    | 跳过派生与 UI，防无谓计算  |
| 调试 Flag（若存在） | 可能固定 rng 种子便于截图 |

## 常见问题 FAQ

| 问题       | 回答方向                          |
|----------|-------------------------------|
| 这是安全机制吗？ | 主要是体验一致性，不是金融级安全。             |
| 会影响性能吗？  | 派生为 O(1) 计算，可忽略相对 LLM 延迟。     |
| mod 社区呢？ | 第三方皮肤与官方 integrity 可能冲突，属预期外。 |

## 小结

- 反作弊在此语境 = 配置篡改无效化，通过会话级 canonical 重算实现。
- 6 眼 / 8 帽 / Common 无帽 必须在派生函数中硬编码约束，而非仅信 UI。
- 与 15.3 的 PRNG 配套：物种稀有可长期稳定，外观可会话刷新——二者并不矛盾，取决于复合种子设计。

## 课后自测

1. 写出一个「磁盘偏好」结构体，并标注哪些字段是「只读展示」、哪些是「可写用户设置」（如静音彩蛋）。
2. 解释为何「会话重算」能阻断「复制他人配置文件」攻击路径。
3. 绘制测试矩阵中 T3 的断言伪代码。

上一节：[15.3 Buddy 宠物](#)

下一节：[15.5 Deep Planning Mode](#)

# 第十五部分 · 15.5 Deep Planning Mode — 长达 30 分钟的单任务深度规划

导航：[← 15.4 反作弊](#) · [15.6 内部彩蛋](#) [→](#)

## 学习目标

完成本节学习后，你应该能够：

1. 定义 Deep Planning Mode：在单一规划任务上允许显著延长的思考/工具/子步骤预算（教学口径：约 30 分钟），用于复杂重构、迁移或跨模块设计。
2. 区分 普通对话回合限制与「深度规划」在超时、心跳、取消语义上的差异。
3. 识别 该模式在源码中可能与哪些 Flag、调度器、任务队列共同出现。
4. 评估 成本与风险：Token、API 账单、用户等待体验、长时间占用的并发槽位。



## 生活类比：快餐套餐 versus 私房慢炖

- **普通模式**像写字楼下的**快餐套餐**：15 分钟内上菜，适合「改一个函数签名」。
- **Deep Planning Mode**像周末预约的**慢炖私房菜**：厨师（规划器）可以在厨房连续工作半小时，中间多道备料步骤（读文件、画依赖图、列风险清单），最后一次性呈上「完整菜单」（里程碑计划）。

关键不是「聊天更久」，而是**同一个规划任务**被系统识别为**允许长跑**的工作类型。

## 能力边界（概念表）

| 维度      | 普通规划     | Deep Planning（教学口径） |
|---------|----------|---------------------|
| 单任务时间上限 | 较短（分钟级）  | 约 30 分钟             |
| 中断与恢复   | 依赖会话检查点  | 可能需要显式 checkpoint   |
| 工具调用深度  | 受默认预算约束  | 放宽迭代轮次              |
| 适用输入    | 大多数用户提示  | 明确「规划类」意图或 Flag     |
| 并发      | 与常规任务共用池 | 可能占用更长槽位            |

## Mermaid：深度规划作业生命周期

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid：调度与超时层级

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：超时策略（示意）

---

```
// planning-timeouts.ts (示意)
export const DEFAULT_PLANNING_BUDGET_MS = 3 * 60 * 1000; // 3 分钟
export const DEEP_PLANNING_BUDGET_MS = 30 * 60 * 1000; // 30 分钟

export function resolvePlanningBudget(
  flags: FeatureSet,
  taskKind: 'quick' | 'plan' | 'deep_plan'
): number {
  if (taskKind === 'deep_plan' && flags.deepPlanning) {
    return DEEP_PLANNING_BUDGET_MS;
  }
  if (taskKind === 'plan') {
    return DEFAULT_PLANNING_BUDGET_MS;
  }
  return 60 * 1000;
}
```

```
// planning-runner.ts (示意)
export async function runPlanningJob(job: PlanningJob, budgetMs: number) {
  const controller = new AbortController();
  const timer = setTimeout(() => controller.abort(), budgetMs);
  try {
    await orchestrateTools(job, { signal: controller.signal });
  } finally {
    clearTimeout(timer);
  }
}
```

```
// checkpoint.ts (示意)
export async function maybeCheckpoint(state: PlannerState) {
  if (state.elapsedMs > 5 * 60 * 1000) {
    await persistPlannerCheckpoint(state);
  }
}
```

## 何时应开启？（决策表）

| 场景             | 建议  | 原因             |
|----------------|-----|----------------|
| 单体仓库跨 50+ 文件迁移 | 适合  | 需要广泛只读扫描与一致性计划 |
| 修复一个拼写错误       | 不适合 | 浪费长跑槽位与费用      |
| CI 中自动规划       | 谨慎  | 30 分钟作业可能阻塞流水线 |
| 安全相关渗透测试计划     | 视政策 | 可能需审计日志与人工门控   |

## 与其他子系统交互

| 子系统   | 交互点                                                            |
|-------|----------------------------------------------------------------|
| 工具系统  | 更多 <code>Grep</code> / <code>Glob</code> / <code>LSP</code> 轮次 |
| 上下文窗口 | 需要摘要与 checkpoint 防溢出                                           |
| 权限    | 长跑中多次触及敏感工具 → <code>PermissionRequest</code> Hook              |
| Hooks | <code>PreToolUse</code> 可在深度任务中更严格拦截                           |

## 可观测性

| 指标                               | 说明       |
|----------------------------------|----------|
| <code>planning_wall_ms</code>    | 墙钟时间     |
| <code>planning_tool_calls</code> | 工具次数     |
| <code>planning_tokens</code>     | 估算 Token |
| <code>planning_aborted</code>    | 取消率      |

## 用户体验建议

| 实践    | 说明                                |
|-------|-----------------------------------|
| 显式进度  | 每 N 分钟输出阶段标题                      |
| 可取消   | Ctrl+C / Stop 必须可靠                |
| 结果结构化 | 强制 Markdown 章节：目标 / 风险 / 里程碑 / 回滚 |
| 费用提示  | 长跑前二次确认（若产品支持）                    |

## 风险与缓解

| 风险      | 缓解              |
|---------|-----------------|
| 模型陷入循环  | 最大轮次 + 重复检测     |
| 上下文爆炸   | 中间摘要压缩          |
| 误触发深度模式 | 意图分类 + Flag 双门槛 |
| 资源挤占    | 每用户并发上限         |

## 常见问题 FAQ

| 问题                 | 回答方向                   |
|--------------------|------------------------|
| 是否等于「Thinking 更长」？ | 更接近「规划作业预算」，仍受工具与模型限制。 |
| 能否拆成多会话？           | 可以，但需 checkpoint 文件接力。 |
| 离线可用？              | 取决于本地模型与实现；通常仍受算力限制。   |

## 与 15.1 Flag 矩阵的映射

| Flag（示意名）            | 作用       |
|----------------------|----------|
| DEEP_PLANNING        | 打开门控     |
| PLANNING_MAX_MINUTES | 若存在可调优配置 |

## 延伸：与企业版 SLA

| 维度   | 个人    | 企业           |
|------|-------|--------------|
| 最长规划 | 受账户策略 | 可能被管理员调低以控成本 |
| 审计   | 可选    | 常强制          |

## 小结

- Deep Planning Mode 把「规划」从短跑升级为中长跑（~30 分钟单任务预算的教学口径）。
- 工程关键是超时策略 + checkpoint + 工具预算三位一体。
- 使用上遵循「大事慢炖，小事快餐」，并与 Hooks、权限、遥测联动审视。

## 课后自测

1. 画一张时间轴：0—30 分钟内应至少出现几次 checkpoint 才合理？
2. 写伪代码：检测连续三次相同工具相同参数时中断循环。
3. 解释为何深度规划对「并发槽位」是产品级问题而非纯算法问题。

## 附录：伪 API（教学）

```
interface DeepPlanningRequest {
  goal: string;
  constraints: string[];
  nonGoals: string[];
}

interface DeepPlanningResult {
  milestones: { title: string; acceptance: string }[];
  risks: { severity: 'low' | 'med' | 'high'; note: string }[];
  rollback: string;
}
```

上一节：[15.4 反作弊](#)

下一节：[15.6 内部彩蛋与 USER TYPE](#)

## 第十五部分 · 15.6 内部彩蛋 — `USER_TYPE === 'ant'` 与实验输出策略

导航: [← 15.5 深度规划](#) · [返回 15.1](#)

### 学习目标

完成本节学习后，你应该能够：

1. 识别 源码中可能存在的**内部用户类型**分支，例如 `USER_TYPE === 'ant'`（教学代号），用于区分**雇员/内测**与公众默认路径。
2. 解释「更激进输出策略」在工程上可能指：更高温度、更少保守拒答模板、启用实验工具、提前曝光未发布 Flag。
3. 论证 为何此类分支**不应**被普通用户依赖：稳定性、合规、合同条款与随时下线。
4. 对照 [15.2 Undercover](#)：内部能力扩张 vs. 对外标识弱化是**不同维度**。

### 生活类比：试飞员 versus 民航乘客

- **民航乘客**（默认用户）：遵循保守操作规程，安全带演示、限流、延误以安全为先。
- **试飞员**（内部 `ant` 分支）：可以在**指定空域**测试更高推重比、触发更多传感器日志——数据用于改进下一版民航软件。

你在 GitHub 上读到的「`ant` 分支」不是让你**混进驾驶舱**的漏洞，而是提醒你：**存在一条为内部验证预留的跑道**。

## 概念要素表

| 要素   | 说明                                                 |
|------|----------------------------------------------------|
| 判别信号 | 登录身份、组织租户、设备证书、构建变体等（实现相关）                         |
| 分支键  | <code>USER_TYPE</code> 枚举，教学示例值 <code>'ant'</code> |
| 行为差异 | 更激进输出、实验功能可见性、诊断日志级别                               |
| 生命周期 | 可能随版本重构改名或删除                                       |

## Mermaid：用户类型路由

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid：策略差异（示意）

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：分支守卫（示意）

```
// user-profile.ts (示意)
export type UserType = 'default' | 'ant' | 'partner';

export function resolveUserType(ctx: AuthContext): UserType {
  if (ctx.claims.internalStaff) return 'ant';
  if (ctx.claims.partnerLab) return 'partner';
  return 'default';
}
```

```
// model-policy.ts (示意)
export function buildOutputPolicy(userType: UserType): OutputPolicy {
  if (userType === 'ant') {
    return {
      temperatureBoost: 0.1,
      enableExperimentalTools: true,
      systemPromptSuffix: INTERNAL_LAB_SUFFIX,
    };
  }
  return DEFAULT_OUTPUT_POLICY;
}
```

```
// feature-gates.ts (示意)
export function mergeFeatureFlags(
  base: FeatureSet,
  userType: UserType
): FeatureSet {
  if (userType === 'ant') {
    return {
      ...base,
      buddy: true,
      deepPlanning: true,
      undercover: false, // 内部调试可能关闭某些合规包装
    };
  }
  return base;
}
```

**警告：**最后一段 `undercover: false` 仅为展示「内部矩阵可不同」的教学虚构；真实矩阵以合规为准，切勿据此推断可绕过公开仓策略。

## 「更激进输出」拆解

| 可能含义  | 工程实现线索                                        | 对终端用户影响      |
|-------|-----------------------------------------------|--------------|
| 解码参数  | <code>temperature</code> 、 <code>top_p</code> | 回答更多样        |
| 系统提示  | 附加「可推测未公开 API」条款（内部）                          | 不可依赖         |
| 工具可见性 | 暴露诊断类 tool                                    | 安全风险上升 → 仅内部 |
| 拒答阈值  | 降低安全拦截敏感度                                     | 绝不适用于对外默认    |



## 道德与合规边界

| 原则      | 说明                                  |
|---------|-------------------------------------|
| 不冒充内部   | 伪造 <code>USER_TYPE</code> 可能违反服务条款。 |
| 不传播绕过技巧 | 本指南只讲只读理解源码结构。                      |
| 数据最小化   | 内部分支常伴随更强日志，需员工培训。                  |

## 与 Undercover 的对比表

| 维度   | Undercover (15.2) | <code>ant</code> 内部分支 |
|------|-------------------|-----------------------|
| 目标   | 对外弱化标识            | 对内增强能力                |
| 默认触发 | 公开仓 / 环境变量        | 内部认证                  |
| 受众   | 开源贡献者             | 员工/实验室                |
| 可讨论性 | 可公开教学             | 以公司政策为准               |

## 源码考古建议

| 步骤 | 动作                          |
|----|-----------------------------|
| 1  | <code>`rg "USER_TYPE</code> |
| 2  | 跟踪枚举到策略合成函数                 |
| 3  | 对照 changelog 是否重命名          |
| 4  | 区分客户端与网关两侧判断                |

## 测试替身（工程视角）

| 场景   | 做法                                     |
|------|----------------------------------------|
| 单元测试 | 注入 <code>userType: 'ant'</code> 快照策略对象 |
| E2E  | 勿在公共 CI 依赖真实内部账号                       |
| 开源贡献 | 提交前确认不泄露内部 endpoint                    |

## 常见问题 FAQ

| 问题                | 回答方向                      |
|-------------------|---------------------------|
| 我能改本地二进制打开 ant 吗？ | 不建议且可能违法/违约；此处仅学术讨论分支存在性。 |
| ant 会进发行版吗？       | 代码可能在同一二进制，但门控在服务端/证书。    |
| 与 Buddy 有关吗？      | 可能独立；内部或默认全开做 dogfood。    |

## 与其他部分索引

| 章节                   | 关联             |
|----------------------|----------------|
| <a href="#">15.1</a> | Flag 总览        |
| <a href="#">15.5</a> | 内部或更早试用长跑规划    |
| 第十六部分                | Hooks 审计内部工具调用 |

## 小结

- `USER_TYPE === 'ant'` 代表一类内部/实验室策略模板，而非用户可投机开启的「隐藏菜单」。
- 更激进输出与实验功能服务于迭代速度，伴随合规与安全的额外义务。
- 阅读源码的价值在于理解产品线结构，而非寻找捷径。

## 课后自测

1. 画一张表：列出三项「公众策略」与三项「内部策略」差异，并标注哪一侧更适合自动化测试覆盖。
2. 解释为何内部分支常与「更强遥测」共存。
3. 从开源伦理角度，讨论 Undercover 与内部激进策略同时存在是否合理。

## 本部分回顾（15.1–15.6）

| 节    | 关键词                                                 |
|------|-----------------------------------------------------|
| 15.1 | 90+ Flags、registry、检索                               |
| 15.2 | Undercover、公开仓、 <code>CLAUDE_CODE_UNDERCOVER</code> |
| 15.3 | BUDDY、18 宠、Mulberry32、五维属性                          |
| 15.4 | 会话重算、防篡改外观                                          |
| 15.5 | Deep Planning、~30 分钟                                |
| 15.6 | <code>ant</code> 、内部实验策略                            |

上一节： [15.5 Deep Planning](#)

返回： [15.1 Feature Flags](#)

## 第16篇：Hooks/Skills/Plugins

### 第十六部分 · Hooks / Skills / Plugins 生态 (16.1)

#### — 扩展点总览

**导航：**本部分共 8 节。本节建立钩子 (Hooks)、技能 (Skills)、\*\*插件 (Plugins)\*\* 三者的边界与协作模型；后续各节深入 `PreToolUse`、Skills、Plugins、MCP 注入、斜杠命令、延迟加载与综合演练。

#### 学习目标

完成本节学习后，你应该能够：

1. 定义 Hooks：用户声明的 Shell / HTTP / LLM 回调，在运行时特定生命周期点被调度执行。
2. 枚举 六个 Hook 事件：`SessionStart`、`UserPromptSubmit`、`PreToolUse`、`PermissionRequest`、`PostToolUse`、`Stop`。
3. 对比 Skills（基于 Markdown 的轻量工作流，可声明 `allowed-tools`）与 Plugins（更重：自定义命令、独立目录配置、`disable-model-invocation` 等硬性标记）。
4. 叙述 端到端管线：事件触发 → 收集注册 Hooks → matcher 过滤 → callback 执行 → 聚合决策（`allow` / `block` / `modify`）。
5. 预告 MCP 动态指令注入、斜杠自定义命令、`defer_loading` 与本部分的衔接章节。

#### 生活类比：机场联检流水线

把一次完整的 Claude Code 会话想象成国际航班抵达通道：

- **SessionStart**：航班靠桥，边检系统开机自检（环境变量、工作区、MCP 连接）。
- **UserPromptSubmit**：旅客递交入境卡——Hook 可预审表单（过滤敏感词、附加审计 ID）。
- **PreToolUse**：行李过 X 光机：可放行 / 扣留 / 改贴标签（改写工具名与参数）。
- **PermissionRequest**：海关抽查需人工柜台——Hook 可自动批准或升级审批。

- **PostToolUse**：行李离开传送带后**二次拍照存档**（写日志、同步外部工单）。
- **Stop**：旅客离开隔离区——**做清场与埋点**。

**Skills**像「海关张贴的多语言告示」：轻量、可复制、告诉模型遇到某类场景**必须按某流程走**。

**Plugins**像「航站楼里的特许经营店」：自带装修（目录结构）、员工手册（命令）、甚至「禁止总部代下单」（`disable-model-invocation`）。

### 三件套对比总表

| 维度     | Hooks            | Skills                 | Plugins         |
|--------|------------------|------------------------|-----------------|
| 载体     | 配置 + 脚本/HTTP/LLM | Markdown + 元数据         | 包/目录 + manifest |
| 主要用途   | 治理、审计、拦截         | 工作流模板、工具白名单提示          | 命令、深度集成、强约束     |
| 执行时机   | 生命周期事件           | 模型匹配场景时注入              | 安装期 + 运行期       |
| 能否阻止工具 | 是（尤其 PreToolUse） | 间接（提示 + allowed-tools） | 依实现；常配合 Hooks   |
| 典型用户   | 平台/安全团队          | 团队模板作者                 | 扩展开发者           |

### 六个 Hook 事件速查

| 事件                             | 触发时机（概念）  | 典型用途            |
|--------------------------------|-----------|-----------------|
| <code>SessionStart</code>      | 会话建立后尽早   | 环境探测、注入审计上下文    |
| <code>UserPromptSubmit</code>  | 用户消息进入管线前 | 过滤、改写、附加标签      |
| <code>PreToolUse</code>        | 工具调用执行前   | <b>拦截/改写/允许</b> |
| <code>PermissionRequest</code> | 需要显式授权时   | 自动策略、工单联动       |
| <code>PostToolUse</code>       | 工具返回后     | 日志、同步外部 CMDB    |
| <code>Stop</code>              | 会话或任务终止   | 清理、指标上报         |

### Mermaid：Hook 管线总览

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid: Skills / Plugins / MCP 共生

[Mermaid 图表 — 请在在线版本中查看]

### 源码片段: Hook 注册表 (示意)

```
// hooks-registry.ts (示意)
export type HookEvent =
  | 'SessionStart'
  | 'UserPromptSubmit'
  | 'PreToolUse'
  | 'PermissionRequest'
  | 'PostToolUse'
  | 'Stop';

export type HookCallback =
  | { kind: 'shell'; command: string }
  | { kind: 'http'; url: string; method: 'POST' | 'GET' }
  | { kind: 'llm'; model: string; promptTemplate: string };

export interface RegisteredHook {
  id: string;
  event: HookEvent;
  matcher?: ToolMatcher;
  callback: HookCallback;
  priority: number;
}
```

```
// hook-runner.ts (示意)
export async function dispatchHooks(
  event: HookEvent,
  payload: HookPayload,
  hooks: RegisteredHook[]
): Promise<HookDecision> {
  const candidates = hooks
    .filter((h) => h.event === event)
    .filter((h) => !h.matcher || matchTool(h.matcher, payload.tool))
    .sort((a, b) => a.priority - b.priority);

  let decision: HookDecision = { type: 'allow' };
  for (const h of candidates) {
    const partial = await runCallback(h.callback, payload);
    decision = mergeDecisions(decision, partial);
    if (decision.type === 'block') break;
  }
  return decision;
}
```

PreToolUse

为何被单独成章？

| 原因   | 说明                                                            |
|------|---------------------------------------------------------------|
| 权力最大 | 可直接改变工具调用图                                                    |
| 安全关键 | 数据外泄、破坏性命令多在此拦截                                               |
| 复杂匹配 | 支持 MCP 工具名如 <code>mcp__server__tool</code>                    |
| 决策三态 | <code>allow</code> / <code>block</code> / <code>modify</code> |

详见 [16.2 PreToolUse](#)。

Skills 核心句柄：

allowed-tools

Skills Markdown frontmatter 中常见：

```
---
name: sql-migration-review
allowed-tools: Bash, Grep, FileRead
---
```

运行时可将此清单与系统策略合并，迫使模型在匹配场景下优先调用声明工具（具体 enforcement 以版本为准）。

Plugins 的「重型」特征（节选）

| 特征                                    | 含义             |
|---------------------------------------|----------------|
| 自定义命令                                 | 新增斜杠或 CLI 子命令  |
| 独立目录                                  | 资源隔离、版本升级      |
| <code>disable-model-invocation</code> | 硬性禁止模型自动调用某些路径 |

详见 [16.4 Plugins](#)。

## MCP 与系统提示词

MCP 设备连接成功后，其 `instructions` 字段常被动态拼接进系统提示词，使模型「知道」此外设的约束与能力。见 [16.5](#)。

## defer\_loading（预告）

工具注册表可使用 [延迟加载](#)避免冷启动时一次性解析全部工具定义，保护本地缓存与首屏时延。见 [16.7](#)。

## 与本指南其他部分的衔接

| 概念            | 参考    |
|---------------|-------|
| 工具系统          | 第六部分  |
| 权限模型          | 第七部分  |
| Feature Flags | 第十五部分 |

## 常见问题 FAQ

| 问题                 | 回答方向                    |
|--------------------|-------------------------|
| Hooks 会显著拖慢吗？      | Shell/HTTP 可能；需超时与并发控制。 |
| Skills 能替代单元测试？    | 不能；它是流程与提示治理。           |
| Plugins 与 MCP 冲突吗？ | 正交；注意工具名空间去重。           |

## 小结

- **Hooks** = 生命周期回调 + 决策聚合；**PreToolUse** 是皇冠上的明珠。
- **Skills** = Markdown 轻量封装 + 工具白名单语义。
- **Plugins** = 重型扩展面；**MCP** 动态注入 instructions；**defer\_loading** 优化工具加载。



## 课后自测

1. 将六个事件按时间顺序手写排列，并各举一例非安全类用途。
2. 画一张 Venn 图文字描述：Skills 与 Plugins 的重叠与差异。
3. 解释「matcher 过滤」在  $O(n)$  Hook 列表下的性能注意点。

下一节：[16.2 PreToolUse — 拦截、放行与改写](#)

## 第十六部分 · 16.2 PreToolUse — 最强大的 Hook： block / allow / modify

导航：[← 16.1 总览](#) · [16.3 Skills →](#)

### 学习目标

完成本节学习后，你应该能够：

1. 解释 **PreToolUse** 在六事件中的特殊地位：唯一在工具执行前拥有对调用图的结构影响力。
2. 使用 三种决策语义：**block**（终止调用）、**allow**（原样通过）、**modify**（改写工具名与/或参数）。
3. 编写 matcher 规则以匹配 MCP 工具名形态：**mcp\_\_server\_\_tool**（双下划线分段，教学口径）。
4. 复述 处理管线：事件触发 → 收集 Hooks → matcher 过滤 → callback 执行 → 决策合并。
5. 识别 与权限系统、**PermissionRequest** 事件的协作与边界。

### 生活类比：海关开箱权

普通 Hooks 像「监控摄像头」——记录发生了什么。

**PreToolUse** 像海关关员手里的「退运章」与「改单笔」：

- **block**：整箱货物不准入境（该次工具调用直接作废）。

- **allow**：单据齐全，放行（透传模型生成的调用）。
- **modify**：箱单写错了货值，关员**当场划线更正**再放行（改参数或替换为安全等价调用）。

没有这一环，事后审计只能「追责」，难以「止损」。

## 决策语义对照表

| 决策     | 对执行器影响    | 对模型可见性    | 典型场景                         |
|--------|-----------|-----------|------------------------------|
| allow  | 继续执行原调用   | 通常无额外消息   | 默认路径                         |
| block  | 不执行       | 返回结构化拒绝原因 | <code>rm -rf /</code> 、未授权域名 |
| modify | 执行被编辑后的调用 | 可记录 diff  | 将绝对路径改为沙箱路径                  |

## MCP 工具名匹配

| 形式     | 示例                                   | 说明                                             |
|--------|--------------------------------------|------------------------------------------------|
| 内置工具   | <code>FileRead</code>                | 单段 PascalCase（示意）                              |
| MCP 规范 | <code>mcp_github_create_issue</code> | <code>mcp__&lt;server&gt;__&lt;tool&gt;</code> |
| 正则要点   | <code>^mcp__</code>                  | 批量策略：所有 MCP 先审                                 |

## Mermaid：PreToolUse 决策流

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid：modify 决策与参数补丁

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：matcher 与 MCP（示意）

```
// tool-matcher.ts (示意)
export type ToolMatcher =
  | { type: 'exact'; name: string }
  | { type: 'prefix'; value: string }
  | { type: 'regex'; pattern: RegExp };

export function matchTool(m: ToolMatcher, name: string): boolean {
  switch (m.type) {
    case 'exact':
      return name === m.name;
    case 'prefix':
      return name.startsWith(m.value);
    case 'regex':
      return m.pattern.test(name);
  }
}

// MCP 工具名例: mcp__notion__search
export const MCP_TOOL_PATTERN = /^mcp__[^\_]+\_[^\_]+$/;
```

```
// pre-tool-use-decision.ts (示意)
export type HookDecision =
  | { type: 'allow' }
  | { type: 'block'; reason: string }
  | { type: 'modify'; name?: string; args?: Record<string, unknown> };

export function mergeDecisions(
  acc: HookDecision,
  next: HookDecision
): HookDecision {
  if (acc.type === 'block' || next.type === 'block') {
    return next.type === 'block' ? next : acc;
  }
  if (next.type === 'modify') {
    return {
      type: 'modify',
      name: next.name ?? (acc.type === 'modify' ? acc.name : undefined),
      args: {
        ...(acc.type === 'modify' ? acc.args : {}),
        ...next.args,
      },
    };
  }
  return acc;
}
```

```
// example-hook.json (示意配置)
{
  "event": "PreToolUse",
  "matcher": { "type": "prefix", "value": "mcp__" },
  "callback": {
    "kind": "shell",
    "command": "./bin/audit-mcp.sh"
  },
  "priority": 10
}
```

## block vs PermissionRequest

| 维度   | PreToolUse block | PermissionRequest Hook |
|------|------------------|------------------------|
| 时机   | 工具调用已生成          | UI/系统即将向用户要授权          |
| 用户感知 | 可能直接失败           | 可出现对话框                 |
| 适合   | 硬规则（黑名单）         | 策略引擎（风险评分）             |

## Shell / HTTP / LLM 三种 callback 权衡

| 类型    | 优点        | 风险           |
|-------|-----------|--------------|
| Shell | 与现有脚本生态集成 | 注入、超时、可移植性   |
| HTTP  | 中心化策略服务   | 网络故障、SSRF 防护 |
| LLM   | 灵活语义判断    | 成本、延迟、非确定性   |

## 合并策略与优先级

| 规则                       | 说明                                           |
|--------------------------|----------------------------------------------|
| <code>priority</code> 数值 | 通常 <b>越小越先</b> 或相反——以具体实现为准，本节强调 <b>存在全序</b> |
| <code>block</code> 吞噬    | 一旦出现 block，后续 allow 无效                       |
| <code>modify</code> 叠加   | 多 Hook 依次 patch，需注意键冲突                       |

## 测试矩阵（节选）

| 用例 | 输入工具名                                   | 期望                   |
|----|-----------------------------------------|----------------------|
| A  | <code>Bash</code> + <code>rm -rf</code> | block                |
| B  | <code>mcp_slack_post</code>             | modify 去掉 channel id |
| C  | <code>FileRead</code> 路径穿越              | block 或 modify 规范化   |
| D  | 无 matcher Hook                          | 应跳过 callback         |

## 性能与超时

| 实践   | 建议                                   |
|------|--------------------------------------|
| 回调超时 | 200—2000ms 分级                        |
| 并发   | 同事件 Hook 可否并行？需读实现                   |
| 缓存   | 对重复 <code>FileRead</code> 路径策略可做 LRU |

## 常见问题 FAQ

| 问题             | 回答方向                             |
|----------------|----------------------------------|
| modify 能换工具名吗？ | 可以，相当于重定向；需防范循环。                 |
| MCP 名大小写？      | 以注册为准；matcher 建议 case-sensitive。 |
| 与 Skills 冲突？   | Skills 偏提示；PreToolUse 硬治理。       |

## 小结

- PreToolUse 是 block / allow / modify 三态决策器，位于工具真正执行之前。
- MCP 工具名 `mcp_server_tool` 是编写跨服务器策略的关键把手。
- 管线记住八字：收集 → 过滤 → 执行 → 合并。

## 课后自测

1. 写一条正则：匹配任意 MCP 工具但排除 `mcp__internal__debug`。
2. 解释为何 `block` 应优先于 `modify` 合并。
3. 设计一个 HTTP callback 的 JSON schema（请求/响应字段）。

上一节： [16.1 总览](#)

下一节： [16.3 Skills](#)

## 第十六部分 · 16.3 Skills — Markdown 封装的轻量工作流

导航： [← 16.2 PreToolUse](#) · [16.4 Plugins](#) →

### 学习目标

完成本节学习后，你应该能够：

1. 定义 Skill：以 **Markdown** 为载体、带元数据（YAML frontmatter）的可发现工作流片段。
2. 解释 `allowed-tools` 声明的意图：在匹配场景下**约束/强烈提示**模型可调用工具集合，与平台策略求交。
3. 描述 系统如何在「模型匹配到某场景」时注入 Skill 正文到上下文，并**强制遵循流程**（enforcement 程度因版本而异）。
4. 区分 Skill 与系统提示、项目 `AGENTS.md`、Plugin 命令文档的边界。

### 生活类比：后厨标准作业程序（SOP）卡片

把 Skill 想象成餐厅墙上塑封的 SOP 卡片：

- 每张卡片（一个 `.md` 文件）写明：**何时启用**（frontmatter 里的描述/标签）、**允许用哪些厨具**（`allowed-tools`）、**分步操作**（正文 Markdown）。

- 主厨（模型）在接到「海鲜过敏客诉」这类信号时，应翻牌执行对应 SOP，而不是即兴发挥。
- SOP 不是把整个厨房搬走（那是 Plugin），也不是监控摄像头（那是 Hook）。

## Skill 文件结构（示意）

```
---
name: database-migration-checklist
description: 在检测到 SQL 迁移相关请求时启用
allowed-tools: FileRead, Grep, Bash
version: 1
---

## 步骤

1. 阅读 `migrations/` 下最新文件。
2. 搜索破坏性变更关键字 `DROP`, `TRUNCATE`。
3. 生成回滚建议段落。
```

## frontmatter 字段参考表

| 字段            | 必填 | 说明            |
|---------------|----|---------------|
| name          | 建议 | 稳定标识符         |
| description   | 建议 | 触发语义，供检索与模型路由 |
| allowed-tools | 可选 | 逗号分隔工具列表      |
| version       | 可选 | 便于迁移与缓存失效     |

## Mermaid：Skill 注入时机

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid: allowed-tools 与系统强制

[Mermaid 图表 — 请在在线版本中查看]

### 源码片段: Skill 加载 (示意)

```
// skill-types.ts (示意)
export interface SkillFrontmatter {
  name: string;
  description: string;
  allowedTools?: string[];
  version?: string;
}

export interface LoadedSkill {
  id: string;
  frontmatter: SkillFrontmatter;
  bodyMarkdown: string;
}
```

```
// skill-matcher.ts (示意)
export function selectSkills(
  userTurn: string,
  catalog: LoadedSkill[],
  topK: number
): LoadedSkill[] {
  const scored = catalog.map((s) => ({
    skill: s,
    score: semanticScore(userTurn, s.frontmatter.description),
  }));
  return scored
    .filter((x) => x.score > 0.35)
    .sort((a, b) => b.score - a.score)
    .slice(0, topK)
    .map((x) => x.skill);
}
```

```
// tool-gate.ts (示意)
export function effectiveTools(
  globalAllow: Set<string>,
  skill?: LoadedSkill
): Set<string> {
  if (!skill?.frontmatter.allowedTools?.length) return globalAllow;
  const skillSet = new Set(skill.frontmatter.allowedTools);
  return new Set([...globalAllow].filter((t) => skillSet.has(t)));
}
```



## 「系统强制模型匹配场景必须调用」的含义

| 层级  | 行为                                  |
|-----|-------------------------------------|
| 软提示 | 仅把 Skill 文本当作建议                     |
| 硬门控 | 工具选择器过滤非 <code>allowed-tools</code> |
| 审计  | 若模型跳过 Skill，记录 telemetry            |

具体产品落在哪一档，应以你所用版本的实现与文档为准；本指南强调设计空间。

## Skills 与 Hooks 协作

| 场景                     | Hook           | Skill    |
|------------------------|----------------|----------|
| 拦截非法 <code>Bash</code> | PreToolUse     | 提供合规替代步骤 |
| 数据外泄                   | block          | 正文写明脱敏清单 |
| 长流程一致性                 | PostToolUse 记录 | 步骤模板     |

## 组织治理建议

| 实践   | 说明                                      |
|------|-----------------------------------------|
| 单一职责 | 一 Skill 一主题，避免万行 Markdown               |
| 可测试  | 给 <code>description</code> 写触发样例句       |
| 版本化  | <code>version</code> + changelog        |
| 评审   | PR 模板检查 <code>allowed-tools</code> 是否过宽 |

## 反模式表

| 反模式                              | 后果         |
|----------------------------------|------------|
| <code>allowed-tools: *</code> 语义 | 失去约束意义     |
| 复制粘贴旧流程                          | 模型上下文膨胀    |
| 无 matcher 关键词                    | Skill 永不触发 |

## 与 Plugins 的边界（预告）

| 对比    | Skills              | Plugins        |
|-------|---------------------|----------------|
| 交付形态  | 多数 <code>.md</code> | 目录包 + manifest |
| 命令扩展  | 少                   | 多（斜杠命令）        |
| 运行时成本 | 低                   | 中高             |

## 常见问题 FAQ

| 问题                 | 回答方向                  |
|--------------------|-----------------------|
| Skill 能访问密钥吗？      | 不应明文；走环境/密钥管理。        |
| 多 Skill 同时命中？      | 需 merge 策略与 token 预算。 |
| 与 Cursor Rules 关系？ | 同属「提示治理」，载体不同。        |

## 小结

- Skills = Markdown + frontmatter，主打轻量、可检索、可组合工作流。
- `allowed-tools` 是连接「流程描述」与「工具沙箱」的桥梁，常与全局策略取交。
- 与 PreToolUse 叠加可形成软+硬双层治理。

## 课后自测

1. 为一个「安全依赖升级」主题起草 frontmatter（含 `allowed-tools` 列表）。

2. 解释 `effectiveTools = global n skill` 在空 `allowed-tools` 时应如何退化。
3. 列举两个应放在 Hook 而非 Skill 的策略例子。

上一节: [16.2 PreToolUse](#)

下一节: [16.4 Plugins](#)

## 第十六部分 · 16.4 Plugins — 重型扩展：命令、目录与硬性标记

导航: [← 16.3 Skills](#) · [16.5 MCP 注入](#) [→](#)

### 学习目标

完成本节学习后，你应该能够：

1. 定义 Plugin：以独立目录与 `manifest` 组织的扩展包，可注册自定义命令、静态资源与配置片段。
2. 解释 `disable-model-invocation`（或等价标记）的用途：禁止模型自动调用某些路径/工具，forcing 显式用户触发。
3. 对比 Plugin 与 Skill 在交付物、生命周期、权限面上的差异。
4. 列举 插件开发中常见的工程约束：名称空间、版本兼容、与 MCP 工具名碰撞处理。

### 生活类比：机场里的「品牌专柜」

- Skill 像机上杂志里的服务流程页——轻、可复制、贴在座椅后背。
- Plugin 像航站楼里占地几十平米的免税店：有独立装修（目录）、专属店员培训（命令实现）、合同条款（manifest 里的硬性标记），甚至规定「某些高价值商品只能人工收银」（`disable-model-invocation`）。

## Plugin 目录结构（示意）

```
my-plugin/  
  plugin.json      # manifest  
  commands/  
    deploy.md      # 斜杠命令文档 + 参数 schema  
  hooks/           # 可选：随包分发 Hook 片段  
  resources/  
    templates/...
```

## manifest 字段教学表

| 字段                     | 说明             |
|------------------------|----------------|
| name                   | 插件唯一标识         |
| version                | SemVer         |
| commands               | 命令注册表          |
| permissions            | 额外声明的网络/文件范围   |
| disableModelInvocation | 硬性关闭模型自动调用特定能力 |

实际键名以你所用平台的 schema 为准；本节用 camelCase 示意。

## Mermaid：Plugin 生命周期

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid：disable-model-invocation 语义

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：manifest 类型（示意）

```
// plugin-manifest.ts (示意)
export interface PluginCommand {
  id: string;
  title: string;
  description: string;
  entry: string; // markdown 或 handler 路径
}

export interface PluginManifest {
  name: string;
  version: string;
  commands: PluginCommand[];
  disableModelInvocationFor?: string[]; // tool/command ids
}
```

```
// plugin-loader.ts (示意)
export function loadPlugin(root: string): LoadedPlugin {
  const manifest = readJson<PluginManifest>(path.join(root, 'plugin.json'));
  validateManifestOrThrow(manifest);
  const commands = manifest.commands.map((c) =>
    loadCommandDoc(path.join(root, c.entry))
  );
  return { manifest, commands, root };
}
```

```
// tool-gate-integration.ts (示意)
export function isModelInvocationAllowed(
  toolName: string,
  plugin: LoadedPlugin
): boolean {
  const blocked = plugin.manifest.disableModelInvocationFor ?? [];
  return !blocked.includes(toolName);
}
```

## 自定义命令与 Slash（衔接 16.6）

| 维度 | 说明                                     |
|----|----------------------------------------|
| 注册 | manifest <code>commands[]</code>       |
| 解析 | 用户输入 <code>/deploy staging</code>      |
| 文档 | <code>commands/deploy.md</code> 内嵌参数说明 |

详见 [16.6](#)。

## 与 Hooks / Skills 的组合矩阵

| 组合                  | 用途              |
|---------------------|-----------------|
| Plugin + PreToolUse | 随包分发安全策略        |
| Plugin + Skill      | 插件内置 SOP 模板     |
| Plugin + MCP        | 插件启动时注册 MCP 子进程 |

## 名称空间与冲突

| 冲突类型     | 缓解                                     |
|----------|----------------------------------------|
| 工具名与 MCP | 前缀 <code>plugin__myplugin__tool</code> |
| 命令与内置斜杠  | manifest 校验拒绝保留字                       |
| 环境变量     | 使用 <code>PLUGIN_MYPLUGIN_*</code>      |

## 测试与发布

| 阶段 | 建议                                 |
|----|------------------------------------|
| 本地 | <code>plugin.json</code> schema 测试 |
| CI | 打包体积与许可证扫描                         |
| 发布 | SemVer + changelog                 |

## 安全表

| 风险            | 缓解                    |
|---------------|-----------------------|
| 恶意 Hook       | 仅安装可信来源；PreToolUse 审计 |
| 过度权限 manifest | 代码评审 + 最小权限           |
| 供应链           | 锁定版本与哈希               |

## 常见问题 FAQ

| 问题               | 回答方向                     |
|------------------|--------------------------|
| Plugin 能改核心二进制吗？ | 通常不能；扩展在沙箱 API 内。        |
| 禁用模型调用后 UX？      | 应用斜杠/按钮引导。               |
| 与 monorepo 兼容？   | 本地路径安装 + workspace 相对引用。 |

## 小结

- Plugins 提供 manifest + 目录 + 命令 的重型扩展面。
- `disable-model-invocation` 把高危能力从「模型随手可调」降级为「用户显式」。
- 与 Skills、Hooks、MCP 可叠加，需注意名称空间与策略合并顺序。

## 课后自测

1. 设计一个插件命令：仅允许人工触发生产数据库只读查询，并标注 manifest 字段。
2. 解释为何「禁用模型调用」不等于「禁用工具存在」。
3. 列出插件加载失败时应有的三级降级策略。

上一节：[16.3 Skills](#)

下一节：[16.5 MCP 动态指令注入](#)

# 第十六部分 · 16.5 MCP 动态指令注入 — instructions 与系统提示词拼接

导航：← [16.4 Plugins](#) · [16.6 自定义命令](#) →

## 学习目标

完成本节学习后，你应该能够：

1. 解释 MCP（Model Context Protocol）设备连接后，其 `instructions` 如何作为动态文本进入系统提示词合成管线。
2. 区分 静态系统提示、项目级 `AGENTS.md`、Skill 注入与 MCP instructions 的时序与优先级（概念级）。
3. 识别 动态注入带来的风险：提示注入、工具描述膨胀、多 MCP 源冲突。
4. 列举 缓解策略：摘要、分层、Hook 审计、defer 加载（见 16.7）。

## 生活类比：外聘专家的「当天简报」

把 MCP 服务器想象成按项目聘请的外部专家：

- 专家进会议室（连接成功）时，会递上一份当天简报（`instructions`）：「我今天带的仪器能测什么、不能测什么、报告格式要求」。
- 会议主席（系统提示合成器）把简报夹进总议程（系统提示词），让所有人（模型）知道边界。
- 若三位专家同时发言且彼此矛盾，主席需要合并规则或标注优先级——这就是多 MCP 注入的工程难题。

## 数据流要素表

| 要素                        | 来源                      | 去向          |
|---------------------------|-------------------------|-------------|
| MCP 元数据                   | 握手/清单                   | 运行时注册表      |
| <code>instructions</code> | 服务器声明                   | 系统提示片段      |
| 工具 schema                 | <code>tools/list</code> | 工具选择器 + 描述块 |
| 资源模板                      | 可选                      | 上下文引用       |



## Mermaid：连接与注入时序

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid：多 MCP 片段合并

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：合成器（示意）

```
// system-prompt-composer.ts (示意)
export function composeSystemPrompt(ctx: {
  base: string;
  projectRules: string;
  skills: string[];
  mcp: { serverId: string; instructions: string }[];
}): string {
  const mcpBlock = ctx.mcp
    .map(
      (m) =>
        `## MCP Server: ${m.serverId}\n${trimInstructions(m.instructions)}`
    )
    .join('\n\n');
  return [ctx.base, ctx.projectRules, ...ctx.skills, mcpBlock]
    .filter(Boolean)
    .join('\n\n---\n\n');
}
```

```
// mcp-session.ts (示意)
export class McpSession {
  constructor(
    public readonly serverId: string,
    private readonly conn: McpConnection
  ) {}

  async hydrateIntoContext(composer: SystemPromptComposer) {
    const meta = await this.conn.getServerMeta();
    composer.appendMcpInstructions({
      serverId: this.serverId,
      instructions: meta.instructions ?? '',
    });
    await this.registerTools(meta.tools);
  }
}
```

```
// token-budget.ts (示意)
export function trimInstructions(text: string, maxChars: number): string {
  if (text.length <= maxChars) return text;
  return text.slice(0, maxChars) + '\n...[truncated]';
}
```

## 与 PreToolUse 的协同

| 场景                          | 说明               |
|-----------------------------|------------------|
| instructions 诱导调用恶意工具       | PreToolUse 以策略覆盖 |
| 工具名 <code>mcp_*</code> 批量审计 | matcher 前缀       |
| 敏感数据经 MCP 返回                | PostToolUse 脱敏   |

## 提示注入风险表

| 攻击面                   | 示例           | 缓解           |
|-----------------------|--------------|--------------|
| instructions 内含「忽略上文」 | 经典注入         | 分隔符 + 模型安全训练 |
| 工具 description 过长     | 挤压用户消息       | 摘要与 defer    |
| 多源互相覆盖                | A 说可删库 B 说不行 | 显式优先级段落      |

## Token 经济学

| 组件                 | 影响             |
|--------------------|----------------|
| 每 MCP instructions | +N tokens / 请求 |
| 工具 schema          | 更大头；可延迟加载      |
| Skills             | 与 MCP 共享预算     |

## 可观测性

| 指标                       | 用途   |
|--------------------------|------|
| mcp_instruction_chars    | 预算调优 |
| mcp_connect_failures     | 可靠性  |
| tool_register_latency_ms | 启动性能 |

## 版本与缓存

| 实践        | 说明     |
|-----------|--------|
| 服务器版本变更   | 失效合成缓存 |
| 工具列表 etag | 减少重复拉取 |

## 常见问题 FAQ

| 问题                    | 回答方向                 |
|-----------------------|----------------------|
| instructions 可热更新吗？   | 视客户端是否支持重连/刷新。       |
| 无 instructions 的 MCP？ | 仅用工具 schema 片段。      |
| 与企业代理冲突？              | TLS 与 allowlist 需预配。 |

## 小结

- MCP 动态指令注入 = 连接后将 `instructions` 拼进系统提示词。
- 多 MCP 需要合并策略 + Token 预算 + 安全审计。
- 与 Hooks、`defer_loading` 形成「能连、能控、能省」三角。

## 课后自测

1. 手写一个三段式 system 模板：`BASE`、`PROJECT`、`MCP[{{id}}]`。
2. 解释为何工具 schema 往往比 instructions 更耗 Token。
3. 设计一个 `PostToolUse` 事件：检测响应里是否包含疑似注入短语。

## 术语表（中英）

| 中文    | 英文                         | 备注                    |
|-------|----------------------------|-----------------------|
| 系统提示词 | system prompt              | 与 user/assistant 消息并列 |
| 清单    | manifest / capability list | MCP 握手返回              |
| 截断    | truncation                 | 需配合「已截断」提示防模型误判       |
| 合成器   | composer                   | 负责拼接各片段的纯函数或服务        |

## 与 defer\_loading 的衔接说明

当 `instructions` 较短而 `tool schema` 极大时，常见架构是：

1. 首包仅把 `instructions` 与工具名称列表送入 system；
2. 具体 JSON Schema 在 `ensureHydrated(toolName)` 时加载（见 [16.7](#)）；
3. `PostToolUse` 仍可对「已加载工具集」做审计，无需等待全量 schema。

该拆分能显著降低「连接 10 个 MCP」时的冷启动 Token。

上一节：[16.4 Plugins](#)

下一节：[16.6 自定义斜杠命令](#)

## 第十六部分 · 16.6 自定义命令 — Slash 命令开发

导航: [← 16.5 MCP 注入](#) · [16.7 defer loading →](#)

### 学习目标

完成本节学习后，你应该能够：

1. **定义** 斜杠（Slash）命令：以 `/` 前缀触发的用户意图快捷方式，映射到**固定提示模板**、脚本或 **Plugin 处理器**。
2. **描述** 自定义命令从**注册**（manifest / 配置）到**解析**（参数分词）再到**执行**（注入对话上下文）的链路。
3. **列举** 良好命令设计的要素：帮助文本、参数 schema、示例、权限声明。
4. **关联** [16.4 Plugins](#) 中的 `commands/` 目录约定。

### 生活类比：快餐编号点餐

把 Slash 命令想象成快餐店的**编号餐**：

- 顾客不说长句，只喊 `/12 套餐` ——收银机（解析器）知道对应**固定配方**（模板）与**可加选项**（参数）。
- 自定义命令就是门店总部允许加盟店在收银机里**新增编号**，但必须附带**价目表说明**（help）与**过敏原提示**（权限/风险）。

### 命令形态示例

| 输入                           | 语义            |
|------------------------------|---------------|
| <code>/review</code>         | 无参，触发代码审查 SOP |
| <code>/deploy staging</code> | 位置参数          |
| <code>/sql --dry-run</code>  | 标志位           |

## Mermaid: Slash 解析管线

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid: Plugin 命令与内置命令共存

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：命令定义（示意）

```
// slash-command.ts (示意)
export interface SlashCommand {
  name: string; // 不含前导 /
  description: string;
  usage: string;
  handler:
    | { type: 'template'; path: string }
    | { type: 'script'; bin: string }
    | { type: 'delegate'; plugin: string };
}
```

```
// parser.ts (示意)
export function parseSlashInput(line: string): ParsedSlash | null {
  if (!line.startsWith('/')) return null;
  const tokens = line.slice(1).trim().split(/\s+/);
  const name = tokens.shift() ?? '';
  return { name, args: tokens };
}
```

```
<!-- commands/review.md (示意) -->
---
command: review
description: 对当前 git diff 做结构化审查
---

请按以下结构输出：

1. 变更摘要
2. 风险等级（低/中/高）
3. 测试建议
4. 可改进命名

参数: {{args}}
```

## manifest 注册（衔接 Plugin）

```
{
  "commands": [
    {
      "id": "review",
      "title": "Code Review",
      "description": "结构化 diff 审查",
      "entry": "commands/review.md"
    }
  ]
}
```

## 参数 schema 建议

| 字段                      | 类型                    | 说明                       |
|-------------------------|-----------------------|--------------------------|
| <code>positional</code> | <code>string[]</code> | 按顺序                      |
| <code>flags</code>      | <code>record</code>   | <code>--key value</code> |
| <code>env</code>        | <code>string[]</code> | 需要的环境变量                  |

可用 JSON Schema 生成 IDE 补全（视平台支持）。

## 与 `disable-model-invocation` 联动

| 场景                           | 做法                   |
|------------------------------|----------------------|
| 高危 <code>/deploy prod</code> | 仅允许 slash，不允许模型自拟工具链 |
| 只读 <code>/audit</code>       | 模型可调用只读工具            |

## UX 清单

| 项                       | 说明                                   |
|-------------------------|--------------------------------------|
| <code>/help</code> 列出命令 | 可发现性                                 |
| 错误信息                    | 未知命令给建议                              |
| 别名                      | <code>r</code> → <code>review</code> |

## 测试表

| 用例 | 输入                     | 期望           |
|----|------------------------|--------------|
| T1 | <code>/review</code>   | 装载 review 模板 |
| T2 | <code>/unknown</code>  | 友好错误         |
| T3 | <code>/sql ; rm</code> | 被安全层拒绝或转义    |

## 国际化

| 话题   | 建议          |
|------|-------------|
| 命令名  | 保持 ASCII 稳定 |
| 帮助文本 | 可本地化文件      |



## 常见问题 FAQ

| 问题                 | 回答方向                                  |
|--------------------|---------------------------------------|
| Slash 与 @ 文件引用优先级？ | 平台相关；通常先解析 slash。                     |
| 命令能嵌套吗？            | 一般不建议；用子命令 <code>/git commit</code> 。 |
| 与 MCP 工具重名？        | 命名空间隔离；文档说明。                          |

## 小结

- 自定义 Slash 命令把重复 workflows 压缩为可注册、可文档化的入口。
- Plugin 通过 manifest + markdown 模板扩展命令表。
- 与 权限、PreToolUse、disable-model-invocation 组合可构造安全的人工闸。

## 课后自测

1. 为 `/migrate-db` 起草 `usage` 字符串与两个示例调用。
2. 写伪代码：在 `parseSlashInput` 后把结果转为「合成用户消息」。
3. 解释为何命令帮助应独立于模型自由文本。

## 合成用户消息（示意伪代码）

下列片段展示如何把 Slash 解析结果映射为结构化用户消息，便于在 transcript 中区分「显式命令」与自由输入：

```
function slashToSyntheticUserMessage(parsed: ParsedSlash, template: string) {
  const filled = template.replace('{{args}}', parsed.args.join(' '));
  return {
    role: 'user' as const,
    content: filled,
    metadata: { kind: 'slash', name: parsed.name },
  };
}
```

## 保留字与命名空间（建议表）

| 类别 | 示例                                       | 策略          |
|----|------------------------------------------|-------------|
| 内置 | <code>/help</code> , <code>/clear</code> | 插件不可覆盖      |
| 组织 | <code>/corp-*</code>                     | 内部插件统一前缀    |
| 实验 | <code>/x-*</code>                        | 随时可删，不入正式文档 |

上一节: [16.5 MCP 注入](#)

下一节: [16.7 defer loading](#)

# 第十六部分 · 16.7 defer\_loading — 工具延迟加载与缓存保护

导航: [← 16.6 自定义命令](#) · [16.8 综合演练 →](#)

## 学习目标

完成本节学习后，你应该能够：

1. 解释 `defer_loading`（或等价机制）在工具注册中的目标：**推迟**重型工具定义的物化，直到**首次需要或显式预热**。
2. 描述 延迟加载如何**保护缓存**：避免每次会话启动**全量失效**与**重复解析**大 schema（尤其 MCP）。
3. 对比 eager load 与 deferred load 在**首响延迟**、**内存占用**、**调试复杂度**上的权衡。
4. 关联 [16.5 MCP 注入](#) 中的 schema 体积问题。

## 生活类比：图书馆闭架借书

- Eager loading像开业时把**所有书**都摊在大厅——找得快一眼看完，但搬书累死（启动慢）、灰尘多（缓存抖动）。

- **Defer loading**像闭架书库：目录卡片在前台（轻量索引），真有人借某类书时，管理员才去库房取（**按需加载**），并且同一本书当天只取一次（**缓存命中**）。

## 核心概念表

| 概念         | 说明                       |
|------------|--------------------------|
| 工具 stub    | 仅含名称与轻描述，占位              |
| hydrate    | 首次调用前拉全 schema           |
| cache key  | 通常含插件版本、MCP 会话 id、工具列表哈希 |
| invalidate | 连接变更、manifest 更新         |

## Mermaid：延迟加载状态机

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid：启动路径对比

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：注册与惰性解析（示意）

```
// tool-registry.ts (示意)
export interface ToolRegistration {
  name: string;
  kind: 'builtin' | 'mcp';
  loader?: () => Promise<ToolSchema>;
  schema?: ToolSchema; // hydrated
}

export class ToolRegistry {
  private tools = new Map<string, ToolRegistration>();

  registerDeferred(t: ToolRegistration) {
    this.tools.set(t.name, { ...t, schema: undefined });
  }

  async ensureHydrated(name: string): Promise<ToolSchema> {
    const reg = this.tools.get(name);
    if (!reg) throw new Error('unknown tool');
    if (reg.schema) return reg.schema;
    if (!reg.loader) throw new Error('no loader');
    reg.schema = await reg.loader();
    return reg.schema;
  }
}
```

```
// cache-layer.ts (示意)
export function makeCacheKey(parts: {
  mcpServerId: string;
  toolListHash: string;
}): string {
  return `mcp:${parts.mcpServerId}:tools:${parts.toolListHash}`;
}

export async function getOrLoad<T>({
  key: string,
  loader: () => Promise<T>,
  ttlMs: number
}): Promise<T> {
  const hit = lruGet<T>(key);
  if (hit && !isStale(hit, ttlMs)) return hit.value;
  const value = await loader();
  lruSet(key, { value, fetchedAt: Date.now() });
  return value;
}
```

```
// planner-integration.ts (示意)
export async function planToolCall(
  registry: ToolRegistry,
  call: { name: string; args: unknown }
) {
  const schema = await registry.ensureHydrated(call.name);
  validateArgs(schema, call.args);
  return execute(call);
}
```

## 缓存保护含义

| 威胁             | defer 的缓解       |
|----------------|-----------------|
| 每次全量 schema 下载 | 仅 hydrate 用到的子集 |
| 大对象 JSON parse | 分摊到多次 idle      |
| 冷启动阻塞 UI       | 先渲染会话，再后台预热     |

## 预热策略

| 策略   | 描述                             |
|------|--------------------------------|
| 用户显式 | <code>/tools warmup</code>     |
| 启发式  | 根据最近历史预测下一工具                   |
| 空闲时  | requestIdleCallback 批量 hydrate |

## 与 Hooks 的交互

| 事件                        | 说明                      |
|---------------------------|-------------------------|
| <code>SessionStart</code> | 可触发 selective preload   |
| <code>PreToolUse</code>   | hydrate 若失败 → block 并提示 |

## 调试清单

| 步骤 | 动作                           |
|----|------------------------------|
| 1  | 日志记录 <code>hydrate_ms</code> |
| 2  | 对比 eager vs defer 的 P95 启动   |
| 3  | 压测 20+ MCP 同时连接              |

## 反模式

| 反模式             | 后果        |
|-----------------|-----------|
| 每次调用都重新拉 schema | 延迟爆炸      |
| cache key 过粗    | 陈旧 schema |
| cache key 过细    | 命中率趋近 0   |

## 常见问题 FAQ

| 问题             | 回答方向               |
|----------------|--------------------|
| defer 影响确定性吗？  | 不应；仅影响何时加载。        |
| 与流式首 token 关系？ | 先返回对话，再后台 hydrate。 |
| 离线模式？          | stub 需含降级错误文案。     |

## 小结

- `defer_loading` 通过 `stub` + 按需 `hydrate` 降低冷启动成本。
- 缓存 `key` 设计决定新鲜度 vs 命中平衡。
- 与 MCP 大 `schema`、多插件场景强相关。

## 课后自测

- 为 `ensureHydrated` 添加并发合并 (coalescing) 伪代码：同一工具并发请求只触发一次 loader。
- 计算：100 个工具每个 schema 8KB，全量 eager 与仅 5 个 defer hydrate 的内存差（量级）。
- 设计缓存失效：MCP `tools/list` 变更时的监听点。

## 并发合并 (coalescing) 示意

```
const inflight = new Map<string, Promise<ToolSchema>>();

async function ensureHydratedCoalesced(
  name: string,
  loader: () => Promise<ToolSchema>
): Promise<ToolSchema> {
  const existing = inflight.get(name);
  if (existing) return existing;
  const p = loader().finally(() => inflight.delete(name));
  inflight.set(name, p);
  return p;
}
```

同一工具在极短窗口内的重复请求将共享 单次 `loader()`，避免惊群与重复网络 I/O。

## 监控面板建议字段

| 字段                                 | 类型        | 说明             |
|------------------------------------|-----------|----------------|
| <code>defer_hydrate_ms</code>      | histogram | hydrate 耗时分布   |
| <code>defer_cache_hit_ratio</code> | gauge     | 命中比例           |
| <code>defer_inflight</code>        | gauge     | 当前并发 hydrate 数 |

上一节：[16.6 自定义命令](#)

下一节：[16.8 综合演练](#)

## 第十六部分 · 16.8 综合演练 — 从 Hooks 到 defer 的端到端蓝图

导航: [← 16.7 defer\\_loading](#) · [返回 16.1](#)

### 学习目标

完成本节学习后，你应该能够：

1. **整合** 本部分知识点：在同一虚构项目中同时配置 `PreToolUse`、`Skill`、`Plugin` 斜杠命令、`MCP instructions` 与 `defer` 工具加载。
2. **演练** 一次完整会话：`SessionStart` → 用户提交 → `MCP` 连接 → `Skill` 命中 → 模型发起 `MCP` 工具 → `PreToolUse` 审计 → `PostToolUse` 记录 → `Stop` 清理。
3. **输出** 可交付团队的检查清单与回滚策略。
4. **复盘** `Token`、延迟、安全三者的权衡决策。

### 生活类比：开一家「受控自动化」咖啡店

你要开一家店：

- `MCP` 是外部原料供应商，每天送货时附当天品控说明（`instructions` 注入）。
- `Skill` 是员工手册里的「客诉处理 SOP」。
- `Plugin` `/refund` 是收银机上的退款专用键（自定义命令），规定只能店长按（`disable-model-invocation`）。
- `PreToolUse` 是店长在出杯前最后一眼：发现异常配方立刻 `block`。
- `defer_loading` 是库房按需取豆：早上不把 30 种豆子全倒上台面。



## 演练场景设定

| 要素  | 取值                                     |
|-----|----------------------------------------|
| 仓库  | <code>payments-service</code> monorepo |
| MCP | <code>mcp__jira__create_ticket</code>  |
| 风险  | 禁止模型自动建高优先级工单                          |
| 合规  | 所有 MCP 调用写审计日志                         |

## Mermaid：端到端序列（综合）

[Mermaid 图表 — 请在在线版本中查看]

## Mermaid：组件依赖图

[Mermaid 图表 — 请在在线版本中查看]

## 配置蓝图（示意 YAML）

```
hooks:
  - event: SessionStart
    callback: { kind: shell, command: ./scripts/bootstrap-audit.sh }
  - event: PreToolUse
    matcher: { type: prefix, value: 'mcp__jira__' }
    callback: { kind: shell, command: ./scripts/jira-guard.sh }
  - event: PostToolUse
    matcher: { type: prefix, value: 'mcp__' }
    callback: { kind: http, url: https://audit.internal/mcp }
  - event: Stop
    callback: { kind: shell, command: ./scripts/teardown-cache.sh }

skills:
  searchPaths: ['.claude/skills']

plugins:
  - path: ./plugins/payments-plugin

mcp:
  servers:
    - id: jira
      command: npx
      args: ['-y', '@example/mcp-jira']

tools:
  deferLoading: true
```

## Skill 摘要（示意）

```
---
name: payments-regression
description: 当讨论支付回归、对账、退款链路时启用
allowed-tools: FileRead, Grep, mcp__jira__create_ticket
---

## 流程
1. 定位相关模块 `payments/`.
2. 列出潜在边界条件。
3. 如需工单，仅使用 Jira MCP，并设置优先级为 Low，除非用户显式授权。
```

## Plugin 命令片段（示意）

```
---
command: triage
description: 人工触发高优先级工单流程（需审批）
disableModelInvocationFor:
  - mcp__jira__create_ticket
---
用户已显式运行 /triage。允许以下例外： .....
```

## PreToolUse 决策表（演练）

| 调用                                     | 决策    | 理由        |
|----------------------------------------|-------|-----------|
| mcp__jira__create_ticket priority=High | block | 无 /triage |
| 同上 priority=Low                        | allow | 符合 Skill  |
| Bash curl 外网                           | block | 数据外泄      |

## defer\_loading 演练步骤

| 步骤 | 操作           | 观察            |
|----|--------------|---------------|
| 1  | 冷启动会话        | 首屏时间          |
| 2  | 首次触发 Jira 工具 | hydrate_ms 峰值 |
| 3  | 第二次调用        | 命中缓存          |

## 回滚策略

| 层级 | 动作                              |
|----|---------------------------------|
| L1 | 关闭单个 Hook                       |
| L2 | 禁用插件 manifest                   |
| L3 | defer_loading: false 临时恢复 eager |
| L4 | 断开 MCP                          |

## Token 复盘表

| 片段                 | 粗估影响           |
|--------------------|----------------|
| MCP instructions   | +0.5—2k tokens |
| Skill 正文           | +0.2—1k        |
| 工具 schema（defer 后） | 首调前较小          |

## 团队分工 RACI

| 工作        | R    | A     | C   | I  |
|-----------|------|-------|-----|----|
| Hook 策略   | 安全   | 平台负责人 | SRE | 研发 |
| Skill 内容  | 业务架构 | 工程经理  | QA  | 全员 |
| Plugin 发布 | 工具链  | 发布经理  | 安全  | 用户 |

## 常见问题 FAQ

| 问题     | 回答方向                |
|--------|---------------------|
| 演练环境？  | 独立 staging + 假 Jira |
| 失败重试？  | PostToolUse 应幂等     |
| 成本谁买单？ | MCP + LLM 双计费       |

## 交付物清单

| 交付物     | 格式                                   |
|---------|--------------------------------------|
| Hook 脚本 | <code>scripts/*.sh</code> + 单元测试     |
| Skill 库 | <code>.claude/skills/*.md</code>     |
| Plugin  | <code>plugins/.../plugin.json</code> |
| 运行手册    | 本节 + 内部 wiki                         |

## 小结

- 本演练把 Hooks / Skills / Plugins / MCP / defer 穿成一条可审计链路。
- PreToolUse 是安全闸，Skill 是流程，Slash 是人工钥匙，defer 是性能垫。
- 回滚分层与 RACI 让实验可进可退。

## 课后自测（大作业）

1. 将示意 YAML 转为你团队真实路径，并列出三条验证命令。
2. 写一次「事故复盘」模板：当 PreToolUse 误 block 时如何快速旁路。
3. 估算全链路 P99：从用户发送到 MCP 首次返回。

## 本部分回顾（16.1–16.8）

| 节    | 关键词                              |
|------|----------------------------------|
| 16.1 | 六事件、总览、三件套                       |
| 16.2 | PreToolUse、MCP 匹配                |
| 16.3 | Skills、allowed-tools             |
| 16.4 | Plugins、disable-model-invocation |
| 16.5 | MCP instructions 注入              |
| 16.6 | Slash 命令                         |
| 16.7 | defer_loading、缓存                 |
| 16.8 | 综合演练                             |

上一节：[16.7 defer loading](#)

返回：[16.1 总览](#)

# 第17篇：性能与成本

## 第17篇 · 17.1 Token 经济学与定价心智模型

**本篇导读：**理解 Claude Code 背后的「按 Token 计费」逻辑，是把「用得爽」和「花得值」统一起来的第一步。本节建立定价表、成本直觉与决策框架。

### 学习目标

完成本节后，你应能够：

- 1. **复述** Anthropic API 在 Opus / Sonnet 档位下的输入、输出与缓存读写单价（美元 / 百万 Token）。
- 2. **解释** 为何「输出 Token」通常比「输入 Token」贵，以及这对长对话、长 diff 的影响。
- 3. **对比** 纯对话场景与「工具 + 上下文膨胀」场景下的成本结构差异。
- 4. **建立** 个人或团队的「Token 预算」心智：何时升级模型、何时缩短 system 提示。
- 5. **将** 定价数字与后续各节（缓存、懒加载、流式）形成连贯的优化叙事。

### 核心定价速览（教学用基准）

下列数字为教学场景下常用的公开档位示意（实际以 [Anthropic 官方定价页](#) 为准，会随产品与区域调整）。

| 项目                | Opus（示意）        | Sonnet（示意）                     |
|-------------------|-----------------|--------------------------------|
| 输入（Input）         | \$15 / 百万 Token | \$3 / 百万 Token                 |
| 输出（Output）        | \$75 / 百万 Token | \$15 / 百万 Token                |
| 缓存写入（Cache write） | 依产品档            | \$3.75 / 百万 Token（Sonnet 档示意）  |
| 缓存读取（Cache read）  | 依产品档            | \$0.375 / 百万 Token（Sonnet 档示意） |

### 生活类比：餐厅「座位费」与「点菜价」

- **输入 Token** 像你已坐在店里占用的座位时间：菜单、环境说明、你带进去的故事，都算「进场成本」。
- **输出 Token** 像厨师现做的一道道热菜：工序多、不可预测长度，所以**单价更高**。
- **缓存** 像提前炖好的一锅高汤：第一次写入贵一点，后面「舀一勺」便宜一个数量级。

[Mermaid 图表 — 请在在线版本中查看]

### 成本直觉：一次「典型请求」如何拆解

假设某次 Agent 回合向 API 发送：

- 系统提示 + 工具定义 + 历史 + 当前文件摘要：**200k 输入 Token**（含可缓存前缀）
- 模型返回：**8k 输出 Token**

#### Opus 档粗算（无缓存、全按输入计价）

| 科目 | 计算                              | 小计（美元）           |
|----|---------------------------------|------------------|
| 输入 | $200 \times (\$15 / 1\text{M})$ | \$3.00           |
| 输出 | $8 \times (\$75 / 1\text{M})$   | \$0.60           |
| 合计 |                                 | $\approx \$3.60$ |

同一请求若换 **Sonnet**（仍无缓存）：

| 科目 | 计算                             | 小计（美元）           |
|----|--------------------------------|------------------|
| 输入 | $200 \times (\$3 / 1\text{M})$ | \$0.60           |
| 输出 | $8 \times (\$15 / 1\text{M})$  | \$0.12           |
| 合计 |                                | $\approx \$0.72$ |

可见：**模型档位切换**对账单的影响往往是**数倍级**，而非百分之几。

## 源码片段（概念层）：客户端如何组装「可计费」载荷

真实 Claude Code 代码库会封装 HTTP 客户端与消息格式。下面用伪代码说明「哪些东西会吃掉输入 Token」：

```
// 概念示意：非生产代码
type TurnPayload = {
  system: string;           // 系统提示 → 输入 Token
  tools: ToolDefinition[]; // 每个工具 schema → 输入 Token
  messages: Message[];     // 多轮历史 → 输入 Token
  metadata: {
    model: "opus" | "sonnet";
    enablePromptCache: boolean;
  };
};

function estimateBillableInput(p: TurnPayload): number {
  const systemTokens = countTokens(p.system);
  const toolTokens = p.tools.reduce((s, t) => s + countTokens(JSON.stringify(t)), 0);
  const historyTokens = p.messages.reduce((s, m) => s + countTokens(m.content), 0);
  return systemTokens + toolTokens + historyTokens;
}
```

要点：

- 工具定义越长、工具越多，每轮「固定输入」越重。
- 把大段文档无差别塞进 system，是常见的「贵但未必更聪明」的做法。

## 决策树：何时值得用 Opus？

[Mermaid 图表 — 请在在线版本中查看]



## Token「膨胀」的五大来源

| 来源        | 说明          | 降费思路（预告）                 |
|-----------|-------------|--------------------------|
| 系统提示与规则   | 每次回合重复发送    | 稳定前缀 + Prompt 缓存（见 17.2） |
| 工具 Schema | JSON 模式长    | 懒加载工具（见 17.4）            |
| Git / 目录树 | 状态快照大       | 并行预取 + 裁剪展示（见 17.3）      |
| 模型输出      | diff / 解释冗长 | 流式体验 + 提示约束（见 17.7）      |
| UI 调试残留   | 日志灌进上下文     | 生产配置隔离（见第 18 篇）          |

## 与后续小节的映射

| 小节文件                                     | 主题         | 与 Token 经济学的关系              |
|------------------------------------------|------------|-----------------------------|
| <a href="#">02-prompt-caching.md</a>     | 提示缓存       | 直接降低重复输入与缓存读单价              |
| <a href="#">03-parallel-prefetch.md</a>  | 并行预取       | 减少「等待时间」≠ 直接减 Token，但改善 SLA |
| <a href="#">04-lazy-loading.md</a>       | 懒加载        | 削减每轮工具 schema 体积            |
| <a href="#">05-sub-agent-cache.md</a>    | 子 Agent 缓存 | 多进程共享前缀，提高缓存命中率             |
| <a href="#">06-render-performance.md</a> | React 性能   | 前端流畅，间接减少「人类重试」             |
| <a href="#">07-streaming-pipeline.md</a> | 流式管道       | 感知延迟下降，优化交互轮次               |
| <a href="#">08-cost-cheatsheet.md</a>    | 省钱速查       | 把本节数字落到场景表                  |

## 自测清单

- 同样 100k 输入 + 20k 输出，Opus 与 Sonnet 的费用比大约是多少倍？（用本节表估算）
- 为什么说「输出 Token」对「长解释 + 大 patch」更敏感？
- 缓存读取单价约为 Sonnet 正常输入的百分之几？（用表：\$0.375 vs \$3）

## 延伸阅读建议

- 官方定价与「prompt caching」说明（以最新文档为准）。
- 本篇 [08-cost-cheatsheet.md](#)：把抽象美元换算成「每天一杯咖啡」式直觉。

- 第 18 篇：遥测与生命周期——用数据验证你的降费策略是否生效。

## 术语表

| 术语      | 含义                          |
|---------|-----------------------------|
| Token   | 模型处理文本的最小计费单位，不等价于「字符」或「单词」 |
| 输入 / 输出 | 分别对应请求体中的提示与模型生成内容          |
| 前缀匹配    | 缓存键通常按字节级前缀匹配，稳定前缀越长、命中越高   |

## 小结

- Opus 输入 \$15/M、输出 \$75/M；Sonnet 输入 \$3/M、输出 \$15/M 是理解账单的数量级锚点。
- 缓存写 / 读（Sonnet 示意 \$3.75 / \$0.375）使「重复上下文」从成本中心变成优化杠杆。
- 降费不是单一技巧，而是 模型选择 × 上下文工程 × 缓存 × 工具加载 的乘积；后续小节将逐项拆开。

## 进阶：把「百万 Token」换算成日常单位

下列换算仅帮助建立直觉，真实消耗以仪表盘为准。

| 若你的项目...                              | 可近似理解为...                                   |
|---------------------------------------|---------------------------------------------|
| 每天 50 次 Sonnet 调用，平均每次 20k 输入 + 2k 输出 | 输入约 $\$3/M \times 1M = \$3/\text{天}$ 量级（示意） |
| 周报需要对比「切 Opus 的一周」                    | 用本节表 × 你的 Token 日志                          |

[Mermaid 图表 — 请在在线版本中查看]

## 常见误区

| 误区          | 实际情况               |
|-------------|--------------------|
| 「模型越贵越省时间」  | 若任务简单，贵模型会付出更高输出溢价 |
| 「缓存免费」      | 缓存写入仍计费；命中读才显著便宜   |
| 「缩短用户消息就够了」 | 系统提示与工具定义往往才是大头    |

## 团队协作：Token 预算表（模板）

| 角色   | 默认模型   | 日 Token 上限（示例） | 例外审批        |
|------|--------|----------------|-------------|
| 一线开发 | Sonnet | 5M 输入（示例）      | 架构评审用 Opus  |
| SRE  | Sonnet | 按值班周调整         | 事故复盘可加 Opus |
| 文档   | Sonnet | 低输出配额          | 长文生成走批处理    |

## 与 API 错误率的经济学

高延迟或失败导致的人类重试，在账单上表现为：

- 重复的输入前缀（若未命中缓存 → 全价输入）
- 重复的输出（若客户端丢弃半成品 → 输出浪费）

因此「稳定性」与「成本」同构；第 18 篇将从生命周期与遥测角度收紧这一环。

## 参考公式（背公式不如背数量级）

设输入单价  $P_i$ （\$/M）、输出  $P_o$ （\$/M），本轮输入 Token 为  $T_i$ 、输出  $T_k$ （千 Token 换算为 M 时需除以 1000）：

$$\text{Cost} \approx \frac{T_i}{10^6} P_i + \frac{T_o}{10^6} P_o$$

有缓存时，将  $T_i$  拆成「未命中部分」与「缓存读部分」，分别乘以对应单价即可（详见 17.2）。

### 课堂练习（书面）

- 1. 题 A：输入 120k、输出 15k，分别用 Opus 与 Sonnet 估算费用。
- 2. 题 B：说明为何「把 README 全文粘进 system」可能比「放进 RAG 片段」更贵。
- 3. 题 C：画一张草图（可用纸笔）描述你当前项目的「Token 从哪进、从哪出」。

### 附录：与 Claude Code 产品形态的对应（概念）

| 产品概念         | 经济学含义                   |
|--------------|-------------------------|
| Agent 多轮工具循环 | 每轮可能重复发送大段 system/tools |
| 子 Agent      | 若前缀一致，有利于缓存前缀匹配（见 17.5） |
| IDE 插件       | 前端性能影响「体感」与重试率（见 17.6）  |

下一节：[02-prompt-caching.md](#) — Prompt 缓存与命中率工程。

## 17.2 Prompt 缓存：从「每轮全价」到「前缀命中」

**本节焦点：** Prompt Caching 如何把重复出现的长前缀从「全价输入」变成「低价缓存读」，从而在长会话中实现数量级节省。

### 学习目标

- 1. 说明 Prompt 缓存的「前缀一致」原理与字节级匹配直觉。
- 2. 量化 无缓存与有缓存场景下，100 轮 Opus 会话的成本区间（教学用：\$50—100 vs \$10—19，约 80% 节省）。
- 3. 解释 缓存读取成本约为正常输入 10%（Sonnet：\$0.375 vs \$3 / 百万）意味着什么。
- 4. 列举 提高命中率的工程手段：稳定 system、工具顺序固定、避免无意义抖动。
- 5. 规避 反模式：在每轮前缀末尾追加时间戳、随机 UUID 等「破坏前缀」的做法。

## 生活类比：图书馆编目与「快速取书」

想象你在写博士论文，每天都要引用同一套基础参考书目 + 实验室安全守则（几十页）。

- **无缓存**：每天去图书馆，馆员把整套守则重新复印一遍再给你 —— 贵、慢。
- **有缓存**：第一天「登记入库」（缓存写，一次性较贵），之后只给你索书号条（缓存读，极便宜）。

模型侧的「前缀」就是那套永远不变的章节目录；你每轮新问的问题只是目录后面的新页。

## 核心数据（教学用）

| 场景                 | 粗算账单（示意）               | 相对          |
|--------------------|------------------------|-------------|
| 100 轮会话，无缓存，Opus 档 | 约 \$50—100             | 基准 100%     |
| 同会话，有缓存、高中命中率      | 约 \$10—19              | 约节省 80%     |
| Sonnet 缓存读 vs 正常输入 | \$0.375 / M vs \$3 / M | 读 ≈ 10% 输入价 |

注：真实数字取决于每轮输入输出 Token、前缀长度、是否跨模型失效等。此处强调数量级。

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段（概念）：标记可缓存块

Anthropic 系 API 常见模式是在消息结构中标注 `cache_control`。下面为教学伪代码：

```
// 教学伪代码：展示「前缀可缓存」思想
const stablePrefix = [
  {
    type: "text",
    text: SYSTEM_PROMPT + TOOL_SCHEMAS_FROZEN_ORDER,
    cache_control: { type: "ephemeral" }, // 示意：可缓存段
  },
];

const userTurn = {
  type: "text",
  text: userMessage, // 通常不放 cache_control，避免污染前缀键
};

async function sendTurn(history: Message[]) {
  return await anthropic.messages.create({
    model: "claude-3-5-sonnet-latest",
    system: stablePrefix,
    messages: history,
    tools: toolsPossiblyCached,
  });
}
```

要点：

- 可缓存段应尽可能长且稳定。
- 易变字段（时间、随机 id）应放在前缀之后，否则整段前缀失效。

## 命中率工程检查表

| 检查项          | 建议           | 反例                           |
|--------------|--------------|------------------------------|
| system 文本    | 版本化、可复用      | 每轮插入 <code>Date.now()</code> |
| 工具列表顺序       | 固定排序         | 按对象哈希无序遍历                    |
| 依赖文件摘要       | 摘要 hash 变才更新 | 全量文件每轮重传                     |
| 模型与 endpoint | 切换可能使缓存失效    | 频繁 A/B 切模型                   |

## 成本对比表：同一 100 轮会话

假设（简化）：每轮除前缀外新增 2k 输入、1k 输出；前缀合计 80k Token；Opus 输入 \$15/M、输出 \$75/M。

| 模式  | 前缀每轮计费方式            | 100 轮前缀累计（示意） |
|-----|---------------------|---------------|
| 无缓存 | 80k × 100 × 全价输入    | 极高            |
| 有缓存 | 首轮写 + 99 轮读（~10% 价） | 显著下降          |

[Mermaid 图表 — 请在在线版本中查看]

## 与 Sonnet 定价的联动

当团队默认 Sonnet 时：

- 正常输入 \$3/M 是「未命中」锚点。
- 缓存读 \$0.375/M 让你愿意把「稳定知识」往前放。
- 缓存写 \$3.75/M 提醒：不要为极小前缀启用缓存 —— 可能得不偿失。

| Token 类型 | Sonnet 示意单价 | 策略含义     |
|----------|-------------|----------|
| 输入（全价）   | \$3/M       | 压缩可变部分   |
| 缓存读      | \$0.375/M   | 扩大稳定前缀   |
| 缓存写      | \$3.75/M    | 控制「重写」频率 |

## 调试：如何知道「命中了没有」

真实集成应读取：

- API 响应中的 `usage` 字段（`cache_creation_input_tokens`、`cache_read_input_tokens` 等，以官方字段名为准）。
- 内部仪表盘：命中率、每轮前缀长度分布。

教学用日志片段（伪）：

```
{
  "usage": {
    "input_tokens": 1200,
    "cache_creation_input_tokens": 0,
    "cache_read_input_tokens": 80000,
    "output_tokens": 900
  }
}
```

当 `cache_read_input_tokens` 与「稳定前缀长度」接近时，说明工程目标达成。

---

## 与子 Agent 缓存的衔接（预告 17.5）

---

多 Agent 若统一以同一字符串开头，例如：

```
Fork started – processing in background
```

则不同进程的请求在字节级前缀上对齐，利于共享同一缓存键空间（具体以平台实现为准）。详见 `05-sub-agent-cache.md`。

---

## 常见面试式问答

---

**Q：缓存会降低模型「聪明度」吗？**

A：缓存是传输与计费优化，不改变模型权重；但若错误地把动态状态放进「假稳定」前缀，会造成陈旧上下文问题，那是工程失误而非缓存本身。

**Q：小团队要不要上缓存？**

A：只要存在长 system + 多工具 + 长会话，通常值得；短问答收益有限。

---

## 自测

---

1. 用 Sonnet 价：80 万 Token 前缀若全走缓存读，输入侧费用约多少美元？
  2. 举三种「破坏前缀」的代码写法。
  3. 解释「缓存写」为何可能比「普通输入」更贵或同量级（查表对比 \$3.75 vs \$3）。
-



## 小结

- 无缓存长会话可在 Opus 档达到 \$50—100（百轮示意）；有缓存可压到 \$10—19，约 80% 节省是合理宣传口径（视场景浮动）。
- 缓存读  $\approx$  正常输入 10%（Sonnet: \$0.375 / \$3）是记忆锚点。
- 工程核心是：长且稳的前缀 + 有序工具 + 无随机污染。

上一节: [index.md](#) · 下一节: [03-parallel-prefetch.md](#)

## 17.3 并行预取：启动路径上的「不等公交」

**本节焦点：**在 Agent / IDE 启动阶段，**并行**拉取系统信息、Git 状态、环境检测，避免串行等待造成的秒级甚至十秒级卡顿。

### 学习目标

1. 对比 串行 `await A; await B; await C` 与 `Promise.all` 的总耗时模型。
2. 设计 启动阶段任务DAG：哪些可并行、哪些必须顺序（例如依赖 `$PATH` 的检测）。
3. 解释 并行预取与 Token 账单 的关系：主要改善 延迟与体验，间接减少人类因等待而重试。
4. 识别 资源争用风险：磁盘 IO、Git 锁、CPU 密集检测的背压策略。
5. 落地 可观测性：为每个预取任务打点耗时（衔接第 18 篇）。

### 生活类比：出差前的「并行收拾行李」

串行像：先找袜子 → 再找充电器 → 再订车，每一步干等。

并行像：袜子、充电器、订车同时进行，最后在一个「汇合点」检查是否齐全。

前提是：订车不依赖袜子是否穿好 —— 依赖关系要画清。

## 串行 vs 并行：时间线直觉

[Mermaid 图表 — 请在在线版本中查看]

并行墙钟时间  $\approx \max(200, 300, 150) = 300\text{ms}$ ；串行  $\approx 650\text{ms}$ 。

## 源码片段：并行预取骨架

```
// 教学示例：启动时并行预取
type PrefetchResult = {
  sys: SystemInfo;
  git: GitStatus;
  env: EnvironmentProbe;
};

async function prefetchAll(signal: AbortSignal): Promise<PrefetchResult> {
  const sysP = getSystemInfo({ signal });
  const gitP = getGitStatus({ signal });
  const envP = probeEnvironment({ signal });

  const [sys, git, env] = await Promise.all([sysP, gitP, envP]);
  return { sys, git, env };
}

// 在应用入口
async function bootstrap() {
  const ac = new AbortController();
  const prefetch = prefetchAll(ac.signal);
  renderSplash(); // 不阻塞在空白屏
  const ctx = await prefetch;
  mountAgentUI(ctx);
}
```

要点：

- 统一 **AbortSignal**：用户取消启动或超时合并时一并取消子任务。
- 先渲染壳子再等待数据，改善感知性能。

## 任务分类表

| 任务                         | 典型耗时来源                       | 并行安全? | 备注      |
|----------------------------|------------------------------|-------|---------|
| <code>uname</code> / OS 版本 | 进程 spawn                     | 是     | 注意沙箱权限  |
| <code>git status</code>    | 仓库大小                         | 通常    | 大仓库可加超时 |
| Node/Bun 版本                | <code>process.version</code> | 是     | 极快      |
| MCP 服务发现                   | 网络                           | 视情况   | 建议独立超时  |
| 读取超大 <code>.env</code>     | 磁盘                           | 是     | 应限制读取大小 |

## 依赖 DAG（何时不能并行）

[Mermaid 图表 — 请在在线版本中查看]

若 B 必须知道根路径，则 A 先于 B；C 与 B 在 A 之后可并行。

## 进阶：限制并发与优先级

并行不是「无限线程」：

```
async function prefetchWithPool<T>(  
  tasks: Array<() => Promise<T>>,  
  limit: number  
) : Promise<T[]> {  
  const results: T[] = [];  
  let i = 0;  
  async function worker() {  
    while (i < tasks.length) {  
      const idx = i++;  
      results[idx] = await tasks[idx]();  
    }  
  }  
  await Promise.all(Array.from({ length: limit }, worker));  
  return results;  
}
```

适用于：数十个 MCP 探测或子目录扫描，避免把机器打满。

## 与 Token 经济学的关系

| 维度   | 说明                           |
|------|------------------------------|
| 直接计费 | 预取本地信息通常不走模型，不耗 Token        |
| 间接计费 | 卡顿 → 用户取消 → 重试 → 重复发送上下文     |
| 团队体验 | 快启动 → 更少「抱怨式」加长 system 来解释环境 |

## 错误处理：一个失败要不要全失败？

| 策略                                   | 适用                |
|--------------------------------------|-------------------|
| <code>Promise.all</code> 全失败         | 强一致：缺 Git 就不能工作   |
| <code>Promise.allSettled</code> 部分成功 | IDE 可降级：无 Git 仍可用 |
| 超时兜底                                 | 网络类预取必备           |

```
async function withTimeout<T>(p: Promise<T>, ms: number, label: string): Promise<T> {
  let to: ReturnType<typeof setTimeout>;
  const timeout = new Promise<never>((_, rej) => {
    to = setTimeout(() => rej(new Error(`${label} timeout`)), ms);
  });
  try {
    return await Promise.race([p, timeout]);
  } finally {
    clearTimeout(to);
  }
}
```

## 与懒加载的边界（预告 17.4）

预取解决「启动时要已知的全局状态」；懒加载解决「工具/schema 不必一次性进上下文」。二者互补：

- 预取：墙钟时间优化。
- 懒加载：上下文体积优化。

## 检查清单

- [ ] 所有预取任务是否支持 `abort`?
- [ ] 是否为 `git` / 网络 设了 超时?
- [ ] 是否在 UI 上暴露 「仍在加载」 而非白屏?
- [ ] 日志是否带 任务名 + 耗时?

## 自测

1. 画出一个你熟悉项目的启动 DAG。
2. 解释为何 `Promise.all` 中任一 `reject` 会导致整体 `reject`，以及如何用 `allSettled` 改进。
3. 并行预取如何间接降低 Token 成本?

## 小结

- 并行预取把系统信息、Git、环境检测从「串行加法」变成「并行取 max」，是启动性能的基本盘。
- 配好 `AbortSignal`、超时、部分成功 才能在真实仓库与弱网下站稳。
- 与 缓存、懒加载 组合，形成「快 + 省」的双轮驱动。

上一节: [02-prompt-caching.md](#) · 下一节: [04-lazy-loading.md](#)

## 17.4 懒加载：四十二般兵器不必一次背上身

**本节焦点：**将 42 个工具（教学口径）从「启动全量加载」改为**按需加载**：通过 `ToolSearchTool` 注入、条件 `require` 等机制，削减每轮请求中的 schema 体积。

## 学习目标

1. 解释 全量工具加载如何线性放大 输入 Token（工具 JSON Schema 之和）。
2. 描述 `ToolSearchTool` 式「元工具」如何把「工具目录」与「当前可用工具子集」分离。

- 3. 实现 条件 `require` / 动态 `import()` 的边界：Node/Bun 下的冷启动与循环依赖注意点。
- 4. 对比 懒加载与 `Prompt 缓存`：前者减「每轮子集」，后者减「重复前缀」。
- 5. 制定 工具分层策略：核心工具常驻、领域工具按项目类型启停。

## 生活类比：瑞士军刀 vs 工具箱在车库里

若你每次出门都把整个车库背在背上（全量工具），走路都费劲。

懒加载是：口袋里只留一把常用小刀（核心工具），需要电钻时回家取（动态加载）或通过目录卡片（ToolSearch）让店员递给你。

## 问题陈述：Schema 体积爆炸

假设每个工具定义平均 1.5k Token（示意）：

| 策略       | 每轮附带 schema 量 | 42 工具示意        |
|----------|---------------|----------------|
| 全量加载     | 全部            | ~63k Token 仅工具 |
| 懒加载 + 子集 | 5—8 个常用       | ~9—12k Token   |

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：ToolSearchTool 注入模式（概念）

```
// 概念：元工具返回「可加载工具名列表」或「内联说明」
const toolSearchTool = {
  name: "tool_search",
  description: "根据用户意图检索可调工具的名称与摘要。",
  input_schema: {
    type: "object",
    properties: {
      query: { type: "string" },
    },
    required: ["query"],
  },
  async run({ query }: { query: string }) {
    const hits = searchToolCatalog(allToolMetadata, query); // 仅元数据，非全 schema
    return { tools: hits.map((h) => h.name) };
  },
};

// 会话状态：当前已「挂载」的工具
let mountedTools = new Set<string>(["read_file", "grep", "tool_search"]);

function buildAnthropicTools() {
  return [...mountedTools].map((name) => loadToolSchema(name));
}
```

### 流程：

1. 首轮只挂载 核心 + tool\_search。
2. 模型调用 `tool_search` → 返回 `["docker_exec", "k8s_apply"]`。
3. 运行时 `mountedTools.add(...)` 并 动态拉取 对应 schema。

## 条件 require 与动态 import

### CommonJS 风格（示意）

```
function loadExecutor(name) {
  if (name === "git") return require("./executors/git");
  if (name === "npm") return require("./executors/npm");
  throw new Error(`unknown ${name}`);
}
```

## ESM 动态 import（推荐）

```
async function loadToolModule(name: string) {
  switch (name) {
    case "git":
      return import("./tools/git.js");
    case "container":
      return import("./tools/container.js");
    default:
      throw new Error(`unknown tool ${name}`);
  }
}
```

| 方式                        | 优点     | 注意                            |
|---------------------------|--------|-------------------------------|
| <code>require</code> 条件分支 | 简单     | 打包器需识别以拆分 chunk               |
| <code>import()</code>     | 标准 ESM | 需 <code>await</code> ，错误处理异步化 |

## 死代码消除：Bun 打包与 tree shaking

Tree shaking 指 bundler 基于静态分析删除从未被引用的导出，减小产物体积与解析成本。Bun 在 `bun build` 时会做死代码消除；与懒加载结合时，注意 `import` 路径可静态解析。

| 实践                                                       | 作用                 |
|----------------------------------------------------------|--------------------|
| <code>sideEffects: false</code>                          | 告诉打包器哪些包可安全摇树      |
| 避免 <code>export *</code> 滥用                              | 减少「误保留」导出          |
| 动态 <code>import("./x")</code> + <code>--splitting</code> | 罕见工具进独立 chunk，主包更瘦 |

[Mermaid 图表 — 请在在线版本中查看]

### 教学命令（示意）：

```
bun build ./src/main.tsx --outdir=dist --minify --splitting
```

懒加载若在源码层写成静态字符串路径，`bun build --splitting` 更易生成独立 chunk；若路径完全动态（如 `import(\`./tools/${name}\`)`），需配置或白名单，否则可能无法分块或整包打入。



## 工具分层表（建议模板）

| 层级    | 示例工具                       | 加载策略             |
|-------|----------------------------|------------------|
| L0 核心 | read, write, grep, glob    | 会话开始即挂载          |
| L1 通用 | run_terminal_cmd, mcp_list | 第二批或首次使用时        |
| L2 领域 | k8s, terraform, ios        | 项目探测后注入          |
| L3 罕见 | 许可证生成器                     | 仅 tool_search 命中 |

## 与 Prompt 缓存的联合效果

[Mermaid 图表 — 请在在线版本中查看]

- 缓存优化「重复的大前缀」。
- 懒加载优化「每轮都在变的工具块」若也能稳定成「当前挂载集合」则更可预测。

## 反模式

| 反模式                     | 后果              |
|-------------------------|-----------------|
| 每次挂载按随机顺序序列化工具          | 缓存键抖动           |
| tool_search 返回整份 schema | 元工具失去意义         |
| 懒加载不做并发锁                | 两次并行加载同一工具重复 IO |

简单互斥锁示意：

```
const inflight = new Map<string, Promise<ToolDef>>>();

async function getTool(name: string): Promise<ToolDef> {
  if (inflight.has(name)) return inflight.get(name)!;
  const p = loadToolSchema(name);
  inflight.set(name, p);
  try {
    return await p;
  } finally {
    inflight.delete(name);
  }
}
```

## 安全与权限

懒加载不是「隐藏能力」：

- ACL 仍应在 **执行层** 校验（谁允许 `docker`）。
- `tool_search` 应对 **敏感工具** 打标签，默认不向某些工作区暴露。

## 观测指标

| 指标              | 含义         |
|-----------------|------------|
| 平均每轮挂载工具数       | 懒加载是否过「碎」  |
| tool_search 调用率 | 模型是否常找不到工具 |
| 动态 import 失败次数  | 打包路径/权限问题  |

## 自测

1. 用假设计算：42×1.5k vs 8×1.5k Token 的输入侧费用差（Sonnet \$3/M）。
2. 说明 ToolSearchTool 与「把 42 个 schema 全写进 system」的本质差异。
3. 为何工具顺序稳定对缓存友好？

## 课堂扩展：伪代码 —— 会话状态机

```
type SessionState = {
  mounted: Set<string>;
  projectFlavor: "node" | "go" | "unknown";
};

function onProjectDetected(s: SessionState, flavor: SessionState["projectFlavor"]) {
  s.projectFlavor = flavor;
  if (flavor === "node") {
    ["npm_tool", "eslint_tool"].forEach((t) => s.mounted.add(t));
  }
}
```

## 小结

- 42 工具全量加载是 Token 与解析 的双重税；按需加载是规模化必选项。
- ToolSearchTool 注入把「发现」与「装载」拆开，是常见架构模式。
- 条件 require / 动态 import 应配合 打包拆分 与 稳定顺序 才能与缓存策略共振。

上一节: [03-parallel-prefetch.md](#) · 下一节: [05-sub-agent-cache.md](#)

## 17.5 子 Agent 缓存：统一前缀与字节级命中

**本节焦点：**多 Agent / 子进程协作时，通过**统一前缀**（教学示例：`Fork started - processing in background`）对齐请求开头，利用**字节级前缀匹配**提高 Prompt 缓存命中率。

### 学习目标

1. **解释** 为何「子 Agent 的首条可见文本」应高度一致、可预测。
2. **描述** 字节级前缀匹配与「字符串相等」的差异直觉（BOM、换行符、空格）。
3. **设计** 主 Agent 与子 Agent 的 **system 拼接规范**，避免一字之差导致整段失效。
4. **评估** 统一前缀对可观测性的好处：日志 grep、指标归类。
5. **规避** 在多语言环境下因翻译或本地化破坏前缀的问题。

### 生活类比：合唱团的「起调音」

主唱与子歌手若同一音高起调，指挥一挥手大家就对齐。

若每个子歌手用不同方言喊「开始」，听众以为换了首歌 —— 缓存键「对不上」。

### 统一前缀模式

教学用常量（勿在业务中硬编码魔法字符串而不文档化）：

```
export const SUB_AGENT_PUBLIC_PREFIX =
  "Fork started – processing in background\n";

export function wrapSubAgentSystem(base: string): string {
  return SUB_AGENT_PUBLIC_PREFIX + base;
}
```

| 要求          | 说明                                           |
|-------------|----------------------------------------------|
| 固定 UTF-8 编码 | 避免平台默认编码差异                                   |
| 结尾换行一致      | <code>\n</code> vs <code>\r\n</code> 会导致前缀不同 |
| 不插入动态时间     | 时间戳应放在前缀之后                                   |

[Mermaid 图表 — 请在在线版本中查看]

### 源码片段：主 / 子 共用缓存块

```
// 共享：可缓存的稳定块（示意）
const SHARED_STABLE = {
  type: "text",
  text: CORPORATE_POLICY + TOOL_CATALOG_MINIMAL,
  cache_control: { type: "ephemeral" },
};

function buildMainSystem(extra: string) {
  return [SHARED_STABLE, { type: "text", text: extra }];
}

function buildSubSystem(role: string) {
  const header = SUB_AGENT_PUBLIC_PREFIX + `Role:${role}\n`;
  return [SHARED_STABLE, { type: "text", text: header }];
}
```

关键： `SHARED_STABLE` 在主、子请求中逐字节相同，最易命中。

## 字节级前缀匹配：工程师自查表

| 项目          | 检查                         |
|-------------|----------------------------|
| BOM         | 确保文件保存为 无 UTF-8 BOM 或全项目一致 |
| 空格          | 全角空格 与半角 不同                |
| Unicode 规范化 | NFC vs NFD 可能不同（如 ä 组合字符）  |
| 尾部空格        | 看似相同，实际多一个空格即失效            |

### Node 侧规范化（教学）：

```
import { normalize } from "node:path"; // 路径用
// 文本建议：统一 NFC
function toNfc(s: string) {
  return s.normalize("NFC");
}
```

## 与「Fork」语义的结合

统一前缀不仅是缓存，也是 UX：

- 用户在 Transcript 里搜索 `Fork started` 即可过滤子 Agent 输出。
- 告警规则可对包含此前缀的 span 单独采样。

[Mermaid 图表 — 请在在线版本中查看]

## 失败案例库（教学）

| 现象             | 可能原因                                       |
|----------------|--------------------------------------------|
| 子 Agent 从未命中缓存 | 前缀中插入了 <code>pid</code>                    |
| 偶发命中           | 某些机器 <code>\r\n</code> 、某些 <code>\n</code> |
| 升级后命中骤降        | 常量字符串「文案润色」改了两个字                           |

## 与并行预取、懒加载的关系

| 机制   | 子 Agent 场景             |
|------|------------------------|
| 并行预取 | 子 Agent 启动时并行拉取其工作副本状态 |
| 懒加载  | 子 Agent 仅挂载与其子任务相关的工具  |
| 统一前缀 | 与主会话共享 大块稳定前缀          |

三者叠加：又快又省。

## 监控 SQL / 伪查询（示意）

```
-- 示意：按前缀聚合缓存读 Token
SELECT
  CASE WHEN prompt LIKE 'Fork started - processing%' THEN 'sub' ELSE 'main' END AS lane,
  SUM(cache_read_tokens) AS crt
FROM turns
GROUP BY 1;
```

## 安全注意

统一前缀不是认证：

- 仍应对子 Agent 的 凭证 做隔离（最小权限）。
- 前缀可公开，不要在其中放 密钥。

## 自测

1. 写出两种「肉眼相同但字节不同」的前缀变体。
2. 解释为何要把动态内容放在前缀之后。
3. 主、子共享 `SHARED_STABLE` 时，懒加载工具应如何影响共享块？

## 扩展阅读（概念）

- Unicode Normalization Forms（Unicode 标准）

- Anthropic Prompt Caching 文档中的「可缓存块」限制与失效条件

## 术语对照

| 英文              | 中文              |
|-----------------|-----------------|
| Prefix match    | 前缀匹配            |
| Ephemeral cache | 短期/会话级缓存（依厂商定义） |
| Fork            | 分支/子任务（进程或逻辑）   |

## 小结

- 统一前缀 `Fork started - processing in background` 是教学示例，核心是跨 Agent 字节级一致。
- 字节级细节（换行、规范化、BOM）决定「真命中」还是「假一致」。
- 与 共享可缓存块 组合，是 multi-agent 降费的关键拼图。

上一节: [04-lazy-loading.md](#) · 下一节: [06-render-performance.md](#)

# 17.6 React 渲染性能：流畅 UI 如何「省下重试」

**本节焦点：**在 Claude Code 类 IDE 界面中，通过 **虚拟滚动**、**条件渲染**、`React.memo` 等手段降低主线程压力，改善长会话体验，**间接**减少因卡顿导致的重复请求与 Token 浪费。

## 学习目标

1. **解释** 长 Transcript（数千消息）下 DOM 节点膨胀的问题。
2. **实现** 虚拟列表（windowing）的直觉模型与关键 props（item 高度估计、overscan）。
3. **使用** `memo` / `useMemo` / `useCallback` 的边界：何时有效、何时无效。
4. **设计** 条件渲染：折叠工具详情、懒渲染 Markdown 预览。
5. **连接** 性能与成本：流畅 UI → 更少误操作重试。

## 生活类比：超市价签

若超市把一万件商品的价签同时贴在天花板上（全量渲染），你抬头就累。  
虚拟滚动是：只贴你眼前货架那一截，你走它再换 —— 看得见的才存在。

## 三大手段速览

| 手段   | 解决什么            | 典型 API                                                  |
|------|-----------------|---------------------------------------------------------|
| 虚拟滚动 | 长列表 DOM 过多      | <code>react-window</code> 、 <code>react-virtuoso</code> |
| 条件渲染 | 不可见模块仍 diff     | <code>&amp;&amp;</code> 、早期 <code>return</code>         |
| memo | 子树 props 不变仍重渲染 | <code>memo</code> ， <code>useMemo</code>                |

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：虚拟列表骨架

```
import { FixedSizeList as List } from "react-window";

type Row = { id: string; role: "user" | "assistant"; text: string };

function TranscriptPane({ rows }: { rows: Row[] }) {
  const Row = ({ index, style }: { index: number; style: React.CSSProperties }) => (
    <div style={style}>
      <MessageBubble row={rows[index]} />
    </div>
  );

  return (
    <List
      height={600}
      itemCount={rows.length}
      itemSize={72}
      width="100%"
      overscanCount={8}
    >
      {Row}
    </List>
  );
}
```



注意：

- `itemSize` 若可变，需 可变高度列表 或 测量缓存。
- `MessageBubble` 内部仍应用 `memo` 避免行内重绘。

memo 与稳定 props

```
import { memo } from "react";

const MessageBubble = memo(function MessageBubble({ row }: { row: Row }) {
  return (
    <article>
      <header>{row.role}</header>
      <p>{row.text}</p>
    </article>
  );
});
```

失效场景：

| 情况                                               | 结果       |
|--------------------------------------------------|----------|
| 父组件每次 <code>inline</code> 新对象 <code>{row}</code> | memo 无效  |
| <code>new Date()</code> 作 prop                   | 每帧变      |
| 子组件消费 Context 频繁变                                | memo 挡不住 |

对策：`useCallback` / 拆分 Context / selector 库（如 `zustand`、`use-context-selector`）。

条件渲染：工具调用折叠

```
function ToolCallCard({ call }: { call: ToolCall }) {
  const [open, setOpen] = useState(false);
  if (!open) {
    return (
      <button type="button" onClick={() => setOpen(true)}>
        {call.name} (点击展开参数)
      </button>
    );
  }
  return <HeavyJsonViewer value={call.args} />;
}
```

- `HeavyJsonViewer` 仅在展开时挂载，避免千次工具调用的 JSON 高亮拖垮页面。

[Mermaid 图表 — 请在在线版本中查看]

## useMemo 用于昂贵派生

```
function Stats({ rows }: { rows: Row[] }) {
  const tokenishEstimate = useMemo(() => {
    return rows.reduce((s, r) => s + r.text.length / 4, 0); // 粗估非精确
  }, [rows]);
  return <footer>≈ {tokenishEstimate.toFixed(0)} tokens</footer>;
}
```

避免每次父级无关 state 变化都  $O(n)$  扫描。

## 与流式输出的协同（衔接 17.7）

流式追加文本时：

- 虚拟列表应 `scrollToItem` 或 sticky 到底部。
- 防抖更新：每 16ms 合并多次 chunk，减少 React commit 次数。

```
import { useEffect, useRef } from "react";

function useRafThrottle<T>(value: T, cb: (v: T) => void) {
  const raf = useRef<number | null>(null);
  useEffect(() => {
    if (raf.current !== null) cancelAnimationFrame(raf.current);
    raf.current = requestAnimationFrame(() => cb(value));
    return () => {
      if (raf.current !== null) cancelAnimationFrame(raf.current);
    };
  }, [value, cb]);
}
```

## 性能检查清单

- [ ] React DevTools Profiler：哪颗子树最红？
- [ ] 长会话采样：滚动 FPS 是否掉下 50？
- [ ] 内存： detached DOM 是否泄漏？

# 与 Token 成本的因果链

[Mermaid 图表 — 请在在线版本中查看]

优化 UI 是 **成本治理** 的隐藏杠杆。

## 自测

- 1. 说明 `memo` 与 `useMemo` 分别缓存什么。
- 2. 虚拟列表 `overscanCount` 增大有何利弊？
- 3. 为何流式输出时要节流 React state 更新？

## 常见坑

| 坑                     | 修复                       |
|-----------------------|--------------------------|
| 列表 key 用 index        | 重排时状态错乱，用稳定 id           |
| 全局 dark mode Context  | 拆分子树或 memo + 拆分 Provider |
| Markdown 全量 highlight | 延迟到可见或 Web Worker        |

## 小结

- 虚拟滚动治理长 Transcript；条件渲染推迟昂贵子树；**memo** 家族削减无效 diff。
- 与 **流式节流** 搭配，可把「跟手」做到极致。
- 流畅 UI 通过减少 **误操作重试** 间接保护 Token 预算。

上一节: [05-sub-agent-cache.md](#) · 下一节: [07-streaming-pipeline.md](#)

## 17.7 流式管道：async generator 与逐词到达

**本节焦点：**用 `async generator`、`yield` 分片推送模型输出，构建**不等完整响应**的 UX 与背压可控的管道，降低**\*\*首字延迟（TTFB 体感）\*\***并改善取消语义。

### 学习目标

1. 实现 基于 `async function*` 的流式读取循环，配合 `for await` 消费。
2. 对比 「攒齐全文再渲染」与「边到边渲染」在**感知性能与可中断性**上的差异。
3. 解释 流式与 **React 状态更新** 的节流策略（衔接 17.6）。
4. 描述 背压：下游慢时如何暂停上游 `pull`（示意）。
5. 识别 工具调用与流式混排时的解析边界（JSON 片段、SSE 事件）。

### 生活类比：听播客 vs 等整季下载完

- 非流式：整季下载完才能听第一秒。
- 流式：边下边播；不想听了就停（abort），已缓冲的还能保留。

### 核心思想：不等完整响应

[Mermaid 图表 — 请在在线版本中查看]

用户更早开始阅读 → 更少焦虑性重复提问 → 间接省 Token。

## 源码片段：async generator 包装 SSE

```
async function* streamClaudeText(
  body: ReadableStream<Uint8Array>,
  signal: AbortSignal
): AsyncGenerator<string> {
  const reader = body.getReader();
  const decoder = new TextDecoder();
  try {
    while (true) {
      if (signal.aborted) throw new DOMException("aborted", "AbortError");
      const { value, done } = await reader.read();
      if (done) break;
      const text = decoder.decode(value, { stream: true });
      // 真实场景需按 SSE 事件切分，这里教学简化为按块 yield
      if (text) yield text;
    }
  } finally {
    reader.releaseLock();
  }
}

// 消费端
async function renderLoop(gen: AsyncGenerator<string>, onDelta: (s: string) => void) {
  for await (const delta of gen) {
    onDelta(delta);
  }
}
```

## 与 fetch 结合（概念）

```
async function startTurn(prompt: string, signal: AbortSignal) {
  const res = await fetch("/api/claude/stream", {
    method: "POST",
    body: JSON.stringify({ prompt }),
    signal,
    headers: { "content-type": "application/json" },
  });
  if (!res.body) throw new Error("no body");
  for await (const chunk of streamClaudeText(res.body, signal)) {
    appendToTranscript(chunk);
  }
}
```

## 解析复杂度：工具调用穿插

| 模式                | 说明                          |
|-------------------|-----------------------------|
| 纯文本 SSE           | <code>yield</code> 字符串即可    |
| JSON tool_calls 流 | 需增量解析或等结构化块                 |
| 混排                | 常见做法是服务端先流式文本，工具走独立 channel |

[Mermaid 图表 — 请在在线版本中查看]

## 背压示意（教学）

若 UI 渲染极慢，可暂停读取：

```
async function* throttled<T>(source: AsyncIterable<T>, ms: number) {
  for await (const item of source) {
    yield item;
    await new Promise((r) => setTimeout(r, ms));
  }
}
```

生产环境更常用 `requestAnimationFrame` 或 队列 + max pending。

## Abort：用户按「停止」时

```
const ac = new AbortController();
startTurn("...", ac.signal).catch((e) => {
  if (e.name === "AbortError") return; // 用户取消
  showError(e);
});

// 用户点击停止
function onStop() {
  ac.abort();
}
```

流式让 `abort` 更有意义：不必等整包返回。

## 与虚拟滚动联动

长回答边生成边增长度：

- 列表应 **auto-scroll** 到底部，但避免与用户手动上翻冲突（「粘底」标志位）。
- 大块 Markdown 应 **分阶段 highlight**，避免每字符全量 re-parse。

## 错误恢复（预告第 18 篇）

| 事件   | 流式策略                                |
|------|-------------------------------------|
| 网络闪断 | 断点续传需服务端 <code>last-event-id</code> |
| 5xx  | 自动重试应 <b>丢弃</b> 不完整的工具 JSON         |
| 429  | 退避 + 用户可见倒计时                        |

## 性能指标

| 指标       | 目标直觉               |
|----------|--------------------|
| TTFB     | 首 chunk 到达时间       |
| chunk 间隔 | 过长 → 「卡住」感         |
| 取消耗时     | abort 后多久停止计费（依平台） |

## Node/Bun 侧等价

在服务端转发模型流时：

```
import { Readable } from "node:stream";

function generatorToReadable(gen: AsyncGenerator<Uint8Array>) {
  return Readable.from(
    (async function* () {
      for await (const u8 of gen) {
        yield u8;
      }
    })()
  );
}
```

便于接入 Express / Hono 的 `response` 流。

## 自测

- `for await` 与 `while + next()` 消费 async generator 有何区别？
- 为何工具调用 JSON 不适合逐字符展示给用户？
- abort 后客户端还应做哪些 UI 状态 清理？

## 与成本的关系（重申）

| 链路     | 效果                       |
|--------|--------------------------|
| 更快可见输出 | 用户更少打断 / 重发              |
| 可中断    | 少付「不需要的尾部输出」（视 API 计费粒度） |
| 更细粒度观测 | 可对慢 chunk 打点优化 prompt    |

## 小结

- `async generator + yield` 是表达「异步拉取 + 分段产出」的自然模型。
- 流式优化体感首延迟与取消，与 React 节流、虚拟列表 组合完整。
- 工具调用与 JSON 需单独设计，避免半拉子结构污染 UX。

上一节: [06-render-performance.md](#) · 下一节: [08-cost-cheatsheet.md](#)

# 17.8 省钱速查表：场景 × 模型 × 动作

本节汇总 第 17 篇定价锚点与优化手段，用可操作的表格做场景化估算与决策。



## 学习目标

- 1. 快速查阅 Opus / Sonnet 输入输出与缓存读写单价。
- 2. 估算 典型开发日、长会话、子 Agent 派发的费用区间（数量级）。
- 3. 对照 每项优化手段的预期收益与实施成本。
- 4. 制定 团队「默认策略」：何时 Sonnet、何时 Opus、何时开缓存。
- 5. 复盘 如何用遥测验证估算（衔接第 18 篇）。

## 定价锚点表（教学）

| 项目  | Opus（示意）  | Sonnet（示意）   |
|-----|-----------|--------------|
| 输入  | \$15 / 1M | \$3 / 1M     |
| 输出  | \$75 / 1M | \$15 / 1M    |
| 缓存写 | （以官方为准）   | \$3.75 / 1M  |
| 缓存读 | （以官方为准）   | \$0.375 / 1M |

缓存读 / 正常输入（Sonnet）： $\$0.375 \div \$3 = 12.5\%$ （教学中常记 约 10%）。

## 场景估算表 A：单次大回合

假设：150k 输入（含工具与历史）+ 12k 输出。

| 模型     | 输入费用                            | 输出费用                           | 合计（约）         |
|--------|---------------------------------|--------------------------------|---------------|
| Opus   | $150 \times 15 / 1000 = \$2.25$ | $12 \times 75 / 1000 = \$0.90$ | <b>\$3.15</b> |
| Sonnet | $150 \times 3 / 1000 = \$0.45$  | $12 \times 15 / 1000 = \$0.18$ | <b>\$0.63</b> |

[Mermaid 图表 — 请在在线版本中查看]

## 场景估算表 B：百轮会话（与缓存）

| 策略  | Opus 区间（教学） | 说明      |
|-----|-------------|---------|
| 无缓存 | \$50—100    | 大前缀每轮全价 |
| 有缓存 | \$10—19     | 约省 80%  |

实际随每轮 Token、命中率波动；用于**立项沟通**足够。

## 场景估算表 C：工具策略

| 策略              | 相对输入 Token | 适用     |
|-----------------|------------|--------|
| 42 工具全量         | 高          | 不推荐长会话 |
| 懒加载 8 工具        | 低          | 默认推荐   |
| tool_search 元工具 | 中          | 大目录必备  |

## 场景估算表 D：前端与流式

| 动作   | 直接省 Token? | 间接收益      |
|------|------------|-----------|
| 虚拟滚动 | 否          | 减少重试      |
| 流式输出 | 依平台        | 减少焦虑性重复提问 |
| 并行预取 | 否          | 快启动 → 少放弃 |

## 死代码消除：Bun 打包与 tree shaking（速查）

| 命令/配置（示意）                                         | 作用                |
|---------------------------------------------------|-------------------|
| <code>bun build --minify</code>                   | 体积 ↓              |
| <code>bun build --splitting</code>                | 动态 import 拆 chunk |
| <code>"sideEffects": false</code> in package.json | 助 tree shake      |

[Mermaid 图表 — 请在在线版本中查看]

教学片段：

```
// 若从未引用 heavy, 可被摇掉
export const light = () => 1;
export const heavy = () => import("./heavy"); // 动态则进分包
```

决策速查：今天该用哪档模型？

| 信号            | 建议                 |
|---------------|--------------------|
| 大规模重构 / 安全审计  | Opus 或 Opus 抽检     |
| CRUD、格式化、单测补齐 | Sonnet             |
| 上下文已 >200k    | Sonnet + 缓存 + 分片   |
| 预算告警触发        | 全员 Sonnet + 强制缓存审查 |

优化手段 × 收益 × 成本

| 手段               | 预期收益     | 实施成本    | 见节   |
|------------------|----------|---------|------|
| Prompt 缓存        | 高（长会话）   | 中（前缀工程） | 17.2 |
| 并行预取             | 中（体验）    | 低       | 17.3 |
| 懒加载工具            | 高（输入）    | 中       | 17.4 |
| 子 Agent 统一前缀     | 中（缓存命中）  | 低       | 17.5 |
| React 虚拟滚动       | 中（间接）    | 中       | 17.6 |
| 流式管道             | 中（体验/中断） | 中       | 17.7 |
| Bun tree shaking | 低（前端体积）  | 低       | 本节   |

一日账单「咖啡换算」（纯属直觉）

假设 Sonnet, 2M 输入 + 200k 输出 / 天：

- 输入：2 × \$3 = \$6
- 输出：0.2 × \$15 = \$3
- 合计 ~\$9/天 → 约两杯精品咖啡（城市差异大，仅作参考锚）

## 团队政策模板（可复制）

1. 默认模型：Sonnet

2. system 与工具顺序：锁定版本号，禁止随机字段

3. 子 Agent 前缀：使用统一常量 SUB\_AGENT\_PREFIX

4. 代码审查：新增工具必须声明层级（L0-L3）

5. 每周复盘：cache\_read / input 比值

## 与第 18 篇的接口

| 你要证明优化有效 | 需要遥测                       |
|----------|----------------------------|
| 缓存命中上升   | cache_read_input_tokens 系列 |
| 懒加载生效    | 每轮 tools schema token 估计   |
| 流式体验     | 首 chunk 延迟 P95             |

## 常见数字陷阱

| 陷阱            | 说明                      |
|---------------|-------------------------|
| 把「字符数」当 Token | 中英文压缩比不同                |
| 忽略输出          | 长解释同样贵                  |
| 只看模型价         | 工具/MCP 侧也会引入间接成本（人力、机器） |

## 自测答案指引（不写死数字）

1. 用 17.8 表 A 公式重算若输入变为 300k 的费用。

2. 说明 80% 节省依赖哪些前提（长前缀重复、命中率高）。

3. 列举三项 不花钱 但 降 retry 的优化。

# Mermaid：优化组合路线图

[Mermaid 图表 — 请在在线版本中查看]

## 小结

- 速查表把抽象 \$/M 落到 单次 / 百轮 / 团队政策。
- Sonnet 缓存读  $\approx$  输入一折 是最应刻进肌肉记忆的比率之一。
- 优化组合拳：缓存 + 懒加载 + 前缀 + UI + 流式；Bun tree shaking 守护分发体积。

第 17 篇完。继续：[第 18 篇 · 遥测与生命周期](#)

## 第18篇：遥测与生命周期

### 第18篇 · 18.1 进程状态调度：前后台 Agent 与生命周期

**本篇导读：** Claude Code 类系统在长时间运行中会同时存在**前台交互**与**后台子任务**。本节拆解 Agent 前后台无缝切换、独立 `AbortController`、完成通知回调与**进度追踪**的工程设计。

#### 学习目标

1. **描述** 前台 Agent 与后台 Agent 在 **UI 状态机**中的切换条件（用户聚焦、任务排队、子 Agent fork）。
2. **解释** 为何后台 Agent 必须持有**独立** `AbortController`，与主会话解耦取消语义。
3. **实现** 完成后的 **回调 / 事件总线** 模式，避免轮询浪费。
4. **设计** 进度追踪：`phase`、`percent`（若可估）、`heartbeat` 时间戳。
5. **关联** 第 17 篇：后台任务失败不应拖垮前台 **Token 预算**（隔离上下文）。

#### 生活类比：餐厅前台与后厨

- **前台**接待新客、点菜、结账 —— **低延迟响应**。
- **后厨**炖汤、慢烤 —— **可后台进行**，好了**按铃**（回调），不应让前台站着干等。
- 若客人离席（`abort`），前台**取消未上的菜**，但不应误删别桌正在炖的汤 —— **每桌独立** `AbortController`。

# 核心概念表

| 概念              | 含义                   |
|-----------------|----------------------|
| 前台 Agent        | 与用户当前输入绑定的执行上下文      |
| 后台 Agent        | fork / 队列中的子任务，可并行   |
| AbortController | 取消 fetch、子进程、生成器的统一柄 |
| 回调 / Promise    | 任务完成向上层通知            |
| 进度              | 可观测、可展示、可告警的中间状态     |

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：独立 AbortController

```
type AgentHandle = {
  id: string;
  abort: AbortController;
  onComplete: (result: Result) => void;
  onProgress?: (p: Progress) => void;
};

const agents = new Map<string, AgentHandle>();

export function forkBackgroundAgent(job: JobSpec, parentId: string): string {
  const id = crypto.randomUUID();
  const abort = new AbortController();

  const handle: AgentHandle = {
    id,
    abort,
    onComplete: (result) => {
      emit("agent:complete", { id, parentId, result });
      agents.delete(id);
    },
    onProgress: (p) => emit("agent:progress", { id, p }),
  };

  agents.set(id, handle);
  void runAgentLoop(job, { signal: abort.signal, handle });
  return id;
}

export function cancelAgent(id: string) {
  agents.get(id)?.abort.abort();
}
```

- 要点：
- 子 `AbortSignal` 不应用父 `signal` 直接替换，否则父取消会误杀无关后台任务（除非业务明确要求级联）。

## 级联取消 vs 独立取消

| 策略  | 场景                                         |
|-----|--------------------------------------------|
| 独立  | 用户只想停当前输入，后台索引继续跑                          |
| 级联  | 用户「关闭工作区」— 全员 abort                        |
| 可配置 | 设置项 <code>background.detachOnCancel</code> |



[Mermaid 图表 — 请在在线版本中查看]

## 进度追踪字段建议

| 字段      | 类型      | 说明                               |
|---------|---------|----------------------------------|
| phase   | string  | planning / tooling / summarizing |
| step    | number  | 单调递增                             |
| message | string  | 人类可读                             |
| ts      | number  | Date.now() heartbeat             |
| etaMs   | number? | 可估则填，不可则省略                       |

```
type Progress = {
  phase: string;
  step: number;
  message: string;
  ts: number;
  etaMs?: number;
};

function heartbeat(handle: AgentHandle, p: Progress) {
  handle.onProgress?.(p);
}
```

## 与 UI 的绑定：无缝切换

1. 前台 running：输入框禁用或显示「停止」。
2. 后台 running：状态栏显示 N 个后台任务，可点开看进度。
3. 完成：Toast + Transcript 注入摘要，不把整段子上下文合并进前台（避免 Token 爆炸）——只合并结论。

## 并发与资源上限

| 风险       | 缓解                                    |
|----------|---------------------------------------|
| 无限制 fork | 队列 + maxParallel                      |
| 内存膨胀     | 子 Agent 输出落盘，摘要进主会话                   |
| 僵尸进程     | 见 <code>04-resource-cleanup.md</code> |

## 事件命名规范（建议）

| 事件                          | payload                             |
|-----------------------------|-------------------------------------|
| <code>agent:start</code>    | <code>{ id, parentId, mode }</code> |
| <code>agent:progress</code> | <code>{ id, p }</code>              |
| <code>agent:complete</code> | <code>{ id, result }</code>         |
| <code>agent:error</code>    | <code>{ id, error }</code>          |

便于 Perfetto 与 全链路遥测 订阅（后续小节）。

## 自测

- 何时应使用**独立** AbortController，何时应**链接**父 signal？
- 后台完成回调里，应避免把哪些内容合并进主上下文？
- 画出一个三状态 UI（空闲 / 前台运行 / 有后台任务）的切换表。

## 与 18.2–18.6 的路线图

| 文件                     | 主题                 |
|------------------------|--------------------|
| 02-perfetto.md         | 性能追踪注册             |
| 03-telemetry.md        | Transcript、计时、延迟   |
| 04-resource-cleanup.md | Shell / MCP / Hook |
| 05-error-recovery.md   | 熔断、重试、降级           |
| 06-best-practices.md   | 生产运维               |

## 小结

- 前后台分离是长会话产品化的基础；独立 **abort** 是正确取消的底线。
- 回调 + 事件优于忙轮询；进度 **heartbeat** 是体验与遥测的共同数据源。
- 与 资源清理、错误恢复 拼起来，才构成完整生命周期。

## 进阶：任务队列与公平性

当多个后台 Agent 竞争同一资源（CPU、Git 锁、MCP 配额）时，建议引入**轻量队列**：

| 策略    | 描述                         | 代价       |
|-------|----------------------------|----------|
| FIFO  | 先到先得                       | 长任务阻塞短任务 |
| 加权公平  | 按 <b>priority</b> 与已消耗配额排序 | 实现复杂     |
| 用户级隔离 | 每用户并发上限                    | 资源碎片化    |

[Mermaid 图表 — 请在在线版本中查看]

## 与 Windows / Electron 的注意事项

| 平台       | 建议                                                                                          |
|----------|---------------------------------------------------------------------------------------------|
| Windows  | <code>process.kill</code> 语义不同, shell 追踪用 <code>child.pid</code> + <code>taskkill</code> 备选 |
| Electron | 在 <code>window-all-closed</code> 与 <code>before-quit</code> 双钩子里调用清理 (见 18.4)               |
| 沙箱       | 子进程可能无法发信号给孙进程, 需记录整棵 PID 树                                                                 |

## 术语表

| 英文          | 中文       |
|-------------|----------|
| Foreground  | 前台       |
| Background  | 后台       |
| AbortSignal | 取消信号     |
| Callback    | 回调       |
| Heartbeat   | 心跳 / 进度戳 |

下一节: [02-perfetto.md](#) — Perfetto 性能追踪。

## 18.2 Perfetto 性能追踪: `registerPerfettoAgent()`

**本节焦点:** 通过 `registerPerfettoAgent()` (教学命名) 将追踪模块注册进系统, 把**每一次 Agent 运行**建模为一次**微服务调用**: 有入口 span、子 span (工具、模型、渲染), 可在 Perfetto UI 中切片分析。

### 学习目标

1. 解释 Perfetto / Chrome Tracing 模型: `track`、`slice`、`flow` 的基本语义。
2. 设计 `registerPerfettoAgent` 的 API: 注册 name、版本、采样开关。
3. 映射 Agent 回合 → `turn` slice, 工具调用 → child slice, HTTP → nested slice。

4. 评估 开销：生产环境采样率、异步 buffer。
5. 关联 全链路遥测 (18.3)：同一 `traceId` 贯通日志与追踪。

---

## 生活类比：机场航班雷达

---

每次 Agent 运行像一架航班：起飞 (turn 开始)、巡航 (模型思考)、经停 (工具)、降落 (输出完成)。

Perfetto 是雷达屏幕：你看到的是时间轴上的航迹，而不是黑盒。

---

## 核心注册流程（概念 API）

---

```
// 教学用 API 名
type PerfettoAgentModule = {
  name: string;
  version: string;
  onTrace: (emit: PerfettoEmitter) => void;
};

declare function registerPerfettoAgent(mod: PerfettoAgentModule): Disposable;

type PerfettoEmitter = {
  beginSlice(cat: string, title: string, args?: Record<string, unknown>): void;
  endSlice(): void;
  instant(name: string, args?: Record<string, unknown>): void;
};
```

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：在 Agent 回合包一层

```
const perfetto = registerPerfettoAgent({
  name: "claude-code-agent",
  version: "2.0.0",
  onTrace(emit) {
    bus.on("turn:start", ({ id }) => {
      emit.beginSlice("agent", `turn:${id}`, { id });
    });
    bus.on("turn:end", () => emit.endSlice());

    bus.on("tool:start", ({ name }) => {
      emit.beginSlice("tool", name);
    });
    bus.on("tool:end", () => emit.endSlice());

    bus.on("http:start", ({ route }) => {
      emit.beginSlice("http", route);
    });
    bus.on("http:end", ({ status }) => {
      emit.instant("http:status", { status });
      emit.endSlice();
    });
  },
});

// 应用退出
process.on("exit", () => perfetto.dispose());
```

## 微服务心智：每次运行 = 一次调用链

| Span         | 类比微服务            |
|--------------|------------------|
| turn         | API Gateway 入站请求 |
| model.stream | 下游 LLM 服务        |
| tool.grep    | 内部工具微服务          |
| mcp.fetch    | 外部 MCP 提供方       |

[Mermaid 图表 — 请在在线版本中查看]

## 与 `traceId` / `spanId` 对齐（示意）

```
import { randomUUID } from "node:crypto";

function startTurnTrace() {
  const traceId = randomUUID();
  const baggage = { traceId };
  return baggage;
}
```

日志行应带 `traceId`，便于从 Perfetto slice 跳转到 结构化日志。

## 生产开关

| 环境 | 建议                                       |
|----|------------------------------------------|
| 开发 | 100% 追踪或 Chrome <code>trace_event</code> |
| 预发 | 50% 采样                                   |
| 生产 | 1–5% 采样 + 慢请求全量                          |

```
const SAMPLE_RATE = Number(process.env.PERFETTO_SAMPLE ?? "0.01");

function maybe<T>(fn: () => T): T | undefined {
  if (Math.random() < SAMPLE_RATE) return fn();
  return undefined;
}
```

## 输出格式注意

- Perfetto JSON、Chrome Trace Event、OpenTelemetry 可互转；选型依现有栈。
- 时钟域：前端 `performance.now()` 与后端 `hrtime` 需对齐策略（服务器权威 vs 客户端展示）。

## 常见坑

| 坑                                       | 后果                 |
|-----------------------------------------|--------------------|
| <code>begin</code> 无配对 <code>end</code> | trace 文件损坏 / UI 错乱 |
| slice 过密 (每 token)                      | 体积爆炸               |
| PII 打进 args                             | 合规风险               |

脱敏示例：

```
function redact(s: string) {
  return s.length > 200 ? s.slice(0, 200) + "..." : s;
}
```

## 与第 17 篇性能优化的闭环

| 优化   | 在 Perfetto 中验证                           |
|------|------------------------------------------|
| 并行预取 | 启动阶段 slice 墙钟变短                          |
| 懒加载  | <code>tool.register</code> slice 后移      |
| 流式   | <code>model.firstChunk</code> instant 提前 |

## 自测

- 为何要把 `tool` 与 `model` 分成不同 `category`?
- 采样率与 问题可复现性 如何权衡?
- `instant` 事件与 `slice` 的区别?



表：推荐 category 命名

| category | 用途                |
|----------|-------------------|
| agent    | 回合级               |
| model    | API 调用            |
| tool     | 本地工具              |
| mcp      | 远程 MCP            |
| ui       | React commit / 渲染 |

小结

- `registerPerfettoAgent()` 是把追踪插件化的枢纽；每次 Agent 运行视为微服务调用链最利于对齐团队语言。
- 配对 slice、控制粒度、脱敏 是上线三板斧。
- 与 `traceld` 日志贯通后，才能从「感觉慢」走到「哪一段慢」。

上一节: [index.md](#) · 下一节: [03-telemetry.md](#)

18.3 全链路遥测：Transcript、工具计时与 API 延迟

本节焦点：执行副本（Transcript）强制记录、工具调用计时、API 延迟监控 三位一体，构成可审计、可优化、可告警的数据平面。

学习目标

1. 定义 Transcript 的最小不可变字段集（谁、何时、输入摘要、输出摘要、工具轨迹）。
2. 实现 工具包装的 `before/after` 计时与错误捕获。
3. 设计 API 延迟直方图：P50/P95/P99 与模型维度切片。
4. 解释 为何 Transcript 应 `append-only`（合规与复盘）。
5. 对接 Perfetto（18.2）与资源清理（18.4）：同一 `turnId`。

## 生活类比：飞机黑匣子

- **Transcript** 是黑匣子：发生了什么按时间顺序**不可抵赖**地记下。
- **工具计时** 是每个仪表的**读数间隔**：哪块表异常一目了然。
- **API 延迟** 是**塔台响应时间**：管制慢，整条航线堵。

## Transcript 记录模式

| 字段                        | 说明                          |
|---------------------------|-----------------------------|
| <code>turnId</code>       | UUID                        |
| <code>ts</code>           | ISO 时间                      |
| <code>actor</code>        | user / agent / sub-agent id |
| <code>model</code>        | opus / sonnet               |
| <code>inputDigest</code>  | hash 或截断摘要                  |
| <code>outputDigest</code> | 同上                          |
| <code>toolSpans</code>    | 有序数组                        |
| <code>status</code>       | ok / error / aborted        |

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：强制 append Transcript

---

```
type TranscriptEvent =
  | { type: "user"; text: string; turnId: string; ts: string }
  | { type: "assistant"; text: string; turnId: string; ts: string }
  | { type: "tool"; name: string; ms: number; ok: boolean; turnId: string; ts: string };

const sink: TranscriptEvent[] = []; // 教学: 内存; 生产: WAL / S3 / DB

export function appendTranscript(e: TranscriptEvent) {
  Object.freeze(e);
  sink.push(e);
  // 生产: 同步 fsync 或消息队列
}

export function assertTranscriptEnabled() {
  if (process.env.TRANSCRIPT_OFF === "1") {
    throw new Error("Transcript recording cannot be disabled in this environment");
  }
}
```

「强制记录」 = 关键环境变量禁止关闭 + 启动自检。

---

## 工具调用计时包装器

---

```
type ToolFn<TArgs, TRes> = (args: TArgs) => Promise<TRes>;

function instrumentTool<TArgs, TRes>(  
  name: string,  
  fn: ToolFn<TArgs, TRes>,  
  turnId: string  
) : ToolFn<TArgs, TRes> {  
  return async (args) => {  
    const t0 = performance.now();  
    try {  
      const res = await fn(args);  
      appendTranscript({  
        type: "tool",  
        name,  
        ms: performance.now() - t0,  
        ok: true,  
        turnId,  
        ts: new Date().toISOString(),  
      });  
      return res;  
    } catch (e) {  
      appendTranscript({  
        type: "tool",  
        name,  
        ms: performance.now() - t0,  
        ok: false,  
        turnId,  
        ts: new Date().toISOString(),  
      });  
      throw e;  
    }  
  };  
}
```

## API 延迟监控

```
type LatencySample = { route: string; ms: number; model?: string; ts: number };

const samples: LatencySample[] = [];

export function recordApiLatency(s: LatencySample) {
  samples.push(s);
  // 生产: 推送到 Prometheus histogram 或 OTel
}

export function p95(values: number[]) {
  if (!values.length) return 0;
  const a = [...values].sort((x, y) => x - y);
  return a[Math.floor(0.95 * (a.length - 1))];
}
```

| 指标名 (示意)              | 类型        | 标签             |
|-----------------------|-----------|----------------|
| claude_api_latency_ms | histogram | model , region |
| tool_duration_ms      | histogram | tool           |
| turn_total_ms         | summary   | agent_mode     |

## 全链路 ID 贯通

[Mermaid 图表 — 请在在线版本中查看]

## 存储与保留策略

| 层级  | 保留   | 备注   |
|-----|------|------|
| 热存储 | 7 天  | 快速查询 |
| 温存储 | 90 天 | 成本折中 |
| 冷归档 | 1 年+ | 合规   |

## 隐私与脱敏

| 数据     | 策略              |
|--------|-----------------|
| 源码片段   | 仅 hash + 行号     |
| env 秘密 | 永不写入 Transcript |
| 用户消息   | 按策略掩码邮箱/电话      |

## 与成本分析（第 17 篇）

| Transcript 字段             | 用于               |
|---------------------------|------------------|
| <code>model</code>        | 分模型账单核对          |
| <code>toolSpans</code>    | 识别异常频繁工具调用       |
| <code>tokens</code> （若记录） | 与实际 API usage 对账 |

## 自测

- 为何 `Object.freeze` 事件对象（教学）有助于审计心智？
- `instrumentTool` 中若 `fn` 内部再次调用未包装工具，会出现什么观测盲区？
- P95 与 P99 分别适合哪种告警？

## Dashboard 草图（文字）

- Panel A：每分钟 turn 数
- Panel B：API P95 按模型
- Panel C：工具失败率 TopN
- Panel D：aborted 占比

## 小结

- Transcript append-only 是可信叙事的基础；工具计时定位局部慢；API 延迟定位网络/服务端慢。

- `turnId` + `traceId` 把日志、追踪、黑匣子缝成一条线。
- 与 `Perfetto`、资源清理、错误恢复 共用同一套事件总线最省事。

上一节: [02-perfetto.md](#) · 下一节: [04-resource-cleanup.md](#)

## 18.4 资源清理: `killShellTasksForAgent()` 与连接卸载

**本节焦点：** Agent 结束或取消时，系统性回收 `Shell` 子进程、闲置 `MCP` 连接、运行时 `Hook` 与 `状态缓存`，避免僵尸进程与内存泄漏拖垮长会话桌面端。

### 学习目标

1. 实现 `killShellTasksForAgent(agentId)` 的语义：按 Agent 维度猎杀 shell 任务树。
2. 描述 `MCP` 客户端空闲断开策略：超时、引用计数、优雅 shutdown。
3. 列举 常见运行时 `Hook`（`fetch` patch、`console` wrap）的卸载顺序。
4. 清空 状态缓存：LRU、会话级 Map、弱引用何时失效。
5. 把 清理挂到 `abort`、进程退出、窗口关闭 三钩子。

### 生活类比：剧组收工

拍完一场戏要：关灯（断 `MCP`）、遣散群演（杀 `shell`）、拆轨道（卸 `Hook`）、清空化妆台（状态缓存）。

若只喊「散会」不做收工清单，第二天片场全是遗留道具和占着场地的车。

### Shell 僵尸：问题陈述

| 现象       | 根因                         |
|----------|----------------------------|
| 端口仍占用    | 子进程未 <code>SIGTERM</code>  |
| CPU 风扇狂转 | 死循环脚本                      |
| 文件锁      | <code>git</code> / 编译进程未退出 |

[Mermaid 图表 — 请在在线版本中查看]

## 源码片段：按 Agent 追踪 Shell

```
type ShellTask = {
  pid: number;
  agentId: string;
  child: ReturnType<typeof spawn>;
};

const shells = new Set<ShellTask>();

export function registerShellTask(agentId: string, child: ReturnType<typeof spawn>) {
  const pid = child.pid!;
  shells.add({ pid, agentId, child });
  child.on("exit", () => {
    for (const s of shells) {
      if (s.pid === pid) shells.delete(s);
    }
  });
}

export function killShellTasksForAgent(agentId: string) {
  for (const s of [...shells]) {
    if (s.agentId !== agentId) continue;
    try {
      process.kill(-s.pid, "SIGTERM"); // 进程组（若可用）
    } catch {
      s.child.kill("SIGTERM");
    }
    shells.delete(s);
  }
}
```

Windows / 无进程组环境需换策略；此处 Unix 教学为主。



## MCP 连接：强制断开闲置

```
type MCPConn = { id: string; lastUsed: number; close: () => Promise<void> };
const mcps = new Map<string, MCPConn>();

const IDLE_MS = 60_000;
setInterval(async () => {
  const now = Date.now();
  for (const [id, c] of mcps) {
    if (now - c.lastUsed > IDLE_MS) {
      await c.close().catch(() => {});
      mcps.delete(id);
    }
  }
}, 10_000).unref();

export function touchMcp(id: string) {
  mcps.get(id)!.lastUsed = Date.now();
}
```

| 策略         | 适用       |
|------------|----------|
| 固定 idle 超时 | 大多数      |
| 引用计数       | 多消费者共享连接 |
| 立即关        | 单次脚本     |

## 卸载运行时 Hook

```
type HookRestore = () => void;
const restores: HookRestore[] = [];

export function installFetchHook() {
  const orig = globalThis.fetch;
  globalThis.fetch = async (...args) => {
    // 遥测、注入 header...
    return orig(...args);
  };
  restores.push(() => {
    globalThis.fetch = orig;
  });
}

export function uninstallAllHooks() {
  while (restores.length) {
    restores.pop()!();
  }
}
```

顺序：先停新请求 → 等飞行中完成 → 再 `uninstall`。

[Mermaid 图表 — 请在在线版本中查看]

## 清空状态缓存

| 缓存类型      | 清理触发                            |
|-----------|---------------------------------|
| 会话 Map    | <code>session:end</code>        |
| 全局 LRU    | <code>memory pressure</code> 事件 |
| 编译缓存（若内嵌） | <code>workspace:close</code>    |

```
const sessionCache = new Map<string, unknown>();

export function clearStateForSession(sessionId: string) {
  for (const key of [...sessionCache.keys()]) {
    if (key.startsWith(sessionId + ":")) sessionCache.delete(key);
  }
}
```

## 与 AbortController 集成 (18.1)

```
function wireCleanup(agentId: string, signal: AbortSignal) {
  const onAbort = () => {
    killShellTasksForAgent(agentId);
    clearStateForSession(agentId);
  };
  signal.addEventListener("abort", onAbort, { once: true });
}
```

## 检查清单

- [ ] 子进程是否注册到 全局表？
- [ ] MCP `close` 是否 `await`？
- [ ] Hook 是否可 幂等 卸载？
- [ ] 是否在 `before-quit` (Electron) 重复调用？

## 常见坑

| 坑          | 修复                             |
|------------|--------------------------------|
| SIGKILL 过早 | 先 SIGTERM, grace period 后 KILL |
| 进程组 ID 为 1 | 检测并降级为单 PID                    |
| MCP 半开 TCP | 设 keepalive / socket timeout   |

## 自测

- 为何按 `agentId` 杀 shell 而不是全局 `pkill` ?
- Hook 卸载为何要等 `in-flight`?
- idle MCP 断开如何与 `工具调用中` 竞态?

## 表：清理动作与触发源

| 动作                     | turn abort | session end | app quit |
|------------------------|------------|-------------|----------|
| killShellTasksForAgent | ✓          | ✓           | ✓        |
| MCP close              | 可选         | ✓           | ✓        |
| uninstall hooks        |            |             | ✓        |
| clear session cache    | ✓          | ✓           | ✓        |

## 小结

- `killShellTasksForAgent()` 是防止 `shell 僵尸` 的第一道闸。
- MCP 闲置断开 保护句柄与远程配额。
- Hook 卸载 + 缓存清空 让长时间运行的 IDE 可预测地回到基线。

上一节: [03-telemetry.md](#) · 下一节: [05-error-recovery.md](#)

## 18.5 错误恢复：熔断、重试、退避与优雅降级

本节焦点：连续 3 次压缩失败熔断、API 错误自动重试 + 指数退避、优雅降级（换模型、缩上下文、关工具），让系统在故障下仍可「带病运行」。

### 学习目标

1. 实现 压缩失败计数器与熔断状态机（开路 / 半开 / 闭合）。
2. 设计 API 重试：可重试状态码（429、5xx、网络）vs 不可重试（400、401）。
3. 应用 指数退避 + 抖动避免惊群。
4. 定义 降级阶梯：降模型 → 截断历史 → 禁用 MCP → 只读模式。
5. 记录 每次故障到 Transcript（18.3）供事后复盘。

### 生活类比：电梯检修

- 重试：按一次没反应，稍等再按，而不是狂按一百次。
- 熔断：连续三次故障，电梯停运检修（开路），避免把人困在半空反复撞击。
- 降级：货梯坏了，走楼梯（功能降级但仍可达）。

## 压缩失败熔断（教学口径：连续 3 次）

```
type CompressState = { failures: number; openUntil: number };

const COMPRESS_THRESHOLD = 3;
const COOLDOWN_MS = 60_000;

export class ContextCompressor {
  private state: CompressState = { failures: 0, openUntil: 0 };

  async compress(ctx: string): Promise<string> {
    if (Date.now() < this.state.openUntil) {
      throw new Error("compressor circuit open - skip compress");
    }
    try {
      const out = await this.doCompress(ctx);
      this.state = { failures: 0, openUntil: 0 };
      return out;
    } catch (e) {
      this.state.failures += 1;
      if (this.state.failures >= COMPRESS_THRESHOLD) {
        this.state.openUntil = Date.now() + COOLDOWN_MS;
      }
      throw e;
    }
  }

  private async doCompress(ctx: string): Promise<string> {
    // 调用摘要模型或本地算法
    return ctx.slice(0, Math.min(ctx.length, 8000));
  }
}
```

[Mermaid 图表 — 请在在线版本中查看]

## API 重试与退避

```
async function sleep(ms: number) {
  await new Promise((r) => setTimeout(r, ms));
}

function jitter(ms: number) {
  return Math.floor(ms * (0.5 + Math.random()));
}

export async function withRetry<T>(fn: () => Promise<T>, opts?: { max?: number }) {
  const max = opts?.max ?? 4;
  let attempt = 0;
  let delay = 500;
  for (;;) {
    try {
      return await fn();
    } catch (e: any) {
      attempt += 1;
      if (attempt >= max) throw e;
      if (!isRetriable(e)) throw e;
      await sleep(jitter(delay));
      delay = Math.min(delay * 2, 30_000);
    }
  }
}

function isRetriable(e: any) {
  const code = e?.status ?? e?.code;
  if (code === 429) return true;
  if (typeof code === "number" && code >= 500) return true;
  if (e?.name === "TypeError") return true; // 网络 flake
  return false;
}
```

| 错误类型        | 重试?                |
|-------------|--------------------|
| 429         | ✓ (配合 Retry-After) |
| 502/503/504 | ✓                  |
| 401         | ✗ (刷新凭证)           |
| 400         | ✗ (修正请求)           |

## 优雅降级阶梯

[Mermaid 图表 — 请在在线版本中查看]

| 级别 | 用户可见文案（示例）        |
|----|-------------------|
| L1 | 「已切换至经济模式模型」      |
| L2 | 「历史较早部分已折叠，可手动展开」 |
| L3 | 「外部工具暂不可用」        |
| L4 | 「当前为只读协助模式」       |

## 与流式结合（第 17 篇）

若流式中途失败：

- 1. 不要把半拉子 tool JSON 提交给执行器。
- 2. 标记 turn `status=error`，允许用户「重试最后一步」。

```
appendTranscript({
  type: "tool",
  name: "stream",
  ms: 0,
  ok: false,
  turnId,
  ts: new Date().toISOString(),
});
```

## 半开试探（可选）

```
async function halfOpenTry(compressor: ContextCompressor) {
  // 冷却结束后允许一次真实压缩；失败立即再开路
}
```

## 告警阈值建议

| 指标         | 阈值（示例）     |
|------------|------------|
| 压缩熔断次数 / h | > 5        |
| API 429 比例 | > 10%      |
| 重试后仍失败     | 页面级 banner |

## 自测

1. 为何需要 抖动？
2. 连续压缩失败可能暗示哪些根因（模型、网络、上下文过大）？
3. 降级到「只读」时如何防止误导用户以为仍能写文件？

## 表：重试头处理

| 响应头               | 行为   |
|-------------------|------|
| Retry-After       | 优先使用 |
| 无                 | 指数退避 |
| x-ratelimit-reset | 可选对齐 |

## 小结

- 三次压缩失败熔断避免在坏路径上浪费 Token 与 CPU。
- 可重试错误 + 退避 + 抖动 是 API 集成的底线配置。
- 降级阶梯把「全挂」变成「能用但不完美」，并应用文案管理预期。

上一节: [04-resource-cleanup.md](#) · 下一节: [06-best-practices.md](#)

# 18.6 生产最佳实践：监控、告警、日志与调试

**本节焦点：**把 18.1–18.5 的能力产品化：**监控看板、告警路由、结构化日志与一线调试技巧**，形成可复制的运维手册。

## 学习目标

1. 列出 Agent 系统的 SLO 建议：可用性、P95 延迟、错误预算。
2. 设计 告警分级：P1 全站不可用 vs P3 体验退化。



3. 规范 日志字段： `level`、`traceId`、`turnId`、`agentId`、无密钥。
4. 掌握 调试路径：从用户截图 → `traceId` → Transcript → Perfetto。
5. 制定 发布 checklist：特性开关、回滚、灰度。

## 生活类比：医院急诊分诊

- 监控是分诊台体温血压：常态数据。
- 告警是红灯：哪位医生去取决于级别。
- 日志是病历：后续会诊都靠它。
- 调试是影像科：定位病灶，不是猜。

## 监控四维矩阵

| 维度  | 示例指标                | 数据源         |
|-----|---------------------|-------------|
| 可用性 | turn 成功率            | Transcript  |
| 延迟  | API P95、首 token     | telemetry   |
| 成本  | tokens / user / day | billing API |
| 资源  | shell 数、MCP 连接数     | 进程表         |

[Mermaid 图表 — 请在在线版本中查看]

## 告警分级表

| 级别 | 条件（示例）           | 响应 |
|----|------------------|----|
| P1 | 成功率 < 90% 持续 10m | 电话 |
| P2 | P95 > 30s 持续 20m | IM |
| P3 | 单工具失败率突增         | 工单 |

## 结构化日志片段

```
import pino from "pino";

const log = pino({ level: process.env.LOG_LEVEL ?? "info" });

export function logTurn(opts: {
  traceId: string;
  turnId: string;
  msg: string;
  model: string;
}) {
  log.info(
    {
      traceId: opts.traceId,
      turnId: opts.turnId,
      model: opts.model,
    },
    opts.msg
  );
}
```

| 字段                     | 必须?  |
|------------------------|------|
| <code>traceId</code>   | 强烈推荐 |
| <code>userId</code>    | 视合规  |
| <code>prompt</code> 全文 | 通常禁止 |

## 调试剧本（复制即用）

1. 拿到 `turnId`（UI 复制或日志）。
2. 拉取 Transcript 片段：用户输入 → 工具链 → 错误栈。
3. 打开 同时间段 Perfetto trace：看 `model` slice 是否异常长。
4. 检查 资源：该 `agentId` 是否残留 shell（18.4）。
5. 验证 是否为 429：若是，审查重试与配额（18.5）。

[Mermaid 图表 — 请在在线版本中查看]

## 特性开关与灰度

| 实践                            | 说明                 |
|-------------------------------|--------------------|
| <code>FEATURE_SUBAGENT</code> | 可按 workspace 百分比开放 |
| kill switch                   | API 异常时一键关 MCP     |
| 配置远端下发                        | 避免发版才关功能           |

## 日志查询示例（伪 PromQL / Loki）

```
{app="claude-code"} |= "turnId=abc123"
```

```
histogram_quantile(0.95, sum(rate(claude_api_latency_ms_bucket[5m])) by (le))
```

## 值班手册模板

```
## 429 风暴
1. 确认是否全局
2. 检查最近是否上线更高模型默认档
3. 临时: 降采样 + 强制 Sonnet

## 内存上涨
1. heap snapshot
2. 检查 sessionCache 与 transcript buffer
3. killShellTasksForAgent 全量 (维护窗)
```

## 与合规

| 要求     | 做法               |
|--------|------------------|
| GDPR   | 可删除 Transcript 行 |
| SOC2   | 访问审计与加密静态存储      |
| 企业 SSO | 日志脱敏用户标识         |

## 自测

1. 为何 P95 比 平均值 更适合 Agent 延迟 SLO?
2. 日志里放 API key 的替代方案是什么?
3. 灰度发布时最先观察哪三个指标?

## 发布 Checklist

- [ ] 特性开关默认值
- [ ] 回滚脚本 / 镜像 tag
- [ ] 仪表盘已加新 slice category
- [ ] on-call 文档更新

## 小结

- 监控 + 告警 + 结构化日志 是生产三件套；traceId 是串联线索。
- 调试剧本把「玄学」变「工序」。
- 灰度与 kill switch 让 Agent 这种高耦合系统可治理。

## On-call 与 Runbook 联动

将 18.3 Transcript 与 18.2 Perfetto 的查询链接写进 Runbook，可缩短 MTTR：

| 步骤 | 工具                             | 产出       |
|----|--------------------------------|----------|
| 1  | 日志 <code>traceId</code>        | 时间窗口     |
| 2  | Transcript <code>turnId</code> | 决策链还原    |
| 3  | Perfetto slice                 | 慢在模型还是工具 |
| 4  | 指标                             | 是否系统性故障  |

[Mermaid 图表 — 请在在线版本中查看]

# 与文化：「可观测性优先」PR 模板

```
## 可观测性
- [ ] 新增代码路径是否打点 / 日志?
- [ ] 是否增加 PII?
- [ ] 是否需要更新 Grafana 面板?
```

## 小结（全篇收束）

第 18 篇从 进程调度 到 Perfetto、遥测、清理、容错、生产实践，形成闭环：看得见、停得掉、救得起、查得到。

第 18 篇完。回顾性能与成本：[第 17 篇](#)

## 第19篇：实战Lab

### Lab 1：从零搭建最简 Agent Loop

系列：Claude Code 完全指南 V2 · 第 19 篇实战 Lab

前置：已安装 Node.js 18+、npm；拥有 [Anthropic API Key](#)

#### 学习目标

完成本 Lab 后，你将能够：

1. 使用 `npm init` 与 TypeScript 配置搭建一个最小 Node.js 项目。
2. 理解 **Agent Loop** 的核心：`读入` → `调用模型` → `输出` → `判断是否继续`（含工具调用）→ `循环`。
3. 使用官方 `@anthropic-ai/sdk` 调用 Claude Messages API。
4. 在终端中与模型进行多轮对话，并识别响应中的 `tool_use` 块（为后续 Lab 打基础）。

#### 核心概念

##### 什么是 Agent Loop?

Agent 不是「问一句答一句」的静态接口，而是一个**持续运转的循环**：每次把当前对话状态发给模型，根据返回决定是结束、继续对话，还是执行工具并把结果再喂回模型。

[Mermaid 图表 — 请在在线版本中查看]

本 Lab 中，遇到 `tool_use` 时我们先打印说明并继续循环（真正执行工具在 Lab 2）。

## 环境准备

### 步骤 1: 创建项目目录

```
mkdir mini-agent-lab1 && cd mini-agent-lab1
npm init -y
```

### 步骤 2: 安装依赖

```
npm install @anthropic-ai/sdk
npm install -D typescript @types/node tsx
```

- `@anthropic-ai/sdk`: 官方 Anthropic SDK。
- `tsx`: 直接运行 TypeScript, 无需先编译 (开发体验好)。

### 步骤 3: 配置 TypeScript

创建 `tsconfig.json` :

```
{
  "compilerOptions": {
    "target": "ES2022",
    "module": "NodeNext",
    "moduleResolution": "NodeNext",
    "strict": true,
    "skipLibCheck": true,
    "outDir": "dist",
    "rootDir": "src",
    "esModuleInterop": true
  },
  "include": ["src/**/*"]
}
```

### 步骤 4: 配置启动脚本

在 `package.json` 的 `scripts` 中增加:

```
{
  "scripts": {
    "start": "tsx src/main.ts"
  }
}
```

## 步骤 5: 设置 API Key

```
export ANTHROPIC_API_KEY="sk-ant-api03-..."
```

或在项目根目录创建 `.env` (本 Lab 代码用环境变量即可, Lab 8 再整合 dotenv)。

---

## 完整可运行代码

将下列文件保存为 `src/main.ts`。代码约 70 行, 实现: `while(true)` 循环、读取 `readline` 用户输入、调用 Claude、打印助手文本, 并检测 `tool_use` (暂不执行)。



```
/**
 * Lab 1: 最简 Agent Loop
 * 运行: ANTHROPIC_API_KEY=xxx npm start
 */
import * as readline from "node:readline/promises";
import { stdin as input, stdout as output } from "node:process";
import Anthropic from "@anthropic-ai/sdk";
import type {
  Message,
  MessageParam,
  ToolUseBlock,
} from "@anthropic-ai/sdk/resources/messages";

const MODEL = "claude-sonnet-4-20250514";

function extractText(message: Message): string {
  const parts: string[] = [];
  for (const block of message.content) {
    if (block.type === "text") parts.push(block.text);
  }
  return parts.join("\n");
}

function extractToolUses(message: Message): ToolUseBlock[] {
  return message.content.filter(
    (b): b is ToolUseBlock => b.type === "tool_use"
  );
}

async function main(): Promise<void> {
  const apiKey = process.env.ANTHROPIC_API_KEY;
  if (!apiKey) {
    console.error("请设置环境变量 ANTHROPIC_API_KEY");
    process.exit(1);
  }

  const client = new Anthropic({ apiKey });
  const rl = readline.createInterface({ input, output });

  const history: MessageParam[] = [
    {
      role: "user",
      content:
        "你是一个简洁的编程助手。用户会在终端与你对话。若需要调用工具，请先说明意图；本演示环境尚未挂载真实工",
    },
  ];

  console.log("最简 Agent Loop 已启动。输入 exit 或 quit 退出。\\n");

  while (true) {
    const userLine = await rl.question("你: ");
    const trimmed = userLine.trim();
    if (!trimmed) continue;
    if (/^(exit|quit)$/i.test(trimmed)) {
      console.log("再见。");
      rl.close();
    }
  }
}
```

```
        break;
    }

    history.push({ role: "user", content: trimmed });

    const message = await client.messages.create({
        model: MODEL,
        max_tokens: 1024,
        messages: history,
    });

    const assistantText = extractText(message);
    const toolUses = extractToolUses(message);

    history.push({
        role: "assistant",
        content: message.content,
    });

    if (assistantText) {
        console.log("\n助手:\n" + assistantText + "\n");
    }

    if (toolUses.length > 0) {
        console.log(
            "[Loop] 检测到 tool_use (本 Lab 仅打印, Lab 2 将执行): ",
            toolUses.map((t) => t.name).join(", ")
        );
        // 占位: 真实场景需把 tool_result 以 user 消息形式追加, 再调 API
    }
}

main().catch((err) => {
    console.error(err);
    process.exit(1);
});
```

---

## 运行与验证

---

```
cd mini-agent-lab1
export ANTHROPIC_API_KEY="你的密钥"
npm start
```

1. 输入普通问题, 应看到助手回复。
  2. 若模型返回 `tool_use`, 终端会打印 `[Loop] 检测到 tool_use...`。
-

## 步骤指引小结

| 步骤 | 内容                                                                                   |
|----|--------------------------------------------------------------------------------------|
| 1  | <code>npm init</code> 、安装 SDK / TypeScript / tsx                                     |
| 2  | 编写 <code>tsconfig.json</code> 与 <code>src/main.ts</code>                             |
| 3  | 用 <code>readline</code> 读用户输入，维护 <code>history: MessageParam[]</code>                |
| 4  | <code>client.messages.create</code> 发请求，把 <code>message.content</code> 原样追加到 history |
| 5  | 解析文本与 <code>tool_use</code> ，为 Lab 2 预留执行分支                                          |

## 架构一图读懂

[Mermaid 图表 — 请在在线版本中查看]

## 常见问题

Q：为什么要把 assistant 的 `content` 整块放进 history？

A：Claude Messages API 要求多轮对话时保留完整的 assistant 消息（含 `tool_use` 块），后续才能配对 `tool_result`。

Q：本 Lab 为何不执行工具？

A：先保证循环与 API 调用正确；Lab 2 会接入 `tools` 参数与 `tool_result` 回传。

## 完整 `package.json` (Lab 1 可直接使用)

```
{
  "name": "mini-agent-lab1",
  "version": "1.0.0",
  "type": "module",
  "private": true,
  "scripts": {
    "start": "tsx src/main.ts",
    "typecheck": "tsc --noEmit"
  },
  "dependencies": {
    "@anthropic-ai/sdk": "^0.39.0"
  },
  "devDependencies": {
    "@types/node": "^22.0.0",
    "tsx": "^4.19.0",
    "typescript": "^5.7.0"
  }
}
```

使用 ESM 时务必设置 `"type": "module"`，与 `tsconfig` 的 `NodeNext` 一致。

## 术语速查

| 术语                        | 含义                                         |
|---------------------------|--------------------------------------------|
| Agent Loop                | 反复「请求模型 → 处理输出 → 可能执行工具 → 再请求」的循环          |
| <code>MessageParam</code> | 发给 Messages API 的单条角色消息 (user / assistant) |
| <code>tool_use</code>     | 模型在 <code>content</code> 中声明要调用的工具及参数      |
| <code>tool_result</code>  | 你把工具返回值回传给模型的 user 消息块                     |

## 课后作业 (建议)

- 在检测到 `tool_use` 时，打印完整 JSON 的 `input` 到 stderr，观察模型如何填参。
- 增加命令 `/reset`：清空 `history` 仅保留一条 system 风格的首条 user。
- 将 `MODEL` 改为从环境变量 `ANTHROPIC_MODEL` 读取，默认仍为 Sonnet。

## 下一 Lab

[Lab 2: 工具注册与执行](#) 将实现 `Tool` 接口、`ToolRegistry`，以及 `ReadFileTool` / `ListDirectoryTool` 的真实执行与 API 对接。

## Lab 2: 实现工具注册与执行

系列：Claude Code 完全指南 V2 · 第 19 篇实战 Lab

前置：完成 [Lab 1](#)，理解 Messages API 与 `history` 维护方式。

### 学习目标

- 定义统一的 `Tool` 接口：`name`、`description`、`inputSchema`、`execute`。
- 实现 `ToolRegistry`：注册、按名查找、列出全部工具定义（供 API `tools` 参数使用）。
- 实现两个示例工具：`ReadFileTool`、`ListDirectoryTool`（仅读文件系统，注意路径安全）。
- 在 `messages.create` 中传入 `tools`，处理 `stop_reason == "tool_use"`，执行工具并以 `tool_result` `user` 消息追加后继续请求，直到模型输出自然语言结束。

### 概念：Claude 工具调用流程

[Mermaid 图表 — 请在在线版本中查看]

要点：

- `tools` 数组中的每一项需符合 Anthropic 的 `tool schema` (`name`、`description`、`input_schema`)。
- 收到 `tool_use` 后，必须用 同一次对话 追加一条 `user` 消息，`content` 为 `tool_result` 块，且 `tool_use_id` 与响应一致。

## 项目结构（在 Lab 1 基础上扩展）

```
mini-agent-lab2/
├─ package.json
├─ tsconfig.json
├─ src/
│   ├── main.ts
│   ├── tool.ts           # Tool 接口
│   ├── registry.ts       # ToolRegistry
│   └─ tools/
│       ├── read-file.ts
│       └─ list-dir.ts
```

安装依赖与 Lab 1 相同；若从 Lab 1 复制项目，只需新增上述文件。

### 步骤 1: Tool 接口与 JSON Schema 类型

创建 `src/tool.ts` :

```
import type { Tool as AnthropicTool } from "@anthropic-ai/sdk/resources/messages";

/** 单工具执行: 返回给模型的文本（可序列化为字符串） */
export type ToolExecute = (input: Record<string, unknown>) => Promise<string>;

export interface ToolDefinition {
  name: string;
  description: string;
  /** JSON Schema object, 对应 API 的 input_schema */
  inputSchema: Record<string, unknown>;
  execute: ToolExecute;
}

export function toAnthropicTool(def: ToolDefinition): AnthropicTool {
  return {
    name: def.name,
    description: def.description,
    input_schema: def.inputSchema as AnthropicTool["input_schema"],
  };
}
```

### 步骤 2: ToolRegistry

创建 `src/registry.ts` :

```
import type { Tool as AnthropicTool } from "@anthropic-ai/sdk/resources/messages";
import type { ToolDefinition } from "../tool.js";
import { toAnthropicTool } from "../tool.js";

export class ToolRegistry {
  private readonly map = new Map<string, ToolDefinition>();

  register(tool: ToolDefinition): void {
    if (this.map.has(tool.name)) {
      throw new Error(`Tool already registered: ${tool.name}`);
    }
    this.map.set(tool.name, tool);
  }

  get(name: string): ToolDefinition | undefined {
    return this.map.get(name);
  }

  listDefinitions(): AnthropicTool[] {
    return [...this.map.values()].map(toAnthropicTool);
  }
}
```

---

### 步骤 3: `ReadFileTool` 与 `ListDirectoryTool`

---

`src/tools/read-file.ts` :

```
import * as fs from "node:fs/promises";
import * as path from "node:path";
import type { ToolDefinition } from "../tool.js";

function resolveSafe(root: string, userPath: string): string {
  const abs = path.resolve(root, userPath);
  const rootAbs = path.resolve(root);
  if (!abs.startsWith(rootAbs + path.sep) && abs !== rootAbs) {
    throw new Error("Path escapes workspace root");
  }
  return abs;
}

export function createReadFileTool(workspaceRoot: string): ToolDefinition {
  return {
    name: "read_file",
    description: "Read UTF-8 text from a file under the workspace root.",
    inputSchema: {
      type: "object",
      properties: {
        path: { type: "string", description: "Relative path from workspace root" },
      },
      required: ["path"],
    },
    async execute(input) {
      const p = String(input.path ?? "");
      const full = resolveSafe(workspaceRoot, p);
      const data = await fs.readFile(full, "utf8");
      return data;
    },
  };
}
```

src/tools/list-dir.ts :



```
import * as fs from "node:fs/promises";
import * as path from "node:path";
import type { ToolDefinition } from "../tool.js";

function resolveSafe(root: string, userPath: string): string {
  const abs = path.resolve(root, userPath);
  const rootAbs = path.resolve(root);
  if (!abs.startsWith(rootAbs + path.sep) && abs !== rootAbs) {
    throw new Error("Path escapes workspace root");
  }
  return abs;
}

export function createListDirectoryTool(workspaceRoot: string): ToolDefinition {
  return {
    name: "list_directory",
    description: "List non-hidden entries in a directory under the workspace root.",
    inputSchema: {
      type: "object",
      properties: {
        path: {
          type: "string",
          description: "Relative directory path; use . for root",
        },
      },
      required: ["path"],
    },
    async execute(input) {
      const p = String(input.path ?? ".");
      const full = resolveSafe(workspaceRoot, p);
      const entries = await fs.readdir(full, { withFileTypes: true });
      const lines = entries
        .filter((e) => !e.name.startsWith("."))
        .map((e) => (e.isDirectory() ? `${e.name}/` : e.name));
      return lines.length ? lines.join("\n") : "(empty)";
    },
  };
}
```

---

## 步骤 4：主循环（含工具执行）

---

创建 `src/main.ts`（完整可运行）：

```
import * as readline from "node:readline/promises";
import { stdin as input, stdout as output } from "node:process";
import * as path from "node:path";
import Anthropic from "@anthropic-ai/sdk";
import type {
  Message,
  MessageParam,
  ToolResultBlockParam,
  ToolUseBlock,
} from "@anthropic-ai/sdk/resources/messages";
import { ToolRegistry } from "./registry.js";
import { createReadFileTool } from "./tools/read-file.js";
import { createListDirectoryTool } from "./tools/list-dir.js";

const MODEL = "claude-sonnet-4-20250514";

function extractText(message: Message): string {
  const parts: string[] = [];
  for (const block of message.content) {
    if (block.type === "text") parts.push(block.text);
  }
  return parts.join("\n");
}

function extractToolUses(message: Message): ToolUseBlock[] {
  return message.content.filter(
    (b): b is ToolUseBlock => b.type === "tool_use"
  );
}

async function runAgentTurn(
  client: Anthropic,
  history: MessageParam[],
  tools: ReturnType<ToolRegistry["listDefinitions"]>,
  registry: ToolRegistry
): Promise<void> {
  let current = await client.messages.create({
    model: MODEL,
    max_tokens: 2048,
    tools,
    messages: history,
  });

  while (true) {
    history.push({ role: "assistant", content: current.content });

    const text = extractText(current);
    if (text) console.log("\n助手:\n" + text + "\n");

    const uses = extractToolUses(current);
    if (uses.length === 0) break;

    const results: ToolResultBlockParam[] = [];
    for (const u of uses) {
      const tool = registry.get(u.name);
      let out: string;
```

```
        if (!tool) {
            out = `Unknown tool: ${u.name}`;
        } else {
            try {
                const inputObj = (u.input ?? {}) as Record<string, unknown>;
                out = await tool.execute(inputObj);
            } catch (e) {
                out = e instanceof Error ? e.message : String(e);
            }
        }
        results.push({
            type: "tool_result",
            tool_use_id: u.id,
            content: out,
        });
    }

    history.push({ role: "user", content: results });

    current = await client.messages.create({
        model: MODEL,
        max_tokens: 2048,
        tools,
        messages: history,
    });
}
}

async function main(): Promise<void> {
    const apiKey = process.env.ANTHROPIC_API_KEY;
    if (!apiKey) {
        console.error("请设置 ANTHROPIC_API_KEY");
        process.exit(1);
    }

    const workspace = path.resolve(process.cwd());
    const registry = new ToolRegistry();
    registry.register(createReadFileTool(workspace));
    registry.register(createListDirectoryTool(workspace));
    const tools = registry.listDefinitions();

    const client = new Anthropic({ apiKey });
    const rl = readline.createInterface({ input, output });

    const history: MessageParam[] = [
        {
            role: "user",
            content: `工作区根目录: ${workspace}。你可使用 read_file 与 list_directory 协助用户。`,
        },
    ];

    console.log("Lab2 Agent (带工具) 已启动。exit/quit 退出。\\n");

    while (true) {
        const line = (await rl.question("你: ")).trim();
        if (!line) continue;
        if (/^(exit|quit)$/i.test(line)) {
```

```
        rl.close();
        break;
    }
    history.push({ role: "user", content: line });
    await runAgentTurn(client, history, tools, registry);
}
}

main().catch((e) => {
    console.error(e);
    process.exit(1);
});
```

---

## package.json 参考

---

```
{
  "name": "mini-agent-lab2",
  "type": "module",
  "scripts": {
    "start": "tsx src/main.ts"
  },
  "dependencies": {
    "@anthropic-ai/sdk": "^0.39.0"
  },
  "devDependencies": {
    "@types/node": "^22.0.0",
    "tsx": "^4.19.0",
    "typescript": "^5.7.0"
  }
}
```

使用 ESM 时请在 `tsconfig.json` 中保持 `"module": "NodeNext"`。

---

## 运行与验证

---

```
export ANTHROPIC_API_KEY=...
npm start
```

尝试提问：「列出当前目录有哪些文件」或「读取 package.json 的内容」。应观察到模型发起 `tool_use`，终端打印工具执行后的第二轮助手回复。

---

## 内部状态机 (Mermaid)

[Mermaid 图表 — 请在在线版本中查看]

## 安全提示

- `resolveSafe` 限制路径必须在 `workspaceRoot` 下，避免任意文件读取。
- 生产环境还需限制文件大小、二进制检测等（本 Lab 从简）。

## 小结与下一 Lab

| 能力     | 说明                                                                                         |
|--------|--------------------------------------------------------------------------------------------|
| 注册表    | 集中管理工具名与实现                                                                                 |
| API 对齐 | <code>listDefinitions()</code> → <code>tools</code>                                        |
| 闭环     | <code>assistant(tool_use)</code> → <code>user(tool_result)</code> → 再次 <code>create</code> |

下一篇：[Lab 3: 权限控制](#) 将在执行工具前加入确认与黑名单策略。

# Lab 3：添加权限控制

系列：Claude Code 完全指南 V2 · 第 19 篇实战 Lab

前置：完成 [Lab 2](#)，已能执行 `read_file` / `list_directory`。

## 学习目标

- 在工具真正执行前插入统一的权限检查层（Policy / Gate）。
- 实现简单规则：**读类操作默认通过**；**写类或敏感操作**需用户在终端输入 `y / n` 确认。
- 维护**命令黑名单**（如包含 `rm -rf`、`mkfs` 等子串的命令一律拒绝），防止模型或用户误触发毁灭性操作。
- 将拒绝原因以字符串返回给模型（作为 `tool_result` 内容），使模型能向用户解释并重试。

---

## 权限模型概览

---

[Mermaid 图表 — 请在在线版本中查看]

本 Lab 在 Lab 2 的 `ToolRegistry` 执行路径外包一层 `PermissionGuard`，主循环只调用 `guard.run(toolName, input, () => tool.execute(input))`。

---

## 步骤 1：定义工具风险等级与策略接口

---

创建 `src/permissions/types.ts`：

```
export type RiskLevel = "read" | "write" | "command";

export interface PermissionContext {
  /** 同步询问用户；返回 true 表示允许 */
  confirm(message: string): Promise<boolean>;
}

export type ClassifyFn = (
  toolName: string,
  input: Record<string, unknown>
) => RiskLevel;
```

---

## 步骤 2：黑名单与分类逻辑

---

创建 `src/permissions/policy.ts`：

```

import type { ClassifyFn, PermissionContext, RiskLevel } from "./types.js";

const COMMAND_BLACKLIST_PATTERNS: RegExp[] = [
  /\brm\s+-rf\b/i,
  /\bmkfs\b/i,
  /\:(\(\)\{\s*:\|:&\s*\}\;:\/, // fork bomb 简例
  />\s*\dev\/sd/, // 危险重定向示例
];

export function isBlacklistedCommand(cmd: string): string | null {
  const c = cmd.trim();
  for (const re of COMMAND_BLACKLIST_PATTERNS) {
    if (re.test(c)) return `黑名单命中: ${re.source}`;
  }
  return null;
}

/** 演示用分类: 按工具名粗分; 你可按 input 细化 */
export const classifyTool: ClassifyFn = (
  toolName: string,
  _input: Record<string, unknown>
): RiskLevel => {
  if (toolName === "read_file" || toolName === "list_directory") return "read";
  if (toolName === "run_shell") return "command";
  return "write";
};

export async function ensureAllowed(
  level: RiskLevel,
  toolName: string,
  input: Record<string, unknown>,
  ctx: PermissionContext
): Promise<{ ok: true } | { ok: false; reason: string }> {
  if (level === "read") return { ok: true };

  if (level === "command") {
    const cmd = String(input.command ?? input.cmd ?? "");
    const bl = isBlacklistedCommand(cmd);
    if (bl) return { ok: false, reason: bl };
    const ok = await ctx.confirm(
      `[命令执行] 是否允许运行?\n工具: ${toolName}\n命令: ${cmd}`
    );
    return ok ? { ok: true } : { ok: false, reason: "用户拒绝执行命令" };
  }

  // write 或其他
  const ok = await ctx.confirm(
    `[写操作] 是否允许工具 ${toolName} 修改系统状态?\n输入: ${JSON.stringify(
      input
    )}`
  );
  return ok ? { ok: true } : { ok: false, reason: "用户拒绝写操作" };
}

```

## 步骤 3: `PermissionGuard` 包装执行

---

创建 `src/permissions/guard.ts` :

```
import type { ToolDefinition } from "../tool.js";
import type { PermissionContext } from "../types.js";
import { classifyTool, ensureAllowed } from "../policy.js";

export class PermissionGuard {
  constructor(private readonly ctx: PermissionContext) {}

  async wrapExecute(
    def: ToolDefinition,
    input: Record<string, unknown>
  ): Promise<string> {
    const level = classifyTool(def.name, input);
    const decision = await ensureAllowed(level, def.name, input, this.ctx);
    if (!decision.ok) return `[权限拒绝] ${decision.reason}`;
    return def.execute(input);
  }
}
```

## 步骤 4: 演示用 `run_shell` 工具（可选但推荐）

---

创建 `src/tools/run-shell.ts` （仅用于教学，生产请用更严格的沙箱）：



```
import { execFile } from "node:child_process";
import { promisify } from "node:util";
import type { ToolDefinition } from "../tool.js";

const execFileAsync = promisify(execFile);

export function createRunShellTool(): ToolDefinition {
  return {
    name: "run_shell",
    description:
      "Run a single shell command (non-interactive). Dangerous; requires user confirmation a",
    inputSchema: {
      type: "object",
      properties: {
        command: { type: "string", description: "Single command line" },
      },
      required: ["command"],
    },
    async execute(input) {
      const command = String(input.command ?? "");
      // 极简演示: 拆成可执行文件 + 参数 (真实场景需更严谨解析)
      const [bin, ...args] = command.split(/\s+/);
      if (!bin) throw new Error("empty command");
      const { stdout, stderr } = await execFileAsync(bin, args, {
        maxBuffer: 1024 * 1024,
      });
      return (stdout || stderr || "(no output)").toString();
    },
  };
}
```

---

## 步骤 5: 整合到主循环 `main.ts`

---

在 Lab 2 的 `runAgentTurn` 中, 将执行段改为使用 `PermissionGuard`。下面给出关键修改片段 (完整文件 = Lab 2 结构 + 下列替换)。

```
import * as readline from "node:readline/promises";
import { stdin as input, stdout as output } from "node:process";
import { PermissionGuard } from "../permissions/guard.js";
import { createRunShellTool } from "../tools/run-shell.js";
// ... 其余 import 同 Lab2

async function runAgentTurn(
  client: Anthropic,
  history: MessageParam[],
  tools: ReturnType<ToolRegistry["listDefinitions"]>,
  registry: ToolRegistry,
  guard: PermissionGuard
): Promise<void> {
  let current = await client.messages.create({
    model: MODEL,
    max_tokens: 2048,
    tools,
    messages: history,
  });

  while (true) {
    history.push({ role: "assistant", content: current.content });
    const text = extractText(current);
    if (text) console.log("\n助手:\n" + text + "\n");

    const uses = extractToolUses(current);
    if (uses.length === 0) break;

    const results: ToolResultBlockParam[] = [];
    for (const u of uses) {
      const def = registry.get(u.name);
      let out: string;
      if (!def) out = `Unknown tool: ${u.name}`;
      else {
        const inputObj = (u.input ?? {}) as Record<string, unknown>;
        out = await guard.wrapExecute(def, inputObj);
      }
      results.push({
        type: "tool_result",
        tool_use_id: u.id,
        content: out,
      });
    }

    history.push({ role: "user", content: results });
    current = await client.messages.create({
      model: MODEL,
      max_tokens: 2048,
      tools,
      messages: history,
    });
  }
}

async function main(): Promise<void> {
  const rl = readline.createInterface({ input, output });
```

```
const guard = new PermissionGuard({
  async confirm(msg: string) {
    console.log("\n" + msg + "\n允许? [y/N]: ");
    const ans = (await rl.question("")).trim().toLowerCase();
    return ans === "y" || ans === "yes";
  },
});

const registry = new ToolRegistry();
registry.register(createReadFileTool(workspace));
registry.register(createListDirectoryTool(workspace));
registry.register(createRunShellTool());

// ... 循环里 await runAgentTurn(..., guard);
}
```

## 行为矩阵

| 工具             | 分类      | 黑名单 | 用户确认 |
|----------------|---------|-----|------|
| read_file      | read    | —   | 否    |
| list_directory | read    | —   | 否    |
| run_shell      | command | 是   | 是    |
| 未来 write_file  | write   | 可扩展 | 是    |

## 测试用例建议

- 读文件：应无确认提示。
- `run_shell` + `echo hello`：应弹出确认，`y` 后成功。
- `run_shell` + `rm -rf /`：应直接拒绝，无需确认（黑名单）。

## Mermaid：与模型交互时的拒绝路径

[Mermaid 图表 — 请在在线版本中查看]

## 设计取舍

- 为何不把黑名单放在模型侧？ 模型可被诱导；执行前硬编码策略更可靠。
- 为何读操作不确认？ 降低摩擦；若工作区含密钥文件，应改为「敏感路径二次确认」（扩展练习）。

## 下一 Lab

[Lab 4: 流式响应](#) 将把 `messages.create` 改为流式 API，实现终端逐 token 输出。

## Lab 4: 实现流式响应

系列：Claude Code 完全指南 V2 · 第 19 篇实战 Lab

前置：完成 [Lab 2](#) (Lab 3 权限可与本 Lab 独立合并)。

## 学习目标

1. 使用 Anthropic SDK 的 `stream: true`（或 `messages.stream`）消费 Server-Sent Events 风格的分片。
2. 将文本增量实时写入终端（`process.stdout.write`），改善交互体验。
3. 从流中重组完整 `Message`（或等价结构），以便正确提取 `tool_use` 块并走与 Lab 2 相同的工具执行闭环。
4. 理解流式场景下 `message_start` / `content_block_start` / `content_block_delta` / `message_delta` 等事件顺序。

## 流式事件管线

[Mermaid 图表 — 请在在线版本中查看]

SDK 的具体事件名以当前版本为准；核心是：累积 `text delta`，并对 `tool_use` 块累积 `partial_json` 直至块结束。

---

## 步骤 1: 依赖与模型常量

---

与前面 Lab 相同，确保：

```
npm install @anthropic-ai/sdk
```

---

## 步骤 2: 流式辅助 — 累积完整内容与 tool\_use

---

创建 `src/streaming/accumulate.ts`（适配 `@anthropic-ai/sdk` 的 stream 迭代器；若类型有差异，以官方类型为准微调）：

```

import type Anthropic from "@anthropic-ai/sdk";
import type {
  Message,
  MessageStreamEvent,
} from "@anthropic-ai/sdk/resources/messages";

type ToolUsePartial = {
  id: string;
  name: string;
  inputJson: string;
};

/**
 * 消费 async iterable 流，边打印文本边累积最终 Message 所需结构。
 * 返回与 messages.create 类似的对象，便于复用 Lab2 的 extractToolUses。
 */
export async function streamToMessage(
  stream: AsyncIterable<MessageStreamEvent>,
  onTextDelta: (t: string) => void
): Promise<Message> {
  const textParts: string[] = [];
  const toolUses: ToolUsePartial[] = [];
  let currentTextBlockIndex = -1;
  let currentToolIndex = -1;
  let stopReason: Message["stop_reason"] | null = null;
  let usage: Message["usage"] | undefined;

  for await (const event of stream) {
    switch (event.type) {
      case "message_start": {
        // usage 可能在 message_delta 中更新
        break;
      }
      case "content_block_start": {
        if (event.content_block.type === "text") {
          currentTextBlockIndex = event.index;
          textParts.push("");
        } else if (event.content_block.type === "tool_use") {
          currentToolIndex = event.index;
          toolUses.push({
            id: event.content_block.id,
            name: event.content_block.name,
            inputJson: "",
          });
        }
        break;
      }
      case "content_block_delta": {
        if (event.delta.type === "text_delta") {
          const t = event.delta.text;
          onTextDelta(t);
          if (currentTextBlockIndex >= 0) {
            textParts[currentTextBlockIndex] =
              (textParts[currentTextBlockIndex] ?? "") + t;
          }
        } else if (event.delta.type === "input_json_delta") {

```

```

        const chunk = event.delta.partial_json ?? "";
        const tu = toolUses[currentToolIndex];
        if (tu) tu.inputJson += chunk;
    }
    break;
}
case "message_delta": {
    if (event.delta.stop_reason)
        stopReason = event.delta.stop_reason as Message["stop_reason"];
    if (event.usage) usage = event.usage as Message["usage"];
    break;
}
default:
    break;
}
}

const content: Message["content"] = [];

for (let i = 0; i < textParts.length; i++) {
    if (textParts[i]) content.push({ type: "text", text: textParts[i] });
}
for (const tu of toolUses) {
    let input: Record<string, unknown> = {};
    try {
        input = tu.inputJson ? JSON.parse(tu.inputJson) : {};
    } catch {
        input = { _raw: tu.inputJson };
    }
    content.push({
        type: "tool_use",
        id: tu.id,
        name: tu.name,
        input,
    });
}

return {
    id: "stream-local",
    type: "message",
    role: "assistant",
    content,
    model: "stream",
    stop_reason: stopReason ?? "end_turn",
    stop_sequence: null,
    usage: usage ?? { input_tokens: 0, output_tokens: 0 },
};
}

```

**说明：**不同 SDK 版本事件字段可能略有不同；若编译报错，请对照

`node_modules/@anthropic-ai/sdk` 中 `MessageStreamEvent` 定义逐项对齐。

## 步骤 3：主循环中改用 `stream`

`src/main.ts` 核心片段：

```
import Anthropic from "@anthropic-ai/sdk";
import { streamToMessage } from "../streaming/accumulate.js";

const MODEL = "claude-sonnet-4-20250514";

async function createMessageStreaming(
  client: Anthropic,
  params: Omit<Anthropic.Messages.MessageCreateParams, "stream"> & {
    stream?: true;
  }
): Promise<import("@anthropic-ai/sdk/resources/messages").Message> {
  const stream = await client.messages.create({
    ...params,
    stream: true,
  });
  process.stdout.write("\n助手: ");
  const message = await streamToMessage(stream, (t) => {
    process.stdout.write(t);
  });
  process.stdout.write("\n\n");
  return message;
}

// 在 runAgentTurn 内将 client.messages.create 替换为 createMessageStreaming
```

完整逻辑仍为：

1. `createMessageStreaming` 得到 `message`。
2. `history.push({ role: "assistant", content: message.content })`。
3. 若有 `tool_use`，执行工具，`history.push(tool_result)`，再次流式请求，直到无工具。

## 终端体验对比

[Mermaid 图表 — 请在在线版本中查看]

## 调试建议

1. 临时 `console.error(JSON.stringify(event))` 打印事件，确认 `content_block` 索引与 `text/tool` 交错顺序。



2. `tool_use` 的 `input_json` 可能分多片到达，必须拼接完再 `JSON.parse`。
3. 若模型先输出说明性文字再输出 `tool_use`，用户会先看到流式文本，再进入工具分支，体验与 Claude Code 类似。

## 与 Lab 3 合并

将 `createMessageStreaming` 与 `PermissionGuard.wrapExecute` 放在同一 `runAgentTurn` 即可；无额外冲突。

## 小结

| 要点  | 内容                                   |
|-----|--------------------------------------|
| UX  | <code>stdout.write</code> 逐字输出       |
| 正确性 | 累积完整 <code>content</code> 再写 history |
| 工具  | <code>input_json_delta</code> 拼接     |

## 非流式与流式统一封装（可选）

若希望同一套 `runAgentTurn` 在配置中切换模式，可提取：

```
type CreateMsg = typeof Anthropic.prototype.messages.create;

async function callModel(
  client: Anthropic,
  params: Anthropic.Messages.MessageCreateParamsNonStreaming,
  streaming: boolean
): Promise<Anthropic.Message> {
  if (!streaming) {
    return client.messages.create(params);
  }
  const stream = await client.messages.create({ ...params, stream: true });
  process.stdout.write("\n助手: ");
  const message = await streamToMessage(stream, (t) => process.stdout.write(t));
  process.stdout.write("\n\n");
  return message;
}
```

注意：`streamToMessage` 返回的本地对象在 `id / usage` 上与真实 REST 响应可能略有差异，但 `content` 结构足以驱动 `extractToolUses` 与 history 追加。

## SDK 版本与事件类型

`MessageStreamEvent` 的联合成员名随 `@anthropic-ai/sdk` 小版本可能增减。若 TypeScript 报错：

1. 在 `node_modules/@anthropic-ai/sdk/resources/messages` 中搜索 `MessageStreamEvent`。
2. 为 `content_block_delta` 分支增加对新 `delta.type` 的 `switch` 分支或 `default` 吞掉未知事件。
3. 升级 SDK 时运行 `npm run typecheck`，用编译器引导你补齐处理逻辑。

## 错误处理与重试（教学提示）

流中途若网络断开，`for await` 会抛错。可在循环外层包一层：

```
async function withRetry<T>(fn: () => Promise<T>, times = 2): Promise<T> {
  let last: unknown;
  for (let i = 0; i <= times; i++) {
    try {
      return await fn();
    } catch (e) {
      last = e;
      if (i === times) break;
      await new Promise((r) => setTimeout(r, 500 * (i + 1)));
    }
  }
  throw last;
}
```

生产环境还需区分 429 / 5xx 与 4xx 业务错误（Anthropic SDK 会抛出带 `status` 的错误对象）。

## 自检清单

- [ ] 纯文本对话时，终端逐字出现且无乱码。
- [ ] 含 `tool_use` 时，流结束后仍能解析出完整 `name` 与 `input`。
- [ ] 多轮工具调用中，每一轮 assistant `content` 均已写入 `history`。

## 下一 Lab

[Lab 5: 构建 MCP 服务器](#) 将用 `@modelcontextprotocol/sdk` 实现 stdio MCP，并把远程工具挂入 Agent。

# Lab 5: 构建 MCP 服务器（stdio）并在 Agent 中使用

系列：Claude Code 完全指南 V2 · 第 19 篇实战 Lab

前置：完成 [Lab 2](#)；理解工具在 API 中的声明方式。

## 学习目标

1. 使用 `@modelcontextprotocol/sdk` 搭建一个最小 MCP Server，通过 stdio 与客户端通信。
2. 实现一个演示工具 `get_weather`（可返回模拟 JSON，不必调用真实气象 API）。
3. 在 Node Agent 中使用 `@anthropic-ai/sdk` 之外的 MCP 客户端（或用 `child_process` 启动 server 并用 MCP 客户端库连接）。本 Lab 采用：独立 `mcp-server` 子包 + Agent 内 `@modelcontextprotocol/sdk` Client + `StdioClientTransport`。
4. 将 MCP 暴露的工具动态合并进 `messages.create` 的 `tools` 列表，并把执行路由到 MCP `callTool`。

## 架构总览

[Mermaid 图表 — 请在在线版本中查看]

## 步骤 1: 工作区布局

```
lab5-mcp/
├── package.json
├── tsconfig.json
├── src/
│   ├── agent/
│   │   └── main.ts
│   └── mcp-server/
│       └── index.ts
```

根 `package.json` :

```
{
  "name": "lab5-mcp",
  "type": "module",
  "scripts": {
    "mcp": "tsx src/mcp-server/index.ts",
    "agent": "tsx src/agent/main.ts"
  },
  "dependencies": {
    "@anthropic-ai/sdk": "^0.39.0",
    "@modelcontextprotocol/sdk": "^1.12.0",
    "zod": "^3.25.0"
  },
  "devDependencies": {
    "@types/node": "^22.0.0",
    "tsx": "^4.19.0",
    "typescript": "^5.7.0"
  }
}
```

## 步骤 2: MCP Server (stdio)

当前 npm 上的 `@modelcontextprotocol/sdk` (v1.x) 推荐使用高层 `McpServer` + `registerTool`，并需安装 peer 依赖 `zod` (可与 SDK 一样使用 `zod/v4` 或 `zod/v3`，见 SDK README)。

`src/mcp-server/index.ts` :

```
#!/usr/bin/env node
import { McpServer } from "@modelcontextprotocol/sdk/server/mcp.js";
import { StdioServerTransport } from "@modelcontextprotocol/sdk/server/stdio.js";
import * as z from "zod/v4";

const server = new McpServer({ name: "weather-demo", version: "0.1.0" });

server.registerTool(
  "get_weather",
  {
    description: "Get fake weather for a city (demo only).",
    inputSchema: {
      city: z.string().describe("City name"),
    },
  },
  async ({ city }) => {
    const payload = {
      city,
      unit: "C",
      condition: "sunny",
      temperature: 22,
      note: "This is simulated data for Lab 5.",
    };
    return {
      content: [{ type: "text", text: JSON.stringify(payload) }],
    };
  }
);

async function main() {
  const transport = new StdioServerTransport();
  await server.connect(transport);
  console.error("MCP weather server (stdio) ready.");
}

main().catch((e) => {
  console.error(e);
  process.exit(1);
});
```

**注意：**MCP server 进程应只通过 `stdout` 写协议数据；调试请用 `console.error` 打到 `stderr`，避免破坏 JSON-RPC 流。

若你使用的是已拆分的 **v2 预览包** ( `@modelcontextprotocol/server` )，请将 `import` 改为该包的 `server/mcp` 与 `server/stdio` 路径，逻辑保持一致即可。

## 步骤 3: Agent 连接 MCP 并桥接到 Claude

---

`src/agent/main.ts` (教学版: 启动时 `spawn` 子进程运行同一套 `tsx mcp-server`, 再用 `StdioClientTransport` 连接; 为缩短篇幅, 内置工具可省略, 仅演示 MCP):

```
import { spawn } from "node:child_process";
import * as path from "node:path";
import { fileURLToPath } from "node:url";
import Anthropic from "@anthropic-ai/sdk";
import { Client } from "@modelcontextprotocol/sdk/client/index.js";
import { StdioClientTransport } from "@modelcontextprotocol/sdk/client/stdio.js";

const MODEL = "claude-sonnet-4-20250514";

const __filename = fileURLToPath(import.meta.url);
const __dirname = path.dirname(__filename);
const root = path.resolve(__dirname, "../..");

async function startMcpClient(): Promise<{
  client: Client;
  transport: StdioClientTransport;
}> {
  const serverPath = path.join(root, "src/mcp-server/index.ts");
  const transport = new StdioClientTransport({
    command: "npx",
    args: ["tsx", serverPath],
    cwd: root,
  });
  const client = new Client({ name: "lab5-agent", version: "0.1.0" });
  await client.connect(transport);
  return { client, transport };
}

function mcpToolToAnthropic(t: {
  name: string;
  description?: string;
  inputSchema?: unknown;
}) {
  return {
    name: t.name,
    description: t.description ?? "",
    input_schema: t.inputSchema as Record<string, unknown>,
  };
}

async function main() {
  const apiKey = process.env.ANTHROPIC_API_KEY;
  if (!apiKey) throw new Error("ANTHROPIC_API_KEY required");

  const { client: mcp, transport } = await startMcpClient();
  const listed = await mcp.listTools();
  const anthropicTools = listed.tools.map(mcpToolToAnthropic);

  const anthropic = new Anthropic({ apiKey });

  const userPrompt =
    "请用 get_weather 查询 Beijing 的天气，并用一句话中文总结。";

  let messages: Anthropic.Messages.MessageParam[] = [
    { role: "user", content: userPrompt },
  ];
}
```

```
while (true) {
  const msg = await anthropic.messages.create({
    model: MODEL,
    max_tokens: 1024,
    tools: anthropicTools,
    messages: messages,
  });

  messages.push({ role: "assistant", content: msg.content });

  const toolUses = msg.content.filter((b) => b.type === "tool_use");
  if (toolUses.length === 0) {
    const text = msg.content
      .filter((b) => b.type === "text")
      .map((b) => b.text)
      .join("\n");
    console.log("助手:", text);
    break;
  }

  const results: Anthropic.Messages.ToolResultBlockParam[] = [];
  for (const u of toolUses) {
    const res = await mcp.callTool({
      name: u.name,
      arguments: (u.input ?? {}) as Record<string, unknown>,
    });
    const text = res.content
      .filter((c) => c.type === "text")
      .map((c) => c.text)
      .join("\n");
    results.push({
      type: "tool_result",
      tool_use_id: u.id,
      content: text,
    });
  }
  messages.push({ role: "user", content: results });
}

await mcp.close();
await transport.close();
}

main().catch((e) => {
  console.error(e);
  process.exit(1);
});
```

---

## 运行方式

---

终端 1 (可选手动调试 server):



```
npm run mcp
```

运行 Agent（会自动拉起子进程 server）：

```
export ANTHROPIC_API_KEY=...  
npm run agent
```

## MCP 与 Anthropic tools 字段映射

[Mermaid 图表 — 请在在线版本中查看]

## 常见问题

1. `npx tsx` 找不到：在 `lab5-mcp` 根目录先 `npm install`。
2. Windows 路径：`StdioClientTransport` 的 `command` 可改为 `node` + 预编译 `dist`。
3. 与 Claude Code 对比：真实 Claude Code 通过内置 MCP 配置管理多 server；本 Lab 为单 server 硬编码演示。

## 扩展练习

- 把 Lab 2 的 `read_file` 与 MCP `get_weather` 合并到同一 `tools` 数组，路由器根据 `name` 前缀或注册表分流。
- 增加 `resources` / `prompts` 能力（SDK 支持），体验完整 MCP。

## 下一 Lab

[Lab 6: 多 Agent 协调](#) 将实现主从 Agent：子 Agent 只读探索，主 Agent 决策。

# Lab 6: 多 Agent 协调 (主 Agent + 只读子 Agent)

系列: Claude Code 完全指南 V2 · 第 19 篇实战 Lab

前置: 完成 [Lab 2](#); 理解 `messages` 与工具闭环。

## 学习目标

1. 实现 主 Agent (Orchestrator) 与 子 Agent (Explorer) 两个独立调用链 (两次 `messages.create` 序列, 或两套 system 提示)。
2. 约束 子 Agent 仅使用只读工具 (`read_file`、`list_directory`), 并在 system prompt 中明确禁止写操作与 shell。
3. 主 Agent 接收 用户原始目标, 先委派子任务 (例如「总结 src 目录结构」), 再将 子 Agent 的最终文本报告 作为上下文, 由主 Agent 给出决策与面向用户的答案。
4. 用清晰的 TypeScript 类型表达「子任务输入 / 输出」, 便于后续替换为并行子 Agent。

## 角色与数据流

[Mermaid 图表 — 请在在线版本中查看]

## 设计说明

- 子 Agent 使用较短 `max_tokens`, 专用 system: 仅输出结构化 Markdown 报告, 不直接对用户说话。
- 主 Agent 拥有完整工具集 (若已做 Lab 3/5, 可只给主 Agent `run_shell` / MCP, 而子 Agent 工具列表过滤)。
- 本 Lab 串行执行: 先跑完子 Agent, 再跑主 Agent; Lab 扩展可做 `Promise.all` 多子任务。

## 完整示例代码

`src/multi-agent.ts` :

```

import Anthropic from "@anthropic-ai/sdk";
import type { MessageParam, Tool } from "@anthropic-ai/sdk/resources/messages";
import * as path from "node:path";
import { ToolRegistry } from "./registry.js";
import { createReadFileTool } from "./tools/read-file.js";
import { createListDirectoryTool } from "./tools/list-dir.js";

const MODEL = "claude-sonnet-4-20250514";

const SUB_SYSTEM = `你是代码探索子代理。你只能使用提供的只读工具。
禁止建议写文件、执行 shell、或任何副作用操作。
输出一份简洁的 Markdown 报告：目录结构要点、关键文件、与任务相关的发现。`;

const MAIN_SYSTEM = `你是主代理。你会先收到子代理的探索报告，再结合用户目标给出最终方案。
你可以使用完整工具集（若可用）。回答用户时使用中文，条理清晰。`;

/**
 * 子 Agent：仅使用 readOnlyTools；工具体通过 registry 执行（与主循环一致）。
 */
async function runSubAgentWithRegistry(
  client: Anthropic,
  task: string,
  registry: ToolRegistry,
  readOnlyTools: Tool[]
): Promise<string> {
  const messages: MessageParam[] = [
    { role: "user", content: `探索任务:\n${task}` },
  ];

  for (let turn = 0; turn < 8; turn++) {
    const msg = await client.messages.create({
      model: MODEL,
      max_tokens: 2048,
      system: SUB_SYSTEM,
      tools: readOnlyTools,
      messages,
    });

    messages.push({ role: "assistant", content: msg.content });

    const uses = msg.content.filter((b) => b.type === "tool_use");
    if (uses.length === 0) {
      return msg.content
        .filter((b) => b.type === "text")
        .map((b) => b.text)
        .join("\n");
    }

    const results = [];
    for (const u of uses) {
      const def = registry.get(u.name);
      let out: string;
      if (!def) out = `unknown tool ${u.name}`;
      else {
        try {
          out = await def.execute((u.input ?? {}) as Record<string, unknown>);
        }
      }
    }
  }
}

```

```

        } catch (e) {
            out = e instanceof Error ? e.message : String(e);
        }
    }
    results.push({
        type: "tool_result" as const,
        tool_use_id: u.id,
        content: out,
    });
}
messages.push({ role: "user", content: results });
}
return "(subagent) max turns exceeded";
}

async function runMainAgent(
    client: Anthropic,
    userGoal: string,
    subReport: string,
    mainTools: Tool[],
    registry: ToolRegistry
): Promise<void> {
    const messages: MessageParam[] = [
        {
            role: "user",
            content: `用户目标:\n${userGoal}\n\n--- 子代理探索报告 ---\n${subReport}`,
        },
    ];

    for (let turn = 0; turn < 10; turn++) {
        const msg = await client.messages.create({
            model: MODEL,
            max_tokens: 4096,
            system: MAIN_SYSTEM,
            tools: mainTools,
            messages,
        });

        messages.push({ role: "assistant", content: msg.content });

        const text = msg.content
            .filter((b) => b.type === "text")
            .map((b) => b.text)
            .join("\n");
        if (text) console.log("\n[主代理]\n" + text + "\n");

        const uses = msg.content.filter((b) => b.type === "tool_use");
        if (uses.length === 0) return;

        const results = [];
        for (const u of uses) {
            const def = registry.get(u.name);
            let out: string;
            if (!def) out = `unknown tool ${u.name}`;
            else {
                try {
                    out = await def.execute((u.input ?? {}) as Record<string, unknown>);
                }
            }
        }
    }
}

```

```
        } catch (e) {
            out = e instanceof Error ? e.message : String(e);
        }
    }
    results.push({
        type: "tool_result" as const,
        tool_use_id: u.id,
        content: out,
    });
}
messages.push({ role: "user", content: results });
}
}

async function main() {
    const apiKey = process.env.ANTHROPIC_API_KEY;
    if (!apiKey) throw new Error("ANTHROPIC_API_KEY required");

    const root = path.resolve(process.cwd());
    const registry = new ToolRegistry();
    registry.register(createReadFileTool(root));
    registry.register(createListDirectoryTool(root));

    const allTools = registry.listDefinitions();
    const readOnlyTools = allTools; // 本例仅只读工具; 若有 run_shell, 请 filter 掉再给子 Agent

    const client = new Anthropic({ apiKey });

    const userGoal = "我了解这个项目的目录与 package.json 里定义了哪些脚本。";
    const subTask = `请在项目根目录下列出顶层文件, 并读取 package.json 摘要关键字段。工作区: ${root}`;

    console.log("---- 子代理探索中 ----\n");
    const subReport = await runSubAgentWithRegistry(
        client,
        subTask,
        registry,
        readOnlyTools
    );
    console.log("---- 子代理报告 ----\n", subReport, "\n");

    console.log("---- 主代理答复 ----\n");
    await runMainAgent(client, userGoal, subReport, allTools, registry);
}

main().catch((e) => {
    console.error(e);
    process.exit(1);
});
```

## 工具权限分离（Mermaid）

[Mermaid 图表 — 请在在线版本中查看]

实现方式：`readOnlyTools = allTools.filter(t => READ_SET.has(t.name))`。

## 与 Claude Code 的类比

| 概念  | Claude Code   | 本 Lab                                    |
|-----|---------------|------------------------------------------|
| 子代理 | Explore / 子任务 | <code>runSubAgentWithRegistry</code>     |
| 主代理 | 主对话 + 决策      | <code>runMainAgent</code>                |
| 隔离  | 进程 / 权限模型     | 不同 <code>tools</code> 数组 + system prompt |

## 子任务路由策略（何时启用子 Agent）

不必每笔用户消息都跑子 Agent。常见启发式：

| 用户意图           | 建议                                |
|----------------|-----------------------------------|
| 「这个仓库是干什么的」    | 启用子 Agent 做目录 + README 扫描         |
| 「把 foo 改成 bar」 | 主 Agent 直接处理；子 Agent 可选做一次只读确认    |
| 「解释某文件第 N 行」   | 主 Agent <code>read_file</code> 即可 |

实现上可在 `main.ts` 增加轻量分类：先用 **小模型或关键词**（教学）判断 `needsExplorer`，再调用 `runSubAgentWithRegistry`。

[Mermaid 图表 — 请在在线版本中查看]

## 陷阱与建议

- 不要把子 Agent 的 `messages` 与主 Agent 混在一个 history，除非你做显式角色标注；否则模型易混淆。
- 子 Agent 输出可能过长：可要求「最多 20 行」或在主 Agent 侧二次摘要（衔接 Lab 7）。

3. **成本**：两次调用链  $\approx$  双倍 token；重要场景再启用子 Agent。

## 下一 Lab

[Lab 7: 上下文压缩](#) 将在历史过长时自动摘要旧消息。

# Lab 7：实现上下文压缩（摘要替换旧历史）

**系列**：Claude Code 完全指南 V2 · 第 19 篇实战 Lab  
**前置**：完成 [Lab 1](#)~[Lab 2](#)；理解 `MessageParam[]` 的维护。

## 学习目标

- 在每次用户发话前（或每 N 轮后）评估上下文长度；本 Lab 用 **字符数近似 token**（教学用），避免引入额外 tokenizer 依赖。
- 当总字符数超过阈值 `MAX_CHARS` 时，调用 Claude 生成一段**不丢失关键决策与未完成事项**的对话摘要。
- 用摘要 **替换** 旧消息，但 **保留最近 `KEEP_LAST` 条「轮次」**（用户消息 + 其后 assistant/tool 链算一轮，实现时可简化为保留数组尾部 K 个元素）。
- 压缩后的 history 仍以合法 `MessageParam[]` 形式继续调用 `messages.create`。

## 压缩策略流程图

[Mermaid 图表 — 请在在线版本中查看]

## 近似 Token 与阈值

真实场景应使用 `tiktoken` 或厂商 tokenizer；为保持依赖最小，本 Lab：

```
export function approxChars(messages: import("@anthropic-ai/sdk/resources/messages").Message[]): number {
  return JSON.stringify(messages).length;
}
```

经验系数: `chars / 4 ≈ tokens` (英文); 中文偏差更大, 可把 `MAX_CHARS` 调低。

---

## 步骤 1: `compactMessages` 实现

---

创建 `src/context/compaction.ts` :



```
import Anthropic from "@anthropic-ai/sdk";
import type { MessageParam } from "@anthropic-ai/sdk/resources/messages";

const MODEL = "claude-sonnet-4-20250514";

export function approxChars(messages: MessageParam[]): number {
  return JSON.stringify(messages).length;
}

/**
 * 保留尾部 keepLast 条消息，将其余前缀压缩为一条 user 摘要消息。
 */
export async function maybeCompact(
  client: Anthropic,
  messages: MessageParam[],
  options: { maxChars: number; keepLast: number }
): Promise<MessageParam[]> {
  if (approxChars(messages) <= options.maxChars) return messages;
  if (messages.length <= options.keepLast) return messages;

  const head = messages.slice(0, -options.keepLast);
  const tail = messages.slice(-options.keepLast);

  const summaryPrompt: MessageParam[] = [
    {
      role: "user",
      content: `请将下列对话历史压缩为简洁中文摘要，保留：
- 用户目标与约束
- 已执行的重要工具结果（路径、结论）
- 未完成的行动项与错误信息

原始消息(JSON): \n${JSON.stringify(head).slice(0, 120_000)}\`,
    },
  ];

  const sum = await client.messages.create({
    model: MODEL,
    max_tokens: 1024,
    messages: summaryPrompt,
  });

  const text = sum.content
    .filter((b) => b.type === "text")
    .map((b) => b.text)
    .join("\n");

  const summaryMessage: MessageParam = {
    role: "user",
    content: `[上下文已压缩] 以下为此前对话的摘要，请在此基础上继续协助用户： \n\n${text}`,
  };

  return [summaryMessage, ...tail];
}
```

**注意：** `JSON.stringify(head).slice(0, 120_000)` 防止摘要请求本身过大；生产应分块或多段摘要。

## 步骤 2：接入主循环

在每次 `history.push(用户输入)` 之后、调用模型 之前：

```
import { approxChars, maybeCompact } from "../context/compaction.js";

// ...
history.push({ role: "user", content: line });

if (approxChars(history) > 50_000) {
  console.error("[context] 压缩中...");
  history = await maybeCompact(client, history, {
    maxChars: 40_000,
    keepLast: 6,
  });
}

await runAgentTurn(client, history, tools, registry);
```

`keepLast: 6` 表示保留最近 6 条 `MessageParam`（简化版）；更严谨可按「轮」切分。

## 保留最近 N 轮的 Mermaid 说明

[Mermaid 图表 — 请在在线版本中查看]

## 风险与缓解

| 风险                                        | 缓解                                                     |
|-------------------------------------------|--------------------------------------------------------|
| 摘要丢失 <code>tool_use/tool_result</code> 配对 | 保留段内应包含 <b>未完成</b> 工具链；或压缩前强制结束 <code>open tool</code> |
| 摘要请求超上下文                                  | 对 <code>head</code> 分段多次摘要                             |
| 字符近似不准                                    | 换用官方 tokenizer                                         |

## 与 Claude Code 的「Compaction」

Claude Code 在上下文将满时会做结构化压缩与检查点；本 Lab 用「单条摘要 user 消息 + 尾部原样」展示最小可行实现。

### 扩展练习

1. 压缩时剔除纯寒暄轮次。
2. 将摘要存盘为 `~/.mini-agent/sessions/id.json`，支持断点续聊。
3. 使用 `claude-3-5-haiku` 等更小模型做摘要以节省成本。

### 按「轮次」切分（进阶）

简化版按「消息条数」保留尾部，在工具密集对话中可能把同一轮 `assistant(tool_use)` 与 `user(tool_result)` 切开。更稳妥做法是：

1. 将 `MessageParam[]` 从左扫描，按「用户一条 user（非 tool\_result）」作为一轮起点。
2. 其后直到下一条「用户新意图」之前的 `assistant / user(tool_result)` 都归入该轮。
3. 压缩时永远保留完整最近 K 轮，而不是 K 条消息。

[Mermaid 图表 — 请在在线版本中查看]

伪代码：

```
function splitIntoTurns(messages: MessageParam[]): MessageParam[][] {
  const turns: MessageParam[][] = [];
  let cur: MessageParam[] = [];
  for (const m of messages) {
    if (m.role === "user" && cur.length > 0 && !isOnlyToolResults(m)) {
      turns.push(cur);
      cur = [];
    }
    cur.push(m);
  }
  if (cur.length) turns.push(cur);
  return turns;
}

function isOnlyToolResults(m: MessageParam): boolean {
  return (
    Array.isArray(m.content) &&
    m.content.length > 0 &&
    m.content.every((b) => typeof b === "object" && b && "type" in b && b.type === "tool_res
  );
}
```

将 `maybeCompact` 中的 `head` / `tail` 改为基于 `splitIntoTurns` 的合并结果，可显著降低 `tool` 配对被破坏 的概率。

---

## 使用更小模型做摘要（可选代码）

---

在 `maybeCompact` 内把 `MODEL` 换为环境变量 `SUMMARY_MODEL`（如 `claude-3-5-haiku-20241022`），可降低成本：

```
const SUMMARY_MODEL =
  process.env.SUMMARY_MODEL ?? "claude-sonnet-4-20250514";

const sum = await client.messages.create({
  model: SUMMARY_MODEL,
  max_tokens: 1024,
  messages: summaryPrompt,
});
```

摘要任务不要求最强推理，Haiku 类模型通常足够。

---

## 压测脚本思路

---

1. 写脚本循环向 Agent 发送「再讲一个段子」类请求 30 次。
2. 观察 `approxChars(history)` 曲线与压缩触发次数。

3. 调整 `maxChars` / `keepLast`，在成本与遗忘率之间折中。

## 小结

| 项目 | 建议                                               |
|----|--------------------------------------------------|
| 度量 | 教学用 <code>JSON.stringify</code> 长度；上线换 tokenizer |
| 切分 | 优先按「轮」保留尾部                                       |
| 摘要 | 专用短模型 + 明确 bullet 保留项                            |

## 下一 Lab

[Lab 8: 完整项目整合](#) 将把 Lab 1~7 模块化为可配置工程并附运行检查清单。

# Lab 8：完整项目整合与架构对照

系列：Claude Code 完全指南 V2 · 第 19 篇实战 Lab

前置：建议按顺序完成 [Lab 1](#)~[Lab 7](#)。

## 学习目标

- 将 Agent Loop、工具注册表、权限、流式输出（可选开关）、MCP 客户端（可选）、多 Agent（可选）、上下文压缩 整合为单一可运行仓库。
- 使用 配置文件（`mini-agent.config.json`）管理模型名、阈值、是否启用 MCP 路径等，避免硬编码散落。
- 提供 `npm run build` / `npm start` 与最小「冒烟」验证步骤。
- 对照 真实 Claude Code 的架构维度做总结，明确教学项目与生产产品的差距。

## 推荐最终目录结构

[Mermaid 图表 — 请在在线版本中查看]

```
mini-agent-full/  
├─ package.json  
├─ tsconfig.json  
├─ mini-agent.config.json  
├─ src/  
│   ├─ main.ts  
│   ├─ config.ts  
│   ├─ agent/  
│   │   └─ loop.ts  
│   ├─ tools/  
│   │   ├─ registry.ts  
│   │   ├─ types.ts  
│   │   ├─ read-file.ts  
│   │   ├─ list-dir.ts  
│   │   └─ run-shell.ts  
│   ├─ permissions/  
│   │   ├─ guard.ts  
│   │   └─ policy.ts  
│   ├─ streaming/  
│   │   └─ accumulate.ts  
│   ├─ context/  
│   │   └─ compaction.ts  
│   ├─ mcp/  
│   │   └─ client.ts  
│   └─ multi/  
│       └─ orchestrator.ts  
└─ README.md (可选, 本教程不强制)
```

## mini-agent.config.json 示例

---

```
{
  "model": "claude-sonnet-4-20250514",
  "maxTokens": 4096,
  "streaming": true,
  "compaction": {
    "enabled": true,
    "maxChars": 48000,
    "keepLast": 8
  },
  "mcp": {
    "enabled": false,
    "serverScript": "src/mcp-server/index.ts"
  },
  "multiAgent": {
    "enabled": false
  }
}
```

---

**src/config.ts**

```
import * as fs from "node:fs";
import * as path from "node:path";

export interface AppConfig {
  model: string;
  maxTokens: number;
  streaming: boolean;
  compaction: {
    enabled: boolean;
    maxChars: number;
    keepLast: number;
  };
  mcp: {
    enabled: boolean;
    serverScript: string;
  };
  multiAgent: {
    enabled: boolean;
  };
}

const DEFAULTS: AppConfig = {
  model: "claude-sonnet-4-20250514",
  maxTokens: 4096,
  streaming: false,
  compaction: {
    enabled: true,
    maxChars: 50000,
    keepLast: 6,
  },
  mcp: { enabled: false, serverScript: "src/mcp-server/index.ts" },
  multiAgent: { enabled: false },
};

export function loadConfig(cwd: string): AppConfig {
  const p = path.join(cwd, "mini-agent.config.json");
  if (!fs.existsSync(p)) return DEFAULTS;
  const raw = JSON.parse(fs.readFileSync(p, "utf8")) as Partial<AppConfig>;
  return { ...DEFAULTS, ...raw, compaction: { ...DEFAULTS.compaction, ...raw.compaction }, mcp: { ...DEFAULTS.mcp, ...raw.mcp }, multiAgent: { ...DEFAULTS.multiAgent, ...raw.multiAgent } };
}
```



## src/main.ts (整合入口骨架)

```
import * as readline from "node:readline/promises";
import { stdin as input, stdout as output } from "node:process";
import * as path from "node:path";
import Anthropic from "@anthropic-ai/sdk";
import { loadConfig } from "./config.js";
import { createAgentLoop } from "./agent/loop.js";

async function main() {
  const apiKey = process.env.ANTHROPIC_API_KEY;
  if (!apiKey) {
    console.error("缺少 ANTHROPIC_API_KEY");
    process.exit(1);
  }
  const cwd = process.cwd();
  const cfg = loadConfig(cwd);
  const client = new Anthropic({ apiKey });
  const rl = readline.createInterface({ input, output });

  const loop = await createAgentLoop({
    client,
    cfg,
    workspaceRoot: path.resolve(cwd),
    rl,
  });

  console.log("Mini Claude-Agent 已启动 (Lab8 整合版)。exit 退出。\\n");
  while (true) {
    const line = (await rl.question("你: ")).trim();
    if (!line) continue;
    if (/^(exit|quit)$/i.test(line)) break;
    await loop.handleUserLine(line);
  }
  rl.close();
}

main().catch((e) => {
  console.error(e);
  process.exit(1);
});
```

## src/agent/loop.ts (模块职责说明)

`createAgentLoop` 应内部完成：

1. 初始化 `ToolRegistry`，注册 `read_file` / `list_directory` / `run_shell` (可选)。
2. 若 `cfg.mcp.enabled`，按 Lab 5 启动 MCP 并把工具并入 `tools`。
3. `PermissionGuard` 包装执行 (Lab 3)。

4. 若 `cfg.compaction.enabled`，在模型调用前 `maybeCompact` (Lab 7)。
5. 若 `cfg.multiAgent.enabled`，对复杂查询先走 `orchestrator` (Lab 6)。
6. 根据 `cfg.streaming` 选择 `messages.create` 或 `stream: true` + `streamToMessage` (Lab 4)。

完整实现建议直接把 Lab 2~4 的 `runAgentTurn` 抽成函数，再逐段插入上述分支；为控制篇幅，此处不重复粘贴数百行，读者应以自己 Lab 代码为「源码真值」。

---

## `package.json` 脚本

---

```
{
  "scripts": {
    "build": "tsc -p .",
    "start": "tsx src/main.ts",
    "start:dist": "node dist/main.js",
    "typecheck": "tsc -p . --noEmit"
  }
}
```

---

## 运行测试（冒烟清单）

---

[Mermaid 图表 — 请在在线版本中查看]

1. `npm run typecheck`：无 TypeScript 错误。
2. 工具：能列出目录、读取小文件。
3. 权限：`run_shell` + 黑名单命令应拒绝。
4. 配置热读：修改 `mini-agent.config.json` 后重启进程验证。

---

## 与真正 Claude Code 架构对比

---

[Mermaid 图表 — 请在在线版本中查看]

| 维度  | 教学 Mini-Agent    | Claude Code               |
|-----|------------------|---------------------------|
| 上下文 | 单文件 history + 摘要 | 多子系统、持久化会话、产品级 compaction |
| 工具  | 自注册 + 可选 MCP     | 丰富内置能力 + 生态 MCP           |
| 安全  | 路径校验 + 黑名单 + 确认  | 企业策略、审计、沙箱执行              |
| 体验  | 终端 + 可选流式        | 编辑器内 diff、应用编辑、多模态        |
| 工程  | 单进程演示            | 版本化协议、兼容矩阵、规模化测试          |

## 结语

八个 Lab 串联了「能对话 → 能调用工具 → 能控权 → 能流式 → 能接 MCP → 能分工 → 能控长度 → 能工程化」的完整学习路径。建议你在整合时**每合并一个 Lab 就提交一次 git**，便于回滚与对比。

恭喜你完成 Claude Code 完全指南 V2 · 第 19 篇实战 Lab 全部内容。

## 系列索引

- [Lab 1: 最简 Agent Loop](#)
- [Lab 2: 工具注册与执行](#)
- [Lab 3: 权限控制](#)
- [Lab 4: 流式响应](#)
- [Lab 5: MCP 服务器](#)
- [Lab 6: 多 Agent](#)
- [Lab 7: 上下文压缩](#)
- **Lab 8: 完整整合 (本文)**

# 第20篇：总结与展望

## 第 20 篇 · 总结与展望（一）AI Agent 的壁垒在哪里

**本篇定位：**在通读全书约 20 篇、200+ 节之后，把「Agent 到底难在哪」从感性体验拉回到工程现实。核心论点只有一句：**AI Agent 约 90% 的工作量落在 AI 模型之外。**

### 1. 开篇：体验好 ≠ 模型强

许多团队第一次接触「编程 Agent」时，会把成败归因到「Prompt 写得好不好」。短期演示里，这往往成立；但一旦进入真实仓库、真实权限、真实 CI 与真实评审流程，**模型能力的天花板很快让位给系统能力的天花板。**

[Mermaid 图表 — 请在在线版本中查看]

表 20-1：常见误判 vs 更贴近事实的解释

| 现象                | 易误判原因 | 更接近事实的解释        |
|-------------------|-------|-----------------|
| Agent 经常「改错文件」    | 模型不聪明 | 索引/检索/工作区边界未治理  |
| Agent 反复读大文件      | 模型浪费  | 上下文策略与缓存未设计     |
| Agent 乱执行命令       | 模型不听话 | 权限模型与确认流缺失      |
| Agent 输出看似正确但不可合并 | 模型幻觉  | 测试/类型检查/评审未嵌入闭环 |

### 2. 核心论点：「90% 在 AI 之外」意味着什么

把一次「从需求到可合并 PR」的旅程拆开，你会发现模型主要负责的是**语言层面的推理与生成**；而下面这些工作，几乎全部由**产品与工程底座**承担：

- **任务分解与状态机：**何时读、何时写、何时停、何时问人。
- **工具契约：**每个工具输入输出的 schema、错误语义、幂等与重试。

- **权限与审计**：谁能动什么、动了什么能否追溯。
- **上下文经济学**：哪些进窗口、如何压缩、如何命中缓存。
- **人机协同**：卡点、回滚、代码评审、发布门禁。

[Mermaid 图表 — 请在在线版本中查看]

表 20-2：工作量粗粒度拆分（教学用示意，非精确测量）

| 范畴         | 典型活动                | 相对占比（示意） |
|------------|---------------------|----------|
| 系统与产品工程    | 工具、权限、上下文、UI/CLI 流程 | ~45%     |
| 质量与流程      | 测试、CI、评审模板、回滚策略     | ~25%     |
| 数据与评估      | 日志、轨迹、回归集、对比实验      | ~15%     |
| 模型与 Prompt | 指令、样例、约束、风格         | ~15%     |

教学目的：**不要把「Agent 项目」当成「调模型项目」**。模型是发动机，但整车能不能上路，取决于底盘、刹车与交规。

### 3. 「4756 个模块」的隐喻：复杂度来自组合爆炸

书中常以大型代码库为例说明：**行数与文件数本身不是壁垒，模块间耦合与隐式协议才是**。所谓「4756 个模块」在此作为教学隐喻，指向三件事：

1. **边界数量**：每一个可被 Agent 调用的能力，都是一条「对外协议」。
2. **失败面**：任意模块的模糊语义，都会在长链路里被放大成事故。
3. **演进成本**：没有版本化与兼容策略时，Agent 行为会「随小改小动而漂移」。

[Mermaid 图表 — 请在在线版本中查看]

表 20-3：模块膨胀时优先治理的「杠杆点」

| 杠杆点       | 为什么优先     | 典型抓手        |
|-----------|-----------|-------------|
| 工具 schema | 一处模糊，处处翻车 | 严格类型、示例、错误码 |
| 权限默认      | 默认过宽等于裸奔  | 白名单、分级、可审计  |
| 上下文入口     | 垃圾进窗口，贵且慢 | 索引、摘要、分层加载  |
| 可观测性      | 不能复盘就无法迭代 | 结构化日志、轨迹导出  |

## 4. 好体验不是「几段 Prompt 忽悠出来的」

高质量 Agent 体验通常具备以下特征，它们几乎都依赖工程而非话术：

- **可预测**：在相同约束下，行为稳定、可解释。
- **可中断**：用户随时能接管、回滚、改方向。
- **可验证**：改动能自动被测试/类型检查/静态规则捕获。
- **可审计**：关键动作有记录，便于追责与复盘。

[Mermaid 图表 — 请在在线版本中查看]

表 20-4：「Prompt 主义」 vs 「系统工程」对照

| 维度 | Prompt 主义陷阱 | 系统工程做法         |
|----|-------------|----------------|
| 安全 | 让模型「答应不乱来」  | 运行时强制策略与审计     |
| 质量 | 让模型「自我检查」   | CI + 测试 + 人类评审 |
| 成本 | 反复塞全文       | 缓存、分层、只读必要片段   |
| 协作 | 一个超级 Prompt | 角色拆分与交接协议      |

## 5. 你需要什么：制度、角色、卡点、Token 预算

### 5.1 把好行为写成制度

制度不是官僚文本，而是把「正确路径」变成默认路径：分支策略、提交规范、评审清单、发布窗口、回滚预案。Agent 在制度清晰的团队里更稳，因为环境信号一致、反馈回路短。

## 5.2 拆分「写代码 / 审代码」角色

教学上推荐把心智拆成两类责任（可由同人分阶段承担，也可由多 Agent 承担）：

- **实现者**：专注交付最小可验证改动。
- **审查者**：专注风险、边界、测试充分性与可维护性。

[Mermaid 图表 — 请在在线版本中查看]

## 5.3 层层卡点与工具治理

卡点不是刁难，而是把高风险动作暴露为显式决策：网络访问、写盘范围、执行命令、修改配置密钥等。工具治理的目标是：默认安全、逐步放权、全程可审计。

## 5.4 Token 当预算精打细算

把 Token 视作云资源预算：能缓存的别重读，能结构化的别啰嗦，能工具化的别长篇推理。长期成本与延迟往往由上下文策略决定，而不是由「模型是不是最强」决定。

表 20-5：Token 预算的「四类账户」

| 账户      | 花在哪   | 节省策略            |
|---------|-------|-----------------|
| 系统提示与规范 | 稳定规则  | 分层加载、版本化片段      |
| 仓库上下文   | 代码与文档 | 检索、摘要、相关文件集合    |
| 工具往返    | 输入输出  | schema 瘦身、分页、增量 |
| 对话历史    | 多轮澄清  | 压缩、里程碑摘要、结构化状态  |

## 6. Anthropic 的答卷：不能关笼子，也不能裸奔

业界常见两种极端：

- **关笼子**：能力阉割到几乎不可用，安全但创新停滞。
- **裸奔**：权限与流程缺位，短期爽、长期事故与合规风险。

更可持续的路线，是把它做成操作系统式底座：进程（会话）、系统调用（工具）、权限（capabilities）、文件系统（工作区）、审计（日志）、更新机制（版本与兼容）。

[Mermaid 图表 — 请在在线版本中查看]

表 20-6：底座能力清单（教学抽象）

| 能力域   | 用户可感知价值   | 工程要点       |
|-------|-----------|------------|
| 会话与循环 | 「能持续推进任务」 | 状态、重试、失败恢复 |
| 工具系统  | 「能真实改代码」  | 注册、校验、沙箱   |
| 权限    | 「敢用」      | 默认拒绝、分级授权  |
| 上下文   | 「快且准」     | 缓存、压缩、检索   |
| 观测    | 「能改好」     | 轨迹、指标、回归   |

## 7. 与全书脉络的衔接：从「会用」到「能建」

若你把前 19 篇看作「拆解一辆车的结构」，本篇要做的是提醒你：**造车厂的主要工时不在发动机广告词，而在供应链与质检**。下一节（20.2）将从产品形态维度对比主流工具；20.3 讨论趋势；20.4—20.5 落到个人与团队成长路径；20.6 收束全书。

[Mermaid 图表 — 请在在线版本中查看]

## 8. 本篇小结（背板）

- **壁垒主要在系统**：工具、权限、上下文、流程与观测，决定上限。
- **模块规模大是信号**：需要协议化、版本化与治理，而不是更大 Prompt。
- **体验来自制度与工程**：可预测、可验证、可审计比「会聊天」更重要。
- **Token 是预算**：治理上下文就是治理成本与延迟。
- **产品哲学**：在限制与放开之间，用「OS 底座」换取长期可持续。

## 9. 自测清单（读完本篇应能回答）

1. 为什么说「90% 在 AI 之外」并不贬低模型价值？
2. 举出三类「看起来像模型问题、其实是系统问题」的例子。
3. 「实现者 / 审查者」拆分解决的是什么组织难题？



4. 用你自己的项目举一个「应设卡点」的高风险工具调用。
5. 「OS 底座」与「裸奔 / 关笼子」各差在哪三个要素？

## 10. 延伸阅读钩子

- 对比维度：见 `02-comparison.md`。
- 趋势判断：见 `03-future.md`。
- 个人与团队行动项：见 `04-insights.md` 与 `05-learning-path.md`。
- 术语与命令速查：见附录 `glossary-en-zh.md`、`cheatsheet.md`。

## 11. 术语对齐（本篇）

| 英文/产品语           | 中文教学表述  |
|------------------|---------|
| Tool / MCP       | 工具与扩展协议 |
| Permission model | 权限模型    |
| Context window   | 上下文窗口   |
| Audit trail      | 审计轨迹    |

## 12. 常见反对意见与回应

| 反对意见          | 教学回应                  |
|---------------|-----------------------|
| 「我们团队小，不需要制度」 | 小团队更需要默认路径，否则知识在人不在系统 |
| 「卡点会降低效率」     | 卡点针对高风险动作；低风险应自动化     |
| 「最强模型可以解决一切」  | 模型不能替代运行时约束与合规要求      |

## 13. 最小行动建议（本周可做）

1. 画一张你当前 Agent 调用的工具清单，标注风险等级。
2. 写一份「审查者检查表」10 条，用于 PR 或自审。
3. 记录一次会话的 Token/耗时粗账，找最重的三类上下文来源。

## 14. 图表索引（本篇）

| 编号     | 类型        | 主题          |
|--------|-----------|-------------|
| 图 20-1 | flowchart | 表象层与系统层     |
| 图 20-2 | mindmap   | Agent 工程心智图 |
| 图 20-3 | graph TB  | 模块关系示意      |
| 图 20-4 | sequence  | 验证闭环        |
| 图 20-5 | flowchart | 实现者与审查者     |
| 图 20-6 | flowchart | 极端与 OS 底座   |
| 图 20-7 | timeline  | 全书进阶        |

## 15. 结语（过渡到 20.2）

当你能稳定地用 Agent 提效之后，下一步的自然问题是：**在工具丛林里如何选型与组合**。下一文件从架构、安全、扩展性、成本、上下文、多 Agent 与开源程度等维度，给出一张「多维对比」总表，便于你把本篇的抽象判断落到具体产品差异上。

文档版本：V2 教学稿 · 第 20 篇第 1 节 · 行数与结构服务于课堂与自学复盘。

# 20.2 深度对比：Claude Code、Cursor、GitHub Copilot Agent、Windsurf、Aider、Continue

**本节目标：**用同一张「多维评分表」对齐六类代表性形态，避免「谁广告多谁就好」的选型误区。下列评级为**教学用相对刻度（1—5）**，侧重架构与工程特征，不代表对任何商业产品的官方评价；真实选型请以你团队约束为准。

## 1. 对比对象如何界定

| 产品/形态                | 本节所指范围（教学界定）                               |
|----------------------|--------------------------------------------|
| Claude Code          | Anthropic 体系下的终端/Agent 式编程助手及其工具链生态        |
| Cursor               | 基于 VS Code 派生的 AI IDE，强调编辑器内 Agent 与补全     |
| GitHub Copilot Agent | 以 GitHub / VS Code 生态为中心的 Agent 能力（含托管侧协作） |
| Windsurf             | 强调 Flow/Agent 体验的 AI IDE 类产品（迭代快，以官方能力为准）  |
| Aider                | 开源终端型 pair programmer，强调 git 集成与补丁式改码      |
| Continue             | 开源 IDE 扩展，强调可插拔模型与本地/远程配置                  |

[Mermaid 图表 — 请在在线版本中查看]

说明：分区仅为帮助建立直觉；不同版本发布会导致边界漂移。精确坐标对比见下方总表。

## 2. 总表 A：架构与交互范式

| 维度     | Claude Code   | Cursor    | Copilot Agent | Windsurf   | Aider     | Continue |
|--------|---------------|-----------|---------------|------------|-----------|----------|
| 主战场    | 终端 + 仓库       | IDE 工作区   | IDE + GitHub  | IDE 工作区    | 终端 + git  | IDE 扩展   |
| 会话内核   | Agent 循环 + 工具 | 多模式 Agent | 生态集成 Agent    | Agent Flow | 对话 + 补丁   | 聊天 + 工具  |
| 项目表征   | 仓库级上下文        | 工作区索引     | 仓库/PR 语境      | 工作区索引      | 文件树 + git | 工作区文件    |
| 适合长任务链 | 强             | 强         | 中—强           | 强          | 中         | 中（视配置）   |
| 适合快速点改 | 中             | 强         | 强             | 强          | 强         | 强        |

[Mermaid 图表 — 请在在线版本中查看]

### 3. 总表 B：安全、权限与合规

| 维度       | Claude Code | Cursor | Copilot Agent  | Windsurf | Aider   | Continue |
|----------|-------------|--------|----------------|----------|---------|----------|
| 企业治理成熟度  | 高（视部署与策略）   | 高      | 高（GitHub 生态）   | 中—高      | 中（自管为主） | 中（自管为主）  |
| 数据驻留可控性  | 依赖方案        | 依赖方案   | 依赖 GitHub/租户策略 | 依赖方案     | 高（本地为主） | 高（可本地模型） |
| 密钥与秘文风险面 | 中           | 中      | 中              | 中        | 中—高     | 中        |
| 审计友好度    | 中—高         | 中—高    | 高              | 中        | 中       | 中        |

教学提示：安全不是「是否开源」单选题，而是 默认权限 + 可审计 + 可回滚 的组合题。

[Mermaid 图表 — 请在在线版本中查看]

### 4. 总表 C：扩展性（工具 / 协议 / 插件）

| 维度         | Claude Code | Cursor      | Copilot Agent | Windsurf | Aider     | Continue |
|------------|-------------|-------------|---------------|----------|-----------|----------|
| 工具扩展故事     | 强（生态工具链）    | 强（规则/命令/生态） | 强（平台集成）       | 强        | 中（脚本/工作流） | 强（开放配置）  |
| MCP 等协议态度  | 友好          | 友好          | 渐进            | 友好       | 视集成       | 友好       |
| 可脚本化       | 强           | 中—强         | 中             | 中—强      | 强         | 中        |
| 自建「第二大脑」成本 | 中           | 中           | 中—高           | 中        | 低—中       | 低—中      |

## 5. 总表 D：成本与 Token 经济学

| 维度       | Claude Code  | Cursor   | Copilot Agent | Windsurf | Aider       | Continue |
|----------|--------------|----------|---------------|----------|-------------|----------|
| 计费透明度    | 中—高          | 中（套餐/用量） | 中（订阅/组织）      | 中        | 高（自带KEY）    | 高（自带KEY） |
| 上下文「烧钱点」 | 仓库+工具往返      | 索引+多文件编辑 | PR/对话+代码      | 多步Flow   | 多轮+大文件      | 视模型与检索   |
| 成本优化抓手   | 缓存/分层/工具输出瘦身 | 规则/排除/模式 | 工作项拆分         | Flow 设计  | map/repo 策略 | 本地模型/路由  |

[Mermaid 图表 — 请在在线版本中查看]

## 6. 总表 E：上下文管理（检索、压缩、缓存）

| 维度       | Claude Code | Cursor | Copilot Agent | Windsurf | Aider | Continue |
|----------|-------------|--------|---------------|----------|-------|----------|
| 仓库级理解    | 强           | 强      | 强             | 强        | 中—强   | 中—强      |
| 「只读该读」能力 | 强（依赖实现）     | 强      | 中—强           | 强        | 中     | 中—强      |
| 多文件一致性编辑 | 强           | 强      | 强             | 强        | 强     | 中—强      |
| 可插拔检索    | 中—高         | 高      | 中—高           | 中—高      | 中     | 高        |

表 20–2–1：上下文策略检查清单（通用）

| 检查项 | 通过标准（教学）        |
|-----|-----------------|
| 相关性 | 变更点 2-hop 内文件优先 |
| 重复读 | 同文件同版本不反复塞全文    |
| 秘文  | 秘文不进窗口或已脱敏      |
| 里程碑 | 长任务有阶段性摘要状态     |

## 7. 总表 F：多 Agent / 多角色协作

| 维度          | Claude Code | Cursor | Copilot Agent | Windsurf | Aider | Continue |
|-------------|-------------|--------|---------------|----------|-------|----------|
| 一等公民多 Agent | 中—强         | 中—强    | 中             | 中—强      | 弱—中   | 弱—中      |
| 子任务委派模型     | 有           | 有      | 有             | 有        | 主要靠人  | 主要靠配置    |
| 适合「实现/审查」拆分 | 强           | 强      | 中—强           | 强        | 中     | 中        |

[Mermaid 图表 — 请在在线版本中查看]

## 8. 总表 G：开源程度与可 fork 性

| 维度       | Claude Code | Cursor | Copilot Agent | Windsurf | Aider | Continue |
|----------|-------------|--------|---------------|----------|-------|----------|
| 客户端开源    | 否/部分信息      | 否      | 否             | 否        | 是     | 是        |
| 可自托管模型   | 视方案         | 受限     | 受限            | 视方案      | 强     | 强        |
| 社区插件生态   | 中—高         | 高      | 高             | 中—高      | 中     | 高        |
| 适合「学习内核」 | 中（文档/外围）    | 中      | 中             | 中        | 强     | 强        |

**重要声明：** 闭源产品也可能更安全、更易治理；开源也不等于默认可信——供应链与配置同样关键。

## 9. 综合雷达：教学评分（1—5）

表 20–2–2：综合评分（示意，5 为更强）

| 能力域          | Claude Code | Cursor | Copilot Agent | Windsurf | Aider | Continue |
|--------------|-------------|--------|---------------|----------|-------|----------|
| 架构清晰度（可讲清楚）  | 4           | 4      | 4             | 4        | 5     | 5        |
| 企业治理与审计      | 4           | 4      | 5             | 3        | 3     | 3        |
| 扩展性与协议友好     | 5           | 5      | 4             | 4        | 3     | 5        |
| 成本可控（BYOK 等） | 4           | 3      | 3             | 3        | 5     | 5        |
| 上下文工程上限      | 5           | 5      | 4             | 5        | 4     | 4        |
| 多 Agent 协作   | 4           | 4      | 3             | 4        | 2     | 3        |
| 开源可审计        | 2           | 2      | 2             | 2        | 5     | 5        |

[Mermaid 图表 — 请在在线版本中查看]

注：雷达图在不同渲染器支持不一，本节以 表 20-2-2 为权威对照。

## 10. 选型决策树（教学版）

[Mermaid 图表 — 请在在线版本中查看]

## 11. 典型组合打法（非排他）

| 组合                   | 适用场景          | 风险点           |
|----------------------|---------------|---------------|
| IDE Agent + 终端 Agent | IDE 写码，终端跑批任务 | 上下文分裂，需要明确主会话 |
| 开源扩展 + 商用模型          | 可控配置 + 强模型    | 合规与数据流要梳理     |
| PR 机器人 + 本地 Agent    | 自动化评审 + 本地改码  | 权限与 token 泄露  |

## 12. 对比方法论：如何避免「站队」

1. 先列约束：数据能否出网、是否 SSO、是否必须审计日志。
2. 再列任务：重构、修 bug、写测试、写文档，权重不同。

3. 再做 PoC：同一 issue 用同一验收标准跑三轮。
4. 最后看 TCO：订阅 + 人工等待 + 事故成本。

### 13. 与 20.1 的呼应

20.1 指出壁垒在系统；本节说明：不同产品把「系统」做成了不同形状——有的偏 IDE 集成，有的偏终端与 git，有的偏平台托管。没有 universally best，只有 约束最优。

### 14. 反模式清单

| 反模式        | 后果              |
|------------|-----------------|
| 仅按「模型强弱」选型 | 权限与流程不匹配，事故概率上升 |
| 多工具并行无主编   | 上下文冲突、重复劳动      |
| 忽略审计与密钥扫描  | 合规与安全问题后置，代价高   |

### 15. 本节小结

- 用 七张总表 对齐架构、安全、扩展、成本、上下文、多 Agent、开源。
- 用 决策树 把「约束优先」落到路径。
- 评分与象限均为 教学示意，真实版本迭代会改变结论，应以 PoC 为准。

### 16. 迁移检查表（从工具 A 迁到 B）

| 步骤 | 内容              |
|----|-----------------|
| 1  | 导出规则、命令、忽略清单    |
| 2  | 重建工具白名单与秘文策略    |
| 3  | 跑一轮黄金 issue 集回归 |
| 4  | 对齐 CI 与评审模板     |



## 17. 术语表（本节）

| 术语   | 解释                            |
|------|-------------------------------|
| BYOK | Bring Your Own Key，自备 API Key |
| PoC  | Proof of Concept，验证性试点        |
| TCO  | Total Cost of Ownership，总拥有成本 |

## 18. 练习

1. 为你的团队填写「约束表」10 行，再重排本节各表权重。
2. 选一个真实 issue，用两种形态各做一次，记录耗时与返工次数。
3. 画一张你们当前的「工具组合架构图」，标出数据流。

## 19. 过渡到 20.3

对比解决「现在各家长什么样」；下一节讨论「未来两年大概率往哪走」——更长上下文、缓存、多模态、自主性、安全与信任的再平衡。

教学稿 V2 · 第 20 篇第 2 节 · 产品快速迭代可能导致特征变化，请以官方文档为准。

# 20.3 AI 编程工具未来趋势：上下文、成本、多模态、自主性与信任

**本节目标：**把「未来两年高概率发生的事」从营销话术里剥离出来，用**可验证的技术方向**描述：更长上下文、更好缓存、多模态 Agent、自主性提升，以及安全与信任的再平衡。

## 1. 趋势总览：五股力量同向挤压

[Mermaid 图表 — 请在在线版本中查看]

表 20-3-1：趋势 → 对工程实践的含义

| 趋势    | 对用户可见的变化       | 你需要提前准备的工程动作      |
|-------|----------------|-------------------|
| 更长上下文 | 更少「硬压缩」带来的信息丢失 | 仍要坚持分层加载，避免「全能塞入」 |
| 更好缓存  | 更低重复读成本        | 设计稳定缓存键、减少无意义抖动   |
| 多模态   | UI/截图/线框图驱动改码  | 建立截图脱敏与权限边界       |
| 自主性   | 更长任务链自动化       | 强化卡点、回滚与审计        |
| 信任    | 更严格的企业采纳       | 日志、租户隔离、供应链治理     |

## 2. 更长上下文：从「能塞」到「该塞什么」

上下文变长并不会自动让 Agent 更聪明；它主要降低**机械式截断**带来的失败。未来更可能的工程方向是：

- **分层记忆**：工作记忆（当前任务） / episodic（会话轨迹） / semantic（仓库知识）。
- **结构化引用**：用指针（路径、符号、commit）代替粘贴全文。
- **动态预算**：根据任务类型分配读、写、验证的 token 配额。

[Mermaid 图表 — 请在在线版本中查看]

表 20-3-2：长窗口下的三类反模式

| 反模式    | 症状      | 修复方向                |
|--------|---------|---------------------|
| 万物皆可塞  | 成本高、噪声大 | 相关性子图加载             |
| 长而不结构化 | 模型抓错重点  | schema、标题、要点列表      |
| 无版本指针  | 读后即过期   | 绑定 commit / 文件 hash |

### 3. 更好缓存 → 更低成本：从「重复纳税」到「可复用资产」

缓存不仅是 HTTP 层概念，在 Agent 体系里至少分三层：

- 1. **检索缓存**：同一查询的索引结果。
- 2. **工具输出缓存**：只读命令、构建产物列表等。
- 3. **摘要缓存**：对稳定大文件的摘要。

[Mermaid 图表 — 请在在线版本中查看]

表 20-3-3：缓存键设计检查项

| 检查项 | 说明           |
|-----|--------------|
| 稳定性 | 避免无意义时间戳进键   |
| 粒度  | 过粗失效慢，过细命中率低 |
| 失效  | 文件变更、依赖变更要联动 |
| 安全  | 缓存层不可泄漏跨租户数据 |

### 4. 多模态 Agent：视觉 + 代码的联合作战

多模态让「指哪打哪」更自然：圈选 UI、贴错误截图、上传架构白板照片。代价是：

- **隐私面扩大**：截图常含个人信息与商业信息。
- **解释性变难**：需要把视觉证据链接到具体文件/行号。
- **评测复杂**：同样截图在不同主题下渲染不同。

[Mermaid 图表 — 请在在线版本中查看]

表 20-3-4：多模态落地三阶段

| 阶段 | 能力            | 风险      |
|----|---------------|---------|
| L1 | 截图 → 文字说明     | 低       |
| L2 | 截图 → 定位组件/文件  | 中       |
| L3 | 截图 → 自动改样式/布局 | 高（需强验证） |

## 5. 自主性提升：自动化加深后的「新事故类型」

自主性不是「少点确认」，而是把确认转移到更正确的层级：

- 低风险高频动作自动化。
- 高风险动作保留显式授权或可回滚窗口。
- 长链路需要检查点与人类接管点。

[Mermaid 图表 — 请在在线版本中查看]

表 20-3-5：自主性提升时的必备「安全带」

| 安全带    | 作用              |
|--------|-----------------|
| 变更集边界  | 限制 blast radius |
| 自动测试门禁 | 捕获隐性破坏          |
| 特性开关   | 快速止血            |
| 审计轨迹   | 事后追责与学习         |

## 6. 安全与信任的平衡：从「能用」到「敢规模化用」

未来竞争焦点会逐渐从「单次炫技」转向「组织级采纳」：

- **数据最小化**：默认不收集、收集必说明、可删除。
- **供应链**：插件、模型路由、第三方工具链的可信根。
- **人机共治**：关键决策保留人类签名（审批、发布）。

[Mermaid 图表 — 请在在线版本中查看]

表 20-3-6：信任成熟度模型（教学简化）

| 级别 | 特征              |
|----|-----------------|
| T1 | 个人试用，无统一策略      |
| T2 | 团队规范 + 秘文扫描     |
| T3 | 组织策略 + SSO + 审计 |
| T4 | 合规映射 + 定期红队     |

## 7. 与 20.1、20.2 的关系

- 20.1：壁垒在系统。
- 20.2：各产品在不同维度取舍。
- 20.3：行业会把「长上下文、缓存、多模态、自主性」做成默认能力，**差距将更集中在治理与评测。**

## 8. 未来 12—24 个月的「高概率清单」

| 预测项        | 依据类型    | 对你意味着什么      |
|------------|---------|--------------|
| 上下文继续变长    | 模型与硬件迭代 | 仍要分层，不迷信「全塞」 |
| 缓存成为标配能力   | 成本压力    | 你要会设计失效策略    |
| 多模态进入主路径   | 产品差异化   | 提前做脱敏与权限     |
| Agent 更长链路 | 自动化诉求   | 投资测试与可观测性    |
| 企业采购更严     | 合规趋势    | 提前准备审计与数据流说明 |

## 9. 低概率但高影响：黑天鹅提示

- 监管突变导致某些部署模式受限。
- 关键开源组件供应链事件。
- 模型服务商定价策略剧烈调整。

[Mermaid 图表 — 请在在线版本中查看]

## 10. 路线图模板（可直接复制到团队文档）

| 季度 | 目标             | 度量                |
|----|----------------|-------------------|
| Q1 | 建立工具白名单与审计     | 违规调用次数            |
| Q2 | 引入黄金 issue 回归集 | 通过率               |
| Q3 | 上下文成本账         | \$/任务 或 tokens/任务 |
| Q4 | 多模态试点          | 脱敏违规=0            |

## 11. 本节练习

1. 为你的仓库画「分层上下文」三层内容各 5 条。
2. 列出三种可缓存工具输出，并写缓存失效条件。
3. 设想一张含隐私信息的截图，写一条处理规范。

## 12. 小结

- 更长上下文减轻截断，但不取消结构化责任。
- 缓存是降本核心杠杆，键与失效策略决定成败。
- 多模态提升交互自然度，同时扩大隐私与验证面。
- 自主性必须与门禁、审计、回滚同向演进。
- 信任将成为规模化采纳的分水岭。

## 13. 过渡到 20.4

趋势回答「世界往哪走」；下一节回答「开发者个人与团队该如何站位」——架构思维、Token 经济学、安全优先与多 Agent 协作。

## 14. 参考框架图（能力叠进）

[Mermaid 图表 — 请在在线版本中查看]

## 15. 术语

| 英文           | 中文    |
|--------------|-------|
| multimodal   | 多模态   |
| blast radius | 影响半径  |
| canary       | 金丝雀发布 |

## 16. 与全书主题词对照

| 全书主题 | 在本节的落点      |
|------|-------------|
| 工具治理 | 缓存与工具输出治理   |
| 权限   | 多模态与自主性下的边界 |
| 成本   | 缓存与分层加载     |

## 17. 批判性思考

「更长上下文」是否会让工程师更懒于抽象？教学观点：**会**，因此更需要 code review 与模块边界文化。

## 18. 企业读者检查表

- ☐ 是否定义了可接受的数据流（含截图）？
- ☐ 是否有密钥与令牌轮换策略？
- ☐ 是否有 Agent 相关变更的发布审批？

## 19. 个人读者检查表

- ☐ 是否记录过自己的 token 热点？
- ☐ 是否有一套「实现/自审」习惯？
- ☐ 是否为常用任务建立了可复用提示模板（结构化）？

## 20. 图表索引

| 图        | 类型           | 主题    |
|----------|--------------|-------|
| 图 20-3-1 | flowchart    | 五股力量  |
| 图 20-3-2 | stateDiagram | 上下文演进 |
| 图 20-3-3 | flowchart    | 缓存分层  |
| 图 20-3-4 | sequence     | 多模态链路 |
| 图 20-3-5 | flowchart    | 自主性等级 |
| 图 20-3-6 | mindmap      | 信任模型  |
| 图 20-3-7 | flowchart    | 风险规划  |
| 图 20-3-8 | flowchart    | 年份叠进  |

教学稿 V2 · 第 20 篇第 3 节

## 20.4 对开发者的启示：架构、Token 经济学、安全与多 Agent

**本节目标：**把前文的「产品视角」转译为个人与团队可执行的成长项：学习 AI Agent 架构设计、理解 Token 经济学、坚持安全优先、掌握多 Agent 协作模式。

### 1. 四个支柱：你真正要升级的「新栈」

[Mermaid 图表 — 请在在线版本中查看]

表 20-4-1：四支柱 → 具体能力映射



| 支柱        | 3 个月内可验证产出             |
|-----------|------------------------|
| 架构设计      | 能画出自己的 Agent 数据流与工具边界图 |
| Token 经济学 | 能解释一次任务的 token 主要消耗点   |
| 安全优先      | 能为工具调用分级并写白名单          |
| 多 Agent   | 能写一份「领导者-工作者」任务包模板     |

## 2. 启示一：学习 AI Agent 架构设计

Agent 架构不是「调用 OpenAI API」那么简单，它至少包含：

- **控制循环**：观察 → 计划 → 行动 → 校验。
- **工具层**：schema、错误语义、幂等。
- **状态管理**：会话、检查点、人类在环。

[Mermaid 图表 — 请在在线版本中查看]

表 20-4-2：架构学习路径（自学周计划示意）

| 周次 | 主题        | 实践作业                 |
|----|-----------|----------------------|
| W1 | 控制循环与状态   | 用伪代码写一个「修 lint」循环    |
| W2 | 工具 schema | 给三个内部脚本写 JSON schema |
| W3 | 校验闭环      | 把单元测试接入「行动后必跑」       |
| W4 | 观测        | 导出一次完整轨迹并复盘          |

## 3. 启示二：理解 Token 经济学

Token 不是会计细节，是**性能与成本的联合约束**。你需要建立三类直觉：

1. **输入贵还是输出贵**：依模型与任务而异，要会测。
2. **重复读是最大浪费**：检索与缓存优先级常高于换大模型。
3. **结构化便宜**：列表、表格、指针优于散文。

[Mermaid 图表 — 请在在线版本中查看]

注：比例为教学示意，真实分布请用你的日志估计。

表 20-4-3：降本手段优先级（一般情况）

| 优先级 | 手段     | 说明         |
|-----|--------|------------|
| P0  | 去重与缓存  | often 立竿见影 |
| P1  | 分层加载   | 控制噪声       |
| P2  | 工具输出分页 | 避免巨型 JSON  |
| P3  | 换更强模型  | 最后手段       |

## 4. 启示三：安全优先设计（默认不信任）

安全优先不是「不用 Agent」，而是 **默认最小权限 + 可审计 + 可回滚**：

- **工具白名单**：从只读开始，逐步放开。
- **秘文治理**：.env、密钥、令牌不进窗口或脱敏。
- **供应链**：插件来源固定、版本锁定。

[Mermaid 图表 — 请在在线版本中查看]

表 20-4-4：风险分级示例

| 等级 | 示例动作   | 建议策略   |
|----|--------|--------|
| L0 | 搜索只读   | 自动     |
| L1 | 改测试代码  | 自动+CI  |
| L2 | 改业务核心  | 评审     |
| L3 | 改密钥或权限 | 强审批+审计 |

## 5. 启示四：多 Agent 协作模式

多 Agent 的价值不在「人多势众」，而在角色分离减少认知污染：

- **领导者**：拆任务、定验收、合并结果。
- **工作者**：实现、跑测、迭代。
- **审查者**：威胁建模、边界条件、测试缺口。

[Mermaid 图表 — 请在在线版本中查看]

表 20-4-5：任务包最小字段（模板）

| 字段    | 内容             |
|-------|----------------|
| 目标    | 一句话可验证目标       |
| 约束    | 不可触碰模块/接口      |
| 验收    | 测试命令与期望        |
| 上下文指针 | 文件路径、符号、commit |
| 风险    | 已知坑            |

## 6. 与职业阶段的关系

| 阶段 | 侧重点                |
|----|--------------------|
| 初级 | 会用工具 + 会写清晰任务包     |
| 中级 | 会设计校验闭环 + 会控成本     |
| 高级 | 会定团队规范 + 会评估供应商方案  |
| 专家 | 能造内部平台 + 能做安全与合规映射 |

## 7. 常见误区

| 误区                | 实际情况      |
|-------------------|-----------|
| 「会 Chat 就会 Agent」 | 缺工具与闭环难落地 |
| 「模型越强越省钱」         | 可能更贵且更难控  |
| 「开源=安全」           | 配置错误同样危险  |

## 8. 团队层面的最小制度包

- Agent 变更同样走 PR（除非明确豁免清单）。
- CI 必须通过才能合并。
- 秘文扫描与依赖审计不降级。
- 重大工具权限需要登记与审批。

## 9. 个人层面的每日习惯

- 开始任务前写 5 行「验收标准」。
- 结束任务后记录「token/耗时/返工原因」各一行。
- 每周复盘一次「最大浪费上下文」。

## 10. 本节与全书技能树对齐

[Mermaid 图表 — 请在在线版本中查看]

## 11. 案例骨架（课堂可用）

**背景：**修复一个 flaky 测试。

**错误做法：**让 Agent 直接改生产逻辑凑绿。

**更好做法：**任务包要求先复现、再加最小隔离测试、最后改实现。

**安全点：**禁止扩大网络权限去「抓外部数据凑数」。

## 12. 度量建议

| 指标           | 用途    |
|--------------|-------|
| tokens/合并 PR | 成本趋势  |
| 返工轮次         | 任务包质量 |
| CI 失败率       | 闭环可靠性 |
| 秘文告警数        | 安全基线  |

## 13. 过渡到 20.5

本节回答「该学什么」；下一节给出按角色划分的学习路径与推荐资源，便于你把启示落到时间表。

## 14. 小结背板

- **架构**：循环 + 工具 + 校验 + 观测。
- **Token**：先治理上下文，再考虑换模型。
- **安全**：默认拒绝，逐步放权，全程可审计。
- **多 Agent**：用角色分离降低认知噪声，靠任务包对齐。

## 15. 自测题（口试）

1. 控制循环四步各举一例工具调用。
2. 说出一个「P0 降本」手段及适用条件。
3. 解释为何 L3 风险需要双人复核。
4. 任务包五字段缺一会怎样？

## 16. 术语

| 英文                | 中文   |
|-------------------|------|
| human-in-the-loop | 人类在环 |
| least privilege   | 最小权限 |

## 17. 延伸阅读钩子

详见 `05-learning-path.md` 与附录 `further-reading.md` 。

## 18. 图表索引

| 图        | 类型                |
|----------|-------------------|
| 图 20-4-1 | flowchart 四支柱     |
| 图 20-4-2 | sequence 控制循环     |
| 图 20-4-3 | pie token 示意      |
| 图 20-4-4 | flowchart 权限渐进    |
| 图 20-4-5 | flowchart 多 Agent |
| 图 20-4-6 | mindmap 技能树       |

## 19. 结语句（可引用）

Agent 时代工程师的核心竞争力，是把不确定性关进系统，而不是把希望寄托在模型的「自觉」上。

## 20. 版本记录

| 版本 | 说明           |
|----|--------------|
| V2 | 与全书 20 篇结构对齐 |

教学稿 V2 · 第 20 篇第 4 节

## 20.5 推荐学习路径：按角色划分的进阶路线与资源

**本节目标：**把「读完 20 篇之后干什么」拆成可执行路线。覆盖 小白 / 开发者 / 架构师 三类读者，并给出推荐项目、练习与资源入口（详细外链见附录 `further-reading.md`）。

### 1. 路线总览：三条主径与一个交汇点

[Mermaid 图表 — 请在在线版本中查看]

表 20-5-1：角色 → 12 周目标（建议节奏）

| 角色  | 第 4 周    | 第 8 周       | 第 12 周       |
|-----|----------|-------------|--------------|
| 小白  | 独立完成小改修  | 能带测试提交 PR   | 能自查秘文与权限     |
| 开发者 | 建立个人模板库  | token 成本账清晰 | 能指导同事任务包     |
| 架构师 | 出团队规范 v0 | PoC 对比两家方案  | 建立黄金 issue 集 |

### 2. 小白路径：从「敢用」到「用得稳」

#### 2.1 前置条件

- 会用 git 基本操作（branch、commit、push）。
- 能读懂项目 README 与简单报错。

#### 2.2 第 1-2 周：环境与心智

| 日   | 主题       | 产出           |
|-----|----------|--------------|
| 1-2 | 安装与最小工作流 | 跑通 hello 级任务 |
| 3-4 | 任务描述结构化  | 个人模板 3 份     |
| 5-7 | 第一次 PR   | 带截图或测试说明     |

[Mermaid 图表 — 请在在线版本中查看]

## 2.3 第 3—4 周：安全与习惯

- 学会识别秘文与 `.env`。
- 养成「改前先拉分支、改后跑最小测试」。

表 20–5–2：小白常见坑

| 坑                | 预防          |
|------------------|-------------|
| 让 Agent 直接改 main | 分支策略写进模板第一行 |
| 复制密钥进聊天          | 用占位符+本地文件引用 |
| 不做自测就合并          | 任务包强制写验收命令  |

## 2.4 推荐练手项目类型

| 类型        | 说明        |
|-----------|-----------|
| 小型 CLI 工具 | 边界清晰      |
| 带测试的库     | 易验证       |
| 文档型仓库     | 低风险练结构化写作 |

# 3. 开发者路径：从「会用」到「可复用工程化」

## 3.1 核心模块

1. 工具与权限：理解 allowlist、沙箱、确认流。
2. 上下文策略：相关文件集、摘要、缓存意识。
3. 闭环：lint、typecheck、test 成为默认后缀动作。

[Mermaid 图表 — 请在在线版本中查看]



### 3.2 第 5–8 周专题

| 周 | 专题         | 深度练习           |
|---|------------|----------------|
| 5 | 工具 schema  | 给内部脚本加输入校验     |
| 6 | Token 账    | 记录三次任务的消耗结构    |
| 7 | 多文件重构      | 要求 Agent 输出变更表 |
| 8 | Flaky 测试治理 | 禁止「关测试」式修复     |

表 20–5–3：开发者「模板库」最小集合

| 模板名      | 用途          |
|----------|-------------|
| bugfix   | 复现→定位→测试→修复 |
| refactor | 行为不变清单+增量步骤 |
| feature  | 接口草案+验收用例   |
| docs     | 读者对象+目录结构   |

### 3.3 推荐开源项目（学习向）

| 项目         | 学习点        |
|------------|------------|
| Continue   | 扩展结构、模型路由  |
| Aider      | git 集成、补丁流 |
| 各 MCP 示例服务 | 工具协议与鉴权    |

具体仓库链接见附录 [further-reading.md](#)。

## 4. 架构师路径：从「团队能用」到「组织敢用」

### 4.1 关注面

- 治理：SSO、租户、数据流说明、留痕。
- 平台化：统一模板、统一工具注册、统一观测。
- 评测：黄金 issue、对比实验、回归节奏。

[Mermaid 图表 — 请在在线版本中查看]

4.2 交付物清单（v1）

| 交付物    | 内容              |
|--------|-----------------|
| 数据流白皮书 | 哪些数据出网、存多久      |
| 工具注册表  | 名称、风险级、owner    |
| 变更策略   | Agent 改码的 PR 规则 |
| 事件响应   | 误执行、秘文泄露预案      |

表 20–5–4：架构师 PoC 评分卡（节选）

| 项     | 权重 |
|-------|----|
| 合规映射  | 高  |
| 审计完整性 | 高  |
| TCO   | 中  |
| 开发者体验 | 中  |
| 可迁移性  | 中  |

5. 交汇点：所有人都应掌握的「四项基本功」

- 1. 结构化任务描述（目标/约束/验收）。
- 2. 最小权限意识。
- 3. 可验证闭环（测试或静态检查）。
- 4. 复盘记录（一次一行也能积少成多）。

## 6. 与全书 20 篇的映射（如何复盘）

| 全书板块（示意） | 小白 | 开发者 | 架构师  |
|----------|----|-----|------|
| 入门与安装    | 精读 | 扫读  | 指定标准 |
| 工具与权限    | 遵守 | 精读  | 制定   |
| 上下文与成本   | 感知 | 精读  | 平台化  |
| 多 Agent  | 了解 | 实践  | 设计   |
| 安全合规     | 遵守 | 实践  | 负责   |

## 7. 学习节奏建议（防 burnout）

[Mermaid 图表 — 请在在线版本中查看]

## 8. 里程碑认证（自娱式）

| 徽章    | 条件             |
|-------|----------------|
| 绿通新手  | 10 个带测试的小 PR   |
| 闭环工程师 | 连续 20 次 CI 一次过 |
| 成本守门员 | 文档化三次降本改动      |
| 治理共建者 | 输出团队规范并被采纳     |

## 9. 社群与协作学习

- 参与 **code review**：看别人如何写任务包。
- 组织 **brown bag**：每周 30 分钟分享一次翻车与修复。
- 维护 **团队 playbook**：比个人笔记更能沉淀。

## 10. 与附录联动

| 需求      | 附录文件                            |
|---------|---------------------------------|
| 命令与配置速查 | <code>cheatsheet.md</code>      |
| 术语中英    | <code>glossary-en-zh.md</code>  |
| 自测巩固    | <code>quiz.md</code>            |
| 外链资源    | <code>further-reading.md</code> |
| 源码地图    | <code>source-index.md</code>    |

## 11. 常见时间陷阱

| 陷阱     | 调整         |
|--------|------------|
| 沉迷换模型  | 先固定模型优化上下文 |
| 沉迷收藏文章 | 每周一篇精读+实践  |
| 无验收标准  | 先写验收再允许开工  |

## 12. 过渡到 20.6

下一节为全书收束：回顾 20 篇 200+ 节、致谢读者，并强调开源社区与共建的意义。

## 13. 小结

- 小白：稳、慢、带测试；先习惯与模板。
- 开发者：闭环、成本、模板库；能带人。
- 架构师：治理、平台、评测；能担责。
- 资源：全书 + 附录 + 你手头的真实仓库。

### 14. 30 天打卡表（可复制）

| 天   | 任务        |
|-----|-----------|
| 1   | 安装与环境验证   |
| 2   | 任务包模板 v0  |
| 3   | 第一个 PR    |
| ... | ...       |
| 30  | 复盘文 800 字 |

### 15. 图表索引

| 图        | 类型              |
|----------|-----------------|
| 图 20-5-1 | flowchart 三路径   |
| 图 20-5-2 | journey 小白旅程    |
| 图 20-5-3 | flowchart 开发者闭环 |
| 图 20-5-4 | flowchart 架构师三角 |
| 图 20-5-5 | pie 时间分配        |

### 16. 术语

| 英文        | 中文    |
|-----------|-------|
| brown bag | 午餐学习会 |
| playbook  | 实战手册  |

### 17. 自测

1. 写出你所在角色的「12 周目标」三行版。
2. 列出你模板库还缺的两种场景。
3. 架构师 PoC 你最看重哪两项权重？为什么？

## 18. 版本

| 版本 | 说明      |
|----|---------|
| V2 | 与附录资源联动 |

教学稿 V2 · 第 20 篇第 5 节

## 20.6 结语：回顾全书、致谢读者与开源共建

**本节目标：**为《Claude Code 完全指南 V2》画上句号——回顾 **20 篇、200+ 节** 的知识地图，真诚感谢读者投入的时间，并重申 **开源社区** 在这一波 Agent 工程浪潮中的位置：不是附属品，而是主航道的一部分。

### 1. 我们一同走过的路：全书结构回眸

全书以「从能用到敢用、从个人到组织」为隐线，约 **20 篇** 正文与 **200+ 节** 细粒度拆解，试图覆盖以下主干（篇名以教学抽象表述，便于对照你手中的目录）：

[Mermaid 图表 — 请在在线版本中查看]

表 20-6-1：全书能力栈（再浓缩版）

| 层级   | 你应带走的句子              |
|------|----------------------|
| 认知层  | Agent 是系统问题，模型是其中一环。 |
| 工具层  | schema 与权限决定可靠性与边界。  |
| 上下文层 | Token 是预算，分层与缓存是纪律。  |
| 协作层  | 任务包与评审把「聪明」变成「可合并」。  |
| 组织层  | 没有审计与制度的规模化，只是放大风险。  |

## 2. 数字背后的意义：200+ 节到底在对抗什么

「200+ 节」不是为了厚，而是为了对抗碎片化学习：Agent 领域文章很多，但缺少把 **CLI**、**工具**、**权限**、**上下文**、**测试**、**治理** 连成闭环的叙述。每一节尽量做到：

- 可单独阅读（有局部结论）；
- 可串成路径（有前后引用）；
- 可落到仓库（有检查表或练习）。

[Mermaid 图表 — 请在在线版本中查看]

## 3. 致谢读者：你完成的不仅是阅读

如果你一路读到第 20 篇，你完成的远不止一次「看完」。你很可能已经：

- 在自己的项目上做过 **PoC**；
- 为同事写过 **模板** 或 **规范**；
- 在 **CI** 红绿之间理解过 **Agent** 的代价。

这些实践比任何书评都更珍贵。谢谢你愿意把时间与真实代码交给这套叙事来对照。

## 4. 致谢贡献者与社区：工具链是集体作品

无论闭源产品还是开源生态，背后都有 **维护者**、**文档作者**、**插件作者**、**踩坑分享者**。本书若引用或对齐了某些社区经验，本质上都是站在他人肩膀上的教学整理。

[Mermaid 图表 — 请在在线版本中查看]

表 20-6-2：你可以回馈社区的微行动

| 行动          | 影响      |
|-------------|---------|
| 提交文档 PR     | 降低新人门槛  |
| 发布最小复现      | 节省他人小时  |
| 写清 issue 边界 | 维护者更高效  |
| 分享失败案例      | 比成功学更稀缺 |

## 5. 开源精神在 Agent 时代的变体

传统开源强调 **源码可得**；Agent 时代还要强调：

- **行为可得**：工具做什么、默认权限是什么，应可读、可审计。
- **数据流可得**：哪些内容离开本机，应有清晰说明。
- **评测可得**：宣称的能力应用可重复脚本验证。

这不是道德绑架，而是 **规模化协作的摩擦系数**——摩擦越低，创新越快。

## 6. 闭源与开源如何共处（务实看法）

本书立场：**不必二极管**。团队可以选择闭源产品获得整合体验，同时用开源组件补齐 **可审计模块** 与 **逃生通道**。关键是：

- 合同与租户策略清晰；
- 关键资产有备份与迁移计划；
- 不把「方便」建立在「不可追溯」上。

[Mermaid 图表 — 请在在线版本中查看]

## 7. 全书金句复盘（便于引用）

1. AI Agent 约 90% 的工作量在 AI 之外。
2. 好体验来自制度与工程，而不是几段 Prompt。
3. Token 是预算，上下文治理是专业度。
4. 安全是默认拒绝 + 可审计 + 可回滚。



5. 多 Agent 的本质是角色分离与任务包协议。

8. 未完成之书：为什么指南必须持续更新

Agent 产品与模型迭代极快，任何「完全指南」都只是 快照：

- 命令与参数会变；
- 最佳实践会迁移；
- 合规要求会加码。

因此本书建议与 官方文档、变更日志 同步阅读；附录中的外链与索引也会需要读者自行刷新。

9. 你带走的三样东西

| 物品   | 说明               |
|------|------------------|
| 一张图  | 你自己的 Agent 数据流   |
| 一份表  | 团队工具白名单与风险级      |
| 一个习惯 | 先写验收再允许 Agent 动手 |

10. 最后的练习：写给你的「一年后信」

用 10 行回答：

- 我希望 Agent 在我们团队解决的三类问题是什么？
- 我绝不让它碰的三类红线是什么？
- 一年后我希望成本与质量指标各变成什么样？

把信存进仓库的 docs/ ，一年后打开——这比任何结论都有力量。

## 11. 附录导航（收束全书）

| 文件                              | 用途      |
|---------------------------------|---------|
| <code>source-index.md</code>    | 源码与模块索引 |
| <code>cheatsheet.md</code>      | 命令与配置速查 |
| <code>quiz.md</code>            | 50 题自测  |
| <code>further-reading.md</code> | 主题化外链   |
| <code>glossary-en-zh.md</code>  | 中英术语    |

## 12. 再见不是终点

若本书帮你少翻一次车、少熬一夜、少吵一场会，它就完成了使命。更希望你能带着 **工程化与治理** 的视角，参与下一波共建——无论你写的是业务代码、平台代码，还是一片诚实的复盘帖。

[Mermaid 图表 — 请在在线版本中查看]

## 13. 致谢（编辑与教学伙伴）

感谢在草稿阶段提出「太像广告」「不够可验证」「缺检查表」的审阅者——你们的刻薄让稿子更耐读。

## 14. 许可与使用建议

- 教学幻灯可拆解引用，建议保留出处与日期。
- 商业转载请遵守原仓库声明（若有）。

## 15. 联系与勘误（占位）

- 欢迎在原书配套仓库提交 **issue** 指正事实错误与过期命令。
- 区分「版本变更」与「内容错误」：前者更新快，后者应修。

## 16. 全书路线图时间轴（象征）

[Mermaid 图表 — 请在在线版本中查看]

## 17. 小结背板

- 20 篇 / 200+ 节：对抗碎片化，服务闭环实践。
- 感谢读者：时间投入即贡献。
- 社区与开源：Agent 时代更需要可审计与可复现。
- 指南会过期：与官方文档并行阅读。
- 带走习惯：验收、白名单、复盘。

## 18. 终章一句

愿你在自动化浪潮里，仍然是自己系统的 **架构师**——而不是幸运的乘客。

《Claude Code 完全指南 V2》· 第 20 篇第 6 节 · 全书终

## 附录

### 附录 A · 源码文件索引（教学用模块地图）

**说明：**下列路径、行数与职责为 **V2 教程级「示意索引」**，用于建立阅读大型 Agent/CLI 代码库时的心智模型；真实上游仓库随版本迭代会变化，**请以你本地 checkout 为准**（可用 `wc -l`、`cloc` 自行核对）。核心论点不变：**入口薄、循环厚、工具散、权限横切**。

#### 1. 总览：从入口到一次请求的旅程

[Mermaid 图表 — 请在在线版本中查看]

表 A-1：旅程与典型文件职责

| 阶段 | 典型模块         | 读者关注点           |
|----|--------------|-----------------|
| 启动 | CLI 入口       | 参数解析、配置加载、日志初始化 |
| 会话 | 会话状态         | 历史、压缩、检查点       |
| 循环 | query / loop | 模型往返、工具选择、终止条件  |
| 执行 | tools/*      | IO 边界、错误格式、超时   |
| 横切 | permissions  | 默认拒绝、升级路径、审计钩子  |

#### 2. 核心文件索引（高信号）

表 A-2：关键源码文件（教学示意）

| 文件路径（示意）                                                     | 行数（示意） | 功能描述                           |
|--------------------------------------------------------------|--------|--------------------------------|
| <code>src/main.tsx</code> 或 <code>cli/main.ts</code>         | ~4684  | CLI 入口、命令分发、全局初始化、部分 UI 渲染     |
| <code>src/query.ts</code> （或 <code>session/query.ts</code> ） | ~1730  | <b>核心循环</b> ：模型调用、工具编排、消息折叠与重试 |
| <code>src/Tool.ts</code> （或 <code>tools/Tool.ts</code> ）     | ~793   | 工具抽象基类：schema、执行、权限声明、结果封装     |
| <code>src/tools/**/*.ts</code>                               | 若干     | 读文件、写文件、执行命令、搜索、提交等原子能力        |
| <code>src/permissions/**</code>                              | 若干     | 策略引擎：允许列表、会话级授权、持久化偏好          |
| <code>src/coordinator/**</code>                              | 若干     | 子 Agent 委派、任务分片、结果合并           |
| <code>src/config/**</code>                                   | 若干     | 配置合并：文件、环境变量、CLI 覆盖            |
| <code>src/mcp/**</code> 或 <code>integrations/mcp</code>      | 若干     | MCP 客户端生命周期与路由                 |
| <code>src/ui/**</code> / <code>tui/**</code>                 | 若干     | 终端 UI、富文本、进度与确认框               |

行数为 **教程标注量级**，用于表达「何处最厚」；非精确审计。

### 3. 按模块分组的大表

#### 3.1 CLI 与启动链

| 路径模式                       | 职责                                                                |
|----------------------------|-------------------------------------------------------------------|
| <code>**/cli*.ts</code>    | 子命令： <code>chat</code> 、 <code>doctor</code> 、 <code>mcp</code> 等 |
| <code>**/argv*.ts</code>   | 参数规范化                                                             |
| <code>**/logger*.ts</code> | 结构化日志、脱敏                                                          |

### 3.2 会话与上下文

| 路径模式                         | 职责           |
|------------------------------|--------------|
| <code>**/session*.ts</code>  | 会话 ID、元数据    |
| <code>**/context*.ts</code>  | 上下文装配、相关文件解析 |
| <code>**/compress*.ts</code> | 历史摘要与折叠策略    |
| <code>**/cache*.ts</code>    | 工具/检索缓存（若有）  |

### 3.3 模型与供应商

| 路径模式                          | 职责              |
|-------------------------------|-----------------|
| <code>**/anthropic*.ts</code> | 官方 API 客户端、流式解析 |
| <code>**/model*.ts</code>     | 路由、降级、重试        |
| <code>**/streaming*.ts</code> | SSE/chunk 处理    |

### 3.4 工具系统

| 路径模式                                               | 职责        |
|----------------------------------------------------|-----------|
| <code>**/Tool.ts</code>                            | 接口与注册     |
| <code>**/tools/read*.ts</code>                     | 安全读盘与编码   |
| <code>**/tools/write*.ts</code>                    | 补丁应用、冲突检测 |
| <code>**/tools/bash*.ts</code> 或 <code>exec</code> | 命令执行与沙箱   |
| <code>**/tools/grep*.ts</code> / <code>glob</code> | 仓库内搜索     |

[Mermaid 图表 — 请在在线版本中查看]

### 3.5 权限与策略

| 路径模式                                   | 职责           |
|----------------------------------------|--------------|
| <code>**/permissions/policy*.ts</code> | 规则求值         |
| <code>**/permissions/store*.ts</code>  | 用户选择持久化      |
| <code>**/sandbox*.ts</code>            | 进程/网络隔离（若存在） |

### 3.6 协调与多 Agent

| 路径模式                             | 职责        |
|----------------------------------|-----------|
| <code>**/coordinator/*.ts</code> | 任务拆分、子会话  |
| <code>**/agent*.ts</code>        | 角色定义、提示边界 |
| <code>**/handoff*.ts</code>      | 上下文交接格式   |

### 3.7 测试与夹具

| 路径模式                         | 职责      |
|------------------------------|---------|
| <code>**/__tests__/**</code> | 单元与集成   |
| <code>**/e2e/**</code>       | 端到端（若有） |
| <code>**/fixtures/**</code>  | 仓库样例    |

## 4. 「4756 个模块」在索引中的读法

教学上可把「模块」理解为：每一个可独立失败、需要独立版本策略的单元——不仅指文件数，也指 npm 包、动态加载插件、MCP 服务器进程。索引的价值是让你知道 **该从哪里下刀**：

| 症状      | 优先翻的目录                                      |
|---------|---------------------------------------------|
| 工具调用失败  | <code>tools/*</code> + <code>Tool.ts</code> |
| 权限弹窗异常  | <code>permissions/*</code>                  |
| 循环卡住/重复 | <code>query.ts</code> + 会话压缩                |
| MCP 连不上 | <code>mcp/*</code> + 配置合并                   |

## 5. 阅读顺序建议（8 小时版）

| 小时  | 文件/目录                          | 目标           |
|-----|--------------------------------|--------------|
| 1   | CLI 入口                         | 知道命令如何落到会话   |
| 2–3 | <code>query.ts</code>          | 理解主循环与工具调度   |
| 4   | <code>Tool.ts</code> + 2 个具体工具 | 理解契约         |
| 5   | <code>permissions</code>       | 理解默认拒绝       |
| 6   | <code>coordinator</code>       | 理解多 Agent    |
| 7–8 | 测试夹具                           | 看「官方认为的正确行为」 |

## 6. 与本书正文的映射

| 正文主题    | 索引锚点                                        |
|---------|---------------------------------------------|
| 工具治理    | <code>Tool.ts</code> 、 <code>tools/*</code> |
| 安全      | <code>permissions/*</code> 、危险工具实现          |
| 上下文     | <code>query.ts</code> 、压缩与缓存模块              |
| 多 Agent | <code>coordinator/*</code>                  |

## 7. 自助生成索引命令（读者可执行）

```
# 按行数列出前 30 个 TypeScript 文件（示例）
find . -name "*.ts" -not -path "*/node_modules/*" -print0 | xargs -0 wc -l | sort -nr | head -n 30
```

## 8. 变更追踪建议

| 实践       | 说明                                             |
|----------|------------------------------------------------|
| 锁版本 tag  | 对照 release note                                |
| 记 diff   | 升级后重跑「8 小时阅读」中的关键路径                            |
| 维护团队私有索引 | 在 <code>docs/internal/</code> 放一页「我们 fork 的差异」 |



## 9. 小结

- 入口薄：CLI。
- 心脏厚：query/loop。
- 手脚散：tools/。
- 交规横切：permissions/。
- 车队可选：coordinator/。

## 10. 图表索引

| 图     | 类型               |
|-------|------------------|
| 图 A-1 | flowchart 总览     |
| 图 A-2 | classDiagram 工具类 |

附录 A · V2 教学稿 · 路径与行数需以实际仓库为准

# 附录 B · 命令速查表

**说明：**斜杠命令与 CLI 开关随版本变化较快，下表为 **V2 教程整理版**。若与终端内 `help` 或官方文档冲突，**以运行时输出为准**。路径以 macOS/Linux 为例。

## 1. 配置与状态文件位置（总表）

| 类别       | 典型路径（示意）                                               | 用途             |
|----------|--------------------------------------------------------|----------------|
| 用户级配置    | <code>~/.config/claude-code/</code> 或产品文档指定目录          | 全局偏好、默认模型、忽略规则 |
| 项目级配置    | 仓库根目录 <code>.claude/</code> 、 <code>CLAUDE.md</code> 等 | 项目规则、文档入口      |
| Shell 集成 | <code>~/.zshrc</code> / <code>~/.bashrc</code>         | PATH、别名、非交互变量  |
| 凭据       | OS 钥匙串 / <code>~/.config/...</code> 下加密存储（依实现）         | API Key、令牌     |
| 日志       | 产品日志目录（见 <code>doctor</code> 输出）                       | 排障、审计          |

[Mermaid 图表 — 请在在线版本中查看]

## 2. 常用 CLI 参数（大表）

| 参数 / 子命令                                              | 说明          | 备注          |
|-------------------------------------------------------|-------------|-------------|
| <code>--help</code> / <code>-h</code>                 | 显示帮助        | 首选真相源       |
| <code>--version</code>                                | 版本号         | 提 issue 时附上 |
| <code>doctor</code> / <code>diagnose</code> （若有）      | 自检网络、权限、模型  | 排障第一步       |
| <code>--print</code> / <code>non-interactive</code> 类 | 脚本/CI 非交互   | 注意秘文进日志     |
| <code>--allowedTools</code> 类                         | 限制工具集       | 安全相关        |
| <code>--mcp-config</code> 类                           | 指定 MCP 配置路径 | 多项目隔离时常用    |
| <code>--verbose</code> / <code>--debug</code>         | 详细日志        | 生产流水线慎用     |

具体子命令名请以 `claude --help` 为准；不同发行渠道命名可能不同。

### 3. 环境变量（大表）

| 变量名（示意）                          | 作用            | 安全提示             |
|----------------------------------|---------------|------------------|
| <code>ANTHROPIC_API_KEY</code>   | 调用 Claude API | 勿提交 git；CI 用密钥管理 |
| <code>ANTHROPIC_BASE_URL</code>  | 自定义网关         | 确认证书与租户策略        |
| <code>HTTP(S)_PROXY</code>       | 代理            | 企业网络常见           |
| <code>NO_COLOR</code>            | 关闭 ANSI 颜色    | 日志采集友好           |
| <code>CI=true</code>             | 检测 CI 环境      | 可触发非交互/限权模式      |
| <code>NODE_OPTIONS</code>        | Node 运行时参数    | 可能影响子进程          |
| <code>CLAUDE_CODE_*</code> 类（若有） | 产品专属开关        | 查官方文档列表          |

### 4. 斜杠命令 / 内置指令（教学汇总）

下列为 常见模式 的分类表；实际以交互界面提示为准。

#### 4.1 会话与上下文

| 斜杠命令（示意）              | 功能        |
|-----------------------|-----------|
| <code>/clear</code>   | 清空或折叠上下文  |
| <code>/compact</code> | 压缩历史      |
| <code>/resume</code>  | 恢复会话（若支持） |
| <code>/rename</code>  | 重命名会话     |

#### 4.2 仓库与搜索

| 斜杠命令（示意）             | 功能          |
|----------------------|-------------|
| <code>/init</code>   | 初始化项目助手配置   |
| <code>/index</code>  | 触发/刷新索引（若有） |
| <code>/grep</code> 类 | 快速搜索入口      |

### 4.3 工具与权限

| 斜杠命令（示意）                  | 功能       |
|---------------------------|----------|
| <code>/permissions</code> | 查看或重置授权  |
| <code>/mcp</code>         | MCP 连接管理 |
| <code>/tools</code>       | 列出可用工具   |

### 4.4 帮助与调试

| 斜杠命令（示意）           | 功能         |
|--------------------|------------|
| <code>/help</code> | 命令帮助       |
| <code>/bug</code>  | 收集诊断信息（若有） |

[Mermaid 图表 — 请在在线版本中查看]

## 5. 自然语言指令模板（可复制）

| 场景     | 模板                                                                             |
|--------|--------------------------------------------------------------------------------|
| Bug 修复 | 「在分支 <code>{branch}</code> 上修复 <code>{issue}</code> ：先复现，再写失败测试，再最小修复；禁止扩大范围。」 |
| 重构     | 「行为不变重构 <code>{module}</code> ：列出公开 API 不变清单；每步可编译通过。」                         |
| 加测试    | 「为 <code>{path}</code> 补充覆盖 <code>{场景}</code> 的测试，使用现有测试框架与风格。」                |
| 文档     | 「面向 <code>{读者}</code> 更新 <code>{path}</code> ：包含示例命令与常见错误。」                    |
| 安全审查   | 「审查 <code>{PR}</code> 的秘文、注入、路径穿越风险；输出分级结论。」                                   |
| 性能     | 「分析 <code>{path}</code> 的热路径，给出可验证的基准或复杂度说明。」                                  |

## 6. Git 与 Agent 协作常用命令

| 命令                                   | 用途          |
|--------------------------------------|-------------|
| <code>git status -sb</code>          | 快速看改动面      |
| <code>git diff</code>                | 审查 Agent 改动 |
| <code>git stash push -m "wip"</code> | 人类临时接管前保存   |
| <code>git worktree add</code>        | 并行尝试不同方案    |

## 7. MCP 速查

| 项    | 说明                    |
|------|-----------------------|
| 配置文件 | 常在项目或用户目录 JSON（见官方文档） |
| 启动失败 | 先查端口、路径、Node 版本       |
| 权限   | 按最小暴露注册工具             |

## 8. 排障清单（命令级）

| 步骤 | 命令/动作                                 |
|----|---------------------------------------|
| 1  | 运行 <code>doctor</code> 或等价自检          |
| 2  | <code>curl</code> / 代理探测（网络）          |
| 3  | 降 <code>verbose</code> 日志并脱敏后提交 issue |
| 4  | 最小仓库复现                                |

## 9. CI 片段示例（伪代码）

```
# 示例：仅教学，非可直接运行配置
jobs:
  agent-smoke:
    runs-on: ubuntu-latest
    env:
      ANTHROPIC_API_KEY: ${ secrets.ANTHROPIC_API_KEY }
    steps:
      - uses: actions/checkout@v4
      - name: Print version
        run: claude --version
      - name: Non-interactive task
        run: claude --help
```

## 10. 与模型相关的常用开关（示意表）

| 开关类型 | 示例（依产品而定）                                 | 用途                |
|------|-------------------------------------------|-------------------|
| 模型别名 | <code>opus</code> / <code>sonnet</code> 等 | 切换能力/成本档位         |
| 温度   | <code>temperature</code>                  | 创造性 vs 确定性（编程常偏低） |
| 最大输出 | <code>max_tokens</code> 类                 | 限制单次生成长度          |

## 11. 工作区忽略与索引（概念表）

| 文件/机制                            | 用途                |
|----------------------------------|-------------------|
| <code>.gitignore</code>          | git 与部分索引器尊重的忽略规则 |
| <code>.cursorignore</code> 类产品文件 | 限制 IDE/Agent 索引范围 |
| <code>CLAUDE.md</code> 类项目说明     | 注入稳定项目规范（注意秘文）    |

## 12. 常用排错环境变量

| 变量（示意）                                    | 场景         |
|-------------------------------------------|------------|
| <code>SSL_CERT_FILE</code>                | 企业 MITM 证书 |
| <code>NODE_TLS_REJECT_UNAUTHORIZED</code> | 慎用，仅排障临时   |
| <code>DEBUG</code> / <code>DEBUG=*</code> | 库级详细日志     |

## 13. 自然语言指令扩展模板

| 场景     | 模板                                                                     |
|--------|------------------------------------------------------------------------|
| 依赖升级   | 「将 <code>{pkg}</code> 升级到 <code>{version}</code> ：跑测试与类型检查，列出破坏性变更对照。」 |
| 国际化    | 「为 <code>{组件}</code> 抽取文案到 <code>{i18n路径}</code> ，禁止硬编码中文留在 JSX。」      |
| API 对接 | 「实现 <code>{endpoint}</code> 客户端：超时、重试、错误映射、单元测试各一例。」                   |
| 迁移     | 「从 <code>{旧库}</code> 迁到 <code>{新库}</code> ：分 PR、每步可运行、附回滚说明。」          |

## 14. 小结

- 真相源：`--help` > 官方文档 > 本书附录。
- 安全：环境变量与日志脱敏优先于「先跑起来」。
- 模板：自然语言也要 结构化（目标/约束/验收）。

附录 B · V2 教学稿

## 附录 C · 自测题（50 题）

说明：覆盖全书 20 篇核心概念，分 基础（1—20）、进阶（21—40）、专家（41—50） 三级。  
每题 4 选项，答案在文末。

## 基础（1—20）

---

1. 本书强调「AI Agent 壁垒」主要在哪一侧？

- A. 完全取决于模型参数量
- B. 约 90% 在模型之外的系统与流程
- C. 只与 Prompt 字数有关
- D. 只与编程语言有关

2. 「好体验不是几段 Prompt 忽悠出来的」最接近的含义是？

- A. Prompt 完全无用
- B. 可靠体验依赖制度、工具、权限与验证闭环
- C. 体验只与主题皮肤有关
- D. 只要把提示写长即可

3. Token 在工程上最贴切的比喻是？

- A. CPU 核数
- B. 预算与窗口资源的计量单位
- C. 网络带宽
- D. 磁盘 inode

4. 最小权限原则是指？

- A. 不给任何权限
- B. 仅授予完成任务所需的最小能力
- C. 默认管理员权限
- D. 权限与任务无关

5. 工具（Tool）在 Agent 中的主要作用是？

- A. 替代版本控制
- B. 将模型输出连接到真实世界操作（读/写/执行等）
- C. 仅用于美化终端
- D. 仅用于统计字数

6. MCP 的主要价值是？

- A. 替代 Git
- B. 标准化扩展工具与资源的协议
- C. 加密磁盘
- D. 压缩图片

7. 上下文窗口（Context window）描述的是？

- A. 显示器分辨率
- B. 模型单次可处理的 token 上限



- C. CPU 缓存行
- D. 并发线程数

**8. CI 在 Agent 协作中的首要价值是？**

- A. 取代代码评审
- B. 提供可重复的自动化验证门禁
- C. 删除所有测试
- D. 禁止本地开发

**9. PR (Pull Request) 与 Agent 的最佳关系是？**

- A. Agent 改动可不经评审直接上主分支
- B. Agent 产出也应进入与人类相同的评审与 CI 流程
- C. PR 只给人类用
- D. PR 与质量无关

**10. 秘文 (Secret) 管理的首要原则是？**

- A. 贴在 README 方便协作
- B. 避免进入模型上下文与日志，使用密钥管理
- C. 用短密码即可
- D. 只在生产环境保密

**11. 「人类在环 (Human-in-the-loop)」强调？**

- A. 完全自动化无需人
- B. 关键决策点保留人工确认或审批
- C. 人类不能看代码
- D. 仅用于 UI 动画

**12. 审计轨迹 (Audit trail) 主要解决？**

- A. 提高模型温度
- B. 事后追溯谁做了什么、何时做
- C. 加快打字速度
- D. 自动生成图标

**13. 对 Agent 而言，「结构化任务描述」通常不包含？**

- A. 目标与验收标准
- B. 约束与不可触碰范围
- C. 纯情绪宣泄无验收
- D. 相关文件或模块指针

**14. 沙箱 (Sandbox) 用于？**

- A. 玩游戏
- B. 限制命令或进程的实际破坏范围

- C. 加密模型权重
- D. 替换编程语言

15. Flaky test（不稳定测试）对 Agent 的危害是？

- A. 无影响
- B. 让自动化验证不可信，掩盖真实回归
- C. 一定代表实现正确
- D. 可自动忽略

16. 缓存（Cache）在 Agent 成本上的直接意义是？

- A. 增加重复计算
- B. 减少重复读与重复计算，降低延迟与费用
- C. 只影响 UI 颜色
- D. 与 token 无关

17. 「实现者 / 审查者」角色拆分的核心目的是？

- A. 增加 headcount
- B. 降低认知污染，分离构建与风险审视
- C. 禁止写代码
- D. 只用于招聘标题

18. 多模态 Agent 相比纯文本的主要新增风险是？

- A. 无风险
- B. 截图等可能携带隐私与敏感信息
- C. 不能使用工具
- D. 不能使用 Git

19. 企业采纳 Agent 时，「信任」通常更关注？

- A. 图标是否好看
- B. 数据流、租户隔离、审计与合规映射
- C. 仅关注模型名字
- D. 仅关注 Star 数

20. 本书对「4756 个模块」类说法的教学用途是？

- A. 精确文件计数
- B. 说明组合复杂度与协议治理的重要性
- C. 证明应放弃模块化
- D. 与工程无关

## 进阶（21—40）

---

21. 控制循环（Observe—Plan—Act）中，哪一步最依赖工程化工具返回？

- A. Observe 若无法结构化读取环境则后续易失真
- B. 仅 Plan 需要工具
- C. Act 不需要反馈
- D. 循环与工具无关

22. JSON Schema 对工具系统的价值主要是？

- A. 装饰
- B. 约束输入输出、便于校验与文档化
- C. 替代 HTTP
- D. 生成随机数

23. 长上下文时代仍要坚持分层加载，主要因为？

- A. 长上下文无上限
- B. 噪声与成本仍可能拖垮质量与预算
- C. 分层非法
- D. 模型无法读文件

24. 自主性等级提高时，必须同步加强的是？

- A. 仅 slogan
- B. 审计、回滚、门禁与影响半径控制
- C. 删除所有测试
- D. 关闭 CI

25. coordinator 模块通常负责？

- A. 渲染网页 CSS
- B. 多 Agent 的任务拆分与合并
- C. 数据库 B+ 树
- D. DNS 解析

26. permissions 目录在架构上属于？

- A. 纯 UI
- B. 横切关注点：策略与授权
- C. 与 Agent 无关
- D. 仅文档

27. 「关笼子 vs 裸奔」隐喻中，本书更推崇？

- A. 极端关笼子
- B. 极端裸奔

- C. 操作系统式底座：有边界、可组合、可审计
- D. 不使用任何工具

**28. TCO（总拥有成本）在选型时应包含？**

- A. 仅订阅费
- B. 订阅、人工等待、事故与合规成本等综合
- C. 仅 GPU 电费
- D. 仅键盘磨损

**29. 对终端型 Agent（如 Claude Code / Aider）的典型优势是？**

- A. 绝对不能访问仓库
- B. 与 git/脚本工作流结合紧，适合批处理
- C. 不能做重构
- D. 不能使用模型

**30. IDE 深度集成型产品（如 Cursor 类）的典型优势是？**

- A. 完全不能索引
- B. 编辑、跳转、诊断与 Agent 同屏协同强
- C. 禁止运行测试
- D. 与文件系统无关

**31. BYOK 方案的主要受益场景是？**

- A. 需要自备密钥与成本可控的组织
- B. 完全不想管理密钥
- C. 禁止任何 API
- D. 仅移动端

**32. 开源客户端（如 Continue）学习价值高主要是因为？**

- A. 一定更安全
- B. 可读实现与配置，理解 Agent harness
- C. 无配置
- D. 无社区

**33. RAG 与「把全文塞进窗口」相比，通常更有利于？**

- A. 提高噪声与成本
- B. 按相关性检索，控制上下文规模
- C. 删除索引
- D. 禁止查询

**34. 黄金 issue 集（回归集）用于？**

- A. 装饰 KPI
- B. 可重复评估 Agent/流程改动是否退化

- C. 替代生产监控
- D. 删除文档

35. 任务包 (Task packet) 不建议缺少的字段是？

- A. 验收标准
- B. 随机笑话
- C. 与目标无关的图片
- D. 无约束词

36. 对工具输出做分页/截断的主要目的是？

- A. 让日志不可读
- B. 避免巨型 JSON/文本吞噬上下文
- C. 禁止错误信息
- D. 增加幻觉

37. 供应链安全在 Agent 场景的含义更接近？

- A. 只买贵设备
- B. 插件、MCP 服务、依赖包的来源与版本可信
- C. 不用网络
- D. 不写代码

38. 「Prompt 注入」防御在系统层更可靠的做法是？

- A. 仅要求模型发誓
- B. 运行时权限与数据边界，默认不信任外部文本
- C. 关闭防火墙
- D. 公开所有密钥

39. Git worktree 对 Agent 实验的价值是？

- A. 禁止并行尝试
- B. 同一仓库多目录并行试方案，减少切换成本
- C. 删除历史
- D. 替代远程仓库

40. 本书对比多产品时强调的方法论是？

- A. 只看广告
- B. 先列约束与任务，再做 PoC 与 TCO
- C. 只看开源
- D. 随机抽签

## 专家 (41–50)

---

41. 在设计工具幂等性时，最关键的是？

- A. 每次随机副作用
- B. 重复调用不会造成额外危险副作用（在约束条件下）
- C. 禁止重试
- D. 不需要文档

42. 多 Agent 交接 (Handoff) 最容易出问题的点是？

- A. 上下文过度压缩导致丢失约束
- B. 交接永远无风险
- C. 不能有结构化格式
- D. 不需要领导者

43. 企业数据最小化原则与 Agent 的结合点是？

- A. 上传全部磁盘镜像
- B. 默认少收集、说明用途、可删除与可隔离
- C. 禁止审计
- D. 日志永久明文保存所有秘文

44. 评测 Agent 时，「单次炫技成功」与「规模化可靠」的 gap 主要来自？

- A. 无差距
- B. 治理、回归集、门禁与观测是否到位
- C. 仅字体大小
- D. 仅主题颜色

45. 将 `query.ts` 视为「心脏」文件的教学理由是？

- A. 它一定最短
- B. 主循环、模型往返与工具编排高度集中
- C. 它不参与运行
- D. 它只含 CSS

46. 对 `Tool.ts` 抽象层最应强调的契约是？

- A. 无 schema
- B. schema、错误语义、风险声明与结果封装
- C. 仅字符串拼接
- D. 不可测试

47. 在「操作系统式底座」隐喻中，「系统调用」最接近？

- A. 用户聊天
- B. 受控的工具调用

- C. 桌面壁纸
- D. 邮件客户端

**48. 长任务中设置检查点（Checkpoint）主要是为了？**

- A. 增加失败不可恢复性
- B. 可恢复、可回滚、便于人类接管
- C. 禁止中断
- D. 删除日志

**49. 对闭源与开源工具的务实态度，本书更接近？**

- A. 二极管互斥
- B. 按约束组合：体验与可审计/逃生通道可并存
- C. 只用闭源
- D. 只用开源

**50. 全书结语希望你带走的习惯不包括？**

- A. 先写验收再动手
  - B. 维护工具白名单与风险分级
  - C. 完全不做复盘记录
  - D. 把不确定性关进系统
-

## 答案

| 题号 | 答案 | 题号 | 答案 | 题号 | 答案 | 题号 | 答案 |
|----|----|----|----|----|----|----|----|
| 1  | B  | 14 | B  | 27 | C  | 40 | B  |
| 2  | B  | 15 | B  | 28 | B  | 41 | B  |
| 3  | B  | 16 | B  | 29 | B  | 42 | A  |
| 4  | B  | 17 | B  | 30 | B  | 43 | B  |
| 5  | B  | 18 | B  | 31 | A  | 44 | B  |
| 6  | B  | 19 | B  | 32 | B  | 45 | B  |
| 7  | B  | 20 | B  | 33 | B  | 46 | B  |
| 8  | B  | 21 | A  | 34 | B  | 47 | B  |
| 9  | B  | 22 | B  | 35 | A  | 48 | B  |
| 10 | B  | 23 | B  | 36 | B  | 49 | B  |
| 11 | B  | 24 | B  | 37 | B  | 50 | C  |
| 12 | B  | 25 | B  | 38 | B  |    |    |
| 13 | C  | 26 | B  | 39 | B  |    |    |

单行答案串（便于批量核对）：

1—10: **BBBBBBBBBB**

11—20: **BBBBBBBBBB**

21—30: **ABBBBBBBBB**

31—40: **BBBBBBBBBB**

41—50: **BABBBBBBBC**

## 评分建议

| 得分率    | 建议                  |
|--------|---------------------|
| 80%+   | 可进入平台化/治理阅读         |
| 60—80% | 重读第 20 篇与工具/权限章节    |
| <60%   | 从基础工作流与 CI 闭环章节系统重学 |



[Mermaid 图表 — 请在在线版本中查看]

附录 C · V2 教学稿

附录 D · 推荐阅读与外部资源

说明：链接均为 教程编写时常见入口，不保证永久有效；请在访问前核对域名与证书。企业内网需遵守贵司上网策略。

1. 资源地图

[Mermaid 图表 — 请在在线版本中查看]

2. 官方文档（英文为主）

| 主题              | URL                                                                                                                                       | 备注        |
|-----------------|-------------------------------------------------------------------------------------------------------------------------------------------|-----------|
| Anthropic 文档总入口 | <a href="https://docs.anthropic.com/">https://docs.anthropic.com/</a>                                                                     | 模型、API、策略 |
| Claude API 参考   | <a href="https://docs.anthropic.com/en/api/">https://docs.anthropic.com/en/api/</a>                                                       | 请求/流式/错误码 |
| 消息与工具使用指南       | <a href="https://docs.anthropic.com/en/docs/build-with-claude/tool-use">https://docs.anthropic.com/en/docs/build-with-claude/tool-use</a> | 工具调用模式    |
| MCP 介绍          | <a href="https://modelcontextprotocol.io/">https://modelcontextprotocol.io/</a>                                                           | 协议与生态     |
| MCP 规范仓库        | <a href="https://github.com/modelcontextprotocol/specification">https://github.com/modelcontextprotocol/specification</a>                 | 版本与变更     |

### 3. 源码与仓库（学习向）

| 主题                 | URL                                                                                                                         | 备注                  |
|--------------------|-----------------------------------------------------------------------------------------------------------------------------|---------------------|
| MCP TypeScript SDK | <a href="https://github.com/modelcontextprotocol/typescript-sdk">https://github.com/modelcontextprotocol/typescript-sdk</a> | 服务端/客户端示例           |
| Continue           | <a href="https://github.com/continuedev/continue">https://github.com/continuedev/continue</a>                               | 开源 IDE 扩展           |
| Aider              | <a href="https://github.com/Aider-AI/aider">https://github.com/Aider-AI/aider</a>                                           | 终端 pair programming |

### 4. 源码分析文章（中文）

| 主题               | 说明与检索建议                                 |
|------------------|-----------------------------------------|
| 「Claude Code 架构」 | 在搜索引擎或技术社区检索 Claude Code 源码 分析、Agent 循环 |
| 「MCP 实战」         | 检索 MCP 服务器 搭建、工具注册                      |
| 「上下文工程」          | 检索 RAG Agent 编程、长上下文 成本                 |

具体文章 URL 变化频繁，本书以 **检索词 + 官方一手文档** 为主锚点。

### 5. 源码分析文章（英文）

| 主题                          | 说明与检索建议                               |
|-----------------------------|---------------------------------------|
| Tool use / function calling | 检索 Anthropic tool use best practices  |
| Agent harness               | 检索 LLM agent loop design、ReAct        |
| Security                    | 检索 prompt injection、LLM agent sandbox |

## 6. 视频教程（检索向）

| 平台       | 检索关键词                                      |
|----------|--------------------------------------------|
| YouTube  | Claude Code tutorial , MCP server tutorial |
| Bilibili | Claude Code 教程 , MCP 入门                    |

## 7. 社区讨论

| 平台                 | URL 或说明                                                                          |
|--------------------|----------------------------------------------------------------------------------|
| GitHub Discussions | 各项目仓库内                                                                           |
| Hacker News        | <a href="https://news.ycombinator.com/">https://news.ycombinator.com/</a> 检索相关标题 |
| Reddit             | r/LocalLLaMA、r/MachineLearning 等（注意噪声）                                           |

## 8. 安全与合规阅读

| 主题   | 检索建议                   |
|------|------------------------|
| 供应链  | SLSA、npm supply chain  |
| 数据保护 | GDPR、贵司隐私政策            |
| 秘文管理 | GitHub secret scanning |

## 9. 经典论文与概念（选读）

| 概念            | 说明               |
|---------------|------------------|
| ReAct         | 推理与行动交错范式        |
| RLHF / RLAIIF | 对齐与行为塑造（背景向）     |
| RAG           | 检索增强生成与 Agent 结合 |

## 10. 本书建议阅读顺序

[Mermaid 图表 — 请在在线版本中查看]

## 11. 引用规范建议

- 优先引用 版本化 文档（带日期或 commit）。
- 转载社区文章请核对 许可证。

## 12. 安全纵深阅读（链接或检索）

| 主题               | 入口                                                |
|------------------|---------------------------------------------------|
| OWASP LLM Top 10 | 检索 <code>OWASP LLM Top 10</code> 官方页面             |
| Prompt Injection | 检索 <code>prompt injection mitigation</code>       |
| SLSA 供应链         | <a href="https://slsa.dev/">https://slsa.dev/</a> |

## 13. 工程实践书单（检索向）

| 方向   | 检索词                                                         |
|------|-------------------------------------------------------------|
| 可观测性 | <code>OpenTelemetry</code> 、分布式追踪                           |
| 测试   | <code>testing pyramid</code> 、契约测试                          |
| 发布   | <code>feature flags</code> 、 <code>canary deployment</code> |

## 14. 中文社区与聚合（注意甄别质量）

| 类型  | 说明             |
|-----|----------------|
| 聚合站 | 关注 原文出处 与 发布日期 |
| 翻译文 | 对照英文一手文档核对术语   |

附录 D · V2 教学稿 · 外链需读者自行验证时效性

## 附录 E · 中英术语对照表

**说明：**同一英文词在不同产品中有不同译法，本表取 **教学常用译法**；括号内为可替换说法。

# 术语表 (60+)

| 英文                    | 中文                 | 简要解释                             |
|-----------------------|--------------------|----------------------------------|
| Agent                 | 智能体 / Agent        | 能感知环境、调用工具、多步推进任务的系统             |
| AI Agent              | AI 智能体             | 以模型为推理核、外围含工具与策略的 Agent          |
| Anthropic             | Anthropic (人名/公司名) | Claude 系列模型与相关产品的开发方             |
| API Key               | API 密钥             | 访问云端 API 的凭据，需保密与轮换              |
| API                   | 应用程序接口             | 程序间调用约定，含认证、限流、错误语义              |
| Allowlist             | 允许列表               | 仅显式列出的操作/工具可被自动执行                |
| Audit trail           | 审计轨迹               | 关键动作的可追溯记录                       |
| Bash / Shell          | Shell / 终端命令       | 在操作系统层执行命令的接口                    |
| Blast radius          | 影响半径               | 一次故障或错误变更波及的系统范围                 |
| Branch                | 分支                 | 版本控制中并行开发线                       |
| BYOK                  | 自备密钥               | Bring Your Own Key, 使用自有 API Key |
| Cache                 | 缓存                 | 复用先前结果以降低延迟与成本                   |
| Canary deployment     | 金丝雀发布              | 小流量验证后再全量发布                      |
| Checkpoint            | 检查点                | 长任务中可恢复或回滚的中间状态                  |
| CI                    | 持续集成               | Continuous Integration, 自动化构建与测试 |
| CLI                   | 命令行界面              | Command Line Interface           |
| Claude                | Claude             | Anthropic 的对话与工具模型产品线名称          |
| Claude Code           | Claude Code        | 面向终端/仓库的编程 Agent 产品形态 (教程语境)     |
| Commit                | 提交                 | 将变更记录到版本库的快照                     |
| Compression (context) | (上下文) 压缩           | 将历史折叠为摘要以节省窗口                    |
| Context window        | 上下文窗口              | 模型单次可见的 token 上限                 |
| Context               | 上下文                | 提示、历史、检索结果等的总和                   |
| Coordinator           | 协调器                | 多 Agent 任务拆分与合并模块                |
| Copilot               | 副驾驶 / Copilot      | 辅助编程产品统称 (多厂商使用)                 |
| Cursor                | Cursor             | 一款 AI IDE 产品名                    |
| Denylist / Blocklist  | 拒绝列表               | 明确禁止的动作或域                        |

| 英文                | 中文         | 简要解释                               |
|-------------------|------------|------------------------------------|
| Diff              | 差异         | 两个版本之间的改动对比                        |
| E2E test          | 端到端测试      | 覆盖完整用户路径的测试                        |
| Embedding         | 嵌入向量       | 文本/代码的稠密向量表示，用于检索                  |
| Flaky test        | 不稳定测试      | 间歇性失败、与环境或时序相关的测试                  |
| Fork              | 分叉 / 复刻    | 复制仓库以独立演进                          |
| Git               | Git        | 分布式版本控制工具                          |
| GitHub            | GitHub     | 代码托管与协作平台                          |
| Hallucination     | 幻觉         | 模型生成看似合理但事实错误的内容                   |
| Harness           | 套架 / 运行时框架 | 驱动模型循环、工具调用与状态机的代码                 |
| Human-in-the-loop | 人类在环       | 关键步骤需人工确认或审批                       |
| IDE               | 集成开发环境     | Integrated Development Environment |
| Indexing          | 索引         | 为检索对仓库建立结构或向量索引                    |
| Issue             | 议题 / 工单    | 缺陷或任务跟踪条目                          |
| JSON Schema       | JSON 模式    | 描述 JSON 结构与字段约束                    |
| JWT               | JWT 令牌     | 常用于无状态认证的令牌格式（若涉及网关）               |
| Least privilege   | 最小权限       | 仅授予完成任务所需的最小能力                     |
| Lint              | 静态检查       | 代码风格与部分错误的静态分析                     |
| LLM               | 大语言模型      | Large Language Model               |
| MCP               | 模型上下文协议    | Model Context Protocol，工具与资源的标准化扩展 |
| Merge             | 合并         | 将分支变更合入目标分支                        |
| Multimodal        | 多模态        | 同时处理文本、图像等多种输入                     |
| OAuth             | OAuth      | 一种常见授权协议（企业集成场景）                   |
| Patch             | 补丁         | 一组文件的增量改动                          |
| Permission        | 权限         | 能否执行某类操作的判定                        |
| Plugin            | 插件         | 可扩展能力的模块                           |
| PoC               | 概念验证       | Proof of Concept，小规模试点             |
| PR                | 拉取请求       | Pull Request，代码评审与合并请求             |
| Prompt            | 提示词        | 发给模型的指令与上下文片段                      |

| 英文            | 中文         | 简要解释                           |
|---------------|------------|--------------------------------|
| RAG           | 检索增强生成     | Retrieval-Augmented Generation |
| Rate limit    | 限流         | API 调用频率上限                     |
| Retry         | 重试         | 失败后的再次尝试（需幂等配合）                |
| Rollback      | 回滚         | 恢复到变更前状态                       |
| Sandbox       | 沙箱         | 隔离执行环境以限制破坏范围                  |
| Schema        | 模式 / 结构约束  | 数据或工具输入输出的结构定义                 |
| Secret        | 密钥 / 秘文    | 密码、令牌等敏感信息                     |
| Session       | 会话         | 一次连续交互的状态边界                    |
| Slash command | 斜杠命令       | 以 <code>/</code> 开头的交互式指令      |
| SSE           | 服务器推送事件    | Server-Sent Events，流式响应常用      |
| SSO           | 单点登录       | Single Sign-On                 |
| Streaming     | 流式输出       | 分块返回模型生成内容                     |
| Subagent      | 子智能体       | 被委派子任务的 Agent 实例               |
| System prompt | 系统提示       | 会话级稳定规则与角色设定                   |
| Telemetry     | 遥测         | 使用情况与诊断数据的上报（需合规）              |
| Token         | 词元 / Token | 模型计费与窗口计量的基本单位                 |
| Tool use      | 工具调用       | 模型发起结构化工具请求的能力                 |
| Tool          | 工具         | Agent 可调用的外部能力单元               |
| TUI           | 终端用户界面     | Text-based UI                  |
| Typecheck     | 类型检查       | 静态类型系统验证                       |
| Windsurf      | Windsurf   | 一款 AI IDE 产品名                  |
| Worktree      | 工作树        | Git 同一仓库多检出目录的机制               |
| Zero trust    | 零信任        | 默认不信任、持续验证的安全模型                |

## 概念关系简图

[Mermaid 图表 — 请在在线版本中查看]



---

# 使用建议

---

- 写 中英文混排 PR 描述 时，可固定一套译名（本表为起点）。
  - 对外文档 首次出现 可用「中文（English）」格式。
- 

附录 E · V2 教学稿 · 共 67 条术语

---

Claude Code 完全指南 V2 | 仅用于教育学习目的

Claude Code 源码版权归 Anthropic, PBC 所有 | MIT License

[https://github.com/bcefgjh/claude-code-complete-guide\\_v2](https://github.com/bcefgjh/claude-code-complete-guide_v2)